From Zero to AMD GPU Kernel Engineer

SID-Devu

From Zero to AMD GPU Kernel Engineer

From Zero to AMD GPU Kernel Engineer

- ⚠️ NEW — Read the LEARNING PATH first
- How to use this book
- Table of Contents
- Prerequisites for the labs
- How this book differs from LDD3 and from random tutorials
- A note on the path

THE LEARNING PATH — read this first

- The big picture
- Stage-by-stage table
- Why this order? (the philosophy)
- Daily rhythm (the secret to actually finishing)
- Three checkpoints — can you answer “yes”?
- What if I’m in a hurry?
- What if I get stuck for a week?
- The single hardest moment
- Now go

Part 0 — Absolute Zero

- Who this part is for
- How long it takes
- How to read this part
- Table of Contents
- Why we use Python first
- What you need

Chapter 0 — Start here

- You are not behind
- The three rules
- What “understanding” means
- What good vs bad looks like
- What “real-world” means in this book
- Try it (your first homework)
- A note about feeling lost

Chapter 1 — What is a computer? (in plain English)
- The kitchen analogy
- The five parts you should know by name
- “What is the GPU then?”
- Where Linux fits in
- The screen, the keyboard, and “everything is a device”
- “But what is *inside* the CPU?”
- “What about the cloud, the internet, AI, …”
- Try it (5 minutes, no code yet)
- Self-check

Chapter 2 — Using Linux: the terminal
- Open a terminal
- Your first command
- What is a “command”?
- A useful starter session, end to end
- The 12 commands you must memorize
- Tab completion (huge timesaver)
- Up arrow
- Try it (15 minutes)
- What if I get stuck?
- Self-check

Chapter 3 — Files and paths
- Everything is a file
- The filesystem is a tree
- What is a “path”?
- Practice this until you don’t have to think
- Hidden files
- Permissions, very briefly
- Useful path tricks
- Special directories you should know exist
- Try it (10 minutes)
- Self-check

Chapter 4 — Text editors
- The three you need to know
- `nano` — the friendly one
- VS Code (or Cursor)
- `vim` — eventually
- Why text editors matter
- A common editor mistake
- Try it (10 minutes)
- Self-check

Chapter 5 — What is code?
- Code is just text in a file
- What does “running” a program mean?
- A program is a list of instructions
- What happens when you run a program (the kid version)
- “But what is a programming language *really*?”
- “Source code” vs “binary”
- A driver is just a program too
- Try it (10 minutes)
- Self-check

Chapter 6 — Your first program (in Python)
- Three things every program does
- Variables
- Types of values
- Arithmetic
- Strings
- Reading input
- Comments
- Putting it together
- Lists (a peek)
- Try it (15 minutes)
- Common beginner mistakes (and how to fix)
- Self-check

Chapter 7 — Decisions and repeats
- Decisions: `if` / `elif` / `else`
- Repeats: `while`
- Repeats: `for`
- `break` and `continue`
- A real-feeling example: the FizzBuzz
- Combining decisions and repeats: number guessing
- Try it (20 minutes)
- Common beginner mistakes
- Self-check

Chapter 8 — Functions
- The motivation
- Defining a function
- Arguments (input)
- Return values (output)
- A useful example
- Why functions matter (for real)
- Variable scope

- Default arguments
- Putting it together
- What makes a good function
- Try it (20 minutes)
- Common mistakes
- Self-check

Chapter 9 — From Python to C

- Get the C compiler
- Hello, C
- Variables in C
- Arithmetic in C
- `if` / `else` in C
- Loops in C
- FizzBuzz, side by side
- Functions in C
- Reading input
- The compile-edit-run loop
- Try it (30 minutes)
- Differences you must internalize
- Self-check

Chapter 10 — When things break (the most important chapter)

- The mindset
- The 90% checklist
- Reading C compiler errors
- Reading Python errors
- Print debugging
- The 30-minute rule
- How to ask a good question
- When the program “just hangs”
- When the kernel “just hangs” (foreshadowing)
- A concrete debugging example
- Try it (30 minutes)
- Self-check

Chapter 11 — Bridge to Part I

- What changes in Part I
- Recommended pacing for Part I
- How to read Part I (and the rest of the book)
- What to expect emotionally
- A sneak preview: the same FizzBuzz, kernel-style
- A reminder of the journey
- A request

Part 0 Foundations — side topics every engineer needs
A1 — Numbers: binary, hex, decimal
How counting works (you already know this)
Binary — base 2
Hex — base 16
Why both?
Converting in your head
Powers of 2 — you must know these cold
Quick conversions to memorize
Negative numbers — two's complement (just FYI)
A real-world example
Try it
A2 — Bits, bytes, words
The hierarchy
The byte is the fundamental unit
Sizes (memorize)
How big is a byte's range?
KB vs KiB — the trap
Endianness — which byte goes first?
Alignment
Bit ordering inside a byte
Common bit-twiddling idioms
Try it
A3 — Boolean logic: AND, OR, NOT, XOR
The four operations
Logical vs bitwise (the most common beginner trap)
Truth tables tell you everything
Why this matters for software
Hardware view (one paragraph, then we move on)
Try it
A4 — Shell power tour
Pipes — `|`
Redirects — `>`, `>>`, `<`, `2>`
`grep` — find text
`find` — find files
`xargs` — run a command per line of input
Variables and `$`
Command substitution — `$(...)`
Chaining: `&&`, `||`, `;`
Globs — `*`, `?`, `[]`, `{}`
History

- Useful one-liners
- Shell scripts
- Functions in scripts
- Loops
- Real-world combos
- Aliases — keep your hands fast
- Cheat-sheet card (print this)
- Try it

A5 — Character encoding: ASCII, UTF-8, and why text is harder than it looks

- ASCII — the original 128 characters
- Unicode — one number per character (forever)
- UTF-8 — the only encoding you should use
- UTF-16 (heads up)
- C strings and Unicode
- Byte-order mark (BOM)
- Newlines: LF vs CRLF
- Locales — the `LANG` and `LC_*` variables
- In the kernel
- Try it

Chapter 1 — What is a program, a compiler, and an operating system?

- 1.1 The machine
- 1.2 What a CPU actually does
- 1.3 What a compiler does
- 1.4 What an operating system does
- 1.5 Where a device driver lives
- 1.6 The languages of the trade
- 1.7 Mental model checklist

Chapter 2 — The toolchain and your first C programs

- 2.1 The five tools you must own
- 2.2 Hello, world
- 2.3 The structure of a real C program
- 2.4 A first Makefile
- 2.5 Reading compiler warnings is a skill
- 2.6 Debugging your first segfault
- 2.7 The five rules of writing C from day one
- 2.8 Exercises

Chapter 3 — Types, integer promotion, and undefined behavior

- 3.1 The fundamental types
- 3.2 Sizes and `sizeof`
- 3.3 Implicit conversions and integer promotion
- 3.4 Signed integer overflow is undefined behavior

3.5 Other undefined behaviors you must memorize
3.6 Strict aliasing
3.7 The `volatile` keyword
3.8 The `const` keyword
3.9 The `static` and `extern` keywords
3.10 Type aliases for hardware
3.11 Exercises

Chapter 4 — Pointers, arrays, and the C memory model

4.1 What a pointer is
4.2 The four memory regions
4.3 Pointer arithmetic
4.4 Arrays decay to pointers (and when they don't)
4.5 Strings
4.6 The two-headed sword: `char *` and `void *`
4.7 Arrays of pointers, pointers to pointers
4.8 Function pointers
4.9 The `restrict` keyword
4.10 Common pointer bugs and how to find them
4.11 The C memory model in one paragraph
4.12 Exercises

Chapter 5 — Structs, unions, bitfields, packing, alignment

5.1 The basic struct
5.2 Alignment
5.3 Why misaligned access matters
5.4 The `__attribute__((packed))` directive
5.5 Unions
5.6 Bitfields
5.7 Endianness
5.8 The `container_of` macro
5.9 Forward declarations and opaque types
5.10 Anonymous structs and unions
5.11 Real-world: a hardware-register struct
5.12 Exercises

Chapter 6 — Bit manipulation: the language of hardware

6.1 The six basic operators
6.2 The Linux kernel idioms
6.3 Common bit tricks
6.4 Bitmaps
6.5 Reading a register: full pattern
6.6 Bit hacks you should at least recognize
6.7 Exercises

Chapter 7 — Dynamic memory and writing your own allocator
- 7.1 The libc heap
- 7.2 Where does memory come from?
- 7.3 Memory layout of a small allocation
- 7.4 Patterns and anti-patterns
- 7.5 The arena allocator
- 7.6 A free-list allocator
- 7.7 The SLUB / slab idea
- 7.8 Lifetimes: who owns the memory?
- 7.9 Defensive habits
- 7.10 Exercises

Chapter 8 — The preprocessor, headers, linkage, build systems
- 8.1 Directives
- 8.2 Macro pitfalls
- 8.3 `#` and `##`
- 8.4 Header guards and `#pragma once`
- 8.5 What goes in a header
- 8.6 Linkage
- 8.7 The `inline` keyword
- 8.8 The build pipeline of a real C project
- 8.9 `pkg-config`
- 8.10 Static vs. dynamic libraries
- 8.11 The kernel build system (Kbuild)
- 8.12 Reading a real driver Makefile
- 8.13 Exercises

Chapter 9 — Function pointers, callbacks, opaque types, plugin design
- 9.1 The syntax, one more time
- 9.2 Callbacks
- 9.3 Vtables in C: structs of function pointers
- 9.4 Opaque types and the C version of OOP
- 9.5 Inheritance via embedded structs
- 9.6 The `_init` / `_destroy` symmetry
- 9.7 Plugin design
- 9.8 The `EXPORT_SYMBOL` mechanism
- 9.9 Exercises

Chapter 10 — Defensive C: const, restrict, error handling, kernel style
- 10.1 Const-correctness
- 10.2 `restrict`
- 10.3 Error handling
- 10.4 Cleanup with `goto`
- 10.5 The kernel coding style — what you must absolutely follow

10.6 `assert`, `BUG_ON`, `WARN_ON`
10.7 Avoiding undefined behavior
10.8 A sample function reviewed
10.9 Exercises
Part DSA — Data Structures & Algorithms (in C)
Why DSA is non-negotiable
How DSA fits with the rest of this book
⚠️ Read this first — DSA-00 — How to read this part (without running code)
How to study this part
Table of contents
What you'll have at the end
Notes on language
DSA 0 — How to read this part (without running code)
What changed
How to read each chapter
A note on diagrams
What if I really want to run something?
A note on the original chapters
Last thing — be patient
DSA 1 — Big-O and complexity (read-to-learn version)
The big idea
The 7 labels you must know
How big is each, really? (numbers!)
Reading code to find Big-O
A trace: how `binary_search` actually runs
Best, average, worst case
What Big-O ignores (and why)
Space complexity (memory, not time)
Common confusions (resolve them now)
A picture summary
Common mistakes — what NOT to do
Recap (read this every day for a week)
Self-check (no code-running needed — answer in your head)
Read deeper (if curious — not required)
DSA 2 — Arrays (read-to-learn version)
The big idea
How operations behave (memorize)
Static vs dynamic vs growable
A multi-dimensional array (matrix)
Worked example: reverse an array in place
Worked example: maximum subarray (Kadane's algorithm)

- Common bugs (drawings included)
- Recap
- Self-check (no code-running needed)
- Read deeper (optional)

DSA 3 — Strings (read-to-learn version)

- The big idea
- Two ways to write strings in C
- strlen — how long is the string?
- strcmp — compare two strings
- The dangerous library functions (and the safe ones)
- Common string algorithms (traced)
- Convert string to integer (safely)
- Pointer-to-literal vs array (review with picture)
- Common bugs (with pictures)
- Recap
- Self-check (answer in your head)

DSA 4 — Linked lists (read-to-learn version)

- The big idea
- When linked lists win, when arrays win
- The struct that defines a node
- Building a list, step by step
- list_push_front (in code)
- Walk the list (find a value)
- Reverse a linked list (the famous interview question)
- Detect a cycle (Floyd's tortoise & hare)
- Doubly linked list
- The Linux kernel's `list_head` (the most important DS in the kernel)
- Iterating while deleting (the classic trap)
- Common bugs (with pictures)
- Recap
- Self-check (in your head)

DSA 5 — Stacks and queues (read-to-learn version)

- The big idea
- Stack: pictures and operations
- Stack as an array
- Why stacks matter — the call stack
- Classic problem: balanced brackets
- Queue: pictures and operations
- The naive queue (and why it's slow)
- Circular queue (ring buffer) — fast queue
- Where you see queues in real systems
- SPSC ring buffer — fast queue between two threads

- Stack vs queue — when to use what
- Common bugs
- Recap
- Self-check (in your head)

DSA 6 — Hash tables (read-to-learn version)

- The big idea
- What a hash function does
- Bucket selection: hash modulo size
- Collisions — when two keys land in the same bucket
- Full example: chained hash table with 4 buckets
- The full code (chained hash table)
- Why resize?
- Why hash tables fail (worst case)
- Hash tables vs other structures
- Picture of resize (4 → 8 buckets)
- Common application: the LRU cache (preview)
- Common bugs
- Linux kernel hash tables (preview)
- Recap
- Self-check (in your head)

DSA 7 — Binary trees & BSTs (read-to-learn version)

- The big idea
- Vocabulary (memorize)
- A binary tree node in C
- Binary Search Tree (BST) — the rule
- How search works in a BST
- Insert into a BST
- In-order traversal (visits values in sorted order!)
- Three traversal orders (memorize)
- Why “balance” matters: the worst case
- Counting the height
- Delete from a BST (the tricky one)
- Why two pointers per node makes a tree more powerful than a list
- Common bugs
- Recap
- Self-check (in your head)

DSA 8 — Self-balancing trees: AVL & Red-Black

- The problem balanced trees solve
- Rotations — the universal primitive
- AVL trees
- Red-Black trees
- Linux `rb_tree` — practical usage

- Other balanced trees you should know exist
- When to use what (cheat sheet)
- Common bugs (you will hit them)
- Try it
- Read deeper

DSA 9 — B-trees and B+trees

- Why not just use a red-black tree?
- B-tree definition
- Operations
- B+tree
- Cache and performance
- Concurrency
- In the kernel / on disk
- When B-trees aren't right
- LSM trees (briefly, for completeness)
- Try it (you don't need to write a full B-tree)
- Read deeper

DSA 10 — Heaps & priority queues (read-to-learn version)

- The big idea
- The heap shape and rules
- The trick: store a tree as an array (no pointers!)
- Insert into a min-heap (sift up)
- Remove the min (extract-min, sift down)
- The full code (min-heap, array-backed)
- Heap operations summary
- Build heap in O(n) — Floyd's bottom-up
- Heap sort (sort an array in place)
- Top-K problem (interview classic)
- Where heaps appear in real systems
- Common bugs
- Recap
- Self-check (in your head)

DSA 11 — Graphs: representation & traversal

- Definitions
- Representations
- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- What DFS unlocks
- Bipartite check (BFS/DFS coloring)
- Floyd-Warshall (for completeness)
- In the kernel / GPU stack
- Common bugs

Try it
Read deeper
DSA 12 — Shortest paths: Dijkstra, Bellman-Ford, A*
Single-source shortest path
A* — heuristic-guided search
Bidirectional search
When to use what
Negative cycles
Practical issues
In the real world
Common bugs
Try it
Read deeper
DSA 13 — Tries
The structure
Why a trie beats a hash table
Why a trie loses
Compressed trie / Patricia trie / Radix tree
Real applications
In the kernel
Common bugs
Try it
Read deeper
DSA 14 — Sorting algorithms (read-to-learn version)
Why so many sorts?
Bubble sort
Insertion sort
Merge sort (divide & conquer)
Quick sort (divide & conquer with a twist)
When to use which sort
In Linux
Common bugs
Recap
Self-check (in your head)
DSA 15 — Searching (read-to-learn version)
The big idea
Linear search (the obvious one)
Binary search (the magic one)
The off-by-one minefield (read carefully)
Variant: find the **first** occurrence (lower_bound)
Other search variants

- Ternary search (only on unimodal functions)
- Search in a 2D matrix
- When binary search isn't applicable
- In Linux
- Common bugs
- Recap
- Self-check (in your head)

DSA 16 — Recursion deep

- What recursion is
- How recursion uses the stack
- Tail recursion — the only "free" recursion
- Common recursion families
- Master theorem (computing recursion complexity)
- When recursion is wrong
- Converting recursion to iteration
- Worked examples
- Common bugs
- Try it (do all of these)
- Read deeper

DSA 17 — Dynamic programming

- When DP applies
- Two approaches
- DP recipe
- Classic problems
- State space caveats
- Bitmask DP (when n is small)
- DP on trees
- DP on intervals
- DP on digits ("digit DP")
- Memoization gotchas
- Common DP patterns to recognize
- Common bugs
- Try it (top 10 must-do DPs)
- Read deeper

DSA 18 — Greedy algorithms

- What "greedy" means
- When greedy works (and you can prove it)
- Classic greedy algorithms
- When greedy fails (and you must use DP)
- Greedy mindset for problem solving
- Real-world greedy
- Common bugs

- Try it
- Read deeper

DSA 19 — Divide & conquer

- The recipe
- Classic D&C algorithms
- Patterns for D&C
- When D&C wins over DP
- Parallel D&C
- Cilk-style work-stealing
- D&C in the real world
- Common bugs
- Try it
- Read deeper

DSA 20 — Bit manipulation tricks

- The basics (cross-link Foundations A3)
- Standard idioms
- Bitfields and packed flags
- Manipulating bit ranges (multi-bit fields)
- Atomic bit operations
- Bitmaps
- Real-world tricks
- In the kernel
- Common bugs
- Try it
- Read deeper

DSA 21 — Two pointers, sliding window, fast/slow

- Two pointers — opposite ends
- Two pointers — same direction (read/write)
- Sliding window
- Fast/slow pointers (Floyd’s tortoise & hare)
- Three pointers
- Patterns to recognize
- Common bugs
- Try it (top patterns)
- Read deeper

DSA 22 — Backtracking

- The pattern
- N-Queens (the classic)
- Permutations
- Subsets (power set)
- Sudoku solver

- Word search (in a grid)
- Pruning — the key to speed
- Common backtracking problems
- Memoization for backtracking
- Iterative backtracking
- Common bugs
- Try it (must-do list)
- Read deeper

DSA 23 — Union-Find / Disjoint-Set Union (DSU)

- Operations
- Implementation
- Kruskal's MST
- Other applications
- Variants
- In the kernel
- Common bugs
- Iterative find (avoids deep recursion)
- Try it
- Read deeper

DSA 24 — Advanced trees: segment, Fenwick (BIT), interval

- The problem they solve
- Fenwick tree (Binary Indexed Tree, BIT)
- Segment tree
- Interval tree
- Range tree
- kd-tree
- When to use what
- In the kernel
- Common bugs
- Try it
- Read deeper

DSA 25 — String algorithms: KMP, Rabin-Karp, Z, suffix arrays

- The problem
- KMP (Knuth-Morris-Pratt)
- Rabin-Karp
- Z-function
- Boyer-Moore
- Aho-Corasick
- Suffix arrays
- Suffix tree
- Edit distance algorithms
- Compression algorithms (briefly)

- In the real world
- Common bugs
- Try it (must-do)
- Read deeper

DSA 26 — System-design data structures

- LRU cache
- LFU cache (frequency-based)
- Bloom filter
- Skip list
- Consistent hashing
- Skiplist vs B-tree vs RB-tree (for indexes)
- Reservoir sampling
- Count-min sketch
- HyperLogLog
- In Linux / kernel-adjacent
- Common bugs
- Try it
- Read deeper

DSA 27 — Concurrent data structures

- Why concurrent DS are special
- Atomic operations (preview)
- Lock-based concurrent queue
- Single-producer, single-consumer (SPSC) lock-free queue
- MPMC queue (multiple producers, multiple consumers)
- ABA problem
- RCU (Read-Copy-Update) — Linux’s superpower
- Hazard pointers
- Per-CPU data
- Sequence locks (seqlock)
- Lock-free hash table
- What this chapter is for
- Common bugs (preview)
- Try it
- Read deeper

DSA 28 — DSA in the Linux kernel — guided tour

- Linked lists
- Hash tables
- Trees (RB-tree)
- Radix trees / XArray
- ID allocators
- Bitmaps
- Queues

- Heaps / priority queues
- Per-CPU data
- Refcounting
- Wait queues / completions
- Locking primitives
- DMA-fences (GPU specifically)
- DMA buffer sharing
- Workqueues / kthreads
- Slab / page allocator
- Page cache
- TCP/networking
- VFS (virtual filesystem)
- Block layer
- Tracing & debugging
- Where the kernel **doesn't** use textbook DSA
- Reading order to study real kernel DSA
- Try it
- Closing

Part USP — Userspace systems programming

- Why this part exists
- Table of contents
- What you'll build
- Mental model
- How to study this part

USP 1 — Unix philosophy & the system call mental model

- The Unix philosophy
- Everything is a file (descriptor)
- System calls — the boundary
- strace — your microscope
- errno — how syscalls report errors
- Man pages — your authoritative reference
- The standard fd numbers
- Process address space — preview
- Shells, environment, arguments
- "Hello, system calls"
- A note on glibc / libc
- Try it
- Read deeper

USP 2 — Processes: fork, exec, wait, exit

- Process — what is it really?
- fork() — make a copy of yourself
- exec() — replace yourself with another program

- fork + exec — the standard combo
- wait() / waitpid() — reap children
- exit() — proper termination
- Orphans, zombies, init
- A tiny shell
- Process groups & sessions (preview)
- getpid, getppid, getuid, getgid
- Resource limits
- Daemonization (briefly)
- Common bugs
- Try it
- Read deeper

USP 3 — Signals: interrupts to processes

- What signals are
- Default actions
- Sending signals
- Catching signals — sigaction (the right way)
- Async-signal-safe — the rule
- Self-pipe trick — bridging signals to a normal event loop
- signalfd — modern alternative
- Blocking signals — sigprocmask
- SIGCHLD — children dying
- Real-time signals (SIGRTMIN..SIGRTMAX)
- Common signals you'll write handlers for
- Signals and threads
- Signals from the kernel (faults)
- Common bugs
- Try it
- Read deeper

USP 4 — Pipes & FIFOs

- Anonymous pipes
- Pipe + fork = command pipeline
- dup2 — replace a file descriptor
- FIFOs (named pipes)
- SIGPIPE — when the reader is gone
- Atomic writes — PIPE_BUF
- Pipe sizes
- splice / tee / vmsplice
- Pipes vs sockets
- Common bugs
- socketpair — bidirectional cousin of pipe
- Try it

- Read deeper
- USP 5 — Shared memory & mmap
  - What `mmap` actually does
  - File-backed mmap
  - Anonymous mmap — for the heap or for shm
  - POSIX shared memory — for unrelated processes
  - Synchronization in shared memory
  - Performance tricks
  - msync — controlling the flush
  - Cost of mmap
  - When NOT to use mmap
  - Real-world examples
  - Common bugs
  - Try it
  - Read deeper
- USP 6 — Message queues, semaphores, futexes
  - POSIX message queues
  - System V message queues
  - POSIX semaphores
  - System V semaphores
  - eventfd — a counter you can poll
  - futex — the kernel primitive under userspace mutexes
  - When to use which IPC
  - Locks across processes
  - Common bugs
  - Try it
  - Read deeper
- USP 7 — Sockets — TCP & UDP
  - The 5-call life of a TCP server
  - The 3-call life of a TCP client
  - Address structures
  - Address families & types
  - TCP vs UDP
  - A minimal echo server
  - A minimal UDP echo server
  - SO_REUSEADDR / SO_REUSEPORT
  - Other useful socket options
  - DNS resolution — getaddrinfo
  - Unix domain sockets
  - Read & write semantics
  - TCP states (briefly)
  - Common bugs

Try it
Read deeper
USP 8 — I/O multiplexing: select, poll, epoll, io_uring
The problem
select — the original (1980s)
poll — improvement
epoll — Linux's scalable answer
A real epoll server skeleton
Edge-triggered drain rule
EPOLLOUT — when fd is writable
io_uring — the modern way
kqueue (BSD/macOS)
eventfd / signalfd / timerfd in epoll
Common bugs
Try it
Read deeper
USP 9 — pthreads deep
Threads vs processes
Creating a thread
Mutexes
Condition variables
Semaphores
Read-write locks
Thread-local storage
pthread_once — initialize once
Cancellation
Thread attributes
Producer / consumer (canonical example)
Atomic operations (C11)
Deadlocks
Sanitizers — your best friends
Per-thread vs shared
Common bugs
Try it
Read deeper
USP 10 — File I/O deep
The two layers
open flags
read, write — partial completes
lseek — random access
pread / pwrite — atomic positioned I/O
readv / writev — vectored I/O

- sendfile — zero-copy
- O_DIRECT — bypass the page cache
- fsync, fdatasync, sync
- File locking
- Inotify — file change notifications
- stat — file metadata
- /tmp considerations
- Async I/O alternatives
- Page cache and writeback
- Common bugs
- Try it
- Read deeper

USP 11 — /proc, /sys, /dev — the magic filesystems

- /proc
- /sys (sysfs)
- /dev
- /dev/shm
- /dev/pts
- /dev/loop, mounting images
- procfs limits
- Kernel-style attributes (preview)
- Container-aware /proc
- Tools that walk these filesystems
- Common bugs
- Try it
- Read deeper

USP 12 — Userspace performance & debug tools

- strace — trace system calls
- ltrace — trace library calls
- perf — Linux’s profiler (the most important tool)
- valgrind — memory error detector
- ThreadSanitizer (TSan)
- UndefinedBehaviorSanitizer (UBSan)
- gdb — the debugger
- addr2line — convert addresses to source
- objdump / readelf / nm — inspect binaries
- ltrace, ldd
- time — basic wallclock
- /usr/bin/time -v
- ftrace, eBPF, bcc, bpftrace — kernel-side tracing
- htop, iotop, iostat, vmstat
- /proc/PID/wchan — what’s it waiting on?

A debugging mindset (the 30-minute rule again)
Try it
Read deeper
Chapter 11 — From C to C++: classes, RAII, references
11.1 What C++ adds over C
11.2 Compiling C++
11.3 Namespaces
11.4 Classes
11.5 RAII — the single biggest reason to use C++
11.6 References
11.7 The four kinds of `T`
11.8 `new` and `delete` (and why we won't use them)
11.9 References vs. pointers — a rule of thumb
11.10 Headers and TUs in C++
11.11 Exercises
Chapter 12 — Object lifetime, special members, rule of 0/3/5
12.1 The six special member functions
12.2 The order of construction and destruction
12.3 Construction options: default, value, copy, move
12.4 A concrete RAII type with the rule of 5
12.5 Why `noexcept` matters
12.6 Inheritance and virtual destructors
12.7 Slicing
12.8 `final`
12.9 `default` and `delete`
12.10 Initializer lists, `{}` initialization
12.11 Avoiding common lifetime bugs
12.12 Exercises
Chapter 13 — Templates, concepts, and the type system
13.1 Function templates
13.2 Class templates
13.3 Specialization and overloading
13.4 Template type deduction
13.5 SFINAE and `enable_if`
13.6 Concepts (C++20)
13.7 Variadic templates and parameter packs
13.8 `constexpr` and compile-time evaluation
13.9 Type traits
13.10 CRTP — Curiously Recurring Template Pattern
13.11 The cost of templates
13.12 Exercises

Chapter 14 — Modern C++: move semantics, smart pointers, lambdas
- 14.1 Why move semantics
- 14.2 Lvalues, rvalues, xvalues
- 14.3 `std::move` and `std::forward`
- 14.4 Smart pointers
- 14.5 Custom deleters
- 14.6 Lambdas
- 14.7 `std::function` vs. function pointers vs. templates
- 14.8 `auto`, `decltype`, `decltype(auto)`
- 14.9 Range-based for, structured bindings
- 14.10 `std::optional`, `std::variant`, `std::expected`
- 14.11 `noexcept` revisited
- 14.12 Exercises

Chapter 15 — STL and writing your own container
- 15.1 What's in the STL
- 15.2 `<algorithm>`
- 15.3 Iterators
- 15.4 Iterator invalidation — the silent killer
- 15.5 The cost of small operations
- 15.6 Allocators (briefly)
- 15.7 Building your own `Vector<T>`
- 15.8 STL string and `string_view`
- 15.9 Hash maps in practice
- 15.10 Exercises

Chapter 16 — C++ for low-level work: PIMPL, polymorphism cost, ABI
- 16.1 The cost of virtual functions
- 16.2 PIMPL — Pointer to IMPLementation
- 16.3 ABI — what it is and why you care
- 16.4 Symbol mangling and `extern "C"`
- 16.5 Linkage and ODR
- 16.6 The "vocabulary types" of systems C++
- 16.7 Exception strategy
- 16.8 The `noreturn`, `[[likely]]`, `[[nodiscard]]` attributes
- 16.9 Inline assembly (briefly)
- 16.10 Where C++ shines in our domain
- 16.11 Exercises

Chapter 17 — What an OS is and the boot path on x86_64
- 17.1 What "operating system" means
- 17.2 The boot path on x86_64
- 17.3 The kernel's view of memory
- 17.4 Privilege rings, CPL, syscall

17.5 The big abstractions
17.6 Modes of running drivers
17.7 The kernel-userspace contract: stable APIs
17.8 The major subsystems we will touch
17.9 Exercises

Chapter 18 — Processes, threads, address spaces, syscalls

18.1 The Linux process
18.2 Processes vs. threads on Linux
18.3 Address space (`mm_struct`)
18.4 fork, exec, wait
18.5 The syscall mechanism on x86_64
18.6 ioctl: the catch-all syscall
18.7 Signals
18.8 epoll/poll/select and async I/O
18.9 Resource limits
18.10 Exercises

Chapter 19 — Virtual memory, paging, TLBs, MMUs

19.1 Why virtual memory
19.2 Pages
19.3 A page table walk
19.4 Huge pages
19.5 The TLB
19.6 The kernel's page-table view
19.7 Page faults
19.8 Cache attributes
19.9 The page allocator
19.10 Reverse mappings and `struct page`
19.11 NUMA briefly
19.12 IOMMUs
19.13 Exercises

Chapter 20 — Scheduling: CFS, deadline, real-time

20.1 Scheduling classes
20.2 The CFS algorithm
20.3 Preemption
20.4 Blocking, sleeping, scheduling
20.5 Atomic context
20.6 Affinity, CPU pinning, isolation
20.7 IRQ threads and kthreads
20.8 Latency budgets
20.9 The `current` macro and `task_struct`
20.10 Exercises

Chapter 21 — The I/O stack: VFS, page cache, block layer
- 21.1 The VFS
- 21.2 The page cache
- 21.3 Block layer (briefly)
- 21.4 Network stack (briefer)
- 21.5 The `inode` and the `dentry`
- 21.6 sysfs and debugfs
- 21.7 `procfs` (`/proc`)
- 21.8 The driver-author's view of the I/O stack
- 21.9 Exercises

Chapter 22 — Interrupts, exceptions, IRQ delivery
- 22.1 What an interrupt is
- 22.2 Legacy and modern interrupt delivery
- 22.3 The kernel's IRQ subsystem
- 22.4 The handler
- 22.5 Top half and bottom half
- 22.6 amdgpu interrupt flow
- 22.7 Affinity
- 22.8 NMI and watchdogs
- 22.9 IPIs
- 22.10 Receiving interrupts in atomic vs. process context
- 22.11 Polling fallback
- 22.12 Exercises

Part OST — Classic OS Theory (the academic foundation)
- Why this part exists
- The classic OS topics
- Table of contents
- Recommended companion text
- How this part fits

OST 1 — Process model: the formal view
- The illusion
- Process control block (PCB)
- Process state diagram
- Context switch
- Process creation models
- Threads — finer granularity
- Process vs thread vs fiber
- Birth and death
- Resource isolation: namespaces (preview)
- CPU scheduling preview
- Linux specifics

- Try it
- Read deeper

OST 2 — CPU scheduling theory

- The scheduling problem
- Metrics
- Classic algorithms
- Linux’s scheduling classes
- Multi-CPU scheduling
- Preemption — when does a task lose CPU?
- Wake-up & sleep
- Real-time scheduling
- Schedulers in the wild
- What’s hard
- Try it
- Read deeper

OST 3 — Synchronization theory: Peterson, semaphores, monitors

- Why we need sync
- Mutual exclusion — the goal
- Peterson’s algorithm (theoretical, software-only)
- Hardware support
- Spinlock
- Mutex (sleeping lock)
- Reader-writer lock
- Semaphore (Dijkstra, 1965)
- Monitors (Hoare/Hansen)
- Memory ordering — the modern wrinkle
- Lock-free vs wait-free
- Common synchronization primitives — summary
- Common bugs
- Try it
- Read deeper

OST 4 — Classic synchronization problems

- Producer / Consumer (bounded buffer)
- Readers / Writers
- Dining philosophers
- Sleeping barber
- Cigarette smokers (a teaching trap)
- H2O problem
- Once initialization
- Token-passing (the producer/consumer’s cousin)
- Barrier
- Reader-Writer with priority

- Banker’s algorithm (deadlock avoidance)
- What this teaches
- Try it
- Read deeper

OST 5 — Deadlock: prevention, avoidance, detection

- The four conditions for deadlock (Coffman, 1971)
- Prevention strategies
- Avoidance — the Banker’s algorithm
- Detection + recovery
- Livelock
- Priority inversion (and inheritance)
- Lockdep — Linux’s deadlock detector
- Common deadlock-shaped bugs
- Strategies in real systems
- Algorithms tangentially related
- Common bugs
- Try it
- Read deeper

OST 6 — Memory theory: paging, segmentation, swapping

- The illusion
- Address translation
- Page table entries (PTEs)
- Page faults
- TLB (Translation Lookaside Buffer)
- Page sizes
- Swapping
- Working set & locality
- Memory zones (Linux)
- NUMA
- Allocation strategies (the OS picks frames)
- Real address spaces (Linux x86-64)
- What about virtualization?
- What you should remember
- Try it
- Read deeper

OST 7 — Page replacement algorithms

- The setup
- The optimal (Belady’s MIN)
- FIFO (First-In, First-Out)
- LRU (Least Recently Used)
- Approximate LRU — the practical reality
- LFU (Least Frequently Used)

Working-set algorithm
Modern variants
Linux’s actual policy
Anonymous vs file-backed
Major vs minor faults
Thrashing
In databases
Common pitfalls
Try it
Read deeper
OST 8 — File system theory & journaling
What a filesystem must provide
On-disk layout (classic ext-style)
Inode
Path resolution
Allocation strategies
Concurrency in FS
Crash consistency — the hard part
fsync & friends — durability boundaries
File system zoo
VFS — Virtual Filesystem Switch
Network filesystems
Performance issues
Try it
Read deeper
OST 9 — Crash consistency & write ordering
The problem
Tools the OS uses
Approaches
Atomic rename — the user’s friend
Disk surprises you should know
Real-world durability (databases)
Common pitfalls
Try it
Read deeper
OST 10 — Distributed systems intro: consensus, RPC, time
Why a kernel engineer should care
Definitions
The fundamental impossibility — FLP
CAP theorem
Time and ordering
Consensus algorithms

RPC (Remote Procedure Call)
Replication
Quorums
Eventual consistency
Stronger models (reading)
CRDTs (Conflict-free Replicated Data Types)
Failure detectors
Saga / transactions across services
Real-world systems to know about
What this means for GPUs
Try it (mostly reading)
Read deeper
Closing the OS theory part
Chapter 23 — Threads, the C/C++ memory model
23.1 What a thread is
23.2 pthreads
23.3 The data race
23.4 The C/C++ memory model
23.5 The acquire-release pattern
23.6 Sequential consistency
23.7 Hardware memory ordering
23.8 What `volatile` is and isn't
23.9 Memory model in practice — a tiny lock-free queue
23.10 Thread-local storage
23.11 False sharing
23.12 Exercises
Chapter 24 — Mutexes, condition variables, semaphores
24.1 Mutex (mutual exclusion)
24.2 Recursive mutexes
24.3 Reader/writer locks
24.4 Spin lock
24.5 Deadlock
24.6 Condition variables
24.7 The producer/consumer queue
24.8 Semaphore
24.9 Once and lazy init
24.10 Barrier and latch (C++20)
24.11 Common bugs
24.12 Exercises
Chapter 25 — Atomics, lock-free data structures
25.1 The atomic operations
25.2 The lock-free counter

25.3 Lock-free stack (Treiber stack)
25.4 Single-producer single-consumer queue (revisited)
25.5 The MCS / queue lock
25.6 Read-Copy-Update (RCU)
25.7 Hazard pointers (briefly)
25.8 Memory barriers in the kernel
25.9 The `seq_lock`
25.10 Practical guidance
25.11 Exercises

Chapter 26 — The Linux kernel concurrency toolbox

26.1 Spinlock
26.2 Mutex
26.3 Semaphore (rare in modern kernel)
26.4 Completion
26.5 Wait queues
26.6 Reader-writer locks
26.7 RCU primitives
26.8 Atomic ops in the kernel
26.9 Per-CPU variables
26.10 Memory barriers
26.11 `lockdep`
26.12 KCSAN
26.13 Practical patterns
26.14 Exercises

Chapter 27 — GPU concurrency: fences, queues, doorbells

27.1 The mental model
27.2 Doorbells
27.3 Fences
27.4 Sync objects (`drm_syncobj`)
27.5 dma-buf and cross-driver sharing
27.6 Memory ordering CPU↔GPU
27.7 Coherent vs. write-combining vs. uncached for GPU
27.8 GPU scheduler in the kernel
27.9 Memory operations from the GPU's perspective
27.10 Practical takeaways for our future driver work
27.11 Exercises

Chapter 28 — The kernel source tree and how to build it

28.1 The tree
28.2 Configure
28.3 Build
28.4 Boot in a VM (faster + safer)
28.5 The first patch you write

28.6 Out-of-tree module
28.7 Reading the source — tools
28.8 Patch flow
28.9 Exercises
Chapter 29 — Loadable kernel modules: your first module
29.1 Anatomy of a module
29.2 `__init` and `__exit` annotations
29.3 The `printk` levels
29.4 Module dependencies
29.5 Symbol naming and collisions
29.6 Module parameters
29.7 Auto-loading via `MODULE_DEVICE_TABLE`
29.8 Init order
29.9 The `THIS_MODULE` and refcount
29.10 Loading and unloading machinery
29.11 A non-trivial module: chardev skeleton
29.12 Exercises
Chapter 30 — Kernel data structures: lists, rbtrees, xarray, idr
30.1 The intrusive doubly-linked list
30.2 hlist — hashed linked list
30.3 Red-black trees (`rb_tree`)
30.4 Interval trees
30.5 IDR / IDA / xarray
30.6 Lookup-and-insert atomic patterns
30.7 Bitmaps (review)
30.8 The `kfifo`
30.9 The `llist` — lock-less list
30.10 Reference counting
30.11 Putting it together — driver-internal indices
30.12 Exercises
Chapter 31 — Kernel memory: kmalloc, vmalloc, slab, page allocator
31.1 The hierarchy
31.2 GFP flags
31.3 `kmalloc` and `kfree`
31.4 `kvmalloc`
31.5 `vmalloc`
31.6 The slab allocator
31.7 The page allocator
31.8 `devm_*` — auto-free on detach
31.9 Reading memory pressure
31.10 KASAN, KFENCE, and DEBUG_VM
31.11 The `dma-buf` allocator (preview)

31.12 Practical guidance
31.13 Exercises
Chapter 32 — Time, timers, workqueues, kthreads
32.1 Time bases
32.2 Sleeping (process context only)
32.3 Busy-waiting (atomic context)
32.4 Timers — one-shot deferred work
32.5 High-resolution timers (`hrtimer`)
32.6 Workqueues — sleeping deferred work
32.7 Tasklets (deprecated)
32.8 Kthreads
32.9 RCU's `call_rcu`
32.10 The completion (review)
32.11 Time-outs in waits
32.12 Exercises
Chapter 33 — Locking in the kernel
33.1 Decision tree
33.2 Order of locks
33.3 Common lock idioms
33.4 Sleeping vs. atomic context — the bug
33.5 IRQ-safe variants
33.6 Read-side cost
33.7 Lock-free counters (per-CPU)
33.8 lockdep features
33.9 Deadlock avoidance recipes
33.10 Exercises
Chapter 34 — The driver model: kobject, sysfs, devices, drivers, buses
34.1 The objects
34.2 Buses you'll touch
34.3 sysfs
34.4 The probe/remove flow
34.5 Hot-plug
34.6 The component framework
34.7 Class
34.8 Power-management hooks
34.9 Attaching to the device tree (briefly, for SoCs)
34.10 Reading source
34.11 Exercises
Chapter 35 — Character drivers from scratch
35.1 Char vs. block
35.2 The major and minor numbers

35.3 `cdev` — the kernel object
35.4 The read/write contract
35.5 A real ring-buffer chardev
35.6 What we did right
35.7 What's missing (next chapters)
35.8 Exercises
Chapter 36 — `ioctl`, `mmap`, `poll`, async I/O
36.1 `ioctl` — control commands
36.2 `mmap`
36.3 `poll` / `epoll` / `select`
36.4 Async I/O / `O_NONBLOCK` / signal-driven
36.5 Notifying user via fd
36.6 Compat ioctls
36.7 Exercises
Chapter 37 — The PCI/PCIe subsystem and writing a PCI driver
37.1 The PCI fabric
37.2 The configuration space
37.3 The driver model on PCI
37.4 The QEMU `edu` device
37.5 Probe path
37.6 What's important
37.7 Beyond `edu`
37.8 Exercises
Chapter 38 — DMA, IOMMU, coherent vs streaming mappings
38.1 What DMA is
38.2 The two DMA mappings
38.3 Coherent example
38.4 Streaming example
38.5 Scatter-gather
38.6 DMA mask
38.7 The IOMMU
38.8 SWIOTLB / bounce buffer
38.9 dma-buf
38.10 Cache management
38.11 Minimal end-to-end DMA in our edu driver
38.12 Exercises
Chapter 39 — MMIO, register access, barriers
39.1 What MMIO is
39.2 The accessors
39.3 Posted writes
39.4 Memory barriers between CPU and device

39.5 Write combining
39.6 Endianness
39.7 Address validation
39.8 Debugging MMIO
39.9 amdgpu register naming
39.10 Exercises
Chapter 40 — Power management, runtime PM, suspend/resume
40.1 Two axes of PM
40.2 The `dev_pm_ops` struct
40.3 What suspend does, in order
40.4 Runtime PM lifecycle
40.5 amdgpu PM landscape
40.6 The “atomic suspend” rule
40.7 D-states (PCIe)
40.8 ACPI integration
40.9 Practical advice
40.10 Exercises
Chapter 41 — debugfs, sysfs, tracing, ftrace, perf
41.1 dmesg / printk
41.2 sysfs vs debugfs
41.3 ftrace
41.4 perf
41.5 BPF / bpftrace
41.6 GDB on the kernel
41.7 crash and pstore
41.8 KASAN, KCSAN, lockdep, DEBUG_VM
41.9 Memory leak detection
41.10 amdgpu-specific debug knobs
41.11 ROCm-side tools
41.12 Reading dmesg traces
41.13 Exercises
Chapter 42 — GPU architecture for software engineers
42.1 What a GPU is
42.2 The blocks you’ll touch
42.3 The command-processor model
42.4 The ISA (briefly)
42.5 GPU memory architecture
42.6 The whole stack, top to bottom
42.7 The KFD path (ROCm)
42.8 What a GPU “kernel” is
42.9 Bring-up overview (preview of Chapter 48)
42.10 Exercises

Chapter 43 — The Linux graphics stack from app to silicon
- 43.1 The big picture
- 43.2 The userspace driver's job
- 43.3 The display path: DRM/KMS
- 43.4 `/dev/dri/cardN` vs `/dev/dri/renderDN`
- 43.5 Key userspace bits
- 43.6 Patches typically span layers
- 43.7 Wayland and X11 differences (briefly)
- 43.8 ROCm path
- 43.9 Putting your first userspace prog together
- 43.10 Exercises

Chapter 44 — DRM, the kernel's GPU framework
- 44.1 What DRM provides
- 44.2 Where DRM glues in
- 44.3 The driver-author's deliverables
- 44.4 DRM file_operations
- 44.5 The ioctl table
- 44.6 The `drm_device`
- 44.7 The `drm_file`
- 44.8 Authentication and master
- 44.9 dma-buf / PRIME
- 44.10 KMS at a glance (full chapter later)
- 44.11 The DRM scheduler (full chapter later)
- 44.12 The minimal DRM driver
- 44.13 Exercises

Chapter 45 — GEM — buffer objects
- 45.1 What a GEM object is
- 45.2 Handles vs. objects
- 45.3 The lifecycle
- 45.4 amdgpu BO placement
- 45.5 Reserving and fences
- 45.6 The mmap path in depth
- 45.7 The VA bind path (GPU-side mapping)
- 45.8 dma-buf export/import
- 45.9 BO list
- 45.10 Userptr BOs
- 45.11 Reserved and pinned BOs
- 45.12 GEM helpers and "shmem" path
- 45.13 Exercises

Chapter 46 — TTM — the memory manager amdgpu uses
- 46.1 The problem TTM solves

46.2 The pieces
46.3 Placement
46.4 Migration and fences
46.5 Eviction
46.6 The “reservation” lock
46.7 The amdgpu_bo struct (real)
46.8 The buddy VRAM allocator
46.9 Sub-allocator
46.10 Performance pitfalls
46.11 The kernel call you’ll see most
46.12 Exercises
Chapter 47 — KMS — display, planes, CRTCs, atomic modesetting
47.1 The objects
47.2 The display path
47.3 Atomic modesetting
47.4 The driver’s job
47.5 EDID and modes
47.6 vblank
47.7 Page flip
47.8 Cursor and overlay planes
47.9 HDR, color, gamma
47.10 The DC architecture (briefly)
47.11 Headless / KFD
47.12 Exercises
Chapter 48 — The amdgpu driver — top-level architecture
48.1 Source map
48.2 The probe path
48.3 IP blocks
48.4 The amdgpu_device struct
48.5 The IH (Interrupt Handler) pipeline
48.6 The CS path (preview)
48.7 The amdgpu_ring abstraction
48.8 The fence path
48.9 GPUVM
48.10 RAS, reset, recovery
48.11 Reading the source: a recommended path
48.12 Exercises
Chapter 49 — Command submission, IBs, rings, doorbells
49.1 The big picture
49.2 An IB (Indirect Buffer)
49.3 The ring
49.4 The CS ioctl in detail

49.5 BO reservation with ww_mutex
49.6 Validation
49.7 Building the drm_sched_job
49.8 Doorbells
49.9 Multiple engines
49.10 MES — the firmware scheduler
49.11 Performance notes
49.12 Exercises
Chapter 50 — The GPU scheduler and dma-fence
50.1 The DRM scheduler
50.2 Entities
50.3 The lifecycle of a job
50.4 Dependencies
50.5 dma-fence
50.6 amdgpu fences
50.7 Timeouts and reset
50.8 Priority and preemption
50.9 Syncobjs and timeline semaphores
50.10 Locking model
50.11 amdgpu jobs in code
50.12 Exercises
Chapter 51 — VM, page tables, and unified memory
51.1 Why per-process VM
51.2 The structures
51.3 Mapping a BO
51.4 The PDE / PTE format
51.5 Eviction and re-binding
51.6 Userptr BOs and MMU notifiers
51.7 Shared Virtual Memory (SVM)
51.8 The GMC
51.9 Page faults
51.10 Concurrency
51.11 Summary mental model
51.12 Exercises
Chapter 52 — Userspace: libdrm, Mesa, ROCm, KFD
52.1 libdrm and libdrm_amdgpu
52.2 Mesa
52.3 ROCm runtime
52.4 Reading HIP/ROCm source
52.5 A minimal compute submission via libdrm_amdgpu
52.6 The Vulkan layer at AMD
52.7 The X11/Wayland integration

52.8 Tools you should have
52.9 Debugging userspace ↔ kernel
52.10 Exercises
Chapter 53 — Reset, recovery, RAS, and the real world
53.1 What “reset” means
53.2 Reset levels (modes)
53.3 The recovery code path
53.4 What the userspace sees
53.5 RAS — Reliability, Availability, Serviceability
53.6 Hang detection
53.7 Page-fault path
53.8 The watchdog and the “TDR” experience
53.9 Production-quality testing
53.10 What junior engineers can land
53.11 Example real-world patch shape
53.12 Exercises
Chapter 54 — Computer architecture for systems programmers
54.1 What a single core actually does
54.2 The pipeline
54.3 The memory hierarchy
54.4 Branch prediction and speculation
54.5 Out-of-order and barriers
54.6 SIMD
54.7 ARM/AArch64 differences
54.8 RISC-V briefly
54.9 GPU architecture vs. CPU
54.10 What this means for you
54.11 Exercises
Chapter 55 — Caches, coherence, and memory ordering
55.1 Cache structure
55.2 Coherence vs. consistency
55.3 The store buffer
55.4 Memory barriers
55.5 The infamous “double-checked locking”
55.6 The CPU↔device case
55.7 GPU caches
55.8 IOMMU and coherence
55.9 Practical guidance
55.10 Exercises
Chapter 56 — PCIe deep dive
56.1 PCIe is a layered packet protocol

56.2 Lanes and generations
56.3 TLP types
56.4 Posted vs. non-posted
56.5 Ordering rules (briefly)
56.6 BARs
56.7 ATS, PRI, ACS
56.8 Speed / latency
56.9 Common PCIe issues
56.10 Multi-GPU and P2P
56.11 Reading a real GPU’s PCIe state
56.12 Exercises
Chapter 57 — Performance: profiling, flamegraphs, GPU profilers
57.1 The scientific method
57.2 perf
57.3 Flame graphs
57.4 ftrace
57.5 bpftrace / bcc
57.6 GPU profiling
57.7 GPU performance counters
57.8 Where time goes in amdgpu
57.9 Memory bandwidth
57.10 Power and thermal
57.11 Practical tips
57.12 Exercises
Chapter 58 — Debugging: gdb, kgdb, crash, ftrace, dmesg, lockdep
58.1 dmesg first, always
58.2 gdb on userspace
58.3 gdb on the kernel (under QEMU)
58.4 kgdb on real hardware
58.5 crash and kdump
58.6 ftrace for kernel function tracing
58.7 lockdep, KASAN, KCSAN, KMSAN, UBSAN, KFENCE, DEBUG_VM
58.8 Driver-specific: amdgpu
58.9 Reproducing intermittent bugs
58.10 Reading core dumps
58.11 Static analysis
58.12 The triage checklist
58.13 Exercises
Chapter 59 — What AMD / NVIDIA / Qualcomm / Intel actually look for
59.1 The roles
59.2 What they’re really screening for
59.3 The bar for “competent junior” in 2026

59.4 What earns you FTE conversion
59.5 The 90-day plan when you join
59.6 Companies' cultures (rough)
59.7 The non-technical signals
59.8 Exercises
Chapter 60 — System-software interview question bank with answers
60.1 C language
60.2 C++
60.3 Operating systems
60.4 Multithreading and concurrency
60.5 Memory model and architecture
60.6 PCIe and DMA
60.7 GPU concepts
60.8 Debugging
60.9 System design
60.10 Exercises
Chapter 61 — Projects that get you hired
61.1 Tier-0 (you must have at least these)
61.2 Tier-1 (interview gold)
61.3 Tier-2 (top-tier)
61.4 Tier-3 (signature work)
61.5 What to put on github
61.6 Anti-patterns (avoid these)
61.7 The resume
61.8 The cover letter
61.9 Exercises
Chapter 62 — Contributing to the upstream kernel and Mesa
62.1 The kernel patch flow
62.2 The commit message
62.3 The DRM/amdgpu specifics
62.4 Mesa patch flow
62.5 ROCm and KFD specifics
62.6 Code review etiquette
62.7 Mistakes that get rejected
62.8 An example first patch
62.9 Long-term contribution rhythm
62.10 Resources
62.11 Exercises
Chapter 63 — The 12-month plan from contractor to FTE
Months 1–2: foundations
Months 3–4: kernel and concurrency

Months 5–6: DRM and amdgpu orientation
Months 7–8: ownership area
Months 9–10: real bug ownership
Months 11–12: convert
A weekly cadence
On burnout
On asking for help
On luck
Final words
Part X — The Senior / Principal / PMTS Track
Sections
How to use this part for FTE conversion
What this part is not
Chapter 64 — GPU microarchitecture: RDNA
Top-level block diagram
The hierarchy from top to bottom
Wave32 vs wave64 (the RDNA shift)
RDNA generations at a glance
How a triangle becomes pixels — the path through RDNA
What you should be able to do after this chapter
Read deeper
Chapter 65 — GPU microarchitecture: CDNA & matrix engines
How CDNA differs from RDNA
CDNA lineage
The matrix engine: MFMA
Memory subsystem on CDNA
Software differences vs RDNA
Multi-die, chiplets, packaging (CDNA 2 onward)
ECC
What you should be able to do after this chapter
Read deeper
Chapter 66 — Wavefronts, occupancy, VGPRs/SGPRs
What a wave actually is
Workgroup → wave mapping
Occupancy
Spilling (the enemy)
Divergence
Wave-level intrinsics
RDNA’s dual-issue (RDNA 3+)
CDNA’s MFMA pipeline
Practical occupancy tuning

- What the kernel driver has to know
- What you should be able to do after this chapter
- Read deeper

Chapter 67 — GPU memory subsystem: HBM, GDDR, controllers, coalescing

- Memory types
- VRAM: HBM vs GDDR
- Memory controllers
- Coalescing — the most important access pattern
- Address translation on the GPU
- Cache coherence on the GPU
- Bandwidth math
- VRAM layout decisions made by the kernel
- Memory hot paths a senior engineer cares about
- What you should be able to do after this chapter
- Read deeper

Chapter 68 — GPU cache hierarchy & coherence

- The hierarchy on RDNA (gen-typical, varies)
- On CDNA
- Cache-line size
- Coherence: the GPU model
- Wait counters and barriers
- Caching policy — `glc`, `slc`, `dlc`
- Texture cache (TC / TCC / TCP)
- Cache misses and how to fix them
- Inspecting cache behavior
- Coherence with the CPU (for SVM / ATS)
- Implications for the driver
- What you should be able to do after this chapter
- Read deeper

Chapter 69 — PM4 packets, command processor, firmware engines

- The firmware engines (one chip, many CPUs)
- What PM4 is
- The command processor pipeline
- How the kernel emits PM4 (in `amdgpu`)
- MES — the next-generation scheduler
- RLC — the unsung hero
- PSP boot order
- Versioning and pain
- Attack surface
- What you should be able to do after this chapter
- Read deeper

Chapter 70 — Geometry pipeline: vertex → tess → geom → mesh shaders
- The classic pipeline (DX10/OpenGL 3 era)
- How AMD actually maps stages — the merged-shader pipeline
- NGG (Next-Generation Geometry) — RDNA's big change
- Mesh shaders
- Primitive culling
- Index/vertex fetch
- Tessellation (HS + Tess + DS)
- Geometry shader — the cautionary tale
- Position-only / streamout
- Driver responsibilities
- Performance traps in geometry
- What you should be able to do after this chapter
- Read deeper

Chapter 71 — Rasterization, hierarchical Z, primitive culling
- What rasterization actually computes
- Pixel quads — the 2×2 unit
- Hierarchical Z (HiZ) and Hi-Stencil
- Render targets, depth, MSAA
- Primitive culling, again — at the rasterizer level
- Tile-based vs immediate-mode
- RDNA-specific: WGP-pair issuing
- Common rasterization perf issues
- Driver responsibilities
- Display interaction
- What you should be able to do after this chapter
- Read deeper

Chapter 72 — Pixel pipeline: RBE, ROPs, blending, MSAA
- Block diagram of the RBE
- ROPs ("Raster OPerations" units)
- Depth and stencil
- Blending
- DCC (Delta Color Compression)
- CMASK and FMASK
- Fast clears
- MSAA on the back end
- VRS (Variable Rate Shading) interaction
- Pixel shader output paths
- Cross-stage dependencies
- Driver responsibilities
- What you should be able to do after this chapter
- Read deeper

Chapter 73 — Ray tracing hardware
- What ray tracing actually computes
- The BVH
- AMD’s ray-accelerator approach
- RDNA 3 and 4 improvements
- DXR / VKR — the API picture
- Inline ray tracing (a.k.a. “Ray Queries”)
- BVH build
- Performance considerations
- Driver responsibilities
- Lumen, RTGI, path tracers
- What you should be able to do after this chapter
- Read deeper

Chapter 74 — KFD, HSA, AQL packets, MES
- The dual-driver story
- What HSA is
- AQL packet format
- The user-mode queue model
- Doorbells
- MES: scheduling user-mode queues
- KFD ioctls — the compute API
- Kernel object & code object
- Signals
- SVM (Shared Virtual Memory)
- Userspace stack
- Driver responsibilities (KFD)
- What you should be able to do after this chapter
- Read deeper

Chapter 75 — CWSR, priorities, preemption
- The problem
- Granularity options
- What CWSR saves
- When CWSR runs
- Time-slicing
- Priorities
- CU masking and partitioning
- Realtime + display interaction
- What kernels and runtimes worry about
- Preemption + reset
- Debug interactions
- Performance considerations
- Driver responsibilities

- What you should be able to do after this chapter
- Read deeper

Chapter 76 — ROCm runtime deep

- The big picture
- HIP — the CUDA-shaped API
- hipcc and the compiler chain
- Streams and events
- Multi-GPU
- hipBLASLt and rocBLAS
- MIOpen
- Composable Kernel (CK)
- PyTorch on ROCm
- Triton, JAX, TF, XLA
- Debugging compute
- Production realities
- What you should be able to do after this chapter
- Read deeper

Chapter 77 — Display Core (DCN) deep dive

- What “display” actually does
- DCN block diagram (RDNA / CDNA-A era)
- Pipes
- Planes
- Atomic modeset, in DC’s view
- Modes, timings, links
- EDID / DisplayID
- HDR and color
- Variable refresh (FreeSync / VRR / Adaptive Sync)
- DSC (Display Stream Compression)
- DMCUB — display firmware microcontroller
- Audio
- Reset and recovery
- Driver responsibilities (massive)
- What you should be able to do after this chapter
- Read deeper

Chapter 78 — Multimedia (VCN), codecs

- What VCN does
- VCN generations
- How decode works
- How encode works
- Latency targets
- VA-API and the userspace stack
- DRM (digital rights management)

DPB management
JPEG engine
VCN driver pieces in kernel
Cross-IP coordination
Power management for VCN
Testing and validation
What you should be able to do after this chapter
Read deeper
Chapter 79 — SMU, DPM, voltage/frequency, telemetry
What SMU manages
DPM — Dynamic Power Management
OPPs and DPM tables
Telemetry
SVI3 and serial voltage
Thermal management
Power capping
Customer/ISV-specific tuning
Power features per platform
RuntimePM in the kernel
SMU interface — kernel side
SMU firmware updating
What goes wrong
What you should be able to do after this chapter
Read deeper
Chapter 80 — Mesa, radv, ACO, NIR
Mesa structure (quick map)
NIR — Mesa's IR
ACO — the AMD-compute optimizer
radv — Vulkan on AMD
radeonsi — OpenGL on AMD
OpenCL on Mesa: rusticl
Compositor / WSI
How a draw call becomes silicon (Mesa view)
Testing
How to contribute
What you should be able to do after this chapter
Read deeper
Chapter 81 — LLVM AMDGPU backend
Why both LLVM and ACO?
LLVM source structure (AMDGPU side)
Compile flow
TableGen — the instruction database

Memory model handling
Wave size handling
Register allocation challenges
Optimization levels
AMDGCN-specific intrinsics
Code object layout
Multi-arch fat binaries
ROCm-LLVM vs upstream LLVM
What you should be able to do after this chapter
Read deeper
Chapter 82 — Vulkan & DX12 deep
Why explicit APIs
Pipeline state objects (PSOs)
Descriptor sets / descriptors / descriptor heaps
Synchronization
Render passes (Vulkan classic) vs dynamic rendering
Pipeline barriers — the bug factory
Compute in Vulkan
Mesh and ray tracing in Vulkan
DX12 on Linux
Drawing best practices (what every AAA team learns)
Driver architecture impact
Validation, tools
What you should be able to do after this chapter
Read deeper
Chapter 83 — SR-IOV and GPU virtualization
The problem
PCIe-level model
Host vs guest drivers
The Mailbox
VRAM and address-space partitioning
Time-slice vs space-slice
Reset isolation
Live migration
MIG-like vs partition
Security implications
Display & multimedia in SR-IOV
Tooling
What kernel code to read
What you should be able to do after this chapter
Read deeper

Chapter 84 — VM faults, page tables, recovery
- GPU page tables (recap from chapter 51)
- What “VM fault” means
- Two flavors
- The page-fault path in `amdgpu`
- Why recovery is hard
- TLB management
- Eviction and re-binding
- Userptr and MMU notifiers
- SVM (Shared Virtual Memory) extras
- Reset-and-recovery interactions (continued from chapter 53)
- Watching for faults in production
- Common fault signatures
- What you should be able to do after this chapter
- Read deeper

Chapter 85 — Security: PSP, FW signing, attestation, SEV
- Why GPUs need a security processor
- PSP basics
- Trusted Applications (TAs)
- Kernel↔PSP interface
- Firmware update flow
- Anti-rollback
- Confidential Computing on GPU (SEV-SNP era)
- Attestation
- HDCP and content protection
- Side-channel concerns
- Threat-model for a PMTS reviewer
- Where PSP code lives
- What you should be able to do after this chapter
- Read deeper

Chapter 86 — SQTT, RGP, perf counters, RGD
- The tools
- SQTT (Shader Quality Trace)
- How RGP capture works
- Hardware perf counters
- SPM — Streaming Performance Monitor
- Reading an RGP capture
- Reading a rocprof / omniperf capture
- Hot-path identification: a checklist
- RGD — crash forensics
- Capture tips
- Driver instrumentation

Counter combinations to learn
What you should be able to do after this chapter
Read deeper
Chapter 87 — Cross-stack debug & regression triage
The triage funnel
Reproducing
Localizing the layer
Bisecting
Reading dmesg
Reading trace-cmd output
umr — direct hardware introspection
rocgdb walkthrough
Cross-stack reasoning example
When the bug is in firmware
When it’s an upstream regression you didn’t cause
Production patterns
What you should be able to do after this chapter
Read deeper
Chapter 88 — Customer / ISV / game studio interaction
Who the customers are
What they want from you
A typical engagement
Profiling with a customer
Reporting customer-found driver bugs
Pre-silicon engagement
OEM dynamics
Cloud customer dynamics
ISV best practices for AMD
Conferences
Public communication
Ethical considerations
What you should be able to do after this chapter
Read deeper
Chapter 89 — Maintainership and upstream leadership
What “maintainer” means in Linux
How you become a maintainer
Reviewing patches
Sending pull requests
Conflict resolution
ABI stability — sacred
Working with downstream distros
The 6-month merge cycle

- Mesa / Mesa-like maintainership
- Working with bugbots
- Mentoring
- What you should be able to do after this chapter
- Read deeper

Chapter 90 — Silicon bring-up, post-silicon validation

- The phases
- Lab bring-up — the first hours
- Pre-tapeout simulation work
- Bring-up artifacts
- Communicating with hardware
- Post-silicon validation (PSV)
- Driver test plans for new silicon
- Errata handling pattern
- Pre-launch software polish
- After launch — sustaining
- Mentoring during bring-up
- What you should be able to do after this chapter
- Read deeper

Chapter 91 — Cross-vendor architecture: NVIDIA, Apple, ARM, Imagination

- The roster
- NVIDIA — the standard you measure against
- Apple — vertical integration
- ARM Mali
- Imagination Tech (PowerVR)
- Qualcomm Adreno
- Intel Arc / Xe
- How a PMTS uses cross-vendor knowledge
- Reading whitepapers
- Industry standards work
- What you should be able to do after this chapter
- Read deeper

Chapter 92 — The PMTS career arc — the 24-year view

- The shape of a career
- What changes at each level
- What kills a career at each stage
- The compounding investments
- The non-technical skills
- Money, family, location
- The horrible truth: not everyone makes PMTS
- A 24-year plan, sketched
- The alternative paths

- A final word
- Appendix — Toolchain cheat sheet
  - Build
  - Inspect
  - Debug
  - Kernel build
  - Out-of-tree module Makefile
  - Useful kernel debug knobs
  - git daily
  - DRM/amdgpu
  - ROCm
- Appendix — A QEMU + custom-kernel dev VM (do this once, save your life forever)
  1. Install host packages
  2. Get the kernel source
  3. Configure and build
  4. Build a minimal rootfs
  5. Boot it
  6. Attach gdb to the running kernel
  7. Add the QEMU `edu` device for the PCI lab
  8. Enable AMD GPU passthrough (optional, advanced)
  9. Snapshots
  10. The 30-second iterate loop
- Appendix — Kernel coding style (the rules that get patches accepted)
  1. Indentation
  2. Line length
  3. Braces
  4. Naming
  5. Comments
  6. Functions
  7. Locking
  8. Macros
  9. Don’t
  10. Run before sending
  11. Commit message
- Appendix — Glossary (every acronym you will hit)
- Appendix — Checklists you can paste into your dotfiles
  - “I just hit a kernel oops” checklist
  - “I just hit a hang” checklist
  - “I’m about to send a kernel patch” checklist
  - “I’m reviewing someone’s patch” checklist
  - “I’m starting at AMD on Monday” checklist (Day 0)

- “I’m building a portfolio commit” checklist
- Appendix — Curated reading list
  - Books — read end-to-end
  - Books — reference (not cover-to-cover)
  - Documentation you should bookmark
  - Talks (YouTube — search and watch)
  - Source code (read this, not just docs)
  - Mailing lists / forums
  - Reading order for an absolute beginner
- Appendix — A guided tour of `drivers/gpu/drm/amd/amdgpu/`
  - Tier 1 — read these first
  - Tier 2 — the resource managers
  - Tier 3 — command path
  - Tier 4 — interrupt + reset
  - Tier 5 — display + per-IP
  - Tier 6 — KFD (compute)
  - How to navigate it
  - Suggested reading sequence (your first month)
- Labs
  - Prerequisites
  - Tips

From Zero to AMD GPU Kernel Engineer

A self-contained textbook covering C, DSA, userspace systems programming, OS theory, the Linux kernel, device drivers, and the AMD GPU stack — for engineers aspiring to FTE roles at AMD, NVIDIA, Qualcomm, Intel, Broadcom.

Author: SID-Devu **Repository:** https://github.com/SID-Devu/amdgpu-learning **Generated:** 2026

This PDF combines the entire textbook into one document for offline reading. It includes:

- Part 0 — Absolute Zero (no programming knowledge required)
- Part 1 — C
- Part DSA — Data Structures & Algorithms (read-to-learn rewrites for chapters 1–7, 10, 14, 15)
- Part USP — Userspace Systems Programming
- Part 2 — C++
- Part 3 — Linux OS
- Part OST — Classic OS Theory
- Part 4 — Multithreading
- Part 5 — Kernel
- Part 6 — Device Drivers
- Part 7 — GPU & DRM
- Part 8 — Hardware/Software interface
- Part 9 — Career
- Part X — Principal/PMTS track
- Appendix
- Labs

Read the LEARNING-PATH section first for the recommended order.

From Zero to AMD GPU Kernel Engineer

> A self-contained, deeply technical textbook covering C, C++, operating systems, multithreading, the Linux kernel, Linux device drivers, and the AMD GPU (DRM/amdgpu) stack — written for a fresh graduate aiming at an FTE role at **AMD, NVIDIA, Qualcomm, Intel, Broadcom, MediaTek, Marvell** or any low-level systems / GPU / SoC team.

This book assumes **no prior programming knowledge at all**. We start from "what is a computer" and "how do I open a terminal" in **Part 0**. By the end (Part IX) you can:

- read and modify `drivers/gpu/drm/amd/amdgpu/*.c`,
- write a PCIe device driver from scratch,
- explain command submission, fences, TTM, and KMS in an interview,
- pass system-software interview loops at AMD/NVIDIA/Qualcomm.

⚠️ NEW — Read the LEARNING PATH first

Before anything else, read LEARNING-PATH.md. It tells you what order to read this book in (you do **not** read parts numerically the first time — there's a real path).

The 14-stage path: computer literacy → programming basics → C → **DSA** → **userspace systems programming** → C++ → multithreading → **OS theory** → kernel → drivers → GPU → HW/SW → career → senior track.

If reading "Chapter 1: What is a program, a compiler, an OS?" feels too advanced, **you are in the right place but on the wrong page**. Go to **Part 0 — Absolute Zero** first. It assumes you know nothing — not even how to use a terminal — and brings you up to the level where Part I makes sense.

If you already know basic Linux + Python or a little C, skim Part 0 and start at Part I.

How to use this book

1. **Read sequentially the first time.** Every part builds on the previous.
2. **Type every code example.** Don't copy-paste. Muscle memory matters.
3. **Do the labs** in `labs/` after each part.
4. **Re-read** the GPU parts after you've shipped your first kernel module.

Estimated time: **6–9 months** at 2 hrs/day, or **3 months** full-time. Add 2–3 weeks if you're starting from Part 0.

You will build, in order:

- a C memory allocator,

- a C++ smart pointer & a tiny STL,
- a userspace cooperative scheduler,
- a producer/consumer ring buffer (lock-based and lock-free),
- a Linux kernel module,
- a character device driver with `ioctl` and `mmap`,
- a virtual PCI driver against QEMU's `edu` device,
- a tiny DRM driver,
- a userspace GPU command submitter using `libdrm` against amdgpu.

Table of Contents

Part 0 — Absolute Zero (no programming knowledge required)

1. Start here
2. What is a computer? (in plain English)
3. Using Linux: the terminal
4. Files and paths
5. Text editors
6. What is code?
7. Your first program (in Python)
8. Decisions and repeats
9. Functions
10. From Python to C
11. When things break — debugging mindset
12. Bridge to Part I

Part DSA — Data Structures & Algorithms (in C) — *Stage 4 in the LEARNING PATH*

> **READ AFTER PART I.** Required for interviews **and** kernel work. 28 chapters.

See **partDSA/README.md** for the full table of contents. Topics: complexity, arrays, strings, linked lists, stacks/queues, hash tables, BSTs, RB/AVL trees, B-trees, heaps, graphs, shortest paths, tries, all the sorts, all the searches, recursion, DP, greedy, divide & conquer, bit tricks, two pointers, backtracking, union-find, segment / Fenwick / interval trees, KMP / Rabin-Karp / suffix arrays, system-design DS (LRU, bloom filter, skip list, consistent hashing), concurrent DS (RCU, lock-free queues), and a guided tour of the kernel's data structures.

Part USP — Userspace Systems Programming — *Stage 5 in the LEARNING PATH*

> **MASTER USERSPACE FIRST. THEN KERNEL.** 12 chapters.

See **partUSP/README.md** for the full table of contents. Topics: Unix philosophy & syscalls, processes (fork/exec/wait), signals, pipes/FIFOs, shared memory & mmap, message queues / semaphores / futex, sockets (TCP/UDP/Unix), epoll / io_uring, pthreads deep, file I/O deep (`O_DIRECT`, `sendfile`), `/proc` / `/sys` / `/dev`, perf tools (strace, perf, valgrind, gdb).

Part OST — Classic OS Theory — *Stage 8 in the LEARNING PATH*

> Academic foundation that complements Part III (Linux-specific). 10 chapters.

See **partOST/README.md** for the full table of contents. Topics: process model, scheduling theory (FCFS/SJF/RR/MLFQ/lottery/CFS/EDF), synchronization theory (Peterson/semaphores/monitors), classic problems (dining philosophers etc.), deadlock, memory theory (paging/segmentation/swap), page replacement, file system theory & journaling, crash consistency, distributed systems intro (consensus, RPC, time, CAP).

Part I — C from Zero to Driver-Grade

1. What is a program, a compiler, an OS?
2. Your first C programs and the toolchain
3. Types, integer promotion, undefined behavior
4. Pointers, arrays, the memory model of C
5. Structs, unions, bitfields, packing, alignment
6. Bit manipulation — the language of hardware
7. Dynamic memory and writing your own allocator
8. The preprocessor, headers, linkage, build systems
9. Function pointers, callbacks, opaque types, plugin design
10. Defensive C: const correctness, restrict, MISRA-ish style

Part II — C++ for Systems Programmers

1. From C to C++: classes, RAII, references
2. Object lifetime, special members, rule of 0/3/5
3. Templates, concepts, and the type system
4. Modern C++: move semantics, smart pointers, lambdas
5. STL and writing your own container
6. C++ for low-level work: PIMPL, polymorphism cost, ABI

Part III — Operating Systems Internals

1. What an OS is and the boot path on x86_64
2. Processes, threads, address spaces, syscalls
3. Virtual memory, paging, TLBs, MMUs
4. Scheduling — CFS, deadline, real-time
5. The I/O stack — VFS, page cache, block layer
6. Interrupts, exceptions, IRQ delivery

Part IV — Multithreading & Concurrency

1. Threads, the C/C++ memory model

2. Mutexes, condition variables, semaphores
3. Atomics, lock-free data structures
4. The Linux kernel concurrency toolbox
5. GPU concurrency — fences, queues, doorbells

Part V — Linux Kernel Foundations

1. The kernel source tree and how to build it
2. Loadable kernel modules — your first module
3. Kernel data structures: lists, rbtrees, xarray, idr
4. Kernel memory: kmalloc, vmalloc, slab, page allocator
5. Time, timers, workqueues, kthreads
6. Locking in the kernel
7. The driver model — kobject, sysfs, devices, drivers, buses

Part VI — Linux Device Drivers

1. Character drivers from scratch
2. `ioctl`, `mmap`, polling, async I/O
3. The PCI/PCIe subsystem and writing a PCI driver
4. DMA, IOMMU, coherent vs streaming mappings
5. MMIO, register access, barriers
6. Power management, runtime PM, suspend/resume
7. debugfs, sysfs, tracing, ftrace, perf

Part VII — The Linux GPU Stack (DRM + amdgpu)

1. GPU architecture for software engineers
2. The Linux graphics stack from app to silicon
3. DRM — the kernel's GPU framework
4. GEM — buffer objects
5. TTM — the memory manager amdgpu uses
6. KMS — display, planes, CRTCs, atomic modesetting
7. The amdgpu driver — top-level architecture
8. Command submission, IBs, rings, doorbells
9. The GPU scheduler and dma-fence
10. VM, page tables, and unified memory
11. Userspace: libdrm, Mesa, ROCm, KFD
12. Reset, recovery, RAS, and the real world

Part VIII — Hardware/Software Interface

1. Computer architecture for systems programmers
2. Caches, coherence, and memory ordering
3. PCIe deep dive
4. Performance: profiling, flamegraphs, GPU profilers
5. Debugging: gdb, kgdb, crash, ftrace, dmesg, lockdep

Part IX — Getting Hired (AMD / NVIDIA / Qualcomm / Intel)

1. What these companies actually look for
2. System-software interview question bank with answers
3. Projects that get you hired
4. Contributing to the upstream kernel and Mesa
5. The 12-month plan from contractor to FTE

Part X — The Senior / Principal / PMTS Track (the 24-year-engineer's map)

See `partX-principal/README.md` for how to use this part. Reference material — not a tutorial.

Hardware foundations 64. GPU microarchitecture: RDNA 65. GPU microarchitecture: CDNA & matrix engines 66. Wavefronts, occupancy, VGPRs/SGPRs 67. GPU memory subsystem: HBM, GDDR, controllers 68. GPU cache hierarchy & coherence 69. PM4 packets, command processor, firmware engines

The graphics pipeline, deeply 70. Geometry pipeline: vertex → tess → geom → mesh shaders 71. Rasterization, hierarchical Z, primitive culling 72. Pixel pipeline: RBE, ROPs, blending, MSAA 73. Ray tracing hardware

Compute, runtime, queues 74. KFD, HSA, AQL packets, MES 75. CWSR, priorities, preemption 76. ROCm runtime deep

Display, multimedia, power 77. Display Core (DCN) deep dive 78. Multimedia (VCN), codecs 79. SMU, DPM, voltage/frequency, telemetry

Userspace stack deep 80. Mesa, radv, ACO, NIR 81. LLVM AMDGPU backend 82. Vulkan & DX12 deep

Driver advanced topics 83. SR-IOV and GPU virtualization 84. VM faults, page tables, recovery 85. Security: PSP, FW signing, attestation, SEV

Performance & debug at scale 86. SQTT, RGP, perf counters, RGD 87. Cross-stack debug & regression triage

Engineering at staff/principal level 88. Customer / ISV / game studio interaction 89. Maintainership and upstream leadership 90. Silicon bring-up, post-silicon validation 91. Cross-vendor architecture: NVIDIA, Apple, ARM, Imagination 92. The PMTS career arc — the 24-year view

Labs

See `labs/README.md` — runnable C, C++, kernel, and DRM exercises:

1. `01-arena/` — arena allocator (C)
2. `02-spsc/` — lock-free SPSC ring buffer (C++)
3. `03-thread-pool/` — mutex+condvar thread pool (C++)
4. `04-hello-mod/` — first kernel module
5. `05-myring-chardev/` — char driver with `read` / `write` / `poll` / `ioctl`
6. `06-edu-pci/` — PCI driver against QEMU's `edu` device
7. `07-vkms-skeleton/` — read & extend the in-tree minimal DRM driver
8. `08-libdrm-amdgpu/` — userspace amdgpu BO + IB submission

Appendix

- Toolchain cheat sheet — build, debug, kernel commands
- Kernel coding style — the rules that get patches accepted
- Reading the amdgpu source map — guided tour of `drivers/gpu/drm/amd/amdgpu/`
- QEMU + custom-kernel dev VM — your daily reproducer
- Reading list — books, papers, talks, source
- Checklists — oops, hang, patch, review, day-zero
- Glossary — every acronym you will hit

Prerequisites for the labs

You need a Linux box (Ubuntu 22.04+ or Fedora 39+ recommended) with:

```
sudo apt install build-essential gdb clang lld llvm \
                 linux-headers-$(uname -r) \
                 git make bison flex libssl-dev libelf-dev \
                 qemu-system-x86 qemu-utils \
                 libdrm-dev mesa-utils \
                 strace ltrace perf trace-cmd
```

For amdgpu-specific work you also want:

```
sudo apt install libdrm-amdgpu1 libdrm-dev rocm-smi-lib
```

How this book differs from LDD3 and from random tutorials

- **LDD3** (Linux Device Drivers, 3rd ed., Corbet/Rubini/Kroah-Hartman) is the canonical reference but it stops at kernel ~2.6.10 (2005). Many APIs have changed: `cdev_init`, `class_create`, `device_create`, `dma-buf`, `iommu`, the DRM/GEM/atomic-KMS stack — all post-LDD3. We rebuild on a modern (≥6.x) kernel.
- **Random tutorials** teach you `hello_world.ko` and stop. We go all the way to *"why does `amdgpu_cs_ioctl` need a `drm_sched_job` and a `dma_fence`?"*.
- **This book** is opinionated about what is real-world for a junior at AMD: *PCIe, DMA, ring buffers, fences, TTM, KMS, reset paths, runtime PM, lockdep*. Those are the things you'll actually be debugging in your first year.

A note on the path

If you are currently a contractor and want FTE: the people who convert are the ones who:

1. **Ship code that lands upstream**, even if tiny. (One real `Signed-off-by` in `drivers/gpu/drm/amd/` from you is worth ten LeetCode mediums.)
2. **Debug something nobody else can.** Pick the ugliest open bug nobody is touching and become the expert.

3. **Are visibly competent in code review.** Read patches on `amd-gfx@lists.freedesktop.org`, ask intelligent questions.
4. **Know the hardware.** Read RDNA / CDNA ISA documents. Know what an SDMA queue is.
5. **Don't need hand-holding.** Get the kernel to build, get amdgpu to load, and triage your own bugs before asking for help.

This book is engineered to make all five of those true for you.

Let's begin.

THE LEARNING PATH — read this first

> **You don't know how to program. That is fine. You will become an AMD GPU kernel engineer in 18–24 months if you follow this path. Don't skip stages. Don't read out of order on the first pass.**

The book has many parts. You will not read them all at once and you will not read them in numerical order. **You read them in the order below.** This page is your compass.

The big picture

```
Stage 1: Computer literacy        (you can use Linux at all)
      ↓
Stage 2: Programming basics       (Python + tiny C)
      ↓
Stage 3: C mastery                (the language of the kernel)
      ↓
Stage 4: Data Structures & Algorithms   (DSA — for interviews + kernel)
      ↓
Stage 5: Userspace systems programming   (master Unix as a user)
      ↓
Stage 6: C++ for systems          (Mesa, ROCm, AMF)
      ↓
Stage 7: Multithreading & memory models   (atomics, locks, lock-free)
      ↓
Stage 8: Operating systems theory  (the academic foundation)
      ↓
—— you have now mastered userspace + theory ——
      ↓
Stage 9: Linux kernel foundations  (build it, modules, kernel APIs)
      ↓
Stage 10: Linux device drivers     (char drivers, PCI, DMA, mmap)
      ↓
Stage 11: GPU stack — DRM/amdgpu   (the real job)
      ↓
Stage 12: Hardware/software interface  (caches, PCIe, perf, debug)
      ↓
Stage 13: Career — get the FTE     (interview, projects, upstream)
      ↓
Stage 14: Senior / Principal track  (the 24-year arc)
```

Everything before "you have now mastered userspace + theory" is mandatory. Don't enter the kernel until you've earned it.

Stage-by-stage table

Times assume **1 hour/day, 5 days/week**. Double the pace if you do 2 hours/day.

Stage	Duration	Where
1. Computer literacy	1 week	Part 0 ch 0–4 + Foundations A4

Stage	Duration	Where
2. Programming basics (Python + intro C)	2 weeks	Part 0 ch 5–11
3. C mastery	6 weeks	Part I (10 chapters) + labs 01
4. Data Structures & Algorithms	8–10 weeks	Part DSA
5. Userspace systems programming	4–6 weeks	Part USP
6. C++ for systems	3 weeks	Part II (6 chapters) + labs 02 + 03
7. Multithreading & memory models	4 weeks	Part IV (5 chapters) + lab 02, 03 deep
8. Operating systems theory	4–6 weeks	Part III (6 chapters) + Part OST (10 chapters)
9. Linux kernel foundations	6 weeks	Part V (7 chapters) + labs 04, 05
10. Linux device drivers	6 weeks	Part VI (7 chapters) + lab 06
11. GPU stack — DRM/amdgpu	8–12 weeks	Part VII (12 chapters) + labs 07, 08
12. Hardware/software interface	3 weeks	Part VIII (5 chapters)
13. Career — get FTE	parallel from Stage 9	Part IX (5 chapters)
14. Senior / Principal track	parallel for years	Part X (29 chapters)

Total time to be ready for the FTE conversion attempt: 12–18 months at 1 hr/day.

You can stretch or compress, but **do not skip stages**.

Why this order? (the philosophy)

“Userspace first, then kernel” — the unbreakable rule

Why?

- **You cannot debug a kernel module if you can’t write a 100-line C program.**
- **You cannot reason about driver locking if you can’t write multi-threaded userspace code.**
- **You cannot understand `/dev/dri/card0` if you don’t understand `/dev/null`, `pipe()`, `read()`, `write()`.**

Every kernel concept has a userspace mirror. Master the mirror first; the kernel is then “just like that, but harder.”

“DSA before kernel” — for two real reasons

1. **Interviews require it.** Every interview at AMD/NVIDIA/Qualcomm has a coding round. They will ask: “implement a hash map.” “Reverse a linked list.” “Find the kth largest.” If you can’t do these in 30 minutes in C, you don’t get the offer. Period.

2. **The kernel uses these data structures everywhere.** `list_head` (linked list), `rb_root` (red-black tree), `xarray` (radix tree), `hash_table.h` (hashtable), `kfifo` (queue). If you've implemented them yourself once, the kernel reads like prose.

"OS theory after multithreading" — because...

Multithreading **is** OS theory in practice. Once you've fought race conditions in pthreads, the academic content (Peterson's algorithm, semaphores, dining philosophers, deadlock) makes sense in 1/10th the reading time.

"GPU last" — because the GPU is just a hard PCI device

The GPU stack assumes: - you understand C (Stage 3), - you understand pointers and address spaces (Stage 8), - you understand kernel modules (Stage 9), - you understand PCI/DMA/MMIO (Stage 10), - you understand graphics/compute APIs at least at a conceptual level (Stages 11+).

If any of those are shaky, the GPU chapters will feel impossible. They are not impossible — your foundation is just thin.

Daily rhythm (the secret to actually finishing)

1 hour/day, 5 days/week:

```
┌──────────────────────────────────────────┐
│ 0:00 - 0:05  Open notebook. Read your    │
│              note from yesterday.         │
├──────────────────────────────────────────┤
│ 0:05 - 0:35  Read one new chapter or     │
│              section. Type every code    │
│              example.                     │
├──────────────────────────────────────────┤
│ 0:35 - 0:55  Do the exercises / write    │
│              code. Run it. Break it.     │
├──────────────────────────────────────────┤
│ 0:55 - 1:00  Write 3 sentences in your   │
│              notebook: "today I"         │
└──────────────────────────────────────────┘
```

Three days a week, end with `git push` to your portfolio repo.

One day a week (e.g. Saturday), spend 2 hours doing a **review**: re-read confusing parts, redo a hard lab, or read a kernel patch on lore.kernel.org just to absorb idiom.

Don't binge. People who study 8 hours on Sunday and 0 the rest of the week forget everything. People who study 1 hour every day remember.

Three checkpoints — can you answer "yes"?

Checkpoint 1 — End of Stage 4 (DSA)

- [ ] I can implement a singly linked list, hash table, and binary search tree from scratch in C, in 30 minutes each.
- [ ]

- [ ] I can compute the time complexity of a function I just wrote.
- [ ] I can solve a “medium” Leetcode problem in C without help.
- [ ] I can read kernel `list_head` / `hlist` / `rbtree` code without confusion.

If yes: continue. If no: **stay in DSA for 4 more weeks.**

Checkpoint 2 — End of Stage 8 (OS theory)

- [ ] I can write a multi-threaded program with mutexes and condition variables that doesn’t race.
- [ ] I can write a program that uses `fork`, `exec`, `pipe`, `signal` correctly.
- [ ] I can explain virtual memory, paging, and the difference between user and kernel mode.
- [ ] I can write a TCP echo server and an `epoll`-based one.

If yes: continue. If no: **redo Stages 5+7+8 with more labs.**

Checkpoint 3 — End of Stage 11 (GPU stack)

- [ ] I have written a working kernel module with `read`/`write`/`ioctl`/`mmap`.
- [ ] I have written a PCI driver against QEMU `edu`.
- [ ] I have used libdrm_amdgpu to allocate a BO and submit a tiny PM4 IB.
- [ ] I can explain DMA, DRM scheduler, dma-fence, and GPUVM.
- [ ] I have read at least one full file in `drivers/gpu/drm/amd/amdgpu/` and understood ≥80%.

If yes: **start interviewing for FTE.** If no: pick the weakest area and re-deepen.

What if I’m in a hurry?

If you have a deadline (e.g., “I have 6 months to convert from contractor to FTE”):

Time you have	Strategy
6 months	2 hours/day. Skip Part X (you can read after FTE). Cut some DSA depth (do top 100 problems, not 200). Total: ~365 hours.
3 months	Full-time (8 hours/day). All stages. Will be intense but doable. ~480 hours.
< 3 months	Stage 9–11 only, plus sufficient C. Cram the rest. Risky; reread later.

In all cases: **don’t fake mastery.** Recruiters and senior engineers can tell.

What if I get stuck for a week?

Welcome to engineering. The 30-minute rule (Part 0 chapter 10): if stuck > 30 min, walk + write up your problem + search + ask. If a whole week is gone:

1. **Reduce scope**: pick the one specific concept you're stuck on; ignore the rest.
2. **Get a smaller test case**: 5 lines that exhibit your confusion.
3. **Ask in public**: amd-gfx@lists.freedesktop.org for AMD Q's, kernelnewbies.org for kernel Q's, Stack Overflow for C/Linux.
4. **Sleep on it 2 nights.** Often this alone fixes things.
5. **Go back one chapter.** Sometimes the gap is in the prerequisite.

There is no week of confusion you can't escape. Every senior engineer has had them. They keep going.

The single hardest moment

Around month 4–6, you'll feel like "everyone is faster than me, I'll never get this." That moment passes. It always passes. Push through.

The 24-year PMTS engineer at AMD also had this moment. Probably more than once.

Now go

Open Part 0 / Chapter 0 — Start here and begin.

Or, if you've done Stage 1 already, go to your current stage from the table above.

Welcome to the path.

Part 0 — Absolute Zero

> **Read this if you have never written a line of code in your life.**
>
> This part assumes **nothing**. Not even that you know how to open a terminal. We will go from "what is a computer" to "I just wrote my first C program," step by step, slowly, with no jargon.

The rest of the book (Parts I–IX) is hard. It has to be — your job is hard. But you cannot start there. You have to walk before you run, and run before you fly. **Part 0 is the walking.**

Who this part is for

- People who got a degree but never really wrote code.
- People who feel "everyone else seems to already know this stuff."
- People who tried programming books before and gave up because chapter 1 was already too much.

That's most fresh graduates. You are not behind. You are just starting. Welcome.

How long it takes

About **2–3 weeks** at 1 hour a day. Less if you already know some of this; more if you take your time. **Take your time.** There is no prize for finishing fast.

How to read this part

1. Read one chapter a day. Don't binge.
2. **Type every command** by hand. Do not copy-paste. Your fingers learn what your eyes don't.
3. If you don't understand a sentence, **read it twice more**, then move on. It will make sense by chapter 5.
4. After each chapter, do the "Try it" boxes. They take 5 minutes each.
5. **You are allowed to be stuck.** Being stuck is normal. Chapter 10 is about exactly this.

Table of Contents

#	Chapter	What you'll learn
0	Start here	What this part is, what mindset to bring
1	What is a computer?	CPU, RAM, disk — in plain English
2	Using Linux: the terminal	Open a terminal. Type commands. Don't be afraid

#	Chapter	What you'll learn
3	Files and paths	What `/`, `~`, `.`, and `..` mean
4	Text editors	How to actually create and edit a file
5	What is code?	Source code, running a program, the big picture
6	Your first program (in Python)	`print`, variables, arithmetic — gently
7	Decisions and repeats	`if`, `while`, `for`, in Python
8	Functions	Naming a piece of code so you can reuse it
9	From Python to C	The same ideas in C — your real working language
10	When things break	The debugging mindset (the most important chapter)
11	Bridge to Part I	You are ready. Here's the map.

Foundations side-track (read on demand)

These are *remedial* topics. If you ever feel "everyone seems to assume I know this," read the relevant one and come back.

#	Chapter	Read when
A1	Numbers — binary, hex, decimal	You see `0xCAFE` or `1 << 12` and don't know what they mean
A2	Bits, bytes, words	You're unsure what a byte is, or KB vs KiB
A3	Boolean logic — AND/OR/NOT/XOR	Bit operations confuse you
A4	Shell power tour	You want pipes, grep, scripts beyond chapter 2
A5	Character encoding — ASCII / UTF-8	Text shows up garbled or you hit a CRLF mystery

See `foundations/README.md` for the index.

Why we use Python first

You will end up writing **C** for your job (and that's what Part I onwards teaches). But C is unforgiving — small mistakes cause weird crashes that confuse beginners.

Python is friendly. It tells you exactly what went wrong in plain English. Using it for chapters 6–8 lets you focus on *the ideas* (variables, loops, functions) without fighting the language.

In chapter 9, we re-do every Python program in C, side by side, so you see the same idea in both. Then we never come back to Python.

What you need

- A computer with **Linux installed** (Ubuntu 22.04 or later is easiest).
  - If you only have Windows: install **WSL2** (one Google search away). It gives you a real Linux inside Windows.
  - If you have Mac: most things in this book work, but get a Linux VM eventually.
- An internet connection.
- 1 hour a day for 2–3 weeks.
- A notebook (paper). Yes, paper. You will draw boxes and arrows.

That's it. Let's start.

➡️ Next: Chapter 0 — Start here

Chapter 0 — Start here

This chapter has no code in it. Just words. **Read it slowly.** It will save you weeks of confusion later.

You are not behind

Most people who join AMD, NVIDIA, or any chip company started exactly where you are. They were 22, fresh out of college, and the first time they opened the Linux kernel source they thought “I will never understand this.” And then 6 months later they did.

The brain learns this stuff. It just takes repetition.

The trick is **don’t quit when something is confusing.** Confusion is not a sign that you are stupid. Confusion is the feeling of your brain *building the model*. If you push through, the confusion clears. If you give up, it doesn’t.

The three rules

Rule 1: Type, don’t copy.

When this book shows a command or a piece of code, you will type it yourself, character by character, into your terminal. No copy-paste. This feels slow. It is the fastest way to learn. Your fingers and your eyes need to see the same thing in two places before your brain remembers it.

Rule 2: When something doesn’t work, read the error message.

90% of beginners’ problems are because they didn’t read the error. The computer literally tells you what is wrong. It might use words you don’t know yet — that’s OK, you will learn them. But it is *not* random. The error is the answer.

We will practice reading error messages in chapter 10.

Rule 3: One thing at a time.

You will see code that looks long and scary. Always **break it into small pieces**. Read one line. Understand it. Then the next. Don’t try to understand a 50-line function at once.

What “understanding” means

You don’t need to memorize anything in this book. You need two things:

1. **Recognition** — “I have seen this before. I know roughly what it does. If I need details, I know where to look.”
2. **Recall of the few core ideas** — Pointers. Memory. Pages. Locks. These you must internalize.

Everything else, you look up. **Senior engineers Google constantly.** They look up function signatures, header file paths, syscall numbers, register layouts. The job isn’t to memorize. The job is to think clearly.

What good vs bad looks like

Bad day: You read a chapter, understand 60% of it, feel frustrated, watch YouTube instead.

Good day: You read the chapter, understand 60% of it, write down the 3 things that confused you in your notebook, and move on. Tomorrow, after the next chapter, 2 of those 3 things make sense. The 3rd you ask in a forum or to me in another chat. By the end of the week the entire chapter clicks.

The goal is not to *not be confused*. The goal is to **stay calm while confused** and keep going.

What "real-world" means in this book

Some books teach you toy programs that have no use anywhere. This book is different. Every example here is something you will actually see at AMD:

- The terminal commands in Chapter 2 are the ones you will run on your work laptop every day.
- The C programs in Chapter 9 use the same patterns the kernel uses.
- The debugging chapter (10) is exactly how senior engineers diagnose driver crashes.

You are not learning "school stuff." You are learning **the job.** Slowly, from the bottom.

Try it (your first homework)

Right now, before chapter 1:

1. Get a paper notebook. Title the first page "Linux device drivers — daily log."
2. Write today's date. Write one sentence: "Today I started Part 0."
3. At the end of every study session, write 1–3 sentences:
    - what you learned
    - what confused you
    - what you'll try tomorrow

That notebook is going to be your most valuable tool for the next year. Don't skip it.

A note about feeling lost

You will, at many points in this book, feel completely lost. That is the experience. Every single engineer at AMD has felt the same way at some point. Some of them still do, and they ship code anyway.

Lost is OK. Stuck is OK. **Quitting is not OK.**

When you feel lost, the answer is almost always one of:

- Re-read the previous chapter.
- Type out the code by hand again.
- Run the program with one small change to see what happens.
- Sleep. The next morning, it often clicks.

OK. Take a breath. Let's meet the computer.

➡️ Next: Chapter 1 — What is a computer?

Chapter 1 — What is a computer? (in plain English)

We will eventually talk about caches, page tables, MMIO, and DMA. **Not today.** Today we just learn what a computer *is*, in everyday words.

The kitchen analogy

Imagine a small kitchen with one cook. The cook can do simple things very fast: chop, stir, taste.

Kitchen	Computer
The **cook**	The **CPU** (the thing that does work)
The **counter** in front of the cook	**RAM** (memory you can use *right now*)
The **fridge / pantry**	The **disk** (storage that survives even when off)
The **recipe** the cook is following	The **program**
Ingredients on the counter	**Data** in memory

Three rules of this kitchen:

1. The cook can only work with what's on the counter. If an ingredient is in the fridge, somebody has to bring it to the counter first. (CPUs only operate on what's in RAM, or even closer — in registers/caches.)
2. The counter is small. The fridge is huge. (RAM is GBs; disk is hundreds of GBs to TBs.)
3. The cook is *very* fast. Going to the fridge is *very* slow compared to looking at the counter. (RAM is ~100 times slower than the CPU's tiny on-chip caches; disk is ~100,000 times slower than RAM.)

Most performance problems in computers come from rule 3.

The five parts you should know by name

```
        ┌──────────────────────┐
        │        CPU           │   does work
        │   (cook)             │
        └──────────────────────┘
                  │
        ┌──────────────────────┐
        │        RAM           │   short-term workspace
        │    (counter)         │
        └──────────────────────┘
                  │
        ┌──────────────────────┐
        │        Disk          │   long-term storage
        │    (fridge)          │
        └──────────────────────┘
                  │
     ┌──────────────────────────┐
     │  Other devices           │
     │  GPU, NIC, USB, screen, ...  │
```

```
                    └──────────────────────────┘
                                  │
                      ┌──────────────────────┐
                      │  You (human)         │
                      │  keyboard, mouse     │
                      └──────────────────────┘
```

That's it. That's a computer. Everything else is detail.

"What is the GPU then?"

The GPU is *another* worker. Imagine the cook has a friend in the same kitchen — a friend with **many hands** who can do simple things in parallel: chop 64 carrots at once, but can't read complicated recipes alone.

	CPU	GPU
Number of "workers"	4–32 fast cores	thousands of small cores
Good at	branchy logic, anything	doing the same math on lots of data
Has its own memory?	uses system RAM	usually has **VRAM** (its own RAM)
Talks to it via	direct CPU instructions	"send a recipe over a cable" (PCIe)

The CPU writes a recipe ("multiply these 4 million numbers by 2") on a piece of paper, hands it to the GPU through a slot in the wall (the **PCIe** cable), and goes off to do other things. The GPU does the math, hands the result back through the slot.

That "writing the recipe" + "passing it through the slot" + "telling the GPU when to start" is what an **AMD GPU driver** does. That's your future job in a single sentence.

Where Linux fits in

Imagine the kitchen has many cooks who all want to use the same counter, the same fridge, the same friends-with-many-hands. They can't all just grab things, or chaos. So there is a **head chef** who:

- decides who gets the counter when
- protects the fridge so nobody steals
- keeps every cook from interfering with the others
- talks to the friends-with-many-hands on behalf of every cook

That head chef is the **operating system**. **Linux** is one head chef. **Windows** and **macOS** are other head chefs. They all do roughly the same job, in different ways.

A **device driver** is a part of the head chef that knows how to talk to one specific friend. The amdgpu driver knows how to talk to AMD's GPUs.

The screen, the keyboard, and "everything is a device"

When you press a key on your keyboard:

1. The keyboard hardware sends a tiny electrical signal.

2. A driver inside Linux (the keyboard/USB driver) catches the signal and turns it into “the user pressed the letter A.”
3. Linux passes that information to whatever program you’re typing into.

When a program wants to draw a window:

1. The program asks Linux to draw something.
2. Linux passes the request to the GPU driver.
3. The GPU driver turns it into instructions the GPU can run.
4. The GPU does the drawing into a piece of memory.
5. The display hardware reads that memory and lights up the pixels you see.

This is the *whole* model. Everything we do for the rest of the book is filling in details about each step.

“But what is *inside* the CPU?”

Just enough to know:

- The CPU has a small set of named slots called **registers** — like cubbies on the cook’s apron. Each holds one number. Reading from a register is the fastest possible thing the CPU does.
- It also has tiny, fast **caches** (called L1, L2, L3) — basically a magnetic strip *on the counter itself* that holds copies of recently-used ingredients so the cook doesn’t keep walking to the fridge.

That’s all you need for now. We’ll come back to caches in Part VIII when it actually matters.

“What about the cloud, the internet, AI, …”

A “server” is just a computer in someone else’s room. The “cloud” is just lots of those rooms. The “internet” is wires connecting rooms.

When you watch a YouTube video: the video file is on some server’s *disk*, sent over the *network* to your computer’s RAM, decoded by *your CPU and GPU*, drawn to *your screen*. Same five parts. Just two computers connected by a wire.

It’s all the same picture, made bigger.

Try it (5 minutes, no code yet)

In your notebook, draw the diagram from this chapter: CPU, RAM, disk, GPU, you. Label every box. Draw arrows for “data goes this way” between them.

Don’t peek at the chapter while you do it. Drawing it from memory is the point.

If you get it wrong, that’s fine — flip back, find what was wrong, and re-draw.

Self-check

- What’s the difference between RAM and disk? (Hint: counter vs fridge.)
- Why is going from CPU to disk slow? (Hint: distance and design — it’s not about cables, it’s about the kind of memory.)

- Why does a GPU need a special driver and a regular keyboard doesn't, sort of? (Trick question — the keyboard does have a driver. But the GPU's driver is way more complex because the GPU is a whole *other computer* that takes instructions, while a keyboard just sends key codes.)

You now know what a computer is. Tomorrow we will actually open one.

➡️ Next: Chapter 2 — Using Linux: the terminal

Chapter 2 — Using Linux: the terminal

The **terminal** (also called the **shell** or **command line**) is a window where you type instructions and the computer does them. It looks scary because it's all black and text. It is not scary. It is your most powerful tool.

Everyone at AMD uses a terminal every day. After this chapter, so will you.

Open a terminal

- **On Ubuntu/Linux Mint**: press `Ctrl + Alt + T`.
- **On WSL2 (Windows)**: open the **Ubuntu** app from your Start Menu.
- **On Mac**: press `Cmd + Space`, type "Terminal", press Enter. (For full Linux compatibility, install Linux in a VM later.)

You'll see something like this:

```
sudhdevu@laptop:~$
```

That's called the **prompt**. It is waiting for you to type. Every time you see `$` in this book, that's the prompt — you don't type the `$`, you type after it.

Your first command

Type this and press Enter:

```
echo "hello"
```

You should see:

```
hello
```

That's it. You used a computer the way professionals use it.

`echo` is a command that just prints whatever you give it. Try a few more:

```
echo "I am learning Linux"
echo 1 + 1
```

The second one prints `1 + 1`, not `2`. The terminal does not do math. `echo` just repeats words.

What is a "command"?

A command is a tiny program that does one thing. Linux comes with hundreds of them. You will only use about 30 every day. Here are the absolute essentials. **Type each one.** Don't read them.

See where you are

```
pwd
```

`pwd` = **print working directory**. The "directory" is the folder you are currently inside. It will print something like `/home/sudhdevu`. We'll explain what that means in chapter 3.

List what's here

```
ls
```

`ls` = **list**. Shows the files and folders in the current directory.

Variations:

```
ls -l        # long form, shows sizes and dates
ls -a        # also shows "hidden" files (those starting with .)
ls -la       # both
ls /etc      # list a specific directory
```

The `-l`, `-a` parts are called **flags** or **options**. They modify what the command does.

Move into a directory

```
cd Documents
```

`cd` = **change directory**. Now you are inside `Documents`. Run `pwd` and `ls` to confirm.

```
cd ..        # go back up one directory
cd ~         # go to your home directory
cd /         # go to the root of the filesystem
cd -         # go back to the last directory you were in
```

Make a folder

```
mkdir my-first-folder
ls
cd my-first-folder
pwd
```

You just created a folder, listed your current dir to confirm, moved into it, and printed where you are.

Make a file (the easy way)

```
touch hello.txt
ls
```

`touch` creates an empty file (or, if it exists, just updates its timestamp).

See what's inside a file

```
echo "this is some text" > note.txt
cat note.txt
```

The `>` says "take whatever this command would print, and put it in a file instead." So `echo "this is some text" > note.txt` creates `note.txt` with the text inside.

`cat` = **concatenate**, but in practice it just dumps the contents of a file to your screen.

For long files, use `less`:

```
less /etc/services
```

In `less`: press `Space` to scroll down, `b` to scroll up, `/word` to search, `q` to quit.

Copy, move, delete

```
cp note.txt note-copy.txt        # copy
mv note-copy.txt renamed.txt     # rename / move
rm renamed.txt                   # delete (PERMANENT — no recycle bin!)
```

`rm` does not move things to a recycle bin. Once it's gone, it's gone. Be careful.

The single most dangerous command in Linux is:

```
rm -rf /        # NEVER RUN THIS — it deletes everything
```

Don't run that. Don't even type it.

Find help

Every command has a manual page. Try:

```
man ls
```

(`q` to quit.) The man pages are dense. As a beginner, prefer:

```
ls --help
```

It usually fits on one screen.

A useful starter session, end to end

Type all of this. Don't skip any step.

```
cd ~                                    # go home
mkdir ldd-textbook                      # make a folder for this book's work
cd ldd-textbook                         # enter it
pwd                                     # confirm
echo "I started Linux Device Drivers" > log.txt
cat log.txt
ls -l
```

Output of the last line will look like:

```
-rw-r--r-- 1 sudhdevu sudhdevu 30 May  7 14:00 log.txt
```

This is a file listing. Most of those columns are explained in chapter 3 (file permissions). For now: there's one file, named `log.txt`, and it's 30 bytes.

The 12 commands you must memorize

Command	What it does
`pwd`	print where you are
`ls`	list files here
`cd path`	change directory
`mkdir name`	make folder
`touch name`	make empty file
`cp src dst`	copy
`mv src dst`	move / rename
`rm name`	delete
`cat file`	print file
`less file`	view file scrolling
`echo "text"`	print text
`clear`	clear the screen

Add 6 more for everyday work:

Command	What it does
`man cmd`	show manual
`cmd --help`	show short help
`which cmd`	where the command lives
`history`	show what you typed before
`Ctrl+C`	cancel a running command
`Ctrl+L`	same as `clear`

That's 18 things. Master them this week.

Tab completion (huge timesaver)

When you type the start of a name, press **Tab**. The shell completes it for you.

```
cd Doc<Tab>             # becomes cd Documents/
ls /etc/syst<Tab>       # becomes ls /etc/system... or shows possibilities
```

Tab is your friend. **Use it all the time.** It also prevents typos.

Up arrow

Press the **Up Arrow** to recall the previous command. Press it again for the one before. This is how engineers re-run a build 50 times: they just hit Up + Enter.

Try it (15 minutes)

1. Open a terminal.
2. Make a folder called `practice` in your home directory.

3. Inside it, make 5 files: `a.txt`, `b.txt`, `c.txt`, `d.txt`, `e.txt`. Each should contain its own letter (use `echo "a" > a.txt`, etc.).
4. List them long form (`ls -l`).
5. Make a folder called `letters` and move all of them into it.
6. Inside `letters`, copy `a.txt` to `aa.txt`.
7. Print all of them concatenated to your screen using `cat *.txt` (the `*` means "all matching files").
8. Delete `aa.txt`.
9. Go up one level and delete the whole `practice` folder.
   - Hint: `rm -r practice` (the `-r` means "recursive — go into directories").

If you made it through, you have all the Linux skills you need to get started.

What if I get stuck?

You will. That's chapter 10. For now, the rule is:

- If a command says "command not found," you typed it wrong.
- If a command says "Permission denied," you don't have rights to that file (don't worry, normal in `/etc`).
- If you see a screen you don't recognize and don't know how to leave, try **q**, then **Esc**, then **Ctrl+C**, then close the terminal and reopen.

Self-check

- What does `pwd` mean and why would you use it?
- What's the difference between `cd ..` and `cd ~`?
- What's the difference between `rm` and a recycle bin?
- Why is Tab completion important?

You now control Linux from the keyboard. Tomorrow we'll talk about *where* files actually live.

➡️ Next: Chapter 3 — Files and paths

Chapter 3 — Files and paths

In chapter 2 you ran `ls` and saw filenames. Where are those files actually stored, and what does `/home/sudhdevu/Documents/` even mean? That's this chapter.

Everything is a file

Linux has a famous saying: **"everything is a file."** It's not literally true, but it's a useful starting point.

- Documents are files.
- Photos are files.
- Programs (the things you run) are files.
- Even your terminal's connection to the keyboard is a file (`/dev/tty`).
- Even your GPU device is a file (`/dev/dri/card0`). Yes, really.

This is why Linux has so many small commands like `cat`, `ls`, `cp` — once you can manipulate files, you can manipulate almost everything.

The filesystem is a tree

Files live in **folders** (also called **directories** — same thing). Folders can hold other folders. The whole structure is a *tree* with one root:

```
/                           ← the root, called "slash"
├── bin/                    ← essential programs
├── boot/                   ← kernel and bootloader files
├── dev/                    ← device files (your GPU lives here)
├── etc/                    ← system configuration
├── home/                   ← users' home directories
│   └── sudhdevu/           ← your home (named after your username)
│       ├── Documents/
│       ├── Downloads/
│       └── ldd-textbook/
├── lib/                    ← shared libraries
├── proc/                   ← live kernel info (not real files)
├── root/                   ← the root user's home (not the same as /)
├── sys/                    ← live device tree
├── tmp/                    ← temporary files
├── usr/                    ← installed software
│   ├── bin/
│   ├── lib/
│   └── share/
└── var/                    ← logs and changing data
```

Two things to notice:

1. **There is one root, `/`.** Unlike Windows, where each disk is `C:\`, `D:\`, etc. In Linux, every disk is mounted *under* `/`. Your USB drive might appear at `/media/usb`. Still under `/`.
2. **Slashes go forward.** `path/to/thing`, never `path\to\thing`.

What is a “path”?

A **path** is a string that names a location in the tree. There are two kinds.

Absolute paths

Start with `/`. Spelled out from the root:

```
/home/sudhdevu/ldd-textbook/log.txt
```

This means: from `/`, go into `home`, then `sudhdevu`, then `ldd-textbook`, find `log.txt`.

Absolute paths always work no matter where you are.

Relative paths

Start with anything **else**. Read relative to your current directory.

If you are inside `/home/sudhdevu`, then:

```
ldd-textbook/log.txt
```

is short for `/home/sudhdevu/ldd-textbook/log.txt`. The shell fills in the front.

`.` and `..`

- `.` (a single dot) = “the current directory” (where you are now).
- `..` (two dots) = “the parent directory” (one level up).

Examples, assuming you are in `/home/sudhdevu/ldd-textbook/`:

You type	Means
`cat log.txt`	`/home/sudhdevu/ldd-textbook/log.txt`
`cat ./log.txt`	exactly the same — `.` makes it explicit
`cat ../Documents/x.txt`	`/home/sudhdevu/Documents/x.txt` (go up one, then into Documents)
`cd ..`	go to `/home/sudhdevu/`
`cd ../..`	go to `/home/`
`cd ../../..`	go to `/`

`~`

`~` (tilde) means “your home directory.” Almost always `/home/yourname/`.

```
cd ~                # same as cd /home/sudhdevu
cd ~/Documents      # same as cd /home/sudhdevu/Documents
echo $HOME          # prints your home dir
```

Practice this until you don’t have to think

In your terminal, do this exact sequence and predict the output of each `pwd` *before* you press Enter:

```
cd /
pwd
cd home
pwd
```

```
cd ~
pwd
cd ..
pwd
cd ./../..
pwd
cd
pwd
```

(`cd` with no arguments = `cd ~`.)

Did you predict every step right? If not, draw the tree on paper and walk through it.

Hidden files

Files whose names start with `.` are **hidden**. `ls` doesn't show them by default; `ls -a` does.

```
cd ~
ls
ls -a
```

You will see things like `.bashrc`, `.vimrc`, `.ssh/`. These are usually configuration files. The dot doesn't make them special, it's just a convention to keep them out of the way.

Permissions, very briefly

When you ran `ls -l` you saw something like:

```
-rw-r--r-- 1 sudhdevu sudhdevu 30 May 7 14:00 log.txt
```

The `-rw-r--r--` is the **permissions string**. Read it as four groups:

```
-   rw-   r--   r--
type owner group others
```

- **type**: `-` means regular file, `d` means directory, `l` means symbolic link, `c` / `b` mean character/block device files.
- **owner / group / others**: who can `r`ead, `w`rite, e`x`ecute the file.

For now you mostly don't worry about this. When something says "Permission denied," you'll come back here. Or use `chmod` / `chown`, which we'll cover later.

Useful path tricks

Wildcards

```
ls *.txt          # all files ending in .txt
ls a*             # all files starting with 'a'
ls ?.txt          # exactly 1 char + .txt (a.txt yes, ab.txt no)
ls /etc/*.conf    # full paths work too
```

`*` and `?` are expanded by the shell **before** the command runs. So `ls *.txt` becomes `ls a.txt b.txt c.txt`.

Two filenames at once with `{}`

```
mkdir -p {src,build,test}     # makes 3 dirs
cp file.{txt,bak}             # copy file.txt to file.bak
```

Finding files

```
find ~/ldd-textbook -name "*.md"    # find all .md files under that dir
which gcc                            # which file gets run when I type 'gcc'
locate amdgpu.ko                     # fast search of a precomputed db (if installed)
```

You will use `find` often when working in the kernel tree.

Special directories you should know exist

Path	What's there
`/home/youruser/`	your stuff
`/etc/`	system configuration files
`/var/log/`	system logs (`/var/log/syslog` , `/var/log/dmesg`)
`/usr/include/`	C header files (`stdio.h` , etc.)
`/usr/bin/` , `/bin/`	installed programs
`/usr/lib/` , `/lib/`	shared libraries
`/proc/`	live process info (e.g., `/proc/cpuinfo` shows your CPU)
`/sys/`	live device tree (e.g., `/sys/class/drm/` shows your GPUs)
`/dev/`	device files (e.g., `/dev/dri/card0` is your GPU)
`/tmp/`	scratch space, cleared on reboot

Try these now:

```
cat /proc/cpuinfo | head      # info about your CPU
ls /sys/class/drm             # your display devices
ls /dev/dri                   # your GPU files (if you have a GPU)
```

That `card0` you may see *is* the file your future driver code will create. You're already touching it.

Try it (10 minutes)

1. From your home directory, navigate to `/usr/include/` using only relative paths (no absolute path, no `~`). How many `..` do you need?
2. Make this exact tree under `~/practice` :

   ```
   practice/
   ├── parts/
   │   ├── part0/
   │   └── part1/
   └── README.md
   ```

   (Hint: `mkdir -p` creates parents as needed.)
3. From inside `parts/part0/` , write a relative path that names `practice/README.md` .
4. From inside `parts/part0/` , write a relative path that names `parts/part1/` .

5. Use `find` to find every `.md` file under `~/ldd-textbook`. (If you've copied the book, there should be many.)

Self-check

- What does `/` mean by itself? What about `/home/`?
- What's the difference between `~`, `.`, and `..`?
- If your current directory is `/a/b/c`, what is `../../../etc`? (Answer: `/etc`.)
- Why is Linux's "everything is a file" model useful for drivers?

You now know how to find anything on your computer. Next we learn how to *edit* files, since "open Notepad" doesn't apply here.

➡️ Next: Chapter 4 — Text editors

Chapter 4 — Text editors

To write code you have to edit text files. There are many editors. Beginners get into religious wars over them. **Don't.** Use whatever lets you make progress today, and learn `vim` later because the kernel community uses it.

The three you need to know

Editor	When to use
`nano`	Right now, today, while you're new. Easiest.
VS Code (or "Cursor", same idea)	When you write more than 50 lines at a time.
`vim`	Eventually. Mandatory if you want to be quick over SSH and read kernel patches comfortably.

You don't have to pick. You'll use all three over time.

`nano` — the friendly one

`nano` is the easiest editor on Earth. To open a file:

```
nano hello.txt
```

You see a screen with the file contents at the top and a strip of help at the bottom:

```
GNU nano 6.2                    hello.txt

Hello, this is my first file.
_

^G Help    ^O Write Out   ^X Exit
```

Read the bottom strip. `^X` **means** `Ctrl+X`**.** Just type your text, then:

- `Ctrl+O` then Enter to **save** ("write out").
- `Ctrl+X` to **quit**.
- `Ctrl+K` to cut a line.
- `Ctrl+U` to paste.
- `Ctrl+W` to search.

That's enough to write and edit any file. Use `nano` for the next few chapters until something annoys you about it.

Try it

```
cd ~
nano my-name.txt
# Type: My name is <your name>.
# Ctrl+O, Enter, Ctrl+X
cat my-name.txt
```

VS Code (or Cursor)

For long code, you want a real editor with autocomplete, syntax colors, and a file tree.

- **VS Code**: free, install from your distribution's package manager or from https://code.visualstudio.com/.
- **Cursor**: VS Code with AI built in. Same UI.

To open a folder in VS Code from the terminal:

```
cd ~/ldd-textbook
code .          # open the current directory
```

You'll get a panel on the left showing all the files, and a tabbed editor on the right.

You don't need to learn 100 shortcuts now. Just know:

- `Ctrl+S` save
- `Ctrl+P` quick-open a file by name (huge time saver)
- `Ctrl+` (backtick) to open a terminal *inside* VS Code
- `Ctrl+/` to comment/uncomment a line

Use VS Code for **all the C and Python work** in this book. Use `nano` over SSH or for quick edits.

`vim` — eventually

`vim` is hostile to beginners on purpose. It's also extremely fast once you know it, and the kernel community lives in it. You should learn it within the first 6 months. Today is not that day.

If you accidentally end up in `vim`:

- Press `Esc` (this exits any sub-mode).
- Type `:q!` and press Enter.
- That's "quit without saving." You're back at the shell.

If you have 30 spare minutes, run:

```
vimtutor
```

It's a built-in interactive tutorial. Do it once. Then don't worry about `vim` again until chapter 28 or so.

vim cheat sheet (for when you're ready)

```
Modes
  Esc          go to "normal" mode (this is where you navigate)
  i            switch to "insert" mode (where you type)

Movement (normal mode)
  h j k l      left, down, up, right
  w            next word
  b            previous word
  0            start of line
  $            end of line
  gg           top of file
  G            bottom of file
  /pattern     search forward (then n / N for next / prev)
```

```
Edit
  i            insert before cursor
  a            insert after cursor
  o            open new line below
  x            delete one char
  dd           delete a line
  yy           copy ("yank") a line
  p            paste below
  u            undo
  Ctrl+R       redo

Save / quit
  :w           save
  :q           quit
  :wq          save + quit
  :q!          quit without saving
```

Print this. Tape it next to your monitor. In 2 weeks you won't need it.

Why text editors matter

You will spend more hours in your editor than in any other program. Pick one, get fast at it. **Speed of editing is real productivity** — every minute you save fumbling with the cursor is a minute you spend thinking about the problem.

But this is a slow improvement, not an urgent one. Don't optimize before you have a workflow. **Today: open `nano`, write text, save, quit.** That's it.

A common editor mistake

If you `nano file.c` then later run a command and edit the same file in VS Code, you might end up with two views of the file open. If you save in one and then save in the other, the second save **overwrites** the first.

Rule: edit a file in **one place at a time**. Save before switching.

Try it (10 minutes)

1. Use `nano` to create `~/ldd-textbook/notes.md`. Write 3 sentences about chapter 3. Save and quit.
2. Print it with `cat`.
3. Install VS Code if you haven't (`sudo apt install code` may work, otherwise download from the website).
4. Open `~/ldd-textbook/` in VS Code.
5. Open `notes.md` in VS Code, add a 4th sentence, save (`Ctrl+S`).
6. Back in the terminal, `cat notes.md` again. Confirm the new sentence is there.

Self-check

- How do you save and quit `nano`?
- How do you save and quit `vim`? (Trick: include "what if you didn't save and want to abandon changes?")
- Why might you prefer VS Code for big projects but `nano` over SSH?

You can now edit any file. Now we get to the actual point of this book — writing code.

➡️ Next: Chapter 5 — What is code?

Chapter 5 — What is code?

You've heard "code" your whole life. Let's nail down what it actually means.

Code is just text in a file

When someone "writes code," they open a text editor (like `nano` or VS Code from chapter 4), type some sentences in a special language the computer understands, and save the file.

That's it. Code is text. It's not magic, it's not a black box, it's literally a `.txt`-like file with a different extension.

Examples:

File	Language
`hello.py`	Python
`hello.c`	C
`hello.cpp`	C++
`hello.sh`	Bash (shell script)
`hello.html`	HTML (a web page, not really "code" but text)
`hello.rs`	Rust
`hello.java`	Java

The extension doesn't make a file work. It's just a hint to humans (and editors) about what's inside.

What does "running" a program mean?

You **wrote** text. To **run** it you need a program that reads your text and *does* what the text says.

There are two main flavors of "run":

Flavor 1: an interpreter reads your file and acts on it line by line

This is how Python and Bash work.

```
python3 hello.py
```

This launches the program `python3`, which opens `hello.py`, reads the first line, does what it says, then the second line, then the third, etc.

The interpreter is itself a program (someone wrote `python3` in C, mostly). It exists on your computer because it was installed.

Flavor 2: a compiler turns your file into a new program, which you then run

This is how C and C++ work.

```
gcc hello.c -o hello       # compile: hello.c → hello (a binary)
./hello                    # run the binary
```

Step 1 produces a brand new file (`hello`) made of machine code — the language the CPU itself speaks. Step 2 runs that file.

Why two flavors?

Interpreter	Compiler
Easy. You change your code, run it.	More setup. You change, compile, then run.
Slow. The interpreter does work on every line every time.	Fast. The compiler did the hard work once; running is direct CPU instructions.
Friendlier error messages.	Less friendly errors, but better hardware control.
Python, Bash, JavaScript, Ruby.	C, C++, Rust, Go.

The Linux kernel and amdgpu are written in C. That's what we're heading towards. But for chapters 6–8 we use Python because it's gentler.

A program is a list of instructions

Whatever the language, a program is fundamentally a list of instructions:

```
1. take this number
2. add 1 to it
3. print it
4. if it's less than 10, go back to step 2
```

That's actual logic. We'll dress it up in Python syntax in the next chapter:

```
n = 0
while n < 10:
    n = n + 1
    print(n)
```

Same thing. The fancy parts are just how the language wants you to write it.

What happens when you run a program (the kid version)

You type:

```
./hello
```

What happens, in slow motion:

1. The shell finds the file `hello` (you'll learn about `PATH` later).
2. The shell asks the **operating system** (Linux) to run it.
3. Linux makes a new **process** — a sandbox with its own memory and a place to track the program.
4. Linux loads the program's instructions from disk into the process's memory.
5. Linux starts the CPU executing the instructions.
6. The instructions run. They might do calculations, print to the screen, read files.
7. When the program finishes, Linux cleans up the process. The shell prompt comes back.

You don't have to remember every step. Just internalize: **a program is *loaded into memory* and *the CPU runs it*.**

"But what is a programming language *really*?"

A programming language is a set of rules ("syntax") for how you write code, and a meaning ("semantics") for what each piece of code does.

For example, in Python, this is valid:

```
print("hello")
```

In C, the same thing looks like:

```
#include <stdio.h>
int main(void) {
    printf("hello\n");
    return 0;
}
```

C requires more ceremony because it gives you more control over the machine. Python hides the ceremony but takes the control away. Different trade-offs.

"Source code" vs "binary"

- **Source code**: the human-readable text you wrote. `hello.c` is source code.
- **Binary** (or **executable**): the machine-code file produced by the compiler. `hello` (no extension on Linux) is a binary.

You only ship source code to other developers. You only ship binaries to users (or both).

You can peek inside a binary:

```
file ./hello
# ELF 64-bit LSB pie executable, x86-64, dynamically linked, ...

ls -l ./hello
# -rwxr-xr-x 1 sudhdevu sudhdevu 16640 May 7 14:30 hello
```

`x` in the permissions means "executable." That's why `./hello` works — the file is marked as runnable.

A driver is just a program too

Important grounding: the Linux kernel is a (huge) C program. A driver like amdgpu is a smaller C program *inside* it.

When you eventually write `myring.c` in lab 05, it will be compiled into `myring.ko` — a binary, just a special one that gets loaded **into the kernel** instead of run as a user process. We'll explain that in detail in Part V.

For now: don't be scared of the word "driver." A driver is just code. You can read it, write it, change it. There is no priesthood.

Try it (10 minutes)

We'll do the *very* first hands-on:

```
cd ~/ldd-textbook
mkdir hello-py
cd hello-py
nano hello.py
```

In `nano`, type exactly:

```
print("Hello, world!")
```

Save (`Ctrl+O`, Enter), quit (`Ctrl+X`).

Then run it:

```
python3 hello.py
```

You should see:

```
Hello, world!
```

Congratulations. You have written and run a program.

If `python3` is not installed:

```
sudo apt install python3
```

If it doesn't print "Hello, world!" but instead complains, **read the error message carefully**. 99% of the time it's a typo in your file.

Self-check

- What's the difference between an interpreter and a compiler? Name an example of each.
- Why does running a C program take 2 commands but running a Python program takes 1?
- What is a "binary"? Where does it come from?
- Is the amdgpu driver fundamentally different from `hello.py`? (Spoiler: no, just bigger and runs in a different mode.)

You wrote code. Tomorrow we make it actually do useful things.

➡️ Next: Chapter 6 — Your first program (in Python)

Chapter 6 — Your first program (in Python)

Today we learn what code can actually *do*. We'll use Python as training wheels because it's friendly. Don't worry — Part I onwards is all C.

Make sure you've done the "Try it" from chapter 5 (`hello.py` working). Now we'll grow it.

Three things every program does

1. **Stores values** (variables).
2. **Computes** with them (arithmetic, text manipulation).
3. **Prints** results (or sends them somewhere).

That's the whole game. Decisions and loops (next chapter) are added on top.

Variables

A **variable** is a name for a value. In Python:

```
name = "Sudhanshu"
age = 22
pi = 3.14159
```

You read this as "assign the value `"Sudhanshu"` to the name `name`." Now whenever you use `name`, Python substitutes the value.

```
print(name)
print(age)
print(pi)
```

Save as `vars.py` and run with `python3 vars.py`:

```
Sudhanshu
22
3.14159
```

You can change a variable later:

```
age = 23
print(age)         # now prints 23
```

The old value is gone. Variables are not permanent — they hold whatever you last assigned.

Types of values

Python has a few basic kinds of values:

Type	Example	Description
integer (`int`)	`42` , `-7` , `0`	whole numbers
float (`float`)	`3.14` , `-0.5` , `2.0`	numbers with decimals
string (`str`)	`"hello"` , `"abc"`	text, in quotes
boolean (`bool`)	`True` , `False`	yes/no, on/off

You don't declare types in Python. The value's type is whatever it is.

Arithmetic

Python knows math:

```
a = 5
b = 3
print(a + b)        # 8
print(a - b)        # 2
print(a * b)        # 15
print(a / b)        # 1.6666...
print(a // b)       # 1     (integer division)
print(a % b)        # 2     (remainder)
print(a ** b)       # 125   (a to the power b)
```

`%` is called **modulo** or "remainder." `5 % 3` is `2` because 5 ÷ 3 is 1, remainder 2. You will see `%` constantly in real code (especially for ring buffers — already a hint of Part VI).

Strings

Strings are text. They live in quotes:

```
greeting = "hello"
name = "Sudhanshu"
message = greeting + ", " + name + "!"
print(message)        # hello, Sudhanshu!
```

`+` on strings means **concatenate** (glue together).

A friendlier way is **f-strings**:

```
print(f"hello, {name}!")
```

The `f` before the `"` lets you embed variables in `{...}`. This is what you'll use in real life.

Reading input

Programs are more interesting when they accept input from a human:

```
name = input("What is your name? ")
print(f"Nice to meet you, {name}!")
```

`input("prompt: ")` displays the prompt, waits for the user to type, and returns whatever they typed *as a string*.

If you want a number, you have to convert:

```
age_str = input("How old are you? ")
age = int(age_str)              # convert string to integer
print(f"In 10 years you will be {age + 10}.")
```

If the user types “twenty”, `int(...)` will crash because “twenty” is not a number-looking string. We’ll handle errors in chapter 10.

Comments

A line starting with `#` is a **comment**. Python ignores it. Comments are for humans:

```
# this calculates the area of a circle
radius = 5
area = 3.14159 * radius * radius
print(area)
```

Use comments to explain **why** code is doing something non-obvious, not **what** it does. (Good code should be readable on its own.)

Putting it together

Type this exactly into `~/ldd-textbook/hello-py/temperature.py`:

```
# convert Fahrenheit to Celsius
print("Fahrenheit to Celsius converter")
print("--------------------------------")

f_str = input("Enter a temperature in F: ")
f = float(f_str)

c = (f - 32) * 5 / 9

print(f"{f} F is {c} C")
```

Run it:

```
python3 temperature.py
```

Try a few inputs: `32` (should give 0), `212` (should give 100), `98.6` (body temperature, should give 37).

That’s a useful program. You are no longer a beginner-beginner.

Lists (a peek)

A list holds multiple values:

```
numbers = [10, 20, 30, 40]
print(numbers[0])        # 10  (first element, indexed from 0!)
print(numbers[3])        # 40
print(len(numbers))      # 4

numbers.append(50)
print(numbers)           # [10, 20, 30, 40, 50]
```

Indexes start at **0**, not 1. This is the universal convention in programming. The first element is index 0. Get used to it now.

We'll use lists in the next chapter for loops.

Try it (15 minutes)

Write a program `~/ldd-textbook/hello-py/calc.py` that:

1. Prints a banner: `My tiny calculator`.
2. Asks for two numbers.
3. Prints their sum, difference, product, and quotient.

Expected run:

```
My tiny calculator
First number: 6
Second number: 4
6 + 4 = 10
6 - 4 = 2
6 * 4 = 24
6 / 4 = 1.5
```

Hints: - Use `input(...)` and `float(...)` to read numbers. - Use f-strings to format output.

If your code prints `10.0` instead of `10`, that's because `float` keeps the decimal. Don't worry about it.

Common beginner mistakes (and how to fix)

`SyntaxError: invalid syntax` — usually means you have a typo (missing quote, missing parenthesis). Look at the line number Python prints. The error is on or just before that line.

`NameError: name 'foo' is not defined` — you used a variable before assigning to it. Or you misspelled it (`Print` instead of `print`).

`TypeError: can only concatenate str (not "int") to str` — you tried to do `"hello" + 5`. Strings and numbers don't add. Use `str(5)` to convert: `"hello" + str(5)`.

`ValueError: invalid literal for int() with base 10: 'abc'` — you tried `int("abc")` and "abc" isn't a number. Validate input or accept floats.

When you see one of these, stop, read the line number, look at that exact line. The fix is almost always 1 character.

Self-check

- What does `=` mean in `age = 23`? (Answer: assign, not "equals.")
- Why does `int("hello")` crash but `int("42")` doesn't?
- Why are list indexes from 0 and not 1? (You don't have to know the historical answer — just memorize "from 0.")
- What's the difference between `5 / 2` and `5 // 2` in Python?

You can compute. Tomorrow: making the program decide and repeat.

➡️ Next: Chapter 7 — Decisions and repeats

Chapter 7 — Decisions and repeats

A program with only assignments and prints is boring. The two ideas that make programs powerful are:

1. **Decisions** — do this if some condition is true, otherwise do something else.
2. **Repeats** — do something many times.

That's it. Every program in the world, including a 100,000-line GPU driver, is built from these two ideas.

Decisions: `if` / `elif` / `else`

```
age = int(input("Age: "))

if age >= 18:
    print("You can vote.")
else:
    print("Too young to vote.")
```

Read it like English. `if some condition, do block; else, do block.`

Notice **two** important Python rules:

1. **The colon** `:` at the end of `if` and `else`.
2. **Indentation** — the lines inside the block are indented (4 spaces by convention). Python uses indentation to know what's "inside" the `if`.

If you indent wrong, Python complains:

```
IndentationError: expected an indented block
```

This is unique to Python (C uses braces). Don't worry, in C you'll write `{` and `}` and the indentation is just for humans.

Multiple branches with `elif`

```
score = int(input("Score (0-100): "))

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(f"Grade: {grade}")
```

`elif` = "else if." Python checks each in order; first one that's true wins.

Comparisons

```
==     equal to       (note: TWO equals; one equals is assignment!)
!=     not equal to
<      less than
<=     less than or equal
>      greater than
>=     greater than or equal
```

This is the most common beginner bug:

```
if x = 5:           # WRONG — this is assignment, syntax error
    ...

if x == 5:          # right
    ...
```

Combining: `and`, `or`, `not`

```
if age >= 18 and age < 65:
    print("Working age.")

if grade == "A" or grade == "B":
    print("Good job.")

if not is_logged_in:
    print("Please log in.")
```

These are spelled out as words in Python. In C you'll see `&&`, `||`, `!`. Same idea.

Boolean values

```
is_done = True
is_failed = False

if is_done and not is_failed:
    print("Success!")
```

`True` and `False` (capitalized) are the two boolean values.

Repeats: `while`

`while` keeps doing something *as long as* a condition is true.

```
count = 1
while count <= 5:
    print(count)
    count = count + 1

print("done")
```

Output:

```
1
2
3
4
5
done
```

Walk through the logic in your head:

- `count` starts at 1.
- Is `1 <= 5`? Yes. Print 1. Then `count = 1 + 1 = 2`.
- Is `2 <= 5`? Yes. Print 2. `count = 3`.
- … and so on.
- Is `6 <= 5`? No. Stop the loop. Print "done."

The single biggest danger of `while`: **infinite loops**. If you forget to update the condition variable:

```
count = 1
while count <= 5:
    print(count)
    # forgot count = count + 1 !
```

This prints `1` forever. **Press `Ctrl+C` to stop.** This is what `Ctrl+C` is for.

Repeats: `for`

`for` is for going through a sequence (a list, a range of numbers, the lines of a file).

Range of numbers

```
for i in range(5):
    print(i)
```

Prints `0, 1, 2, 3, 4`. (Note: 0 to 4, not 1 to 5. `range(5)` makes 5 numbers starting at 0.)

```
for i in range(1, 6):
    print(i)        # 1, 2, 3, 4, 5

for i in range(0, 10, 2):
    print(i)        # 0, 2, 4, 6, 8 (step 2)
```

Going through a list

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(f"I like {fruit}.")
```

Output:

```
I like apple.
I like banana.
I like cherry.
```

`fruit` is a fresh variable each time, taking the next value from the list.

`for` is just convenient `while`

You could write the same logic with `while`:

```
fruits = ["apple", "banana", "cherry"]
i = 0
while i < len(fruits):
```

```
        print(f"I like {fruits[i]}.")
        i = i + 1
```

Both are valid. `for ... in` is shorter and more readable, so prefer it.

`break` and `continue`

Sometimes you want to **stop a loop early** or **skip an iteration**.

```
# print numbers 1..10, but stop at 7
for n in range(1, 11):
    if n == 7:
        break              # exit the loop entirely
    print(n)
```

```
# print only odd numbers in 1..10
for n in range(1, 11):
    if n % 2 == 0:
        continue           # skip the rest of this iteration
    print(n)
```

Use them when they make the code clearer; not as a habit.

A real-feeling example: the FizzBuzz

This is the classic interview warm-up. Write a program that prints numbers 1 to 20, but:

- if the number is divisible by 3, print “Fizz” instead.
- if it’s divisible by 5, print “Buzz”.
- if it’s divisible by both, print “FizzBuzz”.

Save as `~/ldd-textbook/hello-py/fizzbuzz.py`:

```
for n in range(1, 21):
    if n % 15 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

Run it. Expected output starts:

```
1
2
Fizz
4
Buzz
Fizz
7
...
```

Why `n % 15` first? Because 15 is `3 × 5` — if it’s divisible by 15 it’s divisible by both 3 and 5. If we checked `n % 3` first, “FizzBuzz” inputs would already match “Fizz” and we’d never reach the both-case.

This kind of ordering thinking is what programming is really about.

Combining decisions and repeats: number guessing

```
import random            # standard library: random numbers

secret = random.randint(1, 100)
guesses = 0

print("I'm thinking of a number from 1 to 100.")

while True:                          # loop forever (we'll break out)
    guess = int(input("Your guess: "))
    guesses = guesses + 1

    if guess < secret:
        print("Too low.")
    elif guess > secret:
        print("Too high.")
    else:
        print(f"Correct! It took you {guesses} guesses.")
        break
```

Save as `guess.py` and play it. This is your first program with real interactivity.

Notes: - `import random` brings in a built-in module of utilities. - `random.randint(1, 100)` gives a random integer 1..100 inclusive. - `while True:` loops forever; the only way out is `break`.

Try it (20 minutes)

Write `~/ldd-textbook/hello-py/sum-evens.py` that:

1. Asks the user for a number `N`.
2. Computes the sum of all even numbers from 1 to N (inclusive).
3. Prints the result.

Expected:

```
N: 10
Sum of evens 1..10 = 30
```

(2 + 4 + 6 + 8 + 10 = 30.)

Two ways to write it:

- Using `for` and `if`.
- Using `for` with `range(2, N+1, 2)` (cleaner — no `if` needed).

Try both, see which you find more readable.

Common beginner mistakes

- **Using `=` instead of `==`** in conditions. Will refuse to run.
- **Forgetting the colon `:`** at the end of `if`, `elif`, `else`, `while`, `for`. Python will say `SyntaxError: expected ':'`.
- **Wrong indentation.** Mix of tabs and spaces is a classic. Pick one (4 spaces) and stick to it. Most editors handle this for you.
- **Off-by-one errors.** `range(5)` is 0..4, not 1..5. `range(1, 5)` is 1..4, not 1..5. To get 1..5 use `range(1, 6)`. You will hit this constantly. It's normal.

Self-check

- What's the difference between `=` and `==`?
- What's an infinite loop, and how do you stop one that's running?
- Why is the order of `elif` checks important in FizzBuzz?
- How do you read each line of a list in a `for` loop?

You can now write programs that decide and repeat. That's most of programming. Tomorrow: how to package code so you don't repeat yourself.

➡️ Next: Chapter 8 — Functions

Chapter 8 — Functions

You've now seen variables, decisions, and loops. With those alone you could write programs forever. But you'd repeat yourself a lot.

A **function** is a *named* piece of code that you can call by its name. It's the single most important tool for keeping your code clean.

The motivation

Imagine you want to print a friendly greeting in 5 places:

```
print("=" * 30)
print("   Welcome to the program  ")
print("=" * 30)
```

If you copy-paste this 5 times and later want to change the wording, you'd have to find and edit every copy. Mistakes happen. Functions fix this.

Defining a function

```
def banner():
    print("=" * 30)
    print("   Welcome to the program  ")
    print("=" * 30)
```

Read it: `def` (define) `banner` (the name) `()` (no inputs) `:`. Then the body, indented.

Now to use it:

```
banner()                # call the function

print("here is some content")

banner()                # call again
```

Output:

```
==============================
   Welcome to the program
==============================
here is some content
==============================
   Welcome to the program
==============================
```

You wrote the code *once*, used it *twice*. Change it in one place, both calls update.

Arguments (input)

Functions are more useful when they take **arguments**:

```
def banner(message):
    print("=" * 30)
    print("   " + message + "   ")
    print("=" * 30)

banner("Hello, world!")
banner("Welcome back")
```

Output:

```
==============================
   Hello, world!
==============================
==============================
   Welcome back
==============================
```

`message` is a **parameter** — a placeholder name for whatever the caller passes in.

You can have multiple parameters:

```
def add(a, b):
    print(a + b)

add(2, 3)         # 5
add(10, 20)       # 30
```

Return values (output)

Functions can also **return** a value, so the caller can use it:

```
def add(a, b):
    return a + b

result = add(2, 3)
print(result)             # 5
print(add(10, 20))        # 30
print(add(add(1, 2), 4))  # 7
```

`return` exits the function and gives back the value.

A function without `return` returns a special value `None`, which means "nothing here."

A useful example

```
def is_even(n):
    return n % 2 == 0

def sum_evens_up_to(N):
    total = 0
    for i in range(1, N + 1):
        if is_even(i):
            total = total + i
    return total

print(sum_evens_up_to(10))      # 30
print(sum_evens_up_to(100))     # 2550
```

Notice we built `sum_evens_up_to` *out of* `is_even`. This is the whole point: small functions combine into bigger ones.

Why functions matter (for real)

Every Linux kernel driver function looks like this:

```
static int my_driver_probe(struct pci_dev *pdev, const struct pci_device_id *id)
{
    /* code */
    return 0;
}
```

Same idea — a function with parameters and a return value. The Linux kernel is **6 million lines** of mostly small functions calling each other. If you understand how functions work, you understand 80% of the structure.

Variable scope

A variable created **inside** a function only exists there:

```
def foo():
    secret = 42
    print(secret)

foo()
print(secret)        # NameError: 'secret' is not defined
```

`secret` lives only inside `foo`. Once the function returns, it's gone. This is called **local scope**.

A variable **outside** any function is **global**:

```
counter = 0           # global

def bump():
    global counter    # tell Python "I want the global, not a new local"
    counter = counter + 1

bump()
bump()
bump()
print(counter)        # 3
```

You usually want to avoid globals — they make code harder to reason about. Pass things in as arguments and return values out instead.

Default arguments

You can provide defaults so the caller doesn't have to specify everything:

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Sudhanshu")                # Hello, Sudhanshu!
greet("Sudhanshu", "Hi there")    # Hi there, Sudhanshu!
```

Putting it together

```
def banner(text):
    line = "=" * (len(text) + 4)
    print(line)
    print(f"  {text}  ")
    print(line)

def is_prime(n):
    if n < 2:
        return False
    for d in range(2, n):
        if n % d == 0:
            return False
    return True

def primes_up_to(N):
    result = []
    for n in range(2, N + 1):
        if is_prime(n):
            result.append(n)
    return result

banner("Primes up to 30")
print(primes_up_to(30))
```

Save as `~/ldd-textbook/hello-py/primes.py` and run.

Output:

```
========================
  Primes up to 30
========================
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Three small functions, each does one thing, the last one composes them.

What makes a good function

1. **One job.** If the name has "and" in it, it should probably be two functions.
2. **Short.** A function that doesn't fit on your screen is too long. Aim for under 30 lines.
3. **Honest name.** `add(a, b)` should add. Not subtract. Not print.
4. **Return, don't print.** Whenever possible, *return* the answer; let the caller decide whether to print, save to a file, or something else. Mixing computation and printing makes code less reusable.

Same principles apply to C functions in the kernel. Keep them small, focused, well-named.

Try it (20 minutes)

Write `~/ldd-textbook/hello-py/temperature_v2.py` that has two functions:

```
def f_to_c(f):
    # convert Fahrenheit to Celsius and return
    ...

def c_to_f(c):
    # convert Celsius to Fahrenheit and return
    ...
```

Then a small program that loops asking the user:

```
What do you want?
  1. F → C
  2. C → F
  3. quit
Choice:
```

and uses the right function.

Hint: outer `while True:` loop, `if/elif/else` inside, `break` on choice 3.

Common mistakes

- **Calling a function without parentheses.** `banner` (no parens) is the function itself, not a call. `banner()` calls it. Beginners often write `banner` and wonder why nothing happens.
- **Forgetting `return`.** A function with no `return` returns `None`. If you do `x = compute()` and `compute` doesn't return, `x` will be `None` and later code will crash with weird errors.
- **Modifying globals.** Don't reach out of a function to change variables defined elsewhere. Pass parameters in, return values out.
- **Reusing parameter names.** Inside `def foo(x):`, the local `x` shadows any global `x`. Usually fine, but if you wanted the global one, you have a bug.

Self-check

- What's the difference between defining a function and calling it?
- What's the difference between an argument and a parameter? (Trivia: parameter = the name in the function definition; argument = the value passed at call time. Most people use the words interchangeably; nobody will judge you.)
- Why is "one function, one job" a good rule?
- What does a function return if you don't write `return`?

You can now write reusable code. That's a real skill. Tomorrow we move from Python to C — the language you'll actually be writing for the rest of this book.

➡️ Next: Chapter 9 — From Python to C

Chapter 9 — From Python to C

You now know what variables, decisions, loops, and functions are. **Those ideas exist in C too**, just dressed differently.

This chapter is the **bridge**. We rebuild every Python program from chapters 6–8 in C, side by side. After this you will be able to read Part I of the book.

Get the C compiler

```
sudo apt install build-essential
gcc --version
```

You should see something like `gcc (Ubuntu …) 11.x` or newer.

`gcc` is the GNU C compiler. It turns your `.c` source code into a runnable binary.

Hello, C

`~/ldd-textbook/hello-c/hello.c`:

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

Build and run:

```
cd ~/ldd-textbook/hello-c
gcc hello.c -o hello
./hello
```

Output:

```
Hello, world!
```

That's it. You wrote your first C program. Let's break it down line by line.

Line	What it means	Python equivalent
`#include <stdio.h>`	"I want to use the standard input/output library."	`import sys` (sort of)
`int main(void)`	"Define a function called `main` that returns an `int` and takes no arguments."	`def main():`
`{ ... }`	The function body — like Python's indentation, but explicit.	indented block
`printf("Hello, world!\n");`	Print a string. `\n` is newline.	`print("Hello, world!")`

Line	What it means	Python equivalent
`return 0;`	Exit with status 0 (success).	`return 0`
`;` at end of every statement	Required in C. Forget it and the compiler complains.	Python uses newlines instead.

So C is just Python with:

- explicit types,
- braces instead of indentation,
- semicolons after statements,
- a `main()` function that's the entry point,
- a separate compile step.

That's the entire difference, syntactically. The hard parts of C are *semantic* (memory, pointers, undefined behavior). We get there in Part I.

Variables in C

Python:

```python
name = "Sudhanshu"
age = 22
pi = 3.14159
```

C:

```c
#include <stdio.h>

int main(void)
{
    char name[] = "Sudhanshu";    // a string of characters
    int age = 22;                 // an integer
    double pi = 3.14159;          // a floating-point number

    printf("name = %s\n", name);
    printf("age = %d\n", age);
    printf("pi = %f\n", pi);

    return 0;
}
```

Differences:

- **You declare the type.** `int age = 22;` says "age is an integer." C wants this; Python figures it out.
- **Strings are not as simple.** A C string is "an array of characters." We'll see this in detail in Part I, chapter 4. For now `char name[] = "..."` works.
- **`printf` is fancier.** It uses **format specifiers**:
  - `%d` for `int`,
  - `%f` for `double` (floating point),
  - `%s` for string,
  - `%c` for a single character.
  - `\n` for newline (always remember it; `printf` doesn't auto-add).

Compile and run:

```
gcc vars.c -o vars
./vars
```

Arithmetic in C

Same operators, mostly:

```
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 3;

    printf("a + b = %d\n", a + b);   // 8
    printf("a - b = %d\n", a - b);   // 2
    printf("a * b = %d\n", a * b);   // 15
    printf("a / b = %d\n", a / b);   // 1   ← integer division!
    printf("a %% b = %d\n", a % b);  // 2
    return 0;
}
```

Note the surprise: `5 / 3` in C is `1`, not `1.6666...`. **Integer division** drops the remainder. To get a fractional result, you need at least one of the operands to be a `double`:

```
double a = 5;
double b = 3;
printf("%f\n", a / b);   // 1.666667
```

This is the kind of detail that bites beginners. You'll memorize it after one bug.

(`%%` in `printf` prints a literal `%`. The single `%` is for format specifiers.)

`if` / `else` in C

Python:

```
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

C:

```
if (age >= 18) {
    printf("Adult\n");
} else {
    printf("Minor\n");
}
```

Differences:

- **Parentheses around the condition.** `if (age >= 18)`, not `if age >= 18`.
- **Braces around blocks.** Mandatory if more than one statement (and good practice always).
- **No `elif` keyword.** You use `else if`:

```
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';
} else {
    grade = 'F';
}
```

- **Logical operators are symbols, not words:**

Python	C
`and`	`&&`
`or`	`||`
`not`	`!`
`True`	`1` (or `true` if you `#include <stdbool.h>`)
`False`	`0` (or `false`)

```
if (age >= 18 && age < 65) { ... }
if (!is_logged_in) { ... }
```

Loops in C

Python:

```
for i in range(5):
    print(i)
```

C:

```
#include <stdio.h>

int main(void)
{
    for (int i = 0; i < 5; i = i + 1) {
        printf("%d\n", i);
    }
    return 0;
}
```

The C `for` loop has **three parts** in parentheses:

```
for (initializer ; condition ; update)
```

- `int i = 0` — declare and initialize.
- `i < 5` — keep looping while this is true.
- `i = i + 1` (or `i++`, same thing) — do this each iteration.

Once you see this pattern a few times it becomes natural.

C also has `while` and `do { } while`, identical to Python's `while` and a slight variant:

```
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

`i++` means "add 1 to i." Equivalent to `i = i + 1`. You'll see `++` everywhere.

FizzBuzz, side by side

Python:

```python
for n in range(1, 21):
    if n % 15 == 0:
        print("FizzBuzz")
    elif n % 3 == 0:
        print("Fizz")
    elif n % 5 == 0:
        print("Buzz")
    else:
        print(n)
```

C:

```c
#include <stdio.h>

int main(void)
{
    for (int n = 1; n <= 20; n++) {
        if (n % 15 == 0)      printf("FizzBuzz\n");
        else if (n % 3 == 0)  printf("Fizz\n");
        else if (n % 5 == 0)  printf("Buzz\n");
        else                  printf("%d\n", n);
    }
    return 0;
}
```

Compile and run:

```bash
gcc fizzbuzz.c -o fizzbuzz -Wall -Wextra
./fizzbuzz
```

`-Wall -Wextra` means “turn on all warnings.” Always use them. The compiler will tell you about likely bugs.

Functions in C

Python:

```python
def add(a, b):
    return a + b

print(add(2, 3))
```

C:

```c
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int main(void)
{
    printf("%d\n", add(2, 3));
    return 0;
}
```

The function declaration is more explicit:

- `int add(int a, int b)` — "function `add` that returns `int`, takes two `int` parameters."
- The function definition must be **above** any code that calls it (or you must declare a **prototype** at the top).

Functions in C are pure recipe: type in, type out. No magic.

Multiple-function example

```
#include <stdio.h>

int is_even(int n)
{
    return n % 2 == 0;     // returns 1 (true) or 0 (false)
}

int sum_evens_up_to(int N)
{
    int total = 0;
    for (int i = 1; i <= N; i++) {
        if (is_even(i)) {
            total = total + i;
        }
    }
    return total;
}

int main(void)
{
    printf("%d\n", sum_evens_up_to(10));    // 30
    printf("%d\n", sum_evens_up_to(100));   // 2550
    return 0;
}
```

Same shape as Python. The skeleton transfers directly.

Reading input

C's input is uglier. Don't worry about it for now; we'll do it properly in Part I. For chapters where you need it, you can use:

```
int n;
printf("Enter a number: ");
scanf("%d", &n);
printf("You entered %d.\n", n);
```

The `&n` is a *pointer* to `n` — `scanf` writes the value back into the variable through the pointer. We'll explain pointers in Part I, chapter 4. For now, just type it.

The compile-edit-run loop

In Python:

```
nano hello.py
python3 hello.py
```

In C:

```
nano hello.c
gcc hello.c -o hello -Wall -Wextra
./hello
```

You add a `gcc` step. The compiler will catch many mistakes for you.

A typical session looks like:

```
$ gcc hello.c -o hello -Wall -Wextra
hello.c: In function 'main':
hello.c:5:5: error: 'pirntf' undeclared (first use in this function); did you mean 'printf'?
    5 |     pirntf("hello\n");
      |     ^~~~~~
      |     printf
```

Read the error: file `hello.c`, line 5, you typed `pirntf` instead of `printf`. Fix and rebuild.

This loop — write, compile, fix errors, rebuild — is the heartbeat of C development. Get used to it.

Try it (30 minutes)

Build the C version of your Python `temperature_v2.py` (the menu-driven temperature converter from chapter 8).

Skeleton:

```c
#include <stdio.h>

double f_to_c(double f)
{
    return (f - 32.0) * 5.0 / 9.0;
}

double c_to_f(double c)
{
    return c * 9.0 / 5.0 + 32.0;
}

int main(void)
{
    while (1) {
        printf("\n1. F -> C   2. C -> F   3. quit\nchoice: ");
        int choice;
        scanf("%d", &choice);

        if (choice == 3) break;

        double t;
        printf("temperature: ");
        scanf("%lf", &t);

        if (choice == 1) {
            printf("%f F = %f C\n", t, f_to_c(t));
        } else if (choice == 2) {
            printf("%f C = %f F\n", t, c_to_f(t));
        } else {
            printf("invalid choice\n");
        }
    }
    return 0;
}
```

Build and run. Type some inputs.

(`scanf("%lf", ...)` reads a `double`. The `l` is for “long.”)

Differences you must internalize

Concept	Python	C	Why C is “harder”
Variables	`x = 5`	`int x = 5;`	You must say the type.
Strings	`"abc"` is a string	`"abc"` is an array of `char` ending with `\0`	Manipulating strings requires care.
Memory	automatic	manual (`malloc` / `free`)	Power and footguns both.
Errors	runtime, friendly	compile-time mostly, less friendly	But faster and gives more control.
Build step	none	`gcc`	One more thing to learn.
Speed	slow	fast	Why kernels are in C.

C is harder because the *machine model is exposed*. That’s also why it’s the language of operating systems, drivers, and game engines. You need control over every byte. Python hides all that and trades it for speed.

You’re now allowed to feel like C is annoying. Push through. By the end of Part I it’ll feel natural.

Self-check

- What’s the C equivalent of Python’s `if x: print("yes")`?
- Why does `5/3` give `1` in C but `1.6666...` in Python?
- What does `gcc hello.c -o hello -Wall` do, in your own words?
- Why does C need braces and Python doesn’t?

You can now read C. You can now compile C. You’re ready for Part I.

But first: one more chapter. The most important one in this part.

➡️ Next: Chapter 10 — When things break

Chapter 10 — When things break (the most important chapter)

Programs don't work the first time. Or the second. Or the tenth. **That's not a bug — that's the job.**

A senior engineer is not someone whose code "just works." A senior engineer is someone who *fixes* code faster than a junior. The faster you can debug, the better an engineer you are.

This chapter is the one most beginners skip. **Don't skip it.** Reading it carefully now is worth 10 chapters of code.

The mindset

Three rules.

Rule 1: The computer is not lying to you

When the computer says "Permission denied," it is not picking on you. There genuinely is no permission. When it says `NameError: 'foo' is not defined`, the variable `foo` genuinely is not defined.

Always assume the error message is true. Read it.

99% of beginner debugging goes like this:

1. Get an error.
2. Don't read it.
3. Re-run the same thing hoping it works.
4. Same error.
5. Frustration.
6. Eventually read the error.
7. "Oh, I had a typo on line 5."

Skip steps 2-6. Read the error first.

Rule 2: Read the error from the top

When you get a stack of error output, **the cause is usually at the top, not the bottom**. (Some tools are reversed, but Python and `gcc` show the cause first.)

Example:

```
Traceback (most recent call last):
  File "fizzbuzz.py", line 4, in <module>
    print(n)
NameError: name 'n' is not defined
```

The cause: `n` is not defined. The location: `fizzbuzz.py`, line 4. The fix: define `n` (probably the loop variable was misspelled).

C example:

```
hello.c: In function 'main':
hello.c:5:5: error: expected ';' before '}' token
    5 |     printf("hi\n")
      |     ^
      |     ;
    6 | }
```

Cause: missing `;`. Location: line 5. The compiler even shows where to put it (`^` arrow under the spot, suggested `;` below).

Rule 3: Make one change at a time

When code is broken, the temptation is to change 3 things at once. Don't. Change **one** thing. Run. See what happens. Then the next change.

If you change 3 things and it works, you don't know which fix worked. Worse, you might have introduced 2 new bugs that cancel out.

Single-change debugging feels slow and is fast.

The 90% checklist

When something doesn't work, walk this list. The fix is almost always in here.

1. **Did you save the file?** Many beginners edit, run, get the old behavior, panic. Save (`Ctrl+S` or `:w` in vim).
2. **Are you running the right file?** `python3 fizz.py` when the file is `fizzbuzz.py` produces "no such file."
3. **Spelling.** `pirntf` vs `printf`. `lenght` vs `length`. Run `grep` over your code if in doubt.
4. **Indentation (Python)** or **braces (C)**. A single missed brace can cause weird errors many lines later.
5. **Did you include the right header (C)?** If you use `printf`, you need `#include <stdio.h>`.
6. **Is the file in the directory you think?** `pwd`, `ls`. Easy to be in `~/Documents` when your file is in `~/ldd-textbook`.
7. **Did the compile actually succeed?** If `gcc` printed errors, the binary didn't update. Running `./hello` is running yesterday's `hello`.
8. **Are arguments in the right order?** `cp src dst`, not `cp dst src`. `gcc -o hello hello.c` works; `gcc hello.c -o hello` works; `gcc hello.c hello -o` does not.

Fix any of these one at a time, re-run, see if the symptom is gone or different.

Reading C compiler errors

C compiler errors are scary at first. They follow a pattern:

```
file:line:column: severity: message
```

Common ones:

`error: expected ';' before ...`

You forgot a `;`. The line and column tell you where.

`error: 'foo' undeclared`

You used `foo` without declaring it. Either `int foo;` it, or check spelling.

`warning: implicit declaration of function 'printf'`

You called a function the compiler doesn't know about. Probably missing `#include <stdio.h>`.

`warning: control reaches end of non-void function`

Your function returns `int` but doesn't `return` anything. Add `return 0;` (or whatever).

`error: too few arguments to function 'add'`

You declared `add(int a, int b)` and called `add(5)`. Pass both.

`Segmentation fault (core dumped)`

The program **ran but crashed**. You accessed memory you weren't supposed to (often: a NULL or uninitialized pointer). This is *the* C beginner experience. We dedicate Part I chapter 4 to it. For now, narrow down with print statements.

`undefined reference to 'foo'`

The compiler compiled OK but the **linker** can't find `foo`. Either you forgot to define `foo`, or you forgot to pass the `.c` file or library to `gcc`.

`warning: unused variable 'x'`

You declared `x` but never used it. Delete the declaration, or use it.

Always compile with `-Wall -Wextra` so you see warnings. Treat warnings as errors. They are.

Reading Python errors

Python errors are friendlier. They almost always say:

- the kind (`NameError`, `TypeError`, `ValueError`, `IndexError`, `SyntaxError`, ...).
- the file and line.
- the underlying cause.

Pattern: read the **last** line (the kind + message). Look at the line in your file. Fix.

The most common Python errors:

Error	Means	Fix
`SyntaxError`	invalid grammar	look at the line, count quotes, parens, colons
`IndentationError`	bad spaces	use 4 spaces; don't mix tabs
`NameError`	variable not defined	spell it right, define it before use
`TypeError`	wrong kind of value	`int + str`? convert one side
`ValueError`	right type, wrong content	`int("abc")` — check input
`IndexError`	index out of range	`len(list)` to know how big it is

Error	Means	Fix
`KeyError`	missing dict key	check the key exists first
`ZeroDivisionError`	divided by 0	guard with `if x != 0:`
`AttributeError`	object doesn't have that thing	check spelling, type

Print debugging

The classic move when stuck: add `print` statements to see what the program is doing.

```
def sum_evens_up_to(N):
    total = 0
    for i in range(1, N + 1):
        print(f"DEBUG: i={i}, total={total}")
        if i % 2 == 0:
            total = total + i
    return total
```

Now run it. The `DEBUG:` output shows you what's happening. When you see the bug, fix it, then **delete the print**.

In C:

```
printf("DEBUG: i=%d total=%d\n", i, total);
```

Same idea.

There are fancier tools (`gdb`, `pdb`, `pyspy`, `strace`, `ftrace`, ...). Learn `print` first. It works everywhere, including inside drivers (where it's called `printk`).

The 30-minute rule

If you've been stuck on the same problem for **30 minutes** without progress, do this:

1. **Stand up.** Walk for 5 minutes. No phone.
2. **Re-state the problem in a notebook.** What were you trying to do? What's actually happening? What's the gap?
3. **Reduce.** Can you make a smaller test case that fails? A 5-line program that shows the same bug?
4. **Search.** Copy the *exact* error message into Google. Add the language name. (`gcc "expected ';' before"`. `python3 "TypeError" "NoneType"`.)
5. **Read someone else's question.** Stack Overflow has the answer to most beginner questions. Read the *accepted* answer carefully.
6. **Ask.** If still stuck after 60 minutes, ask in:
   - your team's Slack/Teams (don't sit silently),
   - `#linux-newbie` IRC,
   - a coworker.

There is no medal for struggling alone for 6 hours. Senior engineers ask each other for help all the time. **Asking is a skill, not a weakness.**

How to ask a good question

A bad question:

> hi guys my code dont work plz help

You will get no response.

A good question:

> Trying to compile this 8-line C program (paste). It compiles fine but at runtime I get `Segmentation fault`. I'm running it on Ubuntu 22.04, gcc 11.4. I've already tried changing the array size from 100 to 10. Same crash. What am I missing?

The good question:

- shows the exact code,
- shows the exact error,
- shows the environment,
- shows what you've already tried.

Spending 10 minutes writing a good question often *answers itself* — you discover the bug while writing.

When the program "just hangs"

If a program seems to be doing nothing forever:

- Press `Ctrl+C` to interrupt.
- It's almost always an infinite loop. Trace through: what's the loop condition? What changes it? Maybe nothing.

When the kernel "just hangs" (foreshadowing)

In Part V you'll be writing kernel code, where the consequences are bigger. A bad kernel module can hang or crash the whole machine. That's why we always test in a VM. We'll get there.

For now: any "loop forever" you make in Python or userspace C just runs until you `Ctrl+C` it. No harm done.

A concrete debugging example

Here's a real session. You wrote `factorial.py`:

```
def factorial(n):
    if n == 0:
        return 1
    return n + factorial(n - 1)        # bug

print(factorial(5))
```

Expected: `120`. Actual: `15`. Why?

1. **Is the answer wrong, or did the program crash?** It runs, but answer is wrong. So it's a logic bug, not a syntax/runtime bug.
2. **What do I expect for n=5?** 5×4×3×2×1 = 120.
3. **What does the code compute?** 5 + 4 + 3 + 2 + 1 + 1 = 16… wait, it gives 15. Hmm. Oh: 5 + 4 + 3 + 2 + 1 + factorial(0) = 5+4+3+2+1+1 = 16. Why does it print 15?
4. **Add prints.**

```
def factorial(n):
    print(f"factorial({n}) called")
    if n == 0:
        return 1
    r = n + factorial(n - 1)
    print(f"factorial({n}) returning {r}")
    return r

print(factorial(5))
```

Run. Look at the output. You'll see the chain of calls and what each returned. The bug becomes obvious: `+` should be `*`.

Fix:

```
return n * factorial(n - 1)
```

Run again: `120`. Done.

This is the entire game. Form a hypothesis, add prints, look, refine.

Try it (30 minutes)

I've given you 5 broken Python programs. Your job: find and fix each.

(Type each one as-is, run, read the error, fix.)

bug1.py

```
print("hello world)
```

bug2.py

```
x = 10
y = 20
z = x + Y
print(z)
```

bug3.py

```
for i in range(5)
    print(i)
```

bug4.py

```
n = int(input("number: "))
if n = 0:
```

```
        print("zero")
    else:
        print("nonzero")
```

bug5.py

```
    def square(n):
        n * n

    print(square(5))
```

(Hint for bug5: it prints `None`. Why?)

Solutions are intentionally not given here. Stare at the error and fix. If you get all 5 in 30 minutes, you've cleared the bar.

Self-check

- Why does "make one change at a time" matter?
- What's the 30-minute rule?
- What's a good way to phrase a question when you're stuck?
- If a program prints `None`, what's the most likely cause?
- You see `Segmentation fault (core dumped)` running a C program. Did the compile succeed? (Yes — segfault is a *runtime* crash.)

Debugging is the skill. You will use it every day for the rest of your career. Be friends with errors.

➡️ Next: Chapter 11 — Bridge to Part I

Chapter 11 — Bridge to Part I

You made it. **Take a moment.** Two or three weeks ago you had not opened a terminal. Now you have:

- navigated Linux from the command line,
- written code in Python and C,
- used variables, conditions, loops, and functions,
- compiled C programs,
- read error messages and fixed bugs.

That is genuinely 80% of what most "intro to programming" courses teach in a semester. You did it in chapters.

You are no longer a beginner-beginner. You are now just a beginner. That's a real promotion.

What changes in Part I

Part I is **C, deeply.** The same ideas you learned (variables, loops, functions) but with the *machine-level meaning* exposed.

In Python, when you wrote `x = 5`, the language hid:

- where `x` lives in memory,
- what shape `5` has in bytes,
- what happens if you ask for `x` after deleting it,
- how the operating system gave you that memory in the first place.

In Part I we lift the floorboards. We talk about:

- **Bytes, words, alignment**: what `int` *physically* is.
- **Pointers**: a number that is the address of another byte. The single most powerful and dangerous thing in C.
- **Memory layout**: stack vs heap, what `malloc` and `free` actually do.
- **Undefined behavior**: places where C says "you broke the rules; whatever happens is on you."

Then we apply it to write our own memory allocator, the kind you'd find in a kernel driver.

Part I is **the** part that separates "people who know C" from "people who can write C in a kernel." Don't rush it.

Recommended pacing for Part I

Chapter	Days	Goal
1 — What is a program	1	Re-read with fresh eyes. Most of it should now feel familiar.
2 — Toolchain	2	Get comfortable with `gcc`, `make`, header files.

Chapter	Days	Goal
3 — Types, integer promotion, UB	3	This is dense. Take your time.
4 — Pointers	5+	The famous one. Draw boxes on paper.
5 — Structs, unions, alignment	3	Foundational for kernel data types.
6 — Bit manipulation	2	The language of hardware registers.
7 — Allocators	4	Hands-on: build one.
8 — Preprocessor & build	2	Demystifies `#include`.
9 — Function pointers	2	The mechanism behind `file_operations`.
10 — Defensive C	2	Habits to keep.

About **6 weeks at 1 hour a day**, plus the corresponding labs.

If a chapter feels rough: re-read; type the example three times; sleep; come back. The brain consolidates this stuff while you sleep.

How to read Part I (and the rest of the book)

1. **Read the chapter once for shape.** Don't try to understand everything. Get the *map*.
2. **Type every code block** into your editor. Compile. Run.
3. **Modify the code.** Break it. See what changes. (Compilers can't catch bugs you didn't write.)
4. **Do the exercises.** Even the ones that seem easy.
5. **Check the corresponding lab in `labs/`** if there is one.
6. **Write 3 sentences in your notebook**: what's clear, what's still fuzzy, what you'll do tomorrow.

If after all that something is still fuzzy, **flag it and move on**. Often the *next* chapter clarifies the previous one. Don't get stuck for 2 weeks on one paragraph.

What to expect emotionally

There will be moments — chapter 4 (pointers), chapter 7 (allocators), Part III (virtual memory), Part VII (GPU command submission) — where you will feel like you genuinely don't understand anything. **That is the experience.** Every kernel engineer has felt it. Some still do. They ship anyway.

When you hit that wall:

- Re-read the previous chapter.
- Sleep on it.
- Draw a diagram on paper.
- Come back.

Walls are growth. The ones who quit at the wall remain juniors forever. The ones who push through become senior.

A sneak preview: the same FizzBuzz, kernel-style

Here's what kernel C looks like, just so you've seen it. Don't try to understand it — just notice that it's *the same shape* as your `fizzbuzz.c`:

```c
static int my_module_init(void)
{
    int i;

    pr_info("module loaded\n");

    for (i = 1; i <= 20; i++) {
        if (i % 15 == 0)
            pr_info("FizzBuzz\n");
        else if (i % 3 == 0)
            pr_info("Fizz\n");
        else if (i % 5 == 0)
            pr_info("Buzz\n");
        else
            pr_info("%d\n", i);
    }

    return 0;
}
```

Differences from your fizzbuzz.c:

- `static int my_module_init(void)` instead of `int main(void)` — a function the kernel calls when this module loads.
- `pr_info(...)` instead of `printf(...)` — the kernel doesn't have stdio; `pr_info` writes to the kernel log instead.
- No `#include <stdio.h>` — kernel doesn't have those headers; it has `<linux/...>` ones.

Otherwise: same `for`, `if`, `else if`. **You already know how to read this.** You just need to learn the new vocabulary, which is what Parts V and VI do.

A reminder of the journey

You are following this map:

```
Part 0 (you are here, finishing) — Absolute zero
Part 1 — C deeply
Part 2 — C++ for systems
Part 3 — How the OS works
Part 4 — How threads and memory really work
Part 5 — The Linux kernel itself
Part 6 — Writing real Linux drivers
Part 7 — The GPU stack and the amdgpu driver
Part 8 — Hardware/software interaction (PCIe, DMA, profiling)
Part 9 — Getting hired and growing
```

That's a *years*-long climb in real terms — but you climb a few hundred feet a week and don't look down.

A request

Before you turn the page to Part I:

1. Open your notebook.
2. Write the date.

3. Answer in your own words:
   - “A program is…”
   - “A driver is…”
   - “When my program crashes, the first thing I do is…”
   - “The hardest thing in Part 0 was…”
   - “I’m proud that I…”

That’s your snapshot of Day 0 of your real engineering career. In 12 months, when you’re sending kernel patches, you’ll re-read it and smile.

OK. Let’s go.

➡️ Onward to Part I — C from Zero to Driver-Grade.

Part 0 Foundations — side topics every engineer needs

These five chapters cover concepts that the rest of the book *assumes* you know. If you've programmed before, you can skip them. If chapters in Part 0 left you unsure what "10 in hex" or "the AND of two bits" or "what `<<` does" means, **read these now.**

Read them in any order. Each is short.

#	Chapter	What it teaches
A1	Numbers — binary, hex, decimal	Why hardware uses base 2 / base 16, and how to read both
A2	Bits, bytes, words	What "8-bit" / "32-bit" really mean, KB vs KiB
A3	Boolean logic	AND / OR / NOT / XOR — the alphabet of digital hardware
A4	Shell power tour	Pipes, redirects, grep, env vars, scripting basics
A5	Character encoding	ASCII, UTF-8, why text is harder than it looks

After these, return to whatever chapter you came from.

A1 — Numbers: binary, hex, decimal

You will see numbers like `0xDEADBEEF`, `0b10110100`, `1 << 30` everywhere in driver code. They're not strange — they're just **the same number written in a different base**.

How counting works (you already know this)

When you write `347` you mean:

```
3 × 100 + 4 × 10 + 7 × 1
3 × 10²  + 4 × 10¹ + 7 × 10⁰
```

We use **base 10** because we have 10 fingers. Each digit can be 0–9. The position of a digit multiplies its value by a power of 10.

That's a *choice*, not a law. Computers chose differently.

Binary — base 2

A bit can be 0 or 1. Stack bits in positions, like decimal but with powers of 2:

```
     1   0   1   1   0   1   0   0     ← bits
   128  64  32  16   8   4   2   1     ← place values

= 128 + 32 + 16 + 4 = 180
```

Conventions:

- Written `0b10110100` in C/Python (the `0b` prefix says "binary").
- Bit positions are counted from the right, starting at **bit 0**: the rightmost bit is bit 0, the next bit 1, etc.

Why binary? Because hardware is a bunch of switches. A switch is on or off. 1 or 0. That's it.

Hex — base 16

Binary is annoying to read (`0b1011010001110100`...). Hex is denser. One hex digit = exactly **4 bits**:

Hex	Decimal	Binary
`0`	0	`0000`
`1`	1	`0001`
`2`	2	`0010`
`3`	3	`0011`
`4`	4	`0100`
`5`	5	`0101`
`6`	6	`0110`
`7`	7	`0111`

Hex	Decimal	Binary
`8`	8	`1000`
`9`	9	`1001`
`A`	10	`1010`
`B`	11	`1011`
`C`	12	`1100`
`D`	13	`1101`
`E`	14	`1110`
`F`	15	`1111`

Memorize this table. Right now. It will save you 1000 hours over your career.

A hex number is written with `0x` prefix:

```
0xCAFE = C    A    F    E
       = 1100 1010 1111 1110     ← 16 bits
       = 51966 in decimal
```

You read hex by chunking into 4-bit pieces. `0xDEADBEEF` is 8 hex digits = 32 bits = 4 bytes. Common GPU register addresses look exactly like this.

Why both?

Base	Where you see it
Decimal	Sizes ("buffer is 4096 bytes"), counts ("3 wavefronts"), array indices
Hex	Addresses (`0xFFFF8000`), bit masks (`0x000000FF`), magic constants
Binary	Reading individual bit fields in registers (rare; usually still hex)

Converting in your head

Decimal → binary

Keep dividing by 2, write down the remainders **bottom up**:

```
13 / 2 = 6  rem 1
 6 / 2 = 3  rem 0
 3 / 2 = 1  rem 1
 1 / 2 = 0  rem 1
```

Read remainders from bottom: `1101` = 13. Check: 8 + 4 + 1 = 13. √

Decimal → hex

Same trick with 16:

```
255 / 16 = 15  rem 15 → F
 15 / 16 = 0   rem 15 → F
```

So 255 = `0xFF`.

Binary → hex

Group bits into chunks of 4, **starting from the right**, then look up each chunk:

```
0b 1011 0100   →  B    4   →  0xB4
```

If the bit count isn't a multiple of 4, pad with leading zeros: `0b101 → 0b0101 → 0x5`.

Hex → binary

Each hex digit becomes 4 bits:

```
0xCAFE → 1100 1010 1111 1110
```

That's it. Hex ↔ binary is mechanical.

Powers of 2 — you must know these cold

Power	Decimal	Hex
2^0	1	0x1
2^1	2	0x2
2^2	4	0x4
2^3	8	0x8
2^4	16	0x10
2^5	32	0x20
2^6	64	0x40
2^7	128	0x80
2^8	256	0x100
2^9	512	0x200
2^{10}	1024	0x400
2^{12}	4096	0x1000 ← page size
2^{16}	65,536	0x10000
2^{20}	~1 million	0x100000 ← 1 MiB
2^{30}	~1 billion	0x40000000 ← 1 GiB
2^{32}	~4.3 billion	0x100000000 ← 32-bit limit
2^{64}	huge	(16 hex digits) ← 64-bit limit

Page size = 4096 = 0x1000 = 2^{12}. This will appear so many times it'll burn into your brain.

Quick conversions to memorize

```
0x10 = 16
0x20 = 32
0x40 = 64
0x100 = 256
0x1000 = 4096           ← page
0xFF = 255              ← max byte
0xFFFF = 65535          ← max 16-bit
0xFFFFFFFF = 4294967295 ← max 32-bit
```

Negative numbers — two's complement (just FYI)

Computers represent `-1` in 32-bit as `0xFFFFFFFF`. This is "two's complement." You don't need to internalize it now; just know:

- All bits 1 = -1.
- High bit 1 = negative (for signed types).
- Add 1 to `0xFFFFFFFF`, you get `0x00000000` (it overflows, like an odometer).

A real-world example

In `amdgpu` you'll see code like:

```
#define AMDGPU_FENCE_OWNER_UNDEFINED   ((void *)0ul)
#define AMDGPU_GEM_DOMAIN_VRAM         (1 << 2)   /* = 4 */
#define AMDGPU_GEM_DOMAIN_GTT          (1 << 1)   /* = 2 */
#define AMDGPU_GEM_DOMAIN_CPU          (1 << 0)   /* = 1 */
```

`(1 << 2)` is the same as `0b100` is the same as `0x4` is the same as `4`. They're using bits as flags: "VRAM" = bit 2, "GTT" = bit 1, "CPU" = bit 0. You can OR them: `VRAM | GTT = 0b110 = 0x6 = 6`.

You will see this pattern everywhere. The whole binary/hex apparatus exists to make it readable.

Try it

Convert without a calculator:

1. `0x40` in decimal? (Answer: 64)
2. `170` in hex? (Answer: 0xAA)
3. `0b11010110` in hex? (Hint: split as `1101 0110`. Answer: 0xD6)
4. What's `1 << 12`? (Answer: 4096 = 0x1000 = page size)
5. What's `0xFF & 0x0F`? (Hint: AND of bits — see chapter A3. Answer: 0x0F = 15)

If you got 4 of 5, you're solid. Move on.

A2 — Bits, bytes, words

In chapter A1 you learned what a binary digit (a **bit**) is. Now we group them.

The hierarchy

Unit	Size	Notes
bit	1 binary digit (0 or 1)	smallest possible
nibble	4 bits	one hex digit; rarely named
byte	8 bits	the basic addressable unit on every modern CPU
half-word	16 bits / 2 bytes	C: `int16_t`, `uint16_t`
word	32 bits / 4 bytes (on x86_64/ AArch64)	C: `int32_t`, `uint32_t`
double word / "qword"	64 bits / 8 bytes	C: `int64_t`, `uint64_t`

"Word" is squishy. On x86, "word" historically meant 16 bits; on AArch64, 32 bits. **In Linux kernel C, prefer the explicit names: `u8`, `u16`, `u32`, `u64`** (or `uint8_t` etc. in userspace).

The byte is the fundamental unit

Every memory address points to one byte. When you read or write memory, you read or write some integer number of bytes. The CPU has special instructions to deal with 1, 2, 4, or 8 bytes at a time.

```
Address    Byte
0x1000   ┌──────┐
0x1001   │      │
0x1002   │ each │
0x1003   │ box  │
0x1004   │ is 8 │
0x1005   │ bits │
0x1006   │      │
0x1007   └──────┘
```

A 32-bit `int` at address `0x1000` occupies `0x1000`, `0x1001`, `0x1002`, `0x1003` — four consecutive bytes.

Sizes (memorize)

```
8 bits        = 1 byte
16 bits       = 2 bytes
32 bits       = 4 bytes
64 bits       = 8 bytes
128 bits      = 16 bytes
512 bits      = 64 bytes  ← cache line size on most x86
4096 bits     = 512 bytes
4 KiB         = 4096 bytes  ← page size
2 MiB         = huge page
1 GiB         = sometimes huge page
```

How big is a byte's range?

A byte (8 bits) holds $2^8 = 256$ values.

As C type	Range
`unsigned char` / `u8`	0 to 255 (`0x00` to `0xFF`)
`signed char` / `s8`	−128 to 127

A 32-bit value:

As C type	Range
`u32`	0 to ~4.29 billion
`s32`	~−2.15 billion to ~2.15 billion

A 64-bit value: ~18 quintillion unsigned. (For the curious: `u64` max = `0xFFFF_FFFF_FFFF_FFFF`.)

You will hit overflow bugs eventually. Always know how big your integer can get.

KB vs KiB — the trap

There are **two** conventions for "kilobyte":

Symbol	Meaning	Used by
KB (or kB)	1000 bytes (decimal)	hard drive marketing, network speed
KiB	1024 bytes (binary)	RAM, kernel sizes, programmers

The same trap for MB / MiB, GB / GiB:

```
1 KiB = 1024 bytes   = 2^10
1 MiB = 1024 KiB     = 2^20 = 1,048,576 bytes
1 GiB = 1024 MiB     = 2^30
1 TiB = 1024 GiB     = 2^40
```

In kernel code and this book, when we say "4 KB page" we mean 4 KiB = 4096 bytes. SI purists are correct that "kB" should mean 1000, but tradition wins.

A "16 GB" HDD is 16,000,000,000 decimal bytes. A "16 GiB" RAM stick is 17,179,869,184 binary bytes. **They look the same from the marketing slide; they are not.**

Endianness — which byte goes first?

A 32-bit integer like `0x12345678` is 4 bytes: `12`, `34`, `56`, `78`. But how are they ordered in memory?

- **Little-endian**: lowest byte first. `0x1000` = `0x78`, `0x1001` = `0x56`, …
- **Big-endian**: highest byte first. `0x1000` = `0x12`, `0x1001` = `0x34`, …

x86 and AArch64 (in normal mode) are **little-endian**. Network protocols are big-endian ("network byte order"). When parsing wire formats you'll need `htonl()`, `ntohl()`, etc.

Test it on your machine:

```c
#include <stdio.h>
#include <stdint.h>
```

```
int main(void) {
    uint32_t n = 0x12345678;
    uint8_t  *p = (uint8_t *)&n;
    printf("%02x %02x %02x %02x\n", p[0], p[1], p[2], p[3]);
    // little-endian prints: 78 56 34 12
}
```

Alignment

CPUs are picky. Reading a `u32` is fastest when the address is a multiple of 4. Reading a `u64` from an address that's a multiple of 8. Misaligned reads are either slow (x86 tolerates them) or trap (some ARM modes).

```
Aligned u32:    address 0x1000 (multiple of 4)
                address 0x1004
                address 0x1008

Misaligned u32: address 0x1001 ← bad
```

Compiler will pad structs to keep things aligned:

```
struct s {
    uint8_t  a;     // 1 byte, then 3 padding bytes
    uint32_t b;     // 4 bytes, total 8
    uint8_t  c;     // 1 byte, then 3 padding bytes (struct must end aligned for arrays)
};
sizeof(struct s) == 12
```

In Part I chapter 5 we do this in detail. For now, just know that data has natural lanes.

Bit ordering inside a byte

Bit 0 of a byte is **the rightmost (least significant)** bit. So:

```
binary:    0  1  0  1  1  0  0  1   = 0x59
bit:       7  6  5  4  3  2  1  0
```

You'll see notation like `[7:0]` meaning "bits 7 through 0" (the whole byte) or `[15:8]` meaning "the high byte of a 16-bit value." Read this comfortably; hardware datasheets use it constantly.

Common bit-twiddling idioms

These come up in every driver. We'll write them in C; same idea in Python.

Set bit N

```
x |= (1u << N);     // OR with a mask of just bit N
```

Clear bit N

```
x &= ~(1u << N);    // AND with a mask that has 0 only at bit N
```

Test bit N

```
        if (x & (1u << N)) { ... }
```

Toggle bit N

```
        x ^= (1u << N);     // XOR
```

Extract bits [hi:lo]

```
        uint32_t field = (x >> lo) & ((1u << (hi - lo + 1)) - 1);
```

This is *the* GPU register pattern. A 32-bit register has 5 bit-fields packed in; you extract them with shifts and masks.

We'll have a whole chapter (Part I, ch 6) on this. Bookmark it.

Try it

1. How many bits in `u32`? In `u64`? (32 / 64.)
2. What is `0x80` in binary, and which bit is set? (Answer: `1000 0000`, bit 7.)
3. If `int x = 0xFF;` what is `x & 0x0F`? (Answer: `0x0F` — bottom nibble.)
4. Is your CPU little-endian? (Almost certainly yes. Verify with the snippet above.)
5. Why does padding exist in structs?

A3 — Boolean logic: AND, OR, NOT, XOR

These four operations are the alphabet of digital hardware. Inside a CPU or a GPU, billions of transistors are arranged into networks of these four. **Everything else is built from them.**

The four operations

NOT (also written `!`, `~`, ¬)

Flip a bit.

input	NOT input
0	1
1	0

```
~0b0011 = 0b1100
```

C has both `!` (logical, gives 1 or 0) and `~` (bitwise, flips every bit). Important difference:

```
        !0xF0     // 0   (because 0xF0 is nonzero, "true", so NOT it is "false")
        ~0xF0     // 0x...0F (every bit flipped)
```

AND (also written `&`, `&&`, ∧)

True only if **both** are true. Like multiplication of bits.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

```
0b1100
0b1010
------
0b1000     ← bit-by-bit AND
```

Two C operators: `&` (bitwise) and `&&` (logical, short-circuits).

OR (also written `|`, `||`, ∨)

True if **either** is true.

A	B	A OR B
0	0	0
0	1	1

A	B	A OR B
1	0	1
1	1	1

```
0b1100
0b1010
------
0b1110
```

C: `|` (bitwise) and `||` (logical, short-circuits).

XOR (exclusive OR, also `^`, ⊕)

True if **exactly one** is true. Equivalent to "they're different."

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

```
0b1100
0b1010
------
0b0110     ← differs in 2nd-from-left and rightmost
```

C: `^` (bitwise). No logical XOR keyword (you write `(a && !b) || (!a && b)` or just use `!=`).

XOR has a magic property: `a ^ a = 0`, and `a ^ 0 = a`. This is why XOR is used in hashes, error correction, and the classic "swap without a temp variable" trick:

```
a = a ^ b;
b = a ^ b;     // now b = original a
a = a ^ b;     // now a = original b
```

(Don't actually use this in real code; the compiler does better with a temp.)

Logical vs bitwise (the most common beginner trap)

```
int  flags = 0xC;       // 0b1100
int  mask  = 0x9;       // 0b1001

flags & mask     // 0b1000 = 0x8 (bitwise AND each pair)
flags && mask    // 1 (both nonzero, so logically true)

flags | mask     // 0b1101 = 0xD
flags || mask    // 1
```

Rule: `&` and `|` operate on every bit individually. `&&` and `||` operate on the value as a whole and return either 0 or 1.

In `if`s and conditions, almost always `&&` and `||`. In bit-twiddling, almost always `&` and `|`. Don't mix them up.

Truth tables tell you everything

Anything you can describe with a truth table can be built from AND/OR/NOT. That's mathematically proven.

For example, "true if exactly two of A, B, C are true":

A	B	C	result
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

You can mechanically translate that into a Boolean formula. The hardware engineers who design GPUs do exactly this for every signal.

Why this matters for software

You're writing software, not hardware, but the same operators show up because they're efficient on a CPU.

Flag fields

A driver often packs many true/false properties into one integer:

```
#define IRQ_DISABLED   (1u << 0)   // 0x01
#define DMA_READY      (1u << 1)   // 0x02
#define ERROR          (1u << 2)   // 0x04
#define HOTPLUG        (1u << 3)   // 0x08

uint32_t status = 0;

status |= DMA_READY;                    // mark DMA ready
status |= ERROR | HOTPLUG;              // OR in two at once
if (status & ERROR) { /* handle */ }
status &= ~ERROR;                       // clear error
```

This pattern is in every kernel driver.

Masks

To extract bits 4..7 of a register:

```
uint32_t reg  = readl(REG_X);
uint32_t mask = 0xF0;                    // bits 4..7
uint32_t val  = (reg & mask) >> 4;
```

Boolean math identities (memorize)

```
A AND 0 = 0          A OR 0 = A          A XOR 0 = A
A AND 1 = A          A OR 1 = 1          A XOR 1 = NOT A
```

```
A AND A = A          A OR A = A          A XOR A = 0
NOT NOT A = A
NOT(A AND B) = NOT A OR NOT B            ← De Morgan
NOT(A OR B)  = NOT A AND NOT B           ← De Morgan
```

De Morgan is the trick to swap AND/OR through a NOT. Useful when refactoring conditions.

Hardware view (one paragraph, then we move on)

A “transistor” is a tiny switch controlled by an electrical voltage. A few transistors arranged make a “gate” — AND gate, OR gate, NOT gate (called an inverter). Wire millions of gates together → flip-flops, registers, adders, multipliers. Wire billions of gates → a GPU. The whole pyramid stands on AND/OR/NOT.

Knowing this is why senior engineers can intuit *why* certain CPU instructions are cheap and certain ones are expensive: the cheap ones map to a few gate-delays; the expensive ones (like float divide) require many cascaded blocks.

Try it

1. What is `0b1100 ^ 0b1010` in binary, in hex, in decimal?
2. What is `~0xFF` interpreted as a 32-bit unsigned int? (Answer: `0xFFFFFF00`.)
3. Write a one-liner: clear the bottom 12 bits of `x`. (Answer: `x &= ~0xFFFu;` — clears bits 0..11.)
4. Why is `x % 16` the same as `x & 0xF`? (Answer: 16 = 2^4, so `% 16` keeps just the bottom 4 bits, which is what `& 0xF` does. Faster on hardware.)
5. Translate “the device is ready AND not in error mode” into C, given `status = uint32_t`, `DMA_READY = bit 1`, `ERROR = bit 2`. (Answer: `if ((status & DMA_READY) && !(status & ERROR))`.)

If 4 of 5 are right, you have boolean logic. Move on.

A4 — Shell power tour

You learned `pwd`, `ls`, `cd`, `cp`, `mv`, `rm` in chapter 2. The shell is much more powerful. This chapter shows you the parts you'll use every day for the rest of your career.

We're using **bash** (or zsh — they're 95% identical for this material).

Pipes — `|`

A **pipe** sends the output of one command into the input of another. This is the magic that makes the shell composable.

```
ls /etc | wc -l                # how many entries in /etc?
ps aux | grep firefox          # find firefox processes
dmesg | tail -50               # last 50 kernel log lines
cat file.log | grep ERROR | wc -l   # how many ERROR lines?
```

`wc -l` counts lines. `grep pattern` keeps only lines matching. `tail -N` shows last N. `head -N` shows first N.

Read `A | B | C | D` as "do A, send result to B, send result to C, send result to D." Like a factory line.

Redirects — `>`, `>>`, `<`, `2>`

Where output goes (or input comes from):

Symbol	Meaning
`cmd > file`	overwrite `file` with stdout
`cmd >> file`	append stdout to `file`
`cmd < file`	use `file` as stdin
`cmd 2> file`	write stderr to file
`cmd > out 2> err`	stdout to `out`, stderr to `err`
`cmd > both 2>&1`	stdout AND stderr to `both`
`cmd &> both`	(bash shortcut) same as above
`cmd > /dev/null`	throw stdout away

Examples:

```
make > build.log 2>&1            # capture all build output
gcc x.c 2> errors.txt            # only error messages
sort < unsorted.txt > sorted.txt # read one, write the other
```

`grep` — find text

`grep pattern files` prints lines matching `pattern`.

```
grep "TODO" *.c                  # all TODOs in C files
grep -r "amdgpu_vm_init" .       # search recursively
grep -rn "EXPORT_SYMBOL" .       # -n shows line numbers
grep -ri "fence" drivers/gpu/    # -i = case insensitive
grep -v "^#" config              # invert: lines that DON'T start with #
grep -E "(foo|bar)" file         # extended regex
```

In a kernel tree, `grep -rn` is your most-used command. Modern faster alternative: `rg` **(ripgrep)** — same usage, way faster:

```
sudo apt install ripgrep
rg amdgpu_vm_init drivers/gpu/   # finds in seconds
```

Use `rg` whenever it's installed.

`find` — find files

```
find . -name "*.c"                # all C files under current dir
find . -name "*.h" -newer foo     # files newer than foo
find . -type d -name build        # directories named "build"
find . -size +10M                 # bigger than 10 MB
find . -name "*.tmp" -delete      # delete .tmp files (CAREFUL)
find . -name "*.c" -exec wc -l {} \;   # run command on each match
```

`xargs` — run a command per line of input

```
find . -name "*.c" | xargs grep -l "amdgpu"    # files containing amdgpu
echo "1 2 3 4" | xargs -n 1 echo                # one per line
ls *.txt | xargs rm                             # delete (use -f for force)
```

Variables and `$`

```
NAME="Sudhanshu"
echo "Hello, $NAME"
echo 'Hello, $NAME'        # single quotes don't expand!

PATH=/usr/local/bin:$PATH  # prepend a directory to PATH
export EDITOR=vim          # export = make available to child processes
```

Useful built-ins:

```
$HOME       # your home directory
$USER       # your username
$PWD        # current dir
$$          # process id of the shell
$?          # exit code of last command (0 = success)
$0          # script name
$1, $2, ... # script arguments
```

Command substitution — `$(...)`

Replace a command with its output:

```
echo "today is $(date)"
KERNEL_VER=$(uname -r)
echo $KERNEL_VER
DIR=/lib/modules/$(uname -r)/build
ls $DIR
```

Old syntax with backticks (still works, less readable):

```
echo "today is `date`"
```

Always prefer `$(...)`.

Chaining: `&&`, `||`, `;`

```
make && ./run_tests             # run tests only if make succeeded
make || echo "build failed"     # message only if make failed
mkdir build; cd build           # always do both, ignoring failures
```

Rule: **`&&` for "must succeed", `||` for "or else", `;` for "just keep going."**

You'll see this *all the time*:

```
mkdir -p out && cd out && cmake .. && make -j8
```

If any step fails, the rest don't run. That's correct behavior for a build.

Globs — `*`, `?`, `[]`, `{}`

```
ls *.c              # any C file
ls a*.c             # C files starting with a
ls ?.c              # exactly 1 character then .c
ls [abc].c          # a.c, b.c, or c.c
ls *.{c,h}          # .c or .h files (brace expansion)
ls drivers/**/*.c   # recursive (bash needs `shopt -s globstar`)
```

The shell expands these *before* the command runs. `ls *.c` becomes `ls a.c b.c main.c` ... before `ls` even sees it.

History

```
history                 # show last commands
!!                      # rerun last command
!42                     # rerun command #42 from history
!grep                   # rerun last command starting with "grep"
Ctrl+R                  # interactive reverse search (super useful)
```

`Ctrl+R` then start typing a fragment of a previous command — bash searches history. Press `Ctrl+R` again to find older matches. Press Enter to run, or → to edit.

Useful one-liners

```
# Most-recently-modified files
ls -lt | head
```

```
# Disk usage of current dir, sorted
du -sh * | sort -h

# Find big files
find / -size +1G 2>/dev/null

# Watch a command repeatedly
watch -n 1 "cat /sys/kernel/debug/dri/0/amdgpu_pm_info"

# Process tree
ps fauxw | less
pstree

# Memory + CPU snapshot
top                       # press q to quit
htop                      # nicer (sudo apt install htop)

# Network
ss -tulpn                 # listening ports
ip a                      # interfaces
```

Shell scripts

A shell script is just a file containing shell commands. First line says which shell:

```
#!/bin/bash
set -e                  # stop on first error
set -u                  # error on undefined variables

SRC=~/work/linux
BUILD=$SRC/build

cd "$SRC"
make O="$BUILD" -j$(nproc)
echo "build OK"
```

Save as `build.sh`, then:

```
chmod +x build.sh
./build.sh
```

`set -e` and `set -u` are critical. Without them, scripts swallow errors and you debug for hours. Always use them.

Functions in scripts

```
log() {
    echo "[$(date +%H:%M:%S)] $*"
}

log "Starting build"
make
log "Build complete"
```

Functions take args as `$1`, `$2`, ... or `$@` (all of them).

Loops

```
for f in *.c; do
    echo "compiling $f"
    gcc -c "$f"
done

i=0
while [ $i -lt 10 ]; do
    echo $i
    i=$((i + 1))
done
```

Note: `[ ... ]` is a command (`test`). Spaces around brackets are required. `$(( ... ))` is arithmetic.

Real-world combos

These are patterns you'll re-use forever.

```
# Find every C file containing "foo", show file:line:match
rg -n "foo" --type c

# Count lines of code in a directory
find . -name "*.c" | xargs wc -l | tail -1

# Diff two directories' file lists
diff <(ls a) <(ls b)

# Run a command 100 times and capture timing
for i in {1..100}; do
    time ./benchmark
done 2>&1 | grep real

# Tail a log file as it grows
tail -f /var/log/syslog

# Rerun until success
until make; do
    echo "fail, retry"
    sleep 1
done

# Make a backup before editing
cp file.c file.c.bak
nano file.c
diff file.c.bak file.c
```

Aliases — keep your hands fast

In `~/.bashrc`:

```
alias ll='ls -lah'
alias gs='git status -sb'
alias ka='sudo make modules_install && sudo make install'
alias dmw='sudo dmesg -w'
alias k='cd ~/work/linux'
```

Reload with `source ~/.bashrc`. Saves keystrokes; saves wrist tendons.

Cheat-sheet card (print this)

```
| ; & |   chain commands
| && ||   conditional chain
| > >> <  redirects
| | (pipe) cmd1 | cmd2
| $(...)  command substitution
| $VAR    variable
| *.c     glob
| 2>&1    merge stderr to stdout
| Ctrl+R  search history
| Ctrl+C  cancel
| Ctrl+D  end of input / logout
| Ctrl+L  clear screen
| Ctrl+A / Ctrl+E  start / end of line
| Alt+B / Alt+F   back / forward word
```

Try it

1. Count how many `.c` files are under `/usr/src/linux*/drivers/gpu/` (or any dir on your machine).
2. Find every line containing `EXPORT_SYMBOL` in your home dir's projects, with file and line number.
3. Write a one-liner to print the 5 biggest files in your home dir.
4. Write a 5-line shell script that creates a folder, cd's in, runs `git init`, and creates a README.md.
5. Find a process and kill it (without using `kill -9` first; try just `kill`).

Spend an hour playing with these. The shell is a multi-tool — every minute here saves hours later.

A5 — Character encoding: ASCII, UTF-8, and why text is harder than it looks

A computer doesn't store letters. It stores bytes. To turn bytes into letters you need an **encoding** — a table that says "byte 0x41 means 'A'."

You will hit encoding bugs when:

- a `printf` in your driver outputs garbage,
- a file copied from Windows shows weird characters,
- a regex doesn't match what you expect,
- a shader compiler chokes on a Unicode quote that snuck in.

A short chapter, a useful chapter.

ASCII — the original 128 characters

ASCII (American Standard Code for Information Interchange, 1963) maps 128 codes (0–127) to characters. Every modern encoding starts with these.

The important rows:

```
Code  Char       Notes
----  ----       ----
0x00  NUL        end of string in C ("\0")
0x09  TAB        \t
0x0A  LF         newline (Linux), \n
0x0D  CR         carriage return, \r (Windows uses CR+LF)
0x20  SPACE
0x21  !
0x22  "
0x27  '
0x2F  /
0x30..0x39  0..9
0x41..0x5A  A..Z
0x5C  \
0x61..0x7A  a..z
0x7B  {
0x7D  }
0x7F  DEL
```

Things that fall out:

- `'A' = 0x41`, `'a' = 0x61`. Lowercase = uppercase + 0x20. So `c | 0x20` lowercases an ASCII letter; `c & ~0x20` uppercases it.
- `'0' = 0x30`. So `c - '0'` converts the character `'7'` to the integer 7. (Used in every parser.)

7 bits suffice for ASCII (128 codes); the eighth bit was originally always 0. That spare bit became "extended ASCII" in the 80s — different countries shoved their own characters into bytes 128..255. **Disaster.** Hundreds of incompatible encodings.

Unicode — one number per character (forever)

Unicode is a giant table assigning a unique number (called a **code point**) to every character in every script in the world (and emoji).

```
U+0041   LATIN CAPITAL LETTER A         (= 0x41, same as ASCII for the first 128)
U+00E9   é (e with acute)
U+4E2D   中
U+1F600  😀
```

Code points go up to ~1.1 million. So you can't fit one in a byte. You need an **encoding** — how to turn a code point into bytes.

UTF-8 — the only encoding you should use

UTF-8 is the dominant encoding everywhere except some Windows internals. Use it. Always.

Rules:

- ASCII code points (0..127) → one byte, identical to ASCII. **UTF-8 is a strict superset of ASCII.**
- Code points 128..2047 → two bytes.
- Code points 2048..65535 → three bytes.
- Code points 65536..1.1M → four bytes.

The bit patterns:

```
Code points         UTF-8 bytes (binary)
0x0000 .. 0x007F    0xxxxxxx
0x0080 .. 0x07FF    110xxxxx 10xxxxxx
0x0800 .. 0xFFFF    1110xxxx 10xxxxxx 10xxxxxx
0x10000 .. 0x10FFFF 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
```

Properties:

- The first byte's high bits tell you how long the sequence is.
- Continuation bytes always start with `10`.
- ASCII files are valid UTF-8 byte for byte.
- You can find character boundaries by skipping any byte starting with `10`.

So `é` (U+00E9) is `0xC3 0xA9` in UTF-8 — two bytes. If software treats it as Latin-1 (a one-byte-per-char encoding), it shows as "Ã©". This is the famous **mojibake**.

UTF-16 (heads up)

UTF-16 uses 2-byte code units. Used by Windows internally, by Java, by JavaScript strings. Variable-length when characters need 4 bytes (surrogate pairs). **Avoid in Linux/kernel work.** Just be aware it exists when you parse Windows files.

C strings and Unicode

`char *` in C is a *byte* string, NUL-terminated. UTF-8 fits perfectly here — `strlen` returns byte count (not character count!), but otherwise byte routines mostly work.

```
char s[] = "café";   // in UTF-8: 'c' 'a' 'f' 0xC3 0xA9 0x00
strlen(s)            // 5 — bytes, not characters!
```

If you need actual character counts, you need a Unicode-aware library (ICU, glib, etc.). For driver work you almost never deal with non-ASCII text.

`wchar_t` and `wchar*` are an old C idea (wide chars) — avoid them in new code.

Byte-order mark (BOM)

Some Windows tools prepend `0xEF 0xBB 0xBF` (the UTF-8 BOM) to text files. Linux tools mostly ignore it; some are confused. If a script fails with "syntax error" on the very first line, check for a BOM:

```
hexdump -C -n 4 myscript.sh
# 00000000  ef bb bf 23 ...   ← BOM, then '#'
```

Strip with:

```
sed -i '1s/^\xef\xbb\xbf//' myscript.sh
```

Newlines: LF vs CRLF

Linux uses `\n` (one byte: `0x0A`). Windows uses `\r\n` (two bytes: `0x0D 0x0A`). Mac classic used `\r`.

Symptoms of mixed line endings:

- Bash script fails with `^M: command not found` or `bad interpreter: ... ^M`.
- Text file shows `^M` at end of lines in `vim`.
- `git diff` shows whole-file changes.

Fix:

```
dos2unix file.sh                # convert CRLF → LF
sed -i 's/\r$//' file.sh        # same, no install needed
```

In `git`, set `core.autocrlf=input` (Linux) or use `.gitattributes` with `* text=auto eol=lf` to enforce.

Locales — the `LANG` and `LC_*` variables

The shell, libc, and most tools look at environment variables to decide:

- which language messages should be in,
- how dates are formatted,
- how to sort strings (collation),
- which encoding to assume for stdin/stdout.

```
echo $LANG          # e.g., en_US.UTF-8
locale              # show all LC_* settings
LC_ALL=C make       # force "C" locale — pure ASCII, English errors, fast sort
```

Always set `LC_ALL=C` in scripts that parse output. Otherwise a French developer's `ls -l` may say "20 oct" instead of "20 Oct" and your parser breaks.

In the kernel

Kernel code is plain ASCII almost always. The only places encoding matters:

- file system code (paths can be UTF-8 or whatever userspace passes),
- EDID parsing (display info, sometimes UCS-2),
- Bluetooth / network protocols with text fields.

You generally don't need to think about Unicode in driver code. Just don't paste a Unicode quote (`" "`) into a `printk` by accident — your editor's autocorrect can do this.

Try it

1. What's `'A' - '0'` in C? (Answer: 0x11 = 17. Useful in hex-parsing.)
2. What's `0xC3 0xA9` as UTF-8? (Answer: `é`.)
3. Open a file with `hexdump -C file.txt | head` — find the bytes for one character of your choice.
4. Make a 1-line shell script that fails because of a CRLF line ending. Fix it with `dos2unix`.
5. Why does `LC_ALL=C sort` differ from `sort` on a list with accented characters?

You can now stop being confused about "encoding errors." Back to whichever chapter you came from.

Chapter 1 — What is a program, a compiler, and an operating system?

Before we write a line of C, you must have a correct mental model of what happens when you run a program. If this model is wrong, every later chapter (memory, drivers, GPU command submission) will be confusing. If it is right, everything else falls into place.

1.1 The machine

A modern PC has, at minimum:

- One or more **CPUs**, each with several **cores**, each able to run one (or, with SMT, two) **hardware threads** at once. Each core has private **L1** and **L2** caches; cores share an **L3** cache.
- **DRAM** (main memory): large (GBs), slow compared to caches (~100 ns vs ~1 ns for L1).
- **MMIO devices**: GPU, NIC, NVMe, USB controller, sound. Each appears as a region of physical address space; reading/writing those addresses talks to the device.
- A **PCIe** fabric connecting the CPU to those devices.
- A **boot ROM** (UEFI/BIOS firmware) that runs on power-on.
- A **storage device** (NVMe SSD) that holds the OS and your files.

Every program ultimately runs on this machine. The job of the OS is to *give every program the illusion that it owns the machine alone*.

1.2 What a CPU actually does

A CPU is, at its core, a loop:

```
while (true) {
    instruction = MEMORY[program_counter];
    decode(instruction);
    execute(instruction);
    program_counter += instruction_size;  // or branch target
}
```

That loop runs at a few GHz. Each instruction is a few bytes of binary. The CPU has a small set of **registers** (`rax`, `rbx`, `rsp`, `rip`, etc. on x86_64; `x0` – `x30`, `sp`, `pc` on AArch64) that it operates on. To do anything with main memory it must `LOAD` a value into a register, operate on it, then `STORE` it back.

A "program" is a sequence of those instructions plus initial data. That's it. The whole stack — your editor, Chrome, Linux, amdgpu — is just very long sequences of those instructions running on cores.

1.3 What a compiler does

You don't want to write `LOAD r0, [0x4000]; ADD r0, r1, r0; STORE [0x4004], r0`. You want to write `c = a + b;`.

A **compiler** translates a high-level language (C, C++, Rust) into machine instructions. The classical pipeline:

```
source.c  ──[preprocessor]──>  expanded.c
expanded.c  ──[compiler]──>    source.s        (assembly)
source.s    ──[assembler]──>   source.o        (object file: machine code + symbols)
*.o + libs  ──[linker]──>      a.out / vmlinux (executable)
```

GCC and Clang both do all four steps. We'll use both. Try this:

```
echo 'int main(void){return 1+2;}' > t.c
gcc -O2 -S t.c -o -
```

You will see something like:

```
main:
    mov     eax, 3
    ret
```

The compiler **constant-folded** `1+2` into `3` and emitted exactly two instructions. This is your first hint that the compiler is far cleverer than any code you write. You will spend much of your career either *helping it* (by writing clear code) or *fighting it* (when it optimizes away something you wanted to keep — relevant for memory-mapped I/O).

1.4 What an operating system does

When you run `./a.out`, the OS:

1. **Reads the ELF file** from disk. It contains code, data, and metadata.
2. **Creates a process**: a new address space (a virtual memory map), a kernel data structure (`struct task_struct` in Linux), file descriptor table, etc.
3. **Maps the program's segments** into that address space (`.text` for code, `.data` for initialized globals, `.bss` for zero-initialized globals, plus a stack and heap).
4. **Sets the program counter** to the entry point and **jumps**. Now your code runs.
5. When your code wants something only the kernel can do (open a file, allocate memory from the OS, talk to a device) it issues a **system call** — on x86_64, the `syscall` instruction; on ARM64, `svc #0`. The CPU traps into kernel mode, the kernel does the work, and returns.

Two crucial mental concepts come from this:

- **User mode vs. kernel mode**: a CPU bit decides whether the currently executing code can do privileged things (talk to hardware, change page tables). Your `a.out` runs in user mode. The kernel runs in kernel mode. A driver runs in kernel mode. **Crashing in kernel mode crashes the machine.**
- **Virtual memory**: every process sees a clean, flat address space starting at low addresses. Two processes can both have a pointer `0x4000`, and they refer to *different* physical bytes. The MMU (a hardware unit in the CPU) translates virtual to physical on every memory access using **page tables** maintained by the kernel.

We will return to both ideas many times.

1.5 Where a device driver lives

A **device driver** is code that runs *in kernel mode* and bridges the gap between the OS and a piece of hardware. Examples:

- `e1000e.ko` — Intel gigabit NIC driver.
- `nvme.ko` — NVMe SSD driver.
- `amdgpu.ko` — AMD GPU driver. ~800k lines of C.

Drivers don't usually run as their own process. They're modules linked into the kernel. They expose interfaces to userspace (files in `/dev`, `ioctl`s, `sysfs` attributes, `mmap`able regions) and they talk to the hardware via MMIO, DMA, and interrupts.

Diagram:

```
┌──────────────────────────────────────┐
│      User process (your app)         │
│   reads /dev/dri/card0, calls ioctl  │
└──────────────────────────────────────┘
              │  syscall
              ▼
┌──────────────────────────────────────┐
│           Linux kernel               │
│   ┌──────────────────────────────┐   │
│   │ VFS / DRM core / TTM         │   │
│   │ amdgpu driver (this is us)   │   │
│   └──────────────────────────────┘   │
└──────────────────────────────────────┘
              │  MMIO / DMA / IRQ
              ▼
┌──────────────────────────────────────┐
│           AMD GPU                    │
└──────────────────────────────────────┘
```

Everything in this book is preparation to confidently write and modify the middle box.

1.6 The languages of the trade

- **C** — the language of the Linux kernel and of every driver in `drivers/gpu/drm/amd/amdgpu/`. **Required.**
- **C++** — used in userspace (Mesa, ROCm runtime, ROCclr, HIP). **Required for AMD GPU work.**
- **Assembly** — you don't write much, but you must read x86_64 and AArch64 assembly when debugging.
- **Python** — kernel build scripts, test harnesses, ROCm tools.
- **Rust** — being introduced into the kernel; learn it after you've mastered C.
- **GLSL / SPIR-V / GCN / RDNA ISA** — GPU shader languages and AMD's GPU instruction sets. Read the ISA manuals once you reach Part VII.

Don't try to learn all of these at once. C → kernel → C++ → GPU → Rust is a sane order, and it's the order this book follows.

1.7 Mental model checklist

Before you turn the page, answer aloud:

1. What is the difference between a process and a thread?

2. What does the CPU do at every clock tick?
3. What is the difference between virtual and physical memory?
4. Why can’t a normal program write to a register on the GPU directly?
5. Why does writing to a hardware register require special care from the compiler?

If any answer is fuzzy, re-read the matching section. The whole book sits on top of these ideas.

Next: Chapter 2 — Your first C programs and the toolchain.

Chapter 2 — The toolchain and your first C programs

We learn C the way professionals use it: by understanding the toolchain. If you can't read a linker error or a `-Wall` warning, you can't write driver code.

2.1 The five tools you must own

Tool	What it does	Why you care
`gcc` / `clang`	C compiler	Builds the kernel and every driver.
`make`	Build system	Linux kernel uses Kbuild, a Make extension.
`gdb`	Debugger	Step through code, inspect memory, diagnose crashes.
`objdump` / `readelf` / `nm`	Inspect binaries	Read assembly of your code, find symbols.
`git`	Source control	The kernel and Mesa live in git; you must be fluent.

Install on Ubuntu:

```
sudo apt install build-essential gdb clang lld llvm \
                 binutils git valgrind strace ltrace
```

2.2 Hello, world

Create `hello.c`:

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, world\n");
    return 0;
}
```

Compile and run:

```
gcc -Wall -Wextra -O2 -g -o hello hello.c
./hello
```

You always want `-Wall -Wextra` (warnings on) and `-g` (debug info). `-O2` is what production code uses; do not get used to `-O0` or you'll write code the optimizer breaks.

Decoding what just happened

- `#include <stdio.h>` is a **preprocessor** directive. It textually pastes the contents of `stdio.h` into your file.
- `int main(void)` is the **entry point** the runtime calls. Returning `0` means success; non-zero means error.
- `printf` is a **library function** — it lives in `libc.so.6`, dynamically loaded at runtime.
- The actual entry point is not `main`. It's `_start` (provided by `crt1.o`), which sets up argv/envp/stack canaries, calls `__libc_start_main`, which calls `main`.

To see all four toolchain steps individually:

```
gcc -E hello.c -o hello.i      # Preprocess
gcc -S hello.i -o hello.s      # Compile
gcc -c hello.s -o hello.o      # Assemble
gcc hello.o    -o hello        # Link
```

Inspect the object file:

```
nm hello.o            # symbols (T = text/code, U = undefined, etc.)
objdump -d hello.o    # disassemble
readelf -a hello.o    # ELF metadata
```

A driver engineer must be able to look at `objdump -d driver.ko` and figure out what's wrong. Practice this on hello world.

2.3 The structure of a real C program

A "real" program has multiple files. Here is a tiny library plus a main driver:

```
/* math_util.h */
#ifndef MATH_UTIL_H
#define MATH_UTIL_H

int add(int a, int b);
int mul(int a, int b);

#endif
```

```
/* math_util.c */
#include "math_util.h"

int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }
```

```
/* main.c */
#include <stdio.h>
#include "math_util.h"

int main(void)
{
    printf("2+3 = %d\n", add(2, 3));
    printf("2*3 = %d\n", mul(2, 3));
    return 0;
}
```

Build:

```
gcc -Wall -Wextra -O2 -g -c math_util.c    # → math_util.o
gcc -Wall -Wextra -O2 -g -c main.c         # → main.o
gcc main.o math_util.o -o app
./app
```

Three points to internalize:

1. **Header files declare; .c files define.** A function declared in a header has its body in some .c file. The linker matches them up.
2. **Every .c file is compiled independently** into a `.o`. The compiler doesn't see across files (that's why headers exist). This is called the **translation unit** model.
3. **The linker** stitches `.o` files plus libraries into the final executable. Linker errors mean *missing or duplicated definitions across .o files*.

2.4 A first Makefile

```
CC      := gcc
CFLAGS  := -Wall -Wextra -O2 -g
OBJS    := main.o math_util.o

app: $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c math_util.h
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f *.o app
```

Save as `Makefile`, then `make`. The kernel build system (Kbuild) is an elaborate version of this.

2.5 Reading compiler warnings is a skill

This compiles silently with `-O2` but is dead wrong:

```
#include <stdio.h>
int main(void) {
    int x;
    printf("%d\n", x);   /* x is uninitialized */
    return 0;
}
```

Add `-Wall -Wextra -Wuninitialized -O2`:

```
warning: 'x' is used uninitialized [-Wuninitialized]
```

In a driver, an uninitialized variable can leak kernel memory to userspace. **Treat warnings as errors.** Add `-Werror` to your `CFLAGS`. Real kernel code is built with hundreds of warning flags enabled.

2.6 Debugging your first segfault

```
#include <stdio.h>
int main(void) {
    int *p = NULL;
    *p = 42;       /* boom */
```

```
        return 0;
    }
```

Compile with `-g`, run, see `Segmentation fault`. Now:

```
    gcc -g segfault.c -o segfault
    gdb ./segfault
    (gdb) run
    (gdb) bt          # backtrace: where did it crash
    (gdb) frame 0
    (gdb) print p     # $1 = (int *) 0x0
```

If you can't reach for gdb the second something crashes, you cannot work on drivers. Drivers crash in obscure ways and gdb (or `crash` / `kdb` / `pstore`) is the only way through.

2.7 The five rules of writing C from day one

1. **Initialize every variable.** `int x = 0;` not `int x;`.
2. **Check every return code.** `malloc`, `fopen`, `read`, `ioctl` — all of them can fail.
3. **Free everything you allocate, exactly once.** This is harder than it sounds.
4. **Run with sanitizers.** `gcc -fsanitize=address,undefined -g` in development.
5. **Compile with `-Wall -Wextra -Werror`.** No exceptions.

Internalize these now. They are the difference between a junior who ships and a junior whose patches get rejected.

2.8 Exercises

1. Build `hello.c` four times: with `-O0`, `-O1`, `-O2`, `-O3`. Compare the assembly using `objdump -d hello`. Notice what the optimizer does.
2. Deliberately break the program (typo a function name, omit a return type, leave out `;`) and read each error message until you can predict what each error corresponds to.
3. Write a C program that allocates 1 MB with `malloc`, fills it with the pattern `0xDEADBEEF`, prints the first and last 8 bytes as hex, and frees it. Run it under `valgrind` to confirm zero leaks.

Next: Chapter 3 — Types, integer promotion, and undefined behavior.

Chapter 3 — Types, integer promotion, and undefined behavior

C's type system looks simple. It is not. It is the source of more security bugs and more wrong driver code than any other feature of the language. We must tame it now or pay later.

3.1 The fundamental types

```
char        /* 1 byte, signedness implementation-defined */
signed char /* 1 byte, signed */
unsigned char
short            /* >= 16 bits */
int              /* >= 16 bits, almost always 32 on Linux */
long             /* 32 bits on 32-bit Linux, 64 bits on 64-bit Linux */
long long        /* >= 64 bits */
float, double, long double
```

The C standard promises minimum widths, **not exact widths**. The kernel and any portable code must use the fixed-width types from `<stdint.h>` (userspace) or `<linux/types.h>` (kernel):

```
uint8_t  u8;
uint16_t u16;
uint32_t u32;
uint64_t u64;
int8_t   s8;
/* … and signed variants … */
```

Kernel code uses `u8`, `u16`, `u32`, `u64`, `s32`, etc. — typedefs in `linux/types.h`. Get used to those.

Why this matters for driver code: hardware registers are *fixed width*. A 32-bit register is `u32`, period. Using `int` invites trouble on a future architecture or compiler.

3.2 Sizes and `sizeof`

```
#include <stdio.h>
#include <stdint.h>

int main(void) {
    printf("char  %zu\n", sizeof(char));
    printf("short %zu\n", sizeof(short));
    printf("int   %zu\n", sizeof(int));
    printf("long  %zu\n", sizeof(long));
    printf("ll    %zu\n", sizeof(long long));
    printf("ptr   %zu\n", sizeof(void *));
    printf("u32   %zu\n", sizeof(uint32_t));
    return 0;
}
```

On a 64-bit Linux machine, you'll see `1, 2, 4, 8, 8, 8, 4`. On a 32-bit ARM Cortex-M, `1, 2, 4, 4, 8, 4, 4`. The pointer width and the `long` width track the architecture's bit width on Unix-likes (the **LP64** model). Windows uses **LLP64** where `long` is 32 bits even on 64-bit Windows. Real driver code must not assume.

`sizeof` is an *operator*, not a function. It evaluates at compile time for fixed types. `sizeof(int)` does not compile to a function call.

3.3 Implicit conversions and integer promotion

Here be dragons. When you mix types, the compiler silently converts. The rules:

- Any operand smaller than `int` is **promoted** to `int` (or `unsigned int` if the value won't fit in `int`).
- If both operands are the same after promotion, you're fine.
- If they differ, the *lower-rank* one is converted to the higher-rank one. Mixing signed/unsigned: signed gets converted to unsigned. **This is a frequent bug source.**

Example:

```
unsigned int u = 1;
int s = -1;
if (s < u) {
    /* never executed */
} else {
    puts("WAT");
}
```

Why? `s` is converted to `unsigned int`, becoming `0xFFFFFFFF`, which is greater than `1`. This kind of bug has crashed kernels.

Another classic:

```
size_t len = something_returning_size_t();
int i;
for (i = 0; i < len - 1; i++) { ... }
```

If `len == 0`, then `len - 1` underflows to `SIZE_MAX`, and `i < SIZE_MAX` is true forever. **Always compare loop variables in the same type as the limit.**

3.4 Signed integer overflow is undefined behavior

```
int x = INT_MAX;
x = x + 1;     /* UNDEFINED BEHAVIOR */
```

UB means the standard puts no constraint on what happens. The compiler is allowed to assume it never happens and optimize accordingly. Real example:

```
/* before: */ if (x + 1 < x) overflow();
/* compiler: x+1 < x is impossible if no UB, so delete the branch */
```

This famously bit a TCP sequence-number check. Use `unsigned` for arithmetic that might wrap (unsigned overflow is well-defined: it wraps modulo 2^n). Or use the GCC builtins:

```
int r;
if (__builtin_add_overflow(a, b, &r)) {
    /* overflow */
}
```

The kernel has `check_add_overflow()`, `array_size()`, `struct_size()`, and friends in `<linux/overflow.h>` for exactly this purpose.

3.5 Other undefined behaviors you must memorize

These are the classics. Each has shipped at least one CVE.

UB	Example	What can happen
Signed overflow	`INT_MAX + 1`	Anything; checks deleted
Use after free	`free(p); *p = 0;`	Memory corruption
Double free	`free(p); free(p);`	Heap corruption
Read of uninitialized	`int x; printf("%d", x);`	Garbage; info leak
Out-of-bounds read/write	`int a[4]; a[5] = 0;`	Stack/heap overflow
Null pointer deref	`*(int *)0 = 1;`	Crash, or silent in some kernels
Strict aliasing violation	Reading `int` via `float *`	Wrong values
Misaligned access	`*(uint32_t*)(buf+1)` on ARM	SIGBUS
Data race	Two threads write same var without sync	Anything
Modifying string literal	`char *p = "hi"; p[0]='H';`	SIGSEGV
Shift past width	`int x; x << 32`	Implementation-defined → UB

Run with `-fsanitize=undefined,address` in dev. UBSan and ASan will catch most of these.

3.6 Strict aliasing

The compiler is allowed to assume that an `int *` and a `float *` never point to the same memory. So:

```
void f(float *fp, int *ip) {
    *ip = 1;
    *fp = 0.0f;
    /* compiler may assume *ip is still 1, even though *fp may have written to it */
}
```

If you actually need to reinterpret bytes, use `memcpy`:

```
uint32_t bits;
float    f = 1.0f;
memcpy(&bits, &f, sizeof bits);
```

The compiler optimizes that to a register move. It is the *only* portable, defined way to bit-cast in C. C++ adds `std::bit_cast` (since C++20). The Linux kernel uses `READ_ONCE()` / `WRITE_ONCE()` macros that wrap volatile-cast assignments for similar reasons.

`char *` and `unsigned char *` *are* allowed to alias anything. That's why `memcpy` / `memset` take `void *`.

3.7 The `volatile` keyword

`volatile` tells the compiler "this memory may change underneath you, so don't optimize away reads/writes." It is **only** for:

- memory-mapped hardware registers,
- variables shared with signal handlers,
- variables shared with interrupt handlers in single-threaded contexts.

`volatile` is **not** a substitute for atomics or locks. It does not order accesses across CPUs. It does not make multi-threaded code correct. Re-read that sentence.

In the Linux kernel, you almost never write `volatile` directly. You use `readl()` / `writel()` for MMIO and `READ_ONCE()` / `WRITE_ONCE()` for shared variables. Both expand to volatile casts internally, but the API expresses *intent*.

3.8 The `const` keyword

`const` means "I won't modify this through this name." It is documentation for humans and an optimization hint for the compiler.

```
const int N = 1024;
void print(const char *s);  /* this fn won't modify *s */
```

Two subtle points:

```
const int *p;        /* pointer to const int — *p is const, p is not */
int * const p;       /* const pointer to int — p is const, *p is not */
const int * const p; /* both */
```

Read declarations right-to-left. The kernel uses `const` everywhere for parameters that aren't modified; learn to do the same. Putting tables of function pointers (`struct file_operations`) in `const` lets them go in read-only memory, hardening the kernel.

3.9 The `static` and `extern` keywords

- `static` at file scope (outside a function): "this name is private to this translation unit." Use this for any helper not exported.
- `static` inside a function: "this variable persists between calls." Avoid in driver code; it's per-CPU global state and a hazard.
- `extern`: "this name is defined elsewhere." Used in headers for global variables. Driver code rarely needs it.

A real-world rule of thumb in the kernel: **everything is `static` unless explicitly exported with `EXPORT_SYMBOL`.** This keeps the namespace clean and lets the compiler inline aggressively.

3.10 Type aliases for hardware

Driver headers define register layouts:

```
struct __attribute__((packed)) my_dev_regs {
    u32 ctrl;
    u32 status;
    u32 irq_mask;
    u32 _reserved;
    u32 doorbell;
};
```

`__attribute__((packed))` disables padding. Without it the compiler may insert pad bytes for alignment, breaking your map onto hardware registers. We will revisit packing in chapter 5.

3.11 Exercises

1. Write a program that computes `INT_MAX + 1` two ways: as `int` and as `unsigned`. Compile with `-O2` and look at the assembly. The signed version may be optimized away.
2. Write a function `bool safe_add(int a, int b, int *out)` using `__builtin_add_overflow`.
3. On your machine, write code that reads the bytes of a `float 1.0f` and prints them in hex. (Hint: `memcpy` to a `uint32_t` and print with `%08x`.)
4. Predict the output of:

   ```
   unsigned long size = 1;
   if (size - 2 < 1)
       puts("yes");
   ```

   then run it. Explain.
5. Read through `linux/types.h` (in `/usr/src/linux-headers-*/include/linux/types.h`) and identify every typedef. Recognize that this file is the foundation of every kernel datatype.

Next: Chapter 4 — Pointers, arrays, and the C memory model.

Chapter 4 — Pointers, arrays, and the C memory model

If you cannot draw the memory layout of a function call on a whiteboard, you cannot write driver code. Most kernel bugs are pointer bugs. Most security holes are pointer bugs. We will spend more time here than the language deserves, because the *concept* matters more than the syntax.

4.1 What a pointer is

A pointer is an **integer** that names a memory location. On a 64-bit Linux machine, every pointer is 8 bytes. On a 32-bit machine, 4 bytes. There is nothing magic about it.

```
int x = 42;
int *p = &x;     /* p holds the address of x */
*p = 99;         /* dereference: write through p */
printf("%d\n", x);   /* 99 */
```

Mental picture:

```
Address      Memory
────────     ──────────

0x7fffe000   42 → 99      (x)
0x7fffe008   0x7fffe000   (p)
```

`&x` produces the address. `*p` accesses the memory at that address. `p` itself is a variable (it lives at `0x7fffe008`), and `&p` would be `0x7fffe008`.

4.2 The four memory regions

A typical Unix process has these memory regions, growing in opposite directions:

```
high addresses
┌──────────────────────────┐
│       Stack              │  ← grows down: function locals, return addrs
│       (~8 MB default)    │
├──────────────────────────┤
│          ↓               │
│                          │
│          ↑               │
├──────────────────────────┤
│       Heap               │  ← grows up: malloc()'d memory
├──────────────────────────┤
│       BSS                │  ← zero-initialized globals
├──────────────────────────┤
│       Data               │  ← initialized globals
├──────────────────────────┤
│       Text               │  ← code, read-only
└──────────────────────────┘
low addresses
```

Each region has different lifetimes and permissions:

- **Text** is read-execute. Writing to it is UB and on Linux causes SIGSEGV.

- **Data/BSS** are read-write, persist for the lifetime of the program.
- **Heap** is what you `malloc()`. You're responsible for freeing.
- **Stack** is automatic. Locals appear when a function is entered and **disappear** when it returns.

A pointer to a stack local *outlives* the local only at your peril:

```
int *bad(void) {
    int x = 5;
    return &x;     /* DANGLING POINTER */
}
```

This compiles. The compiler may even warn. Calling `*bad()` is UB. The stack frame is reused immediately by the next function call.

In the kernel, the same applies: stack is small (8 KB or 16 KB *total* per kernel thread on x86_64). You cannot `int big[2048]` on the kernel stack — it will silently corrupt other CPU state.

4.3 Pointer arithmetic

Pointer arithmetic is in **units of the pointee type**:

```
int a[4] = {10, 20, 30, 40};
int *p = a;          /* p points to a[0] */
printf("%d\n", *p);      /* 10 */
printf("%d\n", *(p+1));  /* 20 — moved 4 bytes */
printf("%d\n", p[2]);    /* 30 — same as *(p+2) */
```

`p + 1` advances by `sizeof(*p)` bytes. This is why `void *` cannot be incremented in standard C (you don't know the size of `void`); GCC allows it as an extension.

The corollary: `a[i]` is *literally* `*(a + i)`. So `a[i] == i[a]` is true (don't do this, but be aware C array indexing is symmetric).

4.4 Arrays decay to pointers (and when they don't)

This is the most-asked C interview question, after "explain volatile."

```
void f(int a[]);     /* identical to: void f(int *a); */
void f(int a[10]);   /* identical too! the 10 is ignored */

int main(void) {
    int x[10];
    f(x);            /* x decays to int * */
    printf("%zu\n", sizeof(x));  /* 40 */

    int *p = x;
    printf("%zu\n", sizeof(p));  /* 8 */
}
```

When you pass an array to a function, you pass a pointer. The function does **not** know the array length. You must pass it explicitly:

```
void sum(const int *a, size_t n) {
    int s = 0;
    for (size_t i = 0; i < n; i++) s += a[i];
```

```
        /* … */
    }
```

The kernel pattern: every buffer is a `(ptr, len)` pair. `struct iov_iter`, `struct sg_table`, `struct scatterlist`, every DMA API — all `(ptr, len)`. Hardcode this into your fingers.

Inside `sizeof`, `&`, and the very first expression of a string literal initializer (`char s[] = "hi";`), arrays do **not** decay. Everywhere else, they do.

4.5 Strings

A C string is just a `char` array terminated by a `'\0'` byte. There is no string type. There is no length prefix.

```
char s[] = "hello";   /* 6 bytes: 'h','e','l','l','o','\0' */
char *p  = "hello";   /* p points to a string literal in read-only memory */
```

`p[0] = 'H'` is UB. `s[0] = 'H'` is fine.

`<string.h>` gives you:

```
size_t strlen(const char *s);
char  *strcpy(char *dst, const char *src);   /* DANGEROUS: no length */
char  *strncpy(char *dst, const char *src, size_t n); /* may not NUL-terminate */
size_t strlcpy(char *dst, const char *src, size_t n);  /* GNU/BSD ext: safer */
int    strcmp(const char *a, const char *b);
char  *memcpy(void *dst, const void *src, size_t n);
char  *memset(void *dst, int c, size_t n);
```

In the kernel: never `strcpy`. Use `strscpy()` (kernel-internal). Always check lengths. Most CVEs in C strings come from buffer overflows.

4.6 The two-headed sword: `char *` and `void *`

- `char *` (and `unsigned char *`) is allowed to alias *any* type. So you can `memcpy(buf, &my_struct, sizeof my_struct)`.
- `void *` is the generic pointer type. Every `T *` implicitly converts to `void *` and back without a cast.

```
void *p = malloc(sizeof(int));
int *ip = p;       /* OK in C; in C++ you'd need a cast */
```

In kernel code you'll see:

```
struct foo *f = kzalloc(sizeof *f, GFP_KERNEL);
```

`kzalloc` returns `void *`. We assign to `struct foo *`. No cast needed in C. Notice `sizeof *f` — this is the kernel idiom: you don't repeat the type name, so refactoring is safe.

4.7 Arrays of pointers, pointers to pointers

```
char *names[] = { "Alice", "Bob", "Carol" };  /* array of 3 pointers */
char **np = names;
```

```
        printf("%s\n", *np);       /* "Alice" */
        printf("%s\n", *(np+1));   /* "Bob" */
```

Argv is the canonical example:

```
        int main(int argc, char **argv);
```

`argv` is a pointer to the first element of an array of `char *`. Each element points to a NUL-terminated string. `argv[argc]` is `NULL`.

4.8 Function pointers

```
        int add(int a, int b) { return a + b; }
        int (*fp)(int, int) = add;
        printf("%d\n", fp(2, 3));    /* 5 */
```

Read it as: "fp is a pointer to a function taking (int,int) returning int." The parentheses around `*fp` are required, otherwise it would be "function returning int*."

The kernel uses function pointers everywhere — `struct file_operations`, `struct drm_driver`, `struct pci_driver` — because that's how polymorphism works in C. We dedicate Chapter 9 to this.

4.9 The `restrict` keyword

`restrict` is a C99 promise: "for the lifetime of this pointer, the only way I will access this object is through this pointer." It lets the compiler optimize as if no aliasing exists.

```
        void copy(int * restrict dst, const int * restrict src, size_t n);
```

If `dst` and `src` overlap, the result is UB. `memcpy` is `restrict`; `memmove` is not. The kernel uses `__restrict__` (GCC spelling) sparingly, but in performance-critical math (BLAS, GPU kernel code) `restrict` is essential.

4.10 Common pointer bugs and how to find them

Bug	Diagnostic
Use after free	ASan: `heap-use-after-free`
Double free	ASan: `attempting double-free`
Buffer overflow	ASan: `heap-buffer-overflow`
Stack buffer overflow	ASan: `stack-buffer-overflow`
Memory leak	ASan: `LeakSanitizer` summary
Uninitialized read	MSan / Valgrind: `Conditional jump … depends on uninitialised value`
Dangling stack ptr	`-Wreturn-stack-address` (clang)
Null deref	gdb: SIGSEGV, address `0x0`

Build everything during dev with:

```
        gcc -Wall -Wextra -Werror -O1 -g \
            -fsanitize=address,undefined \
            -fno-omit-frame-pointer
```

You will catch 95% of pointer bugs before they ship.

4.11 The C memory model in one paragraph

Memory is a flat array of bytes addressed by integers. Pointers are typed integers. Allocations come from the stack (automatic, scope-bound), the heap (manual, `malloc` / `free`), or static storage (globals/locals declared `static`). The compiler is free to reorder reads/writes within a single thread as long as the *as-if* rule holds. Across threads or between CPU and device, you need barriers (Chapter 25). Across processes, memory is *not* shared except via explicit `mmap(MAP_SHARED)` or shm primitives.

4.12 Exercises

1. Implement `my_strlen`, `my_strcpy`, `my_strcmp`, `my_memcpy`, `my_memset` from scratch. Test each. This is a standard interview warm-up.
2. Implement a function `void reverse(char *s)` that reverses an in-place string. Don't allocate.
3. Implement `int parse_int(const char *s, int *out)` that returns 0 on success, -1 on failure. Detect overflow.
4. Build a `char **split(const char *s, char delim)` that returns a NULL-terminated array of newly-allocated strings. Write the matching `void free_split(char **)`.
5. Run all of the above under ASan and Valgrind. Fix every leak and overflow.

Next: Chapter 5 — Structs, unions, bitfields, packing, alignment.

Chapter 5 — Structs, unions, bitfields, packing, alignment

Structs are how C describes data. In a driver, a struct often **maps directly onto hardware registers**, a packet on the wire, or a shared memory layout between kernel and userspace. The compiler may add padding, reorder bitfields, choose an endianness — and getting any of those wrong produces silent, terrifying bugs.

5.1 The basic struct

```
struct point {
    int x;
    int y;
};

struct point p = { .x = 1, .y = 2 };  /* designated initializer (C99) */
```

`sizeof(struct point) == 8` on most platforms. Members are stored in the order written. The compiler may insert **padding** to satisfy alignment.

5.2 Alignment

Every type has an *alignment* — the multiple of its address that it must live on. Common alignments on x86_64:

Type	Size	Alignment
`char`	1	1
`short`	2	2
`int`	4	4
`long`, `void *`	8	8
`double`	8	8

A struct's alignment is the *max* of its members'. Members are placed at the smallest offset ≥ their alignment.

```
struct gap {
    char  a;     /* offset 0 */
    /* 3 bytes pad */
    int   b;     /* offset 4 */
    char  c;     /* offset 8 */
    /* 3 bytes pad to make sizeof a multiple of 4 */
};                /* sizeof = 12 */
```

If you reorder:

```
struct nogap {
    int  b;      /* 0 */
    char a;      /* 4 */
```

```
        char c;      /* 5 */
        /* 2 bytes tail pad */
    };               /* sizeof = 8 */
```

Order from largest to smallest to minimize padding. In hot data structures (per-CPU vars, packet headers) this affects cache footprint.

Print sizes of any struct with:

```
    printf("%zu\n", sizeof(struct gap));
```

Or use `pahole` (from `dwarves` package) to dump padding from DWARF:

```
    pahole my_program | less
```

This is a tool you will use often when reading kernel structs.

5.3 Why misaligned access matters

On x86, misaligned loads usually work but are slower. On many ARM cores, on RISC-V, on older SPARC, a misaligned load *traps* (`SIGBUS` in userspace, an oops in the kernel). Driver code runs on every architecture Linux supports — so misaligned access is forbidden.

```
    char buf[8];
    *(uint32_t *)(buf + 1) = 0;  /* BAD: address may be unaligned */
```

Use `memcpy` :

```
    uint32_t v = 0;
    memcpy(buf + 1, &v, 4);
```

The compiler frequently generates the same single instruction on x86 and a safe sequence on ARM. This is the right idiom.

5.4 The `__attribute__((packed))` directive

For hardware register banks, on-the-wire packets, or anything that *must* match an external layout exactly, use `packed` :

```
    struct __attribute__((packed)) ethhdr {
        uint8_t  dst[6];
        uint8_t  src[6];
        uint16_t ethertype;     /* big-endian on the wire */
    };                          /* sizeof == 14, no padding */
```

A packed struct has alignment 1 and may produce slow access on strict-alignment CPUs. The kernel macro `__packed` is just `__attribute__((packed))` .

Rule: use packed only when you must match an external format. Don't pack everything.

5.5 Unions

A `union` overlays multiple types in the same storage. `sizeof(union)` is the size of the largest member. Reading a member that wasn't last written is implementation-defined / UB depending on the rule.

```
union ipv4_or_bytes {
    uint32_t addr;
    uint8_t  octets[4];
};

union ipv4_or_bytes u;
u.addr = 0x0100007F;            /* 127.0.0.1 in little-endian */
printf("%u\n", u.octets[0]);    /* 127 on x86 (LE) */
```

This is the classic endianness probe. It's also used to overlay register layouts: a 32-bit register that has different bit fields depending on mode.

The kernel uses unions extensively in `struct sk_buff`, `struct page`, and many DRM IOCTL structs.

5.6 Bitfields

Bitfields let you name individual bits inside an integer:

```
struct ctrl_reg {
    uint32_t enable      : 1;
    uint32_t mode        : 3;
    uint32_t _reserved   : 4;
    uint32_t length      : 16;
    uint32_t crc         : 8;
};
```

This compiles to integer mask/shift code. **But** the order of bits within the underlying word and the size of "underlying word" are implementation-defined. So bitfields are *not* portable for hardware register layouts on multi-arch code.

The portable kernel idiom for hardware registers is explicit shifts and masks:

```
#define CTRL_ENABLE     BIT(0)
#define CTRL_MODE_MASK  GENMASK(3, 1)
#define CTRL_MODE_SHIFT 1
#define CTRL_LEN_MASK   GENMASK(23, 8)
#define CTRL_LEN_SHIFT  8

u32 v = readl(dev->ctrl);
u32 mode = (v & CTRL_MODE_MASK) >> CTRL_MODE_SHIFT;
v &= ~CTRL_LEN_MASK;
v |= (length << CTRL_LEN_SHIFT) & CTRL_LEN_MASK;
writel(v, dev->ctrl);
```

Memorize this pattern. Bitfields are nice for tightly-controlled IPC where the compiler is fixed, but for hardware registers, mask/shift is the rule.

5.7 Endianness

A 32-bit integer `0x12345678` lives in memory as four bytes. Two orderings exist:

- **Little-endian** (x86, ARM in default mode): bytes `78 56 34 12`. Least significant byte at lowest address.
- **Big-endian** (most network protocols, some PowerPC, MIPS): bytes `12 34 56 78`.

The kernel provides typed endian conversions:

```
__le32 v_on_disk;             /* little-endian */
__be32 v_on_wire;             /* big-endian */
u32 v = le32_to_cpu(v_on_disk);     /* to CPU native */
__le32 d = cpu_to_le32(v);          /* to disk format */
```

Sparse (a kernel static analyzer) flags type mismatches: assigning `__be32` to `__le32` is an error. **Use these typedefs** when describing on-wire/on-disk/over-PCIe formats. They prevent endian bugs from compiling.

GPU registers on AMD are little-endian; PCIe config space is little-endian; many network protocols are big-endian. You must keep track.

5.8 The `container_of` macro

This is the single most important macro in the Linux kernel. Given a pointer to a *member* of a struct, it returns a pointer to the *containing* struct:

```
#define offsetof(TYPE, MEMBER) ((size_t) &((TYPE *)0)->MEMBER)

#define container_of(ptr, type, member) ({                  \
    const typeof(((type *)0)->member) *__mptr = (ptr);      \
    (type *)((char *)__mptr - offsetof(type, member)); })
```

Why? Because the kernel embeds intrusive list/tree nodes inside structs:

```
struct task {
    int id;
    struct list_head node;   /* embedded list node */
    char name[32];
};

struct list_head *p = ... ;  /* I have a node pointer */
struct task *t = container_of(p, struct task, node);  /* recover the task */
```

This is how `list_for_each_entry`, `rb_entry`, and dozens of macros work. **Drill this until it's instinctive.**

5.9 Forward declarations and opaque types

You can declare a struct without defining it. This is *opaque*:

```
/* in foo.h */
struct foo;                         /* opaque */
struct foo *foo_create(void);
void        foo_destroy(struct foo *);
int         foo_do_thing(struct foo *, int);

/* in foo.c */
```

```
struct foo {
    int internal_state;
    /* … */
};
```

Users of the header can hold a `struct foo *` but cannot peek inside. This is **C's encapsulation mechanism**. The kernel uses it heavily: a userspace driver writer holding a `struct drm_device *` cannot see its members; they go through API functions.

5.10 Anonymous structs and unions

C11 (and a GCC extension before that) allows nested anonymous types:

```
struct frame {
    int seq;
    union {
        struct { uint8_t a, b; };  /* anonymous struct */
        uint16_t both;
    };
};

struct frame f = { .seq = 0, .a = 1, .b = 2 };
```

You access `.a`, `.b`, `.both` directly without an extra qualifier. The kernel uses anonymous unions in `struct page` and `struct dma_fence_cb`.

5.11 Real-world: a hardware-register struct

A simplified UART:

```
struct __packed uart_regs {
    u32 data;       /* 0x00: data register */
    u32 status;     /* 0x04: status */
#define UART_STATUS_TX_EMPTY   BIT(0)
#define UART_STATUS_RX_FULL    BIT(1)
#define UART_STATUS_FRAME_ERR  BIT(2)
    u32 ctrl;       /* 0x08: control */
#define UART_CTRL_ENABLE       BIT(0)
#define UART_CTRL_BAUD_MASK    GENMASK(15, 8)
    u32 _reserved[1];
    u32 fifo_level; /* 0x10: fifo occupancy */
};

struct uart_dev {
    struct uart_regs __iomem *regs;
    /* … */
};

static int uart_putc(struct uart_dev *u, char c)
{
    while (!(readl(&u->regs->status) & UART_STATUS_TX_EMPTY))
        cpu_relax();
    writel(c, &u->regs->data);
    return 0;
}
```

Note:

- `__iomem` annotates that this pointer is into device memory, not normal RAM. Sparse will warn if you dereference it directly.
- `readl` / `writel` are 32-bit I/O accessors with appropriate barriers.

- The struct *is* the register layout. Any padding/reordering would point at the wrong register.

5.12 Exercises

1. Reorder this struct to minimize size:

```
struct s { char a; double b; char c; int d; char e; };
```

2. Write a `struct ipv4_packet` (packed) matching the IP header fields. Add the bit shifts for `version` and `IHL` since they share a byte.
3. Write `container_of` from memory and use it: define `struct task` containing an `int id;` and a `struct list_head node;`, take a `node *`, recover the `task *`, print the id.
4. Use `pahole` (or, if unavailable, hand-compute) on `struct task_struct` from your installed kernel headers; identify the largest source of padding.
5. Write a 16-bit and 32-bit `swap_bytes` function and use it instead of compiler builtins. Then read `<linux/swab.h>` to see how the kernel does it.

Next: Chapter 6 — Bit manipulation: the language of hardware.

Chapter 6 — Bit manipulation: the language of hardware

Hardware speaks in bits. A control register is a packed array of 1-bit and N-bit fields. To configure a GPU display engine you set bit 12 of register `0x6118`. To write a kernel driver, bit operations must be as natural to you as addition.

6.1 The six basic operators

```
a & b     /* AND */
a | b     /* OR  */
a ^ b     /* XOR */
~a        /* NOT */
a << n    /* shift left  by n bits */
a >> n    /* shift right by n bits */
```

Truth table for two bits `x`, `y`:

x	y	x&y	x|y	x^y	~x
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

A few rules to memorize:

- `x | (1 << n)` — set bit `n`.
- `x & ~(1 << n)` — clear bit `n`.
- `x ^ (1 << n)` — toggle bit `n`.
- `(x >> n) & 1` — read bit `n`.

Watch out: shifting by ≥ width is UB. `int x; x << 32;` is UB on 32-bit `int`. Always check.

6.2 The Linux kernel idioms

`<linux/bits.h>` provides:

```
#define BIT(nr)                 (1UL << (nr))
#define BIT_ULL(nr)             (1ULL << (nr))
#define GENMASK(h, l)           (((~0UL) << (l)) & (~0UL >> (BITS_PER_LONG-1-(h))))
```

So `BIT(3)` is `0x8`, and `GENMASK(15, 8)` is `0xFF00` — the mask for bits 8 through 15 inclusive.

Setting / clearing fields:

```
#define FIELD_SET(reg, mask, shift, val) \
    (((reg) & ~(mask)) | (((val) << (shift)) & (mask)))
```

```
u32 v = readl(addr);
v = FIELD_SET(v, GENMASK(7,0), 0, 0xAB);
writel(v, addr);
```

The kernel has cleaner primitives in `<linux/bitfield.h>`:

```
#define MY_MODE_MASK   GENMASK(3, 1)

u32 v = readl(addr);
u32 mode = FIELD_GET(MY_MODE_MASK, v);

v &= ~MY_MODE_MASK;
v |= FIELD_PREP(MY_MODE_MASK, 5);
writel(v, addr);
```

`FIELD_GET` extracts and right-shifts. `FIELD_PREP` shifts a value into the field's position. Use these in real driver code; they're checked by `BUILD_BUG_ON_NOT_POWER_OF_2()` etc.

6.3 Common bit tricks

You will see all of these in real kernel and GPU code.

Test if a number is a power of two

```
bool is_pow2(unsigned x) { return x && !(x & (x - 1)); }
```

Round up to the next multiple of `align` (when `align` is a power of two)

```
u64 round_up(u64 x, u64 align) {
    return (x + align - 1) & ~(align - 1);
}
```

The kernel's `ALIGN(x, a)` macro does exactly this. Used in DMA, page allocation, GPU buffer sizing.

Round down

```
u64 round_down(u64 x, u64 a) { return x & ~(a - 1); }
```

Population count (number of set bits)

```
unsigned popcount(uint32_t x) {
    return __builtin_popcount(x);
}
```

GCC emits the `popcnt` instruction on x86 with `-mpopcnt`, otherwise a fallback. The kernel macro is `hweight32()`.

Find first / last set

```
__builtin_ctz(x)   /* count trailing zeros — index of lowest set bit */
__builtin_clz(x)   /* count leading zeros */
```

Kernel: `__ffs()`, `__fls()`, `ffz()`, etc. Useful for: "I have a bitmap of free slots; give me one." The GPU scheduler uses these.

Bit reverse, byte swap

```
__builtin_bswap32(x)   /* swap byte order */
```

Kernel: `__swab32()`, `cpu_to_be32()`, etc.

6.4 Bitmaps

The kernel has a `<linux/bitmap.h>` API for arbitrarily-large bitmaps:

```
DECLARE_BITMAP(map, 256);   /* 256 bits, stored as 4 unsigned long */
bitmap_zero(map, 256);
set_bit(42, map);
clear_bit(42, map);
test_bit(42, map);
unsigned long n = find_first_zero_bit(map, 256);
```

`set_bit` and friends are **atomic** by default (because they're often used to coordinate state across CPUs). The non-atomic versions are `__set_bit`, `__clear_bit`. Use the atomic ones unless you know you have exclusive access.

The amdgpu driver uses bitmaps to track free VMIDs, free SDMA queues, in-flight CSAs, and many other resources.

6.5 Reading a register: full pattern

```
struct gpu_dev {
    void __iomem *mmio;
};

#define REG_CTRL        0x0040
#define CTRL_ENABLE     BIT(0)
#define CTRL_RESET      BIT(1)
#define CTRL_MODE_MASK  GENMASK(7, 4)
#define CTRL_MODE_SHIFT 4

static int gpu_set_mode(struct gpu_dev *g, u8 mode)
{
    u32 v;

    if (mode > 0xF)
        return -EINVAL;

    v  = readl(g->mmio + REG_CTRL);
    v &= ~CTRL_MODE_MASK;
    v |= (mode << CTRL_MODE_SHIFT) & CTRL_MODE_MASK;
    v |= CTRL_ENABLE;
    writel(v, g->mmio + REG_CTRL);

    /* readback to flush posted write */
    (void)readl(g->mmio + REG_CTRL);
    return 0;
}
```

Note the read-modify-write pattern. **Never write a register without reading first** unless the entire register's meaning is well-understood — you would clobber bits the hardware uses. The "readback to flush" is critical on PCIe: writes are *posted* and won't necessarily reach the device before the function returns.

We will spend an entire chapter on MMIO ordering. For now, internalize the RMW pattern.

6.6 Bit hacks you should at least recognize

```
/* Swap two ints without a temp (don't actually do this). */
a ^= b; b ^= a; a ^= b;

/* abs without branch (assuming arithmetic shift). */
int mask = x >> 31;
int abs = (x + mask) ^ mask;

/* min(x, y) without branch. */
int min = y ^ ((x ^ y) & -(x < y));

/* Round to next power of two (32-bit). */
x--;
x |= x >> 1; x |= x >> 2; x |= x >> 4;
x |= x >> 8; x |= x >> 16;
x++;
```

You will not write these in production C — the compiler does them. But you must read them in assembly and in tightly-tuned graphics math.

6.7 Exercises

1. Implement `set`, `clear`, `flip`, `test` on a 64-bit `uint64_t` bitmap.
2. Implement `is_pow2`, `next_pow2`, `log2_floor` for `uint32_t`. Compare your `popcount` to `__builtin_popcount`.
3. Implement `align_up(x, a)` and `align_down(x, a)` for *non-power-of-two* `a`. Then a power-of-two version. Compare assembly with `gcc -O2 -S`.
4. Pack three values `(mode:4, channel:3, enabled:1)` into a `u8`. Then unpack. Then do the same with explicit shifts and masks vs. bitfields and compare assembly.
5. Read `arch/x86/include/asm/bitops.h` in the kernel source. Identify how `set_bit` is implemented (hint: `lock btsl`).

Next: Chapter 7 — Dynamic memory and writing your own allocator.

Chapter 7 — Dynamic memory and writing your own allocator

If you understand allocators, you understand most of operating systems. The kernel itself contains at least four allocators (`buddy`, `slab/SLUB`, `kmalloc`, `vmalloc`) and the GPU stack adds more (`TTM`, `DRM-GEM`, `amdgpu_bo`). This chapter takes you from `malloc` to writing a real free-list allocator.

7.1 The libc heap

`malloc(3)`, `calloc(3)`, `realloc(3)`, `free(3)` come from libc. They manage a *heap* — a region of the process's address space that grows on demand:

```
int *p = malloc(sizeof(int) * 100);
if (!p) abort();
memset(p, 0, sizeof(int) * 100);
free(p);
```

Things you must remember:

- `malloc` may return `NULL`. Always check.
- `malloc` does **not** zero memory. Use `calloc` (which does) or `memset`.
- `realloc(p, n)` can move the buffer — old `p` may become invalid. Always assign the result.
- `free(NULL)` is defined and is a no-op. `free(p)` followed by `free(p)` is UB.

```
char *grow(char *buf, size_t *cap)
{
    size_t new_cap = *cap ? *cap * 2 : 16;
    char *nb = realloc(buf, new_cap);
    if (!nb) return NULL;        /* keep old buf valid */
    *cap = new_cap;
    return nb;
}
```

7.2 Where does memory come from?

`malloc` doesn't talk to hardware. Internally it calls `brk(2)` or `mmap(2)` to get pages from the kernel, then carves them up. The standard glibc allocator (`ptmalloc2`) maintains:

- **Arenas**: per-thread heaps to reduce contention.
- **Bins**: free lists for small/medium/large/huge sizes.
- **Top chunk**: a big slab grown via `sbrk`.

Modern alternatives: `tcmalloc` (Google), `jemalloc` (Facebook/FreeBSD), `mimalloc` (Microsoft). They differ in fragmentation, locality, and concurrency strategy. ROCm runtimes use jemalloc or mimalloc in places.

If you `strace` a libc `malloc(8)` it makes 0 syscalls — the heap is already mapped. `malloc(8 * 1024 * 1024)` may issue an `mmap()`.

7.3 Memory layout of a small allocation

A glibc small allocation looks like:

```
ptr returned to user → ┌──────────────────┐
                       │   payload (n)    │
                       └──────────────────┘

ptr - sizeof(size_t) → ┌──────────────────┐   "chunk header"
                       │  size + flags    │
                       └──────────────────┘
```

`free(p)` reads the size from the header *before* `p` to know how much to release. This is why writing past the start (off-by-one underflow) corrupts the heap silently.

This is also why `free` doesn't take a size argument while `kfree` in the kernel doesn't either. `kmalloc` stashes metadata too.

7.4 Patterns and anti-patterns

```c
/* Pattern: allocate-or-fail */
struct foo *f = malloc(sizeof *f);
if (!f) return -ENOMEM;

/* Pattern: caller-allocates */
void compute(struct result *out);  /* caller allocates, callee fills */

/* Pattern: borrow */
void process(const struct data *d);  /* callee does not own */

/* Anti-pattern: returning an internal buffer */
const char *get_name(void) {
    static char buf[64];   /* not thread-safe */
    /* … */
    return buf;
}
```

Driver code idiomatic patterns:

- `kzalloc(sizeof *p, GFP_KERNEL)` for "give me a zeroed struct."
- `kmem_cache_alloc()` when you allocate the same size repeatedly.
- `devm_kzalloc(dev, size, GFP_KERNEL)` for "free me when device unbinds."
- `dma_alloc_coherent()` for memory the device will DMA from/to.

We will visit each in Part V.

7.5 The arena allocator

When a phase of work has many short-lived allocations and a clear "end of phase," an **arena** is dramatically simpler and faster than malloc. You bump a pointer; you free the whole arena at once.

```c
/* arena.h */
#include <stddef.h>
typedef struct arena {
    char  *base;
```

```
        size_t cap;
        size_t off;
    } arena;

    void  arena_init(arena *a, void *backing, size_t cap);
    void *arena_alloc(arena *a, size_t size, size_t align);
    void  arena_reset(arena *a);
```

```
    /* arena.c */
    #include <assert.h>
    #include <stdint.h>
    #include "arena.h"

    static size_t align_up(size_t x, size_t a) {
        return (x + a - 1) & ~(a - 1);
    }

    void arena_init(arena *a, void *backing, size_t cap) {
        a->base = backing;
        a->cap  = cap;
        a->off  = 0;
    }

    void *arena_alloc(arena *a, size_t size, size_t align) {
        assert((align & (align - 1)) == 0);
        size_t off = align_up(a->off, align);
        if (off + size > a->cap) return NULL;
        void *p = a->base + off;
        a->off  = off + size;
        return p;
    }

    void arena_reset(arena *a) { a->off = 0; }
```

Use:

```
    char backing[1 << 16];
    arena a;
    arena_init(&a, backing, sizeof backing);
    int *xs = arena_alloc(&a, sizeof(int)*100, _Alignof(int));
    arena_reset(&a);   /* all 100 ints "freed" in O(1) */
```

This is the same pattern as the kernel's `mempool`, `dma_pool`, and (loosely) the GPU command buffer allocator. Compilers, IR optimizers, JITs all use arenas.

7.6 A free-list allocator

A real general-purpose allocator must support free of individual blocks. The simplest model that's worth writing yourself:

- Each block has a header: `{size, free_flag, prev_block_size}`.
- A *free list* threads through free blocks by their headers.
- `alloc(n)` walks the free list, finds a fit, splits if too large.
- `free(p)` flips the flag and *coalesces* with adjacent free blocks.

```
    /* freelist_alloc.c — illustrative; not multi-thread safe. */
    #include <stddef.h>
    #include <stdint.h>
    #include <string.h>
    #include <stdbool.h>

    #define ALIGN 16
```

```c
typedef struct hdr {
    size_t size;     /* including header */
    bool   free;
    struct hdr *prev;
    struct hdr *next;   /* in free list */
} hdr;

static char   pool[1 << 20];
static hdr   *free_head;

static size_t align_up(size_t x) { return (x + ALIGN - 1) & ~(ALIGN - 1); }

void fl_init(void) {
    free_head = (hdr *)pool;
    free_head->size = sizeof pool;
    free_head->free = true;
    free_head->prev = NULL;
    free_head->next = NULL;
}

void *fl_alloc(size_t n) {
    size_t need = align_up(n + sizeof(hdr));
    for (hdr *h = free_head; h; h = h->next) {
        if (!h->free || h->size < need) continue;
        if (h->size >= need + sizeof(hdr) + ALIGN) {
            hdr *split = (hdr *)((char *)h + need);
            split->size = h->size - need;
            split->free = true;
            split->prev = h;
            split->next = h->next;
            if (h->next) h->next->prev = split;
            h->size = need;
            h->next = split;
        }
        h->free = false;
        return (char *)h + sizeof(hdr);
    }
    return NULL;
}

void fl_free(void *p) {
    if (!p) return;
    hdr *h = (hdr *)((char *)p - sizeof(hdr));
    h->free = true;
    /* coalesce forward */
    if (h->next && h->next->free) {
        h->size += h->next->size;
        h->next = h->next->next;
        if (h->next) h->next->prev = h;
    }
    /* coalesce backward */
    if (h->prev && h->prev->free) {
        h->prev->size += h->size;
        h->prev->next = h->next;
        if (h->next) h->next->prev = h->prev;
    }
}
```

This is missing the threading the free list separately from the header chain (real allocators use a separate free list to walk only free blocks), and it's not thread-safe, but it captures the *shape*. A weekend of polishing turns this into a teaching allocator. Implementing one is the single best way to understand `kmalloc` / `SLUB`.

7.7 The SLUB / slab idea

For fixed-size, frequently-allocated objects, scanning a free list is wasteful. The slab allocator pre-allocates a *page* of identical objects and hands them out with a single bump or LIFO pop. Free goes back to the slab.

The kernel's `kmem_cache_create()` builds one. You allocate objects from that cache with `kmem_cache_alloc()` and free with `kmem_cache_free()`. This is how `task_struct`, `inode`, `skbuff`, `dma_fence` etc. are managed. We'll use it in Part V.

7.8 Lifetimes: who owns the memory?

Every allocation has an *owner* — the code path responsible for `free`. Encode ownership in your API:

- "out parameter, caller frees": `int parse(const char *src, char **out)` — caller frees `*out`.
- "caller-allocated": `int parse_into(const char *src, char *out, size_t out_cap)`.
- "borrowed": `void log_msg(const char *msg)` — log doesn't free; caller still owns.

Kernel uses *reference counting* (`kref`, `refcount_t`) for objects with multiple owners:

```
struct file {
    refcount_t   ref;
    /* … */
};
```

Each user takes a reference; when the last goes away, the destructor runs. We'll use refcounts heavily for DMA buffers and fences.

7.9 Defensive habits

1. After `free(p)`, set `p = NULL`. Then a use-after-free will crash on the next deref instead of corrupting silently.
2. Allocate one struct at a time; don't reuse buffers for unrelated purposes.
3. Use ASan/UBSan/Valgrind in dev. Use `kasan` and `kfence` in your kernel builds.
4. Never trust a length from userspace until you've verified it.

7.10 Exercises

1. Implement `arena.h/c` exactly as above. Write a test that allocates 1000 random-sized blocks, then resets the arena, and verifies pointers no longer alias the allocator state.
2. Implement the free-list allocator. Test it: allocate 10000 blocks of random sizes, free half at random, allocate 5000 more, verify no overlap.
3. Add coalescing tests: allocate three adjacent blocks A, B, C. Free A, then C, then B. Verify the free list collapses to a single block.
4. Add a `slab` allocator on top of your arena. `slab_create(size_t obj_size)` returns a slab; `slab_alloc()` returns a fixed-size object; `slab_free()` returns it.
5. Run all of the above under ASan to confirm no overruns.

Next: Chapter 8 — Preprocessor, headers, linkage, build systems.

Chapter 8 — The preprocessor, headers, linkage, build systems

The C preprocessor is a textual macro system that runs *before* the compiler sees your code. The kernel uses it heavily. So does every driver. You must read and write macros confidently.

8.1 Directives

```
#include <header.h>     /* search system paths */
#include "header.h"     /* search local first, then system */
#define X 42
#define MAX(a,b) ((a) > (b) ? (a) : (b))
#undef X
#if   defined(CONFIG_FOO)
#elif defined(CONFIG_BAR)
#else
#endif
#ifdef CONFIG_FOO
#ifndef HEADER_H
#error "must define X"
#warning "deprecated"
#pragma once
```

`#include` is **textual paste**. The compiler then sees the union of all the included files as one giant translation unit.

8.2 Macro pitfalls

```
#define SQ(x) x*x
SQ(1+2)            /* expands to 1+2*1+2 = 5, not 9 */
```

Always parenthesize: `#define SQ(x) ((x)*(x))`.

```
#define INCR(x) ((x)+1)
INCR(i++)          /* still safe — no double-eval */

#define MAX(a,b) ((a) > (b) ? (a) : (b))
MAX(i++, j++)      /* increments i or j twice. BAD. */
```

To avoid double-evaluation, kernel uses statement expressions (GCC extension):

```
#define max(a, b) ({                \
    typeof(a) _a = (a);             \
    typeof(b) _b = (b);             \
    _a > _b ? _a : _b;              \
})
```

`typeof` and `({ … })` are GCC extensions that the kernel relies on.

8.3 `#` and `##`

```
#define STR(x)   #x          /* stringify */
#define CAT(a,b) a##b        /* token paste */

STR(hello)         /* "hello" */
CAT(foo, _bar)     /* foo_bar */
```

The kernel uses these in tracing macros and in `EXPORT_SYMBOL`.

8.4 Header guards and `#pragma once`

Every header must be safe to include twice:

```
/* foo.h */
#ifndef FOO_H
#define FOO_H
/* declarations */
#endif
```

Or:

```
#pragma once
```

`#pragma once` is a near-universal extension (GCC, Clang, MSVC). Kernel uses include guards. Either is fine in your own code; be consistent.

8.5 What goes in a header

In a header, put:

- Type declarations (`struct`, `enum`, `typedef`).
- Function *declarations* (prototypes).
- `extern` global declarations.
- Inline functions or `static inline` helpers.
- Macros.

Do **not** put:

- Function *definitions* (bodies). They will be redefined in every TU that includes the header → linker error.
- Global variable definitions. (Same reason.)

Exception: `static inline` and `inline` functions are OK in headers because each TU gets its own copy or because the symbol has special linkage.

8.6 Linkage

Every name has a *linkage*:

- **External linkage**: visible to other translation units (TUs). Default for non-`static` file-scope names.
- **Internal linkage**: visible only within a TU. `static` at file scope.

- **None**: local variables, function parameters.

If you define a function `foo` in two .c files without `static`, the linker complains: multiple definition.

Static helper inside a .c:

```
static int helper(int x) { return x + 1; }
```

You may define this in a header *only* if you also `static inline` it.

8.7 The `inline` keyword

`inline` is a hint. The compiler can inline non-`inline` functions and decline to inline `inline` ones. To *force*:

```
__attribute__((always_inline)) static inline int hot(int x) { return x; }
```

Or to forbid:

```
__attribute__((noinline)) int slow(int x) { return x; }
```

Kernel macros: `noinline`, `__always_inline`, `__force_inline`.

8.8 The build pipeline of a real C project

```
src/main.c     ─┐
src/util.c     ─┼─[preprocess+compile]→ *.o ─[link]→ executable
src/io.c       ─┘
                              ↑
include/ headers ──────────┘ via -I
external libs   ──────────────────────────[ld -l]
```

Build systems:

- **Make / GNU Make**: text-based, used by the kernel (Kbuild) and many older C projects.
- **CMake**: declarative, generates Makefiles or Ninja files. Mesa, ROCm, LLVM use CMake.
- **Meson + Ninja**: simpler than CMake, also used by Mesa.
- **Bazel**: Google's; rare in the Linux ecosystem.

You will primarily use Make for the kernel and CMake/Meson for userspace.

A real Makefile pattern

```
CC      := gcc
CFLAGS  := -std=c11 -Wall -Wextra -Werror -O2 -g -Iinclude
LDFLAGS :=
SRC     := $(wildcard src/*.c)
OBJ     := $(SRC:src/%.c=build/%.o)
BIN     := build/app

all: $(BIN)

$(BIN): $(OBJ)
    $(CC) $(LDFLAGS) -o $@ $^

build/%.o: src/%.c | build
    $(CC) $(CFLAGS) -MMD -MP -c $< -o $@
```

```
build:
    mkdir -p build

-include $(OBJ:.o=.d)

clean:
    rm -rf build

.PHONY: all clean
```

The `-MMD -MP` flags emit `.d` files listing each `.o`'s header dependencies. Then `-include` them so changing a header rebuilds everything that uses it. Get this right and you save hours of debugging stale builds.

8.9 `pkg-config`

For external libraries:

```
        pkg-config --cflags --libs libdrm
        # -I/usr/include/libdrm  -ldrm
```

```
CFLAGS  += $(shell pkg-config --cflags libdrm)
LDFLAGS += $(shell pkg-config --libs libdrm)
```

When you write a userspace GPU tool with libdrm, you'll do exactly this.

8.10 Static vs. dynamic libraries

```
        # Static lib
        ar rcs libutil.a util.o
        gcc main.o -L. -lutil -o app    # links util.o into app

        # Shared lib
        gcc -shared -fPIC -o libutil.so util.o
        gcc main.o -L. -lutil -o app
```

Static: code is copied into the executable; bigger; no runtime dep. Shared: code is loaded at runtime via `ld.so`; smaller; ABI compatibility matters.

The kernel itself is one giant statically-linked executable (`vmlinux`) plus dynamically-loadable modules (`*.ko` — compiled relocatable objects).

8.11 The kernel build system (Kbuild)

The kernel uses `make` plus a top-level `Kbuild` system. A simple module's Makefile:

```
obj-m := hello.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD  := $(shell pwd)

default:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

`obj-m := hello.o` says "build hello.ko." The build runs in the kernel build tree (`-C $(KDIR)`) but compiles files from your dir (`M=$(PWD)`). We will build our first module in Chapter 29.

For a multi-file module:

```
obj-m := mydrv.o
mydrv-y := main.o util.o regs.o
```

For tristate options (`y` / `m` / `n`):

```
obj-$(CONFIG_MYDRV) += mydrv.o
```

The `Kconfig` file declares the option; `make menuconfig` lets the user choose.

8.12 Reading a real driver Makefile

```
# drivers/gpu/drm/amd/amdgpu/Makefile (excerpt)
amdgpu-y := amdgpu_drv.o amdgpu_kms.o amdgpu_device.o \
            amdgpu_atombios.o amdgpu_bo_list.o \
            amdgpu_cs.o amdgpu_object.o amdgpu_gem.o \
            amdgpu_mn.o amdgpu_sa.o amdgpu_sync.o \
            amdgpu_fence.o amdgpu_ttm.o amdgpu_pll.o \
            ...

obj-$(CONFIG_DRM_AMDGPU) += amdgpu.o
```

This builds the giant `amdgpu.ko` module. When you contribute a new file to amdgpu you'll add it here.

8.13 Exercises

1. Write `MAX`, `MIN`, `CLAMP`, `ARRAY_SIZE`, `STRINGIFY`, `CONCAT` macros. Make them work on any types; protect against double-evaluation using `typeof`.
2. Build a 3-file project (header + 2 .c) with a Makefile that supports `make`, `make clean`, and dependency tracking via `-MMD`.
3. Use `pkg-config` to write a Makefile that links against `libpng`. Compile a tiny program that opens a PNG.
4. Write a `Kconfig` and `Makefile` pair so that your `hello.ko` module from Chapter 29 (you'll write it later) can be built either as `=y` (built-in) or `=m` (module) when integrated into a kernel tree.
5. Read the top of `Makefile` in the kernel tree; identify how `KBUILD_CFLAGS` is constructed.

Next: Chapter 9 — Function pointers, callbacks, opaque types, plugin design.

Chapter 9 — Function pointers, callbacks, opaque types, plugin design

The Linux kernel is "C with virtual functions." It implements polymorphism using **structs of function pointers**. Once you internalize this, the kernel suddenly becomes legible.

9.1 The syntax, one more time

```
int add(int a, int b) { return a + b; }

int (*fp)(int, int) = add;     /* pointer to function */
int (*fp2)(int, int) = &add;   /* same — & is optional */

int r = fp(2, 3);              /* call through pointer */
int r2 = (*fp)(2, 3);          /* same */
```

Use `typedef` for readability:

```
typedef int (*binop_fn)(int, int);
binop_fn ops[] = { add, sub, mul, div };
```

To return a function pointer, the syntax is dreadful — let `typedef` save you:

```
typedef int (*op_t)(int, int);
op_t pick(char c) { return c == '+' ? add : sub; }
```

9.2 Callbacks

Pass behavior as data:

```
void each(int *a, size_t n, void (*fn)(int *)) {
    for (size_t i = 0; i < n; i++) fn(&a[i]);
}

static void doubl(int *p) { *p *= 2; }

each(arr, 100, doubl);
```

Always pass a context pointer alongside:

```
void each_ctx(int *a, size_t n, void (*fn)(int *, void *), void *ctx);
```

Callbacks without context become global state and won't compose. The kernel's qsort (`sort()`) takes a `cmp` callback but no ctx; you have to use a per-CPU variable or container_of() trick. Don't repeat that mistake in your own APIs.

9.3 Vtables in C: structs of function pointers

```c
struct file_operations {
    struct module *owner;
    int     (*open)(struct inode *, struct file *);
    int     (*release)(struct inode *, struct file *);
    ssize_t (*read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char __user *, size_t, loff_t *);
    long    (*unlocked_ioctl)(struct file *, unsigned int, unsigned long);
    int     (*mmap)(struct file *, struct vm_area_struct *);
    /* … */
};
```

Each driver instantiates one of these as a `static const` and registers it. The VFS calls `f->f_op->read(...)` polymorphically.

```c
static const struct file_operations my_fops = {
    .owner  = THIS_MODULE,
    .open   = my_open,
    .release= my_release,
    .read   = my_read,
    .write  = my_write,
    .unlocked_ioctl = my_ioctl,
};
```

This is the **single most important pattern** in kernel C. Every subsystem looks like this:

- `struct pci_driver` — for PCI device drivers
- `struct platform_driver` — for SoC devices
- `struct drm_driver` — for DRM drivers
- `struct dma_fence_ops` — for fence operations
- `struct ttm_device_funcs` — for TTM
- `struct drm_sched_backend_ops` — for the GPU scheduler

Once you know “find the ops struct” you can navigate any subsystem.

9.4 Opaque types and the C version of OOP

Encapsulation without classes:

```c
/* counter.h */
struct counter;
struct counter *counter_create(void);
void  counter_destroy(struct counter *);
void  counter_inc(struct counter *);
int   counter_value(const struct counter *);
```

```c
/* counter.c */
struct counter { int n; };
struct counter *counter_create(void) {
    struct counter *c = malloc(sizeof *c);
    if (c) c->n = 0;
    return c;
}
void counter_destroy(struct counter *c) { free(c); }
void counter_inc(struct counter *c) { c->n++; }
int  counter_value(const struct counter *c) { return c->n; }
```

The kernel uses opaque types extensively: `struct mutex`, `struct dma_fence`, `struct drm_device` — you hold pointers, you don't peek inside.

9.5 Inheritance via embedded structs

```
struct base {
    int x;
};

struct derived {
    struct base parent;   /* must be first or use container_of */
    int y;
};

void use_base(struct base *b);

struct derived d = { .parent.x = 1, .y = 2 };
use_base(&d.parent);            /* upcast */
struct derived *dp = container_of(b, struct derived, parent);  /* downcast */
```

This is exactly how `struct kobject` and `struct device` are used. `kobject` is the "base class" of every object that appears in `/sys`; embed it, and your object is sysfs-visible.

```
struct my_dev {
    struct device dev;     /* embed */
    void __iomem *mmio;
    int           irq;
    /* … */
};
```

9.6 The `_init` / `_destroy` symmetry

Every constructor needs a destructor. Driver code is heavy with paired init/destroy; if you forget destroy you leak, often silently for hours.

```
struct foo {
    void   *buf;
    size_t  cap;
};

int  foo_init(struct foo *f, size_t cap)
{
    f->buf = malloc(cap);
    if (!f->buf) return -ENOMEM;
    f->cap = cap;
    return 0;
}

void foo_fini(struct foo *f)
{
    free(f->buf);
    f->buf = NULL;
    f->cap = 0;
}
```

For probe/remove paths, every step adds an "undo" we must perform on error or remove. Use **goto chains**:

```
int probe(struct device *dev)
{
```

```
        struct mydrv *m = kzalloc(sizeof *m, GFP_KERNEL);
        if (!m)                     return -ENOMEM;

        m->mmio = ioremap(addr, size);
        if (!m->mmio)               { ret = -ENOMEM; goto err_free; }

        ret = request_irq(irq, isr, 0, "mydrv", m);
        if (ret)                    goto err_unmap;

        ret = register_chrdev(...);
        if (ret)                    goto err_free_irq;

        return 0;

    err_free_irq:   free_irq(irq, m);
    err_unmap:      iounmap(m->mmio);
    err_free:       kfree(m);
        return ret;
    }
```

Goto for cleanup chains is **idiomatic** in the kernel and *encouraged* by the kernel coding style. Don't be afraid of it.

9.7 Plugin design

A "plugin" registers itself with a core via a function pointer table. The core walks a list of registered plugins and calls into them.

```
    /* core.h */
    struct codec_ops {
        const char *name;
        int (*encode)(const uint8_t *in, size_t n, uint8_t *out);
    };

    int  codec_register(const struct codec_ops *ops);
    void codec_unregister(const struct codec_ops *ops);
    const struct codec_ops *codec_find(const char *name);
```

```
    /* mp3.c */
    static const struct codec_ops mp3_ops = {
        .name   = "mp3",
        .encode = mp3_encode,
    };

    static int __init mp3_init(void) { return codec_register(&mp3_ops); }
    static void __exit mp3_exit(void) { codec_unregister(&mp3_ops); }
```

The Linux kernel implements every subsystem this way. `pci_register_driver`, `platform_driver_register`, `drm_dev_register` are all variations.

9.8 The `EXPORT_SYMBOL` mechanism

Modules can call into the kernel only via *exported* symbols:

```
    int  amdgpu_bo_create(...);
    EXPORT_SYMBOL(amdgpu_bo_create);

    /* or for GPL-only modules */
    EXPORT_SYMBOL_GPL(amdgpu_bo_create);
```

`EXPORT_SYMBOL_GPL` restricts use to GPL-licensed modules. `lsmod` and `modinfo` show dependencies. When your module references a symbol, the linker (`modpost`) records the dep.

9.9 Exercises

1. Implement an in-memory key/value store with vtable for storage backends:

```
struct kv_ops {
    int (*put)(void *self, const char *k, const char *v);
    int (*get)(void *self, const char *k, char *v, size_t cap);
    int (*del)(void *self, const char *k);
};
```

   Provide two backends: in-memory hashtable and a file-backed one.
2. Implement a tiny event system: `event_subscribe(name, fn, ctx)`, `event_publish(name, data)`. Multiple subscribers per event.
3. Implement a state machine using a table of `state_handler` function pointers.
4. Read `include/linux/fs.h` and find `struct file_operations`. Identify each callback. Read at least one real implementation (e.g. `drivers/char/mem.c` — `mem_fops`).
5. Read `include/drm/drm_drv.h` and find `struct drm_driver`. Skim. We will use it heavily later.

Next: Chapter 10 — Defensive C: const correctness, restrict, MISRA-ish style.

Chapter 10 — Defensive C: const, restrict, error handling, kernel style

The difference between code that ships and code that gets rejected on review is not cleverness — it's discipline. This short chapter codifies the habits that survive review on the kernel mailing list.

10.1 Const-correctness

Apply `const` aggressively to arguments and locals you don't mutate.

```
size_t my_strlen(const char *s);                  /* good */
int    parse(const char *src, char *dst, size_t cap);
```

Returning a `const T *` says "you may read but not modify." Use it for accessors that expose internal data:

```
const struct gpu_caps *gpu_get_caps(const struct gpu *g);
```

Top-level data tables that never change: `static const`. Then they live in `.rodata`, are protected by the kernel's `mark_rodata_ro`, and the compiler can fold reads.

```
static const u32 lut[256] = { /* ... */ };
```

10.2 `restrict`

When you know two pointer args don't alias, mark them `restrict`. The compiler can vectorize. Real example:

```
void axpy(size_t n,
          float a,
          const float * restrict x,
          float       * restrict y)
{
    for (size_t i = 0; i < n; i++) y[i] += a * x[i];
}
```

In hot loops in math/graphics code, `restrict` can be a 2x speedup.

10.3 Error handling

C has no exceptions. Pick a discipline and stick to it. The kernel uses **negative errno on failure, 0 on success, positive count on certain reads/writes**:

```
int do_thing(void);
/* return 0 on success, -EINVAL/-ENOMEM/etc on error */
```

For pointers that might be errors, the kernel uses `ERR_PTR()`:

```
struct foo *foo_create(void)
{
    struct foo *f = kzalloc(sizeof *f, GFP_KERNEL);
    if (!f)
        return ERR_PTR(-ENOMEM);
    /* ... */
    return f;
}

struct foo *f = foo_create();
if (IS_ERR(f))
    return PTR_ERR(f);
```

`ERR_PTR(-12)` returns a pointer in the top page (high addresses, kernel-only) that encodes the error. `IS_ERR` checks. `PTR_ERR` decodes. This avoids out-parameters in the common case.

10.4 Cleanup with `goto`

The pattern from the previous chapter, restated as a rule:

> **Every function with > 1 fallible step uses goto for unwind.**

```
int do_complex(struct dev *d)
{
    int ret;

    ret = step_a(d);
    if (ret) return ret;

    ret = step_b(d);
    if (ret) goto out_undo_a;

    ret = step_c(d);
    if (ret) goto out_undo_b;

    return 0;

out_undo_b: undo_b(d);
out_undo_a: undo_a(d);
    return ret;
}
```

Don't nest 8 `if` blocks. Don't duplicate cleanup. Goto chains are *the* idiom.

10.5 The kernel coding style — what you must absolutely follow

Read `Documentation/process/coding-style.rst` in any kernel tree. Highlights:

- Tabs, 8 spaces wide. **Always tabs in kernel code**, not spaces.
- 80-column soft limit (now 100 in some areas).
- Brace placement: opening brace at end of line for control flow; on its own line for functions:

```
        if (x) {
            foo();
        }

        void func(void)
        {
            ...
        }
```

- `if (!ptr)` not `if (ptr == NULL)`.
- Names are `lowercase_with_underscores`. CamelCase is forbidden.
- One declaration per line; no `int a, b, *c;`.
- Variables declared at top of block.
- Functions short — under 80 lines if possible.
- No typedefs for pointers (`typedef struct foo *foo_t` — forbidden).
- Don't put complex logic in macros if a `static inline` works.
- No mixed-case multi-line comments. Use `/* ... */`, kernel uses `//` sparingly (now allowed).
- `Signed-off-by:` your real name + email on every commit.

Run `scripts/checkpatch.pl` on your patches before sending.

10.6 `assert`, `BUG_ON`, `WARN_ON`

In userspace, `assert(p != NULL)` aborts the program when the condition is false. Disabled by `-DNDEBUG`. Don't put logic in assertions; they evaporate.

In kernel:

```
        WARN_ON(condition);   /* dump stack and continue */
        WARN_ON_ONCE(condition);
        BUG_ON(condition);    /* "this is unrecoverable", may panic */
```

`BUG_ON` is rarely correct — prefer `WARN_ON` plus an error return. The kernel must keep going even if a sub-driver hits weirdness; killing the kernel for a recoverable error is hostile to the user.

10.7 Avoiding undefined behavior

A short checklist you can paste above any patch:

- [ ] No signed integer overflow without `__builtin_*overflow` / `check_*overflow`.
- [ ] No shift past width.
- [ ] No misaligned access (use `memcpy` for unaligned loads).
- [ ] No use-after-free; every `free` followed by `= NULL` if the var lives on.
- [ ] No data race on shared mutable state without lock or atomic.
- [ ]

- [ ] All `malloc` / `kmalloc` checked.
- [ ] All paths free what they own.
- [ ] No infinite recursion.
- [ ] Return value of every fallible call checked.

Build dev with `-Wall -Wextra -Wshadow -Wstrict-overflow -Wundef -Werror -fsanitize=address,undefined -fstack-protector-strong`. Kernel: `KASAN=y`, `UBSAN=y`, `KCSAN=y` for race detection.

10.8 A sample function reviewed

Bad:

```
char *get(int n) {
    char *p = malloc(n);
    sprintf(p, "%d", n);
    return p;
}
```

Problems: no NULL check on `malloc`; `sprintf` may overflow if `n` is large enough and the buffer is `n` bytes; caller has to free; nothing documents that.

Better:

```
/**
 * get_decimal_string - format @n as a decimal string.
 * @n:    integer to format.
 * @out:  caller-provided buffer.
 * @cap:  size of @out in bytes.
 *
 * Return: number of bytes written (excluding trailing NUL),
 *         or -EINVAL if @out is NULL or @cap is too small.
 */
int get_decimal_string(int n, char *out, size_t cap)
{
    int len;

    if (!out || cap == 0)
        return -EINVAL;

    len = snprintf(out, cap, "%d", n);
    if (len < 0 || (size_t)len >= cap)
        return -EOVERFLOW;
    return len;
}
```

Caller-allocated, safe `snprintf`, documented contract, errno return. This is the shape of code that ships.

10.9 Exercises

1. Take the bad version above and turn it into the better one. Add a unit test that passes a 1-byte buffer.

2. Convert any tutorial code you have to use kernel-style indentation, 80-col limit, goto-chain cleanup. Run `scripts/checkpatch.pl --strict` against it. Fix every warning.
3. Write `safe_atoi(const char *s, int *out)` that returns 0 on success, `-EINVAL` on garbage, `-ERANGE` on overflow.
4. Read `mm/slub.c` (a kernel core file) and identify ten places that follow this style: const args, errno returns, goto-chain cleanup, `WARN_ON_ONCE` for invariants.

End of Part I. Move to Part II — C++ for systems programmers.

Part DSA — Data Structures & Algorithms (in C)

> **Stage 4 of the LEARNING PATH. 8–10 weeks at 1 hr/day.**

This is the part most engineers skip and most engineers regret skipping. **Don't.**

Why DSA is non-negotiable

Two reasons, both real:

1. **Every interview at AMD/NVIDIA/Qualcomm/Intel has a DSA round.** "Reverse a linked list." "Find the kth largest." "Implement an LRU cache." If you can't do these in 30 minutes in C, you don't get the offer. The hiring bar is the same for new grads and 24-year PMTS — both rounds have DSA.
2. **The Linux kernel is mostly data structures.** Every advanced concept in this book — fences, BO trees, GPUVM page tables, IH ring buffers, dma-buf trackers — is a data structure with constraints. If you've implemented a hash table once yourself, the kernel's `hashtable.h` reads like prose. If not, it reads like Sanskrit.

You have two reasons. Either alone is enough. Both together: **non-negotiable.**

How DSA fits with the rest of this book

- We are at Stage 4. You've finished C (Part I).
- The next stage is **userspace systems programming** (Part USP), which uses DSA constantly (e.g., epoll uses red-black trees internally; you'll write hash tables to track file descriptors; etc.).
- Then OS theory (Part OST) uses DSA at every page (page replacement = LRU = doubly linked list + hash; CPU scheduling = priority queue = heap; etc.).
- Then the kernel — where DSA shows up in `list_head`, `rb_root`, `xarray`, `kfifo`, `radix_tree`, `interval_tree`, `idr/ida`.

So this part has **the highest leverage** of any part in the book. You will use what you learn here for the next 30 years.

⚠️ Read this first — DSA-00 — How to read this part (without running code)

Many chapters have been **rewritten so you can learn purely by reading** — with diagrams, step-by-step traces, and worked examples. **You don't need to run any code** for chapters 1–7, 10, 14, and 15. Read them. Trace them with paper.

The remaining chapters (8, 9, 11–13, 16–28) are still in the original style — they assume you'll run code or solve problems. Get the foundations from the rewritten chapters first; the rest will be much easier.

How to study this part

For the rewritten chapters (1–7, 10, 14, 15):

1. Read once skimming.
2. Read again, slowly tracing each example with a notebook.
3. Without looking, redraw the data structure on a blank page.
4. Re-read to find what you missed.

For the original-style chapters (8, 9, 11–13, 16–28):

1. **Implement it from scratch in C.** No `qsort`, no library, no copying.
2. **Test it with at least 5 cases**, including empty, one, and corner.
3. **Compute the complexity** (time and space) and write it as a comment at the top.
4. **Solve 5 problems using it** on Leetcode or HackerRank, in C.
5. **Find where the Linux kernel uses it.** `git grep` it. Read 1 short usage.

If you do this for every chapter, you'll be ahead of >90% of new grads.

Table of contents

✅ = rewritten in read-to-learn style with diagrams + traces (no code-run required) ○ = original style (run-and-solve)

#	Chapter	Style	Why it matters
0	How to read this part (without running code)	✅	Set expectations
1	Big-O & complexity analysis	✅	The language all engineers speak
2	Arrays — the foundation	✅	RAM is an array; everything starts here
3	Strings (in C, properly)	✅	C strings are tricky; master them
4	Linked lists (single, double, circular)	✅	The kernel's `list_head` is everywhere
5	Stacks & queues (incl. ring buffers)	✅	Function calls, IH ring, kfifo
6	Hash tables (chaining, open addressing)	✅	The most-used data structure in CS
7	Binary trees & BSTs	✅	Foundation for self-balancing trees
8	Self-balancing trees: AVL, Red-Black	○	Linux `rbtree` is in this chapter
9	B-trees and B+trees	○	Filesystems and DBs use these

#	Chapter	Style	Why it matters
10	Heaps & priority queues	✅	Schedulers, Dijkstra, top-K
11	Graphs — representation & traversal (BFS/DFS)	○	Dependency graphs, DRM scheduler deps
12	Shortest paths: Dijkstra, Bellman-Ford, A*	○	Path planning, graph weights
13	Tries	○	Strings, autocomplete, IP routing
14	Sorting algorithms (all of them)	✅	When each is right; kernel's sort.c
15	Searching: linear, binary, ternary, exponential	✅	The most-asked interview pattern
16	Recursion deep	○	The mental model behind DP, trees, backtracking
17	Dynamic programming	○	The interviewer's favorite
18	Greedy algorithms	○	When greedy works and when it doesn't
19	Divide & conquer	○	The mindset behind merge sort, quicksort
20	Bit manipulation tricks	○	Kernel uses tons of these
21	Two pointers, sliding window, fast/slow	○	Problem-solving patterns
22	Backtracking	○	N-queens, sudoku, permutations
23	Union-Find / DSU	○	Kruskal, connectivity, image labeling
24	Advanced trees: segment, Fenwick (BIT), interval	○	Linux `interval_tree` lives here
25	String algorithms: KMP, Rabin-Karp, Z, suffix array	○	Pattern matching, dedup, search
26	System-design DSA: LRU cache, bloom filter, skip list, consistent hashing	○	Every senior interview asks one
27	Concurrency-aware DS: lock-free queue, RCU, hazard pointers	○	The bridge to kernel concurrency
28	DSA in the Linux kernel — guided tour	○	Where in the source tree everything lives

Total: 28 chapters. At 2 chapters/week, that's 14 weeks. At 3 chapters/week (the recommended pace if you have 1 hr/day), it's about 9–10 weeks.

What you'll have at the end

- A folder of ~30 C files, each implementing one classic data structure or algorithm with tests. **This is a portfolio piece.** Push it to GitHub.
- The ability to walk into an interview and write any common DS from memory.
- The ability to read kernel headers (`include/linux/list.h`, `rbtree.h`, `hashtable.h`) and immediately understand them.

Notes on language

We use **C**, not Python or C++. Why?

- C is what the kernel uses. Implementing in C exposes the **memory layout** of every structure — which matters for performance, cache, and concurrency.
- Interview rounds often allow C++. That's fine — once you can do it in C, C++ is a strict superset.
- Python hides too much for our purposes (e.g., it has no real "pointer" — but the kernel is pointers everywhere).

The first time you implement a hash table in C, with hand-rolled hashing, collision chains, and resize logic, **you become a different engineer**. Don't outsource that.

Now → DSA-00 — How to read this part (without running code), then DSA-01 — Big-O.

DSA 0 — How to read this part (without running code)

> "I can't run the code. Can I still learn?" **Yes.** This part is rewritten so you can learn purely by reading.

What changed

The earlier version of these chapters showed code and said "go try it." That's good if you can run a compiler. **It's useless if you can't.**

The rewritten chapters show, for every important algorithm:

1. **A picture** of the data structure in memory.
2. **The code** with each line explained in plain English.
3. **A worked trace** — a step-by-step table showing how variables change as the algorithm runs.
4. **A second worked example** — different input, same algorithm, again traced.
5. **What goes wrong** — drawings of buggy states.
6. **A recap** in 3–5 sentences.

If you read each chapter slowly with a notebook (paper or digital), **you will learn** even without running anything. Most senior engineers learned the basics this way — there were no online compilers in 1995.

How to read each chapter

Pass 1 — get the picture

- Read the **"Big idea"** section.
- Look at the **picture** of the data structure.
- Skip the code.
- Read the **recap** at the end.
- Now you know **what** the chapter is about, even if not **how**.

Pass 2 — follow the trace

- Open your notebook.
- Read the code once. Don't worry if some lines confuse you.
- Read the worked trace **slowly**. At each step, **draw the diagram yourself**.
- Verify the variable values in the trace match what you drew.
- If something doesn't match, re-read the code line for the previous step.

Pass 3 — explain it back

- Close the page.
- On a blank piece of paper, **draw the data structure** from memory.
- **Re-derive** the algorithm — don't copy it; reason it out.
- Read the chapter again to find the gaps.

After pass 3, you "own" the chapter. That's the goal.

A note on diagrams

Almost everything is in **ASCII art**. It looks like this:

```
[10] -> [20] -> [30] -> NULL
```

Don't dismiss it. Every textbook diagram you've seen is a more polished version of this. The shape **is** the lesson.

For trees:

```
      10
     /  \
    5    15
   / \
  3   8
```

For arrays + indices:

```
index:   0    1    2    3    4
value: |10 | 20 | 30 | 40 | 50 |
```

For pointers:

```
p ──► [10 | next ──► [20 | next ──► NULL ]]
```

Read these as if they were pictures in a book. Your brain will fill in the rest.

What if I really want to run something?

You don't have to. But if you do:

- **Online**: compiler-explorer.com (godbolt) lets you compile and step through C in a browser. Free.
- **Online**: replit.com gives you a real Linux box in a tab.
- **Online**: programiz.com/c/online-compiler — just paste C code, hit "Run."

These need only a browser. No installation.

But again — **you don't need them**. This rewritten part is designed to teach you without them.

A note on the original chapters

The original chapters (DSA-01 to DSA-28) are still here. Some of the later ones (advanced topics) still expect you to run code. The **first 10 chapters** have been rewritten in the new style. Continue with the originals once you've mastered the foundations — by then you may want to run code yourself.

Original style	Rewritten — read-to-learn
DSA-01 Big-O	✅ rewritten
DSA-02 Arrays	✅ rewritten
DSA-03 Strings	✅ rewritten
DSA-04 Linked Lists	✅ rewritten
DSA-05 Stacks & Queues	✅ rewritten
DSA-06 Hash Tables	✅ rewritten
DSA-07 Trees	✅ rewritten
DSA-10 Heaps	✅ rewritten
DSA-14 Sorting	✅ rewritten
DSA-15 Searching	✅ rewritten
DSA-08, 09, 11, 12, 13, 16-28	original (still readable)

Chapters not yet rewritten will be in the next round.

Last thing — be patient

Learning a data structure from a diagram + trace **feels slower** than running code. It's not. Running code skips the part where you actually understand. Reading + tracing **is** the understanding.

You don't get faster by skipping steps. You get faster by mastering them.

→ Now read DSA-01 — Big-O and complexity.

DSA 1 — Big-O and complexity (read-to-learn version)

> If you only learn one thing from this part: **how the time of an algorithm grows when the input grows**. That's Big-O.

The big idea

Imagine you have a list of names. You want to find one name in it.

- If the list has **10 names**, it takes you maybe **5 seconds** of looking.
- If the list has **100 names**, it takes you maybe **50 seconds**.
- If the list has **1000 names**, it takes you about **500 seconds**.

Notice: **10× more names → 10× more time.** That relationship is what we call **linear**, written **O(n)**.

Now suppose the list is **sorted alphabetically** and you do "binary search" — look at the middle, then half, then half again. Then:

- 10 names → about **4 looks**.
- 100 names → about **7 looks**.
- 1000 names → about **10 looks**.
- 1,000,000 names → about **20 looks**.

The time grew **very slowly** even though the list got 100,000× larger. That's **logarithmic**, written **O(log n)**.

That's all Big-O is. A label saying *"how does time grow as input gets bigger?"*

The 7 labels you must know

Below, `n` = "size of the input." Read each row as **"as n doubles, time grows by ___."**

Label	Name	When n doubles, time grows by	Example
O(1)	constant	nothing	look up a value in a hash table
O(log n)	logarithmic	a tiny bit (one step)	binary search in sorted list
O(n)	linear	doubles	scan a list to find max
O(n log n)	"n log n"	a bit more than doubles	merge sort, quicksort
O(n^2)	quadratic	quadruples	bubble sort, naive duplicate check
O(2^n)	exponential	**doubles per +1 to n**	brute-force subsets

Label	Name	When n doubles, time grows by	Example
O(n!)	factorial	astronomical	brute-force permutations

Memorize this table. It's the alphabet of complexity.

How big is each, really? (numbers!)

Here's `n = 100`. How many "operations" does each curve take?

```
O(1)        →                      1 op
O(log₂ n)   →                      ~7 ops
O(n)        →                    100 ops
O(n log n)  →                    700 ops
O(n²)       →                 10,000 ops
O(2ⁿ)       →     1,000,000,000,...,000  (30 zeros — universe age)
O(n!)       →     too big to write
```

Now `n = 1000`:

```
O(1)        →                      1 op
O(log₂ n)   →                    ~10 ops
O(n)        →                  1,000 ops
O(n log n)  →                 10,000 ops
O(n²)       →              1,000,000 ops
O(2ⁿ)       →     so huge you can't compute it
```

A modern computer does about **1,000,000,000 operations per second**.

So: - **O(n²) on n=10,000**: 100 million ops → about **0.1 seconds**. Fine. - **O(n²) on n=100,000**: 10 billion ops → about **10 seconds**. Slow. - **O(n²) on n=1,000,000**: a trillion ops → **17 minutes**. Too slow.

That's why an algorithm that "works on small input" can **die** on bigger input.

Reading code to find Big-O

Here's a recipe. For each piece of code:

1. **One loop over n items** = `O(n)`.
2. **Loop over n inside another loop over n** = `O(n²)`.
3. **Loop where index halves each step** = `O(log n)`.
4. **No loops at all** = `O(1)`.
5. **Function calls** = whatever the called function's complexity is.

Example 1 — O(1)

```c
int first_element(int *arr) {
    return arr[0];
}
```

One step: read element 0 and return. Time does not depend on `n`. **O(1).**

Example 2 — O(n)

```c
int sum(int *arr, int n) {
    int s = 0;
```

```
        for (int i = 0; i < n; i++)
            s += arr[i];
        return s;
    }
```

The loop runs `n` times. Each iteration does a constant amount of work (1 add). Total: `n` adds → **O(n).**

If `n = 1000`, the body runs 1000 times. If `n = 1,000,000`, the body runs a million times.

Example 3 — O(n²)

```
    int has_duplicate(int *arr, int n) {
        for (int i = 0; i < n; i++)
            for (int j = i + 1; j < n; j++)
                if (arr[i] == arr[j])
                    return 1;
        return 0;
    }
```

Two nested loops. The outer runs `n` times. The inner runs (on average) `n/2` times. Total: about `n × n/2` = `n²/2` operations.

We **drop the constant** (the `/2`) for Big-O notation. Result: **O(n²).**

If `n = 1000`, the body runs about **500,000** times. If `n = 10,000`, the body runs about **50,000,000** times. **100× slower** for 10× input.

Example 4 — O(log n)

```
    int binary_search(int *a, int n, int x) {
        int lo = 0, hi = n - 1;
        while (lo <= hi) {
            int mid = (lo + hi) / 2;
            if (a[mid] == x) return mid;
            if (a[mid] < x) lo = mid + 1;
            else            hi = mid - 1;
        }
        return -1;
    }
```

The search range starts at size `n` and **halves** each step (we cut to either left half or right half).

How many times can you halve `n` before reaching 1?

```
n = 1000 → 500 → 250 → 125 → 62 → 31 → 15 → 7 → 3 → 1     (10 halvings)
n = 1024 → 512 → 256 → 128 → 64 → 32 → 16 → 8 → 4 → 2 → 1 (10 halvings)
```

That count is `log₂ n`. **O(log n).**

For n = 1 million, that's only **20 halvings**. For n = 1 billion, **30 halvings**. *Almost free.*

A trace: how `binary_search` actually runs

Let's trace `binary_search([10, 20, 30, 40, 50, 60, 70], 7, 60)`.

Array (indices on top, values below):

```
index:    0    1    2    3    4    5    6
value:  [10] [20] [30] [40] [50] [60] [70]
```

We want to find `x = 60`. Initially: `lo = 0`, `hi = 6`.

Step 1

	value
lo	0
hi	6
mid = (0+6)/2	**3**
a[mid] = a[3]	40
40 < 60?	yes → search right half
lo = mid+1	**4**

After step 1: `lo = 4`, `hi = 6`. Search range: indices 4..6, values [50, 60, 70].

Step 2

	value
lo	4
hi	6
mid = (4+6)/2	**5**
a[mid] = a[5]	60
60 == 60?	**YES — return 5**

Done. We found 60 at index 5. **Two steps** to search 7 elements.

For 7 elements `log₂ 7 ≈ 2.8`. The actual count was 2. Matches!

Best, average, worst case

The **same algorithm** can be fast or slow depending on input.

Take "find x in unsorted array, scan from front":

```
for (int i = 0; i < n; i++)
    if (a[i] == x) return i;
return -1;
```

- **Best case**: `x == a[0]` — found in 1 step → **O(1).**
- **Worst case**: `x` not in array — scanned all `n` → **O(n).**
- **Average case** (assuming uniform): `x` somewhere in the middle → about `n/2` steps → still **O(n).**

When we say "Big-O" without qualification, we usually mean **worst case**. That's the safe, conservative bound.

What Big-O ignores (and why)

Big-O **drops constants and lower-order terms**.

Suppose your algorithm runs `5n + 10` steps. We say it's `O(n)`, **not** `O(5n + 10)`.

Why? Because: - The constant `5` depends on the machine and compiler. It changes. - The `+10` is a one-time setup cost. As `n` grows, it becomes negligible. - We want a label that says **how it scales**, not how fast it is on a specific machine.

The 5 might matter when comparing two `O(n)` algorithms in practice — but for the **theoretical class**, we ignore it.

Space complexity (memory, not time)

Same idea, but for **memory used** instead of time.

```
int sum(int *a, int n) {
    int s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return s;
}
```

Memory: just `s` and `i`. **O(1)** extra space.

```
int *copy(int *a, int n) {
    int *out = malloc(n * sizeof(int));   // n ints
    for (int i = 0; i < n; i++) out[i] = a[i];
    return out;
}
```

Memory: a new array of `n` ints. **O(n)** extra space.

```
void print_pairs(int *a, int n) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            printf("%d %d\n", a[i], a[j]);
}
```

This **takes O(n²) time** but only **O(1) extra space** (no new memory; just prints).

Time and space are independent. Always think about both.

Common confusions (resolve them now)

"Two nested loops always means O(n²)"

Not always! What if the inner loop runs only `log n` times?

```
for (int i = 0; i < n; i++)
    for (int j = 1; j < n; j *= 2)    // doubles → log n times
        do_something();
```

Outer: n times. **Inner:** log n times. Total: **O(n log n).**

"If I add an O(n) and an O(n²), what is it?"

Take the **bigger** term. `O(n + n²) = O(n²)`. The smaller term doesn't matter for big n.

"What about two functions, each O(n)?"

If they run **one after the other**: `O(n + n) = O(2n) = O(n)`. If they run **nested** (one inside the other): `O(n × n) = O(n²)`.

“Recursion — how do I see the Big-O?”

Each recursive call is a step. If `f(n)` calls `f(n-1)` once, total calls = n → **O(n)**. If `f(n)` calls `f(n-1)` twice, total calls = 2^n → **O(2^n)** (the famous bad Fibonacci). If `f(n)` calls `f(n/2)` once, total calls = log n → **O(log n)** (binary search). If `f(n)` calls `f(n/2)` twice, work doubles each level — total = O(n) (merge of merge sort).

We’ll trace these in the recursion chapter.

A picture summary

Here’s how the curves look. Imagine the X-axis is `n`, Y-axis is “time.”

```
time
↑
│                                   ●  O(n²)
│                               ●
│                           ●
│                       ●
│                   ●               ●  O(n log n)
│               ●               ●
│           ●               ●       ●  O(n)
│       ●           ●       ●
│   ●           ●       ●           ●  O(log n)
│   ●       ●       ●       ●
└──────────────────────────────────────→ n
1   2   4   8  16  32  64 128
```

O(n^2) explodes upward. O(log n) is nearly flat. O(n) is a straight line. O(n log n) is slightly above linear.

Common mistakes — what NOT to do

Mistake 1: optimizing the wrong thing

If your code is O(n^2) on n = 1,000,000, **no constant-factor tweak** (e.g., turn `*` into `<<`) will save you. You need a different **algorithm class** — go from O(n^2) to O(n log n) or O(n).

Mistake 2: forgetting the worst case

Quicksort is “O(n log n) average” — but **O(n^2) worst** when given already-sorted input with bad pivot. Production code uses introsort (quicksort + fallback) precisely because of this.

Mistake 3: thinking Big-O = fast

Big-O hides constants. Two O(n) algorithms can differ 100× in real speed. For small n, an O(n^2) with tiny constants can beat an O(n log n) with big ones. **Don’t worship Big-O — use it as a guide.**

Mistake 4: counting wrong loops

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < 10; j++)        // 10 is constant!
        do_x();
}
```

This is **O(n)**, not $O(n^2)$. The inner loop runs 10 times **regardless of n**, so it's a constant factor of the outer.

Recap (read this every day for a week)

1. **Big-O** describes how an algorithm's work grows with input size.
2. The 7 classes you must know: **O(1), O(log n), O(n), O(n log n), $O(n^2)$, $O(2^n)$, O(n!).**
3. **Drop constants and lower terms.** Focus on growth.
4. To find Big-O: count loops over n. Nested = multiply. Sequential = take max.
5. **Worst case** is what matters.
6. **Both** time and space have Big-O.

Self-check (no code-running needed — answer in your head)

1. You have a function with this code:

```
for (int i = 0; i < n; i++) printf("hi\n");
for (int i = 0; i < n; i++) printf("bye\n");
```

   What's the Big-O? *(Answer below.)*
2. Same question for:

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        printf("%d %d\n", i, j);
```

3. If an algorithm is O(n) and processes 100 items in 0.01 seconds, how long for 10,000 items?
4. If an algorithm is $O(n^2)$ and processes 100 items in 0.01 seconds, how long for 10,000 items?
5. Which is faster for n = 1,000,000: an $O(n^2)$ algorithm or an O(n log n) algorithm?

Answers:

1. **O(n).** Two sequential O(n) loops = O(n+n) = O(2n) = O(n).
2. **$O(n^2)$.** Nested loops over n.
3. **About 1 second.** Linear: 100× input → 100× time.
4. **About 100 seconds.** Quadratic: 100× input → 10,000× time.
5. **O(n log n).** For n = 1,000,000: $O(n^2) = 10^{12}$ ops, O(n log n) ≈ 2×10^7 ops. About **50,000× faster**.

If you got 4/5, you understand Big-O. If you got 5/5, move on to the next chapter.

Read deeper (if curious — not required)

- *Algorithms* by Sedgewick — has beautiful diagrams.
- "Big-O cheat sheet": https://bigocheatsheet.com/ — print and pin to your wall.

Next → DSA-02 — Arrays: the foundation.

DSA 2 — Arrays (read-to-learn version)

> RAM is one giant array. Once you understand arrays, you understand half of computer memory.

The big idea

An **array** is a row of boxes lined up next to each other in memory. Each box:

- holds **one value**,
- is **the same size** as the others,
- has a **number** (its **index**, starting at 0).

Picture (an array of 5 ints):

```
index:    0      1      2      3      4
        +-----+-----+-----+-----+-----+
value:  | 10  | 20  | 30  | 40  | 50  |
        +-----+-----+-----+-----+-----+
addr:   1000  1004  1008  1012  1016
```

Each int is 4 bytes (`sizeof(int) == 4` on most systems). The array starts at address `1000`. So `arr[0]` is at 1000, `arr[1]` is at 1004, `arr[2]` at 1008, and so on.

This is why arrays are fast: to find the address of `arr[i]`, the computer just calculates `start + i × sizeof(item)`. One multiplication, one addition. Done. **O(1).**

How operations behave (memorize)

Operation	Time	Why
`arr[i]`	**O(1)**	direct address arithmetic
`arr[i] = x`	**O(1)**	write to that address
Search for `x` (unsorted)	**O(n)**	scan everything
Search for `x` (sorted, binary search)	**O(log n)**	halve range
Insert at end (room available)	**O(1)**	just write
Insert at front	**O(n)**	shift everyone right
Delete from end	**O(1)**	decrement size
Delete from front	**O(n)**	shift everyone left

Why insert/delete at front is O(n): to insert at index 0, you must move every existing element one slot right. For 1 million elements, that's 1 million writes.

Picture: we want to insert `5` at index 0:

```
Before:
        +-----+-----+-----+-----+-----+
        | 10  | 20  | 30  | 40  | 50  |     n=5, no room
```

```
        +-----+-----+-----+-----+-----+

Step 1: shift right (start from end!)
        +-----+-----+-----+-----+-----+-----+
        | 10  | 20  | 30  | 40  | 50  |     |
        +-----+-----+-----+-----+-----+-----+
                                      ↑
                                      a[5] = a[4] = 50

        +-----+-----+-----+-----+-----+-----+
        | 10  | 20  | 30  | 40  |     | 50  |
        +-----+-----+-----+-----+-----+-----+
                                ↑
                                a[4] = a[3] = 40

        ... continues until a[1] = a[0] = 10 ...

Step 2: write
        +-----+-----+-----+-----+-----+-----+
        |  5  | 10  | 20  | 30  | 40  | 50  |     n=6
        +-----+-----+-----+-----+-----+-----+
```

That's 5 shifts + 1 write = 6 ops for n=5. For n=1,000,000 it'd be 1,000,001 ops. **O(n).**

Static vs dynamic vs growable

Static array

```
        int arr[5];
```

Size is **fixed** at compile time. Lives on the stack (auto). Free when the function returns.

Dynamic array (`malloc`)

```
        int *arr = malloc(n * sizeof(int));
        // ... use ...
        free(arr);
```

Size known at **runtime**. Lives on the heap. **You must free it.**

Growable array (vector / list)

What if you want it to grow as you add elements?

The trick: keep two numbers — `size` (how many elements you have) and `cap` (how many slots you've allocated). When `size == cap`, **double** `cap` and copy.

```
        typedef struct {
            int    *data;   // pointer to the array
            size_t  size;   // current count
            size_t  cap;    // capacity (allocated slots)
        } vec_t;
```

The growth strategy:

```
Start:     cap=4, size=0
Push:      cap=4, size=1   [a]
Push:      cap=4, size=2   [a, b]
Push:      cap=4, size=3   [a, b, c]
Push:      cap=4, size=4   [a, b, c, d]   ← FULL!
Push e:    DOUBLE cap. realloc to cap=8.   ← copy 4 elements
           cap=8, size=5   [a, b, c, d, e]
```

```
Push:      cap=8, size=6
Push:      cap=8, size=7
Push:      cap=8, size=8   ← FULL!
Push:      DOUBLE cap=16, size=9            ← copy 8 elements
```

Why doubling? It's the magic ingredient that makes `push` **O(1) on average** (amortized).

Why amortized? Most pushes are O(1) (just write into an empty slot). Once in a while you copy n items (O(n)) — but that only happens every `n` pushes. Spread the cost: each push, on average, pays only O(1).

Sum the cost of pushing n items: 1 + 2 + 4 + 8 + ... + n ≈ 2n. **Linear total → O(1) per push, amortized.**

The full vector code (read line by line)

```
void vec_init(vec_t *v) {
    v->data = NULL;
    v->size = 0;
    v->cap  = 0;
}
```

Just zero everything out. We'll allocate later.

```
int vec_push(vec_t *v, int x) {
    if (v->size == v->cap) {                  // (1) full?
        size_t newcap = v->cap == 0 ? 4       // first push: start at 4
                                    : v->cap * 2;
        int *nd = realloc(v->data, newcap * sizeof(int));
        if (!nd) return -1;                   // realloc failed
        v->data = nd;
        v->cap  = newcap;
    }
    v->data[v->size++] = x;                   // (2) write & bump size
    return 0;
}
```

Line (1): if size == cap, we're full → grow. Line (2): `v->data[v->size++] = x` does two things: store `x` at slot `size`, then increment size. Equivalent to `v->data[v->size] = x; v->size = v->size + 1;`.

Trace: pushing 5 elements

Starting: `vec_init` gives `data=NULL, size=0, cap=0`.

Push	size before	cap before	full?	new cap	data after	size after
10	0	0	yes (0==0)	**4**	[10, , , _]	1
20	1	4	no	4	[10, 20, ,]	2
30	2	4	no	4	[10, 20, 30, _]	3
40	3	4	no	4	[10, 20, 30, 40]	4
50	4	4	yes	**8**	[10, 20, 30, 40, 50, , , _]	5

Pushes 1, 5 trigger reallocations (and copies). The other three are pure writes. Average over 5 pushes: still O(1).

A multi-dimensional array (matrix)

A 2D array stored in memory:

```
int m[3][4];
```

Memory layout — **row by row** (row-major):

```
                    column index
              0    1    2    3
            +----+----+----+----+
row 0       |0,0 |0,1 |0,2 |0,3 |    addresses: 1000, 1004, 1008, 1012
            +----+----+----+----+
row 1       |1,0 |1,1 |1,2 |1,3 |    addresses: 1016, 1020, 1024, 1028
            +----+----+----+----+
row 2       |2,0 |2,1 |2,2 |2,3 |    addresses: 1032, 1036, 1040, 1044
            +----+----+----+----+
```

`m[i][j]` is at `base + (i × cols + j) × sizeof(int)`. For `m[1][2]`: `1000 + (1×4 + 2) × 4 = 1000 + 24 = 1024`. √

Why row-by-row vs column-by-column matters

The CPU loads memory in **chunks** called **cache lines** (typically 64 bytes — that's 16 ints). When you access `m[0][0]`, the CPU also pulls in `m[0][1]`, `m[0][2]`, ..., `m[0][15]` for free. They live in fast cache.

```
// FAST — sequential reads (cache-friendly)
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 4; j++)
        m[i][j] = ...;
```

Memory access pattern: 1000, 1004, 1008, 1012, 1016, 1020, ...

```
// SLOW — column-first (cache-thrashing on big matrices)
for (int j = 0; j < 4; j++)
    for (int i = 0; i < 3; i++)
        m[i][j] = ...;
```

Memory access pattern: 1000, 1016, 1032, 1004, 1020, ... — jumping by row size each time.

For a 1000×1000 matrix of ints, column-first can be **10× slower** because cache lines are wasted. **Always loop in row-major order in C.**

Worked example: reverse an array in place

Given `[10, 20, 30, 40, 50]`, we want `[50, 40, 30, 20, 10]`.

The two-pointer trick: one pointer starts at the left, one at the right. They swap and walk toward each other.

```
void reverse(int *a, int n) {
    for (int i = 0, j = n - 1; i < j; i++, j--) {
        int t = a[i];   // save left
        a[i] = a[j];    // left = right
        a[j] = t;       // right = saved
    }
}
```

Trace on `[10, 20, 30, 40, 50]` (n=5)

Step	i	j	i<j?	swap a[i]↔a[j]	array after
Before	0	4	yes	10 ↔ 50	[50, 20, 30, 40, 10]
Iter 2	1	3	yes	20 ↔ 40	[50, 40, 30, 20, 10]
Iter 3	2	2	**no** (2 < 2 false)	stop	

Done. Array is reversed. **n/2 swaps total → O(n) time, O(1) extra space.**

Why this works (the "invariant")

After each iteration, the **outermost** unprocessed pair has been swapped. Inner part is still raw. We stop when the two pointers meet (or cross). At that point, every element has been swapped to its mirror position.

Picture as we go:

```
After 0 swaps:  [10][20][30][40][50]    pointers at 0 and 4
                 ↑               ↑

After 1 swap:   [50][20][30][40][10]    pointers at 1 and 3
                     ↑      ↑

After 2 swaps:  [50][40][30][20][10]    pointers at 2 and 2 — STOP
                         ↑
```

`30` (the middle) doesn't move. That's fine — it's already at its mirror position.

Worked example: maximum subarray (Kadane's algorithm)

Find the contiguous subarray with the largest sum. **Classic interview question.**

Input: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`. Answer: `6` (from subarray `[4, -1, 2, 1]`).

The trick: at each index, ask: *"is it better to extend the previous subarray, or start fresh here?"*

```
int max_subarray(int *a, int n) {
    int best = a[0];     // best so far overall
    int cur  = a[0];     // best ending at current index
    for (int i = 1; i < n; i++) {
        cur = (a[i] > cur + a[i]) ? a[i] : cur + a[i];
        if (cur > best) best = cur;
    }
    return best;
}
```

Trace on `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`

Start: `best = -2, cur = -2`.

i	a[i]	cur + a[i]	new cur (max of two)	best
1	1	-2+1 = -1	max(1, -1) = **1**	max(-2, 1) = **1**
2	-3	1+(-3) = -2	max(-3, -2) = **-2**	max(1, -2) = 1
3	4	-2+4 = 2	max(4, 2) = **4**	max(1, 4) = **4**
4	-1	4+(-1) = 3	max(-1, 3) = **3**	max(4, 3) = 4

i	a[i]	cur + a[i]	new cur (max of two)	best
5	2	3+2 = 5	max(2, 5) = **5**	max(4, 5) = **5**
6	1	5+1 = 6	max(1, 6) = **6**	max(5, 6) = **6**
7	-5	6+(-5) = 1	max(-5, 1) = **1**	max(6, 1) = 6
8	4	1+4 = 5	max(4, 5) = **5**	max(6, 5) = 6

Final: best = 6. √

This is **O(n) time, O(1) space.** A naive O(n^2) brute force tries every (i, j) pair. Kadane's elegance is doing it in one pass.

Common bugs (drawings included)

Bug 1: off-by-one

```
for (int i = 0; i <= n; i++)        // ← should be i < n
    a[i] = ...;
```

When `i == n`, you write past the end:

```
+-----+-----+-----+-----+-----+
|  ?  |  ?  |  ?  |  ?  |  ?  |  ← legitimate slots 0..n-1
+-----+-----+-----+-----+-----+
                                ↑
                                a[n] — OUT OF BOUNDS!
```

C **does not check bounds.** You'll silently corrupt nearby memory. Maybe a crash later. Maybe a security hole.

Bug 2: dangling pointer

```
int *bad(void) {
    int local[10];
    return local;     // local dies when function returns!
}
```

`local` is on the stack of `bad`. After return, the stack frame is gone. Returned pointer points to dead memory. Touch it → undefined behavior.

Fix: allocate on the heap (`malloc`) or pass a buffer in.

Bug 3: forgetting to free

```
void leak(void) {
    int *a = malloc(1000);
    // ... use a ...
    // forgot free(a)
}
```

Every call leaks 1000 bytes. After enough calls, `malloc` fails (out of memory) or the system kills your process.

Bug 4: `sizeof(arr)` after passing to a function

```
void foo(int arr[]) {
    printf("%zu\n", sizeof(arr));    // prints 8 (a pointer), not the array size!
}

int main(void) {
    int x[100];
    foo(x);
}
```

Inside `foo`, `arr` is **a pointer**, not an array. `sizeof(pointer) == 8` on 64-bit systems. **Always pass the size as a separate argument.**

Recap

1. An array is a contiguous block of memory; element address is `base + i × sizeof(item)`.
2. Indexing is **O(1)**. Search is **O(n)** (or O(log n) if sorted + binary search).
3. Insert/delete at front is **O(n)** (shifts everyone).
4. A growable array doubles capacity → **O(1) amortized** push.
5. 2D arrays are row-major in C; **iterate row-by-row** for cache speed.
6. Common bugs: off-by-one, dangling pointers, leaks, sizeof on parameter pointer.

Self-check (no code-running needed)

1. An array of 1,000,000 ints. How long does `arr[500000]` take? *Constant or proportional to n?*
2. You insert at the **front** of a 1,000,000-element array a million times. How many total operations?
3. A growable array starts at cap=4 and doubles. After pushing 100 items, how many reallocations did you do? *(Hint: 4, 8, 16, 32, 64, 128 — count the steps.)*
4. In a 1000×1000 matrix, you sum all elements. Two loops. Which order is faster: row-then-col, or col-then-row?
5. Trace this short reverse on `[1, 2, 3]`:

   ```
   i=0, j=2 → swap a[0] ↔ a[2] → ?
   i=1, j=1 → loop ends → ?
   ```

   What is the final array?

Answers:

1. **Constant.** `arr[i]` is just one address calculation. Doesn't depend on the array size.
2. About **10^{12}** (a trillion). Each insert at front is O(n), and you do it n times: $n \times n = n^2$. For n=1,000,000 → 10^{12} operations.
3. **5 reallocations.** cap goes 4 → 8 → 16 → 32 → 64 → 128. (At each double, you copy the old contents.)
4. **Row-then-col.** Cache-friendly. (See the matrix section above.)
5. After step 1: `[3, 2, 1]`. Step 2 doesn't run (i==j → loop condition false). **Final: [3, 2, 1].** ✓

If you got 4/5, you understand arrays. If you got 5/5, move on.

Read deeper (optional)

- *The C Programming Language* (K&R), chapter on arrays.
- “What every programmer should know about memory” (Drepper), section on cache.

Next → DSA-03 — Strings.

DSA 3 — Strings (read-to-learn version)

> A C string is "an array of characters that ends with a special byte." That's the secret.

The big idea

In C, a **string** is just an array of `char` with one rule: **the last character is `'\0'`** (the null terminator — a byte whose value is zero).

Picture the string `"hi"` in memory:

```
        +----+----+----+
chars:  | h  | i  |\0  |
        +----+----+----+
addr:    1000 1001 1002
```

That's **3 bytes**, not 2! The `\0` is real, takes one byte, and tells `printf`, `strlen`, and friends "the string ends here."

In ASCII, `'h' = 104`, `'i' = 105`, `'\0' = 0`. So in raw bytes:

```
        +-----+-----+-----+
        | 104 | 105 |  0  |
        +-----+-----+-----+
```

`printf("%s", s)` prints characters starting at `s` until it hits a `0` byte.

Two ways to write strings in C

Way 1: read-only string literal

```
        char *p = "hello";
```

The text `"hello"` is in **read-only memory** (the program's `.rodata` section). `p` just points to it.

```
                        (read-only memory)
                  ┌──────────────────────────┐
                  ↓
            +---+---+---+---+---+---+
p ──►       | h | e | l | l | o |\0 |
            +---+---+---+---+---+---+
```

You **cannot** modify `p[0] = 'H'` — that's writing to read-only memory. Often a SIGSEGV crash.

Way 2: writable array

```
        char a[] = "hello";
```

This **copies** the literal into an array on the stack. Now you have your own private copy:

```
                        (stack — writable)
            +---+---+---+---+---+---+
a ──►       | h | e | l | l | o |\0 |
            +---+---+---+---+---+---+
```

`a[0] = 'H';` works. Result: `"Hello"`.

Rule of thumb: if you intend to modify, use `char arr[]`, not `char *p`.

strlen — how long is the string?

```
size_t my_strlen(const char *s) {
    const char *p = s;
    while (*p) p++;       // walk until we hit '\0'
    return p - s;         // bytes walked
}
```

`while (*p)` reads as: "while the byte pointed to by `p` is non-zero." A `\0` byte is zero, so the loop ends there.

Trace on `"hi"` (s points to address 1000)

Step	p (addr)	*p (the char)	*p non-zero?	action
start	1000	'h' (104)	yes	p++
	1001	'i' (105)	yes	p++
	1002	'\0' (0)	**no**	exit loop

After loop: `p = 1002`, `s = 1000`. Return `p - s = 2`. **Length is 2** (excluding the null terminator). √

Cost: O(n). `strlen` always **scans the whole string.** Calling `strlen` inside a loop over a string is a classic performance bug — O(n) inside O(n) = O(n^2).

strcmp — compare two strings

```
int my_strcmp(const char *a, const char *b) {
    while (*a && *a == *b) {
        a++;
        b++;
    }
    return (unsigned char)*a - (unsigned char)*b;
}
```

Walk through both strings together. Stop when either: - `*a` is `'\0'` (a is done), or - `*a != *b` (mismatch).

Return the difference of the bytes where they differed (or 0 if equal).

Trace: compare `"cat"` and `"car"`

Step	a points to	b points to	match?	action
start	'c'	'c'	yes	both++
	'a'	'a'	yes	both++

Step	a points to	b points to	match?	action
	't'	**'r'**	NO	exit loop

Return `(int)'t' - (int)'r' = 116 - 114 = 2`. **Positive → "cat" > "car"** lexicographically.

Trace: compare `"abc"` and `"abc"`

Step	a points to	b points to	match?	action
start	'a'	'a'	yes	both++
	'b'	'b'	yes	both++
	'c'	'c'	yes	both++
	'\0'	'\0'	yes (but `*a` is 0)	exit loop

Return `0 - 0 = 0`. **Equal.**

The convention: `strcmp` returns **0** for equal, **negative** if `a < b`, **positive** if `a > b`.

The dangerous library functions (and the safe ones)

`strcpy` — DANGEROUS

```
char *strcpy(char *dst, const char *src);
```

Copies `src` into `dst` until `\0`. **No check** that `dst` is big enough. Classic buffer overflow:

```
char dst[5];
strcpy(dst, "hello world");      // "hello world" is 12 bytes
```

Picture the disaster:

```
Stack:
       +---+---+---+---+---+
dst:   | h | e | l | l | o |     ← 5 bytes; out of room
       +---+---+---+---+---+
        | w | o | r | l | d | \0 |  ← writes past end into the next variable!
        +---+---+---+---+---+----+
        ↑ corrupts whatever was here
```

This is how decades of security holes were created. **Avoid `strcpy`.**

`snprintf` — SAFE

```
char dst[5];
snprintf(dst, sizeof(dst), "%s", "hello world");
```

`snprintf` won't write more than `sizeof(dst)` bytes (and always null-terminates inside that limit). Result: `dst = "hell"` (truncated to 4 chars + `\0`).

Truncation may not be **right**, but it's not a security hole. **Use `snprintf` for every "build a string" task.**

`strscpy` (Linux kernel)

The kernel's preferred copy. Always null-terminates; returns the length copied (or `-E2BIG` on truncation). Read `lib/string.c` once you finish Part V.

Common string algorithms (traced)

Reverse a string in place

```
void reverse_str(char *s) {
    size_t n = strlen(s);
    for (size_t i = 0, j = n - 1; i < j; i++, j--) {
        char t = s[i];
        s[i] = s[j];
        s[j] = t;
    }
}
```

Same two-pointer pattern as the array reverse from chapter 2.

Trace on `"hello"` (n=5)

```
Before: [h][e][l][l][o]
         ↑           ↑
         i=0         j=4
```

Step	i	j	swap	array after
1	0	4	h ↔ o	[o][e][l][l][h]
2	1	3	e ↔ l	[o][l][l][e][h]
3	2	2	i<j false → stop	(no change)

Result: "olleh" √

Check palindrome

```
int is_palindrome(const char *s) {
    size_t n = strlen(s);
    for (size_t i = 0, j = n - 1; i < j; i++, j--)
        if (s[i] != s[j]) return 0;
    return 1;
}
```

Same two pointers. Instead of swapping, **check if mirror chars match**.

Trace on `"racecar"` (n=7)

Step	i	j	s[i]	s[j]	match?
1	0	6	'r'	'r'	yes
2	1	5	'a'	'a'	yes
3	2	4	'c'	'c'	yes
4	3	3	i<j false → stop		

Return `1` (palindrome). √

Trace on `"hello"` (n=5)

Step	i	j	s[i]	s[j]	match?
1	0	4	'h'	'o'	**NO** → return 0

Not a palindrome. √

Anagram check (count of each char)

Two strings are anagrams if they have the same letters with the same counts (any order). E.g., `"listen"` and `"silent"`.

The trick: a **frequency table** of size 256 (one slot per possible byte). Increment for the first string, decrement for the second. If all counts end at zero, they're anagrams.

```
int is_anagram(const char *a, const char *b) {
    int cnt[256] = {0};
    while (*a) cnt[(unsigned char)*a++]++;
    while (*b) cnt[(unsigned char)*b++]--;
    for (int i = 0; i < 256; i++)
        if (cnt[i] != 0) return 0;
    return 1;
}
```

Trace on `"abc"` vs `"cab"`

After phase 1 (count `"abc"`):

```
cnt['a'] = 1
cnt['b'] = 1
cnt['c'] = 1
all others = 0
```

After phase 2 (decrement for `"cab"`):

```
cnt['c']-- → 0
cnt['a']-- → 0
cnt['b']-- → 0
```

All zero → **anagrams.** √

Trace on `"abc"` vs `"abd"`

After phase 1: `cnt['a']=1, cnt['b']=1, cnt['c']=1`.

After phase 2: `cnt['a']--=0, cnt['b']--=0, cnt['d']--=-1`, but **cnt['c'] still 1**.

Final scan: cnt['c']=1, cnt['d']=−1 → both non-zero → **not anagrams.** √

Time: O(n + m). **Space:** O(1) (256 is constant — alphabet size). Beautiful.

Convert string to integer (safely)

`atoi("42")` returns 42 — **but** if the input is `"abc"`, `atoi` silently returns 0, with no way to detect the error. Use `strtol` instead:

```
int parse_int(const char *s, int *out) {
    char *end;
    errno = 0;
    long v = strtol(s, &end, 10);
    if (errno != 0)        return -1;       // overflow
    if (end == s)          return -1;       // no digits at all
    if (*end != '\0')      return -1;       // trailing garbage
    if (v < INT_MIN || v > INT_MAX) return -1;
    *out = (int)v;
    return 0;
}
```

`end` is set by `strtol` to "the first character it didn't consume." Check it to validate.

Input	v	end points to	Result
"42"	42	'\0'	OK
"42abc"	42	'a'	reject (trailing garbage)
"abc"	0	'a' (= s)	reject (no digits)
""	0	'\0' (= s)	reject (empty)
"99999999999999"	LONG_MAX	'\0'	reject (errno set)

Defensive parsing. Always validate input.

Pointer-to-literal vs array (review with picture)

```
char *p = "hello";       // pointer to read-only memory
char a[] = "hello";      // copy of literal in writable memory
```

```
(read-only)                    (stack)
+---+---+---+---+---+---+      +---+---+---+---+---+---+
| h | e | l | l | o |\0 |      | h | e | l | l | o |\0 |
+---+---+---+---+---+---+      +---+---+---+---+---+---+
↑                              ↑
p                              a
```

`p[0] = 'H'` → **crash**. `a[0] = 'H'` → fine. Different worlds.

Common bugs (with pictures)

Bug 1: forgetting the null terminator

You allocate a buffer of length `n` and write `n` characters into it. **Where's the `\0`?** Nowhere. `strlen` will run past the end.

```
char buf[5];
buf[0] = 'h'; buf[1] = 'e'; buf[2] = 'l'; buf[3] = 'l'; buf[4] = 'o';
printf("%s\n", buf);                  // ← reads past the end!
```

```
        +---+---+---+---+---+
buf:    | h | e | l | l | o |  ← no '\0'!
        +---+---+---+---+---+---+---+---+...
                              | ? | ? | ? |  whatever junk is here
```

`printf` keeps printing until it accidentally hits a zero byte somewhere in junk.

Always reserve room for `\0`. A buffer for an n-character string needs **n+1** bytes.

Bug 2: comparing strings with `==`

```
char *a = "abc";
char *b = "abc";
if (a == b) ...    // compares POINTERS, not contents
```

`a == b` checks if they point to the **same** address. Sometimes the compiler shares string literals (so this might "work" by accident). **Don't rely on it.** Use `strcmp(a, b) == 0`.

Bug 3: modifying a literal

```
char *s = "hello";
s[0] = 'H';        // CRASH: literal is in read-only memory
```

We saw this above. Fix: `char s[] = "hello";`.

Bug 4: `strcat` overflow

```
char dst[10] = "hello";
strcat(dst, " world");        // "hello world" is 12 chars; only 10 in dst
```

`strcat` blindly appends past the end. Same disaster as `strcpy`. Use `snprintf` or `strscpy`.

Recap

1. A C string = char array ending in `\0`.
2. **Length is bytes-to-`\0`-not-counting-`\0`.** Buffer must be at least length+1 bytes.
3. **Avoid** `strcpy`, `strcat`, `sprintf`, `gets`, `atoi`. **Use** `snprintf`, `strscpy`, `strtol`.
4. Pointer-to-literal is read-only. Array-from-literal is writable.
5. Common patterns: two pointers, frequency tables.

Self-check (answer in your head)

1. How many bytes does the string `"go"` occupy?
2. Trace `is_palindrome("aa")` step by step. What does it return?
3. What's the time complexity of:

   ```
   for (int i = 0; i < strlen(s); i++) ...
   ```

   (Hint: `strlen(s)` is called every iteration!)
4. After this code:

   ```
   char a[10] = "abc";
   ```

   What's at `a[3]`? `a[5]`? `a[10]`?
5. Why does `is_anagram` use `unsigned char` when indexing `cnt`?

Answers:

1. **3 bytes**: `'g'`, `'o'`, `'\0'`.
2. n=2. i=0, j=1. s[0]='a', s[1]='a' → match. i=1, j=0 → loop ends (i<j false). Return 1 (palindrome). √
3. **O(n²).** Each iteration calls `strlen`, which is O(n). The loop runs n times. n × n = n^2. Fix: compute length once before the loop.
4. `a[3]` = `'\0'`. `a[5]` = `'\0'` (uninitialized portion is zeroed for `char a[10] = ...` — array initialization rule). `a[10]` is **out of bounds** (valid indices are 0..9). Don't read it.

5. Because `char` may be signed on some platforms. With negative values, `cnt[(int)c]` could index out of bounds (negative). Casting to `unsigned char` ensures the index is in [0, 255].

If you got 4/5, you understand strings well. If 5/5, move on.

Next → DSA-04 — Linked lists.

DSA 4 — Linked lists (read-to-learn version)

> A linked list is a chain of "boxes," each pointing to the next. The Linux kernel uses linked lists in **thousands of places**.

The big idea

In an **array**, all elements live next to each other in memory. To find element 5, you skip ahead 5 slots (constant time).

In a **linked list**, elements live **anywhere** in memory. Each element has a value **plus a pointer** to the next element. To find element 5, you must follow 5 pointers (linear time).

```
Array (contiguous):

  +-----+-----+-----+-----+-----+
  | 10  | 20  | 30  | 40  | 50  |
  +-----+-----+-----+-----+-----+
  ↑
  one block of memory


Linked list (scattered):

  head ──▶ [10|next]──▶ [20|next]──▶ [30|next]──▶ [40|next]──▶ [50|next]──▶ NULL
           (addr 1000)  (addr 2540)  (addr 700)   (addr 9020)  (addr 4400)

  Each node is its own little allocation, anywhere in memory.
  They're connected only via the next pointer.
```

The dashes `──▶` are **pointers** — addresses of the next box.

`head` is a single pointer that holds the address of the first node. To walk the list, you follow `head → next → next → next → ... → NULL`. **NULL** marks the end.

When linked lists win, when arrays win

	Array	Linked list
Random access (`arr[i]`)	**O(1)**	**O(n)** (must walk)
Insert at front	**O(n)** (shift all)	**O(1)** (just relink)
Delete a node you have a pointer to	**O(n)** (shift)	**O(1)** (relink)
Memory layout	contiguous, cache-friendly	scattered, cache-unfriendly
Memory overhead per element	none	one pointer per node (8 bytes on 64-bit)

Use a linked list when you do many insertions/deletions in the middle and have direct pointers to nodes. **Use an array** for random access or tight loops where cache speed matters.

The struct that defines a node

```
typedef struct node {
    int val;
    struct node *next;
} node_t;
```

A node holds a value (here, an int) and a pointer to the next node.

In memory each node looks like:

```
+----+--------+
| val|  next  |   ← 16 bytes total (4 + 4 padding + 8 ptr on x86_64)
+----+--------+
  ↑       ↑
  data    pointer to next node (or NULL)
```

Building a list, step by step

We want to build the list `[10, 20, 30]` by **pushing each at the front**.

Step 1: empty list

```
node_t *head = NULL;
```

```
head ──► NULL
```

Step 2: push 30 at the front

```
node_t *n1 = malloc(sizeof(node_t));
n1->val  = 30;
n1->next = head;     // currently NULL
head     = n1;
```

```
head ──► [30|next]──► NULL
```

Step 3: push 20 at the front

```
node_t *n2 = malloc(sizeof(node_t));
n2->val  = 20;
n2->next = head;     // points to the 30 node
head     = n2;
```

Picture:

```
                +---------+        +---------+
head ──►  n2 ──► |20 | next|──► n1 ──►|30 | next|──► NULL
                +---------+        +---------+
```

`head` now points to n2, which points to n1, which points to NULL.

Step 4: push 10 at the front

```
node_t *n3 = malloc(sizeof(node_t));
n3->val  = 10;
n3->next = head;     // points to the 20 node
head     = n3;
```

Final:

```
head ──► [10|next]──► [20|next]──► [30|next]──► NULL
```

That's the list `[10, 20, 30]`. **3 mallocs**, no array resizing, ever.

list_push_front (in code)

```
int list_push_front(node_t **head, int v) {
    node_t *n = malloc(sizeof(node_t));   // (1) allocate
    if (!n) return -1;
    n->val  = v;                          // (2) fill in value
    n->next = *head;                      // (3) link to old head
    *head   = n;                          // (4) update head
    return 0;
}
```

Why `node_t **head` (pointer to pointer)? Because we need to **change** what `head` points to. If we wrote `node_t *head` (single pointer), we'd just modify a copy of the pointer — the caller's `head` wouldn't change.

Trace: push 10, 20, 30 onto an empty list

We track `head` (caller's variable) and `*n` (newly allocated node) at each step.

Push	Action	Memory after
start	head = NULL	`head → NULL`
30	malloc node A; A.val=30; A.next=NULL; head=A	`head → [30,NULL]`
20	malloc node B; B.val=20; B.next=A; head=B	`head → [20]→[30,NULL]`
10	malloc node C; C.val=10; C.next=B; head=C	`head → [10]→[20]→[30,NULL]`

Walk the list (find a value)

```
int list_contains(node_t *head, int v) {
    for (node_t *p = head; p != NULL; p = p->next)
        if (p->val == v)
            return 1;
    return 0;
}
```

`p` walks: starts at `head`, then `p->next`, then `p->next->next`, until `NULL`.

Trace: search for 20 in `[10, 20, 30]`

Step	p points to	p->val	match?	next action
start	node holding 10	10	no	p = p->next
	node holding 20	20	**yes**	return 1

Two pointer-follows. **O(n)** time.

Trace: search for 99

Step	p points to	p->val	match?	next action
start	node holding 10	10	no	p = p->next
	node holding 20	20	no	p = p->next
	node holding 30	30	no	p = p->next
	NULL			exit loop

Return 0 (not found). Walked the whole list — **n** pointer-follows.

Reverse a linked list (the famous interview question)

Given `[10, 20, 30] → NULL`, return `[30, 20, 10] → NULL`.

We use **three pointers**: `prev`, `cur`, `next`. The trick: at each step, **flip cur's next pointer** to point to `prev` instead of forward.

```
node_t *reverse(node_t *head) {
    node_t *prev = NULL;
    node_t *cur  = head;
    while (cur != NULL) {
        node_t *next = cur->next;   // (1) save the next node before we lose it
        cur->next = prev;            // (2) flip the arrow backward
        prev = cur;                  // (3) advance prev
        cur  = next;                 // (4) advance cur
    }
    return prev;                    // new head
}
```

Trace step by step on `[10] → [20] → [30] → NULL`

We'll show `prev`, `cur`, `next` at each iteration, plus the **state of links**.

Initial state:

```
prev = NULL
cur  ──▶ [10]──▶[20]──▶[30]──▶NULL
```

Iteration 1:

```
next = cur->next;     // next → [20]
cur->next = prev;     // [10] → NULL
prev = cur;           // prev → [10]
cur  = next;          // cur  → [20]
```

```
After iter 1:
        [10]──▶NULL                    (broken off)
prev ──▶ [10]
cur  ──▶ [20]──▶[30]──▶NULL
```

Iteration 2:

```
        next = cur->next;     // next → [30]
        cur->next = prev;     // [20] → [10]
        prev = cur;           // prev → [20]
        cur  = next;          // cur  → [30]
```

```
After iter 2:
        [20]──▶[10]──▶NULL            (the reversed part so far)
prev ──▶ [20]
cur  ──▶ [30]──▶NULL
```

Iteration 3:

```
        next = cur->next;     // next = NULL
        cur->next = prev;     // [30] → [20]
        prev = cur;           // prev → [30]
        cur  = next;          // cur  = NULL
```

```
After iter 3:
[30]──▶[20]──▶[10]──▶NULL
prev ──▶ [30]
cur  = NULL              ← loop exits
```

Return `prev` = `[30] → [20] → [10] → NULL`. **Reversed!** √

Why each line of code matters

Line	Why we need it
`next = cur->next`	We're about to overwrite `cur->next`. Save it first or we lose the rest of the list.
`cur->next = prev`	The actual reversal — flip this arrow backward.
`prev = cur`	Move `prev` forward to the just-reversed node.
`cur = next`	Move `cur` forward to keep walking.

Time: O(n). Space: O(1). **Memorize this code.** It's asked in every interview.

Detect a cycle (Floyd's tortoise & hare)

If a list has a cycle (some `next` loops back to a previous node), naive walk **never ends**.

The cycle detector uses **two pointers**: slow (steps 1) and fast (steps 2). If there's a cycle, the fast catches up to slow and they meet inside the loop.

```
        int has_cycle(node_t *head) {
            node_t *slow = head, *fast = head;
            while (fast && fast->next) {
                slow = slow->next;
                fast = fast->next->next;
                if (slow == fast) return 1;     // they met → cycle
            }
            return 0;                           // fast hit NULL → no cycle
        }
```

Why fast catches slow

Imagine you're on a circular running track. Two runners start together — one runs at 1 lap/min, one at 2 laps/min. After enough time, the fast runner laps the slow one. They'll be at the same point.

Trace on `[10] → [20] → [30] → [40] → [20]` (cycle at 20)

```
[10] → [20] → [30] → [40] ─┐
         ↑                 │
         └─────────────────┘
```

Step	slow at	fast at	slow == fast?
start	10	10	yes (but we haven't started looping; first move)
1	20	30	no
2	30	20	no
3	40	40	**yes — return 1**

Three steps, cycle found. √

Trace on a list without a cycle

`[10] → [20] → [30] → NULL`

Step	slow at	fast at
1	20	30
2	30	NULL

`fast == NULL` → loop exits, return 0. √

Doubly linked list

Each node has **both** `prev` and `next`:

```
typedef struct dnode {
    int val;
    struct dnode *prev, *next;
} dnode_t;
```

Picture (5 nodes connected in both directions):

```
NULL←┌──────┐┌──────┐┌──────┐┌──────┐┌──────┐
     │  10  ├←┤  20  ├←┤  30  ├←┤  40  ├←┤  50  ├ → NULL
     │      │→│      │→│      │→│      │→│      │
     └──────┘└──────┘└──────┘└──────┘└──────┘

       ↑                                ↑
       head                             tail
```

Why bother with double the pointers?

- O(1) deletion of a known node (no need to scan to find its predecessor).
- O(1) walk backward.

Cost: one extra pointer per node = 8 more bytes. For million-node lists with tiny payloads, that's 8 MB extra. Trade-off.

The Linux kernel's `list_head` (the most important DS in the kernel)

In `include/linux/list.h`:

```
struct list_head {
    struct list_head *next, *prev;
};
```

That's the **whole** struct: just two pointers, no data field! How can a list contain things if the node has no data?

The trick: you embed `list_head` inside your own struct.

```
struct task {
    int pid;
    char name[16];
    struct list_head node;   // embedded link
};
```

Now `struct task` has its own data (`pid`, `name`) AND a `list_head` field for joining lists.

A list of tasks looks like:

```
list_head head ──► [task#1.node] ──► [task#2.node] ──► [task#3.node] ──► back to head

Where each task has more fields:

   task#1: { pid=100, name="bash",  node = {prev,next} }
   task#2: { pid=200, name="vim",   node = {prev,next} }
   task#3: { pid=300, name="cat",   node = {prev,next} }
```

The `next`/`prev` pointers chain together the `node` field of each task — not the task itself.

So how do we recover the **whole task** from a `list_head` pointer? With a tiny piece of arithmetic magic: `container_of`.

container_of — the kernel's most-used macro

```
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))
```

It says: "given a pointer to the `member` field, subtract the offset of `member` within `type` to get a pointer to the whole `type`."

Picture:

```
struct task layout in memory (offsets in bytes):

  offset 0   :  pid           (4 bytes)
  offset 4   :  padding        (4 bytes)
  offset 8   :  name[16]      (16 bytes)
  offset 24  :  node           (16 bytes - prev + next pointers)
  total size :  40 bytes

If you have ptr → node (offset 24 of some task instance),
then `ptr - 24` gives you ptr → start of that task instance.
```

So `container_of(node_ptr, struct task, node)` returns the task that owns that `node`.

This lets you embed `list_head` in **any** struct without templates or generics. **One linked-list implementation works for everything.** Brilliant.

Walking a kernel list

```
struct task *t;
list_for_each_entry(t, &mylist, node) {
    pr_info("pid=%d\n", t->pid);
}
```

The macro `list_for_each_entry` expands to a `for` loop that: - starts at `mylist.next`, - on each iteration, applies `container_of` to recover the `struct task`, - assigns to `t`, - stops when we cycle back to `&mylist`.

You write your code as if iterating tasks; the macro handles the link arithmetic.

Why this matters for you

Once you understand `list_head` + `container_of`, **half the kernel reads like prose**. Drivers, schedulers, the GPU stack — all use this pattern.

Iterating while deleting (the classic trap)

```
// WRONG
for (node_t *p = head; p; p = p->next) {
    free(p);              // we just freed p; reading p->next is undefined
}
```

After `free(p)`, the memory at `p` is gone. Reading `p->next` is reading freed memory — could crash, could return garbage.

```
// RIGHT
node_t *p = head;
while (p) {
    node_t *next = p->next;   // save before freeing
    free(p);
    p = next;
}
```

Save the next pointer **before** freeing the current node.

The kernel's `list_for_each_entry_safe` does this for you. Always use the `_safe` variant when deleting during iteration.

Common bugs (with pictures)

Bug 1: lost head

```
node_t *head = ...;
head = head->next;       // we just lost the original head — leak!
```

```
Before:  head ──▶ [10]──▶[20]──▶[30]
After:   head ──▶ [20]──▶[30]

                  [10]   ← still in memory, but unreachable! LEAK
```

Bug 2: traversal off the end

```
        for (node_t *p = head; p->next != NULL; p = p->next) ...   // wrong condition
```

If `head` is NULL, you dereference NULL on iteration zero. **Crash.**

Right condition: `p != NULL`.

Bug 3: cycle introduction

```
        node_t *a = ..., *b = ...;
        a->next = b;
        b->next = a;             // cycle! list_for_each_* will loop forever
```

Always reason about whether your `next` writes break or create cycles.

Recap

1. A linked list is a chain of nodes, each with value + next pointer.
2. **Push at front: O(1).** Search/index/insert-in-middle (without a pointer to the spot): O(n).
3. **Three operations to memorize**: walk the list, reverse the list, detect a cycle (Floyd's).
4. The Linux kernel uses **embedded** `list_head` + `container_of` — read `include/linux/list.h` once you understand chapter 4.
5. When deleting during iteration, **save next first** (or use `_safe` variant).

Self-check (in your head)

1. You have a 100-node linked list. To access the 50th node, how many pointer follows?
2. Trace `reverse([1, 2])` step by step. What's the final list?
3. In a singly linked list, you have a pointer to the **second-to-last** node. How fast can you delete the **last** node? (O(1) or O(n)?)
4. In the same setup but **doubly** linked, how fast can you delete the **last** node?
5. Why does `container_of` not need to know the type of the field — only the type of the containing struct and the field name?

Answers:

1. **49 pointer follows** (start at head; that's node 1; follow 49 times to reach node 50). Linear in position.
2. After iter 1: prev → [1] → NULL, cur → [2]. After iter 2: prev → [2] → [1] → NULL, cur = NULL. **Final: [2, 1]** √
3. **O(1).** You have the prev; just set `prev->next = NULL`. (Don't forget to free the last node.)
4. Also **O(1).** And easier — the last node points back via `prev`, so given a pointer to the last node, you fix `last->prev->next = NULL` and free.
5. Because `offsetof(type, member)` computes the byte offset at **compile time** — that's all that's needed. The pointer math doesn't care about the field's type, only its offset.

If you got 4/5 you understand linked lists. If 5/5, move on.

Next → DSA-05 — Stacks and queues.

DSA 5 — Stacks and queues (read-to-learn version)

> Two of the simplest data structures, used everywhere from function calls to GPU command rings.

The big idea

A **stack** is a pile of plates. You can only: - put a plate on **top** (push), - take a plate from the **top** (pop).

The plate at the bottom is the one you put down first. Last in, first out — **LIFO**.

A **queue** is a line at a coffee shop. Customers: - join at the **back** (enqueue), - leave from the **front** (dequeue).

The first customer to join is the first to be served. First in, first out — **FIFO**.

Stack: pictures and operations

```
Stack:

        | 30 |   ← top (just pushed)
        | 20 |
        | 10 |   ← bottom (pushed first)
        +----+

operations:
  push(x):  put x on top
  pop():    remove and return the top
  peek():   look at the top without removing
```

Trace: push 10, 20, 30, then pop, pop

Step	Operation	Stack (top on right)
0	(empty)	`[ ]`
1	push(10)	`[ 10 ]`
2	push(20)	`[ 10, 20 ]`
3	push(30)	`[ 10, 20, 30 ]`
4	pop() → returns 30	`[ 10, 20 ]`
5	pop() → returns 20	`[ 10 ]`

Last in (30) was first out. That's LIFO.

Stack as an array

The simplest implementation: an array plus a `size` counter telling you where the top is.

```
typedef struct {
    int *data;
    size_t size;     // number of items currently in stack
    size_t cap;
} stack_t;
```

Picture:

```
data: [ 10 | 20 | 30 |  ?  |  ?  |  ?  ]
         0    1    2    3     4     5
                      ↑
                      size = 3 (next push goes to index 3)
```

`data[size - 1]` is the top. Push: `data[size++] = x`. Pop: `return data[--size]`.

```
void s_push(stack_t *s, int x) {
    if (s->size == s->cap) {
        s->cap = s->cap ? s->cap * 2 : 8;
        s->data = realloc(s->data, s->cap * sizeof(int));
    }
    s->data[s->size++] = x;
}

int s_pop(stack_t *s) {
    return s->data[--s->size];
}

int s_peek(stack_t *s) {
    return s->data[s->size - 1];
}
```

Push and pop are both **O(1)** (amortized for push because of growing).

Why stacks matter — the call stack

Every running program has a **call stack** — a literal stack of "where was I" markers.

When function A calls B, the CPU **pushes** A's return address onto the stack. When B returns, it **pops**. If B calls C, C pushes too.

Picture during nested calls A → B → C:

```
Stack of return addresses (and local variables for each call):

   |  C's locals  |   ← top (the function currently running)
   |  C's return  |
   |  B's locals  |
   |  B's return  |
   |  A's locals  |
   |  A's return  |   ← bottom (the original caller)
   +--------------+
```

When C returns, its frame pops. We're back in B. When B returns, it pops. We're back in A.

Recursion **piles up** stack frames — too deep, and you blow the stack (`Segmentation fault`).

Classic problem: balanced brackets

Given a string like `({[]})`, decide if every opening bracket has a matching close in the right order.

Idea: push each opener; on close, pop and check it matches.

```c
int balanced(const char *s) {
    char st[1024];
    int top = 0;
    for (; *s; s++) {
        char c = *s;
        if (c == '(' || c == '[' || c == '{') {
            st[top++] = c;
        } else {
            if (top == 0) return 0;       // close without matching open
            char o = st[--top];
            if (c == ')' && o != '(') return 0;
            if (c == ']' && o != '[') return 0;
            if (c == '}' && o != '{') return 0;
        }
    }
    return top == 0;                       // all openers matched
}
```

Trace on `"({[]})"`

`top` starts at 0. We track the stack as a list (top on right).

char	type	action	stack
`(`	open	push	`[ ( ]`
`{`	open	push	`[ (, { ]`
`[`	open	push	`[ (, {, [ ]`
`]`	close	pop `[`, matches	`[ (, { ]`
`}`	close	pop `{`, matches	`[ ( ]`
`)`	close	pop `(`, matches	`[ ]`

End of string. Stack is empty → return 1. **Balanced.** √

Trace on `"([)]"`

char	type	action	stack
`(`	open	push	`[ ( ]`
`[`	open	push	`[ (, [ ]`
`)`	close	pop `[`. close is `)`, open is `[`. Mismatch!	return 0

Not balanced. √

Queue: pictures and operations

```
front                                 back
↓                                        ↓
[ A | B | C | D | E ]

dequeue():  remove A from front
enqueue(F): add F to back
```

After dequeue + enqueue:

```
front                                 back
↓                                        ↓
[ B | C | D | E | F ]
```

The naive queue (and why it's slow)

If you implement a queue as a normal array with `enqueue = append, dequeue = remove front`, **dequeue is O(n)** because you have to shift every remaining element left.

```
Before dequeue:  [ A | B | C | D | E ]
                   ↑
                   front
After dequeue:   [ B | C | D | E |   ]      shifted everyone left

That shift takes n-1 writes. O(n) per dequeue. Bad.
```

Circular queue (ring buffer) — fast queue

The fix: don't shift. Use a **head** index for the front, a **tail** index for the back. **Wrap around** when you reach the end of the array.

```
typedef struct {
    int *buf;
    size_t cap;
    size_t head, tail, size;
} ring_t;
```

Picture (cap = 5):

```
index:    0    1    2    3    4
        +----+----+----+----+----+
buf:    | A  | B  | C  |    |    |
        +----+----+----+----+----+
         ↑               ↑
         head=0          tail=3
         size = 3
```

Enqueue D: write to `buf[tail]`, then `tail = (tail+1) % cap`.

```
buf:    | A  | B  | C  | D  |    |
                             ↑
                             tail=4
         head=0
         size = 4
```

Enqueue E: tail wraps from 4 to (4+1)%5 = 0… wait no, **tail goes to 5%5 = 0** after writing index 4.

Actually let me redo this carefully.

```
state:
         head=0, tail=4, size=4   buf = [A, B, C, D, _]

enqueue(E):
         buf[4] = E                  buf = [A, B, C, D, E]
         tail = (4+1) % 5 = 0        head=0, tail=0, size=5  ← FULL
```

Now the ring is full. `head == tail` and `size == cap`.

Dequeue: read `buf[head]`, then `head = (head+1) % cap`.

```
dequeue() → A
         head = (0+1) % 5 = 1        buf = [_, B, C, D, E]   head=1, tail=0, size=4
dequeue() → B
         head = 2                    buf = [_, _, C, D, E]   head=2, tail=0, size=3
```

Notice: front is at `head=2`. Back is at `tail=0`. The "front-to-back" range **wraps around** the array. That's the magic of a ring buffer.

Enqueue F:

```
buf[0] = F                  buf = [F, _, C, D, E]
tail = 1                    head=2, tail=1, size=4
```

The data F goes back at index 0 because that slot is free.

The full code

```
int r_enq(ring_t *r, int v) {
    if (r->size == r->cap) return -1;        // full
    r->buf[r->tail] = v;
    r->tail = (r->tail + 1) % r->cap;
    r->size++;
    return 0;
}

int r_deq(ring_t *r, int *out) {
    if (r->size == 0) return -1;             // empty
    *out = r->buf[r->head];
    r->head = (r->head + 1) % r->cap;
    r->size--;
    return 0;
}
```

Both **O(1).** No shifting. The only overhead: a modulo.

If `cap` is a **power of two**, you can replace `% cap` with `& (cap - 1)` — same result, one cheaper instruction. The kernel does this everywhere.

Where you see queues in real systems

- **Print job queue** — first job submitted prints first.
- **Network packet buffers** — packets in, packets out, FIFO.
- **GPU command rings** (`amdgpu_ring`) — CPU writes commands, GPU reads them. **Literally a ring buffer.**
- **Interrupt-handler ring** in `amdgpu_irq.c` — hardware writes events, CPU drains.
- **Producer/consumer pipelines** — one thread fills the queue, another drains it.
- **Linux's `kfifo`** — kernel ring-buffer macros. Read `include/linux/kfifo.h`.

SPSC ring buffer — fast queue between two threads

If exactly **one** thread enqueues and **one** thread dequeues, you can write a queue **without locks** using only atomic head/tail indices. This is called **single-producer single-consumer** (SPSC).

```
// producer thread:
size_t t = atomic_load(&tail);
if ((t + 1) % cap == atomic_load(&head)) return FULL;     // full
buf[t] = item;
atomic_store(&tail, (t + 1) % cap);                       // publish

// consumer thread:
size_t h = atomic_load(&head);
if (h == atomic_load(&tail)) return EMPTY;
*out = buf[h];
atomic_store(&head, (h + 1) % cap);
```

Why this works: - Only the producer writes `tail`. - Only the consumer writes `head`. - The producer reads `head` to check fullness; the consumer reads `tail` to check emptiness. - No two threads write the same variable → no need for a mutex.

The actual code needs **memory barriers** (acquire/release) for correctness on weak-memory-ordering CPUs (ARM). We'll cover this in Part IV (multithreading). For now, just absorb the **shape**.

This pattern powers `kfifo`, GPU rings, audio buffers, video frame queues — anywhere one fast producer feeds one fast consumer.

Stack vs queue — when to use what

Need	Use
Undo / redo	stack (last action first to undo)
Function call tracking	stack
Match brackets / parsing	stack
Depth-first search (DFS)	stack
Print jobs / task queue	queue
Breadth-first search (BFS)	queue
Producer-consumer	queue (often ring buffer)

If you need **fast random access** in either, you don't want stack/queue — you want a different structure.

Common bugs

Bug 1: pop from empty stack

```
int s_pop(stack_t *s) {
    return s->data[--s->size];      // if size==0, this becomes data[-1] → bad
}
```

Always check: `if (s->size == 0) return -1;` (or use a separate "is empty" check).

Bug 2: ring buffer fullness off-by-one

The classic mistake: `head == tail` is ambiguous — does it mean **empty** or **full**?

Two solutions: 1. Keep a separate `size` counter (what we did above). 2. Always leave one slot empty. Then `(tail+1)%cap == head` means full, `head == tail` means empty.

Both work. Pick one and stick to it across enqueue/dequeue.

Bug 3: mixing up head and tail

In a queue, **dequeue from head**, **enqueue at tail**. Mix them up → you've got a stack!

Recap

1. **Stack** = LIFO. Push and pop on the same end.

2. **Queue** = FIFO. Enqueue at back, dequeue at front.
3. Array-backed stack: simple, fast.
4. Naive array queue: O(n) dequeue (shifts).
5. **Ring buffer** = fast O(1) queue. Used everywhere from kernel `kfifo` to GPU rings.
6. **SPSC ring buffer** = lock-free queue between exactly one producer and one consumer.

Self-check (in your head)

1. Push 1, 2, 3, 4 onto an empty stack. Pop twice. What's on the stack?
2. Enqueue 1, 2, 3, 4 into an empty queue. Dequeue twice. What's the front element now?
3. In a ring buffer with `cap=5`, after these ops: enq A, enq B, enq C, deq, enq D, enq E, enq F — what's `head`, `tail`, `size`?
4. The string `"((()"` — does `balanced` return 0 or 1? Trace the stack to verify.
5. Why is dequeue **O(1)** for a ring buffer but **O(n)** for a naive array queue?

Answers:

1. After pushes: `[1, 2, 3, 4]`. After two pops (4 then 3 leave): **`[1, 2]`**.
2. After enqueues: `[1, 2, 3, 4]` (front=1). After two dequeues (1 then 2 leave): **front element is now 3**, queue is `[3, 4]`.
3. Trace:
   - Start: head=0, tail=0, size=0
   - enq A: buf[0]=A, tail=1, size=1
   - enq B: buf[1]=B, tail=2, size=2
   - enq C: buf[2]=C, tail=3, size=3
   - deq: A out, head=1, size=2
   - enq D: buf[3]=D, tail=4, size=3
   - enq E: buf[4]=E, tail=0 (wrap), size=4
   - enq F: buf[0]=F, tail=1, size=5

   Final: head=1, tail=1, size=5 (full). Buffer = [F, B, C, D, E].
4. Trace:
   - `(` push → `[(]`
   - `(` push → `[(, (]`
   - `(` push → `[(, (, (]`
   - `)` pop → `[(, (]`
   - end of string. Stack is **not empty** (size=2) → return 0.

   Not balanced. √
5. Ring buffer: dequeue just **reads** index `head` and bumps `head` by one. No data moves. Naive array queue: dequeue must **shift** all remaining elements left. Shift is O(n).

If you got 4/5 you understand stacks and queues. 5/5, move on.

Next → DSA-06 — Hash tables.

DSA 6 — Hash tables (read-to-learn version)

> The data structure that finds anything in **constant time** — by computing a magic number from the key and using it as an index.

The big idea

Imagine you have 10,000 names and you want to look one up.

- **Array search**: scan all 10,000. **O(n).** Slow.
- **Sorted array + binary search**: ~14 lookups. **O(log n).** Faster.
- **Hash table**: usually 1 lookup. **O(1).** Fastest.

How? A **hash function** turns a key (like a name) into a number — the **bucket index**. We store the value in that bucket. To find it later, we compute the same number from the key and go straight there.

```
Key "alice"  ──hash──▶  number 173  ──% 10──▶  bucket 3
Key "bob"    ──hash──▶  number 8200 ──% 10──▶  bucket 0
Key "carol"  ──hash──▶  number 9931 ──% 10──▶  bucket 1

       buckets:
       0  ──▶ "bob"   = 25
       1  ──▶ "carol" = 30
       2  ──▶ (empty)
       3  ──▶ "alice" = 22
       ...
       9  ──▶ (empty)
```

To look up “alice”: compute hash, modulo 10 → bucket 3 → value is 22. **One step.**

What a hash function does

A hash function is any function that: 1. Takes a key (string, int, struct). 2. Returns a number — usually 32 or 64 bits. 3. Spreads keys **uniformly** across the number range.

A simple hash for a string (FNV-1a):

```
unsigned int fnv1a(const char *s) {
    unsigned int h = 2166136261u;
    for (; *s; s++) {
        h ^= (unsigned char)*s;
        h *= 16777619u;
    }
    return h;
}
```

For each byte: XOR it with the running hash, then multiply by a magic prime. The result is a 32-bit number that “looks random” relative to the input.

Three keys, three different hashes:

key	fnv1a hash
“alice”	0x4f5cea64 (1,247,843,940)
“bob”	0xfc8a3a93 (4,236,635,795)
“carol”	0x6cd66bf3 (1,825,701,875)

Two important properties:

- Same key always hashes to the same number — **deterministic.**
- Tiny input changes → wildly different hash — **avalanche.** (“alice” vs “alicd” should look unrelated.)

Bucket selection: hash modulo size

Once you have the hash, reduce it to your bucket count:

```
        bucket = hash % n_buckets;
```

For `fnv1a("alice") = 1,247,843,940`, with `n_buckets = 10`:

```
1,247,843,940 % 10 = 0
```

“alice” goes to bucket 0.

If `n_buckets` is a power of 2 (say 16):

```
        bucket = hash & (n_buckets - 1);     // faster than %
```

The bottom 4 bits of the hash become the bucket index.

Collisions — when two keys land in the same bucket

Two different keys can hash to the same bucket. That’s a **collision**. Even with great hash functions, collisions are inevitable: with 10 buckets and 11 keys, **at least two share a bucket** (pigeonhole).

Two main ways to handle them.

Way 1: Chaining (linked list per bucket)

Each bucket holds a linked list of (key, value) pairs that hashed there. To insert: append to the bucket’s list. To find: walk the list comparing keys.

```
buckets:

  0 ──▶ ["bob"=25] ──▶ ["dave"=99] ──▶ NULL    ← two keys collided here
  1 ──▶ ["carol"=30] ──▶ NULL
  2 ──▶ NULL
  3 ──▶ ["alice"=22] ──▶ NULL
  ...
```

If chains are short (1–2 entries), look-up is still O(1) on average. If chains grow long, look-up degrades toward O(n).

Way 2: Open addressing (probe to next slot)

If the bucket is taken, try the next slot, and the next, until empty. To find later, probe the same way.

Picture: cap=10, “bob” at bucket 0. Now “dave” hashes to 0 too:

```
buckets:
  0:  "bob"=25
  1:  (empty) ← dave will land here

Insert "dave": probe 0 (taken), probe 1 (empty) → put dave here.
```

```
  0:  "bob"=25
  1:  "dave"=99
```

To **find “dave”**: hash → 0; check slot 0, key is “bob” not “dave”; probe to slot 1, found.

Full example: chained hash table with 4 buckets

Insert 5 keys:

key	hash (made up)	bucket = hash % 4
“alice”	11	3
“bob”	8	0
“carol”	17	1
“dave”	12	0
“eve”	5	1

After all inserts:

```
buckets:

  0 ──► ["bob"=...] ──► ["dave"=...] ──► NULL
  1 ──► ["carol"=...] ──► ["eve"=...] ──► NULL
  2 ──► NULL
  3 ──► ["alice"=...] ──► NULL
```

Look up “eve”: 1. Hash “eve” → 5. 2. 5 % 4 = 1. 3. Walk bucket 1’s list: first entry is “carol” — not eve. Next: “eve” — match! 4. Return its value.

Two pointer follows. **Approximately O(1).**

The full code (chained hash table)

```c
typedef struct entry {
    char *key;
    int   val;
    struct entry *next;
} entry_t;

typedef struct {
    entry_t **buckets;
    size_t   n_buckets;
    size_t   size;
} hmap_t;

void hmap_init(hmap_t *h, size_t n) {
    h->buckets   = calloc(n, sizeof(entry_t *));   // all NULL
    h->n_buckets = n;
```

```
        h->size      = 0;
    }
```

`calloc` zeros out the buckets so all start as `NULL`.

Insert

```
    int hmap_put(hmap_t *h, const char *key, int val) {
        unsigned int idx = fnv1a(key) % h->n_buckets;

        // (1) check if key already exists; update if so
        for (entry_t *e = h->buckets[idx]; e != NULL; e = e->next) {
            if (strcmp(e->key, key) == 0) {
                e->val = val;
                return 0;
            }
        }

        // (2) prepend new entry to bucket's list
        entry_t *e = malloc(sizeof(*e));
        if (!e) return -1;
        e->key  = strdup(key);
        e->val  = val;
        e->next = h->buckets[idx];   // point to old head
        h->buckets[idx] = e;          // become new head
        h->size++;

        // (3) maybe resize if too full
        if (h->size > h->n_buckets * 3 / 4) hmap_resize(h, h->n_buckets * 2);
        return 0;
    }
```

1. **Walk the chain** to see if key exists. If yes, update value and return.
2. Otherwise, **prepend** new entry to the chain (cheap — O(1) prepend, see DSA-04).
3. If the table is **75% full**, **double** the bucket count and rehash all entries.

Look-up

```
    int hmap_get(hmap_t *h, const char *key, int *out) {
        unsigned int idx = fnv1a(key) % h->n_buckets;
        for (entry_t *e = h->buckets[idx]; e != NULL; e = e->next) {
            if (strcmp(e->key, key) == 0) {
                *out = e->val;
                return 0;
            }
        }
        return -1;     // not found
    }
```

Compute hash → walk the chain → return value (or -1 if missing).

Trace: get("dave") on our example table

1. `fnv1a("dave")` = some big number; `% 4` = 0.
2. `e = buckets[0]` → entry "bob". `strcmp("bob", "dave")` ≠ 0. e = e->next.
3. `e` → entry "dave". `strcmp("dave", "dave")` == 0 → return val.

Two strcmps, two pointer follows. **O(1)** on average (chain length is short).

Why resize?

The "**load factor**" is `size / n_buckets` — average chain length.

- Load factor 0.5: most buckets have 0 or 1 entry. Lookups are essentially O(1).
- Load factor 1.0: chains average length 1. Still mostly O(1).
- Load factor 5.0: chains average length 5. Lookups are 5 strcmps. Becoming O(n).

To keep performance, **resize** when the load factor gets too high — typically when it crosses **0.75**.

Resize: 1. Allocate a new buckets array (e.g., 2× the size). 2. Walk every entry in the old buckets. 3. Re-hash each into the new table (the bucket index changes because `n_buckets` changed!). 4. Free the old buckets.

This is **O(n)** but happens rarely. Amortized cost per insert: still O(1).

Why hash tables fail (worst case)

If all your keys hash to the same bucket, you've got one giant chain. Lookups become **O(n)**.

How could that happen? - **Bad hash function:** every key produces the same hash. - **Adversarial input:** an attacker chooses keys that hash to the same bucket (a denial-of-service attack on web servers).

Real systems use **randomized hashes** (SipHash with a per-boot random seed) so attackers can't predict bucket placement. The Linux kernel uses SipHash for hash tables that face untrusted input.

Hash tables vs other structures

	Hash table	BST	Sorted array
Lookup (avg)	**O(1)**	O(log n)	O(log n)
Lookup (worst)	O(n)	O(log n) (balanced)	O(log n)
Sorted iteration	impossible (random order)	**O(n) sorted**	O(n) sorted
Range query "all keys in [a,b]"	impossible	O(log n + k)	O(log n + k)
Memory	bucket array + entries	2 ptrs + value per node	just the values

Use hash table for "raw fast key→value lookup." **Use BST or sorted array** when you need ordered iteration or range queries.

Picture of resize (4 → 8 buckets)

Before resize (size=3, cap=4, load=0.75):

```
buckets:
0 ──► ["bob",8]  ──► ["dave",12] ──► NULL
1 ──► NULL
2 ──► NULL
3 ──► ["alice",11] ──► NULL
```

We compute new bucket for each entry under cap=8:

key	hash	new bucket = hash % 8
“bob”	8	0
“dave”	12	4
“alice”	11	3

After resize:

```
buckets:
0 ──▶ ["bob",8] ──▶ NULL
1 ──▶ NULL
2 ──▶ NULL
3 ──▶ ["alice",11] ──▶ NULL
4 ──▶ ["dave",12] ──▶ NULL
5 ──▶ NULL
6 ──▶ NULL
7 ──▶ NULL
```

“bob” and “dave” used to share bucket 0; with twice the buckets, they spread out. New load = 3/8 ≈ 0.4. Plenty of room until next resize.

Common application: the LRU cache (preview)

A common interview question: implement a cache that: - maps key → value (fast, O(1)), - evicts the **least-recently-used** key when full.

Solution: hash table + doubly linked list.

- Hash table: key → pointer to list node.
- Doubly linked list: kept in **most-recently-used → least-recently-used** order.

To `get(key)`: hash lookup → move that node to front of list. To `put(key, val)`: insert new node at front; if at capacity, drop the last node.

Both operations: O(1).

Used in: every CPU cache, every web cache, the kernel’s page cache (with twists). We trace the full implementation in DSA-26.

Common bugs

Bug 1: bad hash function

```
unsigned int bad_hash(const char *s) {
    return s[0];           // hash by first char only — terrible!
}
```

All “apple”, “ant”, “axe” → hash ‘a’ = 97. They all collide. Lookup degrades to O(n).

Bug 2: forgetting to compare keys

```
// WRONG — only checks hash, not key
for (entry_t *e = buckets[idx]; e; e = e->next)
    return e->val;          // wrong! returns first entry, even if keys differ
```

Always `strcmp` (or memcmp, etc.) the actual key on lookup. Hash equality doesn’t mean key equality.

Bug 3: modifying a key after insertion

```
char *k = strdup("hello");
hmap_put(h, k, 42);
strcpy(k, "world");            // mutating the key!
hmap_get(h, "world", ...);     // looks in a different bucket — won't find it
```

Hash keys must be **effectively immutable** while in the table.

Bug 4: forgetting to resize

A growing hash table without resize gets slower and slower. Always have a resize policy.

Linux kernel hash tables (preview)

The kernel has its own hash-table macros — read `include/linux/hashtable.h`. They use **embedded `hlist_node`** (similar to `list_head` from DSA-04).

The genius: - No `malloc` per entry — caller embeds `hlist_node` in their own struct. - No type erasure — `container_of` recovers the typed struct. - Fast: written for cache locality.

Once you've understood the array+linked-list version above, the kernel version reads naturally.

Recap

1. Hash table = **bucket array** + **hash function** that maps key → bucket.
2. **Average lookup is O(1).** Worst case O(n) (all collisions in one bucket).
3. Two collision strategies: **chaining** (linked list per bucket) or **open addressing** (probe).
4. Resize when load factor > 0.75 (or similar). Amortized cost stays O(1).
5. **Don't use** for ordered iteration or range queries.
6. The hash function is critical. Bad hash = lots of collisions = slow.

Self-check (in your head)

1. With `n_buckets = 8` and `hash("foo") = 19`, which bucket does "foo" go to?
2. You insert 10 keys into a hash table with 4 buckets. What's the average chain length? What about with 100 buckets?
3. Two keys hash to the same number. What does the chained hash table do?
4. Why is comparing two keys via `==` (or just hash equality) a bug?
5. A hash table is at load factor 0.9 and getting slower. What do you do?

Answers:

1. `19 % 8 = 3`. **Bucket 3.**
2. With 4 buckets: 10 / 4 = **2.5 average chain length**. With 100 buckets: 10 / 100 = **0.1** — most buckets empty, no chains needed.
3. Both end up in the **same bucket**, in the bucket's chain (linked list). Lookup walks the chain and uses key comparison to distinguish them.

4. Different keys can have the same hash (collision). Same hash ≠ same key. Always compare the actual key to confirm.
5. **Resize.** Allocate a bigger bucket array (typically double); rehash every entry into the new buckets. Load factor drops back near 0.4–0.5; lookups speed up.

If 4/5 you understand hash tables. 5/5, move on.

Next → DSA-07 — Trees & BSTs.

DSA 7 — Binary trees & BSTs (read-to-learn version)

> A tree is “a list, but it forks.” Once it forks, you can search log(n) nodes instead of n.

The big idea

A **binary tree** is a structure where each “node” can have up to **two** children: a left child and a right child.

```
          10          ← root (top of the tree)
         /  \
        5    15       ← children of 10
       / \     \
      3   8    20     ← children of children
               / \
             17  25   ← deeper still
```

Why is this useful? Because if the tree is **balanced** (left and right halves roughly equal), and you sort it cleverly, you can find any value in **log n** steps — same as binary search, but for a structure you can grow and shrink.

Vocabulary (memorize)

```
          10
         /  \
        5    15
       / \     \
      3   8    20
               / \
             17  25
```

- **Root**: the top node (10).
- **Leaf**: a node with no children (3, 8, 17, 25).
- **Internal node**: a node with at least one child (5, 15, 20, 10).
- **Parent / child**: 5 is parent of 3; 3 is child of 5.
- **Sibling**: nodes with the same parent (3 and 8 are siblings).
- **Subtree**: a node + all its descendants. (Subtree rooted at 5 = {5, 3, 8}.)
- **Depth of a node**: number of edges from root to that node. (10 has depth 0; 5 has depth 1; 3 has depth 2.)
- **Height of a tree**: longest path from root to any leaf. (Above tree: height = 3.)

A binary tree node in C

```c
typedef struct tnode {
    int val;
    struct tnode *left;
```

```
        struct tnode *right;
    } tnode_t;
```

A node has a value and **two** pointers (left subtree, right subtree). Either pointer can be `NULL` (no child on that side).

Binary Search Tree (BST) — the rule

A **BST** adds one rule: for **every** node,

- All values in the **left** subtree are **less** than the node's value.
- All values in the **right** subtree are **greater** than the node's value.

The example above is a BST. Verify:

- Root 10: left subtree {5, 3, 8} all < 10 √; right subtree {15, 20, 17, 25} all > 10 √.
- Node 5: left {3} < 5 √; right {8} > 5 √.
- Node 15: no left; right {20, 17, 25} all > 15 √.
- Node 20: left {17} < 20 √; right {25} > 20 √.

The rule applies **recursively** at every node.

How search works in a BST

To find a value: - Start at root. - If the value matches → done. - If smaller → go left. - If bigger → go right. - If you fall off the tree (NULL) → not found.

```
tnode_t *bst_find(tnode_t *root, int v) {
    while (root) {
        if (v == root->val) return root;
        if (v < root->val)  root = root->left;
        else                 root = root->right;
    }
    return NULL;
}
```

Each step **halves** the work (in a balanced tree). Total: **O(log n).**

Trace: find 17 in the example

```
        10
       /  \
      5    15
     / \     \
    3   8     20
             / \
            17  25
```

Step	At node	val	comparison	go where
1	10	10	17 > 10	right
2	15	15	17 > 15	right
3	20	20	17 < 20	left
4	17	17	match!	**return 17**

4 steps to find 17. The tree has 7 nodes; $\log_2(7) \approx 2.8$ — close.

Trace: find 99 (not present)

Step	At node	val	comparison	go where
1	10	10	99 > 10	right
2	15	15	99 > 15	right
3	20	20	99 > 20	right
4	NULL			**return NULL**

Not found. Took 3 steps to fall off the tree.

Insert into a BST

To insert: walk down like a search; when you hit a NULL, attach the new node there.

```
tnode_t *new_node(int v) {
    tnode_t *n = malloc(sizeof(*n));
    n->val = v; n->left = n->right = NULL;
    return n;
}

tnode_t *bst_insert(tnode_t *root, int v) {
    if (!root) return new_node(v);
    if (v < root->val) root->left  = bst_insert(root->left,  v);
    else               root->right = bst_insert(root->right, v);
    return root;
}
```

This is **recursive**. The recursion goes: “insert into the left subtree” or “into the right subtree.” It bottoms out when we hit NULL → make a new node.

Trace: insert 12 into the example tree

```
Initial:
            10
           /  \
          5    15
         / \     \
        3   8    20
                / \
               17  25
```

Step	At node	comparison	direction
1	10	12 > 10	right
2	15	12 < 15	left
3	NULL	(empty)	attach new node 12 here

Result:

```
            10
           /  \
          5    15
         / \   / \
        3   8 12  20
                 / \
                17  25
```

12 became the **left child of 15**, which had no left child before.

In-order traversal (visits values in sorted order!)

The magic of BST: walk it in a specific order, and you visit values **sorted ascending**.

The order is: **left, self, right.**

```
void inorder(tnode_t *root) {
    if (!root) return;
    inorder(root->left);
    printf("%d ", root->val);
    inorder(root->right);
}
```

Trace on the example tree

```
        10
       /  \
      5    15
     / \     \
    3   8     20
             / \
            17  25
```

We start at 10 and call `inorder(10)`:

```
inorder(10):
  inorder(5):                              ← go left from 10
    inorder(3):
      inorder(NULL)        ← left of 3
      print 3              ▶ output: 3
      inorder(NULL)        ← right of 3
    print 5                ▶ output: 3 5
    inorder(8):
      inorder(NULL)
      print 8              ▶ output: 3 5 8
      inorder(NULL)
  print 10                 ▶ output: 3 5 8 10
  inorder(15):                             ← go right from 10
    inorder(NULL)          ← left of 15
    print 15               ▶ output: 3 5 8 10 15
    inorder(20):
      inorder(17):
        inorder(NULL)
        print 17           ▶ output: 3 5 8 10 15 17
        inorder(NULL)
      print 20             ▶ output: 3 5 8 10 15 17 20
      inorder(25):
        inorder(NULL)
        print 25           ▶ output: 3 5 8 10 15 17 20 25
        inorder(NULL)
```

Final output: 3 5 8 10 15 17 20 25 — sorted! That's the BST property at work.

Three traversal orders (memorize)

Name	Order	Use
In-order	left, self, right	sorted output of BST
Pre-order	self, left, right	clone a tree; serialize
Post-order	left, right, self	free a tree (children before parent)

Picture each on the example tree:

```
in-order:    3 5 8 10 15 17 20 25     (sorted)
pre-order:   10 5 3 8 15 20 17 25
post-order:  3 8 5 17 25 20 15 10     (children of each node visited before it)
```

For pre-order, every “self” comes **before** its children. For post-order, every “self” comes **after** all its descendants.

Why “balance” matters: the worst case

Suppose you insert sorted: 1, 2, 3, 4, 5.

step	what you insert	tree
1	1	`[1]`
2	2	`[1] → right [2]`
3	3	every insert goes right
4	4	every insert goes right
5	5	every insert goes right

Result: it’s just a linked list pretending to be a tree.

```
1
 \
  2
   \
    3
     \
      4
       \
        5
```

Look-ups now take **n** steps, not log n. The tree’s “balance” is broken.

Self-balancing BSTs (AVL trees, Red-Black trees) automatically rotate after insert/delete to keep heights logarithmic. The Linux kernel uses **Red-Black trees** in `include/linux/rbtree.h` for many things — process tree, deadline scheduler, file extents, GPU virtual address space (`amdgpu_vm_bo_va`).

We cover red-black rotations in DSA-08. For now, just remember: **a regular BST can degrade to a list. Balanced BSTs prevent that.**

Counting the height

```c
int height(tnode_t *root) {
    if (!root) return 0;
    int l = height(root->left);
    int r = height(root->right);
    return 1 + (l > r ? l : r);
}
```

The recursion: height of a tree = 1 + max(height of left subtree, height of right subtree). Empty tree has height 0.

Trace: height of our example tree

We compute bottom-up.

```
height(3) = 1 + max(0, 0) = 1
height(8) = 1 + max(0, 0) = 1
height(5) = 1 + max(1, 1) = 2

height(17) = 1 + max(0, 0) = 1
height(25) = 1 + max(0, 0) = 1
height(20) = 1 + max(1, 1) = 2
height(15) = 1 + max(0, 2) = 3

height(10) = 1 + max(2, 3) = 4
```

Tree height = 4. (Counting nodes; some books count edges.)

Delete from a BST (the tricky one)

Three cases:

1. **Node has no children** → just remove it.
2. **Node has one child** → replace it with that child.
3. **Node has two children** → find the **in-order successor** (smallest value in the right subtree), copy its value into this node, then delete the successor (which has at most 1 child).

Picture case 3

Delete 15 from the example:

```
Before:
            10
           /  \
          5    15           ← delete this
         / \     \
        3   8    20
                / \
               17  25
```

Find smallest value in 15's right subtree → walk left from 20: hit 17 (17 has no left). **17 is the in-order successor.**

Copy 17 into 15's position. Then delete the original 17 (which had no children).

```
After:
            10
           /  \
          5    17           ← was 15, now holds 17
         / \     \
        3   8    20
                / \
               (gone) 25
```

(17 had no left child and we removed it as the lower copy. Now 20 has only a right child 25.)

Why two pointers per node makes a tree more powerful than a list

Each step in a list: only ONE choice (next). Each step in a BST: TWO choices (left or right). And each choice **prunes** half the remaining tree.

That's why log n vs n. Same memory, structured better.

Common bugs

Bug 1: forgetting to handle NULL

```
int height(tnode_t *r) {
    int l = height(r->left);   // crashes if r is NULL
    ...
}
```

Always check `if (!root) return ...` at the top of any tree function.

Bug 2: not maintaining the BST property

```
void bad_insert(tnode_t *r, int v) {
    if (v < r->val) bad_insert(r->left, v);     // works
    else            bad_insert(r->right, v);    // works
    // BUT: forgot to actually create the node when r->left is NULL!
}
```

Always check for NULL and create the node there.

Bug 3: stack overflow on deep trees

A recursive tree function uses the call stack — one frame per node depth. For an extremely deep (unbalanced) tree, this can crash with stack overflow.

Solution: balance the tree (DSA-08), or rewrite recursion as iteration with an explicit stack.

Bug 4: assuming the tree stays sorted after a value change

```
n->val = some_new_value;
```

If you mutate a node's value, the BST property might break (the value may belong elsewhere). **Always remove + reinsert** instead.

Recap

1. A binary tree has up to two children per node.
2. A **BST** maintains: left < node < right (recursively).
3. Search and insert in a balanced BST: **O(log n).** Worst case (unbalanced): O(n).
4. **In-order** traversal yields **sorted** output.
5. Self-balancing BSTs (Red-Black, AVL) prevent worst-case degradation. Linux uses RB-trees heavily.
6. Three traversal orders: pre, in, post — pick by what you need.

Self-check (in your head)

1. Insert 7, 3, 11, 1, 5, 9 into an empty BST in that order. Draw the tree.
2. In-order traversal of your tree above — what does it print?
3. In a perfectly balanced BST with 1023 nodes, how many steps to find any value?
4. What's the height of a tree with just one node?

5. Why does *deleting a node with two children* require finding the in-order successor — why can't you just remove it?

Answers:

1.
```
        7
       / \
      3   11
     / \  /
    1  5 9
```
   - Insert 7 → root.
   - Insert 3 → 3 < 7, left.
   - Insert 11 → 11 > 7, right.
   - Insert 1 → 1 < 7 left, 1 < 3 left.
   - Insert 5 → 5 < 7 left, 5 > 3 right.
   - Insert 9 → 9 > 7 right, 9 < 11 left.
2. **1 3 5 7 9 11.** (Sorted! That's BST + in-order.)
3. $\log_2(1023) \approx$ **10 steps.**
4. **1** (the single node is itself the only level).
5. Because removing the node leaves a "hole" with two children. You can't just glue them together — one is < and one is >. The in-order successor (smallest in the right subtree) is the **next-larger value**; replacing the deleted node with it keeps the BST property intact.

If 4/5 you understand BSTs. 5/5, move on.

Next → DSA-10 — Heaps & priority queues. (We're skipping DSA-08, 09 — balanced trees and B-trees — for the read-to-learn rewrite; the originals are still readable.)

DSA 8 — Self-balancing trees: AVL & Red-Black

> A plain BST can degrade to O(n). Self-balancing trees keep height **Θ(log n)**, always. The Linux kernel's `rb_tree` is one of them.

The problem balanced trees solve

Insert `1, 2, 3, 4, 5` into a plain BST → a chain of right children, height 5. Lookup is now O(n).

Solution: after each insert/delete, **rotate** the tree to restore some height invariant.

Rotations — the universal primitive

A **rotation** changes the structure without breaking the BST invariant. Two flavors: **left** and **right**.

```
Right rotate at y:
       y                    x
      / \                  / \
     x   c       →        a   y
    / \                      / \
   a   b                    b   c
```

```
tnode_t *rotate_right(tnode_t *y) {
    tnode_t *x = y->left;
    y->left = x->right;
    x->right = y;
    return x;   // new root of this subtree
}

tnode_t *rotate_left(tnode_t *x) {
    tnode_t *y = x->right;
    x->right = y->left;
    y->left = x;
    return y;
}
```

Constant time. Preserves BST ordering. The whole rebalancing trick is "after a possibly-bad insert, do at most a constant number of rotations to fix the invariant."

AVL trees

Invariant: for every node, **|height(left) − height(right)| ≤ 1**.

Each node stores its height (or balance factor `+1, 0, -1`).

Insert

1. Insert as in a normal BST.
2. Walk back up; recompute height; if unbalanced, rotate (4 cases: LL, LR, RR, RL).

```
typedef struct anode {
    int val, height;
    struct anode *left, *right;
} anode_t;

int h(anode_t *n) { return n ? n->height : 0; }
int bf(anode_t *n) { return h(n->left) - h(n->right); }
void update_h(anode_t *n) { n->height = 1 + (h(n->left) > h(n->right) ? h(n->left) : h(n-
>right)); }

anode_t *avl_insert(anode_t *r, int v) {
    if (!r) { anode_t *n = calloc(1, sizeof(*n)); n->val = v; n->height = 1; return n; }
    if (v < r->val) r->left = avl_insert(r->left, v);
    else if (v > r->val) r->right = avl_insert(r->right, v);
    else return r;

    update_h(r);
    int b = bf(r);
    // LL
    if (b > 1 && v < r->left->val) return rotate_right(r);
    // RR
    if (b < -1 && v > r->right->val) return rotate_left(r);
    // LR
    if (b > 1 && v > r->left->val) { r->left = rotate_left(r->left); return rotate_right(r); }
    // RL
    if (b < -1 && v < r->right->val) { r->right = rotate_right(r->right); return rotate_left(r); }
    return r;
}
```

(Note: rotate_left/rotate_right need to also call update_h; left as exercise.)

Performance

- Strict balance → **fastest lookups** of any common balanced tree.
- More rotations on insert/delete than red-black trees. So **lookups faster, modifications slower**.
- Used when reads dominate writes.

Red-Black trees

Invariant: every node is **red** or **black**, plus 4 rules: 1. Root is black. 2. Red nodes have black children (no two reds in a row). 3. Every path from root to a NULL pointer passes through the same number of black nodes (**black-height invariant**). 4. (NULL pointers count as black.)

These rules together guarantee **height ≤ 2 $\log_2(n+1)$** — O(log n).

Looser balance than AVL → fewer rotations on insert/delete (at most 2 rotations + recoloring per insert; at most 3 rotations per delete). So **modifications faster, lookups slightly slower**.

This is why Linux uses red-black trees: kernel data structures often see frequent inserts/deletes (e.g., adding/removing VMAs, scheduler runqueue, GPU memory ranges).

Code? Don't write it.

Real RB-tree code is dozens of cases (insert fixup vs delete fixup, with sibling-color and uncle-color subcases). It's correct, but tedious. **Read** Linux's `lib/rbtree.c`; don't reimplement unless you must.

The Linux RB-tree API is **augmented** — you can attach a callback that gets called whenever a subtree changes, used to maintain extra info like `interval_tree`'s "max endpoint in subtree" for fast interval queries.

Linux `rb_tree` — practical usage

`include/linux/rbtree.h`:

```
struct rb_node {
    unsigned long  __rb_parent_color;
    struct rb_node *rb_right, *rb_left;
};

struct rb_root { struct rb_node *rb_node; };

#define RB_ROOT (struct rb_root) { NULL, }
```

Just like `list_head`, the link is **embedded** in your struct via `rb_node`, and `rb_entry` (which is `container_of`) recovers the containing struct.

You provide your own insert function (the kernel can't compare your keys generically):

```
struct gpu_bo {
    u64 vaddr;
    struct rb_node node;
    /* ... */
};

void bo_insert(struct rb_root *root, struct gpu_bo *new) {
    struct rb_node **p = &root->rb_node, *parent = NULL;
    while (*p) {
        struct gpu_bo *e = rb_entry(*p, struct gpu_bo, node);
        parent = *p;
        if (new->vaddr < e->vaddr) p = &(*p)->rb_left;
        else if (new->vaddr > e->vaddr) p = &(*p)->rb_right;
        else { /* duplicate handling */ return; }
    }
    rb_link_node(&new->node, parent, p);
    rb_insert_color(&new->node, root);
}

struct gpu_bo *bo_find(struct rb_root *root, u64 addr) {
    struct rb_node *n = root->rb_node;
    while (n) {
        struct gpu_bo *e = rb_entry(n, struct gpu_bo, node);
        if (addr < e->vaddr) n = n->rb_left;
        else if (addr > e->vaddr) n = n->rb_right;
        else return e;
    }
    return NULL;
}
```

This pattern is **everywhere in amdgpu** — for tracking VAs, PT entries, BO ranges. Memorize it.

Other balanced trees you should know exist

- **Splay trees** — recently-accessed elements bubble to the root. Not balanced in the classical sense, but amortized O(log n). Used in some kernels (BSD), some compilers.
- **Treaps** — BST + heap-on-priorities (priorities random). Probabilistically balanced. Simple code.
- **Skip lists** — randomized linked-list-of-linked-lists. Lock-friendly. Used in concurrent contexts (e.g., LevelDB, Java's `ConcurrentSkipListMap`).
- **B-trees / B+trees** — many children per node. Disk- and cache-friendly. Filesystems (xfs, btrfs, ext4 dirs) and DBs use them. We do these next.

When to use what (cheat sheet)

Need	Use
Fastest get / put unordered	hash table
Ordered iteration	BST (red-black or AVL)
Mostly reads	AVL
Mostly writes	Red-Black
On-disk / huge dataset	B-tree / B+tree
Lock-free ordered	Skip list
Range queries with intervals	interval tree (RB-augmented)

Common bugs (you will hit them)

1. After rotating, **forgetting to update height/color** before rotating again.
2. Updating height **before** doing the rotation, getting wrong values.
3. Mismanaging the **parent pointer** in RB trees (Linux stores parent + color packed into one word — read carefully).
4. Augmenting (adding extra info per node) without telling the rebalance code → bugs after every rotation.

Try it

1. Implement AVL insert and the four rotation cases. Test by inserting 1..1000 sorted and verifying height ≈ $\log_2(1000)$.
2. Implement AVL delete (similar idea — rebalance up the path).
3. Use the Linux RB-tree pattern in a userspace test (you can copy `lib/rbtree.c` + `include/linux/rbtree.h` to userspace; many hobby projects do).
4. Build an **interval tree**: tree of intervals, augmented with "max endpoint in subtree." Support insert, delete, and "find any interval overlapping [a,b]" in O(log n).
5. Read `lib/rbtree.c`. Identify `__rb_insert`, `__rb_erase_color`. Don't try to memorize — appreciate the structure.

Read deeper

- CLRS chapter 13 (Red-Black Trees) and chapter 14 (Augmenting Data Structures).
- Documentation: `Documentation/core-api/rbtree.rst`.
- LWN article: “An introduction to the rbtree library” — short and excellent.
- For B-trees, Hellerstein/Stonebraker’s “Anatomy of a Database System.”

Next → B-trees and B+trees.

DSA 9 — B-trees and B+trees

> The data structure that runs your filesystem and your database.

Why not just use a red-black tree?

Red-black trees are great in RAM. But:

- A disk seek is ~100,000× slower than a RAM access.
- A red-black tree of 1 billion entries is 30 levels deep → up to 30 disk seeks per lookup.
- Cache lines and disk blocks are **4 KB** (or 8/16/64 KB). Reading 24 bytes wastes the rest.

B-tree idea: each node holds many keys (e.g., 100–500). Tree fanout is huge → tree depth is tiny (e.g., 4 levels for a billion entries). Each node fills exactly one disk block.

B-tree definition

A B-tree of order `m`:

- Each node has between `⌈m/2⌉` and `m` children (except the root).
- Each node has between `⌈m/2⌉ - 1` and `m - 1` keys.
- All leaves at the same depth.
- Keys within a node are sorted; subtrees between consecutive keys hold values in that range.

Example, order 4 ("2–3–4 tree"):

```
                [10 | 20]
              /     |     \
        [3,7]   [13,17]   [25,30]
```

A search for `15`: at root, `10 < 15 < 20` → go to middle child → at `[13, 17]`, `13 < 15 < 17` → between → leaf → not found.

Each step moves down a level — O(log_m n) levels.

Operations

Search

- Inside a node, binary search for the key.
- Go to the appropriate child if not found at this level.

Insert

- Always insert at a leaf.
- If the leaf overflows (`m` keys), **split**: middle key goes up to parent.
- If parent overflows, recurse upward. Sometimes the root splits → tree grows by 1.

Delete

- Find key.
- If at internal node, swap with predecessor or successor (now in a leaf).
- Delete from leaf.
- If leaf underflows, **borrow** from sibling, or **merge** with sibling.
- Recurse upward; root may shrink.

Insert and delete are involved (lots of cases). Most engineers **read** the algorithm rather than reimplement it.

B+tree

A variation:

- All values stored **only at leaves**. Internal nodes hold only keys for routing.
- **Leaves are linked in a list** → sequential / range queries are fast.

```
                  Internal: routing keys only
                  /            |              \
        [leaves linked: ↔ ↔ ↔ ↔  containing actual records]
```

This is what almost all modern databases and filesystems actually use: - **InnoDB** (MySQL): B+tree per index. - **PostgreSQL**: B-tree by default for indexes (technically Lehman-Yao, a concurrent B+tree variant). - **ext4**: directory entries are stored in a B-tree-ish structure (Htree). - **xfs, btrfs**: B+tree based. - **ZFS**: B-tree variants.

Cache and performance

The right node size is “1 cache line” for in-RAM B-trees, “1 block” for disk. Modern CPU L1 line is 64B; L2 is 1MB; B-tree variants exist that exploit cache hierarchy (B-trees of B-trees, “fractal trees”). For most database work, **B-tree of pages** (a page = 4–16 KB) is canonical.

Concurrency

Naive B-tree concurrency: lock the tree. Bad — every operation serializes.

Lehman-Yao B+tree (used by Postgres): allow concurrent readers and writers via “high-key” pointers and per-node latches. Read paths are mostly lock-free; writers latch only nodes they touch.

This level of detail is beyond DSA basics, but you should know it exists.

In the kernel / on disk

- `fs/btrfs/ctree.c` — btrfs’s “copy-on-write B-tree.” Read this if you want filesystem mastery.
- `Documentation/filesystems/btrfs.rst`.
- ext4 directory hashing (`fs/ext4/dir.c`).

When B-trees aren't right

- For pure RAM, in-process — RB-tree or hash beats B-tree. (Cache locality of B-tree helps mainly when nodes are bigger than a line; RAM B-tree of 16-key nodes is great too — see "B-tree" by Bender et al.)
- For append-only workloads — LSM trees (RocksDB, LevelDB) win.
- For point lookups where you don't need sorted iteration — hash beats both.

LSM trees (briefly, for completeness)

Log-Structured Merge tree: - Writes go into a small in-RAM sorted structure (skip list). - When full, flushed to disk as a sorted "SSTable." - Older SSTables periodically **compacted** (merge-sorted) into bigger ones.

Optimizes writes; pays at read time (look in multiple levels). Used in: RocksDB, LevelDB, Cassandra, HBase, Bigtable.

Try it (you don't need to write a full B-tree)

1. Read the Wikipedia "B-tree" article. Draw the insertion of `1..7` into an order-3 B-tree. Confirm the tree shape after each insert.
2. Implement a 2-3 tree (B-tree of order 3) — only ~150 lines if you're patient, and it's a great exercise. Test by inserting 1..1000 and verifying invariants.
3. Read `Documentation/filesystems/btrfs.rst`. Identify how btrfs uses copy-on-write to avoid in-place writes.
4. Compare RB-tree vs B-tree of small order in RAM on 10 million inserts. Time both. Comment on cache effects.
5. Bonus: read about **fractional cascading** — a technique to speed up multi-tree searches.

Read deeper

- CLRS chapter 18 (B-trees).
- Lehman & Yao (1981) "Efficient locking for concurrent operations on B-trees" — the Postgres approach.
- Modern: "The Bw-Tree: A B-tree for new hardware" (Microsoft Research) for lock-free B-tree variants.

Next → Heaps & priority queues.

DSA 10 — Heaps & priority queues (read-to-learn version)

> A heap is a tree pretending to be an array — and it gives you the **min** (or **max**) of millions of items in **constant time**.

The big idea

A **priority queue** is a data structure that always lets you grab the **highest priority** item. Used everywhere:

- OS scheduler picks the highest-priority runnable task.
- Dijkstra's algorithm picks the closest unvisited node.
- Top-K problems: "find the 100 largest sales today."
- Linux's deadline scheduler (`include/linux/rbtree.h` in scheduling code).

A **heap** is the simplest, fastest way to implement a priority queue.

The heap shape and rules

A **min-heap**: 1. Is a **complete** binary tree (every level full except possibly the last; last level filled left-to-right). 2. **Heap property**: every node's value is ≤ its children's values.

The smallest value is always at the **root**. Given any node, both its children are ≥ it.

Picture (values 1, 3, 6, 5, 9, 8, 7):

```
          1                    ← root (smallest)
        /   \
       3     6
      / \   / \
     5   9 8   7
```

Verify the property: - 1 ≤ 3 √, 1 ≤ 6 √ - 3 ≤ 5 √, 3 ≤ 9 √ - 6 ≤ 8 √, 6 ≤ 7 √

A **max-heap** is the same idea, flipped: every node ≥ its children. Largest at root.

The trick: store a tree as an array (no pointers!)

Because the tree is **complete**, we can pack it into an array level by level.

```
tree:                              array:

          1                        index:  0  1  2  3  4  5  6
        /   \                      value: [1, 3, 6, 5, 9, 8, 7]
       3     6
      / \   / \
     5   9 8   7
```

The mapping: - `arr[0]` = root. - For node at index `i`: - **left child** is at `2*i + 1` - **right child** is at `2*i + 2` - **parent** is at `(i - 1) / 2`

Verify on the example

index	value	left child idx	left value	right child idx	right value	parent idx	parent value
0	1	1	3	2	6	—	—
1	3	3	5	4	9	0	1
2	6	5	8	6	7	0	1
3	5	(none)		(none)		1	3
4	9	(none)		(none)		1	3
5	8	(none)		(none)		2	6
6	7	(none)		(none)		2	6

Every parent ≤ its children. The heap property holds.

Why arrays? No pointer overhead, contiguous memory (cache-friendly), no malloc per node. Tons of efficiency over a tree-with-pointers.

Insert into a min-heap (sift up)

To add a new element: 1. Append at the end of the array. 2. **Sift up**: while the new element is smaller than its parent, swap them.

Trace: insert 2 into our heap

Start:

```
            1
          /   \
         3     6
        / \   / \
       5   9 8   7

arr = [1, 3, 6, 5, 9, 8, 7]
```

Step 1: append

```
arr = [1, 3, 6, 5, 9, 8, 7, 2]
                            ↑
                            new at index 7
```

```
            1
          /   \
         3     6
        / \   / \
       5   9 8   7
      /
     2     ← inserted at next free slot
```

The heap property is broken — 2's parent is 5, but 2 < 5.

Step 2: sift up

Index 7's parent: `(7-1)/2 = 3`, value 5. 2 < 5 → **swap.**

```
arr = [1, 3, 6, 2, 9, 8, 7, 5]
                ↑
                2 is now at index 3
```

```
            1
          /   \
```

```
          3     6
         / \   / \
        2   9 8   7
       /
      5
```

Still broken — 2's parent is 3 (index `(3-1)/2 = 1`). 2 < 3 → **swap.**

```
arr = [1, 2, 6, 3, 9, 8, 7, 5]
          ↑
          2 is now at index 1
```

```
            1
          /   \
         2     6
        / \   / \
       3   9 8   7
      /
     5
```

2's parent is 1 (index 0). 2 > 1 → **stop.** Heap property restored.

Step	swap?	between	after
0	append	—	`[1, 3, 6, 5, 9, 8, 7, 2]`
1	yes	indices 7 ↔ 3	`[1, 3, 6, 2, 9, 8, 7, 5]`
2	yes	indices 3 ↔ 1	`[1, 2, 6, 3, 9, 8, 7, 5]`
3	no (2 > 1)	—	done

Two swaps. O(log n) — at most height of the tree.

Remove the min (extract-min, sift down)

To remove the root: 1. Save root's value (this is the answer). 2. Move the **last element** of the array into the root. 3. Decrement size. 4. **Sift down**: swap with the smaller child while needed.

Trace: extract-min from `[1, 2, 6, 3, 9, 8, 7, 5]`

Start:

```
            1
          /   \
         2     6
        / \   / \
       3   9 8   7
      /
     5
```

Step 1: save 1. That's the answer.

Step 2: move last (index 7, value 5) to root.

```
arr = [5, 2, 6, 3, 9, 8, 7]      (size 7 now)
```

```
            5
          /   \
         2     6
        / \   / \
       3   9 8   7
```

Heap property broken — 5 > 2 (its left child).

Step 3: sift down. Compare 5 with its children (2, 6); pick the smaller (2). 5 > 2 → swap.

```
arr = [2, 5, 6, 3, 9, 8, 7]
```

```
          2
        /   \
       5     6
      / \   / \
     3   9 8   7
```

Now 5 is at index 1. Children: 3 (index 3), 9 (index 4). Smaller is 3. 5 > 3 → swap.

```
arr = [2, 3, 6, 5, 9, 8, 7]
```

```
          2
        /   \
       3     6
      / \   / \
     5   9 8   7
```

5 is at index 3. Children: indices 7, 8 — but the array only has 7 elements (indices 0–6). 5 has **no children** → stop.

Step	action	array	tree
0	save root	`[1,2,6,3,9,8,7,5]`	(above)
1	last → root	`[5,2,6,3,9,8,7]`	top has 5
2	swap with smaller child (2)	`[2,5,6,3,9,8,7]`	5 down
3	swap with smaller child (3)	`[2,3,6,5,9,8,7]`	5 down again
4	no children — stop		

Returned: 1. Two sift-down swaps. O(log n).

The tree's height is at most ~$\log_2(n)$. Sifting up or down can move at most that many levels. Both insert and extract-min are **O(log n).**

The full code (min-heap, array-backed)

```c
typedef struct {
    int *a;
    size_t size, cap;
} heap_t;

static void heap_swap(heap_t *h, size_t i, size_t j) {
    int t = h->a[i]; h->a[i] = h->a[j]; h->a[j] = t;
}

void heap_push(heap_t *h, int v) {
    if (h->size == h->cap) {
        h->cap = h->cap ? h->cap * 2 : 16;
        h->a = realloc(h->a, h->cap * sizeof(int));
    }
    size_t i = h->size++;
    h->a[i] = v;
    while (i > 0) {
        size_t parent = (i - 1) / 2;
        if (h->a[parent] <= h->a[i]) break;     // heap restored
        heap_swap(h, parent, i);
        i = parent;
    }
}

int heap_pop(heap_t *h) {
    int top = h->a[0];
    h->size--;
```

```
        h->a[0] = h->a[h->size];
        size_t i = 0;
        for (;;) {
            size_t l = 2*i + 1, r = 2*i + 2, smallest = i;
            if (l < h->size && h->a[l] < h->a[smallest]) smallest = l;
            if (r < h->size && h->a[r] < h->a[smallest]) smallest = r;
            if (smallest == i) break;                // heap restored
            heap_swap(h, i, smallest);
            i = smallest;
        }
        return top;
    }
```

Read this carefully against the traces above. Each line maps directly to a step in the trace.

Heap operations summary

Operation	Time
Find min (peek root)	**O(1)**
Insert	**O(log n)**
Extract-min	**O(log n)**
Build heap from n elements	**O(n)** (clever, see below)
Heap sort (sort using heap)	**O(n log n)**

Build heap in O(n) — Floyd's bottom-up

If you have `n` items in a random array and want to make it a heap, you could call `heap_push` n times — that's O(n log n). But there's a faster way.

Idea: start from the **bottom** of the tree. Leaves are already valid heaps (size 1). Sift down each non-leaf node.

```
    void build_heap(int *a, size_t n) {
        if (n < 2) return;
        for (size_t i = n / 2 - 1; ; i--) {       // last non-leaf to root
            sift_down(a, n, i);
            if (i == 0) break;
        }
    }
```

The mathematical reason this is O(n) (not O(n log n)): nodes at depth d are O(n / 2^d), and each sift-down moves O(log n - d) levels. The sum across all depths telescopes to O(n).

You don't need to memorize the proof. Just remember: **build_heap is O(n).** Heap sort then extract-min n times = O(n + n log n) = **O(n log n).**

Heap sort (sort an array in place)

```
    void heap_sort(int *a, size_t n) {
        build_heap(a, n);                  // step 1: O(n)
        for (size_t end = n - 1; end > 0; end--) {
            // (a) swap root (smallest) to end
            int t = a[0]; a[0] = a[end]; a[end] = t;
            // (b) heapify the smaller heap (size = end)
            sift_down(a, end, 0);
```

```
            }
        }
```

Wait — that sorts **descending**? Yes, with a min-heap. Use a **max-heap** for ascending order. (Min/max are symmetric — flip every comparison.)

Heap sort is **O(n log n)** worst case, **O(1)** extra space (in-place). Compare to merge sort (O(n log n) but O(n) extra space). Heap sort is the in-place sort of choice when memory is tight.

Top-K problem (interview classic)

"Given a stream of N numbers, find the K largest."

Naive: sort the whole stream — O(N log N). For huge N this is wasteful — we only need K values.

Better: keep a **min-heap of size K**. For each new value: - If heap has fewer than K items, push. - Else if new value > heap root, pop and push the new value.

End: the heap holds the K largest.

Why min-heap for "K largest"? Because we only need to **evict the smallest of the current K** when a bigger one arrives.

Time: O(N log K). For $N=10^9$, $K=100$, this is $\sim 10^{10}$ ops vs $\sim 3\times 10^{10}$ for full sort. Big win.

Trace: K=3, stream = [4, 1, 7, 2, 9, 3]

Start: empty heap (size ≤ 3).

value	action	heap (sorted view)
4	size<3, push	[4]
1	size<3, push	[1, 4]
7	size<3, push	[1, 4, 7]
2	2 < heap_min (1)? no. **2 > 1 → pop 1, push 2**	[2, 4, 7]
9	9 > 2 → pop 2, push 9	[4, 7, 9]
3	3 > 4? no → skip	[4, 7, 9]

End: top-3 largest = **{4, 7, 9}.** √

Where heaps appear in real systems

- **Linux scheduler (CFS)**: uses a Red-Black tree (similar idea: extract-min is O(log n)) to find the next runnable task.
- **Linux kernel timers**: a heap (`hrtimer`) ordered by expiration time. Next-to-fire is at the root.
- **Dijkstra's algorithm** (next chapter): repeatedly extract the closest unvisited node from a heap.
- **Merge K sorted arrays**: use a heap of K iterators to always pick the smallest next value. (Used in databases, MapReduce, log file merging.)
- **`std::priority_queue` in C++**, `heapq` in Python — built-in heaps you'll use in production code.

Common bugs

Bug 1: forgetting that the array is 0-indexed

If you copied a textbook that uses 1-indexed arrays: - left = `2*i`, right = `2*i + 1`, parent = `i / 2`.

C is **0-indexed**, so: - left = `2*i + 1`, right = `2*i + 2`, parent = `(i - 1) / 2`.

Pick one and stick to it.

Bug 2: bounds check on children during sift-down

```
        size_t l = 2*i + 1;
        if (a[l] < a[i])    // BUG: didn't check that l < size
```

If `l >= size`, you read past the end. Always: `if (l < size && a[l] < a[i])`.

Bug 3: assumption that heap is sorted

A heap is **partially** ordered. You can't read off the array as "sorted." Only the **root** is the min. The 2nd smallest could be left or right of the root. To get sorted output, extract-min repeatedly (heap sort).

Bug 4: comparing in the wrong direction (min vs max)

Min-heap: parent ≤ children (sift up if smaller, sift down to smaller child). Max-heap: parent ≥ children (sift up if larger, sift down to larger child).

Mix them up → "heap" doesn't satisfy any property and operations don't restore it.

Recap

1. A heap is a complete binary tree stored as an **array**. `arr[i]` parent at `(i-1)/2`; children at `2i+1`, `2i+2`.
2. **Min-heap**: parent ≤ children. Min always at root.
3. Insert and extract-min: **O(log n)**. Build a heap from n items: **O(n)**.
4. Heap sort: **O(n log n)** in-place.
5. The top-K problem: keep a min-heap of size K → O(N log K).
6. Used in scheduling, Dijkstra, timers, merge-K, OS task selection.

Self-check (in your head)

1. Given the array `[10, 4, 7, 2, 1, 8]`, is it a min-heap? Why or why not?
2. Insert 0 into the min-heap `[1, 3, 6, 5, 9, 8, 7]`. Trace the sift-up. What's the final array?
3. Extract-min from `[1, 3, 6, 5, 9, 8, 7]`. What value comes out, and what's the array after?
4. In a min-heap of size 1023, what's the maximum number of swaps for one insert?
5. To find the **5 smallest** out of 1 billion numbers, do you use a min-heap or a max-heap of size 5?

Answers:

1. Check parent–child:
   - index 0 (value 10) — left = index 1 (value 4). 10 ≤ 4? **No.** Already broken.

   Not a min-heap.
2. Append 0 at index 7:

   ```
   [1, 3, 6, 5, 9, 8, 7, 0]
   ```

   Sift up from 7: parent (7-1)/2 = 3, value 5. 0 < 5 → swap.

   ```
   [1, 3, 6, 0, 9, 8, 7, 5]
   ```

   Now at index 3. Parent (3-1)/2 = 1, value 3. 0 < 3 → swap.

   ```
   [1, 0, 6, 3, 9, 8, 7, 5]
   ```

   Now at index 1. Parent 0, value 1. 0 < 1 → swap.

   ```
   [0, 1, 6, 3, 9, 8, 7, 5]
   ```

   Now at index 0 — root. Stop.

   Final: `[0, 1, 6, 3, 9, 8, 7, 5]`**.**
3. Returns **1** (root). Move last (7) to root: `[7, 3, 6, 5, 9, 8]` (size 6 now). Sift down from 0: children 3, 6. Smaller is 3 (left). 7 > 3 → swap → `[3, 7, 6, 5, 9, 8]`. Now at 1. Children: 5, 9. Smaller is 5. 7 > 5 → swap → `[3, 5, 6, 7, 9, 8]`. Now at 3. Children: 8, 9 — but only 8 (index 7) exists. 7 < 8 → no swap.

   Final: `[3, 5, 6, 7, 9, 8]`**.**
4. $\log_2(1023) \approx$ **10 swaps.**
5. **Max-heap of size 5.** You're tracking "the 5 smallest seen so far"; the **largest** of those is the candidate for eviction (when a smaller one arrives). Max-heap's root is that largest. Symmetric to top-K largest with min-heap.

If 4/5 you understand heaps. 5/5, move on.

Next → DSA-14 — Sorting algorithms. (Skipping DSA-11/12/13 for the read-to-learn rewrite; originals still readable.)

DSA 11 — Graphs: representation & traversal

> A graph is "things and the relationships between them." The internet is a graph. The kernel's wait-for-fence dependency is a graph. The DRM scheduler's dependency tree is a graph. Master traversal once.

Definitions

- **Graph** `G = (V, E)` — a set of vertices and edges.
- **Directed** vs **undirected**.
- **Weighted** edges have costs.
- **Cyclic** vs **acyclic**.
- **DAG** — directed acyclic graph. Used for build systems, schedulers, dependency resolution.
- **Connected** (undirected) — every pair of vertices has a path between them.
- **Strongly connected** (directed) — every pair has a directed path both ways.

Representations

Two main, plus a few others.

Adjacency matrix

`adj[u][v] = 1` if there's an edge from `u` to `v` (or weight, or 0).

```
int adj[N][N];
```

- Space: $O(V^2)$. Bad for sparse graphs.
- Edge query `(u,v)`: O(1).
- Iterate u's neighbors: O(V).

Good for **dense graphs** or **fast edge lookups** (e.g., chess board adjacency).

Adjacency list

For each vertex, a list of its neighbors.

```
typedef struct edge { int to; int w; struct edge *next; } edge_t;
edge_t *adj[N];

void add_edge(int u, int v, int w) {
    edge_t *e = malloc(sizeof(*e));
    e->to = v; e->w = w; e->next = adj[u];
```

```
        adj[u] = e;
    }
```

- Space: O(V + E).
- Edge query `(u,v)`: O(degree(u)).
- Iterate u's neighbors: O(degree(u)).

Good for sparse graphs (most real graphs).

CSR (compressed sparse row) — read-mostly graphs

Two flat arrays: `offsets[v]` to `offsets[v+1]` gives a slice of `targets[]`. Hugely cache-friendly. Used in graph databases, Pregel-style frameworks.

Breadth-First Search (BFS)

Explore in waves. Uses a **queue**.

```
void bfs(int s) {
    int visited[N] = {0};
    int q[N], head = 0, tail = 0;
    q[tail++] = s; visited[s] = 1;
    while (head < tail) {
        int u = q[head++];
        printf("visited %d\n", u);
        for (edge_t *e = adj[u]; e; e = e->next) {
            if (!visited[e->to]) {
                visited[e->to] = 1;
                q[tail++] = e->to;
            }
        }
    }
}
```

- O(V + E) time and space.
- **BFS finds shortest path in unweighted graphs.**
- Use cases: shortest path on grids, level-order tree traversal, friend-of-friend networks, web crawl, peer-to-peer broadcast.

Depth-First Search (DFS)

Explore deeply, then backtrack. Uses recursion (or an explicit stack).

```
int visited[N];
void dfs(int u) {
    visited[u] = 1;
    printf("visited %d\n", u);
    for (edge_t *e = adj[u]; e; e = e->next)
        if (!visited[e->to]) dfs(e->to);
}
```

- O(V + E) time and space.
- Iterative version uses a stack; helpful for deep graphs that overflow recursion.
- Use cases: cycle detection, topological sort, finding strongly connected components, maze solving, backtracking.

What DFS unlocks

Cycle detection (directed graph)

Three colors: white (unvisited), gray (in stack), black (done). A “back edge” to a gray node = cycle.

```
int color[N];
int has_cycle(int u) {
    color[u] = 1;
    for (edge_t *e = adj[u]; e; e = e->next) {
        if (color[e->to] == 1) return 1;            // back edge
        if (color[e->to] == 0 && has_cycle(e->to)) return 1;
    }
    color[u] = 2;
    return 0;
}
```

Topological sort (DAG only)

“Order vertices so all edges go forward.”

DFS variant: visit + emit on **post-order**, then reverse.

```
int order[N], idx;
void topo(int u) {
    visited[u] = 1;
    for (edge_t *e = adj[u]; e; e = e->next)
        if (!visited[e->to]) topo(e->to);
    order[--idx] = u;   // fill from the back
}
```

Used in: build systems (Make, ninja), task schedulers, course prerequisites, **GPU dependency tracking** (a job depends on its ins; topo gives a valid execution order).

Connected components

Repeated BFS/DFS until all vertices visited. Each pass marks one component.

Strongly connected components (Tarjan / Kosaraju)

For directed graphs. Both run in O(V + E). Tarjan uses one DFS pass; Kosaraju uses two. Used in compiler analysis, social networks.

Bipartite check (BFS/DFS coloring)

“Can vertices be 2-colored so no edge has same-colored endpoints?” Color via BFS; conflict ⇒ not bipartite. Used in scheduling, matching.

Floyd-Warshall (for completeness)

All-pairs shortest paths in $O(V^3)$. Use only for small graphs ($V \leq \sim 500$).

In the kernel / GPU stack

- **DRM scheduler dependencies**: each `drm_sched_job` has a list of `dma-fences` it must wait on. Topological execution of jobs across rings. Cycle detection here would be a bug — each fence has a unique signal-once contract.
- **Module loading** (`kernel/module.c`): a topological sort over dependencies (`MODULE_DEVICE_TABLE` and friends).
- **systemd's unit graph** — runs services in dependency-respecting order.
- **DAG of compilation units** — Make, ninja, bazel.

You will write graph traversal **less often** than tree traversal, but when you do, it's usually high-impact.

Common bugs

1. Forgetting to mark visited before queueing → infinite loop.
2. Marking visited only at dequeue → same node enqueued multiple times.
3. Using DFS on a graph too deep for the call stack — use iterative + stack.
4. Confusing directed and undirected: in undirected, add edge both ways.
5. Bad complexity (e.g., $O(V^2)$ when adjacency lists give $O(V+E)$).

Try it

1. Implement an adjacency-list graph; load from a file with edges; BFS from vertex 0; print BFS order.
2. Detect a cycle in a directed graph using colors. Test on a known DAG and a known cyclic graph.
3. Topological sort. Test on a small dependency graph (e.g., "shoes need socks; pants don't").
4. Find connected components in an undirected graph.
5. Bipartite check on a graph; print the 2-coloring if bipartite.
6. Bonus: implement Tarjan's SCC. Run on a graph with two cycles connected by one edge; verify two SCCs.

Read deeper

- CLRS chapters 22 (Elementary Graph Algorithms) and 23 (MST).
- Sedgewick *Algorithms* — beautiful graph chapter with diagrams.
- For real graph engines: GraphX, Pregel, GraphBLAS.

Next → Shortest paths: Dijkstra, Bellman-Ford, A*.

DSA 12 — Shortest paths: Dijkstra, Bellman-Ford, A*

> "Find the cheapest route from A to B." Maps, packet routing, game AI, build pipelines.

Single-source shortest path

Given a weighted graph and source `s`, find the shortest distance from `s` to every other vertex.

Dijkstra's algorithm — non-negative weights

Greedy: always extract the unvisited vertex with smallest tentative distance; relax its neighbors.

```
dist[s] = 0; dist[others] = ∞
push (0, s) to priority queue
while queue not empty:
    (d, u) = pop min
    if d > dist[u]: continue            # stale
    for (v, w) in adj[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            push (dist[v], v)
```

With a binary heap: O((V + E) log V). With a Fibonacci heap: O(E + V log V), but constants kill it.

```c
typedef struct { int v; int d; } pq_entry_t;   // priority by d
// Use the heap from DSA-10.

void dijkstra(int n, edge_t **adj, int s, int *dist) {
    for (int i = 0; i < n; i++) dist[i] = INT_MAX;
    dist[s] = 0;
    // ... pq setup ...
    while (pq_size) {
        pq_entry_t e = pq_pop();
        if (e.d > dist[e.v]) continue;
        for (edge_t *p = adj[e.v]; p; p = p->next) {
            int nd = dist[e.v] + p->w;
            if (nd < dist[p->to]) {
                dist[p->to] = nd;
                pq_push((pq_entry_t){p->to, nd});
            }
        }
    }
}
```

Limitation: Dijkstra **fails on negative edge weights**. It can settle a vertex too early.

Bellman-Ford — handles negative weights

Relax every edge `V-1` times. If anything still improves on iteration `V`, there's a negative cycle.

```
for V-1 iterations:
    for each edge (u, v, w):
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
```

O(V·E). Slower, but more robust. Used in: - distance-vector routing protocols (RIP); - detection of arbitrage in currency exchange (negative cycle = profit loop); - when negative weights model debits/credits.

Floyd-Warshall — all-pairs

$O(V^3)$. Three nested loops:

```
for k in V: for i in V: for j in V:
    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

Use for small graphs only. The simplicity is deceptive — works on negative weights, doesn't survive negative cycles.

A* — heuristic-guided search

Like Dijkstra but uses a **heuristic** `h(u)` estimating distance from `u` to goal. Push priority = `dist[u] + h(u)`. If `h` is **admissible** (never overestimates), A* finds the optimal path.

Used in: - video games (path on a tile map; `h` = Manhattan or Euclidean distance); - robot navigation; - puzzle solvers.

```
        // Same as Dijkstra, but priority is dist[u] + h(u).
```

If `h` is the actual perfect distance, A* visits a single line. If `h = 0`, A* degenerates to Dijkstra. The art is finding a tight admissible heuristic.

Bidirectional search

For shortest path between two specific points: search from both ends; meet in middle. Often √n improvement. Used in route planners (Google Maps, OSM-routing).

When to use what

Situation	Algorithm
Non-negative weights, single source	Dijkstra
Negative weights, single source	Bellman-Ford
All pairs, small V	Floyd-Warshall
Single src/dst, good heuristic available	A*
Single src/dst, no heuristic, want speedup	Bidirectional Dijkstra
Unweighted	BFS
DAG with topo order	Single pass after topo sort — O(V+E)

Negative cycles

If a graph has a reachable negative-cost cycle, “shortest path” is undefined (you can loop forever to drive cost to $-\infty$). Bellman-Ford detects this on iteration `V`. Always check.

Practical issues

Tie-breaking

When two priorities are equal, your priority queue may return them in any order. If your downstream code is order-sensitive, add a tie-breaker to the priority key.

Numerical stability

Floats and shortest paths don’t mix. Use ints (or scaled ints) when possible. With floats, two equal paths can differ by ε and cause flapping.

Stale heap entries

Standard binary heap doesn’t support decrease-key well. Common idiom: just push the new (smaller) entry; on pop, skip if stale (`if e.d > dist[e.v] continue`). Heap may grow to O(E) — fine.

In the real world

- **Vulkan/DX12 dependency graphs** — bird’s-eye, jobs depend on jobs. The compiler’s barrier inserter walks the graph.
- **Linux’s deadline scheduler** — not exactly shortest path, but uses graph reasoning on temporal constraints.
- **PCIe topology** — Linux’s PCI topology is a tree; “shortest path” is trivial. But device-affinity tools use distance metrics.
- **Memory access** in NUMA — the kernel’s NUMA distance table is essentially used to route memory requests via shortest distance.

Common bugs

1. Using Dijkstra with negative weights — **silently wrong**.
2. Forgetting the “stale entry skip” check — explosive runtime.
3. Wrong heuristic in A* (overestimating) — gives non-optimal paths.
4. Integer overflow when summing weights for huge graphs — use `long long` or saturate.
5. Bad initialization (`INT_MAX + w` overflows — guard before adding).

Try it

1. Implement Dijkstra with a binary heap. Test on a hand-drawn graph.
2. Implement Bellman-Ford and detect a negative cycle (construct one).
3. Implement A* on a 2D grid with obstacles. Heuristic: Manhattan distance. Compare visited node counts vs Dijkstra (`h = 0`).

4. Read a city map (e.g., a small CSV of edges) and find the shortest route between two named nodes.
5. Detect a negative cycle in a currency exchange graph (vertices = currencies, edge `(u, v)` = `-log(rate(u→v))`). If shortest path from `u` to `u` is negative, there's arbitrage.
6. Bonus: implement bidirectional Dijkstra. Compare nodes visited vs unidirectional.

Read deeper

- CLRS chapter 24 (Single-source) and 25 (All-pairs).
- "Contraction Hierarchies" — modern technique used by Google Maps for super-fast routing.
- Russell & Norvig *AI: A Modern Approach* — chapter on heuristic search.

Next → Tries.

DSA 13 — Tries

> A tree where keys are strings (or other sequences), branching at each character. The data structure behind autocomplete, IP routing, dictionaries, and bioinformatics.

The structure

Each node has up to (alphabet size) children, plus a flag for "end of word."

```c
typedef struct trie_node {
    struct trie_node *children[26];   // for lowercase a-z
    int is_word;
} trie_t;

trie_t *trie_new() { return calloc(1, sizeof(trie_t)); }

void trie_insert(trie_t *root, const char *s) {
    trie_t *cur = root;
    for (; *s; s++) {
        int idx = *s - 'a';
        if (!cur->children[idx]) cur->children[idx] = trie_new();
        cur = cur->children[idx];
    }
    cur->is_word = 1;
}

int trie_find(trie_t *root, const char *s) {
    trie_t *cur = root;
    for (; *s; s++) {
        int idx = *s - 'a';
        if (!cur->children[idx]) return 0;
        cur = cur->children[idx];
    }
    return cur->is_word;
}

int trie_starts_with(trie_t *root, const char *prefix) {
    trie_t *cur = root;
    for (; *prefix; prefix++) {
        int idx = *prefix - 'a';
        if (!cur->children[idx]) return 0;
        cur = cur->children[idx];
    }
    return 1;
}
```

Costs

- Insert/find/starts-with: **O(m)** where m = key length. Independent of n words!
- Memory: up to alphabet × #nodes. For ASCII (256), this can balloon.

Why a trie beats a hash table

- **Prefix queries** (`autocomplete`) — hash tables can't do this; trie is natural.
- **Sorted iteration** by walking the trie alphabetically.

- **No collisions** — exact path = exact key. Worst case == average.

Why a trie loses

- **Memory overhead**: each node has 26 pointers (or 256). Sparse tries waste memory.
- **Cache unfriendly** vs. flat hash table.
- **Complexity** of code.

For dense-character alphabets, mitigate with: - a **HashMap** in each node instead of an array (smaller, slightly slower); - **compressed tries** (skip-nodes): collapse single-child chains.

Compressed trie / Patricia trie / Radix tree

When a path has only single children, collapse them into one edge labeled with the substring. Saves memory dramatically for sparse data.

The **Linux radix tree** (`include/linux/radix-tree.h` , gradually replaced by `XArray`) is exactly this. Used to map page indices to `struct page` . The radix tree's keys are integers, not strings, but the principle is identical: branch on chunks of bits.

`XArray` is the modern replacement: a clearer API, easier multi-threaded use, RCU-safe lookups.

Real applications

Autocomplete

After typing "co", traverse to the "co" node. DFS from there, collect all words. Optionally rank by frequency or recency.

IP routing

A router maps an IP destination to the **longest matching prefix** in its routing table. Bit-trie of IP addresses; longest-prefix-match in O(32) for IPv4, O(128) for IPv6.

Implementations: - **Patricia trie** (BSD). - **DXR**, **LC-tries**, **multibit tries** for hardware acceleration.

Dictionary / spellcheck

"Is this a word?" `trie_find` in O(m). Plus near-match suggestions via DFS with edit-distance budget.

Bioinformatics

Suffix tries for genome matching. Built into suffix arrays/trees for sublinear pattern search (DSA-25).

Aho-Corasick

A trie + failure links. Searches multiple patterns in one pass through text. Used in `grep -F` (with multiple patterns), virus scanners, network intrusion detection.

In the kernel

- `lib/radix-tree.c` — was widely used (page cache, IDR).
- `lib/xarray.c` — modern replacement. Now powers page cache mapping.
- `net/ipv4/fib_trie.c` — FIB (forwarding information base) uses an LC-trie.

Read `lib/xarray.c` once you've finished the kernel chapters — it's a beautiful piece of code.

Common bugs

1. **Off-the-end alphabet index**: `'A' - 'a'` is negative; non-ASCII bytes blow up the index.
2. **Shared word + prefix**: forgetting `is_word` and treating any visited node as a word.
3. **Memory leak**: trie cleanup needs post-order recursion.
4. **Collisions in compressed tries**: forgetting to split an edge when a partial match occurs.

Try it

1. Implement the basic trie above. Insert 10,000 dictionary words; test lookups.
2. Build autocomplete: given prefix, return all completions.
3. Implement word-search-II (Leetcode 212): given a board of letters and a word list, find which words are present (cells contiguous, no reuse). Trie + DFS.
4. Implement an IP-prefix matcher: insert (prefix, length, value) tuples; longest-prefix-match an IP.
5. Bonus: implement Aho-Corasick. Search a text for ~10 patterns simultaneously. (~150 lines if you're patient.)

Read deeper

- `lib/xarray.c` and `Documentation/core-api/xarray.rst`.
- Knuth *TAOCP* vol. 3 — searching.
- "Algorithms on Strings, Trees, and Sequences" (Gusfield) — the bible for string-on-trie work.

Next → Sorting algorithms.

DSA 14 — Sorting algorithms (read-to-learn version)

> Six sorts you must know, all traced step-by-step.

Why so many sorts?

Each sort has a different trade-off:

sort	best	average	worst	space	stable?	in place?
Bubble	O(n)	O(n^2)	O(n^2)	O(1)	yes	yes
Insertion	O(n)	O(n^2)	O(n^2)	O(1)	yes	yes
Selection	O(n^2)	O(n^2)	O(n^2)	O(1)	no	yes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	yes	no
Quick	O(n log n)	O(n log n)	O(n^2)	O(log n) stack	no	yes
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	no	yes

Stable: equal elements keep their original relative order. **In place**: needs only O(1)-O(log n) extra memory.

For this chapter, we trace **bubble, insertion, merge, quick** — the four to know cold for interviews.

Bubble sort

Idea: repeatedly walk through the array; swap adjacent pairs that are out of order. Largest element "bubbles" to the end on each pass.

```
void bubble_sort(int *a, size_t n) {
    for (size_t i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (size_t j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j+1]) {
                int t = a[j]; a[j] = a[j+1]; a[j+1] = t;
                swapped = 1;
            }
        }
        if (!swapped) break;          // already sorted
    }
}
```

Trace on `[5, 2, 4, 1, 3]`

Pass 1 (i=0):

j	a[j]	a[j+1]	swap?	array after
0	5	2	yes	[2, **5**, 4, 1, 3]

j	a[j]	a[j+1]	swap?	array after
1	5	4	yes	[2, 4, **5**, 1, 3]
2	5	1	yes	[2, 4, 1, **5**, 3]
3	5	3	yes	[2, 4, 1, 3, **5**]

After pass 1: 5 is at the end. swapped=true.

Pass 2 (i=1, inner runs n-1-i = 3 times):

j	a[j]	a[j+1]	swap?	array after
0	2	4	no	[2, 4, 1, 3, 5]
1	4	1	yes	[2, 1, **4**, 3, 5]
2	4	3	yes	[2, 1, 3, **4**, 5]

After pass 2: 4 in place. swapped=true.

Pass 3 (i=2, inner 2 iterations):

j	a[j]	a[j+1]	swap?	array after
0	2	1	yes	[1, **2**, 3, 4, 5]
1	2	3	no	[1, 2, 3, 4, 5]

After pass 3: looks sorted. swapped=true.

Pass 4 (i=3, inner 1 iteration):

j	a[j]	a[j+1]	swap?	array after
0	1	2	no	[1, 2, 3, 4, 5]

swapped=false → **break.** Done.

Result: `[1, 2, 3, 4, 5]`. 4 passes, ~9 swaps.

Complexity: $O(n^2)$ average and worst. In-place. Stable. **Almost never use it** — but easy to teach.

Insertion sort

Idea: like sorting a hand of cards. Pick up one card at a time, slide it left until it's in the right spot.

```
void insertion_sort(int *a, size_t n) {
    for (size_t i = 1; i < n; i++) {
        int key = a[i];
        size_t j = i;
        while (j > 0 && a[j-1] > key) {
            a[j] = a[j-1];     // shift right
            j--;
        }
        a[j] = key;             // place key
    }
}
```

Trace on `[5, 2, 4, 1, 3]`

i=1, key=2. Compare 2 with 5 → 2 < 5, shift 5 right.

```
Before:  [5, 2, 4, 1, 3]
Step:    j=1, a[j-1]=5 > 2 → shift: a[1]=5, j=0
```

```
Place:   a[0] = 2
After:   [2, 5, 4, 1, 3]
```

i=2, key=4. Compare 4 with 5 → shift. Compare 4 with 2 → stop.

```
Before:  [2, 5, 4, 1, 3]
Step:    j=2, a[j-1]=5 > 4 → shift: a[2]=5, j=1
Step:    j=1, a[j-1]=2 < 4 → stop
Place:   a[1] = 4
After:   [2, 4, 5, 1, 3]
```

i=3, key=1. Shift 5, shift 4, shift 2.

```
Before:  [2, 4, 5, 1, 3]
j=3:     a[2]=5 > 1 → shift: a[3]=5, j=2
j=2:     a[1]=4 > 1 → shift: a[2]=4, j=1
j=1:     a[0]=2 > 1 → shift: a[1]=2, j=0
j=0:     stop (j==0)
Place:   a[0] = 1
After:   [1, 2, 4, 5, 3]
```

i=4, key=3. Shift 5, shift 4. Then 2 ≤ 3 → stop.

```
Before:  [1, 2, 4, 5, 3]
j=4:     a[3]=5 > 3 → shift: a[4]=5, j=3
j=3:     a[2]=4 > 3 → shift: a[3]=4, j=2
j=2:     a[1]=2 < 3 → stop
Place:   a[2] = 3
After:   [1, 2, 3, 4, 5]
```

Result: `[1, 2, 3, 4, 5]`. Done.

Complexity: $O(n^2)$ average and worst, **O(n)** when input is nearly sorted (each key only moves a few slots). Used inside other sorts (e.g., for small subarrays in TimSort and IntroSort).

Merge sort (divide & conquer)

Idea: split the array in half; sort each half (recursively); merge.

```
                 [5, 2, 4, 1, 3]
                 /              \
             [5, 2]            [4, 1, 3]
             /    \            /     \
          [5]    [2]         [4]   [1, 3]
                                      / \
                                    [1]  [3]

   then merge bottom-up:
             [5]+[2] = [2, 5]
             [1]+[3] = [1, 3]
             [4]+[1,3] = [1, 3, 4]
             [2,5]+[1,3,4] = [1, 2, 3, 4, 5]
```

The merge step (the heart of merge sort)

Given two **sorted** subarrays, walk both with two pointers. At each step, take the smaller front element.

Merge `[2, 5]` and `[1, 3, 4]`:

Step	left ptr	right ptr	left[i]	right[j]	take	result
1	0	0	2	1	right (1)	[1]
2	0	1	2	3	left (2)	[1, 2]

Step	left ptr	right ptr	left[i]	right[j]	take	result
3	1	1	5	3	right (3)	[1, 2, 3]
4	1	2	5	4	right (4)	[1, 2, 3, 4]
5	1	exhausted	5	—	left (5)	[1, 2, 3, 4, 5]

Result: `[1, 2, 3, 4, 5]`.

Code:

```
void merge(int *a, int lo, int mid, int hi, int *tmp) {
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi)
        tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i < mid) tmp[k++] = a[i++];
    while (j < hi)  tmp[k++] = a[j++];
    for (int x = lo; x < hi; x++) a[x] = tmp[x];
}

void merge_sort(int *a, int lo, int hi, int *tmp) {
    if (hi - lo < 2) return;            // 0 or 1 element — sorted
    int mid = (lo + hi) / 2;
    merge_sort(a, lo, mid, tmp);
    merge_sort(a, mid, hi, tmp);
    merge(a, lo, mid, hi, tmp);
}
```

Complexity: O(n log n) time (every level does O(n) merge work, log n levels). O(n) extra space (tmp array). **Stable.** Best for **linked lists** (no random access penalty).

Quick sort (divide & conquer with a twist)

Idea: pick a **pivot**. Partition the array into “things < pivot” and “things ≥ pivot”. Recurse on each half.

```
int partition(int *a, int lo, int hi) {
    int pivot = a[hi - 1];
    int i = lo;
    for (int j = lo; j < hi - 1; j++) {
        if (a[j] < pivot) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++;
        }
    }
    int t = a[i]; a[i] = a[hi-1]; a[hi-1] = t;
    return i;
}

void quick_sort(int *a, int lo, int hi) {
    if (hi - lo < 2) return;
    int p = partition(a, lo, hi);
    quick_sort(a, lo, p);
    quick_sort(a, p + 1, hi);
}
```

Trace on `[5, 2, 4, 1, 3]`, pivot = last (3)

Initial: `[5, 2, 4, 1, 3]`. lo=0, hi=5. Pivot=3.

j	a[j]	< pivot?	i before	action	array
0	5	no	0	—	[5,2,4,1,3]

j	a[j]	< pivot?	i before	action	array
1	2	yes	0	swap a[0],a[1]; i=1	[**2**,5,4,1,3]
2	4	no	1	—	[2,5,4,1,3]
3	1	yes	1	swap a[1],a[3]; i=2	[2,**1**,4,5,3]

End of loop. Swap pivot (a[4]=3) with a[i=2]=4:

```
Before swap:  [2, 1, 4, 5, 3]
After swap:   [2, 1, 3, 5, 4]
                        ↑
                        pivot 3 is now at index 2
```

Return p=2. Now recurse on `[2, 1]` (left) and `[5, 4]` (right).

Left subarray `[2, 1]` (lo=0, hi=2): - pivot = a[1] = 1 - j=0: a[0]=2 < 1? no - swap a[i=0] with a[hi-1=1] → `[1, 2]`. p=0. - Recurse on `[1]` (size 1, return) and `[2]` (size 1, return).

Right subarray `[5, 4]` (lo=3, hi=5): - pivot = a[4] = 4 - j=3: a[3]=5 < 4? no - swap a[i=3] with a[4] → `[..., 4, 5]`. p=3. - Recurse on `[4]` (size 1) and `[5]` (size 1).

Final array: `[1, 2, 3, 4, 5]`. √

Complexity: O(n log n) average. **O(n²)** worst case (already-sorted input with naive pivot — every partition is uneven). **In-place.** Not stable.

In production: pick a **random** or **median-of-three** pivot to avoid O(n²) on bad input. Or use **introsort**: quicksort that falls back to heap sort if recursion depth gets too big. C++'s `std::sort` is introsort.

When to use which sort

```
Small array (< ~20 elements)?         insertion sort.
Need stable, O(n log n) guaranteed?    merge sort.
Need O(1) memory?                      heap sort.
Just want it fast in practice?         quicksort (introsort).
Linked list?                           merge sort.
Already (mostly) sorted?               insertion or TimSort.
Tiny range of integers?                counting/radix sort (next chapter).
```

In Linux

The kernel's `sort()` (in `lib/sort.c`) is a **heap sort** variant. It's chosen because: - O(n log n) **worst** case (no surprise stalls in critical paths). - O(1) extra memory (no recursion stack to allocate).

User-space `qsort()` (libc) is usually introsort.

Common bugs

Bug 1: pivoting on the same value forever

```
        // in partition: if (a[j] <= pivot) i++;     // uses <=, not <
```

If many equal values, this still works. But: with `<` and pivot = first element, you may get O(n²) on already-sorted input. Use median-of-three or randomize.

Bug 2: writing past the merge buffer

```
        int tmp[10];
        merge_sort(a, 0, 1000000, tmp);     // BOOM, tmp is too small
```

`tmp` must be at least as big as `a`.

Bug 3: off-by-one on hi/lo

Most recursive sorts use **half-open** intervals `[lo, hi)`. Mixing with `[lo, hi]` (inclusive) is the #1 bug source. Pick one convention and use it consistently.

Bug 4: thinking quicksort is always fastest

Quicksort is fast on **random** input. On adversarial input (sorted, reverse-sorted), it can degrade to $O(n^2)$. That's why all production code uses introsort (quicksort + fallback).

Recap

1. Bubble & insertion: $O(n^2)$, simple, in place. Insertion is great for **small** or **nearly-sorted** input.
2. Merge: O(n log n), stable, **needs O(n) extra memory.**
3. Quick: O(n log n) average, $O(n^2)$ worst, in-place, not stable. Pick pivot smartly.
4. Heap: O(n log n) always, in place, not stable. **Linux kernel's `sort()` uses this.**
5. Production sorts (`qsort`, `std::sort`) use **introsort** — quicksort + heap sort fallback.

Self-check (in your head)

1. Sort `[3, 1, 2]` with bubble sort. How many swaps in pass 1?
2. Insertion sort on `[1, 2, 3, 4, 5]` (already sorted). How many comparisons total?
3. Why is merge sort preferred for sorting **linked lists**?
4. Why does quicksort on already-sorted input perform badly with a "first element as pivot" strategy?
5. Heap sort uses what data structure under the hood?

Answers:

1. Pass 1: j=0: 3>1 swap → [1,3,2]. j=1: 3>2 swap → [1,2,3]. **2 swaps.** Then pass 2 finds no swaps and bails out.
2. Each iteration of the outer loop (i=1..4) does **one comparison** (a[i-1] ≤ a[i] → no shift, place key in same spot). **4 comparisons total**. O(n) on already-sorted input — fast!
3. Linked lists have **no random access** (`a[i]` is O(i)), so random-pivot partitioning (quicksort) is awkward. Merge sort only needs **sequential access** — split list into halves, merge by walking both with two pointers. Natural fit.
4. With "first element as pivot" on a sorted array: partition puts everything in the right half (everything is ≥ pivot). Each recursion shrinks by 1 instead of halving. Total: n + (n-1) + … + 1 = $O(n^2)$ recursive calls.
5. A **binary heap** (typically max-heap for ascending sort).

If 4/5 you understand sorting. 5/5, move on.

Next → DSA-15 — Searching.

DSA 15 — Searching (read-to-learn version)

> Searching = finding a target. The data structure decides how fast.

The big idea

Given a target value, where is it (or is it even there)?

- **Unsorted array** → no shortcuts. Look at every element. **O(n).**
- **Sorted array** → halve the search range each step. **O(log n).** This is **binary search.**
- **Hash table** → compute hash, jump to bucket. **O(1) average.**
- **BST** (balanced) → walk the tree. **O(log n).**

Linear search is trivial; hash and BST are covered in DSA-06 and DSA-07. This chapter focuses on **binary search** — the most asked interview pattern of all time.

Linear search (the obvious one)

```
int linear_search(int *a, int n, int x) {
    for (int i = 0; i < n; i++)
        if (a[i] == x) return i;
    return -1;
}
```

Walk left to right. Found: return index. Reach the end: return -1.

Cost: O(n). No assumption about the array's order. If the input is unsorted, this is the best you can do.

Binary search (the magic one)

If the array is **sorted**, you can be much faster.

Idea: look at the middle. The target is either: - the middle element (done), or - in the **left** half (because target < middle), or - in the **right** half (because target > middle).

Either way, you've eliminated half. Repeat.

```
int binary_search(int *a, int n, int x) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;     // (1) avoids overflow vs (lo+hi)/2
        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid + 1;
        else            hi = mid - 1;
    }
```

```
            return -1;
        }
```

1. `(lo + hi) / 2` can overflow if both are near INT_MAX. `lo + (hi - lo) / 2` is mathematically equal but avoids overflow.

Trace 1: find 60 in `[10, 20, 30, 40, 50, 60, 70]`

```
index:    0    1    2    3    4    5    6
value:  [10, 20, 30, 40, 50, 60, 70]
```

Initial: lo=0, hi=6.

Step	lo	hi	mid	a[mid]	comparison	new lo	new hi
1	0	6	3	40	40 < 60	**4**	6
2	4	6	5	60	match! return 5		

Found at index 5 in 2 steps.

Trace 2: find 35 in the same array (not present)

Step	lo	hi	mid	a[mid]	comparison	new lo	new hi
1	0	6	3	40	40 > 35	0	**2**
2	0	2	1	20	20 < 35	**2**	2
3	2	2	2	30	30 < 35	**3**	2
4	3	2	(lo > hi)	exit loop			

Return -1. **Not found, in 3 steps.**

Why log n?

The range starts at size 7. After step 1: 3 (right half). After step 2: 1. After step 3: 0 (lo > hi).

Each step **halves** the range. Number of halvings before reaching size 0: **$\log_2(n)$**. For n=7, that's about 3.

For n=1,000,000: **20 steps**. For n=1,000,000,000: **30 steps.** Almost free.

The off-by-one minefield (read carefully)

Binary search is famous for off-by-one bugs. **Understand the conventions** to avoid them.

Convention 1: closed interval `[lo, hi]`

- Initialize: `lo = 0, hi = n - 1`.
- Loop: `while (lo <= hi)`.
- Update: `lo = mid + 1` or `hi = mid - 1`.

This is what we used above.

Convention 2: half-open interval `[lo, hi)`

- Initialize: `lo = 0, hi = n`.
- Loop: `while (lo < hi)`.

- Update: `lo = mid + 1` or `hi = mid`.

Many real codebases (including the C++ STL `lower_bound`) use this style. Pick one. **Stick with it.**

Variant: find the first occurrence (lower_bound)

What if the array has duplicates and you want the **leftmost** match?

```
int lower_bound(int *a, int n, int x) {
    int lo = 0, hi = n;            // half-open
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < x) lo = mid + 1;
        else            hi = mid;     // keep going left even on equal
    }
    return lo;     // index of first element >= x
}
```

The trick: when `a[mid] == x`, we **don't return immediately**. We push `hi = mid` to keep searching left.

Trace: lower_bound of 30 in `[10, 20, 30, 30, 30, 40, 50]`

```
index:    0    1    2    3    4    5    6
value:  [10, 20, 30, 30, 30, 40, 50]
```

Step	lo	hi	mid	a[mid]	a[mid] < 30?	action
1	0	7	3	30	no	hi = 3
2	0	3	1	20	yes	lo = 2
3	2	3	2	30	no	hi = 2
4	2	2	exit			

Return **lo = 2.** First occurrence of 30 is at index 2. √

If we'd used the basic binary_search, it might return any of indices 2, 3, 4. lower_bound guarantees the leftmost.

`upper_bound` is the symmetric — find the first element **strictly greater than** x.

Other search variants

Search for "what would be the insertion point"

`lower_bound(a, n, x)` returns where you'd **insert** x to keep the array sorted. Useful for "insert in sorted list."

Search in a rotated sorted array

Classic interview: `[4, 5, 6, 7, 0, 1, 2]` is sorted, then rotated. Search for 0.

The trick: at each step, **one half is still sorted**. Test if target is in the sorted half; if yes, recurse there; otherwise the other half.

```
int search_rotated(int *a, int n, int x) {
    int lo = 0, hi = n - 1;
```

```
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == x) return mid;
            if (a[lo] <= a[mid]) {
                // left half is sorted
                if (x >= a[lo] && x < a[mid]) hi = mid - 1;
                else                          lo = mid + 1;
            } else {
                // right half is sorted
                if (x > a[mid] && x <= a[hi]) lo = mid + 1;
                else                          hi = mid - 1;
            }
        }
        return -1;
    }
```

Still O(log n). The “which half is sorted” trick is the key insight.

Ternary search (only on unimodal functions)

Binary search needs a **monotonic** array (sorted). For some functions that increase then decrease (one peak), you can ternary-search to find the peak — split the range into thirds. Used in optimization, not in everyday data structures.

Search in a 2D matrix

If a 2D matrix is sorted row by row (and the first element of each row > last of previous), treat it as a 1D array of length r×c with index i mapping to `(i / c, i % c)`. Apply binary search.

If it’s only row-sorted and column-sorted (each row sorted left to right; each column sorted top to bottom), use the **stair-case search**:

- Start at top-right corner.
- If target == cell → done.
- If target < cell → go left.
- If target > cell → go down.

O(r + c). Beautiful.

When binary search isn’t applicable

- **Unsorted array** → can’t halve meaningfully. Use linear search or build a hash table.
- **Linked list** → `a[mid]` is O(n). Binary search on linked list is O(n log n) — worse than linear scan O(n)! Don’t.
- **Approximate matches** → binary search finds exact. For “closest match,” extend lower_bound and check both sides.

In Linux

- `lib/sort.c` and `lib/bsearch.c` — kernel’s binary-search-style helpers.
- DRM and amdgpu use `bsearch` for finding GPU page entries, register descriptions, etc.
- The radix tree (`include/linux/radix-tree.h`) is a more advanced search structure used in the page cache — closer to a trie + sparse array.

Common bugs

Bug 1: forgetting the `+1` and `-1`

```
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if (a[mid] < x) lo = mid;       // BUG: should be mid+1
            else            hi = mid;
        }
```

If `a[mid] < x` and you set `lo = mid` instead of `mid+1`, the next iteration has `lo == mid` again — **infinite loop**.

Bug 2: assuming sorted when it isn't

```
        binary_search(unsorted_array, n, x);    // returns garbage
```

Binary search only works on sorted input. Always verify (or sort first).

Bug 3: integer overflow on huge arrays

```
        int mid = (lo + hi) / 2;     // overflows if lo, hi near INT_MAX
```

Use `lo + (hi - lo) / 2`.

Bug 4: closed vs half-open interval mixing

The biggest source of off-by-one bugs. Decide your convention up front and stick to it for **all** binary searches in your codebase.

Recap

1. **Linear search**: O(n). Works on any input.
2. **Binary search**: O(log n). Requires **sorted** input.
3. Halving the range each step is the magic — $\log_2(1B) \approx 30$ steps.
4. **lower_bound** finds the leftmost occurrence (or insertion point).
5. **Rotated sorted array** still allows O(log n) search via "one half is sorted" trick.
6. Don't use binary search on linked lists.

Self-check (in your head)

1. Binary search on `[1, 3, 5, 7, 9]` for x=5. How many steps?
2. Binary search on a sorted array of 1 million ints. Maximum steps?
3. `lower_bound([1, 2, 4, 4, 4, 6], 4)` → returns what?
4. `lower_bound([1, 2, 4, 4, 4, 6], 5)` → returns what?
5. Why is `(lo + hi) / 2` dangerous on huge arrays?

Answers:

1. lo=0, hi=4. mid=2 → a[2]=5 → match! **1 step.**
2. $\log_2(1{,}000{,}000) \approx$ **20 steps.**
3. **2** — the leftmost index where 4 appears.
4. **5** — the index where 5 *would* be inserted to keep sorted (right after the last 4, before 6).
5. Integer overflow: if `lo` and `hi` are near INT_MAX, `lo + hi` exceeds INT_MAX → wraps to a negative number → `mid` is negative → array access at negative index → undefined behavior. Use `lo + (hi - lo) / 2`.

If 4/5 you understand searching. 5/5, move on.

Next → DSA-16 (recursion) — original style still readable. Or jump to DSA-26 — system-design DSA for LRU cache.

DSA 16 — Recursion deep

> If you can think recursively, half of DSA falls in your lap. Trees, DP, divide-and-conquer, backtracking — all recursion in a costume.

What recursion is

A function that **calls itself** on a smaller subproblem. Two ingredients:

1. **Base case** — when to stop.
2. **Recursive case** — reduce the problem.

The classic:

```
int factorial(int n) {
    if (n <= 1) return 1;            // base
    return n * factorial(n - 1);      // recurse
}
```

To trust recursion, **assume the recursive call works**. Then ask: does combining its result give the right answer for `n`?

This is the "**recursive leap of faith**" — trust the smaller call. If that smaller call is correct, your code is correct (by induction).

How recursion uses the stack

Each call gets a **stack frame**: parameters, locals, return address. They stack up.

```
factorial(3) -> 3 * factorial(2)
                      2 * factorial(1)
                             1 (base)
```

Stack peaks at depth `n`. A C program typically has 8 MB stack; a deep recursion (millions deep) can blow the stack — `Segmentation fault`. Convert to iterative or use an explicit stack.

Tail recursion — the only "free" recursion

Tail call: recursion is the **last thing** the function does, so there's nothing to remember after it returns.

```
int fact_tail(int n, int acc) {
    if (n <= 1) return acc;
    return fact_tail(n - 1, acc * n);   // tail call
}
```

A smart compiler converts this to a loop (no stack growth). C does not guarantee tail-call optimization (though `-O2` often performs it). Don't rely on it; write the loop yourself if depth matters.

Common recursion families

1. Linear recursion

`f(n)` calls itself once. Depth = n, total work proportional to recursion depth.

Examples: factorial, sum of array, “is this string a palindrome.”

2. Binary recursion

`f(n)` calls itself twice.

Naive Fibonacci:

```
int fib(int n) {
    if (n < 2) return n;
    return fib(n-1) + fib(n-2);
}
```

This is **O(2^n)** and useless beyond n≈40. Memoization fixes it; see DSA-17.

3. Tree recursion

Each call branches into k subproblems. Total calls = exponential.

Examples: subset enumeration (“for each element, include or exclude”).

4. Mutual recursion

Two functions call each other. `is_even(n)` calls `is_odd(n-1)` and vice versa.

5. Tail recursion

As above.

6. Divide & conquer

`f(n)` solves multiple subproblems of size n/k and combines. See DSA-19.

7. Backtracking

Try all possibilities; undo on failure. See DSA-22.

Master theorem (computing recursion complexity)

For recurrences of the form `T(n) = a · T(n/b) + f(n)`:

- If `f(n) = O(n^(c))` and `c < log_b(a)` → `T(n) = Θ(n^(log_b a))`.
- If `f(n) = Θ(n^(log_b a))` → `T(n) = Θ(n^(log_b a) log n)`.
- If `f(n) = Ω(n^(c))` and `c > log_b(a)` (with some regularity) → `T(n) = Θ(f(n))`.

Useful examples: - Merge sort: `T(n) = 2·T(n/2) + n` → **Θ(n log n)**. - Binary search: `T(n) = T(n/2) + 1` → **Θ(log n)**. - Strassen’s matrix mult: `T(n) = 7·T(n/2) + n²` → **Θ(n^(log₂7)) ≈ Θ(n^2.81)**.

When recursion is wrong

- **Deep call chain on bounded stack** — stack overflow.
- **Repeated subproblems** — exponential blowup. Solution: memoization (DP).
- **Each call costs more than the work it does** — converting to a tight loop wins.
- **Recursive in interrupt context** (kernel) — kernel stack is 8–16 KB. Stay shallow.

Converting recursion to iteration

Pattern: linear recursion → loop

Trivial. Replace with a `for` or `while`.

Pattern: tree recursion → explicit stack

Push the "next subproblem" onto a stack; loop until empty.

```
// DFS using explicit stack instead of recursion
void dfs_iter(node *root) {
    node *stk[N]; int top = 0;
    if (root) stk[top++] = root;
    while (top) {
        node *n = stk[--top];
        // visit n
        if (n->right) stk[top++] = n->right;
        if (n->left)  stk[top++] = n->left;   // pushed last → popped first → preorder
    }
}
```

Same algorithm; bounded stack you control.

Worked examples

Power function (efficient)

```
double power(double x, int n) {
    if (n == 0) return 1;
    if (n < 0) return 1 / power(x, -n);
    if (n % 2 == 0) { double h = power(x, n/2); return h * h; }
    return x * power(x, n - 1);
}
```

O(log n) calls. The doubling trick is fundamental.

GCD (Euclidean algorithm)

```
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
```

Tail recursive. O(log min(a,b)) — and the proof is beautiful (Lamé's theorem connects this to Fibonacci).

Reverse a linked list (DSA-04, recursive form)

```
node *rev(node *head) {
    if (!head || !head->next) return head;
    node *new_head = rev(head->next);
    head->next->next = head;
    head->next = NULL;
    return new_head;
}
```

Tower of Hanoi

```
void hanoi(int n, char from, char via, char to) {
    if (n == 0) return;
    hanoi(n - 1, from, to, via);
    printf("Move %d: %c -> %c\n", n, from, to);
    hanoi(n - 1, via, from, to);
}
```

O(2^n). The classic example.

Common bugs

1. **Forgotten base case** → infinite recursion → stack overflow.
2. **Base case on wrong condition** → wrong answer.
3. **Modifying a global state and forgetting to restore** in backtracking.
4. **Off-by-one in subproblem size** (`n/2` vs `n/2 + 1` for odd n) — careful with integer division.
5. **Using recursion when iteration is clearly better** — premature elegance.

Try it (do all of these)

1. Sum of array recursively. Then iteratively. Compare clarity.
2. Reverse a string recursively.
3. Print all subsets of a set (binary recursion). Trace by hand for `{1,2,3}`.
4. Implement integer power with `O(log n)` calls.
5. Compute GCD using recursion. Compare to iterative version.
6. Compute Fibonacci recursively (no memoization). Time `fib(40)`. Now memoize. Time `fib(40)` again. Notice the difference.

Read deeper

- "How to Design Programs" (Felleisen et al.) — outstanding intro to recursive thinking.
- *Structure and Interpretation of Computer Programs* (SICP) chapters 1.1, 1.2.
- "Programming Praxis" exercises if you want to drill recursion.

Next → Dynamic programming.

DSA 17 — Dynamic programming

> Recursion + memoization, formalized. The interviewer's favorite topic.

When DP applies

A problem is amenable to DP when:

1. **Overlapping subproblems** — the same subproblem is solved many times by naive recursion.
2. **Optimal substructure** — the optimal answer uses optimal answers of subproblems.

Together: don't redo work. Save subproblem answers.

Two approaches

Top-down (memoization)

Just write the recursion, then cache results.

```
int memo[100];   // -1 means uncomputed

int fib(int n) {
    if (n < 2) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fib(n-1) + fib(n-2);
}
```

Easy to write — write the natural recursion, add caching. **The standard answer in interviews.**

Bottom-up (tabulation)

Iterative; build up from base cases.

```
int fib(int n) {
    int dp[n+1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
```

Often faster and uses less memory (you can sometimes throw away old rows). Common after writing the recursion to "see" the pattern.

DP recipe

1. **Define the state**: what determines a subproblem? (e.g., "first i items, capacity j" for knapsack).

2. **Define the recurrence**: how to compute the answer from smaller states.
3. **Identify base cases**.
4. **Choose top-down or bottom-up**.
5. **Optimize space** (often you only need previous row).

Classic problems

Fibonacci

Above. O(n) time, O(1) space (only need last two).

Climbing stairs

"You can take 1 or 2 steps. How many ways to reach step n?"

`ways[n] = ways[n-1] + ways[n-2]` — Fibonacci again.

House robber

Array of values. Pick a subset with max sum, no two adjacent.

`dp[i] = max(dp[i-1], dp[i-2] + a[i])`. O(n), O(1) space.

Coin change (minimum coins)

Given coin denominations, find min coins to make amount V.

```
dp[0] = 0
dp[v] = min over coin c of (dp[v-c] + 1)
```

O(V × #coins).

Coin change (number of ways)

```
ways[0] = 1
for each coin c:
    for v from c to V:
        ways[v] += ways[v-c]
```

Outer loop on coins to avoid double-counting orderings.

0/1 Knapsack

n items with weight `w[i]` and value `v[i]`. Capacity W. Max total value with each item used 0 or 1 times.

```
dp[i][j] = max(dp[i-1][j],                      // skip
               dp[i-1][j - w[i]] + v[i])         // take
```

O(nW). Can be reduced to O(W) space by iterating j backwards.

Longest common subsequence (LCS)

Strings `a` and `b`. Length of longest common subsequence.

```
dp[i][j] = dp[i-1][j-1] + 1                if a[i-1] == b[j-1]
         = max(dp[i-1][j], dp[i][j-1])     otherwise
```

O(n*m).

Edit distance (Levenshtein)

Min insert/delete/replace to convert `a` into `b`.

```
dp[i][j] = min(dp[i-1][j] + 1,                          // delete
               dp[i][j-1] + 1,                          // insert
               dp[i-1][j-1] + (a[i-1] != b[j-1] ? 1 : 0)) // replace
```

O(n*m). Used in spellcheck, diff tools, bioinformatics.

Longest increasing subsequence (LIS)

Length of longest strictly increasing subsequence in an array.

DP: $O(n^2)$: `dp[i] = 1 + max dp[j] for j<i where a[j] < a[i]`. With binary search: **O(n log n)** — classic interview optimization.

Matrix chain multiplication

Given dimensions of n matrices, find the cheapest order.

`dp[i][j] = min over k of (dp[i][k] + dp[k+1][j] + p_{i-1} * p_k * p_j)`. $O(n^3)$. Order matters: `(AB)C` may be cheaper than `A(BC)`.

Word break

Given a string and a dictionary, can the string be split into dictionary words?

`dp[i]` = can prefix of length i be split? `dp[i] = OR over j (dp[j] AND s[j..i] in dict)`. $O(n^2)$.

Subset sum / partition

Subset sum: is there a subset summing to T? `dp[i][j]` = "can we get sum j using first i items?" O(n × T).

State space caveats

- DP runs in **state count × per-state work**.
- If your state space is exponential in n (e.g., subsets), DP is no faster than brute force. Bitmask DP (subsets in `2^n` states with bit-tricks) is good only when n ≤ 20.
- The art of DP is **finding the right state**.

Bitmask DP (when n is small)

State = a subset, encoded as bits. E.g., Traveling Salesman Problem with `dp[mask][last]` = min cost path that visited exactly `mask` set of cities ending at `last`. O(2^n × n^2). Works for n ≤ 20.

DP on trees

Compute `f(node)` from `f(children)` via post-order traversal.

Example: max independent set on a tree. For each node: - `dp[v][0]` = max set NOT including v = sum over children of max(dp[c][0], dp[c][1]) - `dp[v][1]` = max set including v = 1 + sum over children of dp[c][0]

DP on intervals

State indexed by `(left, right)`. Matrix chain mult, palindrome partitioning, optimal BST.

DP on digits ("digit DP")

For problems like "count integers in [L, R] with property P." State: position, tight bound flag, partial state.

Advanced; you'll see it in competitive programming.

Memoization gotchas

- Bound your cache keys (e.g., 2D map for 2-arg functions).
- Beware **non-deterministic recursion** — if a function depends on state outside its args, memoization breaks.
- Recursion depth + memo can be an awkward fit; bottom-up may be cleaner.

Common DP patterns to recognize

- "Min/max ways" → typically DP.
- "Count ways" → typically DP.
- "Longest / shortest something with property" → typically DP.
- "Yes/No achievable" → typically DP (boolean).
- "Choose with constraints, optimize sum" → knapsack-family.

Common bugs

1. **Wrong order of loops** in bottom-up (e.g., 0/1 knapsack iteration in wrong direction → unbounded knapsack).
2. **Forgetting base case** in memoization → infinite loop (or wrong answer if the cache says `0`).
3. **Mixing 0-indexed and 1-indexed** state and array.
4. **Using `int` when sum can overflow** — switch to `long long`.
5. **Optimization too early** (writing 1D array before 2D works) — write the clear version first; optimize space after.

Try it (top 10 must-do DPs)

1. Climbing stairs.
2. House robber.
3. Coin change (min coins) and coin change (number of ways).

4. 0/1 knapsack.
5. LCS — and LIS in O(n log n).
6. Edit distance.
7. Word break.
8. Maximum subarray (Kadane) — O(n) DP.
9. Partition equal subset sum.
10. Longest palindromic substring.

If you can solve all 10 from memory in C, you're DP-fluent.

Read deeper

- CLRS chapter 15 (Dynamic Programming).
- "DP from scratch" tutorials on Codeforces and AtCoder.
- For real-world DP: query optimization in databases (Selinger's algorithm), Smith-Waterman in bioinformatics.

Next → Greedy algorithms.

DSA 18 — Greedy algorithms

> "Make the best local choice; trust it gives the global best." Sometimes works; often doesn't. The skill is knowing when.

What "greedy" means

At each step, pick the choice that looks best **right now**, without backtracking or considering future consequences.

It's the **simplest** algorithmic approach. When it works, it's typically O(n log n) (after sorting) or O(n).

When greedy works (and you can prove it)

A greedy algorithm is correct when the problem has:

1. **Greedy choice property** — making the locally optimal choice leads to a globally optimal solution.
2. **Optimal substructure** — after the greedy choice, the remaining problem is the same kind of problem (smaller).

Often you prove correctness by: - **Exchange argument**: assume some other optimal solution; swap its first choice for the greedy one; show no worse. - **Inductive proof**: greedy choice is consistent with some optimal.

Classic greedy algorithms

1. Activity selection (interval scheduling)

"Pick max # of non-overlapping intervals."

Greedy: sort by **end time**; pick the earliest-ending; skip overlaps; repeat.

```
int activity(interval *a, int n) {
    qsort(a, n, sizeof(interval), cmp_by_end);
    int count = 0, end = -1;
    for (int i = 0; i < n; i++)
        if (a[i].start >= end) { count++; end = a[i].end; }
    return count;
}
```

O(n log n). Optimal — provable by exchange.

2. Huffman coding

Optimal prefix code for compression.

Greedy: repeatedly merge the two least-frequent symbols (priority queue) until one tree remains.

O(n log n). Used in: gzip, zip (DEFLATE), JPEG, MP3.

3. Minimum Spanning Tree

Kruskal's

Sort edges by weight; add to result if it doesn't form a cycle (use union-find — DSA-23).

Prim's

Grow from a starting vertex; always add cheapest edge to an unvisited vertex (priority queue).

Both O(E log V). Used in: network design, clustering, **chip layout**.

4. Dijkstra's shortest path

DSA-12. Greedy: extract closest vertex, relax. Optimal because of non-negative weights.

5. Fractional knapsack

Items with value and weight; can take fractions. Sort by value/weight ratio; take greedily until full. **(Note: 0/1 knapsack is NOT greedy — needs DP.)**

6. Coin change with canonical denominations

US coins (1, 5, 10, 25, 100): greedy works (always take the largest). Arbitrary denominations: greedy can fail. E.g., coins {1, 3, 4}, target 6: greedy gives 4+1+1=3 coins; optimal is 3+3=2 coins. Use DP.

7. Job scheduling with deadlines

"n jobs, each with deadline and profit. Each takes 1 unit. Maximize profit."

Greedy: sort by profit (desc); for each job, place in latest free slot ≤ deadline. Use union-find for fast slot finding.

8. Gas station problem

"Cars at stations, each station has fuel `a[i]`, distance to next is `b[i]`. From which station can you complete a circle?"

Greedy with single pass; classic interview problem.

9. Minimum number of platforms

"Trains arrive and depart; min number of platforms?"

Sort arrivals and departures; sweep and count overlapping intervals.

10. Reordering for minimum cost

Many "rearrange so X is minimized" problems are greedy after a sort.

When greedy fails (and you must use DP)

- 0/1 Knapsack — picking the highest value/weight ratio first can leave you stuck.
- Coin change with arbitrary denominations.

- Edit distance, LCS.
- Longest path (NP-hard in general).

Rule of thumb: if the answer requires considering dependencies between distant choices, greedy fails. DP saves you.

Greedy mindset for problem solving

1. Identify the **cost function** to minimize or **profit function** to maximize.
2. Ask: "what's the locally best move?"
3. Try a small example. Does greedy give the right answer?
4. Try an adversarial example. Does it still work?
5. If not, look for a DP framing.

Real-world greedy

- **Compiler register allocation** — graph coloring is NP-hard, but linear-scan greedy works in production compilers (LLVM has both greedy and basic-PBQP allocators).
- **Schedulers** — pick the next task by priority. Strict greedy (priority queue). Works because we accept "not provably optimal."
- **Cache eviction** — LRU is greedy ("evict the one I haven't used longest"). Belady's optimal needs the future, which we don't have.
- **GPU work-stealing** — each thread greedily steals from longest queue. Heuristic, but practical.

Common bugs

1. **Wrong sort key** — sorting by start time instead of end time in activity selection breaks correctness.
2. **Greedy where DP is required** — produces wrong answer; small testing may not catch it.
3. **Failing to break ties consistently** — flapping between solutions.
4. **Off-by-one in interval inclusion** — `>=` vs `>` for "starts after previous ends."

Try it

1. Activity selection — implement and test.
2. Fractional knapsack — sort by value/weight, fill greedily. Compare to 0/1 knapsack on the same data.
3. Huffman code — build the tree; encode and decode a sample string. Verify length is reduced.
4. Min-platforms — implement and test on sample arrivals/departures.
5. Greedy coin change with `{1,5,10,25}` (US) and `{1,3,4}`. Show greedy succeeds for the first and fails for the second; solve the second with DP.
6. Bonus: Kruskal's MST with union-find (after DSA-23).

Read deeper

- CLRS chapter 16 (Greedy Algorithms).
- “Greedy Algorithms” chapter in *Algorithm Design* by Kleinberg & Tardos — strong on proofs.
- “Linear-scan register allocation” Poletto & Sarkar (1999) — real-world greedy in compilers.

Next → Divide & conquer.

DSA 19 — Divide & conquer

> "Break it into halves, solve each, combine." Merge sort, quicksort, FFT, parallel reductions.

The recipe

1. **Divide** the input into smaller subproblems.
2. **Conquer** each subproblem recursively.
3. **Combine** their solutions to form the answer.

The recurrence is `T(n) = a · T(n/b) + f(n)`. Apply the master theorem (DSA-16) to compute complexity.

Classic D&C algorithms

Merge sort

Two halves, sort each, merge. `T(n) = 2T(n/2) + n` → O(n log n). Stable. See DSA-14.

Quicksort

Partition around a pivot, recurse on each side. `T(n) = T(k) + T(n-k-1) + n`. Expected O(n log n).

Binary search

Halve the search range each step. `T(n) = T(n/2) + 1` → O(log n).

Maximum subarray (D&C version)

For an interval, the max sub-array is either: - in the left half, or - in the right half, or - crossing the middle.

Solve each, take the max. `T(n) = 2T(n/2) + n` → O(n log n).

(Kadane's O(n) DP solution is faster, but this D&C version is the canonical example.)

Closest pair of points (in 2D)

Sort by x; recursively find closest pair in each half; check pairs crossing the dividing line within `min` distance. O(n log n) — a clever O(n) "strip" merge step.

Strassen's matrix multiplication

Multiply two n×n matrices in $O(n^{\log_2 7}) \approx O(n^{2.81})$ instead of $O(n^3)$.

7 recursive multiplications instead of 8, plus extra additions. Used in some BLAS libraries for very large matrices. For typical sizes, the constant overhead means classical n^3 wins.

Karatsuba multiplication

Multiply two n-digit numbers in $O(n^{\log_2 3}) \approx O(n^{1.58})$. The trick: 3 recursive multiplications instead of 4. Used in big-int libraries (GMP).

FFT (Fast Fourier Transform)

Polynomial multiplication / signal transform in O(n log n). Cooley-Tukey D&C: split into even and odd indices, recurse, combine using complex roots of unity.

Foundational; you don't need to write FFT from scratch unless you want to. Know it's D&C, O(n log n).

Patterns for D&C

Balanced split

Halve at each step; depth log n.

Unbalanced split

Quicksort with bad pivot: depth could be n. Worst case bites you.

Multiple smaller problems

Strassen splits into 7 smaller; FFT into 2 even + 2 odd.

When D&C wins over DP

- When subproblems are **independent** (no overlap). DP shines on overlapping subproblems.
- When the combine step is cheap relative to the work.
- When you can **parallelize** halves on different cores/threads.

Parallel D&C

Each recursive call can run on its own thread.

```
void parallel_sort(int *a, int n) {
    if (n < THRESHOLD) { insertion_sort(a, n); return; }
    int m = n / 2;
    #pragma omp task shared(a)
    parallel_sort(a, m);
    #pragma omp task shared(a)
    parallel_sort(a + m, n - m);
    #pragma omp taskwait
    merge(a, m, a + m, n - m);
}
```

Threads have overhead — the `THRESHOLD` switches to serial for small subproblems. This is exactly how `std::sort` parallel modes are implemented.

Cilk-style work-stealing

Each worker thread has a deque. Tasks pushed onto own deque (LIFO). Idle workers steal from others' deques (FIFO). Maximum cache reuse + automatic load balancing. Used in Intel TBB, Rust's Rayon, Go's runtime.

D&C in the real world

- **Compilation**: parallel build (each TU is independent).
- **Map-reduce**: divide data, map, reduce — D&C at scale.
- **GPU radix sort**: divide into tiles, sort each tile, merge.
- **Image processing**: divide image into tiles, process in parallel, stitch.
- **Database query optimization**: parallel hash join, sort-merge join.

Common bugs

1. Bad **base case** (too small, infinite recursion; too large, stack blowup).
2. **Combine step bugs** — you sorted halves, but your merge step has off-by-one.
3. **Sharing state** between recursive calls (one pollutes the other).
4. **Stack overflow** on deep recursion of huge inputs.
5. **Threading**: false sharing (two threads write adjacent ints in the same cache line; Part IV).

Try it

1. Implement merge sort and quicksort. Verify O(n log n) by timing.
2. Implement closest-pair-of-points; test on 1000 random points.
3. Implement Karatsuba multiplication for arbitrary-precision integers (you'll write a tiny big-int).
4. Implement parallel quicksort with OpenMP. Time vs single-thread.
5. Bonus: FFT in C. Implement complex multiplication, test by computing convolution.

Read deeper

- CLRS chapter 4 (Divide and Conquer).
- *Algorithms* by Sedgewick — gentler treatment with diagrams.
- "Cilk Plus" / "Habanero" papers on work-stealing.

Next → Bit manipulation tricks.

DSA 20 — Bit manipulation tricks

> The kernel is full of bit tricks. Memorize the standard set.

The basics (cross-link Foundations A3)

C operators: - `&` AND, `|` OR, `^` XOR, `~` NOT - `<<` left shift, `>>` right shift - For unsigned types these are well-defined; for signed, right shift is implementation-defined for negatives. **Always use unsigned for bit math.**

Standard idioms

Set bit `i`

```
x |= (1U << i);
```

Clear bit `i`

```
x &= ~(1U << i);
```

Toggle bit `i`

```
x ^= (1U << i);
```

Test bit `i`

```
if (x & (1U << i)) { /* set */ }
```

Lowest set bit (isolate)

```
x & -x         // returns 0...010...0, the lowest set bit
```

Used to walk through set bits one at a time, e.g., in segment trees / Fenwick.

Clear lowest set bit

```
x & (x - 1)
```

If this equals 0, then x had exactly one bit set (i.e., is a power of two).

Is `x` a power of two?

```
(x != 0) && ((x & (x - 1)) == 0)
```

Count set bits (popcount)

```
int popcount(unsigned x) {
    int c = 0;
    while (x) { x &= x - 1; c++; }   // each iteration kills lowest set bit
    return c;
}
```

Or use the compiler builtin: `__builtin_popcount(x)` (compiles to single instruction on modern CPUs with `popcnt`).

Count trailing zeros / leading zeros

```
__builtin_ctz(x)   // count trailing zeros (UB if x==0)
__builtin_clz(x)   // count leading zeros
```

These map to single CPU instructions on x86 (`bsf`, `lzcnt`) and ARM (`clz`, `rbit+clz`).

Round up to next power of two

```
unsigned next_pow2(unsigned x) {
    if (x == 0) return 1;
    return 1U << (32 - __builtin_clz(x - 1));
}
```

Modulo by a power of two

```
x % n   →   x & (n - 1)    // when n is a power of two
```

Single AND instead of an integer divide. The kernel does this for ring buffers all the time.

Swap two ints without temp (just don't)

```
a ^= b; b ^= a; a ^= b;
```

Cute, but no faster on modern CPUs (the compiler optimizes a temp away). And blows up if `a == b` (zeros both!). Use a temp.

Bitfields and packed flags

```
#define FLAG_READY   (1U << 0)
#define FLAG_DIRTY   (1U << 1)
#define FLAG_LOCKED  (1U << 2)

unsigned flags;

flags |= FLAG_READY | FLAG_DIRTY;     // set
flags &= ~FLAG_DIRTY;                  // clear
if (flags & FLAG_LOCKED) ...           // test
```

This is **everywhere in the kernel**. Read any header — `O_RDONLY`, `MAP_PRIVATE`, `GFP_KERNEL`, etc. are all bit flags.

Manipulating bit ranges (multi-bit fields)

To read bits `[hi:lo]` from `x`:

```
unsigned bits = (x >> lo) & ((1U << (hi - lo + 1)) - 1);
```

To write a value `v` (small enough) into bits `[hi:lo]`:

```
unsigned mask = ((1U << (hi - lo + 1)) - 1) << lo;
x = (x & ~mask) | ((v << lo) & mask);
```

The kernel uses `GENMASK(hi, lo)` and `FIELD_PREP/FIELD_GET` macros (`include/linux/bitfield.h`) — read them.

Atomic bit operations

Multi-threaded code must use atomic bit ops to avoid lost updates:

- C11: `atomic_fetch_or`, `atomic_fetch_and`, etc.
- Kernel: `set_bit(nr, addr)`, `clear_bit(nr, addr)`, `test_and_set_bit(nr, addr)` — `include/linux/bitops.h`. Each compiles to a hardware atomic on the right architecture.

Bitmaps

A **bitmap** is an array of integers, treated as a flat sequence of bits. Each bit represents one item ("is it allocated?", "is it dirty?", etc.).

Kernel `bitmap_*` API: `bitmap_zero`, `bitmap_set`, `bitmap_clear`, `bitmap_find_first_zero`. Used for: PIDs, page allocator buddy bitmaps, GPU VMID allocation.

```
unsigned long pmap[BITS_TO_LONGS(N)] = {0};
bitmap_set(pmap, idx, 1);
if (test_bit(idx, pmap)) ...
```

Real-world tricks

Reverse bits in a byte

Several techniques (lookup table for speed; bit-reversal permutation as a clever expression).

Count bits set without branches

Brian Kernighan's loop above. Or use parallel-bit-count techniques (SWAR — SIMD Within A Register). Or the `popcnt` instruction.

Detect if integer is divisible by 3 (bit-only)

A surprising amount of literature exists. Practical answer: just use `%`.

Get the position of the only set bit

`__builtin_ctz(x)` if x has exactly one bit set.

Saturating addition

```
unsigned sat_add(unsigned a, unsigned b) {
    unsigned r = a + b;
    return r < a ? UINT_MAX : r;
}
```

Used in counters that should clamp instead of wrap.

Branchless absolute value

```
int abs_branchless(int x) {
    int mask = x >> 31;        // -1 if negative, 0 if non-negative
    return (x ^ mask) - mask;
}
```

Branchless min/max (use compiler; usually it's smarter)

```
int min_branchless(int a, int b) { return b ^ ((a ^ b) & -(a < b)); }
```

Often slower than the obvious `(a < b) ? a : b` — modern compilers generate `cmov`. Don't outsmart them.

In the kernel

- **Page flags**: `flags` field on `struct page` is a bitmask of PG_locked, PG_dirty, etc.
- **GFP flags**: `__GFP_KERNEL`, etc. — flags ORed into one int.
- **PCI capabilities**: bit positions in PCI config space.
- **GPU register fields**: tons of multi-bit fields. AMD's `WREG32_FIELD` and `RREG32_FIELD` macros for read-modify-write.
- **`include/linux/bitops.h`** — your friend.

Common bugs

1. **Signed shift right** on negative ints → implementation-defined.
2. **Shift count >= width** of type → undefined behavior (`x << 32` for `unsigned int` is UB!).
3. **Forgetting parentheses** in macros: `#define MASK(n) 1<<(n)` will fail when MASK is part of an expression. Always parenthesize.
4. **Mixing signed and unsigned** in bit math → integer promotion surprises.
5. **Endianness** when serializing bit fields across the wire.

Try it

1. Write `popcount` four ways: naive loop, Kernighan's, lookup table, builtin. Compare speed on 100M iterations.
2. Write `next_pow2` correctly (handle 0 and 1).
3. Implement a simple bitmap allocator: array of unsigned longs, `alloc()` returns lowest free bit, `free()` clears it. Use `__builtin_ctz` / `__builtin_ctzl`.

4. Read `include/linux/bitfield.h`. Decode `FIELD_PREP(GENMASK(7, 4), 5)`. What does it produce? Try in a small test.
5. Read `bitmap_set` / `bitmap_clear` source. Identify how partial-long boundary handling works.

Read deeper

- *Hacker's Delight* by Henry S. Warren — the bit-trick bible.
- "Bit Twiddling Hacks" — Sean Eron Anderson's classic page (search it).
- `arch/x86/include/asm/bitops.h` — hand-tuned x86 asm for set/clear/test bit.

Next → Two pointers, sliding window, fast/slow.

DSA 21 — Two pointers, sliding window, fast/slow

> Three patterns that turn $O(n^2)$ brute force into $O(n)$ elegance. Learn them once.

Two pointers — opposite ends

Two indices, one at each end, moving inward.

Two-sum in a sorted array

```
int two_sum_sorted(int *a, int n, int target, int *i_out, int *j_out) {
    int i = 0, j = n - 1;
    while (i < j) {
        int s = a[i] + a[j];
        if (s == target) { *i_out = i; *j_out = j; return 0; }
        if (s < target) i++;
        else j--;
    }
    return -1;
}
```

$O(n)$.

Reverse string in place

Already seen. Two pointers from each end.

Trapping rain water (clever 2-ptr O(n) version)

Maintain `left_max`, `right_max`; advance the smaller side. The trapped water at each step is `min(left_max, right_max) - height[i]`.

Container with most water (Leetcode 11)

Two heights, area = `min(h[i], h[j]) * (j - i)`. Move the shorter side inward; track max.

Two pointers — same direction (read/write)

A “fast” pointer that scans, a “slow” pointer that builds the result.

Remove duplicates from sorted array (in place)

```
int dedup(int *a, int n) {
    if (n == 0) return 0;
    int j = 1;
    for (int i = 1; i < n; i++)
        if (a[i] != a[i-1]) a[j++] = a[i];
```

```
        return j;
    }
```

Move zeros to end (DSA-02 again)

Partition by predicate (in place)

```
    int partition_p(int *a, int n, int (*pred)(int)) {
        int j = 0;
        for (int i = 0; i < n; i++) if (pred(a[i])) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            j++;
        }
        return j;   // first index of "false" group
    }
```

Sliding window

Maintain a window `[l, r]` over a sequence. Expand `r`; when window violates a constraint, shrink from `l`. Each element enters and leaves at most once → O(n).

Longest substring with at most K distinct characters

```
    int longest_k_distinct(const char *s, int K) {
        int count[256] = {0}, distinct = 0;
        int best = 0, l = 0;
        for (int r = 0; s[r]; r++) {
            if (count[(unsigned char)s[r]]++ == 0) distinct++;
            while (distinct > K) {
                if (--count[(unsigned char)s[l++]] == 0) distinct--;
            }
            if (r - l + 1 > best) best = r - l + 1;
        }
        return best;
    }
```

Minimum window substring

"Smallest window in s containing all chars of t." Same pattern, plus a counter of "needed counts."

Maximum sum of subarray of size k

Window of size exactly k. Maintain running sum; on each step subtract leaving element, add entering one.

Longest substring without repeating characters

Sliding window with a "last-seen" map.

The pattern: **two indices (`l`, `r`) that both only ever move forward**. Total work O(n).

Fast/slow pointers (Floyd's tortoise & hare)

Cycle detection in linked list

DSA-04. Slow steps 1, fast steps 2.

Find the start of the cycle

Once they meet, reset slow to head; advance both at speed 1; they meet at the cycle entry. Beautiful but tricky to derive — comes up in interviews.

Find duplicate number (Leetcode 287)

Array of `n+1` integers in `[1..n]`, exactly one duplicated. Treat as linked list (`a[i]` = next index). Floyd's algorithm finds the duplicate in O(n) time, O(1) space.

Find middle of linked list (one pass)

DSA-04.

Three pointers

3Sum

"Triples summing to 0." Sort. For each `i`, two-pointer on `[i+1, n-1]`. $O(n^2)$.

Sort colors (Dutch national flag)

Three pointers `lo`, `mid`, `hi`. `lo` for next 0, `hi` for next 2, `mid` scans.

```
void sort012(int *a, int n) {
    int lo = 0, mid = 0, hi = n - 1;
    while (mid <= hi) {
        if (a[mid] == 0) { swap(&a[lo++], &a[mid++]); }
        else if (a[mid] == 1) mid++;
        else { swap(&a[mid], &a[hi--]); }
    }
}
```

One pass, O(n), in place.

Patterns to recognize

- "Subarray / substring with property" → sliding window.
- "Pair / triple summing to X" in sorted → two pointers.
- "Cycle in linked list / sequence" → fast/slow.
- "In-place rearrangement preserving order" → read/write pointers.
- "Find Kth from end" → fast pointer K ahead, then both move.

Common bugs

1. **Wrong shrink condition** in sliding window — over-shrinking gives wrong answer.
2. **Updating window state in wrong order** — check the entering/leaving update sequence.
3. **Off-by-one in window size** (`r - l` vs `r - l + 1`).
4. **Updating `l` past `r`** without bounds check.
5. **Forgetting that two pointers requires sorted input** for sum-style problems.

Try it (top patterns)

1. Two-sum sorted (DSA above) — implement.
2. Longest substring without repeating chars — sliding window.
3. Minimum window substring — sliding window with counts.
4. Trapping rain water — 2-pointer O(n).
5. Sort colors — Dutch national flag.
6. Linked list cycle — Floyd's. Then find cycle start.
7. Bonus: 3Sum, 4Sum — generalize.

Read deeper

- "Two-pointer techniques" tutorials on Leetcode and HackerRank.
- *Cracking the Coding Interview* — chapters on linked lists and arrays.

Next → Backtracking.

DSA 22 — Backtracking

> Try, recurse, undo. The systematic way to enumerate solutions of combinatorial problems.

The pattern

```
backtrack(state):
    if state is a solution: record it
    for choice in candidates(state):
        apply(choice)
        backtrack(state)
        undo(choice)
```

Three ingredients: - **state**: what describes “where we are.” - **choices**: legal options at this state. - **prune**: skip branches that can’t improve.

N-Queens (the classic)

Place N queens on an N×N board so none attack.

```
int N;
int col[N];   // col[r] = chosen column for row r

int safe(int r, int c) {
    for (int i = 0; i < r; i++)
        if (col[i] == c || abs(col[i] - c) == r - i) return 0;
    return 1;
}

void solve(int r) {
    if (r == N) { /* record solution */ return; }
    for (int c = 0; c < N; c++) {
        if (!safe(r, c)) continue;
        col[r] = c;
        solve(r + 1);
        // no explicit undo needed — col[r] is overwritten next iteration
    }
}
```

Permutations

```
void permute(int *a, int l, int n) {
    if (l == n) { /* record */ return; }
    for (int i = l; i < n; i++) {
        swap(&a[l], &a[i]);
        permute(a, l + 1, n);
        swap(&a[l], &a[i]);   // undo
    }
}
```

Subsets (power set)

For each element, either include or exclude.

```
void subsets(int *a, int n, int i, int *cur, int sz) {
    if (i == n) { /* record cur[0..sz-1] */ return; }
    subsets(a, n, i + 1, cur, sz);                // exclude
    cur[sz] = a[i];
    subsets(a, n, i + 1, cur, sz + 1);            // include
}
```

Sudoku solver

```
int board[9][9];

int valid(int r, int c, int v) {
    for (int i = 0; i < 9; i++)
        if (board[r][i] == v || board[i][c] == v) return 0;
    int br = (r / 3) * 3, bc = (c / 3) * 3;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (board[br+i][bc+j] == v) return 0;
    return 1;
}

int solve() {
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++)
            if (board[r][c] == 0) {
                for (int v = 1; v <= 9; v++)
                    if (valid(r, c, v)) {
                        board[r][c] = v;
                        if (solve()) return 1;
                        board[r][c] = 0;
                    }
                return 0;        // no value worked
            }
    return 1;                    // filled
}
```

Word search (in a grid)

Given a grid of letters and a word, does it exist as a connected path?

DFS with mark-visit-unmark.

```
int dfs(int r, int c, const char *w) {
    if (!*w) return 1;
    if (r<0||c<0||r>=R||c>=C) return 0;
    if (board[r][c] != *w) return 0;
    char saved = board[r][c]; board[r][c] = '#';
    int ok = dfs(r+1,c,w+1) || dfs(r-1,c,w+1) || dfs(r,c+1,w+1) || dfs(r,c-1,w+1);
    board[r][c] = saved;
    return ok;
}
```

The “mark + unmark” is the backtracking idiom.

Pruning — the key to speed

Naive backtracking explores every branch — exponential. Pruning cuts branches early.

Branch-and-bound

At each node, compute an **optimistic bound** on the best you could achieve down this branch. If it's worse than your current best, skip.

Example: knapsack with branch-and-bound.

Constraint propagation

Each choice may force other deductions. (Dancing Links + Algorithm X for Sudoku is essentially this.)

Common backtracking problems

- **N-Queens**.
- **Permutations / combinations**.
- **Subsets / power set**.
- **Sudoku**.
- **Word search**.
- **Generate parentheses** ("output all valid `(())()` strings of length 2n").
- **Combination sum**.
- **Letter combinations of a phone number**.
- **Restore IP addresses**.
- **Palindrome partitioning**.
- **Knight's tour**.

Memoization for backtracking

Sometimes a "backtracking" problem has overlapping subproblems → it's actually DP. (Word break, palindrome partitioning min cuts, etc.)

If the same `(state)` is recomputed → memoize → DP.

Iterative backtracking

You can use an explicit stack. Used when recursion is too deep for the call stack. Less common for interview problems.

Common bugs

1. **Forgetting to undo** the change after recursion → state corruption.
2. **Mutating a shared "result" container** without copying when recording a solution.
3. **Not pruning** → time-out on bigger inputs.
4. **Wrong base case** — recording an incomplete solution.
5. **Swapping the recursion call order** when order matters (e.g., lexicographic output).

Try it (must-do list)

1. N-Queens — count solutions for N=8 (should be 92).
2. Permutations of `[1,2,3]` — print all 6.
3. Subsets of `[1,2,3]` — print all 8.
4. Sudoku solver — solve a published puzzle.
5. Word search II (Leetcode 212) — combines trie and backtracking.
6. Combination sum (Leetcode 39) — without and with duplicates.
7. Generate parentheses — exactly `2*n` chars, balanced.

Read deeper

- CLRS chapter 35 (Approximation Algorithms — many start as backtracking).
- *The Art of Computer Programming* vol. 4A (Knuth) — exhaustive coverage of dancing links + Algorithm X.
- "Backtracking algorithm" tutorials on competitive programming wikis.

Next → Union-Find / DSU.

DSA 23 — Union-Find / Disjoint-Set Union (DSU)

> A tiny data structure that supports “are these two things in the same group?” and “merge two groups.” Used in Kruskal’s MST, network connectivity, image segmentation.

Operations

- `make_set(v)` — start a new singleton.
- `find(v)` — return the representative of v’s set.
- `union(a, b)` — merge the two sets.

If implemented well, both operations are **near-constant**: amortized `O(α(n))` where `α` is the inverse Ackermann function — practically `≤ 4` for any n you’ll see.

Implementation

Each node points to a “parent.” The root is the representative.

```
int parent[N], rank[N];

void make_set(int v) { parent[v] = v; rank[v] = 0; }

int find(int v) {
    if (parent[v] == v) return v;
    return parent[v] = find(parent[v]);   // path compression
}

void union_sets(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return;
    if (rank[a] < rank[b]) { int t = a; a = b; b = t; }
    parent[b] = a;
    if (rank[a] == rank[b]) rank[a]++;
}
```

Two optimizations make it fast:

- **Path compression**: `find(v)` flattens the tree by repointing every node on the path to the root.
- **Union by rank** (or **by size**): always attach the shorter tree under the taller. Keeps trees flat.

Either alone gives `O(log n)` amortized; both together give `O(α(n))`. Always use both.

Kruskal's MST

Given a graph with weighted edges:

1. Sort edges ascending by weight.
2. For each edge `(u, v)`: if `find(u) != find(v)`, add to MST and union.
3. Stop when MST has `V-1` edges.

```
qsort(edges, n_edges, sizeof(edge), cmp_by_weight);
for (int i = 0; i < N; i++) make_set(i);
int total = 0, taken = 0;
for (int i = 0; i < n_edges && taken < N - 1; i++) {
    int u = edges[i].u, v = edges[i].v;
    if (find(u) != find(v)) {
        union_sets(u, v);
        total += edges[i].w;
        taken++;
    }
}
```

O(E log E) — sort dominates.

Other applications

Network connectivity

"Are nodes A and B connected?" → `find(A) == find(B)`. Online queries with edge insertions: O(α) per op.

Image segmentation

Pixels are nodes; merge adjacent pixels with similar colors. Output "regions" = connected components.

Cycle detection in undirected graphs

While processing edges, if both endpoints already in the same set, this edge creates a cycle.

Equivalence classes

Anywhere you need "treat these as the same."

Percolation

Statistical physics simulation: random connections; ask "is the top connected to the bottom?"

Compiler optimizations

SSA construction's phi insertion; type unification in Hindley-Milner.

Variants

DSU with weights / "potentials"

Each node tracks a value relative to its root. Useful for: "given equations `a - b = c`, are they consistent?"

Persistent DSU

Snapshot history of merges; query historical state. More complex; useful for offline queries.

Online dynamic connectivity (deletions allowed)

Much harder. Uses link-cut trees or Holm-Lichtenberg-Thorup.

In the kernel

DSU isn't a stock kernel data structure (no `disjoint_set.h`). But the **idea** appears in:

- block-merging in some allocator coalescing,
- ext4's group descriptor management (sort of),
- network connectivity tests.

In userspace tools (e.g., `lvm`, `multipath`), DSU is more directly used.

Common bugs

1. Forgetting **path compression** → trees can become long.
2. **Union by rank wrong** when sizes/ranks flip — careful update of rank.
3. **Recursive find** stack overflow on huge inputs — convert to iterative.
4. **Counting components** wrong: maintain a counter, decrement on every successful union.

Iterative find (avoids deep recursion)

```
int find_iter(int v) {
    int r = v;
    while (parent[r] != r) r = parent[r];
    while (parent[v] != r) {
        int next = parent[v];
        parent[v] = r;
        v = next;
    }
    return r;
}
```

Try it

1. Implement DSU with both path compression and union by rank.
2. Implement Kruskal's MST. Test on a small graph; verify against Prim's.
3. Solve "Number of connected components in undirected graph" (Leetcode 323).
4. Solve "Accounts merge" (Leetcode 721) — DSU on email addresses.

5. Bonus: simulate percolation. Random NxN grid; open cells progressively until top connects to bottom; report how many cells were opened.

Read deeper

- CLRS chapter 21 (Disjoint Sets).
- Tarjan's analysis of `α(n)` — beautiful, hard.
- *Competitive Programmer's Handbook* — concise DSU section.

Next → Advanced trees: segment, Fenwick, interval.

DSA 24 — Advanced trees: segment, Fenwick (BIT), interval

> When you need range queries plus point updates in O(log n), reach for these.

The problem they solve

Given an array, support: - **Point update**: change `a[i]`. - **Range query**: e.g., sum / min / max over `a[l..r]`.

Naive: O(n) per query (just scan). Both segment trees and Fenwick trees give **O(log n)** for both ops.

Fenwick tree (Binary Indexed Tree, BIT)

A clever array-based structure that supports prefix-sum queries and point updates in O(log n).

```
int bit[N + 1];   // 1-indexed

void update(int i, int delta) {
    for (; i <= N; i += i & -i) bit[i] += delta;
}

int prefix_sum(int i) {
    int s = 0;
    for (; i > 0; i -= i & -i) s += bit[i];
    return s;
}

int range_sum(int l, int r) {       // 1-indexed, inclusive
    return prefix_sum(r) - prefix_sum(l - 1);
}
```

The `i & -i` (lowest set bit, DSA-20) is the magic. Each iteration jumps to a higher index whose range covers a longer prefix.

- O(log n) per op.
- O(n) space.
- Tiny code (4 lines). Often beats segment tree in constants.
- Limited: fundamentally about **prefix-aggregate** ops where a “range” can be expressed as “prefix(r) − prefix(l-1)” — that means **invertible** ops (sum, XOR, count). Not min/max (no good “subtract” operation).

Segment tree

A balanced binary tree where each node stores a summary of an interval.

Layout: heap-style array of size 4n.

```
int tree[4 * N];

// build, query, update

void build(int node, int l, int r, int *a) {
    if (l == r) { tree[node] = a[l]; return; }
    int m = (l + r) / 2;
    build(2*node, l, m, a);
    build(2*node+1, m+1, r, a);
    tree[node] = tree[2*node] + tree[2*node+1];
}

void update(int node, int l, int r, int idx, int val) {
    if (l == r) { tree[node] = val; return; }
    int m = (l + r) / 2;
    if (idx <= m) update(2*node, l, m, idx, val);
    else          update(2*node+1, m+1, r, idx, val);
    tree[node] = tree[2*node] + tree[2*node+1];
}

int query(int node, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;             // outside
    if (ql <= l && r <= qr) return tree[node];  // fully inside
    int m = (l + r) / 2;
    return query(2*node, l, m, ql, qr) + query(2*node+1, m+1, r, ql, qr);
}
```

- O(n) build, O(log n) query and update.
- More flexible than BIT: can handle **min, max, GCD, "count of zeros," etc.**
- Larger code, larger memory, slightly slower constants.

Lazy propagation

For **range updates** (e.g., "add 5 to all of [l, r]"): mark each node lazily; defer to descendants until needed. O(log n) for both range update and range query. Much harder to implement correctly. Reach for it when needed.

Interval tree

A red-black tree augmented with "max endpoint in subtree." Supports: "given an interval [a, b], find all stored intervals that overlap it."

Operations: insert/delete in O(log n); single-overlap query in O(log n); all-overlap query in O(k + log n).

The Linux kernel's `interval_tree.h` is exactly this — used in: - KVM page-tracking, - DRM/GPUVM range tracking, - Btrfs free-space tracking, - mmu_notifiers.

```
// see include/linux/interval_tree.h and lib/interval_tree.c
struct interval_tree_node {
    struct rb_node rb;
    unsigned long start, last;
    unsigned long __subtree_last;     // augmentation: max(last) over subtree
};
```

Read this file when you reach Part VII (GPUVM uses it heavily).

Range tree

For 2D and higher: tree of trees. Useful in computational geometry. Out of scope for this book; mention it exists.

kd-tree

Spatial partitioning for k-dimensional points. Used in nearest-neighbor search, ray tracing acceleration structures (BVH is similar), graphics. We'll see BVH in Part X (ray tracing).

When to use what

Need	Choice
Prefix sum / point update	Fenwick tree
Range query (any), point update	Segment tree
Range update + range query	Segment tree with lazy propagation
"Find overlapping interval"	Interval tree (RB-augmented)
Spatial nearest-neighbor	kd-tree, BVH
Just one query type, n small	Just an array — don't over-engineer

In the kernel

- `include/linux/interval_tree.h` — used in mmu_notifiers and elsewhere.
- `lib/interval_tree.c` — read this; one of the cleanest examples of the augmented RB-tree pattern.
- Btrfs uses interval trees for free-space management.
- Out-of-tree GPU drivers use them for VA range tracking.

Common bugs

1. **Off-by-one** with 0- vs 1-indexed in BIT.
2. **Forgetting** to recompute parent in segment tree update.
3. **Inclusive/exclusive** boundary inconsistency in range queries.
4. **Memory blow-up**: 4n for segment tree (some say 2*next_pow2(n)).
5. **Lazy propagation** forgetting to push down before descending.

Try it

1. Implement BIT for prefix-sum + point update. Test against naive on random ops.
2. Implement segment tree for "range max" with point updates.
3. Implement segment tree with lazy propagation for "add x to range" + "sum range."
4. Implement an interval tree (augmented RB-tree). Insert intervals; given a query interval, find all overlapping ones.
5. Read `lib/interval_tree.c`. Trace one insertion by hand on paper.

6. Bonus: solve “Range Sum Query Mutable” on Leetcode using BIT.

Read deeper

- *Competitive Programmer’s Handbook* by Antti Laaksonen — best free intro to these.
- “Augmenting Data Structures” — CLRS chapter 14.
- “Persistent segment trees” if you want to go deep.

Next → String algorithms: KMP, Rabin-Karp, Z, suffix array.

DSA 25 — String algorithms: KMP, Rabin-Karp, Z, suffix arrays

> Sub-quadratic pattern matching. The substrate beneath grep, search engines, bioinformatics.

The problem

Given **text** of length n and **pattern** of length m, find all occurrences of pattern in text.

- Naive: at each text position, compare m characters → O(n·m).
- KMP / Z / Boyer-Moore: O(n + m).
- Rabin-Karp: O(n + m) average, O(n·m) worst.
- Suffix arrays / suffix trees: massively useful for repeated queries.

KMP (Knuth-Morris-Pratt)

Idea: when a partial match fails, **don't go back in the text**. Use a precomputed "failure function" of the pattern that tells you how much of the pattern is also a prefix.

Prefix function ("failure function")

For pattern `P`, compute `pi[i]` = length of the longest proper prefix of `P[0..i]` that's also a suffix.

```
void build_pi(const char *p, int m, int *pi) {
    pi[0] = 0;
    int k = 0;
    for (int i = 1; i < m; i++) {
        while (k > 0 && p[k] != p[i]) k = pi[k-1];
        if (p[k] == p[i]) k++;
        pi[i] = k;
    }
}
```

Matching

```
void kmp(const char *t, int n, const char *p, int m) {
    int pi[m]; build_pi(p, m, pi);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k > 0 && p[k] != t[i]) k = pi[k-1];
        if (p[k] == t[i]) k++;
        if (k == m) {
            printf("match at %d\n", i - m + 1);
            k = pi[k-1];
        }
    }
}
```

O(n + m). One of the cleanest non-trivial algorithms in the book — and one of the most common interview questions for senior roles.

Rabin-Karp

Rolling hash. Compute hash of pattern; slide a window over text computing hash of each m-substring. When hashes match, verify with direct comparison.

```
unsigned long long mod = 1000000007;
unsigned long long base = 31;

unsigned long long hash_str(const char *s, int len) {
    unsigned long long h = 0;
    for (int i = 0; i < len; i++) h = (h * base + s[i]) % mod;
    return h;
}
```

To slide the window, you need `base^(m-1) mod p` precomputed; on each step:

```
new = (old - s[i] * base^(m-1)) * base + s[i+m]
```

all mod p.

- Average O(n + m).
- Worst O(n·m) (adversarial input causing many false hash matches).
- Used in: plagiarism detection, fingerprinting (rsync delta), bytecode dedup.

Always **double-check** with direct compare on hash match — hash collisions are real.

Z-function

For string `s`, `Z[i]` = length of longest substring starting at `i` that matches a prefix of `s`.

Compute in O(n) with a clever two-pointer trick. Used in: - pattern matching (concat pattern + sentinel + text → look for `Z[i] == m`), - detecting repeats, - string compression.

Boyer-Moore

Match from the **right** of the pattern. Bad-character + good-suffix heuristics let you skip large chunks. **Sublinear in practice** (less than n comparisons on natural text). Used in `grep`, `strstr` on glibc.

Implementation is involved (~100 lines); read the wiki article for the rules.

Aho-Corasick

For matching **multiple patterns** simultaneously. Build a trie of patterns; add “failure links” (like KMP’s pi function generalized). Linear time.

Used in: `fgrep`, `snort` (intrusion detection), virus scanners, lexical analyzers.

Suffix arrays

A sorted array of all **suffixes** of a string. Fundamental for many string operations.

For “abc”: - suffixes: “abc”, “bc”, “c” - sorted: “abc”, “bc”, “c” - suffix array: [0, 1, 2]

Construction in O(n log n) (or O(n) with SA-IS). Plus the **LCP array** (longest common prefix between consecutive sorted suffixes) gives O(n) for many problems.

Use cases: - find all occurrences of a pattern in O(m log n + k); - count distinct substrings; - longest repeated substring; - bioinformatics: BWT, FM-index used in bwa for read alignment.

Suffix tree

A trie of all suffixes, compressed (Ukkonen’s algorithm builds it in O(n)). Strictly more expressive than suffix array; uses more memory; harder to implement. In modern practice, **suffix arrays + LCP** are usually preferred.

Edit distance algorithms

- **Levenshtein** distance — DP we did in DSA-17, O(n*m).
- **Wagner-Fischer** — same algorithm, classic name.
- **Bit-parallel** for short patterns — O(n·m/w) where w = word width.

Used in: spell checkers, fuzzy search, DNA alignment.

Compression algorithms (briefly)

Strings overlap heavily with compression: - **Run-length encoding** — consecutive duplicates. - **Huffman coding** — optimal prefix code. - **LZ77 / LZ78** — repeat references; basis of zlib/gzip. - **Burrows-Wheeler Transform** — used in bzip2 and bwa (bioinformatics aligner).

In the real world

- **grep / ripgrep** — Boyer-Moore family for fixed strings; Aho-Corasick for `-F` with multiple patterns; regex engines for the rest.
- **Code search** (GitHub, sourcegraph) — suffix arrays / FM-index for fast on-disk search.
- **AV / IDS** (ClamAV, Snort) — Aho-Corasick.
- **DNA aligners** (BWA, Bowtie) — FM-index.
- **rsync** — rolling hash for delta sync.
- **Compilers** — KMP-style for tokenization in some lexers; suffix tries for token dedup.

Common bugs

1. **Hash collisions** in Rabin-Karp without verification.
2. **Off-by-one in pi function** during recursion (`pi[k-1]` vs `pi[k]`).
3. **Pattern containing the sentinel** in Z-function tricks.
4. **Index types**: use `size_t` or `int64_t` for big strings.
5. **Memory blow-up**: suffix tree for a 100 MB file is huge — use suffix array.

Try it (must-do)

1. Implement KMP. Verify on a few test cases.
2. Implement Rabin-Karp. Confirm hash-collision verification step.
3. Implement Z-function. Use it for pattern matching by `concat(p, '#', t)` trick.
4. Build a suffix array via simple $O(n^2 \log n)$ sort. Run on a small string and verify by hand.
5. Build LCP array using Kasai's algorithm. Use it to find longest repeated substring.
6. Bonus: implement Aho-Corasick on a list of 10 patterns and search a text.

Read deeper

- "Algorithms on Strings, Trees, and Sequences" by Dan Gusfield — the bible.
- *Competitive Programmer's Handbook* — clean intros to KMP, Z, suffix array.
- "BWA: a fast aligner using the Burrows-Wheeler Transform" — Heng Li et al.

Next → System-design DSA: LRU cache, bloom filter, skip list, consistent hashing.

DSA 26 — System-design data structures

> LRU cache, bloom filter, skip list, consistent hashing — the data structures that show up in senior-level "system design" interviews and in real distributed/cache systems.

LRU cache

Need: cap memory at N entries; on overflow, evict the **least recently used**.

O(1) get/put using a hash map + doubly linked list.

```
hash map: key → list node (in O(1))
doubly linked list: head = MRU, tail = LRU (insert/remove in O(1) with known node)
```

```c
typedef struct lru_node {
    int key, val;
    struct lru_node *prev, *next;
} lru_node_t;

typedef struct {
    int cap, size;
    lru_node_t *head, *tail;        // dummy nodes
    hmap_t map;                     // key -> lru_node_t*
} lru_t;

void lru_get(lru_t *l, int key, int *out) {
    lru_node_t *n;
    if (hmap_get(&l->map, key, &n) == 0) {
        // remove from current pos, move to head
        n->prev->next = n->next;
        n->next->prev = n->prev;
        n->next = l->head->next;
        n->prev = l->head;
        l->head->next->prev = n;
        l->head->next = n;
        *out = n->val;
    } else *out = -1;
}

void lru_put(lru_t *l, int key, int val) {
    // similar; if at cap, evict tail->prev
    ...
}
```

Used in: page caches, CPU caches (the policy, simplified), Memcached, Redis (LRU is one of several eviction policies), Linux's dentry cache (with twist), web caches.

Variant: LFU (least frequently used), LRU-K, ARC (Adaptive Replacement Cache used by ZFS).

LFU cache (frequency-based)

Counts accesses; evicts the least-accessed. More state; trickier to make O(1). Done with a list of “frequency buckets” + per-bucket DLLs.

Bloom filter

Need: “have we seen this key?” with tiny memory and acceptable false positives.

- **Bit array** of size m, all zero initially.
- **k hash functions** mapping keys to indices.
- Insert: set the k bits.
- Query: check the k bits. **All set** → maybe present (false positive possible). **Any unset** → definitely not.

```
typedef struct { unsigned long *bits; size_t m; int k; } bloom_t;

void bloom_add(bloom_t *b, const char *key) {
    for (int i = 0; i < b->k; i++) {
        unsigned long h = hash_with_seed(key, i) % b->m;
        b->bits[h / 64] |= 1UL << (h % 64);
    }
}

int bloom_maybe(bloom_t *b, const char *key) {
    for (int i = 0; i < b->k; i++) {
        unsigned long h = hash_with_seed(key, i) % b->m;
        if (!(b->bits[h / 64] & (1UL << (h % 64)))) return 0;
    }
    return 1;
}
```

For n elements and false positive rate p: optimal `m = -n ln p / (ln 2)^2` bits, `k = (m/n) ln 2`.

Used in: BigTable / Cassandra / RocksDB (skip disk reads when key definitely absent), web crawlers (don’t recrawl), spellcheck dictionaries, network packet sampling.

Cannot delete in classic bloom filter. **Counting bloom filter** uses small counters instead of bits → can decrement on delete. Trade memory for delete support.

Skip list

A randomized data structure giving O(log n) expected search/insert/delete with simpler code than balanced trees. Used in: LevelDB / RocksDB memtable, Java’s `ConcurrentSkipListMap`, Redis sorted sets.

Idea: linked list with “express lanes” at randomized higher levels. Each node has a height geometrically distributed.

```
level 3:  head ———————————————> NULL
level 2:  head —> 4 ———> 16 —> NULL
level 1:  head -> 2 -> 4 -> 8 -> 16 -> NULL
```

Search: start at top level; move right while next < target; drop down. Average path length ~log n.

Skip lists are **lock-free-friendly** — easier than balanced BSTs in concurrent settings.

Consistent hashing

Problem: distribute keys across N nodes such that adding/removing a node only **moves a fraction** of keys (not all of them).

Naive: `hash(key) % N`. If N changes from 3 to 4, almost every key moves. Bad.

Consistent hashing: hash both nodes and keys onto a circle (`[0, 2^32)`). A key belongs to the next node clockwise.

```
positions on a 32-bit ring:
    node A at 100
    node B at 1_000_000
    node C at 3_000_000
key  hash 500_000 → goes to node B
```

Adding node D at 700_000: only keys between B's predecessor and 700_000 move from B to D. Only 1/N of keys move on average.

Virtual nodes: each physical node hashed to many positions (200+) → smoother distribution.

Used in: DynamoDB, Cassandra, Memcached client libraries, content-delivery networks, sharded systems.

Variants: - **Rendezvous hashing** (HRW) — `for each node n, compute hash(key, n); pick the node with max hash`. Handles "weights" naturally; no ring. - **Jump consistent hashing** (Lamping & Veach 2014) — small constant time, no metadata. Used by some databases for shard placement.

Skiplist vs B-tree vs RB-tree (for indexes)

Property	Skiplist	B+tree	RB-tree
Concurrency-friendly	Yes	Tricky	Per-node lock OK
Cache-friendly	Mediocre	Yes (block-sized nodes)	Mediocre
Disk-friendly	No	Yes	No
Complexity	Simple	Complex	Mid

For in-memory + concurrent → skiplist. For on-disk → B+tree. For pure single-thread in-memory → RB-tree (low constants).

Reservoir sampling

Choose K random items uniformly from a stream of unknown length without storing all of them.

```
        // R1: keep first k. for i = k+1..n: choose to replace with prob k/i.
        for (int i = 0; i < n; i++) {
            if (i < k) reservoir[i] = a[i];
            else {
                int j = rand() % (i + 1);
                if (j < k) reservoir[j] = a[i];
            }
        }
```

Used in: log sampling, A/B testing, distributed counting.

Count-min sketch

Approximate frequency of items in a stream with sublinear memory. d hash functions, w buckets each. Insert: increment all `cms[j][h_j(x)]`. Query: take the **min** across the row of d.

Bias: never undercounts; can overcount. Trade memory for accuracy.

Used in: heavy-hitter detection, network telemetry.

HyperLogLog

Estimate the **cardinality** (number of distinct items) of a stream in tiny memory.

Idea: count leading zeros of hashes; longer streaks suggest more distinct items. Aggregate via "buckets" of m registers; harmonic mean.

Memory: ~12 KB for billions of items with ~1% error.

Used in: Google's pageview estimation, Redis's `PFCOUNT`.

In Linux / kernel-adjacent

- **Page cache**: LRU (with twists — active/inactive lists).
- **Slab allocator**: sometimes uses bloom-filter-like tricks for fast presence checks.
- **DRBD, GlusterFS, Ceph**: consistent hashing for shard placement.
- **fscrypt** key cache: LRU.

Common bugs

1. **LRU**: forgetting to move on every access; only moving on writes.
2. **Bloom filter**: not enough hash functions → high false positives. Or using non-independent hashes (XOR of similar hashes).
3. **Consistent hashing**: not enough virtual nodes → uneven distribution.
4. **Skip list**: wrong probability for level → degenerate to linked list.
5. **Reservoir sampling**: getting probabilities wrong → biased sampling.

Try it

1. Implement an LRU cache (hash + DLL) with `get` and `put` in O(1). Test eviction on cap=3.
2. Implement a bloom filter; test with 100K inserted keys; measure false-positive rate on 100K random not-inserted keys.
3. Implement a skip list with `insert`, `find`, `delete`. Compare to a BST.
4. Implement consistent hashing with V virtual nodes per physical node. Show that moving 1 node moves ~1/N of keys.
5. Implement reservoir sampling. Sample 100 items from a 10000-item stream; verify uniformity over many runs.
6. Bonus: HyperLogLog. Compare to a hash set on a 1M-cardinality stream; measure memory.

Read deeper

- “Bloom filters in probabilistic verification” — Bloom’s original paper.
- Karger et al., “Consistent Hashing and Random Trees: Distributed Caching Protocols” (the Akamai paper).
- “HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm” — Flajolet et al.
- *Designing Data-Intensive Applications* by Kleppmann — strong on LSM, replication, consensus.

Next → Concurrency-aware DS: lock-free queue, RCU, hazard pointers.

DSA 27 — Concurrent data structures

> Lock-free queues, RCU, hazard pointers — the bridge from "DSA" to "kernel concurrency." This chapter previews ideas you'll learn deeply in Part IV.

Why concurrent DS are special

Two threads accessing the same DS without synchronization can: - Tear writes (one thread sees a half-updated value). - Skip updates (lost updates in counters). - Read freed memory (use-after-free). - Deadlock.

Solutions: 1. **Lock-based** — surround critical sections with mutexes / spinlocks. 2. **Lock-free / wait-free** — use atomic ops + careful memory ordering. 3. **RCU (Read-Copy-Update)** — readers never block; writers reclaim later.

Lock-free is fast and tricky. The Linux kernel uses **all three** depending on the situation.

Atomic operations (preview)

Operations executed indivisibly. Hardware support:

- `atomic_load`, `atomic_store`
- `atomic_fetch_add` (read + add + write atomically)
- `atomic_compare_exchange` (the **CAS** — compare-and-swap):

  ```
  if (*p == expected) { *p = new; return true; } else return false;
  ```

CAS is the universal primitive for lock-free programming.

Memory orderings (C11 / kernel `smp_*`): - **relaxed**: atomic but no ordering with other accesses. - **acquire**: subsequent reads see prior writes from the releasing thread. - **release**: prior writes become visible to acquiring thread. - **seq_cst**: total order across all threads.

Full treatment in Part IV.

Lock-based concurrent queue

Simplest. One mutex; all operations take it.

```
typedef struct {
    queue_t q;
    pthread_mutex_t lock;
    pthread_cond_t  not_empty;
} mq_t;

void mq_enq(mq_t *m, int v) {
    pthread_mutex_lock(&m->lock);
```

```
        q_enq(&m->q, v);
        pthread_cond_signal(&m->not_empty);
        pthread_mutex_unlock(&m->lock);
    }

    int mq_deq(mq_t *m, int *out) {
        pthread_mutex_lock(&m->lock);
        while (m->q.size == 0) pthread_cond_wait(&m->not_empty, &m->lock);
        int rc = q_deq(&m->q, out);
        pthread_mutex_unlock(&m->lock);
        return rc;
    }
```

Bottlenecks at the mutex. Fine for moderate contention.

Single-producer, single-consumer (SPSC) lock-free queue

Two atomic indices, single producer thread, single consumer thread. Already saw the sketch in DSA-05.

```
    size_t head, tail;     // atomic
    T buf[CAP];

    // Producer
    size_t t = atomic_load_relaxed(&tail);
    if ((t + 1) % CAP == atomic_load_acquire(&head)) return FULL;
    buf[t] = item;
    atomic_store_release(&tail, (t + 1) % CAP);

    // Consumer
    size_t h = atomic_load_relaxed(&head);
    if (h == atomic_load_acquire(&tail)) return EMPTY;
    item = buf[h];
    atomic_store_release(&head, (h + 1) % CAP);
```

Why release/acquire? The producer must finish writing `buf[t]` **before** the consumer sees the updated `tail`. The release on `tail` and acquire on `tail` form the synchronization edge.

Lock-free because no thread blocks; if interrupted at any line, others can still progress. **Wait-free** because every operation finishes in O(1) steps regardless of contention.

The kernel's `kfifo` is essentially this.

MPMC queue (multiple producers, multiple consumers)

Much harder. Either: - One queue + a lock — simple, contended. - Lock-free MPMC — Michael-Scott queue (CAS-based linked list); ABA-prone (need hazard pointers / counters).

Boost / Folly / TBB have battle-tested implementations. **Don't write your own** unless for learning. Subtle bugs and they only show up under heavy load on certain CPUs.

ABA problem

CAS reads value `A`, gets preempted; another thread changes value to `B` then back to `A`; CAS succeeds, but the world has changed. Causes lost updates in lock-free linked lists.

Mitigations: - **Tagged pointers** (low/high bits as a counter). - **Hazard pointers** (tell the world "I'm using this pointer; don't free it"). - **RCU** (defer reclamation; when no readers reference, then reclaim).

RCU (Read-Copy-Update) — Linux's superpower

Designed by Paul McKenney for the Linux kernel. Used **everywhere** (dcache, networking, cgroups, SLAB, IDR, KFD).

Idea

1. Readers proceed **without any locks**, just memory barriers.
2. Writers do **copy-on-update**: clone the structure, modify the clone, atomically swap the pointer.
3. Old copy is reclaimed only after a **grace period** during which all preexisting readers have finished.

Reads scale linearly with cores. Writes pay the cost of waiting (or `call_rcu` for deferred).

Userspace API (urcu library)

```
rcu_read_lock();
struct foo *f = rcu_dereference(g_foo);
use(f);
rcu_read_unlock();
```

```
struct foo *new = malloc(sizeof(*new));
*new = *old; modify(new);
rcu_assign_pointer(g_foo, new);
synchronize_rcu();             // wait for old readers
free(old);
```

Linux kernel API

- `rcu_read_lock()` / `rcu_read_unlock()` — readers.
- `rcu_dereference(p)` — read pointer with barrier.
- `rcu_assign_pointer(p, v)` — write pointer with barrier.
- `synchronize_rcu()` — block until all readers done. **Sleeps**; can't use in atomic context.
- `call_rcu(head, fn)` — register a callback to run after the grace period; doesn't sleep.

We'll cover RCU thoroughly in Part IV chapter 4.

Hazard pointers

Each thread publishes which pointers it's currently using. Reclaimers check the published list before freeing.

Lower overhead than RCU for some workloads. Used in some lock-free libraries (Folly, TBB).

Per-CPU data

Avoid contention by giving each CPU its own copy. Update local; read by aggregating across CPUs.

Examples: - Statistics counters (one per CPU; sum at read). - Memory pools (each CPU has private slab; spillover to global). - Network packet stats (each CPU has its own; tools sum).

The kernel API: `DEFINE_PER_CPU`, `per_cpu_ptr`, `__this_cpu_inc`. We'll use these later.

Sequence locks (seqlock)

Optimized for "rare writers, frequent readers."

- Writer increments a sequence counter on enter and exit (odd = in progress, even = done).
- Reader reads counter, reads data, reads counter again. If counter changed or was odd → retry.

Used in: jiffies (kernel time), struct stat updates, GPS time. Reads scale to many cores; writers serialize.

Lock-free hash table

`rhashtable` in the Linux kernel: lock-free reads via RCU, per-bucket spinlocks for writes, automatic resize. Used in TIPC, NFT, and increasingly elsewhere.

What this chapter is for

Knowing **that** these structures exist, **why** they exist, and **roughly how** they work. You'll implement these in Part IV (the multithreading deep-dive). For DSA purposes, recognize:

- "Producer-consumer between two threads" → SPSC ring buffer.
- "Many readers, few writers, on every CPU" → RCU.
- "Counter incremented from many CPUs" → per-CPU counter, summed on read.
- "Random access to data that may resize" → rhashtable / RCU-protected hash.

Common bugs (preview)

1. **Forgetting memory barriers** — works on x86, fails on ARM.
2. **Use-after-free** in lock-free DS — fix with hazard pointers / RCU.
3. **ABA** — fix with tagged pointers.
4. **Reading a partial pointer** on 32-bit (a 64-bit pointer is two writes) — use atomic load.
5. **Treating `volatile` as enough for synchronization** — it is **not**.

Try it

1. Implement an MPMC bounded queue with a single mutex + condvar (the easy way). Stress-test with 4 producers, 4 consumers.

2. Implement a SPSC lock-free ring buffer. Use C11 atomics. Stress test with 1 producer, 1 consumer for 60 seconds; verify nothing lost.
3. Read `Documentation/RCU/whatisRCU.rst` (Linux). Try to summarize in 100 words.
4. Use the urcu library to convert a singly-linked list to RCU-protected. Insertions safe; deletions wait for grace period.
5. Read `lib/llist.c` (lock-less linked list). Identify the trick (single-CAS push, batch pop).

Read deeper

- "Is Parallel Programming Hard, and If So, What Can You Do About It?" by Paul McKenney — free PDF, the bible.
- *The Art of Multiprocessor Programming* by Herlihy & Shavit — the academic foundation.
- `Documentation/RCU/` in the Linux source.
- "C++ Concurrency in Action" by Anthony Williams.

Next → DSA in the Linux kernel — guided tour.

DSA 28 — DSA in the Linux kernel — guided tour

> A walking tour of where every DSA you've learned actually lives in the kernel source. Print this and put it on your wall.

When you finish DSA, this chapter is your "bridge" to Part V. Each item below is a real header or file; clone the kernel and `vim` it yourself.

Linked lists

- `include/linux/list.h` — doubly linked, circular, embedded `list_head`.
- `include/linux/list_bl.h` — bit-locked list (1-bit lock at head).
- `include/linux/llist.h` — lockless singly-linked list (CAS-based push, batch pop).
- `include/linux/hlist.h` — singly linked, used for hash tables (smaller than `list_head`).

Used absolutely everywhere. `git grep "struct list_head" | wc -l` will surprise you.

Hash tables

- `include/linux/hashtable.h` — generic chained hash table (uses `hlist_node`).
- `include/linux/rhashtable.h` — resizable, RCU-safe, lock-friendly. Used by netfilter, IPC, etc.
- `lib/hash.c`, `lib/rhashtable.c`.

Trees (RB-tree)

- `include/linux/rbtree.h` — generic red-black tree.
- `lib/rbtree.c` — implementation.
- `include/linux/rbtree_augmented.h` — augmenting RB-trees (e.g., interval tree).
- `include/linux/interval_tree.h`, `lib/interval_tree.c` — augmented RB for interval overlap queries.

Used in: - `mm/vma.c` (VMAs in process address space), - `kernel/sched/fair.c` (CFS runqueue), - `drivers/gpu/drm/amd/amdgpu/amdgpu_vm.c` (GPU virtual address tracking), - `drivers/iommu/` (IOMMU device tables), - many more.

Radix trees / XArray

- `include/linux/radix-tree.h`, `lib/radix-tree.c` — older radix tree (gradually deprecated).
- `include/linux/xarray.h`, `lib/xarray.c` — modern replacement: cleaner API, RCU-friendly.

Used in: - page cache (`mapping->i_pages`) — keyed by file offset, - IDR/IDA (id allocators), - `pidmap`, - block layer.

Read `Documentation/core-api/xarray.rst` — best entry point.

ID allocators

- `include/linux/idr.h`, `lib/idr.c` — ID R(adix) — assign integer IDs to pointers, lookup pointer by ID.
- `lib/ida` (small case of idr) — bitmap-based IDA for "just give me a free integer."

Used in: PCI device numbers, file descriptors (no, those are arrays — but conceptually similar), `class_id`, anywhere a "small int handle" maps to a pointer.

Bitmaps

- `include/linux/bitmap.h`, `lib/bitmap.c` — array-of-longs bit operations.
- `include/linux/bitops.h` — single-bit ops, with atomic variants.

Used in: CPU masks (`cpumask`), node masks (NUMA), interrupt masks, GPU VMID allocation, slab allocator state, page allocator state.

Queues

- `include/linux/kfifo.h`, `kernel/kfifo.c` — generic FIFO. SPSC lock-free.
- `include/linux/circ_buf.h` — small circular buffer macros.
- DRM/scheduler queues — `drivers/gpu/drm/scheduler/`.

Heaps / priority queues

- `lib/min_heap.c`, `include/linux/min_heap.h` — generic min-heap.
- Used by tracing for keeping bounded sets of "top events."

CFS scheduler uses RB-tree on `vruntime` instead of a heap.

Per-CPU data

- `include/linux/percpu.h`, `mm/percpu.c`.
- `DEFINE_PER_CPU(type, var)`, `this_cpu_inc(var)`, `per_cpu(var, cpu)`.

Used pervasively: counters, allocators (slab uses per-CPU magazines), networking stats.

Refcounting

- `include/linux/refcount.h`, `lib/refcount.c` — saturating counter, robust against overflow/underflow.
- `include/linux/kref.h` — wrap a refcount + a release callback.

Used in: file objects, `struct device`, `drm_file`, GEM objects, `drm_gem_object`, basically anything shared.

Wait queues / completions

- `include/linux/wait.h`, `kernel/sched/wait.c` — wait queues for "sleep until condition."
- `include/linux/completion.h` — one-shot waits ("done"-style signals).

Used in: I/O wait, fence completion, GPU job submission flow.

Locking primitives

- `include/linux/spinlock.h` — spinlocks.
- `include/linux/mutex.h` — sleeping mutex.
- `include/linux/rwsem.h` — reader/writer semaphore.
- `include/linux/rcupdate.h` — RCU.
- `include/linux/seqlock.h` — sequence locks.

Will be deeply covered in Part V/IV.

DMA-fences (GPU specifically)

- `include/linux/dma-fence.h`, `drivers/dma-buf/dma-fence.c` — the queue + waiter pattern for GPU completion. A reference-counted, signaled-once object with callbacks. Built on top of `wait_queue_head_t`.

GPU job ordering and inter-driver synchronization rely on this.

DMA buffer sharing

- `include/linux/dma-buf.h`, `drivers/dma-buf/dma-buf.c` — a graph of buffer objects shared across devices, with attach/map/detach lifecycle. Used by GPU, V4L2, USB, etc.

Workqueues / kthreads

- `include/linux/workqueue.h`, `kernel/workqueue.c`.
- `kthread_create()`, `kthread_run()`.

Used to defer work outside atomic context (interrupts).

Slab / page allocator

- `mm/slab.c`, `mm/slub.c`, `mm/slob.c` — three slab implementations (SLUB is default).
- `mm/page_alloc.c` — buddy page allocator.

Internally uses: per-CPU caches (magazines), partial-slab lists, free-page bitmaps, NUMA-aware free lists, RCU for freeing slabs.

Page cache

- `mm/filemap.c`, `mm/readahead.c`, `mm/swap.c`.

Uses XArray to map file offset → struct page; LRU lists for active/inactive pages; mark-and-sweep for dirty pages.

TCP/networking

- `net/ipv4/tcp.c` — TCP state machine.
- `net/core/dev.c` — packet path.
- `net/core/skbuff.c` — sk_buff (the skb), the kernel's network packet abstraction.

Heavy use of: linked lists (skb queues), hash tables (connection tracking), per-CPU stats, RCU for routes.

VFS (virtual filesystem)

- `fs/dcache.c` — dentry cache (recently-resolved paths). Hash table + LRU.
- `fs/inode.c` — inode cache.
- `fs/file_table.c` — open file table.

Block layer

- `block/blk-mq.c` — multi-queue block layer (per-CPU request queues, dispatch into device).

Uses: bio chains (linked lists), per-CPU request pools, hash tables for in-flight tracking.

Tracing & debugging

- `kernel/trace/ring_buffer.c` — per-CPU lock-free ring buffer for ftrace events. **Must read** if you want to understand high-perf logging.
- `kernel/trace/trace.c` — tracing core.

Where the kernel doesn't use textbook DSA

- **No B-trees in core kernel.** Filesystems have them (`fs/btrfs/ctree.c`); core kernel doesn't.
- **No suffix arrays / tries** in core kernel. Some places use radix tree.
- **No skip lists.** RB-tree usually wins.
- **No regular hash tables in interrupt context** unless they're specifically lock-free / RCU-safe.

Reading order to study real kernel DSA

If you have time, read in this order (small files first):

1. `include/linux/list.h` — short, sweet, masterful.
2. `include/linux/llist.h`.
3. `include/linux/hlist.h`.
4. `include/linux/hashtable.h`.

5. `include/linux/rbtree.h` and `lib/rbtree.c`.
6. `include/linux/interval_tree.h` and `lib/interval_tree.c`.
7. `Documentation/core-api/xarray.rst`, then `include/linux/xarray.h`.
8. `kernel/trace/ring_buffer.c` — masterclass.
9. `lib/refcount.c`, `lib/kref.c`.
10. Documentation/RCU/whatisRCU.rst.

This is **exactly the path that turns a DSA student into a kernel reader.**

Try it

1. Pick three structures from this chapter. For each, find at least one driver in `drivers/gpu/drm/amd/amdgpu/` that uses it. Note the file:line.
2. Read `include/linux/list.h` end to end. Decode `list_for_each_entry`.
3. Read `include/linux/rbtree.h` and identify the macro that recovers the containing struct from an `rb_node`.
4. Read `include/linux/interval_tree.h` and explain (in your notebook) what `__subtree_last` is for.
5. `git grep "kfifo_put"` in the kernel. Pick three usages; read the surrounding code.
6. Trace one allocation through SLUB: `kmalloc` → `__kmalloc` → `kmem_cache_alloc` → … — until you hit a lock or a per-CPU slot.

Closing

This finishes Part DSA. **You now have a base in:** - The standard interview DSA toolkit. - The data structures that actually run the kernel.

Next stage: **Stage 5 of the LEARNING PATH** — userspace systems programming.

→ Part USP — Userspace systems programming.

Part USP — Userspace systems programming

> **Stage 5 of the LEARNING PATH. 4–6 weeks at 1 hr/day.**

You've done C and DSA. Time to **master Unix as a user**. Every concept here has a kernel counterpart you'll learn later — but doing it from userspace first is **non-negotiable**.

Why this part exists

Most books jump from "C basics" to "kernel modules." That's a leap. Between C and the kernel, there's a whole layer:

- How a program becomes a process.
- How processes communicate.
- How threads share memory.
- How files actually get to disk.
- How sockets push bytes across the network.
- How `/proc` exposes the kernel back to userspace.

A junior engineer who's never written `fork()`, `pipe()`, `epoll_wait()`, or `mmap()` has no business poking at GPU kernel code. **Master userspace first.**

When you reach the kernel, every concept will feel familiar — "oh, the kernel `cdev_alloc` is just like `open()`-but-from-the-other-side."

Table of contents

#	Chapter	What you'll be able to do
1	Unix philosophy & system call mental model	Read a man page; trace any program's system calls
2	Processes — fork, exec, wait, exit	Write a tiny shell
3	Signals — interrupts to processes	Handle Ctrl-C, deferred quits, parent notification
4	Pipes & FIFOs — the simplest IPC	Chain commands; build pipelines
5	Shared memory & mmap	Zero-copy IPC; map a file as a buffer
6	Message queues, semaphores, futexes	Synchronize processes
7	Sockets — TCP & UDP	Write a chat server, an echo server
8	I/O multiplexing — select, poll, epoll, io_uring	Handle 10,000 connections in one thread

#	Chapter	What you'll be able to do
9	pthreads deep — locks, condvars, barriers	Write robust multi-threaded servers
10	File I/O deep — fd, blocking, O_DIRECT, sendfile	Read/write large files efficiently
11	/proc, /sys, /dev — the magic filesystems	Inspect any running system
12	Userspace performance & debug tools	Use strace, ltrace, perf, valgrind, gdb fluently

What you'll build

By the end:

- a tiny **shell** that does `fg`, `bg`, pipes, signals;
- a **TCP echo server** scaling to 10,000 connections via `epoll`;
- a **multi-process** producer/consumer with shared memory;
- a **file copy** tool using `sendfile` for zero-copy speed;
- familiarity with `strace`, `perf`, `valgrind`, `gdb` such that you can debug any C program.

These projects + your DSA repo = **the portfolio** that gets you the FTE phone screen.

Mental model

In Unix:

> **Everything is a file descriptor**, **every kernel service is a system call**, **every system call is a number** (`/usr/include/asm-generic/unistd.h`).

A process has: - a memory address space, - a table of open file descriptors, - credentials (uid, gid), - signal handlers, - threads sharing all of the above.

Once you internalize this, the kernel side becomes obvious — the kernel is **the implementer of the system calls**, and a driver is **a backend for some file's read/write/ioctl**.

How to study this part

Each chapter: 1. Read it. Type all examples. 2. Run them. Modify them. 3. Use `strace -f` to see the system calls actually fired. 4. Read the relevant `man` pages — `man 2 fork`, `man 2 epoll_wait`, etc. 5. Do the exercises.

Go in order. Don't skip — chapters depend on each other.

→ USP-01 — Unix philosophy & system call mental model.

USP 1 — Unix philosophy & the system call mental model

> "Do one thing well." Every program in Unix communicates with the kernel through system calls. Master both ideas; everything else falls in.

The Unix philosophy

The four classic rules:

1. **Make each program do one thing well.**
2. **Expect the output of every program to become the input to another.**
3. **Design and build software, even operating systems, to be tried early, ideally within weeks.**
4. **Use tools in preference to unskilled help to lighten a programming task.**

Practically: - `cat`, `grep`, `sort`, `uniq`, `wc` each do one thing. Combine via pipes. - Text streams as the universal interface. - "It's done" = it works. Iterate.

This shapes every Linux/Unix system you'll ever touch.

Everything is a file (descriptor)

Open a regular file → fd. Open a TCP connection → fd. Open a pipe → fd. Open a GPU device → fd (`/dev/dri/card0`).

A **file descriptor** (fd) is a small integer index into a per-process table. Every fd points to a kernel **file object** which knows how to do `read`, `write`, `ioctl`, `mmap`, `poll`, `close`.

The fd table is per-process, copied on `fork`, kept across `exec` (unless `O_CLOEXEC`).

```
int fd = open("hello.txt", O_RDONLY);
char buf[100];
ssize_t n = read(fd, buf, sizeof(buf));
close(fd);
```

This is the **same shape** as opening `/dev/dri/card0` and `read`-ing from it — that's how userspace talks to the GPU driver. `ioctl` is the slightly more structured variant.

System calls — the boundary

A **system call** is the only way a userspace program asks the kernel to do something. They: - run privileged code in **kernel mode**, - have a small numbered table (around 400 on x86-64), - are accessed via a special instruction (`syscall` on x86-64, `svc` on ARM64), - return a value or `-1` (errno set on failure).

You **never** call the syscall instruction directly in normal code. Libc wraps them. `read()` is a libc function that ultimately fires the `read` syscall.

strace — your microscope

Watch any program's system calls:

```
strace -f -e trace=open,openat,read,write,close ./myprogram
strace -c ./myprogram                          # summary stats
strace -p PID                                  # attach to running process
strace -tt -T ./myprogram                      # with timestamps and durations
```

If you can read `strace` output, you can understand any program's interaction with the kernel. **Do this often.**

errno — how syscalls report errors

```
#include <errno.h>
ssize_t n = read(fd, buf, len);
if (n < 0) {
    perror("read");                    // prints "read: Bad file descriptor"
    fprintf(stderr, "errno=%d %s\n", errno, strerror(errno));
}
```

Common errno values: - `ENOENT` — no such file. - `EACCES` — permission denied. - `EINVAL` — invalid argument. - `EAGAIN` / `EWOULDBLOCK` — try again later (non-blocking I/O). - `EINTR` — system call interrupted by signal. - `EBADF` — bad fd. - `EIO` — generic I/O error. - `ENOMEM` — out of memory.

Always check syscall return values. Always `errno` after a `-1`.

`man errno` gives the full list with descriptions.

Man pages — your authoritative reference

Sections you care about: - **2** — system calls (`man 2 read`, `man 2 fork`). - **3** — library functions (`man 3 printf`, `man 3 malloc`). - **5** — file formats (`man 5 proc`, `man 5 services`). - **7** — overviews (`man 7 signal`, `man 7 epoll`, `man 7 socket`).

`apropos signal` lists everything related. Use it.

If you don't have a habit of reading `man` pages: get one **today**. Linux engineers live in `man`.

The standard fd numbers

```
0 = stdin
1 = stdout
2 = stderr
```

These are inherited by every process at start. `printf` writes to fd 1. `perror` writes to fd 2. Pipes and shells redirect them.

`./prog > out.txt 2>&1` — redirect stdout to file, then dup stderr onto stdout's destination.

Process address space — preview

A typical Linux process layout (top to bottom):

```
0x7fff...  +-------------------+
           | kernel mapping    |  (read-only window for vDSO etc)
           +-------------------+
           | stack (grows down)|
           |        ↓          |
           +-------------------+
           |     ...gap...     |
           +-------------------+
           | mmap region       |  (shared libs, mmap files)
           |        ↑          |
           +-------------------+
           |    ...gap...      |
           +-------------------+
           | heap (brk grows up)|
           |        ↑          |
           +-------------------+
           | .bss (uninit data)|
           | .data (init data) |
           | .text (code)      |
0x400000   +-------------------+
```

Ranges are virtual. The MMU maps them to physical pages. We'll go deeper in OS chapters.

`cat /proc/$$/maps` shows your shell's layout right now.

Shells, environment, arguments

A program receives: - `argc`, `argv[]`: command line. - `envp[]` (or `environ`): environment variables. - A working directory. - Open fds 0, 1, 2. - A signal mask + handlers (default). - Resource limits (`ulimit`).

```c
int main(int argc, char *argv[], char *envp[]) {
    for (int i = 0; i < argc; i++) printf("argv[%d]=%s\n", i, argv[i]);
    for (char **e = envp; *e; e++) printf("%s\n", *e);
    return 0;
}
```

"Hello, system calls"

Strip away libc:

```c
#include <unistd.h>
int main(void) { write(1, "hi\n", 3); return 0; }
```

`strace ./hi` will show only one `write` syscall (plus startup). Compare to `printf` — buffered, so `strace` shows the same write but at exit / flush.

A note on glibc / libc

`libc` (glibc on Linux) provides: - syscall wrappers (`open`, `read`, `write`, etc.), - C standard library (`stdio.h`, `string.h`), - thread library (in glibc this is integrated; previously a separate `libpthread`), - locale, internationalization, - math, dynamic loader, etc.

`man 7 libc` is your starting point.

For minimalism / interview understanding, **musl libc** is much smaller, more readable. Worth reading some.

Try it

1. `strace -c ls -la /usr/bin > /dev/null` — what's the most-frequent syscall?
2. Write a 3-line program that uses `write()` to print "hi" without `printf`. Compile, `strace` it.
3. `man 2 open` — read it. Identify what each `O_*` flag means. Try `O_CREAT | O_TRUNC | O_WRONLY` then write into it.
4. Find your shell's PID. `cat /proc/$$/maps` and identify the heap, stack, and shared libs.
5. Check three errno values you don't know yet. Memorize ENOENT, EAGAIN, EINTR.
6. Read `man 7 fanotify` (or anything in section 7). Notice how documentation explains overall subsystems.

Read deeper

- *The Linux Programming Interface* (TLPI) by Michael Kerrisk — the bible for this material. Buy this book.
- *Advanced Programming in the Unix Environment* (APUE) by Stevens — the older bible; still excellent.
- `man 2 syscalls` — list of every Linux system call with introduction version.

Next → Processes — fork, exec, wait, exit.

USP 2 — Processes: fork, exec, wait, exit

> The four system calls that make Unix what it is.

Process — what is it really?

A **process** is a running program plus its environment: - a unique PID (process ID), - its own virtual address space (memory), - a file descriptor table, - credentials (uid/gid), - signal handlers, - a parent process, - maybe children.

`getpid()` gives your PID. `getppid()` gives your parent's. `ps -ef` lists all.

fork() — make a copy of yourself

```
#include <unistd.h>
pid_t pid = fork();
if (pid == 0) {
    // we're the CHILD. fresh PID.
    printf("child: pid=%d ppid=%d\n", getpid(), getppid());
} else if (pid > 0) {
    // we're the PARENT. pid is the child's PID.
    printf("parent: child=%d\n", pid);
} else {
    perror("fork");
}
```

After `fork`: - A new process exists. - Both processes resume **at the same line** (the line after `fork()`). - They have **identical** memory, fds, working directory, etc. — but separate copies (copy-on-write under the hood). - `fork()` returns 0 in the child, the child's PID in the parent.

This is the most fundamental Unix idiom. Read it three times.

exec() — replace yourself with another program

`exec` family is **6 functions** (`execl`, `execlp`, `execv`, `execvp`, `execve`, `execvpe`) — same syscall, different convenience.

```
execl("/bin/ls", "ls", "-la", NULL);
// only reached if exec FAILED
perror("execl");
```

`exec` does not return on success. The current process **becomes** `/bin/ls`. PID stays the same; memory/text/data are replaced.

Variants: - `execl` — args as varargs, full path required. - `execlp` — searches PATH. - `execv` — args as `char *argv[]`. - `execve` — full control: env passed explicitly.

fork + exec — the standard combo

To run a new program **without losing yourself**, fork then exec:

```
pid_t pid = fork();
if (pid == 0) {
    execl("/bin/ls", "ls", "-la", NULL);
    perror("execl"); _exit(127);
}
int status;
waitpid(pid, &status, 0);
```

This is exactly how a shell runs commands. The child becomes `ls`; the parent waits.

wait() / waitpid() — reap children

When a process exits, its kernel state lingers as a **zombie** until the parent calls `wait`. `wait` returns the exit status.

```
int status;
pid_t child = waitpid(pid, &status, 0);
if (WIFEXITED(status)) printf("exited: %d\n", WEXITSTATUS(status));
if (WIFSIGNALED(status)) printf("killed by signal %d\n", WTERMSIG(status));
```

If you don't reap, zombies accumulate (visible in `ps`). Big bug source in long-running programs.

`waitpid(-1, ...)` waits for any child. `waitpid(pid, ..., WNOHANG)` returns immediately if no child has exited (poll style).

exit() — proper termination

```
exit(0);     // calls atexit handlers, flushes stdio, closes fds, then _exit
_exit(0);    // raw: skip atexit, do not flush. Use in child after fork() failure.
return 0;    // from main: same as exit(0)
```

After `fork()` in the child, **before** `exec`, use `_exit` on errors — `exit` would flush parent's stdio buffers (which were inherited as a copy) twice.

Orphans, zombies, init

- **Zombie**: dead but not yet waited for.
- **Orphan**: parent died first; orphans are re-parented to PID 1 (init / systemd), which reaps them automatically.

Long-running daemons traditionally `fork` and let the parent exit so the child becomes an orphan adopted by init — "daemonization."

A tiny shell

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```c
#include <string.h>
#include <stdlib.h>

int main(void) {
    char line[1024];
    while (printf("$ "), fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = 0;        // trim newline
        if (strcmp(line, "exit") == 0) break;
        if (line[0] == 0) continue;

        char *argv[64]; int n = 0;
        char *t = strtok(line, " ");
        while (t && n < 63) { argv[n++] = t; t = strtok(NULL, " "); }
        argv[n] = NULL;

        pid_t pid = fork();
        if (pid == 0) {
            execvp(argv[0], argv);
            perror("execvp"); _exit(127);
        } else if (pid > 0) {
            int status; waitpid(pid, &status, 0);
        }
    }
    return 0;
}
```

That's a (very minimal) shell. Add pipes, redirects, signals, environment substitution, and you have bash.

Process groups & sessions (preview)

- **Process group**: a set of processes that can be signaled together (e.g., `Ctrl-C` kills the whole foreground pipeline).
- **Session**: a set of process groups, typically one terminal.

`setsid()`, `setpgid()`. Critical for shell job control and daemonization. We'll cover these only briefly — TLPI has a great chapter.

getpid, getppid, getuid, getgid

Identity functions. Cheap. Often used in logging.

`getuid()` returns real user id; `geteuid()` effective (after setuid bit).

Resource limits

```c
#include <sys/resource.h>
struct rlimit r;
getrlimit(RLIMIT_NOFILE, &r);
printf("max fds: soft=%lu hard=%lu\n", r.rlim_cur, r.rlim_max);
```

`ulimit -n` in the shell. Used to cap memory, CPU, fd count, core dumps. Essential when running services.

Daemonization (briefly)

Modern systemd-managed services don't daemonize themselves — systemd manages their lifecycle. But classic daemon recipe:

1. `fork()`; parent exits.
2. `setsid()` to detach from terminal.
3. `fork()` again (so we can never reacquire a terminal).
4. `chdir("/")` so we don't keep a busy mount.
5. `umask(0)`.
6. Close fds 0, 1, 2; reopen to `/dev/null`.

Modern: just write a normal foreground program and let systemd run it.

Common bugs

1. **Not checking fork's return value** — race conditions when fork fails.
2. **Forgetting to reap children** — zombie accumulation.
3. **Calling exit() in the child** before exec — flushing buffered I/O twice.
4. **Misordering**: parent reads from a pipe before child has written — depends on schedule, races.
5. **Inherited fds**: parent's fds are inherited; sometimes you want to close them with `O_CLOEXEC` so exec'd program doesn't get them.
6. **PATH security**: `execlp("ls", ...)` searches PATH. Untrusted PATH → arbitrary code.

Try it

1. Write the tiny shell above. Add `cd` (must be built-in — why?). Add `&` for background.
2. Spawn 5 children. Use `waitpid` to reap them in order; print each exit status.
3. Demonstrate a zombie. Fork a child; in parent, sleep 30s without `wait`. In another terminal, `ps -ef | grep defunct`.
4. Demonstrate orphan. Parent exits before child. Inspect parent (ppid) inside the child after parent has died.
5. Read `man 2 fork`. What's NOT inherited by the child? (Hint: pending alarms, file locks under some conditions, async I/O.)
6. Bonus: implement `&` background and `wait` to reap them when they finish, without using SIGCHLD yet (we do signals next).

Read deeper

- TLPI chapters 24, 25, 26, 27 — process creation, exec, exit, monitoring.
- `man 2 fork`, `man 2 execve`, `man 2 wait`, `man 2 _exit`.
- The original "fork" paper (1965, Project MAC) is a curiosity worth a look.

Next → Signals.

USP 3 — Signals: interrupts to processes

> A signal is the kernel saying “drop what you’re doing — this happened.” Powerful, ancient, full of pitfalls.

What signals are

A small fixed set of “interrupt numbers” the kernel can deliver to a process. Each has a name and a default action.

`man 7 signal` lists them. Common ones:

Signal	#	Default	Triggered by
SIGHUP	1	term	terminal hangup; “reload config” by convention
SIGINT	2	term	Ctrl-C
SIGQUIT	3	core dump	Ctrl-\
SIGILL	4	core	illegal instruction
SIGTRAP	5	core	debugger
SIGABRT	6	core	`abort()`
SIGBUS	7	core	bad memory access
SIGFPE	8	core	floating-point exception
SIGKILL	9	term	**uncatchable**
SIGUSR1, SIGUSR2	10, 12	term	user-defined
SIGSEGV	11	core	segfault
SIGPIPE	13	term	wrote to pipe with no reader
SIGALRM	14	term	`alarm()` timer
SIGTERM	15	term	polite “please exit”
SIGCHLD	17	ignore	child status changed
SIGCONT	18	continue	resume after stop
SIGSTOP	19	stop	**uncatchable**
SIGTSTP	20	stop	Ctrl-Z

`SIGKILL` and `SIGSTOP` cannot be caught, blocked, or ignored. The kernel just delivers them.

Default actions

- **term** — exit.
- **core** — exit + write core dump.
- **stop** — pause.

- **continue** — resume.
- **ignore** — discard.

You can override defaults (except KILL/STOP).

Sending signals

```
kill(pid, SIGTERM);          // send to one process
kill(-pgid, SIGINT);          // send to a process group
raise(SIGABRT);               // send to self
pthread_kill(thread, SIGUSR1);// send to a specific thread (in your process)
```

From shell: `kill -TERM 1234`, `kill -9 1234`.

Catching signals — sigaction (the right way)

```
#include <signal.h>
#include <stdio.h>

void on_int(int sig) {
    write(2, "got SIGINT\n", 11);     // async-signal-safe!
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = on_int;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;          // restart interrupted syscalls
    sigaction(SIGINT, &sa, NULL);

    for (;;) pause();
    return 0;
}
```

Don't use `signal()` — its semantics differ between Unixes. Use `sigaction`.

`SA_RESTART` makes most syscalls auto-retry instead of failing with `EINTR`. Useful default.

Async-signal-safe — the rule

Inside a signal handler, you may **only** call functions on the async-signal-safe list: - `write`, `_exit`, `kill`, `signal`, `sigaction`, `sigprocmask`, … - **Not** `printf`, `malloc`, `free`, `fprintf`, `fopen`, `fputs`, anything that takes locks or uses heap.

Why: a signal can interrupt your program **anywhere**. If it interrupts `malloc` and you call `malloc` in the handler → recursive call into a half-mutated allocator → corruption.

Common pattern: handler **only sets a flag**; main loop checks the flag.

```
volatile sig_atomic_t got_int = 0;
void on_int(int sig) { got_int = 1; }

int main(void) {
    /* ... install handler ... */
    while (!got_int) {
        /* normal work */
    }
```

```
        /* clean shutdown */
    }
```

`sig_atomic_t` is a type that can be read/written atomically wrt signal delivery.

Self-pipe trick — bridging signals to a normal event loop

Make a pipe; in the handler, `write(pipefd, "x", 1)`. Add the read end to your `select` / `poll` / `epoll` set. Now signals become events in your event loop. Used in many real servers (libev, libuv).

signalfd — modern alternative

```
    sigset_t mask; sigemptyset(&mask); sigaddset(&mask, SIGINT);
    sigprocmask(SIG_BLOCK, &mask, NULL);
    int fd = signalfd(-1, &mask, 0);
    struct signalfd_siginfo si;
    read(fd, &si, sizeof(si));
```

Now signals come as readable data on a file descriptor — easy to integrate with `epoll`. Linux-specific.

Blocking signals — sigprocmask

```
    sigset_t set, oldset;
    sigemptyset(&set); sigaddset(&set, SIGINT);
    sigprocmask(SIG_BLOCK, &set, &oldset);
    // SIGINT pending but not delivered
    sigprocmask(SIG_SETMASK, &oldset, NULL);
```

Use to protect critical sections that must not be interrupted. Or to mask before `pselect` / `ppoll` so you can atomically unmask while waiting.

SIGCHLD — children dying

When a child dies: - The parent gets `SIGCHLD` (default action: ignore). - The child becomes a zombie until reaped.

To auto-reap with `SIGCHLD`:

```
    void reap(int sig) {
        while (waitpid(-1, NULL, WNOHANG) > 0) ;
    }
    struct sigaction sa = { .sa_handler = reap, .sa_flags = SA_RESTART | SA_NOCLDSTOP };
    sigemptyset(&sa.sa_mask);
    sigaction(SIGCHLD, &sa, NULL);
```

Important: signals are not queued. If 5 children die at the same time, you get **one** SIGCHLD. Always loop with `WNOHANG` until no more.

Real-time signals (SIGRTMIN..SIGRTMAX)

- Are queued.
- Can carry a small integer payload (`sigqueue(pid, sig, value)`).
- Used in real-time apps (POSIX timers, async I/O).

Most apps use the standard signals.

Common signals you'll write handlers for

- **SIGINT/SIGTERM** — graceful shutdown.
- **SIGHUP** — reload config (servers).
- **SIGUSR1/SIGUSR2** — toggle log level, dump stats.
- **SIGCHLD** — reap children (or `SA_NOCLDWAIT` to auto-discard).
- **SIGPIPE** — usually `SIG_IGN` so writes return EPIPE and you handle it.

```
        signal(SIGPIPE, SIG_IGN);   // okay here; for ignore.
```

Signals and threads

Each thread has its own signal mask. A signal directed at the **process** is delivered to **any** thread that doesn't block it. Use `pthread_sigmask` to control per-thread masks. Best practice: **block signals in worker threads; have one dedicated signal-handling thread** that calls `sigwaitinfo`.

Signals from the kernel (faults)

`SIGSEGV` is delivered when the kernel detects you accessed unmapped memory. The siginfo has `si_addr` — the bad address.

You can install a handler and try to recover (e.g., for JITs that handle SEGV from speculative reads). Tricky. Also used by garbage collectors and protected-memory tricks (mark a region read-only and SEGV → handle the write).

Common bugs

1. **Calling unsafe functions in handlers** — undefined behavior. Always remember the list.
2. **Forgetting `volatile sig_atomic_t`** for flag — compiler may optimize away your re-checks.
3. **`signal()` instead of `sigaction()`** — older style, surprising semantics on signal storm.
4. **Not handling EINTR** when not using `SA_RESTART` — your `read` returns -1 errno=EINTR; handle it.
5. **Lost SIGCHLD** because you only call `wait` once.
6. **Race between sigprocmask and pause()** — use `sigsuspend` or `pselect` to atomically unmask + wait.

Try it

1. Write a program that traps `SIGINT` and instead of exiting, prints "press Ctrl-C twice to quit."
2. Use `alarm(5)` and a `SIGALRM` handler to break out of a `read()` if no data arrives in 5 seconds.
3. Use `signalfd` and `epoll` to handle SIGTERM as part of an event loop.
4. Make a parent that forks 5 children. Auto-reap with SIGCHLD and `WNOHANG`. Print PIDs as they exit.
5. Simulate a server that reloads config on SIGHUP. Use a flag in the handler; in main loop, when flag set, reload and clear.
6. Bonus: `gdb` your program; trigger SIGSEGV by writing to NULL. Observe how gdb stops on signal.

Read deeper

- TLPI chapters 20–22 — signals, signal handlers, advanced signals.
- `man 7 signal`, `man 7 signal-safety` (lists async-signal-safe functions).
- *Linux Programming Interface* author's website has signal examples.

Next → Pipes & FIFOs.

USP 4 — Pipes & FIFOs

> The simplest IPC. `cmd1 | cmd2` is two processes sharing a pipe.

Anonymous pipes

```
int fds[2];
pipe(fds);                // fds[0] = read end, fds[1] = write end
```

A pipe is a **kernel ring buffer** (typical 64 KB). One process writes; another reads. Reads block until data; writes block when full. `EOF` on read when all write ends closed.

Anonymous pipes are inherited across `fork`, but otherwise can only be shared between related processes (i.e., a process and its descendants). For unrelated processes, use FIFOs (next section).

Pipe + fork = command pipeline

```
int p[2]; pipe(p);
pid_t a = fork();
if (a == 0) {
    dup2(p[1], 1);          // stdout → pipe write
    close(p[0]); close(p[1]);
    execlp("ls", "ls", "-l", NULL);
    _exit(127);
}
pid_t b = fork();
if (b == 0) {
    dup2(p[0], 0);          // stdin → pipe read
    close(p[0]); close(p[1]);
    execlp("wc", "wc", "-l", NULL);
    _exit(127);
}
close(p[0]); close(p[1]);    // parent has no use for them
waitpid(a, NULL, 0);
waitpid(b, NULL, 0);
```

This is exactly what the shell does for `ls -l | wc -l`. **You must close unused ends in every process** — otherwise the reader never sees EOF and hangs forever.

dup2 — replace a file descriptor

`dup2(oldfd, newfd)`: closes `newfd` and makes it refer to the same file as `oldfd`. Used to "wire" pipes/files to stdin/stdout/stderr before exec.

```
dup2(fd, 1);    // make stdout refer to whatever fd was
```

FIFOs (named pipes)

`mkfifo("/tmp/mypipe", 0644)` creates a special file in the filesystem. Any process can `open` it and read/write — no relationship needed.

Opening `O_RDONLY` blocks until someone opens `O_WRONLY`, and vice versa. Or use `O_NONBLOCK`.

```
mkfifo("/tmp/p", 0644);
// process A:
int fd = open("/tmp/p", O_WRONLY); write(fd, "hi", 2);
// process B:
int fd = open("/tmp/p", O_RDONLY); read(fd, buf, 2);
```

Same kernel ring buffer behind the scenes; you just have a filesystem name to find it.

Modern alternatives: Unix domain sockets (more featureful), eventfd, message queues.

SIGPIPE — when the reader is gone

If you `write` to a pipe whose **read end is closed**, the kernel sends you `SIGPIPE`. Default action: terminate. Useful (auto-die when downstream gone), often surprising.

To handle gracefully:

```
signal(SIGPIPE, SIG_IGN);       // now write returns -1 with errno=EPIPE
ssize_t n = write(fd, buf, len);
if (n < 0 && errno == EPIPE) { /* downstream gone */ }
```

Servers usually `SIG_IGN` SIGPIPE.

Atomic writes — PIPE_BUF

Writes ≤ `PIPE_BUF` bytes (4096 on Linux) are **atomic**: not interleaved with concurrent writers. Bigger writes may be split.

This matters when multiple processes write to the same pipe.

Pipe sizes

Default 64 KB on Linux. `fcntl(fd, F_SETPIPE_SZ, 1<<20)` to grow.

For high-throughput pipelines, big pipes and `splice()` (zero-copy between fds) make a real difference.

splice / tee / vmsplice

Linux extras for zero-copy pipe data movement:

- `splice(fd_in, NULL, fd_out, NULL, len, 0)` — moves bytes between an fd and a pipe (or pipe to fd) without copying through userspace.
- `tee()` — duplicates data between two pipes.
- `vmsplice()` — splice from userspace memory.

Used by tools that move large data between sockets/files efficiently.

Pipes vs sockets

- Pipes: byte stream, half-duplex, simple, fast for parent-child.
- Unix domain sockets: byte or datagram, bidirectional, can pass file descriptors between processes (`SCM_RIGHTS`), can authenticate the peer's UID. The choice for non-trivial IPC.

Common bugs

1. **Forgetting to close write ends** in readers: read never sees EOF.
2. **Forgetting to close read ends** in writers: SIGPIPE never fires; the system just buffers.
3. **Using a single pipe in both directions**: no, pipes are one-way. Make two, or use a socketpair.
4. **Signal-on-write surprise**: not ignoring SIGPIPE → mysterious crashes.
5. **Reads return 0 (EOF), not -1**: handling 0 as success buffer is a classic bug.

socketpair — bidirectional cousin of pipe

```
int sv[2];
socketpair(AF_UNIX, SOCK_STREAM, 0, sv);
```

Like `pipe()` , but full-duplex. Use when you need to read **and** write between two processes. Plus you can pass fds via SCM_RIGHTS.

Try it

1. Write a program that pipes `ls | wc -l` from C using `pipe` , `fork` , `dup2` , `exec` . Verify count matches `bash -c 'ls | wc -l'` .
2. Build a 3-stage pipeline: `cat file | grep error | sort -u` .
3. Make a FIFO. Have two unrelated terminals connect: writer in one, reader in another.
4. Trigger SIGPIPE deliberately. Then ignore it; observe `write` returning -1 with EPIPE.
5. Write a program where two processes communicate bidirectionally via a `socketpair` .
6. Bonus: implement `tee` (the command) using two open files and pipe data through both.

Read deeper

- TLPI chapters 44, 45 (pipes, FIFOs), 57–58 (sockets, Unix domain sockets).
- `man 2 pipe` , `man 2 splice` , `man 7 pipe` .

Next → Shared memory & mmap.

USP 5 — Shared memory & mmap

> The fastest IPC: zero-copy because there's literally one copy, shared. And `mmap` is the same trick on a file.

What `mmap` actually does

`mmap` asks the kernel to make a region of memory in your process appear as a window onto something — usually a file or a chunk of anonymous memory. After `mmap`, you read/write **memory** and the kernel handles "what to do with that data."

```
#include <sys/mman.h>

void *p = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
if (p == MAP_FAILED) perror("mmap");
// now p[0..len) is a window onto fd from offset 0
munmap(p, len);
```

Key knobs: - `PROT_READ`, `PROT_WRITE`, `PROT_EXEC` — protections. - `MAP_SHARED` — writes flush to the file (or shared with other processes). - `MAP_PRIVATE` — copy-on-write; writes never reach the file. - `MAP_ANONYMOUS` — no file; just memory (for `MAP_SHARED` between forked children, or for a custom allocator). - `MAP_FIXED` — pin to a specific address (dangerous; use `MAP_FIXED_NOREPLACE` instead).

File-backed mmap

```
int fd = open("data.bin", O_RDWR);
struct stat st; fstat(fd, &st);
char *p = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
p[0] = 'X';                        // writes the file
msync(p, st.st_size, MS_SYNC);     // force flush
munmap(p, st.st_size); close(fd);
```

Beautiful when you want to treat a big file as a giant array. Used by: - the dynamic loader (mapping `.so` files into memory), - databases (LMDB, SQLite mmap mode), - text editors (mmap your file, edit, save), - bulk data loaders (read 100 GB log without touching `read()`).

Anonymous mmap — for the heap or for shm

```
char *p = mmap(NULL, 1<<20, PROT_READ | PROT_WRITE,
               MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
```

This is **how `malloc`'s big allocations work**. glibc uses `mmap` for >128 KB requests. Smaller go through `brk`/`sbrk` for the heap.

Two processes can share anonymous memory if you `mmap` with `MAP_SHARED | MAP_ANONYMOUS` **before** `fork`.

POSIX shared memory — for unrelated processes

```
#include <sys/mman.h>
#include <fcntl.h>

int fd = shm_open("/myshm", O_CREAT | O_RDWR, 0600);
ftruncate(fd, 4096);
void *p = mmap(NULL, 4096, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
// process B opens the same name and mmaps it.
```

Memory is now shared between any process that opens `/myshm`. Backed by `tmpfs` at `/dev/shm`. Lives until `shm_unlink("/myshm")`.

Synchronization in shared memory

Plain reads/writes by multiple processes race. You need locks: - POSIX **process-shared mutexes**: `pthread_mutexattr_setpshared(&attr, PTHREAD_PROCESS_SHARED)`. Place mutex inside the shared region. - Atomic operations (`stdatomic.h`) — `atomic_int` placed in shared memory works fine. - Semaphores (next chapter). - Futexes (lower-level building block).

Example: shared atomic counter between two processes.

```
#include <stdatomic.h>
atomic_int *counter = mmap(...);
atomic_fetch_add(counter, 1);
```

Performance tricks

MAP_HUGETLB / Transparent Huge Pages

2 MB or 1 GB pages instead of 4 KB. Big TLB win for huge maps. Some workloads (databases, GPU pinning) need it.

MAP_POPULATE

Pre-fault all pages immediately, avoiding the page-fault storm at first access. Latency-sensitive workloads.

madvise

Tell the kernel about your access pattern: - `MADV_SEQUENTIAL` — readahead more aggressively. - `MADV_RANDOM` — read just the page. - `MADV_DONTNEED` — drop the pages, will refault if accessed. - `MADV_HUGEPAGE` — try to use huge pages. - `MADV_DONTFORK` — don't copy this region in `fork`.

```
madvise(p, len, MADV_SEQUENTIAL);
```

msync — controlling the flush

For `MAP_SHARED`, writes go to the page cache; they hit disk on the next writeback. `msync(p, len, MS_SYNC)` forces immediate flush. `MS_ASYNC` schedules.

Cost of mmap

mmap itself is cheap; the cost is the **page faults** when you first touch each page. For a 1 GB map, that's 256K faults of 4 KB each. Use `MAP_POPULATE` or huge pages for bulk loads.

When NOT to use mmap

- Many small reads from many small files — `read()` is simpler and has less syscall overhead.
- Files that grow during use — mmap's size is fixed at the time of the call. (For grow scenarios, remap.)
- Reading network sockets — you can't mmap a socket; use `splice` for zero-copy.

Real-world examples

- **GPU**: BO mappings for CPU visibility (write-combining mappings via `mmap` of `/dev/dri/cardN`). The mmap on a DRM file routes through `drm_gem_mmap` and ends up in your driver's `vm_ops->fault` callback. We'll see this in Part VII.
- **Databases**: SQLite with `mmap_size` pragma, LMDB, MongoDB historically.
- **Browsers**: `MAP_PRIVATE` of fonts and large data files.

Common bugs

1. **Forgetting `ftruncate`**: a freshly created shm has size 0; mmap will SEGV on access.
2. **Wrong PROT**: `PROT_READ` mmap then writing → SIGSEGV.
3. **Mmap past EOF**: with `MAP_SHARED`, accessing past file end → SIGBUS.
4. **`MAP_PRIVATE` confusion**: writes appear local, lost on `munmap`.
5. **Forgot to munmap**: mmap leak; `pmap PID` shows the leak grow.
6. **Concurrent access without sync**: races. Always synchronize.

Try it

1. mmap a file; toggle a byte; flush; verify with `xxd file`.
2. Two processes share a counter via `shm_open` + `mmap` + `atomic_int`. Increment 1M times each; verify final equals 2M.
3. Implement a "tail -f" via mmap: print bytes as they appear in a growing file (you'll need to re-mmap on size changes).
4. Use `madvise(MADV_SEQUENTIAL)` then read a 1 GB file. Compare to without; compare to `read()` only. Time them.
5. Bonus: write a parent that creates an `MAP_SHARED | MAP_ANONYMOUS` region, forks 4 children, each fills 1/4 of the buffer. Parent waits and prints checksum.

Read deeper

- TLPI chapters 49 (memory mappings), 53 (POSIX shared memory).
- `man 2 mmap`, `man 2 madvise`, `man 7 shm_overview`.

- “Linux Page Cache” — search lwn.net.

Next → Message queues, semaphores, futexes.

USP 6 — Message queues, semaphores, futexes

> Three IPC primitives: structured messages, counted blocking, and the lock primitive that makes everything fast.

POSIX message queues

```
#include <mqueue.h>
struct mq_attr a = { .mq_maxmsg = 10, .mq_msgsize = 256 };
mqd_t q = mq_open("/myq", O_CREAT | O_RDWR, 0600, &a);

char buf[256];
mq_send(q, buf, len, /*priority=*/5);
unsigned prio;
mq_receive(q, buf, sizeof(buf), &prio);

mq_close(q);
mq_unlink("/myq");
```

- Bounded queue of fixed-size messages.
- Messages have priority (highest delivered first).
- Lives in the kernel under `/dev/mqueue`.
- Senders block when full; receivers block when empty (configurable nonblocking).

When to use: structured event passing between unrelated processes, with priority delivery.

System V message queues

`msgget`, `msgsnd`, `msgrcv`. Older API (1980s), key-based naming. Avoid; use POSIX or sockets.

POSIX semaphores

A semaphore is a non-negative counter: - `sem_wait` decrements (blocks if 0). - `sem_post` increments (wakes a waiter if any).

Named (between processes)

```
sem_t *s = sem_open("/sem", O_CREAT, 0600, /*initial=*/1);
sem_wait(s);
// critical section
sem_post(s);
sem_close(s);
sem_unlink("/sem");
```

Anonymous (in shared memory)

```
sem_t *s = mmap(...);
sem_init(s, /*pshared=*/1, /*initial=*/1);
```

Counting semaphore vs binary

A semaphore initialized to 1 acts as a mutex (binary). Initialized to N, allows up to N concurrent holders (e.g., a "pool of 10 workers").

Used in: producer/consumer (1 sem for empty slots, 1 for filled), bounded-resource access, simple multi-process synchronization.

System V semaphores

`semget`, `semop`. Older. Powerful but byzantine. Avoid in new code.

eventfd — a counter you can poll

```
int fd = eventfd(0, EFD_NONBLOCK);
uint64_t v = 1; write(fd, &v, 8);   // increment
read(fd, &v, 8);                    // gets and resets
```

Useful for waking an event-loop thread. Drop into `epoll`. Used by libraries like libevent, libuv internally.

futex — the kernel primitive under userspace mutexes

`futex` ("fast userspace mutex") is the syscall that makes lock-free locks possible: - Userspace tries an atomic `cmpxchg` to acquire a lock. - If contended, calls `futex(FUTEX_WAIT, addr, val)` to sleep. - Releaser calls `futex(FUTEX_WAKE, addr, n)` to wake waiters.

You almost never call futex directly. `pthread_mutex_t` uses futex under the hood. Some advanced users (lock-free libraries, JITs) do call it directly.

`man 2 futex` — read it once for awareness.

When to use which IPC

Scenario	Best
Structured messages with priority	POSIX message queue
Mutex/semaphore between processes	POSIX semaphore in shared memory
Wake a thread from epoll loop	`eventfd`
One process sends events to another over a connection	Unix socket
Bidirectional, possibly across machines	TCP socket
Bulk data	shared memory + lock + signaling
Simple "child done"	`wait()` / SIGCHLD

Locks across processes

Two ways to protect shared memory between processes:

POSIX process-shared mutex

```
pthread_mutex_t *m = mmap(...);
pthread_mutexattr_t attr;
pthread_mutexattr_init(&attr);
pthread_mutexattr_setpshared(&attr, PTHREAD_PROCESS_SHARED);
pthread_mutex_init(m, &attr);
```

Now `pthread_mutex_lock(m)` works between processes. Cheap (futex-backed).

Robust mutexes (`pthread_mutexattr_setrobust`) survive owner-process crashes — the next acquirer gets `EOWNERDEAD` and can recover.

POSIX semaphore in shared memory

Simpler: `sem_init` with `pshared=1`. Use as binary mutex.

Common bugs

1. **Forgetting to `unlink`** named queues/semaphores → resource leak (`ls /dev/mqueue`).
2. **Mutex not in shared memory** when used across processes → each process locks its own copy. Subtle.
3. **Initializer running twice** in shared memory → corruption. Initialize from one process only.
4. **Holding lock across long ops or syscalls that may block on disk/net** → contention.
5. **Stale lock state on crash** — use robust mutexes.

Try it

1. POSIX message queue: parent sends 100 messages with random priorities; child reads and prints in order. Verify priority ordering.
2. Two processes share a counter via shared memory; protect with a process-shared `pthread_mutex_t`. Each does 1M increments. Verify final = 2M.
3. Use `eventfd` to integrate “I have work” notifications into an `epoll` loop.
4. Producer/consumer with semaphores: bounded ring buffer of size N; producer blocks when full, consumer when empty.
5. Crash a robust mutex holder; show the next acquirer gets EOWNERDEAD.

Read deeper

- TLPI chapters 51–53 (POSIX IPC).
- `man 7 mq_overview`, `man 7 sem_overview`, `man 2 futex`.
- “Mutexes and Memory Barriers” — useful articles on futex internals.

Next → Sockets — TCP & UDP.

USP 7 — Sockets — TCP & UDP

> Sockets are how Unix programs talk over the network. Same API works for TCP, UDP, Unix domain sockets.

The 5-call life of a TCP server

```
        int s = socket(AF_INET, SOCK_STREAM, 0);
        struct sockaddr_in a = { .sin_family = AF_INET, .sin_port = htons(8080), .sin_addr.s_addr =
INADDR_ANY };
        bind(s, (struct sockaddr*)&a, sizeof(a));
        listen(s, 64);
        int c = accept(s, NULL, NULL);
        read(c, buf, ...); write(c, ...);
        close(c);
```

Five system calls: 1. **socket** — create endpoint. 2. **bind** — assign address. 3. **listen** — mark as a server (backlog of pending connections). 4. **accept** — block until a client connects; returns a new fd for that connection. 5. **read/write/close** — talk and disconnect.

The 3-call life of a TCP client

```
        int s = socket(AF_INET, SOCK_STREAM, 0);
        struct sockaddr_in a = { .sin_family = AF_INET, .sin_port = htons(8080) };
        inet_pton(AF_INET, "1.2.3.4", &a.sin_addr);
        connect(s, (struct sockaddr*)&a, sizeof(a));
        write(s, ...); read(s, ...); close(s);
```

`connect` performs TCP's three-way handshake (SYN, SYN-ACK, ACK) and returns when established (or fails).

Address structures

```
        struct sockaddr_in {                 // IPv4
            sa_family_t  sin_family;         // AF_INET
            in_port_t    sin_port;            // big-endian
            struct in_addr sin_addr;
        };
        struct sockaddr_in6 {                // IPv6
            sa_family_t     sin6_family;
            in_port_t       sin6_port;
            uint32_t        sin6_flowinfo;
            struct in6_addr sin6_addr;
            uint32_t        sin6_scope_id;
        };
        struct sockaddr_un {                 // Unix domain
            sa_family_t sun_family;          // AF_UNIX
            char        sun_path[108];
        };
```

All cast to `struct sockaddr *` for the syscall API. **Network byte order** (big-endian) for ports/IPs — use `htons`, `htonl`, `ntohs`, `ntohl`.

Address families & types

- `AF_INET` / `AF_INET6` — IPv4 / IPv6.
- `AF_UNIX` — local IPC over a filesystem path.
- `AF_PACKET` — raw L2 (Ethernet frames). For sniffers, custom protocols.

Socket types: - `SOCK_STREAM` — TCP, reliable, ordered, byte stream. - `SOCK_DGRAM` — UDP, datagrams, unreliable, may be reordered. - `SOCK_RAW` — raw IP. Need root. - `SOCK_SEQPACKET` — message-oriented but reliable. Used by some IPC protocols.

TCP vs UDP

TCP: - Reliable, ordered, congestion-controlled. - Stateful — connection setup/teardown. - Bytes streaming with no message boundaries.

UDP: - Unreliable, unordered. - Stateless — just send packets. - Use cases: DNS, VoIP, video streaming, time sync, game state, anything with own retransmit logic.

A minimal echo server

```
int s = socket(AF_INET, SOCK_STREAM, 0);
int yes = 1;
setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));   // reuse port immediately
bind(...); listen(s, 64);
for (;;) {
    int c = accept(s, NULL, NULL);
    if (c < 0) { perror("accept"); continue; }
    char buf[4096];
    ssize_t n;
    while ((n = read(c, buf, sizeof(buf))) > 0) {
        if (write(c, buf, n) < 0) break;
    }
    close(c);
}
```

This handles **one client at a time**. To handle many: fork per connection, thread per connection, or `epoll`. We'll do epoll next chapter.

A minimal UDP echo server

```
int s = socket(AF_INET, SOCK_DGRAM, 0);
bind(...);
for (;;) {
    char buf[1500]; struct sockaddr_in peer; socklen_t plen = sizeof(peer);
    ssize_t n = recvfrom(s, buf, sizeof(buf), 0, (struct sockaddr*)&peer, &plen);
    sendto(s, buf, n, 0, (struct sockaddr*)&peer, plen);
}
```

UDP is simpler — no listen/accept, just receive and send.

SO_REUSEADDR / SO_REUSEPORT

- `SO_REUSEADDR` — let you bind even if the port is in TIME_WAIT (recently closed connection still in kernel state).
- `SO_REUSEPORT` — multiple processes can bind to the same port (the kernel load-balances incoming connections). Modern servers use this for multi-process scaling.

Other useful socket options

- `TCP_NODELAY` — disable Nagle's algorithm (small writes go out immediately). For latency-sensitive apps.
- `SO_KEEPALIVE` — kernel sends probes if connection is idle, detects dead peers.
- `SO_RCVBUF` / `SO_SNDBUF` — kernel buffer sizes per socket.
- `SO_LINGER` — control close behavior.

DNS resolution — getaddrinfo

```
struct addrinfo hints = { .ai_family = AF_UNSPEC, .ai_socktype = SOCK_STREAM };
struct addrinfo *res;
getaddrinfo("example.com", "80", &hints, &res);
for (struct addrinfo *p = res; p; p = p->ai_next) {
    int s = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (s < 0) continue;
    if (connect(s, p->ai_addr, p->ai_addrlen) == 0) { /* success */ break; }
    close(s);
}
freeaddrinfo(res);
```

`getaddrinfo` does DNS, returns a linked list of candidates. **Always use this**, never raw `gethostbyname` (deprecated, not thread-safe).

Unix domain sockets

`AF_UNIX` with a filesystem path. Faster than TCP loopback (no checksum, no protocol stack overhead). Used in: - D-Bus, - systemd journal, - container runtimes, - Wayland / X11.

Bonus: pass file descriptors between processes via `SCM_RIGHTS` ancillary messages. Foundational for socket activation, privilege separation.

Read & write semantics

- `read` returns `0` on EOF, `-1` on error, otherwise bytes read.
- `write` may return less than requested → loop.
- `recv` / `send` are like read/write with extra flags (`MSG_PEEK` , `MSG_DONTWAIT` , `MSG_NOSIGNAL`).

Always loop on partial writes:

```
ssize_t write_all(int fd, const void *buf, size_t n) {
    const char *p = buf; size_t left = n;
    while (left) {
        ssize_t w = write(fd, p, left);
```

```
            if (w < 0) { if (errno == EINTR) continue; return -1; }
            p += w; left -= w;
        }
        return n;
    }
```

TCP states (briefly)

```
LISTEN → SYN_RCVD → ESTABLISHED → CLOSE_WAIT or FIN_WAIT_1 → ... → TIME_WAIT → CLOSED
```

`netstat -an` or `ss -ant` shows TCP states for all sockets. **TIME_WAIT** is normal for the active closer; it's why `SO_REUSEADDR` is often needed.

Common bugs

1. **Endianness** — forgetting `htons` / `htonl`.
2. **Address structure size** — `sizeof(struct sockaddr)` (16) vs the right `sockaddr_in` size (16). For `sockaddr_un`, varies.
3. **Buffer too small** for a packet → truncation on UDP recvfrom.
4. **Not handling EAGAIN** with non-blocking sockets.
5. **Concurrent close + read** races — careful with shared sockets.
6. **SIGPIPE on TCP write** to closed connection — set `MSG_NOSIGNAL` or ignore.

Try it

1. Echo server (TCP). Verify with `nc 127.0.0.1 8080`.
2. Echo server (UDP). Verify with `nc -u 127.0.0.1 8080`.
3. Resolve a hostname with `getaddrinfo`; print all returned addresses.
4. Use a Unix domain socket. Pass a file descriptor from one process to another via SCM_RIGHTS.
5. Use `setsockopt(TCP_NODELAY)`. Compare round-trip latency to default.
6. Bonus: write an HTTP/0.9 fetcher (`GET /\r\n\r\n`, then read).

Read deeper

- TLPI chapters 56–62 (sockets).
- *Unix Network Programming* by Stevens — the deepest treatment.
- `man 7 socket`, `man 7 ip`, `man 7 tcp`, `man 7 udp`, `man 7 unix`.

Next → I/O multiplexing — select, poll, epoll, io_uring.

USP 8 — I/O multiplexing: select, poll, epoll, io_uring

> Handle 10,000 connections in one thread. The reason async servers exist.

The problem

A simple TCP server `accept`s, then `read`s. While reading from one client, the next client waits. **One client at a time.**

Solutions: 1. **Process per connection** — `fork` for each. Apache prefork. Heavy. 2. **Thread per connection** — `pthread_create`. Lighter; still limited by # threads. 3. **One thread, many fds, async** — multiplexing. The way modern servers (nginx, Node, Redis) work.

We focus on (3).

select — the original (1980s)

```
fd_set rfds; FD_ZERO(&rfds); FD_SET(fd, &rfds);
struct timeval tv = { .tv_sec = 5 };
int n = select(fd + 1, &rfds, NULL, NULL, &tv);
if (n > 0 && FD_ISSET(fd, &rfds)) /* readable */;
```

- Bitmap-based. Limited to FD_SETSIZE (typically 1024).
- O(n) on every call (kernel scans all bits).
- Awful for many fds.

Use only for compatibility on ancient systems.

poll — improvement

```
struct pollfd pfd[N];
pfd[0] = (struct pollfd){ .fd = sockfd, .events = POLLIN };
int n = poll(pfd, 1, 5000);   // 5s timeout in ms
if (pfd[0].revents & POLLIN) /* readable */;
```

- Array-based. No FD_SETSIZE limit.
- Still O(n) — kernel scans the whole array each call.
- Decent up to a few hundred fds.

epoll — Linux's scalable answer

```
int ep = epoll_create1(0);
struct epoll_event ev = { .events = EPOLLIN, .data.fd = sockfd };
epoll_ctl(ep, EPOLL_CTL_ADD, sockfd, &ev);
```

```
        struct epoll_event events[64];
        int n = epoll_wait(ep, events, 64, /*timeout_ms=*/-1);
        for (int i = 0; i < n; i++) {
            int fd = events[i].data.fd;
            if (events[i].events & EPOLLIN) /* readable */;
        }
```

- O(1) — kernel keeps a ready-list internally.
- No FD limit (well, ulimit -n).
- Edge-triggered or level-triggered modes.

Level-triggered (LT, default)

"Tell me as long as fd is readable." Easy to use; close to poll's semantics.

Edge-triggered (ET, `EPOLLET`)

"Tell me only when fd **becomes** readable." Faster; harder. You **must** drain — read until EAGAIN. Forgetting causes hangs.

```
        ev.events = EPOLLIN | EPOLLET;
```

Modern high-perf code uses ET + non-blocking sockets.

Non-blocking sockets

```
        int flags = fcntl(fd, F_GETFL); fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

Now `read`/`write` return -1 with `errno=EAGAIN` instead of blocking. Required for ET; also good with LT for safety.

A real epoll server skeleton

```
        int s = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK | SOCK_CLOEXEC, 0);
        setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &(int){1}, sizeof(int));
        bind(s, ...); listen(s, 128);

        int ep = epoll_create1(EPOLL_CLOEXEC);
        epoll_ctl(ep, EPOLL_CTL_ADD, s, &(struct epoll_event){.events=EPOLLIN, .data.fd=s});

        for (;;) {
            struct epoll_event evs[64];
            int n = epoll_wait(ep, evs, 64, -1);
            for (int i = 0; i < n; i++) {
                int fd = evs[i].data.fd;
                if (fd == s) {
                    for (;;) {
                        int c = accept4(s, NULL, NULL, SOCK_NONBLOCK | SOCK_CLOEXEC);
                        if (c < 0) { if (errno == EAGAIN) break; perror("accept"); break; }
                        struct epoll_event cev = {.events = EPOLLIN | EPOLLET, .data.fd = c};
                        epoll_ctl(ep, EPOLL_CTL_ADD, c, &cev);
                    }
                } else {
                    handle_data(fd, ep);  // read non-blocking; on EAGAIN, return; on EOF, close +
EPOLL_CTL_DEL
                }
            }
        }
```

This handles tens of thousands of connections in one thread. Add multiple worker threads, each with their own epoll fd, and use `SO_REUSEPORT` for load-balancing — modern nginx-style architecture.

Edge-triggered drain rule

For each readable event:

```
for (;;) {
    ssize_t n = read(fd, buf, sizeof(buf));
    if (n < 0) { if (errno == EAGAIN) break; close_connection(); break; }
    if (n == 0) { close_connection(); break; }   // EOF
    process(buf, n);
}
```

If you don't drain, you may not get another event until **more** data arrives — your existing data sits there.

EPOLLOUT — when fd is writable

For `write`-heavy code, register `EPOLLOUT` only when you have data to send and the kernel buffer is full. When the kernel can accept more, you'll get an event.

Common trick: lazily install `EPOLLOUT` only when needed; remove it when buffer is empty (`EPOLL_CTL_MOD`).

io_uring — the modern way

epoll tells you "fd is ready, now you do the syscall." io_uring lets you submit operations and receive completions — true async I/O for files, sockets, etc.

```
#include <liburing.h>
struct io_uring ring;
io_uring_queue_init(8, &ring, 0);
struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
io_uring_prep_read(sqe, fd, buf, len, off);
io_uring_submit(&ring);

struct io_uring_cqe *cqe;
io_uring_wait_cqe(&ring, &cqe);
ssize_t n = cqe->res;
io_uring_cqe_seen(&ring, cqe);
```

- Two ring buffers (submission, completion) shared with the kernel.
- Reduces syscall overhead drastically.
- Supports nearly every I/O primitive.

io_uring is the future. For now, epoll is more universal; many libraries support both.

kqueue (BSD/macOS)

Equivalent to epoll on BSD/macOS. Slightly different API; same semantics.

eventfd / signalfd / timerfd in epoll

- **eventfd**: send "wake up" between threads as a fd readable.
- **signalfd**: turn signals into fd events.
- **timerfd**: turn timers into fd events.

Drop them all into one epoll loop — clean, single-threaded async server.

Common bugs

1. **Forgetting non-blocking** with edge-triggered → hang.
2. **Not draining** with ET → starvation.
3. **Adding the same fd twice** to epoll without `EPOLL_CTL_MOD` → EEXIST.
4. **Close-before-EPOLL_CTL_DEL** while another thread is in `epoll_wait` — race window. Use `EPOLLEXCLUSIVE` or careful single-threading.
5. **Leak**: epoll_ctl(ADD) on a duplicated fd; the ADD is on the underlying file, not on the descriptor. Use `EPOLL_CTL_DEL` properly.

Try it

1. Implement an `epoll`-based echo server. Use ET + non-blocking. Test with 1000 simultaneous clients (use `ab` or write a small load tool).
2. Add `SO_REUSEPORT` and 4 worker processes. Each binds and `epoll_wait`s. Verify load is spread.
3. Use `timerfd` to fire every 1 second; integrate into your epoll loop alongside sockets.
4. Use `signalfd` for SIGINT; cleanly shut down all connections on Ctrl-C.
5. Benchmark: poll vs epoll on 10000 fds. Most are idle. Measure CPU.
6. Bonus: try `liburing` for the same echo server. Compare throughput to epoll on fast network.

Read deeper

- TLPI chapter 63 (alternate I/O models).
- "The C10K problem" (Dan Kegel, 2002) — original analysis of the scalability problem.
- `man 7 epoll`, `man 2 epoll_ctl`, `man 2 epoll_wait`.
- io_uring docs: kernel.dk and Jens Axboe's blog.

Next → pthreads deep.

USP 9 — pthreads deep

> Threads share everything inside one process. Powerful, dangerous, essential.

Threads vs processes

	Process	Thread
Address space	private	shared with peer threads
File descriptors	private (after fork)	shared
`getpid`	own	same as process
Crashes	isolated (mostly)	crash one = crash all
Communication	IPC (slow)	shared memory (fast)
Creation cost	high	medium

A process is always **one process** with one or more threads. Linux schedules threads (in the kernel they're called "tasks").

Creating a thread

```
#include <pthread.h>

void *worker(void *arg) {
    int id = (int)(long)arg;
    printf("hi from thread %d\n", id);
    return (void*)(long)(id * 2);
}

int main(void) {
    pthread_t t[4];
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, worker, (void*)(long)i);
    for (int i = 0; i < 4; i++) {
        void *ret;
        pthread_join(t[i], &ret);
        printf("thread %d returned %ld\n", i, (long)ret);
    }
}
```

`pthread_join` blocks until the thread finishes. Alternative: `pthread_detach` to "fire and forget" (no return value collected; thread cleans itself).

Mutexes

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&m);
// critical section
pthread_mutex_unlock(&m);
```

If multiple threads contend, all but one block. Backed by futex on Linux — uncontended lock is essentially zero-cost (an atomic CAS in userspace).

Variants

- `pthread_mutex_trylock` — non-blocking; returns EBUSY if locked.
- `pthread_mutex_timedlock` — wait up to a timeout.
- `pthread_rwlock_t` — multiple readers OR one writer.
- Recursive mutex: same thread can lock multiple times. Avoid; usually a sign of bad design.
- Robust mutex: survives owner crash (returns EOWNERDEAD).

Condition variables

For "wait until some condition is true."

```
pthread_cond_t  cv = PTHREAD_COND_INITIALIZER;
pthread_mutex_t m  = PTHREAD_MUTEX_INITIALIZER;
int ready = 0;

// waiter
pthread_mutex_lock(&m);
while (!ready) pthread_cond_wait(&cv, &m);
// safe to access protected data
pthread_mutex_unlock(&m);

// signaler
pthread_mutex_lock(&m);
ready = 1;
pthread_cond_signal(&cv);   // or pthread_cond_broadcast
pthread_mutex_unlock(&m);
```

Always use `while (!cond)`, never `if`. Spurious wakeups are real (rare, but allowed).

`pthread_cond_wait` atomically: drops the mutex + sleeps + re-acquires the mutex on wake. That atomicity is the whole point.

Semaphores

`sem_t` (POSIX). Counting semaphore between threads. Useful for bounded resources.

Read-write locks

```
pthread_rwlock_t rw = PTHREAD_RWLOCK_INITIALIZER;
pthread_rwlock_rdlock(&rw);   // many readers OK
// or
pthread_rwlock_wrlock(&rw);   // exclusive
pthread_rwlock_unlock(&rw);
```

Win when reads vastly outnumber writes. Penalize writers slightly.

Thread-local storage

```
__thread int my_local;        // GCC extension; standard on Linux
// C11:
_Thread_local int my_local;
```

Each thread has its own copy. Used for: per-thread arenas in custom allocators, errno (yes, errno is `__thread`), thread-local random generators.

pthread_once — initialize once

```
pthread_once_t once = PTHREAD_ONCE_INIT;
void init(void) { /* ... */ }

pthread_once(&once, init);
```

Thread-safe initialization. Runs `init` exactly once across all threads.

Cancellation

```
pthread_cancel(thread);
```

Tries to cancel a thread. Cancel points (e.g., `read`, `write`, `pthread_cond_wait`) check and exit. Asynchronous mode is dangerous; avoid.

Better practice: shared “stop” flag the thread checks at safe points.

Thread attributes

```
pthread_attr_t a;
pthread_attr_init(&a);
pthread_attr_setstacksize(&a, 1 << 20);     // 1 MB stack
pthread_attr_setdetachstate(&a, PTHREAD_CREATE_DETACHED);
pthread_create(&t, &a, fn, arg);
pthread_attr_destroy(&a);
```

Important: by default, glibc gives 8 MB stacks; many threads = lots of address space (not RAM until used). Reduce if creating thousands of threads.

Producer / consumer (canonical example)

```
// shared
queue_t q; pthread_mutex_t m; pthread_cond_t not_empty, not_full;

// producer
pthread_mutex_lock(&m);
while (full(&q)) pthread_cond_wait(&not_full, &m);
push(&q, item);
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&m);

// consumer
pthread_mutex_lock(&m);
```

```
        while (empty(&q)) pthread_cond_wait(&not_empty, &m);
        item = pop(&q);
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&m);
```

Memorize this pattern. It's the foundation of every worker-pool server.

Atomic operations (C11)

```
        #include <stdatomic.h>
        atomic_int counter = 0;
        atomic_fetch_add(&counter, 1);     // atomic ++
        atomic_load(&counter);
        atomic_store(&counter, 0);
        atomic_compare_exchange_strong(&counter, &expected, desired);
```

Memory orderings: `relaxed`, `acquire`, `release`, `acq_rel`, `seq_cst`. Default `seq_cst` (sequentially consistent).

Atomics avoid mutexes for small ops (counters, flags). For real lock-free DS, see Part IV.

Deadlocks

Two threads, two locks, opposite order:

```
Thread A: lock(X); lock(Y);
Thread B: lock(Y); lock(X);
```

Both stuck. Avoid: - **Always acquire locks in a consistent global order.** - Use `pthread_mutex_trylock` and back off on conflict. - Run with **lockdep**-style tools (helgrind, TSan) to detect.

Sanitizers — your best friends

```
        gcc -fsanitize=thread -g -O1 prog.c
```

ThreadSanitizer (TSan) catches data races at runtime. Slower (5-10x), worth it during dev. Use it on every multi-threaded program you write.

Helgrind / DRD (valgrind tools) — alternative race detectors.

Per-thread vs shared

- **Shared by default**: heap, globals, file descriptors, signal handlers (process-wide).
- **Per-thread**: stack, errno, thread-local storage.

Common bugs

1. **Forgot to lock** — race condition, hard to reproduce.
2. **Locked but wrong lock** — protecting different objects with different locks; access from wrong context.
3. **Held lock too long** — through I/O or syscalls.

4. **Lost wakeup** — using `if` instead of `while` around `cond_wait`.
5. **Deadlock** — inconsistent lock order.
6. **Leaked thread** — never joined, never detached → resource leak.
7. **Detached thread referencing freed data** — synchronization needed for handoff.

Try it

1. Producer/consumer with N producers, M consumers, bounded queue. Use mutex + condvar. Verify count.
2. Implement a thread pool: N worker threads pulling tasks from a queue. Submit 10000 small tasks; measure throughput.
3. Replace mutex with read-write lock for a "mostly read" map. Measure improvement.
4. Use `_Thread_local` to give each thread its own random generator. Compare to a single shared one + mutex.
5. Build a buggy program with a deliberate race; confirm TSan detects it.
6. Bonus: re-implement a SPSC queue using `atomic_int` indices and `memory_order_acquire/release`. Stress test for 60s.

Read deeper

- TLPI chapters 29–33 (threads, sync, thread-local).
- *C++ Concurrency in Action* (Anthony Williams) — even though it's C++, the model and pitfalls translate.
- *Programming with POSIX Threads* by Butenhof — the classic.

Next → File I/O deep.

USP 10 — File I/O deep

> read, write, lseek, mmap, sendfile, O_DIRECT, fsync. The full menu of how bytes get to and from disk.

The two layers

- **Standard I/O** (`stdio.h`): `fopen`, `fread`, `printf`, `fgets`. Buffered in libc.
- **Low-level I/O** (`<unistd.h>`): `open`, `read`, `write`, `lseek`, `close`. Direct syscalls.

Stdio is convenient. Low-level I/O is the truth.

`fopen` returns a `FILE *`. `open` returns an `int` (the fd). `fileno(fp)` gets the fd from a FILE.

open flags

```
int fd = open(path, O_RDONLY);
int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
int fd = open(path, O_RDWR  | O_CREAT | O_EXCL,  0600);   // fails if exists
int fd = open(path, O_RDWR  | O_DIRECT);                   // bypass page cache
int fd = open(path, O_RDWR  | O_NONBLOCK);                 // for fifos, ttys
int fd = open(path, O_RDWR  | O_APPEND);                   // atomic append
int fd = open(path, O_RDWR  | O_CLOEXEC);                  // fd not inherited by exec
```

Always pass mode (the third arg) when O_CREAT is used, or you get junk perms.

`O_CLOEXEC` is **best practice on every open** unless you specifically need the fd inherited.

read, write — partial completes

```
ssize_t n = read(fd, buf, count);
```

- `n > 0` — bytes read.
- `n == 0` — EOF (regular file) or “no data this time” (some special fds).
- `n < 0` — error; check `errno`.

`read` may return **less than asked**. Loop:

```
size_t got = 0;
while (got < count) {
    ssize_t r = read(fd, buf + got, count - got);
    if (r < 0) { if (errno == EINTR) continue; return -1; }
    if (r == 0) break;
    got += r;
}
```

Same for `write`. Always loop.

lseek — random access

```
        off_t pos = lseek(fd, offset, SEEK_SET);   // SEEK_CUR, SEEK_END
```

For sparse files: `lseek` past EOF and write — kernel allocates only the touched blocks. Sparse files conserve space (e.g., 1 GB log file with most of it zero takes 4 KB on disk).

pread / pwrite — atomic positioned I/O

```
        ssize_t n = pread(fd, buf, count, offset);
```

No `lseek` race. Multiple threads can simultaneously `pread` with different offsets. Best for parallel I/O.

readv / writev — vectored I/O

```
        struct iovec iov[3];
        iov[0] = (struct iovec){.iov_base = "GET ", .iov_len = 4};
        iov[1] = (struct iovec){.iov_base = path,   .iov_len = strlen(path)};
        iov[2] = (struct iovec){.iov_base = "\r\n", .iov_len = 2};
        writev(fd, iov, 3);
```

Single syscall, multiple buffers, all-or-nothing for consistency. Used in HTTP servers, network protocols.

sendfile — zero-copy

```
        ssize_t n = sendfile(outfd, infd, &off, count);
```

Kernel-side copy from one fd to another without going through userspace. Used by web servers serving static files: `sendfile(socket, file, ...)`.

For pipe ↔ fd, use `splice`. For two pipes, `tee`.

O_DIRECT — bypass the page cache

```
        int fd = open(path, O_RDWR | O_DIRECT);
```

Reads/writes go directly between user buffers and the device, skipping kernel page cache.

Constraints (architecture-dependent): - Buffer must be page-aligned. - Buffer size must be a multiple of block size (often 512 or 4096). - Offset must be aligned.

Used by databases that manage their own cache (Oracle, PostgreSQL with direct_io, MySQL InnoDB).

fsync, fdatasync, sync

`write` does not mean “on disk.” It means “in kernel page cache, will be flushed eventually.”

```
fsync(fd);        // flush data + metadata
fdatasync(fd);    // flush data only (skip metadata if not strictly needed)
sync();           // flush everything system-wide
```

Crashes between `write` and `fsync` lose data. Databases call `fsync` at commit boundaries.

File locking

Advisory (most common)

```
struct flock fl = { .l_type = F_WRLCK, .l_whence = SEEK_SET, .l_start = 0, .l_len = 0 };
fcntl(fd, F_SETLKW, &fl);     // block until acquired (whole file)
// ...
fl.l_type = F_UNLCK;
fcntl(fd, F_SETLK, &fl);
```

Other processes that **also** call `fcntl` will respect it; processes that don't, won't (advisory).

`flock(fd, LOCK_EX | LOCK_NB)` — simpler, BSD-style.

Open file description locks (`F_OFD_SETLK`) — modern, properly per-fd.

Used in: PID files, lock files for daemons, multi-process databases.

Inotify — file change notifications

```
int ifd = inotify_init1(IN_CLOEXEC);
int wd = inotify_add_watch(ifd, "/etc/hostname", IN_MODIFY);
char buf[4096];
read(ifd, buf, sizeof(buf));   // returns inotify_event records
```

Watch files/dirs for changes. Used by `tail -f`, file syncers, build tools (`watchman`).

stat — file metadata

```
struct stat st;
stat(path, &st);
printf("size=%ld inode=%lu uid=%d mode=%o\n", st.st_size, st.st_ino, st.st_uid, st.st_mode);
```

Tells you size, perms, owner, mtime, atime, inode, device. Useful constantly.

`fstat(fd, &st)` for an open fd.

/tmp considerations

`/tmp` is often `tmpfs` (RAM-backed) → fast but bounded by RAM. For large temp files, use `/var/tmp` (disk-backed). For application caches, `~/.cache` .

Async I/O alternatives

- **POSIX AIO** (`aio_read` / `aio_write`): existed for years; rarely used; mostly disappointing on Linux.
- **io_uring**: modern, fast, supports everything.

For most apps: synchronous + threads, or epoll + non-blocking sockets.

Page cache and writeback

When you `write`, the kernel marks pages **dirty** and they sit in the page cache. The kernel writeback thread writes them to disk later (based on dirty ratio, time). On crash → unwritten data is lost. Hence `fsync`.

`echo 3 > /proc/sys/vm/drop_caches` drops clean cached pages. Useful for benchmarks (don't run in prod).

Common bugs

1. **Forgetting to loop on partial read/write.**
2. **Using `printf` after fork without flush** — output may appear twice (parent had buffered).
3. **Forgetting `O_CLOEXEC`** — fd leaks into exec'd subprocesses.
4. **Missing `fsync`** in databases — data loss on crash.
5. **Using `gets` (or any unbounded function)** — buffer overflow.
6. **Forgetting close** — fd leak; eventually `EMFILE`.

Try it

1. Write a function `read_file(path)` returning the whole file as a malloc'd buffer. Loop on `read` properly.
2. Implement a copy command using `read` / `write`. Then implement using `mmap`. Then using `sendfile`. Compare speeds on a 1 GB file.
3. Use `lseek` past EOF to make a 1 GB sparse file. `du -sh` shows ~0; `ls -l` shows 1 GB.
4. Implement `tail -f` using inotify.
5. Open with `O_DIRECT` and write a page-aligned 4 KB block. Read it back. Compare timing to `O_RDWR`.
6. Bonus: implement a simple write-ahead log: append, fsync, then return success. Replay on startup.

Read deeper

- TLPI chapters 4–5, 13, 14, 15 — file I/O, buffered I/O, file metadata, file locking.
- `man 2 open`, `man 2 read`, `man 2 sendfile`, `man 2 fsync`, `man 7 inotify`.
- "Files are hard" — Dan Luu's blog post on the surprises in file durability.

Next → /proc, /sys, /dev — the magic filesystems.

USP 11 — /proc, /sys, /dev — the magic filesystems

> Linux exposes the kernel as files. Read them; sometimes write them.

/proc

A virtual filesystem that gives userspace a view of kernel state. Each process has a directory `/proc/<pid>/`. Plus system-wide files at `/proc/...`.

Per-process files

Path	What
`/proc/PID/cmdline`	argv (NUL-separated)
`/proc/PID/exe`	symlink to executable
`/proc/PID/cwd`	symlink to current working dir
`/proc/PID/fd/N`	symlink for fd N
`/proc/PID/maps`	virtual memory map
`/proc/PID/status`	human-readable status (state, RSS, threads)
`/proc/PID/stat`	machine-readable stats
`/proc/PID/io`	bytes read/written
`/proc/PID/limits`	rlimit values
`/proc/PID/environ`	environment variables (NUL-separated)
`/proc/PID/comm`	short process name
`/proc/PID/task/TID/`	per-thread info

```
$ cat /proc/$$/maps        # shell's address space
$ cat /proc/$$/status      # readable status
$ ls -l /proc/$$/fd        # open fds
```

System-wide

Path	What
`/proc/cpuinfo`	CPU details
`/proc/meminfo`	memory totals/free/buffers
`/proc/loadavg`	load averages
`/proc/uptime`	seconds since boot
`/proc/version`	kernel build string
`/proc/cmdline`	kernel command line
`/proc/modules`	loaded modules

Path	What
`/proc/devices`	character/block devices
`/proc/net/tcp`	TCP sockets
`/proc/net/dev`	per-interface stats
`/proc/sys/...`	sysctl tunables
`/proc/interrupts`	per-IRQ counters
`/proc/stat`	system-wide stats
`/proc/buddyinfo`	per-zone free pages
`/proc/slabinfo`	slab allocator stats

sysctl — write to tune

```
echo 1 > /proc/sys/net/ipv4/ip_forward         # enable forwarding
sysctl net.ipv4.tcp_keepalive_time              # query
sysctl -w net.ipv4.tcp_keepalive_time=60        # set
```

`/etc/sysctl.conf` for persistent settings.

/sys (sysfs)

A second virtual filesystem, primarily for **device** info. Mount point `/sys`.

```
/sys/class/        — by device class (net, block, dri, etc.)
/sys/bus/          — by bus (pci, usb, platform)
/sys/devices/      — actual device topology
/sys/module/       — per-module
/sys/kernel/       — kernel core
```

Examples:

```
$ cat /sys/class/dma_buf/                    # dma-buf class
$ cat /sys/class/drm/card0/device/vendor     # GPU vendor (e.g., 0x1002 = AMD)
$ cat /sys/class/net/eth0/operstate
$ cat /sys/block/sda/queue/scheduler         # I/O scheduler
$ echo bfq > /sys/block/sda/queue/scheduler  # set
```

For drivers, sysfs is **how user-visible attributes are exposed**. Your `amdgpu_*` debugfs/sysfs entries (e.g., `pp_dpm_sclk` to read GPU clocks) live here.

When you write a driver chapter (Part VI), you'll create your own sysfs files via `device_create_file` or `DEVICE_ATTR`.

/dev

Device nodes. Each entry is **major:minor** numbers identifying a kernel driver (we'll see this in Part V/VI).

```
$ ls -l /dev/null /dev/zero /dev/random /dev/dri/card0 /dev/sda
crw-rw-rw- 1 root root 1, 3 ... /dev/null
crw-rw-rw- 1 root root 1, 5 ... /dev/zero
crw-rw-rw- 1 root root 1, 8 ... /dev/random
crw-rw---- 1 root render 226, 0 ... /dev/dri/card0
brw-rw---- 1 root disk 8, 0 ... /dev/sda
```

`c` = character device, `b` = block device.

When you `open("/dev/dri/card0", ...)`, the kernel routes to the driver registered with major 226. `read`, `write`, `ioctl`, `mmap` on that fd land in the driver's file_operations.

Key device families: - `/dev/sd*`, `/dev/nvme*` — block devices. - `/dev/tty*`, `/dev/pts/*` — terminals. - `/dev/null`, `/dev/zero`, `/dev/random` — pseudo devices. - `/dev/input/*` — keyboard, mouse, gamepad. - `/dev/dri/card*`, `/dev/dri/renderD*` — GPU. - `/dev/kfd` — AMD compute (KFD). - `/dev/snd/*` — ALSA audio.

/dev/shm

POSIX shared memory backing. `shm_open("/foo", ...)` actually creates `/dev/shm/foo`.

/dev/pts

Pseudo-terminals for SSH/screen/tmux sessions. Each terminal gets a `/dev/pts/N`.

/dev/loop, mounting images

`losetup /dev/loop0 image.img` then `mount /dev/loop0 /mnt` lets you mount a disk image as a filesystem. Useful for running a kernel test from an image without writing to disk.

procfs limits

- Lots of `/proc/*` and `/sys/*` files don't have a meaningful "size" — `stat` shows 0. Read them; size is whatever bytes you get.
- They're not seekable (or seekable in odd ways).
- Cannot mmap.
- Don't expect O(1) reads — `/proc/PID/maps` for a big process is regenerated on each read.

Kernel-style attributes (preview)

When you write your own driver later, you'll expose info via: - **sysfs**: small, named, single-value attributes (`/sys/class/.../foo`). Use for exposed configuration. - **debugfs**: `/sys/kernel/debug/...`. Free-form. Used for debugging-only info. - **procfs**: legacy; new code uses sysfs/debugfs. - **ioctls**: structured commands; opaque to filesystem.

Container-aware /proc

Inside a container, `/proc` shows only the container's view (e.g., its own PIDs). This is done by the **PID namespace**. Some tools (htop, ps) read `/proc` and "just work" inside containers.

Tools that walk these filesystems

- `ps` — reads `/proc/*`.
- `top`, `htop` — read `/proc/*`.
- `lsof` — reads `/proc/*/fd`.
- `pmap` — reads `/proc/PID/maps`.
- `lscpu` — reads `/sys/devices/system/cpu/`.
- `lsblk` — reads `/sys/block/`.
- `lspci` — reads `/sys/bus/pci/devices/`.
- `udev` (systemd) — reacts to events from `/sys`.

You'll be reading these files for the rest of your career.

Common bugs

1. Trying to **mmap** /proc files — fails for most.
2. Assuming **stable size** — many entries report size 0.
3. Parsing brittle **`/proc/stat`** layout — it's column-positional and field counts can change between kernel versions.
4. Reading non-atomic state — between two reads, things may change. Use `/proc/[pid]/status` carefully.

Try it

1. Read `/proc/self/maps` and parse it. List heap, stack, .text.
2. Read `/proc/cpuinfo`. Print only model name and cache size for each CPU.
3. Read `/sys/class/net/*/operstate` for every interface.
4. Find your GPU: `cat /sys/class/drm/card0/device/vendor` (0x1002 = AMD, 0x10de = NVIDIA, 0x8086 = Intel).
5. Watch `/proc/interrupts` over a `for` loop in shell; correlate with activity (move mouse, run a `dd`).
6. Read `/proc/[pid]/io` for a process. Understand bytes_read vs bytes_written. Compare to `/proc/[pid]/stat`.

Read deeper

- `man 5 proc` — exhaustive catalog of /proc.
- TLPI chapter 12 (system info).
- `Documentation/filesystems/proc.rst` and `sysfs.rst` in the kernel.

Next → Userspace performance & debug tools.

USP 12 — Userspace performance & debug tools

> If you can't measure it, you can't fix it. Master these tools.

strace — trace system calls

```
strace ./prog                    # all syscalls
strace -e trace=openat ./prog    # only opens
strace -f ./prog                 # follow forks/threads
strace -p PID                    # attach to running
strace -c ./prog                 # summary (count, time)
strace -tt -T ./prog             # timestamps and durations
strace -o log.txt ./prog         # write to file
```

If your program "hangs," strace tells you on which syscall. If it "fails silently," strace shows the errno.

Limit to interesting syscalls or you drown in `read` / `write` / `mmap`.

ltrace — trace library calls

```
ltrace ./prog
```

Shows libc calls: `malloc`, `printf`, `strcpy`, etc. Useful for "what's this program asking glibc to do?"

perf — Linux's profiler (the most important tool)

Counters

```
perf stat ./prog                 # basic counters: instructions, cycles, IPC
perf stat -e cache-misses,cache-references,page-faults ./prog
perf list                        # available events
```

CPU profile

```
perf record ./prog               # sample call stacks
perf report                      # interactive viewer
perf report --stdio              # text dump
```

For full call graphs:

```
perf record -g -- ./prog
perf report -g graph --no-children
```

Top, live

```
perf top -p PID
```

Trace events

```
perf trace ./prog                   # like strace but lower overhead
```

Flame graphs (read these on a wall)

```
perf record -g -F 99 -- ./prog
perf script | ./stackcollapse-perf.pl | ./flamegraph.pl > out.svg
```

Brendan Gregg's flame graphs (https://www.brendangregg.com/flamegraphs.html) — every senior performance engineer uses them.

valgrind — memory error detector

```
valgrind ./prog
valgrind --leak-check=full --show-leak-kinds=all ./prog
```

Catches: - Use-after-free. - Double-free. - Buffer overruns/underruns. - Uninitialized memory reads. - Memory leaks at exit.

10–50× slower than native, but priceless for catching memory bugs.

Alternative: **AddressSanitizer (ASAN)**, faster (2–3× slowdown), built into gcc/clang:

```
gcc -fsanitize=address -g prog.c -o prog
./prog
```

Use ASAN by default. Valgrind for the cases ASAN misses (rare).

ThreadSanitizer (TSan)

```
gcc -fsanitize=thread -g -O1 prog.c
```

Catches data races, deadlocks. Slower (5–10×).

UndefinedBehaviorSanitizer (UBSan)

```
gcc -fsanitize=undefined -g prog.c
```

Catches signed integer overflow, shift overflow, division by zero, null deref, etc. Often free in CPU; always enable in CI.

gdb — the debugger

Basics

```
gdb ./prog
(gdb) run
(gdb) break main
(gdb) break filename.c:42
(gdb) break funcname
(gdb) run arg1 arg2
(gdb) next            # n: step over
(gdb) step            # s: step into
(gdb) continue        # c
(gdb) finish          # run to return
(gdb) print var       # p var
(gdb) print *ptr@10   # 10-element array
(gdb) info locals
(gdb) backtrace       # bt
(gdb) frame 3
(gdb) watch var       # break when var changes
(gdb) info threads
(gdb) thread 3
(gdb) quit
```

Crash analysis

```
ulimit -c unlimited
./prog                          # crashes, leaves a core file
gdb ./prog core
(gdb) bt
```

Attach to running

```
gdb -p PID
```

Useful commands

- `pretty print`: `(gdb) set print pretty on`
- `tui`: `(gdb) layout src` — split view.
- `rr` (record + replay) — astonishing tool for "I can step backwards to find the bug."

Add `gdbinit` (`~/.gdbinit`) for shortcuts and Python pretty-printers.

addr2line — convert addresses to source

```
addr2line -e prog 0x401234
```

Used when you have a stack trace from a crash.

objdump / readelf / nm — inspect binaries

```
readelf -h prog                    # ELF header
readelf -s prog                    # symbols
nm prog | head                     # symbols simplified
objdump -d prog | less             # disassembly
objdump -h prog                    # section headers
```

```
        file prog                       # what is this file
        strings prog | head             # printable strings
```

Useful for “is this stripped?”, “what symbols does it export?”, “what does this assembly do?”

ltrace, ldd

`ldd prog` — what shared libraries does this binary need? `nm prog | grep ' U '` — undefined symbols (resolved at runtime).

time — basic wallclock

```
        time ./prog
```

```
real    0m3.241s
user    0m2.950s
sys     0m0.180s
```

`real` = wallclock; `user` = CPU in user mode; `sys` = CPU in kernel. `(user + sys) / real` ≈ degree of parallelism.

/usr/bin/time -v

More detail: peak RSS, page faults, voluntary/involuntary context switches.

ftrace, eBPF, bcc, bpftrace — kernel-side tracing

These work mostly on kernel-level events; we’ll cover them in Part V/VIII. Mention here:

- `trace-cmd record / report` — wraps ftrace.
- `bpftrace -e 'tracepoint:syscalls:sys_enter_openat { printf("%s\n", str(args->filename)); }'`
- `bcc` tools (`opensnoop`, `tcpconnect`, `execsnoop`, etc.).

These let you trace **anything** in the kernel, low overhead, without rebuilding.

htop, iotop, iostat, vmstat

System-level health: - `htop` — colorful top. - `iotop` — disk I/O per process. - `iostat -xz 1` — per-disk throughput, await, util. - `vmstat 1` — paging, CPU, IO summary. - `ss -ant` — TCP states. - `dstat` — combined.

/proc/PID/wchan — what’s it waiting on?

```
        cat /proc/PID/wchan
```

Prints kernel function it’s blocked in. Great for “why is it stuck?”

A debugging mindset (the 30-minute rule again)

When you hit a problem:

1. **Reproduce reliably.** A 1-in-100 bug is a nightmare; reproduce often if possible.
2. **Form a hypothesis.** "I think X is failing because Y."
3. **Test it cheaply.** strace, print, single test.
4. **Read the failing code carefully.** Often the bug is on the line you already read.
5. **Bisect.** When did it start working/stop working? `git bisect`. Or remove half the inputs.
6. **Use a sanitizer.** ASAN finds bugs your eye won't.
7. **Get a smaller test case.** A 5-line repro is gold.

Try it

1. Compile a buggy program (use-after-free) under ASAN. Verify it catches the bug.
2. Profile a CPU-heavy program (e.g., dumb merge sort on 10M ints) with `perf record`. Identify the hot function.
3. Run the same with TSan (must be threaded). Try one with deliberately racing threads.
4. Crash a program with NULL deref. Get the core file. Use gdb to print the crashing line.
5. Use `bpftrace -e 'tracepoint:syscalls:sys_enter_open* { @[comm]=count(); }'` for 30 seconds. See which programs open the most files.
6. Use `strace -c` on a few different programs. Compare which syscalls dominate.

Read deeper

- *Systems Performance: Enterprise and the Cloud* by Brendan Gregg — the bible for performance work.
- *BPF Performance Tools* by Brendan Gregg.
- "Mistakes Engineers Make in Large Established Codebases" by Will Larson — broader engineering wisdom.

This finishes Part USP. Next stage in the LEARNING PATH:

- **Stage 6**: C++ for systems (3 weeks)
- **Stage 7**: Multithreading deep (4 weeks)
- **Stage 8**: OS theory deep — Part OST (4–6 weeks) — coming up next.

Then the kernel.

Chapter 11 — From C to C++: classes, RAII, references

The Linux *kernel* is C. But the AMD GPU stack in *userspace* — Mesa, ROCm runtime (HSA), HIP runtime, ROCclr, MIOpen, RCCL, rocBLAS, MIGraphX — is overwhelmingly C++. So is most NVIDIA software (CUDA runtime, cuDNN). You must be fluent.

This part isn't a complete C++ course. It's the slice of C++ that systems engineers use, in the order that matters for our work.

11.1 What C++ adds over C

The big four:

1. **Classes / methods / access control** — encapsulation as a language feature.
2. **RAII (Resource Acquisition Is Initialization)** — destructors give automatic cleanup.
3. **Templates** — type-generic code at compile time.
4. **References** — non-null aliases instead of pointers.

There are dozens of smaller features (operator overloading, exceptions, lambdas, `auto`, namespaces, `constexpr`), but the four above are why C++ exists.

11.2 Compiling C++

```
g++ -std=c++17 -Wall -Wextra -O2 -g hello.cpp -o hello
clang++ -std=c++20 ... # newer dialect
```

`g++` is just `gcc` with C++ defaults. The Linux kernel uses C; userspace GPU stacks are typically C++17 today. ROCm has been moving to C++17/20.

```
#include <iostream>

int main()
{
    std::cout << "Hello, C++\n";
}
```

`std::cout << x` is a function call (`operator<<`). It is, in many ways, a worse `printf` — slower, harder to localize. Most systems C++ avoids `iostream` and uses `fmt::print` (C++20: `std::print`) or sticks to `printf`.

11.3 Namespaces

```
namespace amd {
namespace gpu {
    int do_thing();
}}
```

Or (C++17):

```
namespace amd::gpu {
    int do_thing();
}
```

Use:

```
amd::gpu::do_thing();
using namespace amd;     // pull in everything (avoid in headers)
gpu::do_thing();
```

Headers must never have `using namespace` at top level.

11.4 Classes

```
class Counter {
public:
    Counter() : value_(0) {}
    void  inc() { ++value_; }
    int   get() const { return value_; }
private:
    int value_;
};

Counter c;
c.inc();
int v = c.get();
```

Notes:

- `private` members aren't accessible outside.
- `const` after the parameter list means "this method does not modify state."
- The `:` syntax is the **member-initializer list** — preferred over assigning in the body.
- Trailing underscore on member names is a common convention (Google, ROCm).

11.5 RAII — the single biggest reason to use C++

A class's *destructor* runs when the object goes out of scope. So **every resource** can be tied to an object's lifetime. No manual `free`, no `goto out_unmap`.

```
class FileHandle {
public:
    explicit FileHandle(const char *path)
        : fp_(std::fopen(path, "rb")) {
        if (!fp_) throw std::runtime_error("open failed");
    }
    ~FileHandle() { if (fp_) std::fclose(fp_); }

    FileHandle(const FileHandle&)            = delete;
    FileHandle& operator=(const FileHandle&) = delete;

    FILE *get() const { return fp_; }
private:
    FILE *fp_;
};

void load() {
    FileHandle f("/etc/hostname");
```

```
            /* read from f.get() */
        }   /* fclose runs automatically here */
```

This pattern is used for:

- file descriptors,
- mutex locks (`std::lock_guard`),
- heap allocations (`std::unique_ptr`),
- DMA buffers,
- CUDA/HIP streams,
- GPU command buffers.

ROCm, Mesa, and CUDA samples are filled with RAII wrappers. Internalize it.

The `= delete` lines suppress copy. Without them, copying `FileHandle` would `fclose` the same FILE twice. We'll discuss copy/move in Chapter 12.

11.6 References

A reference is an alias to an object. Once bound, it cannot be rebound. It cannot be null.

```
        int  x = 5;
        int& r = x;     /* r is x */
        r = 10;         /* x is now 10 */
```

Use references for:

- Function parameters that should not be null and should not be copied:

```
        void f(const std::string& s);   /* no copy, no null */
```

- Returning aliases.

In kernel-style C, we pass `T *`. In C++, prefer `T&` (or `const T&`) for parameters that "are" the object, not just optionally point to one. Use `T*` when the parameter may be null.

11.7 The four kinds of `T`

C++ distinguishes:

- `T` — value (object).
- `T&` — lvalue reference (alias).
- `T&&` — rvalue reference (movable temporary). C++11.
- `T*` — pointer.

`const` orthogonal to all. So `const std::string&` means "alias to a string I won't modify." `std::string&&` means "this is a temporary, you can steal its contents."

We will revisit `&&` and move semantics in Chapter 14.

11.8 `new` and `delete` (and why we won't use them)

```
        int *p  = new int(5);
        delete p;
```

```
int *a = new int[100];
delete[] a;
```

Manual `new` / `delete` is C++'s `malloc` / `free`. Modern C++ avoids both — we use `std::unique_ptr` / `std::shared_ptr` /STL containers, which manage memory for us via RAII. We will write our own `unique_ptr` in Chapter 14 to demystify the technique.

11.9 References vs. pointers — a rule of thumb

Need	Use
Optionally-null, may rebind	`T *`
Never-null, never-rebound, doesn't own	`T &`
Never-null, never-rebound, owns	`T` (value) or `std::unique_ptr`
Shared ownership	`std::shared_ptr<T>`
Out-parameter	`T *` (or return a struct)
Input parameter, large object	`const T &`
Input parameter, small (≤16B)	`T` (by value)

11.10 Headers and TUs in C++

Same translation-unit model as C. But:

- `class` definitions go in headers.
- Member function *declarations* go in the class body; *definitions* may go in the header (as `inline` or `template`) or a `.cpp`.
- Template definitions almost always live in headers (more in Chapter 13).

Build systems are CMake or Meson; you'll see them in Mesa and ROCm.

11.11 Exercises

1. Translate `Counter` from above to a C version using opaque types and `_init` / `_destroy`. Notice how much shorter the C++ version is.
2. Implement `class Vec3 { float x, y, z; };` with `+`, `-`, `dot`, `cross`, `length`. Make every member function `const` where possible.
3. Implement `FileHandle` exactly as shown. Then copy it; observe the compile error caused by `= delete`. Remove the deletes; make a copy; observe a double-`fclose` (try it under ASan).
4. Read `include/CL/sycl.hpp` or any ROCm runtime header. Identify three RAII wrappers.

Next: Chapter 12 — Object lifetime, special members, rule of 0/3/5.

Chapter 12 — Object lifetime, special members, rule of 0/3/5

C++ controls the lifecycle of every object via *special member functions*. Get these wrong and you have memory corruption, double-free, or thread races. Get them right and your code is leak-proof and exception-safe.

12.1 The six special member functions

For every class, the compiler may generate:

```
class T {
public:
    T();                            /* default constructor */
    T(const T& other);              /* copy constructor */
    T(T&& other) noexcept;          /* move constructor   (C++11) */
    T& operator=(const T& other); /* copy assignment */
    T& operator=(T&& other) noexcept; /* move assignment (C++11) */
    ~T();                           /* destructor */
};
```

If you don't define any, you get all six. If you define one, the rules about which are auto-generated change. Memorize the safe rule:

> **Rule of 0**: design so you don't need any. Use members that already manage themselves (`std::unique_ptr` , `std::vector` , `std::string`).
>
> **Rule of 5**: if you define one of {dtor, copy ctor, copy assign, move ctor, move assign}, define (or `delete`) all five.
>
> **Rule of 3** (pre-C++11): same idea without moves.

12.2 The order of construction and destruction

Members are constructed in *declaration* order, regardless of the order in the initializer list. They are destroyed in *reverse* declaration order. Base classes are constructed before derived members and destroyed after.

```
class Cache {
public:
    Cache(size_t cap) : data_(cap), tail_(0) {}    /* OK */
private:
    std::vector<int> data_;
    size_t           tail_;
};
```

This compiles and works because `data_` is declared first. If you reverse declaration order to `tail_, data_`, the initializer list still constructs `tail_` first. Compilers may warn about "out of order initialization." Always **match the initializer list to declaration order**.

12.3 Construction options: default, value, copy, move

```
std::string a;          /* default-constructed */
std::string b("hello"); /* constructed from a const char* */
std::string c = b;      /* copy */
std::string d(b);       /* copy */
std::string e = std::move(b); /* move; b is now valid-but-unspecified */
```

A `move` doesn't necessarily change anything. It is a *cast* from `T&` to `T&&` that lets the destination "steal" the source's resources. After the move, the source must remain destructible and assignable, but its content is undefined.

`std::move` is **not** a function that moves. It's a type cast. The actual transfer happens in the move constructor / move assignment of the destination.

12.4 A concrete RAII type with the rule of 5

```
template <class T>
class UniquePtr {
public:
    UniquePtr() noexcept : p_(nullptr) {}
    explicit UniquePtr(T *p) noexcept : p_(p) {}

    ~UniquePtr() { delete p_; }

    /* No copying */
    UniquePtr(const UniquePtr&)            = delete;
    UniquePtr& operator=(const UniquePtr&) = delete;

    /* Move */
    UniquePtr(UniquePtr&& other) noexcept : p_(other.p_) {
        other.p_ = nullptr;
    }
    UniquePtr& operator=(UniquePtr&& other) noexcept {
        if (this != &other) {
            delete p_;
            p_ = other.p_;
            other.p_ = nullptr;
        }
        return *this;
    }

    T*  get()        const noexcept { return p_; }
    T&  operator*()  const          { return *p_; }
    T*  operator->() const noexcept { return p_; }

    explicit operator bool() const noexcept { return p_ != nullptr; }

    T* release() noexcept { T *t = p_; p_ = nullptr; return t; }
    void reset(T *p = nullptr) noexcept { delete p_; p_ = p; }

private:
    T *p_;
};
```

Read every line. This is `std::unique_ptr` minus the deleter template parameter. We:

- Disable copy (a unique pointer cannot be copied).
- Implement move (transfers ownership).
- Provide `release` (give up ownership without deleting) and `reset` (replace, deleting the old).

`noexcept` is critical. Many STL algorithms only use the move constructor if it's `noexcept`; otherwise they fall back to copy.

12.5 Why `noexcept` matters

Consider `std::vector<T>` growing. It must move (or copy) all elements to a bigger buffer. If `T`'s move can throw, then mid-move an exception could leave the vector in a weird state. C++'s strong exception guarantee says: "if you can't move-without-throwing, copy instead."

For systems code, you usually want `noexcept` move ops. They are a perf win and a correctness win.

12.6 Inheritance and virtual destructors

If a class has any `virtual` method, its destructor *must* be virtual:

```
class Base {
public:
    virtual ~Base() = default;
    virtual void op() = 0;          /* pure virtual */
};

class Derived : public Base {
public:
    void op() override { /* ... */ }
};

Base *b = new Derived;
delete b;       /* calls Derived::~Derived() because dtor is virtual */
```

If `~Base()` weren't virtual, deleting through a `Base*` would skip `Derived::~Derived` — UB. Many CUDA/HIP-era bugs stem from missing `virtual` destructors.

`override` (C++11) tells the compiler "this method overrides a base virtual"; if the signature mismatches, you get a compile error. **Always** write `override`.

12.7 Slicing

```
Derived d;
Base   b = d;     /* SLICED: only Base part copied; vtable lost */
```

Passing derived-by-value where a base-by-value is expected silently slices. Always pass polymorphic objects by reference or pointer:

```
void use(const Base& b);
void use(const Base *b);
```

12.8 `final`

```
class Sealed final { ... };
class D : public Base { void op() final override; };
```

`final` on a class disallows further inheritance. `final` on a method disallows further override. Use it when you mean it; it lets the compiler devirtualize.

12.9 `default` and `delete`

```
class T {
public:
    T() = default;        /* explicitly want compiler-generated */
    T(const T&) = delete; /* explicitly want it to NOT exist */
};
```

If you make a class non-copyable, prefer `= delete` over making the copy ctor private without a definition. The error is at compile time and clearer.

12.10 Initializer lists, `{}` initialization

C++11 brace-init:

```
int x{};         /* zero-init */
int a[]{1,2,3};
std::vector<int> v{1,2,3};
struct P{ int x; int y; };
P p{1, 2};
P q = {1, 2};

/* Important: brace-init disallows narrowing */
int n{3.5};      /* error: narrows double to int */
```

Prefer `{}` over `()` for initialization. It's safer and works uniformly across types. Exceptions: when the type has a `std::initializer_list` constructor, `{}` may pick that overload (e.g. `std::vector<int>{10}` is `[10]`, not "10 default-inits").

12.11 Avoiding common lifetime bugs

- **Returning reference to local**: UB.
- **Returning span/view of a temporary**: UB.
- **Storing a reference in a struct then outliving the referent**: UB.
- **Calling a virtual method from a constructor**: only the current class's version runs (the derived class isn't constructed yet).
- **Throwing from a destructor**: program likely terminates if another exception is in flight.

12.12 Exercises

1. Write `class Buffer` that owns a `char *` allocated with `new[]`. Implement the rule of 5. Test: copy, move, resize, exception during copy.
2. Make `Buffer` non-copyable but movable. Implement `operator+=` for concatenation.
3. Convert your `class FileHandle` (Chapter 11) into a templated `Handle<Resource, Closer>` with a deleter callable.

4. Make a base `class GpuObject` with virtual destructor, two derived `class Buffer` and `class Texture`. Store both in `std::vector<std::unique_ptr<GpuObject>>` and call virtual methods.
5. Read `<memory>` (`/usr/include/c++/*/memory`) and find `std::unique_ptr`'s actual definition. Compare to your version.

Next: Chapter 13 — Templates, concepts, and the type system.

Chapter 13 — Templates, concepts, and the type system

Templates are how C++ does generic programming. They are also the principal source of "1500-line error messages." Used correctly, they are how the STL, Eigen, fmt, and most of ROCm runtime are built.

13.1 Function templates

```
template <class T>
T max_of(T a, T b) { return a > b ? a : b; }

int  m1 = max_of(3, 4);              /* T = int */
auto m2 = max_of(1.0, 2.0);          /* T = double */
auto m3 = max_of<int>(3, 4.0);       /* explicit T */
```

The compiler *instantiates* a separate function for each unique `T` you use. Templates are not erased at runtime; they generate concrete code at compile time. This is unlike Java/C# generics.

13.2 Class templates

```
template <class T>
class Vec {
public:
    Vec() : data_(nullptr), size_(0), cap_(0) {}
    /* … */
    void push(T v);
    T&   operator[](size_t i);
private:
    T     *data_;
    size_t size_, cap_;
};

template <class T>
void Vec<T>::push(T v) { /* … */ }
```

Note the funny syntax for out-of-line member definitions. Most class-template implementations live in headers because the compiler must see the body to instantiate.

13.3 Specialization and overloading

You can provide a custom implementation for a specific type:

```
template <class T> struct is_pointer { static constexpr bool value = false; };
template <class T> struct is_pointer<T*> { static constexpr bool value = true; };
```

`is_pointer<int>::value == false`, `is_pointer<int*>::value == true`. This is partial specialization.

Function templates don't have partial specialization, but they do have overloading; use overloads to specialize behavior.

13.4 Template type deduction

Three contexts:

```
template <class T> void f(T x);   /* by value: T = remove_cvref */
template <class T> void g(T& x);  /* by ref: T may be const */
template <class T> void h(T&& x); /* forwarding ref: special rules */
```

`T&&` in a template parameter is a *forwarding reference* (sometimes called "universal reference"): it deduces to `T&` if the argument is an lvalue and `T` if rvalue. Combined with `std::forward<T>(x)`, this is how `emplace_back`, `make_unique`, etc. perfect-forward arguments.

```
template <class T, class... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

Read each piece:

- `class... Args` — variadic template parameter pack.
- `Args&&...` — pack of forwarding references.
- `std::forward<Args>(args)...` — preserves lvalue/rvalue-ness of each.

13.5 SFINAE and `enable_if`

"Substitution Failure Is Not An Error." If template substitution produces an invalid type, the candidate is silently removed from the overload set instead of producing a hard error.

```
template <class T,
          class = std::enable_if_t<std::is_integral_v<T>>>
T abs_int(T v) { return v < 0 ? -v : v; }
```

`enable_if_t<true>` is `void`; `enable_if_t<false>` is ill-formed → SFINAE removes the overload. Hideously verbose. C++20 cleaned this up with **concepts**.

13.6 Concepts (C++20)

```
template <class T>
concept Integral = std::is_integral_v<T>;

template <Integral T>
T abs_int(T v) { return v < 0 ? -v : v; }
```

Or:

```
template <class T>
requires Integral<T>
T abs_int(T v) { ... }
```

Or short:

```
auto abs_int(Integral auto v) { ... }
```

Concepts give clear error messages and act as documentation. ROCm and modern Mesa are starting to adopt C++20 concepts.

13.7 Variadic templates and parameter packs

```
template <class... Ts>
void print_all(Ts... args) {
    (std::cout << ... << args) << '\n';   /* C++17 fold expression */
}

print_all(1, "hi", 3.14);
```

Older syntax used recursion:

```
void print_all() {}                       /* base */
template <class T, class... Ts>
void print_all(T x, Ts... xs) {
    std::cout << x << ' ';
    print_all(xs...);
}
```

Fold expressions (C++17) made this much cleaner.

13.8 `constexpr` and compile-time evaluation

```
constexpr int factorial(int n) { return n <= 1 ? 1 : n * factorial(n-1); }
constexpr int x = factorial(10);   /* computed at compile time */
```

`constexpr` lets a function run at compile time when its inputs are constants. C++20 expands this with `consteval` (always at compile time) and `constinit` (must be initialized at compile time but is a runtime variable thereafter).

`constexpr if`:

```
template <class T>
auto safe_abs(T v) {
    if constexpr (std::is_signed_v<T>)
        return v < 0 ? -v : v;
    else
        return v;
}
```

The unused branch is *not compiled* for that instantiation. This replaces many SFINAE tricks.

13.9 Type traits

In `<type_traits>`:

```
std::is_same_v<T, U>
std::is_integral_v<T>
```

```
std::is_pointer_v<T>
std::is_const_v<T>
std::remove_cv_t<T>
std::decay_t<T>
std::conditional_t<cond, A, B>
```

Use these in metaprogramming and as concepts' building blocks.

13.10 CRTP — Curiously Recurring Template Pattern

A C++ idiom for static polymorphism:

```
template <class Derived>
class Base {
public:
    void run() { static_cast<Derived*>(this)->doit(); }
};

class Foo : public Base<Foo> {
public:
    void doit() { /* … */ }
};

Foo f;
f.run();    /* dispatches to Foo::doit at compile time */
```

No vtable. Useful for hot paths where virtual dispatch is too expensive. Eigen, ROCm runtime command-list builders, and some Mesa code use CRTP.

13.11 The cost of templates

- **Compile time** explodes with template usage. ROCm device builds can take minutes per file.
- **Binary size** can balloon if many instantiations exist.
- **Error messages** are infamous; learn to read them by ignoring 80% of the noise and finding the *first* diagnostic before the cascade.

Mitigations:

- `extern template class Foo<int>;` to prevent instantiation in this TU.
- Move template definitions behind PIMPL (Chapter 16) when possible.

13.12 Exercises

1. Implement `template <class T> class Vector` with `push_back`, `operator[]`, `size`, copy/move semantics, and exception safety. Compare to `std::vector`.
2. Implement `make_unique<T>(args...)` using forwarding references.
3. Implement `Optional<T>` (a poor-man's `std::optional`).
4. Implement a `Tuple<Ts...>` using inheritance recursion. It will be ugly. Then read how the standard implements it.
5. Read `template<class T> class shared_ptr` from `<memory>` (it's intricate). Identify the control block, the `weak_ptr` slot, the deleter type-erasure.

Next: Chapter 14 — Modern C++: move semantics, smart pointers, lambdas.

Chapter 14 — Modern C++: move semantics, smart pointers, lambdas

This chapter is the slice of "modern C++" you'll see in any post-2015 systems codebase. ROCm, Mesa, LLVM, MIOpen: they all live here.

14.1 Why move semantics

Pre-C++11, returning a `std::vector<int>` from a function copied the contents. For a 1 MB vector that meant `memcpy`-ing a megabyte. The compiler often elided the copy via *return value optimization* (RVO), but you couldn't rely on it.

C++11 added rvalue references and move constructors. Now:

```
std::vector<int> make() {
    std::vector<int> v(1'000'000);
    return v;          /* move (or RVO) */
}
auto big = make();     /* no copy */
```

The vector's move constructor swaps three pointers. Done.

14.2 Lvalues, rvalues, xvalues

- **lvalue**: has a name and address. `int x;` — `x` is an lvalue.
- **rvalue**: temporary, can be moved-from. `f()`, `5`, `std::move(x)`.
- **xvalue** ("expiring"): rvalue that names something. `std::move(x)`.

`T&` binds only to lvalues. `T&&` binds only to rvalues. `const T&` binds to both — so it's the safe parameter type when you only read.

```
void f(int& x);        /* only lvalues */
void f(int&& x);       /* only rvalues */
void f(const int& x);  /* both */

int a = 5;
f(a);     /* picks f(int&) */
f(5);     /* picks f(int&&) */
```

14.3 `std::move` and `std::forward`

```
std::string s = "hello";
std::string t = std::move(s);    /* s is now valid-but-unspecified */
```

`std::move(x)` is `static_cast<T&&>(x)`. It does nothing at runtime; it changes the type so that move overloads are picked.

`std::forward<T>(x)` is move-or-not depending on `T`:

```
template <class T>
void wrap(T&& arg) {
    inner(std::forward<T>(arg));   /* preserves lvalue/rvalueness */
}
```

In a function template, prefer `std::forward<T>(arg)` to `std::move(arg)` for forwarding references, otherwise you'll always move and break callers passing lvalues.

14.4 Smart pointers

```
#include <memory>

auto p = std::make_unique<int>(42);     /* unique_ptr<int> */
auto s = std::make_shared<int>(42);     /* shared_ptr<int> */
```

Use:

- `unique_ptr<T>`: sole ownership. Move-only. Zero overhead vs. raw pointer.
- `shared_ptr<T>`: reference-counted. Has a control block (~16-24 bytes extra). Atomic refcount; thread-safe to copy/destroy a `shared_ptr` from multiple threads (but the *pointee* still needs its own synchronization).
- `weak_ptr<T>`: non-owning observer of a `shared_ptr`. Use to break cycles or to cache something whose lifetime is owned elsewhere.

`make_unique` / `make_shared`:

- `make_shared` allocates the control block + object together, saving an allocation, but the object lifetime is tied to `weak_ptr` count too (memory not freed until last `weak_ptr` dies).

In ROCm/HIP code you will see lots of `unique_ptr<Resource>` for GPU streams, kernels, modules.

14.5 Custom deleters

```
struct FileCloser {
    void operator()(FILE *f) const { if (f) std::fclose(f); }
};

std::unique_ptr<FILE, FileCloser> fp(std::fopen("/etc/hostname", "rb"));
```

Or with a lambda:

```
auto fp = std::unique_ptr<FILE, void(*)(FILE*)>(
    std::fopen("/etc/hostname", "rb"),
    [](FILE *f) { if (f) std::fclose(f); });
```

Same trick wraps file descriptors, hipStream_t, drm_fd, etc.

14.6 Lambdas

```
auto add = [](int a, int b) { return a + b; };
add(2, 3);

auto print = [out = std::ofstream("/tmp/log")](auto&&... a) mutable {
```

```
        ((out << a << ' '), ...);
        out << '\n';
    };
```

Lambdas are anonymous classes with `operator()`. The capture list `[…]` declares what they capture:

- `[]` — nothing.
- `[x]` — by value.
- `[&x]` — by reference.
- `[=]` — all locals by value (avoid).
- `[&]` — all by reference (avoid).
- `[this]` — the enclosing object's `this` pointer (lifetime hazard).
- `[x = 5]` — initialized capture (C++14).
- `[x = std::move(p)]` — move-into-capture.

`mutable` lets the lambda mutate by-value captures.

Beware capturing a reference to a stack variable that outlives the lambda. Beware capturing `this` and the object dying.

14.7 `std::function` vs. function pointers vs. templates

```
#include <functional>
std::function<int(int,int)> f = add;     /* type-erased callable */
```

`std::function` is convenient but:

- May allocate (small-buffer-optimized for tiny callables).
- Has indirect call overhead.

For perf-critical callbacks, prefer template parameter:

```
template <class F>
void each(int *a, size_t n, F&& fn) { for (size_t i = 0; i < n; i++) fn(a[i]); }
```

Or function pointer when capture is unnecessary.

14.8 `auto`, `decltype`, `decltype(auto)`

```
auto x = 5;                 /* int */
auto& r = container[0];     /* reference */
auto v = std::vector{1,2,3};

decltype(x) y = 7;          /* int */
decltype(auto) z = f();     /* preserves reference-ness from f */
```

In modern code, `auto` for locals and range-for loops:

```
for (auto& kv : map) { kv.second++; }
```

In headers and APIs, prefer explicit types — `auto` for return types only when necessary or for very local code.

14.9 Range-based for, structured bindings

```
for (const auto& [key, value] : map) { /* ... */ }    /* C++17 */
auto [a, b, c] = std::tuple{1, 2.5, "hi"};
```

These are the polish that makes C++ usable. Use them.

14.10 `std::optional`, `std::variant`, `std::expected`

```
std::optional<int> parse(const std::string& s);
auto v = parse(s);
if (v) use(*v);

std::variant<int, std::string> u = 5;
std::visit([](auto&& x){ std::cout << x; }, u);

std::expected<int, std::string> safe();    /* C++23 */
```

`std::expected` is the ergonomic equivalent of returning `int` and an out-error. ROCm and Mesa are slowly moving to it.

14.11 `noexcept` revisited

Mark functions `noexcept` when they will not throw. The compiler can:

- skip exception-table generation,
- pick more efficient code paths in STL containers,
- enable strong exception guarantees in callers.

For destructors and move ops, `noexcept` is almost always correct. For other functions, only mark `noexcept` if you are *certain*.

14.12 Exercises

1. Write `class String` that owns a heap buffer. Implement copy, move, `operator+=`, `c_str`, with proper `noexcept` markings.
2. Write `template <class T> class SharedRef` (a poor-man's `shared_ptr`) with a control block.
3. Write a `for_each` that takes a `std::function<void(int)>` and another templated on a callable. Benchmark calling each in a tight loop. Compare assembly.
4. Implement an `LRUCache<K,V>` using `std::list<std::pair<K,V>>` plus `std::unordered_map<K, list::iterator>`.
5. Convert any C-style program you wrote in Part I to modern C++ using RAII, smart pointers, and STL containers. Notice the line-count drop.

Next: Chapter 15 — STL and writing your own container.

Chapter 15 — STL and writing your own container

The Standard Template Library is C++'s killer feature. You must know what's in it, when to reach for what, and what each costs. Then we'll build a small container to demystify the implementation.

15.1 What's in the STL

Container	Underlying	Insert	Lookup	Notes
`std::vector<T>`	contiguous array	amortized O(1) push_back	O(1) random	THE default container
`std::array<T,N>`	fixed array	n/a	O(1)	stack-friendly
`std::deque<T>`	array of arrays	O(1) push_front/ back	O(1) random	rarely needed
`std::list<T>`	doubly-linked	O(1) anywhere	O(n)	rarely; cache-hostile
`std::forward_list<T>`	singly-linked	O(1)	O(n)	very rarely
`std::map<K,V>`	red-black tree	O(log n)	O(log n)	ordered
`std::set<T>`	red-black tree	O(log n)	O(log n)	
`std::unordered_map<K,V>`	hash table	avg O(1)	avg O(1)	high constant factor
`std::unordered_set<T>`	hash table	avg O(1)	avg O(1)	
`std::stack<T>`	adapter			wraps deque
`std::queue<T>`	adapter			wraps deque
`std::priority_queue<T>`	adapter	O(log n) push/ pop		binary heap

> **Default to `std::vector<T>`** unless you have a specific reason. Cache locality beats algorithmic complexity for small-to-medium n.

15.2 `<algorithm>`

```
#include <algorithm>
std::sort(v.begin(), v.end());
std::sort(v.begin(), v.end(), [](auto&a, auto&b){ return a.x < b.x; });
auto it = std::find(v.begin(), v.end(), 42);
auto it = std::lower_bound(v.begin(), v.end(), 42);
auto sum = std::accumulate(v.begin(), v.end(), 0);
v.erase(std::remove_if(v.begin(), v.end(),
        [](int x){ return x < 0; }), v.end());        /* erase-remove idiom */
```

C++20 `std::ranges` cleans this up:

```
        std::ranges::sort(v);
        auto it = std::ranges::find(v, 42);
```

15.3 Iterators

Iterators are generalized pointers. Five categories:

- **Input** (read once, advance) — e.g. stream iterators.
- **Output** (write once, advance).
- **Forward** (re-read OK).
- **Bidirectional** — `std::list`.
- **Random-access** — `std::vector`, raw pointers.

Algorithms specify what they need. `std::sort` needs random-access; `std::find` only forward.

In C++20, **ranges** generalize iterator pairs into `std::ranges::range` objects.

15.4 Iterator invalidation — the silent killer

Op	Invalidates
`vector::push_back` if grew	all iterators/references
`vector::insert/erase`	iterators ≥ point; references
`unordered_map` rehash	all iterators (on insert)
`map::erase`	only the erased iterator
`list::insert`	none

Reading a stale iterator is UB. ASan + iterator debug mode (`-D_GLIBCXX_DEBUG`) catches most.

15.5 The cost of small operations

Approximate, modern x86, in-cache:

Op	Cycles
Integer add	1
L1 hit	4
L2 hit	12
L3 hit	30
DRAM access	100–300
Branch mispredict	15–20
`std::vector::push_back` (no grow)	~10
`std::unordered_map::find`	100–300 (chain follow)
`std::map::find`	200–500 (log n with cache misses)
`std::shared_ptr` copy	30 (atomic inc)
Heap alloc (`malloc`)	100–1000

Cache misses dominate. `std::vector` linear search beats `std::map<int,int>` lookup until n is well into the hundreds.

15.6 Allocators (briefly)

Every STL container has an `Allocator` template parameter. Default is `std::allocator<T>` which calls `::operator new`. For systems work you sometimes want custom allocators (arena, pool, GPU pinned memory).

```
template <class T>
struct PoolAlloc {
    using value_type = T;
    T* allocate(size_t n);
    void deallocate(T*, size_t);
};
```

Don't write one until you measure the standard one is the bottleneck.

15.7 Building your own `Vector<T>`

Now we implement a small `std::vector` clone to understand growth, exception safety, allocator awareness, and move ops.

```
template <class T>
class Vector {
public:
    Vector() : data_(nullptr), size_(0), cap_(0) {}

    explicit Vector(size_t n) : Vector() { reserve(n); for (size_t i=0;i<n;i++) emplace_back(); }

    Vector(const Vector& o) : Vector() {
        reserve(o.size_);
        for (size_t i = 0; i < o.size_; i++) push_back(o.data_[i]);
    }
    Vector(Vector&& o) noexcept
        : data_(o.data_), size_(o.size_), cap_(o.cap_) {
        o.data_ = nullptr; o.size_ = 0; o.cap_ = 0;
    }
    Vector& operator=(Vector o) noexcept {     /* copy-and-swap */
        swap(o); return *this;
    }
    ~Vector() {
        clear();
        ::operator delete(data_);
    }

    void swap(Vector& o) noexcept {
        std::swap(data_, o.data_);
        std::swap(size_, o.size_);
        std::swap(cap_,  o.cap_);
    }

    size_t size()     const noexcept { return size_; }
    size_t capacity() const noexcept { return cap_; }

    T&       operator[](size_t i)       { return data_[i]; }
    const T& operator[](size_t i) const { return data_[i]; }

    T*       begin()       { return data_; }
    T*       end()         { return data_ + size_; }
    const T* begin() const { return data_; }
    const T* end()   const { return data_ + size_; }
```

```
        void reserve(size_t n) {
            if (n <= cap_) return;
            T *new_data = static_cast<T*>(::operator new(n * sizeof(T)));
            size_t i = 0;
            try {
                for (; i < size_; i++) new (new_data + i) T(std::move_if_noexcept(data_[i]));
            } catch (...) {
                for (size_t j = 0; j < i; j++) new_data[j].~T();
                ::operator delete(new_data);
                throw;
            }
            for (size_t j = 0; j < size_; j++) data_[j].~T();
            ::operator delete(data_);
            data_ = new_data;
            cap_  = n;
        }

        template <class... Args>
        T& emplace_back(Args&&... args) {
            if (size_ == cap_) reserve(cap_ ? cap_ * 2 : 4);
            new (data_ + size_) T(std::forward<Args>(args)...);
            return data_[size_++];
        }
        void push_back(const T& v) { emplace_back(v); }
        void push_back(T&& v)      { emplace_back(std::move(v)); }

        void pop_back() { data_[--size_].~T(); }

        void clear() {
            for (size_t i = 0; i < size_; i++) data_[i].~T();
            size_ = 0;
        }

    private:
        T     *data_;
        size_t size_, cap_;
    };
```

Things to note:

- We use `::operator new` for raw memory and *placement new* (`new (ptr) T(args)`) to construct in-place. We call destructors explicitly with `data_[i].~T()` . This separation lets us own *capacity* separate from *constructed objects*.
- `move_if_noexcept` moves only when the move ctor is `noexcept` ; otherwise copies, preserving the strong exception guarantee on `reserve` .
- `swap` then `operator=(Vector o)` is the **copy-and-swap idiom**, automatic exception-safe assignment.
- Growth factor 2× is standard; some implementations use 1.5× for better realloc behavior.

This is ~80 lines and approximately what `libstdc++` does in 4000.

15.8 STL string and `string_view`

```
    #include <string>
    #include <string_view>

    void log(std::string_view sv);     /* take by view, no copy, no allocate */
    log("hello");
    log(std::string("world"));
```

`std::string_view` is a non-owning `(ptr, len)` pair. It's the C++17 way to take a "string parameter" without forcing the caller to allocate. Beware: don't return a `string_view` to a temporary string — UB.

`std::span<T>` (C++20) is the same idea for arbitrary contiguous ranges.

15.9 Hash maps in practice

`std::unordered_map` is OK. For perf-sensitive systems code, look at:

- `tsl::robin_map` — open-addressing.
- `absl::flat_hash_map` — Google's; better cache behavior.
- `boost::unordered_flat_map`.

ROCm-related projects often use Abseil. Know the trade-offs.

15.10 Exercises

1. Implement `Vector<T>` exactly as above. Write a test with `int`, with `std::string`, and with a class that throws on copy.
2. Implement `String` with small-string optimization (≤15 bytes inline, longer on heap).
3. Implement an open-addressing hash map with linear probing for `Map<K,V>`.
4. Benchmark `std::vector<int>::push_back` 1e7 times with reserve vs. without. Plot.
5. Implement an LRU cache using your `Vector`-backed allocator and `std::unordered_map`. Test with 1e6 random ops.

Next: Chapter 16 — C++ for low-level work: PIMPL, polymorphism cost, ABI.

Chapter 16 — C++ for low-level work: PIMPL, polymorphism cost, ABI

C++ has features for high-level expressiveness, but in systems code we often turn most of them off. This chapter is about *which subset* of C++ you actually use in driver-runtimes, and what each feature costs in instructions, cache lines, and binary size.

16.1 The cost of virtual functions

```
struct Base   { virtual void op() = 0; };
struct Foo:Base { void op() override { /* … */ } };
```

Each polymorphic class has a **vtable** — a small static array of function pointers. Each instance has a hidden **vptr** at offset 0 pointing to the vtable. A virtual call is two indirect loads:

```
load   r0, [obj]        ; vptr
load   r1, [r0 + 8]     ; slot for op()
call   r1
```

Cost: ~3-5 ns vs ~0.3 ns for a direct call, primarily from branch-prediction and cache. In a tight loop this can dominate. Mesa, ROCm and CUDA hot paths often avoid virtuals in favor of:

- CRTP (compile-time polymorphism, Chapter 13),
- `std::variant` + `std::visit`,
- function pointers in a struct (the C kernel pattern from Chapter 9),
- branchy switch on an enum.

For *cold* paths (config, initialization), virtual is fine.

16.2 PIMPL — Pointer to IMPLementation

PIMPL hides implementation behind an opaque pointer in the header. Pros: build-time decoupling, ABI stability, less recompilation. Cons: an extra heap allocation, an extra indirection.

```
/* foo.h */
class Foo {
public:
    Foo();
    ~Foo();
    Foo(Foo&&) noexcept;
    Foo& operator=(Foo&&) noexcept;

    void do_thing();

private:
```

```
        struct Impl;
        std::unique_ptr<Impl> p_;
    };
```

```
    /* foo.cpp */
    #include "foo.h"
    #include <vector>          /* heavy header — only included here */

    struct Foo::Impl {
        std::vector<int> data;
        /* … */
    };

    Foo::Foo() : p_(std::make_unique<Impl>()) {}
    Foo::~Foo() = default;
    Foo::Foo(Foo&&) noexcept = default;
    Foo& Foo::operator=(Foo&&) noexcept = default;

    void Foo::do_thing() { p_->data.push_back(0); }
```

Use PIMPL for shared-library APIs you must keep ABI-stable, or for headers included from many places.

16.3 ABI — what it is and why you care

ABI (Application Binary Interface) is the low-level contract: how arguments are passed in registers, how `vptr` is laid out, how the linker mangles names. C++ ABI has been stable on Linux (Itanium ABI) since GCC 5 (2015), but small changes happen.

Key ABI facts:

- **Adding a virtual** to a class changes its vtable layout. Every binary that uses the class must be recompiled.
- **Adding a non-static data member** changes its size. Same.
- **Changing an inline function** in a header that's compiled into multiple shared libraries can produce ODR (one-definition rule) violations.
- **Returning a `std::string`** crosses the ABI; old/new libstdc++ used different layouts ("CXX11 ABI"). You'll see `-D_GLIBCXX_USE_CXX11_ABI=0` in old code.

For shared-library work (HSA runtime, libamdhip64.so), you keep ABI stable by:

1. Using opaque pointer / PIMPL.
2. Versioning symbols (`__attribute__((symver(...)))` or ld scripts).
3. Designing C-style APIs at the boundary, with C++ inside.

The HSA runtime, ROCr, libdrm — all expose **C** APIs over C++ implementations for exactly this reason.

16.4 Symbol mangling and `extern "C"`

C++ mangles names to encode types and namespaces:

```
    nm libfoo.so | c++filt
```

`amd::Buffer::flush() const` becomes something like `_ZNK3amd6Buffer5flushEv`.

To export a function from C++ for C linkage:

```
extern "C" {
    void amd_init(void);
    int  amd_doit(int x);
}
```

Inside `extern "C"`, no overloading is allowed (no name mangling). All public APIs of HSA, OpenCL, ROCm runtime are `extern "C"`.

16.5 Linkage and ODR

The One-Definition Rule: every entity may be defined in at most one TU, *except* templates and inline functions, which may be defined identically in many. Violations → UB.

Common ODR breakers:

- Non-`inline` functions defined in headers.
- Class definitions that differ between TUs (e.g. due to `#ifdef`).
- Mixing CXX11 ABI and pre-CXX11 ABI builds.

`-Wodr` (link-time) catches some.

16.6 The "vocabulary types" of systems C++

Use these consistently:

- `std::string` for owned strings; `std::string_view` for parameters.
- `std::vector<T>` as the default container; `std::span<T>` for parameters.
- `std::unique_ptr<T>` for ownership; raw `T*` for non-owning.
- `std::optional<T>` for "may not have a value."
- `std::variant<…>` for tagged unions.
- `std::filesystem::path` — for file paths.
- `std::array<T, N>` — fixed-size, stack-resident.

16.7 Exception strategy

For systems code there are three valid stances:

1. **No exceptions at all** (`-fno-exceptions`). Used by LLVM core, parts of Mesa, and some embedded code. Errors via `std::expected`-style returns, `bool`, or `enum`.
2. **Exceptions for unrecoverable** errors only. ROCm runtime is roughly here.
3. **Exceptions everywhere.** Some app code; rare in systems.

When in Rome: match the codebase. If you join a `-fno-exceptions` codebase, don't sneak in `throw`.

16.8 The `noreturn`, `[[likely]]`, `[[nodiscard]]` attributes

```
[[noreturn]] void abort_msg(const char *m);

[[nodiscard]] int read_byte();   /* warns if return ignored */
```

```
if (cond) [[likely]] { … } else [[unlikely]] { … }
```

`[[nodiscard]]` on factory functions and error returns prevents bugs.

16.9 Inline assembly (briefly)

You will rarely need inline asm in C++. If you do:

```
unsigned long rdtsc() {
    unsigned hi, lo;
    asm volatile ("rdtsc" : "=a"(lo), "=d"(hi));
    return ((unsigned long)hi << 32) | lo;
}
```

Read GCC's "Extended Asm" doc once. The kernel uses inline asm constantly for atomic ops, special instructions, and barriers.

16.10 Where C++ shines in our domain

- **Mesa**: Vulkan/OpenGL drivers; lots of C, modern C++ in the new bits (`mesa/anv`, `mesa/vulkan`).
- **ROCm runtime (ROCr/HSA)**: C++17 with C ABI on top.
- **HIP runtime / ROCclr**: C++ all the way down.
- **MIOpen, rocBLAS, RCCL**: C++ libraries.
- **MIGraphX**: C++ inference engine.
- **LLVM/Clang/AMDGPU backend**: heavy C++.

Get comfortable with each of: smart pointers, STL containers, RAII, template metaprogramming basics, lambdas. You don't need template wizardry, but you must be fluent in everyday modern C++.

16.11 Exercises

1. Take any C kernel header pattern (e.g. `struct file_operations`) and re-implement it in C++ as a `class` with virtual methods. Estimate the runtime cost per call vs. the function-pointer struct.
2. Take your `Vector<T>` from Chapter 15 and convert it to a PIMPL'd `class FixedVec` whose header doesn't expose `<vector>`.
3. Run `nm -D --defined-only libstdc++.so.6 | head`; pick a symbol; demangle it with `c++filt`.
4. Write a header library `result.h` that defines `template <class T, class E> class Result` similar to `std::expected`. Provide `bool ok()`, `T value()`, `E error()`.
5. Compile a tiny C++ file once with `-fno-exceptions` and once without. Compare binary sizes.

End of Part II. Move to Part III — Operating systems internals.

Chapter 17 — What an OS is and the boot path on x86_64

You cannot write a driver without a deep operational picture of the OS. This chapter gives that picture for Linux on x86_64 — and the AArch64 differences where they matter for AMD/Qualcomm engineers.

17.1 What "operating system" means

An OS does three jobs:

1. **Multiplex hardware** — many programs share one machine.
2. **Abstract** — programs deal with files, sockets, pipes; the OS hides disks, NICs, IPC.
3. **Protect** — programs cannot trash each other or the kernel.

A modern monolithic OS like Linux runs in two modes:

- **Kernel mode** (`CPL=0` on x86, `EL1` on AArch64): full privileges, can access any memory and any I/O.
- **User mode** (`CPL=3`, `EL0`): runs your apps; access mediated by page tables and the syscall interface.

A driver runs in kernel mode. A bug in a driver can crash the machine or corrupt anything. Hence the discipline.

17.2 The boot path on x86_64

When you press power:

1. **Reset vector**: CPU starts in 16-bit real mode at `0xFFFFFFF0`.
2. **UEFI firmware** (or legacy BIOS) runs from flash. Initializes RAM, PCIe, integrated devices. On modern UEFI machines, the firmware is in C, sometimes from EDK2.
3. **UEFI boot manager** loads the EFI System Partition's `\EFI\BOOT\BOOTX64.EFI` (or whatever's in `BootOrder`).
4. **GRUB** (or systemd-boot) loads the **Linux kernel** (`bzImage`) and **initrd** (initial RAM disk) into memory.
5. **Kernel decompressor** (`arch/x86/boot/`) runs in 32-bit, switches to 64-bit long mode, sets up initial page tables, jumps to `start_kernel`.
6. **`start_kernel()`** in `init/main.c` initializes everything: the scheduler, memory allocator, VFS, drivers, network stack.
7. **PID 1** (`/sbin/init` or `systemd`) starts the userspace. From here, multiplexing begins.

The driver lifecycle starts somewhere around step 6: PCI devices are enumerated; their drivers are matched and probed; `amdgpu.ko` (built-in or modular) eventually binds to the GPU.

The AArch64 boot is similar (UEFI on most servers, U-Boot on phones/embedded). On Qualcomm SoCs, the boot is more elaborate (PBL → SBL → AOP → ABL → kernel, with secure-world TEE involvement). You don't need to memorize this for AMD work, but if you target Qualcomm, learn the chain.

17.3 The kernel's view of memory

After boot, kernel virtual address space (x86_64, 4-level paging, 48-bit virt):

```
0xffff800000000000 – 0xffffbfffffffffff   direct map of physical RAM
0xffffc00000000000 – 0xffffe7ffffffffff   vmalloc / ioremap
0xffffea0000000000                          memmap (struct page array)
0xffffffff80000000                          kernel image (text, data, bss)
0xffffffffa0000000                          module load area
```

Userspace (per process):

```
0x0000000000400000   text
0x0000000001000000   heap (grows up via brk/mmap)
0x00007fffffffff000   stack base (grows down)
```

The line at `0x0000800000000000` and `0xffff800000000000` is enforced by hardware (canonical addresses). The bottom half is user, the top is kernel — every process has the kernel mapped, but with a privilege bit that makes it inaccessible from userspace.

17.4 Privilege rings, CPL, syscall

x86_64 protection rings: 0 (kernel) … 3 (user). Userspace runs at CPL=3. To enter the kernel:

- **`syscall`** instruction — by convention, syscall number in `rax`, args in `rdi, rsi, rdx, r10, r8, r9`.
- **Interrupt** — `int 0x80` legacy, or hardware IRQs.
- **Exception** — page fault, GPF, etc.

The CPU saves user state (RIP, RSP, RFLAGS), switches stacks (`%gs:cpu_current_top_of_stack`), and jumps to a fixed entry point set in `MSR_LSTAR`. The kernel runs the syscall handler, returns with `sysret`.

Page tables protect kernel memory: pages with the U/S bit clear are unreachable from CPL=3.

17.5 The big abstractions

These are what userspace sees:

- **Process** — `fork()` creates one; `exec()` replaces the image; `wait()` reaps.
- **Thread** — Linux models them as processes that share an address space; `clone(CLONE_VM | CLONE_FS | …)` is the primitive; `pthread_create` is glibc's wrapper.
- **File descriptor** — small integer that indexes into a per-process table of `struct file` pointers. `open`, `read`, `write`, `close`. **Almost everything is a file**: regular files, pipes, sockets, devices, GPUs (yes, `/dev/dri/card0` is a file).
- **Memory mapping** — `mmap` puts file content (or anonymous pages) into your VM. Used for shared memory, large allocations, and **GPU buffer sharing**.

- **Signals** — async notifications (SIGINT, SIGSEGV). Driver authors mostly ignore signals; userspace runtime authors must handle them.
- **Pipes / sockets** — IPC primitives.

The driver writer ultimately lives behind the *file descriptor* abstraction. A user opens `/dev/dri/card0`, gets an fd, calls `ioctl`s on it. We'll spend many chapters on those ioctls.

17.6 Modes of running drivers

- **Built-in**: compiled into `vmlinux`. Loaded at boot.
- **Module**: compiled as `*.ko`, loaded on demand by `modprobe` or by `udev` matching a device.
- **Out-of-tree module**: built against installed headers; common during development.

Modular vs. built-in is mostly an organizational choice. Behavior is the same.

17.7 The kernel-userspace contract: stable APIs

- The **syscall interface** is stable forever.
- The **/proc and /sys interfaces** are stable forever once exposed.
- **Kernel internal APIs** are NOT stable. They change every release. (This is intentional: contributors must update in-tree drivers when they change a core API; out-of-tree drivers must keep up.)
- **DRM ioctls** are stable forever once added. Adding a new one is a multi-list-review process.

Therefore: every `ioctl` you add to amdgpu is a forever commitment. Add them carefully and version them.

17.8 The major subsystems we will touch

```
                    ┌──────────────────────────────────────┐
                    │     User space (apps, libdrm, mesa)  │
                    └──────────────────────────────────────┘
                                    │ syscalls
┌───────────────────────────────────┼──────────────────────────────────┐
│                          Linux kernel                                │
│                                   │                                  │
│   VFS ── Page cache ── Block ── Net ── DRM ── Sound ── USB ── …      │
│                                   │                                  │
│            ┌──────────── Driver model (kobject, sysfs) ────────────┐│
│            │            ↓              ↓              ↓            │
│            │        PCI bus     Platform bus     USB bus           │
│            └──────────────────────────────────────────────────────┘│
│                                   │                                  │
│   Memory mgmt — Scheduler — IRQ — Locks — Kthreads — Workqueues      │
└──────────────────────────────────────────────────────────────────────┘
                                   │
                                   ▼
                              Hardware
```

Driver work touches almost every box. GPU work additionally touches `dma-buf` (cross-driver buffer sharing), `iommu`, `pci`, `drm/*`, and `power/runtime PM`.

17.9 Exercises

1. `strace ls` — observe every syscall. Identify `execve`, `mmap`, `openat`, `read`, `write`, `close`.
2. `cat /proc/cpuinfo`, `/proc/meminfo`, `/proc/interrupts`, `/proc/modules`. Read each.
3. Run `dmesg | head -200` after boot. Identify the lines from your storage controller, your network card, and (if AMD) `[drm] amdgpu`.
4. `lsmod | head` — count loaded modules. `modinfo nvme` and read.
5. Find your AMD GPU in sysfs: `ls /sys/class/drm/`. Walk the directory tree.

Next: Chapter 18 — Processes, threads, address spaces, syscalls.

Chapter 18 — Processes, threads, address spaces, syscalls

18.1 The Linux process

Every process is a `struct task_struct` (about 11000 bytes). It contains everything the kernel knows about a runnable thing: PID, state, priority, files table, address space, scheduling stats, signal info, ptrace info.

The PID namespace tree is rooted at PID 1 (init/systemd). PID 0 is the idle task.

Get a feel:

```
cat /proc/$$/status        # current shell's status
cat /proc/$$/maps          # virtual address layout
cat /proc/$$/fd            # open file descriptors
ps -eLo pid,tid,stat,comm  # threads (TIDs)
```

18.2 Processes vs. threads on Linux

In Linux, both are `struct task_struct`. The difference is what the threads share via `clone(2)` flags:

Flag	Shared
`CLONE_VM`	Address space
`CLONE_FS`	Filesystem info (cwd)
`CLONE_FILES`	File descriptors
`CLONE_SIGHAND`	Signal handlers
`CLONE_THREAD`	Same thread group (TGID = PID)

Two `task_struct`s sharing all of those = "two threads in one process." Two sharing none = two independent processes. Containers tweak this with namespaces.

`getpid()` returns the **TGID** (process id). `gettid()` returns the **PID** (thread id).

18.3 Address space (`mm_struct`)

Each process has a `struct mm_struct` describing its virtual memory. Threads of one process share the same `mm_struct`.

`mm_struct` contains:

- a tree of `vm_area_struct` (VMAs) — contiguous mapped regions,
- a pointer to the page tables (`pgd`),
- per-process counters (RSS, etc.).

A `VMA` describes one mapping: `[start, end)`, permissions (RWX), backing file (if any), flags (anonymous, shared, private). `mmap` creates a VMA. `mprotect` changes its perms.

`/proc/self/maps`:

```
55c5a8000000-55c5a8001000 r-xp 00000000 fd:00 12345 /usr/bin/cat
55c5a8001000-55c5a8002000 r--p 00001000 fd:00 12345 /usr/bin/cat
7f0000000000-7f0000021000 r--p 00000000 fd:00 67890 /lib/x86_64-linux-gnu/ld.so
7ffd1c000000-7ffd1c021000 rw-p 00000000 00:00 0     [stack]
```

In the kernel, you'll often look up which VMA contains a given user address — `find_vma(mm, addr)`.

For GPU drivers, `mmap` is the userspace handle into device memory. `mmap("/dev/dri/card0", offset)` asks the driver to install a VMA that, on page fault, populates from a GEM buffer (Chapter 45).

18.4 fork, exec, wait

```
pid_t pid = fork();
if (pid == 0) {
    /* child */
    execlp("ls", "ls", "-l", NULL);
    _exit(127);
} else if (pid > 0) {
    /* parent */
    int status;
    waitpid(pid, &status, 0);
}
```

`fork` duplicates the process with copy-on-write page tables. `execve` replaces the image. `wait` / `waitpid` reaps a child (without reaping, the child becomes a zombie).

For threads, `pthread_create` calls `clone()` directly with all the share flags.

18.5 The syscall mechanism on x86_64

User code:

```
mov  rax, 1            ; SYS_write
mov  rdi, 1            ; fd = stdout
mov  rsi, msg          ; buf
mov  rdx, msg_len      ; len
syscall
```

`syscall` reads `IA32_LSTAR` MSR (set at boot to `entry_SYSCALL_64`), saves user RIP/RSP/flags, switches to kernel stack via `swapgs` + `gs:`, and jumps. The kernel:

1. Validates the syscall number.
2. Dispatches to `sys_write` via the syscall table.
3. The handler validates user pointers (with `copy_from_user` / `copy_to_user`).
4. Returns; `sysretq` restores user state.

The full table is in `arch/x86/entry/syscalls/syscall_64.tbl`. The number for `write` is 1.

Calling convention is similar to the System V AMD64 user-space ABI but with `r10` instead of `rcx` (because `syscall` clobbers `rcx`).

18.6 ioctl: the catch-all syscall

```
int ioctl(int fd, unsigned long request, void *argp);
```

`ioctl` is a per-driver custom syscall. The driver defines a set of "request" numbers (encoded with `_IOR` / `_IOW` / `_IOWR` macros in `<asm-generic/ioctl.h>`) and a struct for each. We will see *thousands* of GPU ioctls (DRM defines a base set, amdgpu adds vendor-specific ones).

Example (a DRM ioctl):

```
#define DRM_IOCTL_AMDGPU_CS \
    DRM_IOWR(0x46, struct drm_amdgpu_cs_in)

struct drm_amdgpu_cs_in {
    uint32_t ctx_id;
    uint32_t bo_list_handle;
    uint32_t num_chunks;
    uint32_t flags;
    uint64_t chunks;       /* user pointer to array */
};
```

We'll write an entire `ioctl` in Chapter 36 and decode amdgpu ones in Chapter 49.

18.7 Signals

Async per-process notifications. Kernel writes to a queue; the next syscall return checks it; if a handler is registered, kernel synthesizes a frame on the user stack and invokes it.

You will rarely write signal-handling driver code. But you must know that signals can interrupt syscalls — `read` may return `-EINTR`. Userspace runtimes (HSA, HIP) deal with this on every blocking call.

Within a driver, on a wait, you should respect signals:

```
ret = wait_event_interruptible(queue, condition);
if (ret == -ERESTARTSYS) return -ERESTARTSYS;
```

`wait_event_interruptible` (vs. `wait_event`) allows a signal to break the wait. Most user-driven driver waits should be interruptible.

18.8 epoll/poll/select and async I/O

Userspace `poll(2)` / `epoll(7)` waits on multiple fds. The kernel side: every `struct file` whose fops define `.poll` (or `.poll_oneshot`) participates. Drivers implement `.poll` to expose readiness.

Modern: `io_uring` is a fd-and-rings interface for asynchronous I/O. As a driver author you might expose your queue through io_uring eventually, but DRM hasn't gone there yet — DRM uses fences for async.

18.9 Resource limits

`/proc/<pid>/limits`. `setrlimit/getrlimit`. Important for GPU work: `RLIMIT_MEMLOCK` and `RLIMIT_AS` because GPU drivers pin memory and create huge VMAs.

18.10 Exercises

1. Write a C program that `fork`s 5 children, each prints its PID, then exits. Parent waits for all.
2. Write a multi-threaded program (3 threads) that increments a shared counter 1e6 times each. Observe the race; we will fix it in Part IV.
3. Write a tiny syscall using inline assembly:

   ```
   long my_write(int fd, const void *buf, size_t n) {
       long ret;
       asm volatile("syscall"
                    : "=a"(ret)
                    : "0"(1), "D"(fd), "S"(buf), "d"(n)
                    : "rcx", "r11", "memory");
       return ret;
   }
   ```

   Use it to print “hello” without libc.
4. `strace -e ioctl -f -o trace.txt glxgears`; identify amdgpu/i915/nouveau ioctls.
5. Cat `/proc/<pid>/maps` for several processes; understand the layout.

Next: Chapter 19 — Virtual memory, paging, TLBs, MMUs.

Chapter 19 — Virtual memory, paging, TLBs, MMUs

GPU drivers live and die on memory. To write `amdgpu_vm.c` (the GPU's page tables) you must already have a perfect mental model of the CPU's page tables. We build it now.

19.1 Why virtual memory

Early machines had one program, one physical address space. Multitasking + protection requires that:

1. Two processes can both write to "address 0x1000" without conflict.
2. A process cannot read another's memory.
3. Memory can be over-committed (DRAM is finite).

Virtual memory solves all three. Every process sees its own virtual address space, mapped by hardware (the **MMU**) to physical memory through **page tables**.

19.2 Pages

Physical memory is divided into fixed-size **frames** (4 KiB on x86 by default; 16 KiB or 64 KiB possible on some ARM). Virtual memory is divided into matching **pages**.

The MMU's job: given a virtual address, look up the page table to find the physical frame.

Address format on x86_64 with 4 KiB pages, 4-level paging:

```
63                                                                  12 11          0
┌──────────────────────────────────────────────────────────────────────┬──────────────┐
│ canonical sign-extended bits | PML4(9) | PDPT(9) | PD(9) | PT(9)│  page offset │
└──────────────────────────────────────────────────────────────────────┴──────────────┘
```

The 48 low bits of address split into 4× 9-bit indices into 4 levels of tables, plus a 12-bit offset within the 4 KiB page. Each table is itself a page (4 KiB) of 512× 8-byte entries.

Modern x86 supports 5-level paging (`la57`, 57-bit virtual). Most systems still use 4-level.

19.3 A page table walk

```
                      virtual address
  ┌────┬────┬────┬────┬────┬──────┐
  │ ext│ L4 │ L3 │ L2 │ L1 │ off  │
  └────┴────┴────┴────┴────┴──────┘
          │    │    │    │
          ▼    ▼    ▼    ▼
       PML4  →PDPT  → PD  →  PT
                                │
                                ▼
                           physical frame
                                +
                             offset
```

Each table entry contains the physical address of the next-level table or the final frame, plus flag bits:

Bit	Name	Meaning
0	P	Present
1	R/W	Writable
2	U/S	User-accessible
3	PWT	Write-through caching
4	PCD	Cache disable
5	A	Accessed (set by HW)
6	D	Dirty (set by HW for last-level entries)
7	PS	Page size (1 = huge page at this level)
…		
63	XD	Execute disable

The kernel manipulates these bits. A driver rarely touches them directly, but `mmap` for device memory ends up flipping `PCD`, `PWT` to control caching.

19.4 Huge pages

Setting `PS=1` at L2 makes a 2 MiB page; at L3, 1 GiB. Reduces TLB pressure for big buffers. The kernel exposes them via `madvise(MADV_HUGEPAGE)` and transparent huge pages (THP).

GPU drivers use huge mappings on the GPU side (GPUVM) to represent big buffers efficiently. We will revisit in Chapter 51.

19.5 The TLB

The translation lookaside buffer is a hardware cache of recent virtual→physical translations. Without it, every memory access would do 4 (or 5) extra DRAM reads to walk the page table.

Sizes vary; on a Zen 4 core, dTLB has ~64 4 KiB entries (L1d) and 3 K (L2). TLB misses are *expensive* (200+ cycles). Cache-friendly data structures and huge pages help.

When the kernel changes a page-table entry, it must invalidate the affected TLB entries:

- `INVLPG addr` — flush one entry.
- `mov %cr3, %cr3` — flush all (except global pages).
- IPI to other cores — every CPU with the mapping must flush its TLB.

The IPI cost is why `munmap` of a shared mapping is one of the slowest things you can do in Linux.

19.6 The kernel’s page-table view

Every process has its own page tables, but the **upper half** of every process’s PML4 points to the same kernel page tables. So the kernel is mapped into every process at the same VAs.

Switching processes only requires updating the lower half (one CR3 reload). This is fast.

Newer Linux uses **KPTI** (Kernel Page Table Isolation, Meltdown mitigation) which actually swaps in a kernel-only page table on syscall entry. Cost: more TLB flushing.

19.7 Page faults

Every memory access goes through the MMU. If the entry is missing or doesn't have the right perms, the CPU generates a **page fault** exception (vector 14 on x86). The kernel handler:

1. Reads `CR2` (faulting virtual address) and the error code.
2. Finds the VMA covering the address.
3. If the VMA exists and the access is legal, allocates a page (or reads from a file) and installs the PTE.
4. If illegal, sends `SIGSEGV` to the process.

GPU drivers piggyback on this for **demand-faulted GPU buffers**. A user `mmap`s a device handle; when the user touches the buffer, a page fault enters the driver's `vm_ops->fault` callback, which allocates and inserts the page. We will write one in Chapter 36.

19.8 Cache attributes

Each PTE has bits controlling caching for that page. Combined with the **PAT** (Page Attribute Table) the OS can pick:

- **WB** (write-back) — default, fully cached.
- **WT** (write-through) — write-through cache.
- **UC** (uncached) — every access goes to DRAM/MMIO.
- **WC** (write-combining) — uncached but writes coalesce in a small buffer; great for streaming write-only mappings (frame buffers).

For MMIO (device registers), the kernel uses UC. For framebuffers and GPU command buffers being written by the CPU to be read by the GPU, WC. `ioremap()` chooses; `ioremap_wc()` forces WC.

The wrong cache attribute on a GPU buffer will silently corrupt data (the CPU thinks the cache is in sync; the GPU sees stale memory). This is the most insidious driver bug.

19.9 The page allocator

The kernel's lowest-level memory allocator is the **buddy allocator**:

- Free pages are kept in `MAX_ORDER` lists, indexed by power-of-two size (1, 2, 4, …, 1024 pages).
- Allocate by splitting a higher-order block; free by coalescing buddies.

Above it: **slab/SLUB** for fixed-size objects, **kmalloc** for variable, **vmalloc** for non-contiguous virtually-mapped memory. We treat each in Part V.

GPU memory has its own allocator (TTM in amdgpu, Chapter 46). It mirrors the CPU's buddy idea on the device side.

19.10 Reverse mappings and `struct page`

Every physical page has a corresponding `struct page` in a giant array (`mem_map`). This struct stores: refcount, mapcount, flags, owner. Drivers that pin pages (for DMA) frequently use `struct page *` — `get_user_pages_fast()` returns them.

To translate kernel virtual → physical: `__pa(addr)`. Reverse: `__va(paddr)` (works for the direct-mapped region only).

To translate kernel virtual → `struct page *`: `virt_to_page(addr)` (direct-map). For vmalloc'd: `vmalloc_to_page(addr)`.

These helpers are everywhere in DMA code.

19.11 NUMA briefly

On multi-socket boxes (and modern AMD chiplet designs), DRAM is split into **nodes**. Accessing memory on a remote node is slower. The kernel tries to keep allocations local and offers `numa_alloc_local` / `set_mempolicy`.

For GPUs on a multi-socket host, you allocate DMA memory close to the device's PCIe root complex. amdgpu does this via `dev_to_node(dev)`.

19.12 IOMMUs

The IOMMU is the MMU for devices. It maps the device's view of memory ("DMA addresses" or "IOVA") to physical RAM. Without IOMMU, a malicious or buggy device could DMA anywhere. With IOMMU:

- Device sees only what the kernel maps (`dma_map_*`).
- Each device (or device group) can have its own IO page table.
- Protection at the PCIe level.

AMD's IOMMU is "AMD-Vi"; Intel's is VT-d. ARM uses "SMMU." All implement the same idea.

GPU drivers must be IOMMU-aware: `dma_map_*` returns IOVA; the GPU is programmed with IOVA, not physical. The IOMMU translates back to physical when the GPU does DMA.

19.13 Exercises

1. Write a userspace program that does `mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_ANONYMOUS|MAP_PRIVATE, -1, 0)`; read `/proc/self/maps` to see the new VMA.
2. `cat /proc/meminfo`. Identify `MemTotal`, `MemAvailable`, `Slab`, `KReclaimable`, `HugePages_Total`.
3. Write a microbenchmark that measures the cost of a TLB miss: stride through a 1 GiB array in 4 KiB steps. Compare to a 1 MiB array.
4. Read `Documentation/x86/x86_64/mm.rst` from the kernel source. Identify each address-range comment.
5. Read `Documentation/admin-guide/mm/concepts.rst` for the kernel's MM overview.

Next: Chapter 20 — Scheduling — CFS, deadline, real-time.

Chapter 20 — Scheduling: CFS, deadline, real-time

The kernel scheduler decides which task runs on which CPU and for how long. Drivers don't *implement* scheduling, but driver code that runs in process context (your `read`, `write`, `ioctl`) must understand:

- when it can sleep,
- when it must not,
- how long it might block,
- preemption.

20.1 Scheduling classes

Linux has multiple scheduling classes, sorted by priority (highest first):

1. **Stop** — kernel-internal; pre-empts everything.
2. **Deadline (`SCHED_DEADLINE`)** — real-time; tasks declare runtime/deadline/period; EDF + CBS.
3. **Real-time (`SCHED_FIFO`, `SCHED_RR`)** — fixed-priority; preempts CFS.
4. **CFS (`SCHED_NORMAL` / `SCHED_BATCH` / `SCHED_IDLE`)** — fair scheduler; default.
5. **Idle** — runs only when nothing else can.

For driver work, almost everything you create is `SCHED_NORMAL`. Some IRQ threads become `SCHED_FIFO`. GPU schedulers use kthreads at default priority.

20.2 The CFS algorithm

CFS = Completely Fair Scheduler. Tracks per-task **vruntime** (virtual runtime: real runtime weighted by nice value). The runqueue is a red-black tree keyed by `vruntime`. Pick the leftmost. Run it for a slice; resume.

Nice values from -20 (highest priority) to +19 (lowest) scale the rate at which `vruntime` grows. A nice -10 task accumulates `vruntime` ~10x slower than nice 0, so it runs 10x more often.

You won't need to modify CFS. You need to know: **fair, dynamic, no starvation, no fixed priority among `SCHED_NORMAL` tasks**.

20.3 Preemption

Linux is *preemptible*: when a higher-priority task wakes, the running task can be kicked off. Configurable:

- `CONFIG_PREEMPT_NONE` — never preempt in kernel mode (servers).
- `CONFIG_PREEMPT_VOLUNTARY` — preempt at explicit `cond_resched()` points.
- `CONFIG_PREEMPT` — preempt almost anywhere.

- `CONFIG_PREEMPT_RT` — fully preemptible kernel for RT use.

Most desktop distros use `CONFIG_PREEMPT`. Server distros use `CONFIG_PREEMPT_NONE`. As a driver author, **assume `CONFIG_PREEMPT_RT` is possible** — your code must use locks correctly even when interrupted at the worst moments.

20.4 Blocking, sleeping, scheduling

Inside a syscall, your driver code can:

- **Run** (CPU-bound).
- **Sleep** waiting for an event (preferred).
- **Spin** waiting for hardware (only for very short waits).
- **Yield** with `cond_resched()` for long CPU-bound loops.

To sleep on an event, kernel pattern:

```
DECLARE_WAITQUEUE(wait, current);

add_wait_queue(&dev->wq, &wait);
for (;;) {
    set_current_state(TASK_INTERRUPTIBLE);
    if (cond) break;
    if (signal_pending(current)) { ret = -ERESTARTSYS; break; }
    schedule();
}
__set_current_state(TASK_RUNNING);
remove_wait_queue(&dev->wq, &wait);
```

Most code uses the `wait_event_interruptible*` macros that wrap that pattern.

`schedule()` is the call that voluntarily gives up the CPU.

20.5 Atomic context

Some kernel code runs **atomically** — it cannot sleep. If you sleep there, the system will deadlock or bug. The contexts:

- **Hard IRQ handler** (top half).
- **Softirq / tasklet**.
- **Holding a spinlock**.
- **`preempt_disable()`** region.
- **RCU read-side critical section** (cannot block).

In atomic context:

- No `kmalloc(GFP_KERNEL)` (use `GFP_ATOMIC`).
- No mutexes (use spinlocks).
- No `copy_from_user` / `copy_to_user` (they may page-fault and sleep).
- No `msleep` / `schedule` — only `udelay` / `mdelay` (busy-wait).
- No userspace access of any kind.

Driver writers learn this distinction by debugging the first time they put `mutex_lock` in an interrupt handler and crash.

20.6 Affinity, CPU pinning, isolation

```
        sched_setaffinity(pid, sizeof(mask), &mask);
```

Pins a thread to specific CPUs. For latency-critical work (GPU command submission threads), pinning helps cache locality. Boot parameter `isolcpus=2-7` removes CPUs from CFS, letting userspace pin RT work to them.

amdgpu's scheduler kthreads are not pinned by default but can be tuned by the kernel scheduler.

20.7 IRQ threads and kthreads

The kernel can route hardware interrupts to a dedicated **kthread** (rather than running them in hard IRQ context). This lets the handler sleep, take mutexes, and be preempted. Achieved with `request_threaded_irq()`. We will use this pattern in Chapter 22.

A **kthread** is a kernel-only "thread" — `kthread_create`, `kthread_run`, `kthread_stop`. amdgpu has *many* kthreads: command-submission worker, suspend/resume helper, RAS handler.

20.8 Latency budgets

For realtime audio: < 1 ms per buffer. For graphics frame: 16.6 ms (60 Hz). For GPU command submission overhead: < 100 µs from ioctl entry to job queued.

Driver code in the user-syscall path must keep latency in mind. Holding a mutex too long blocks all callers. Doing work in hard IRQ delays everyone.

20.9 The `current` macro and `task_struct`

`current` returns a `struct task_struct *` for the currently-running task. From it you can read:

```
        current->pid, current->comm, current->mm,
        current->files, current->signal,
        current->cred->uid, current->prio
```

But: many fields require locks; not everything in `task_struct` is safe to read from any context. Stick to the obvious fields.

20.10 Exercises

1. Read `Documentation/scheduler/sched-design-CFS.rst`.
2. Run `chrt -f 50 ./your_program` to launch with `SCHED_FIFO` priority 50. (Requires CAP_SYS_NICE.) Observe with `top -p $(pidof your_program)`.
3. Write a kernel module that creates a kthread that prints "tick" every second.
4. In your kthread, deliberately call `mutex_lock` then `msleep` to verify it works in a sleeping context. Then put the same code in a tasklet — observe the BUG.

5. Read `kernel/sched/fair.c::pick_next_task_fair` (a few hundred lines of dense code). Don't try to understand all of it. Recognize the rb-tree traversal.

Next: Chapter 21 — The I/O stack — VFS, page cache, block layer.

Chapter 21 — The I/O stack: VFS, page cache, block layer

For a GPU driver writer this chapter is short — your driver isn't a filesystem. But you must know the shape of the stack because you live above the **VFS** abstraction (you implement `file_operations`), and because GPU buffers occasionally cross the line into mmap-able files via `dma-buf` and `udmabuf`.

21.1 The VFS

The **Virtual File System** is the kernel's polymorphic interface to "things you can read/write." Every file-like object — disk file, pipe, socket, device, GPU — is a `struct file` whose `f_op` is a `struct file_operations`. The kernel implements `read(2)` / `write(2)` / `mmap(2)` by dispatching through `f_op`.

```
struct file {
    struct path                 f_path;
    struct inode               *f_inode;
    const struct file_operations *f_op;
    void                       *private_data;     /* driver's per-fd state */
    fmode_t                     f_mode;
    loff_t                      f_pos;
    /* … */
};
```

When userspace does `open("/dev/dri/card0", O_RDWR)`, the VFS:

1. Resolves the path; finds an inode that says "char device, major 226, minor 0."
2. Looks up the registered char-device class for that major; gets `drm_fops` (DRM's `file_operations`).
3. Creates a `struct file`, sets `f_op = &drm_fops`, calls `drm_fops.open`.
4. The DRM open allocates a per-file structure and stores it in `file->private_data`.

From this point, `read(fd, …)` calls `drm_fops.read` — DRM's read. `ioctl(fd, …)` calls `drm_fops.unlocked_ioctl`. The driver's per-fd state is in `file->private_data`.

Implementing a driver = implementing a `file_operations`. Internalize this.

21.2 The page cache

For real files on disk, the kernel maintains a **page cache**: in-RAM copies of file content keyed by `(inode, offset)`. Reads read from the cache (or fault them in from disk). Writes update the cache and mark dirty; a writeback path eventually flushes to disk.

`mmap` on a file uses the page cache directly: the user's pages *are* the page cache pages. This is why `mmap` + `read` of the same file shares data automatically.

For drivers, the page cache is mostly invisible. But:

- Some drivers expose pseudo-files in `debugfs` / `sysfs`. Those don't use the page cache.

- `dma-buf` and `udmabuf` create file-backed mmap-able objects with custom `vm_ops->fault`. The kernel page-cache machinery touches them (briefly).

21.3 Block layer (briefly)

Below the filesystem is the **block layer**: requests to read/write 4 KiB sectors. Modern: **blk-mq** (multi-queue block layer). Drivers register a `blk_mq_ops` and a request queue; the layer delivers requests to be dispatched to hardware.

NVMe drivers, SCSI, virtio-blk all live here. amdgpu does not. Skip unless you join the storage team.

21.4 Network stack (briefer)

Sockets → protocol stack (TCP/IP) → driver. A NIC driver registers with `netdev`. amdgpu does not implement netdev. Skip.

21.5 The `inode` and the `dentry`

`struct inode` is the kernel's view of a file (size, perms, owner, link count). `struct dentry` is the path-lookup cache (one entry per path component). For drivers, you mostly don't allocate inodes (the device-class machinery does). But understand: `device_create()` plants an inode in `/dev`.

21.6 sysfs and debugfs

These are *kernel virtual filesystems* — not backed by disk, but by callback functions in the kernel.

- **sysfs (`/sys`)**: structured tree of devices, classes, drivers. Each entry corresponds to a `kobject`. Drivers expose attributes via macros like `DEVICE_ATTR_RW(name)` and a show/store pair.
- **debugfs (`/sys/kernel/debug`)**: ad-hoc developer files. amdgpu uses heavily for debugging registers, ring queues, fences. Drivers register a debugfs node with `debugfs_create_file()` and a `file_operations`.

For a driver, sysfs is the user-visible knob; debugfs is the developer's window.

21.7 `procfs` (`/proc`)

Older sibling of sysfs. Drivers shouldn't add new `/proc` entries — use sysfs/debugfs. But you'll read `/proc/interrupts`, `/proc/iomem`, `/proc/meminfo` constantly.

21.8 The driver-author's view of the I/O stack

Every driver is a **producer of `file_operations`** and a **consumer of bus, IRQ, DMA, sysfs, devm APIs**:

```
              ┌──────────────────────┐
              │   open / read / mmap │   userspace
```

```
                    └──────────────────────┘
                                │
                                │ syscall
                    ┌───────────▼──────────┐
                    │          VFS         │
                    └──────────────────────┘
                                │
                                │ f_op->read, etc.
                    ┌───────────▼──────────┐
                    │ Char/DRM driver      │   <-- you implement this
                    │  - parses request    │
                    │  - touches MMIO      │
                    │  - schedules DMA     │
                    │  - waits on IRQ/fence│
                    └──────────────────────┘
                                │
                                ▼
                            Hardware
```

Everything else in this textbook is preparation to write that middle box well.

21.9 Exercises

1. `cat /proc/devices` — see registered char/block majors.
2. Read `Documentation/filesystems/vfs.rst` (a long, dry but excellent doc).
3. List your DRM device tree under `/sys/class/drm/`. Walk `card0/device/` (a symlink into `/sys/devices/pci…/`). Identify `vendor`, `device`, `boot_vga`, `power/`.
4. Run `mount | grep debugfs`; if it isn't mounted, mount it: `sudo mount -t debugfs none /sys/kernel/debug`. List `/sys/kernel/debug/dri/0/`.
5. Read three files in `/sys/kernel/debug/dri/0/`: `amdgpu_pm_info`, `amdgpu_gpu_recover`, `name`. We will revisit these.

Next: Chapter 22 — Interrupts, exceptions, IRQ delivery.

Chapter 22 — Interrupts, exceptions, IRQ delivery

A driver without interrupts is a polling driver — slow, CPU-hogging, fragile. Real hardware *interrupts the CPU* when something happens (DMA done, frame finished, error). This chapter covers the interrupt architecture from silicon up to your `irq_handler_t`.

22.1 What an interrupt is

An interrupt is the CPU saying: "stop what you're doing, jump to a handler, then resume." There are three flavors:

1. **Synchronous exceptions / faults** — caused by execution: page fault, division by zero, GPF.
2. **Hardware interrupts** — async, from a device.
3. **Software interrupts (IPIs)** — from another CPU.

We focus on (2). On x86_64:

- Each interrupt has a vector 0–255.
- A table (the **IDT**) maps vector → handler.
- The CPU pushes RIP/CS/RFLAGS, switches to the kernel stack (via IST or normal), jumps.

22.2 Legacy and modern interrupt delivery

Three eras of interrupt routing on PCs:

1. **PIC / 8259** — original, 16 IRQ lines. Obsolete except for emulated.
2. **APIC / IOAPIC** — per-CPU local APIC + chipset I/O APIC. Supports many vectors and per-CPU delivery.
3. **MSI / MSI-X** — Message-Signaled Interrupts. The device, instead of toggling a wire, **writes** a value to a special memory address that triggers an interrupt. The address encodes "which CPU + which vector"; the data is the vector.

PCIe devices today use MSI/MSI-X. amdgpu requests MSI-X with multiple vectors so different GPU rings can deliver to different CPUs.

```
        ret = pci_alloc_irq_vectors(pdev, 1, max, PCI_IRQ_MSIX | PCI_IRQ_MSI);
```

This is the modern API.

22.3 The kernel's IRQ subsystem

Each device IRQ has a `struct irq_desc` with:

- a chip (the controller it belongs to: `apic_chip`, `intel_ir_chip`, `amd_iommu_chip`),

- a list of action handlers,
- statistics, affinity, etc.

You request an IRQ:

```
        ret = request_irq(irq, my_isr, IRQF_SHARED, "mydrv", dev);
```

On free:

```
        free_irq(irq, dev);
```

The cookie (`dev`) is passed to your handler so you can find your driver's state.

22.4 The handler

```
        static irqreturn_t my_isr(int irq, void *cookie)
        {
            struct mydrv *m = cookie;
            u32 status;

            status = readl(m->regs + REG_INT_STATUS);
            if (!(status & INT_BITS_OF_INTEREST))
                return IRQ_NONE;          /* not ours */

            writel(status, m->regs + REG_INT_ACK);

            /* schedule deferred work; do little here */
            schedule_work(&m->work);
            return IRQ_HANDLED;
        }
```

Rules of an ISR (top half):

- **Cannot sleep**, ever.
- **Must be fast** (microseconds).
- Should detect "is this even our interrupt?" first if shared.
- Acknowledge the device.
- Defer real work to a softirq, tasklet, workqueue, or threaded IRQ.

Returning `IRQ_NONE` from a non-shared IRQ is a bug; from a shared one, it's normal when the IRQ wasn't ours.

22.5 Top half and bottom half

The classic kernel pattern: short top half (hard IRQ context) ACKs and queues; bottom half does the heavy lifting in a context that can take locks/sleep.

Bottom-half mechanisms:

- **Softirq** — built-in, fixed slots (NET_RX, BLOCK, etc.). Drivers don't add new ones.
- **Tasklet** — softirq-based, easy API. Atomic context. Deprecated in newer kernels in favor of workqueues.
- **Workqueue** — runs in a kthread, can sleep. Default for drivers.
- **Threaded IRQ** — request_threaded_irq with primary handler + thread function.

For most drivers today, **threaded IRQ + workqueue** is the right answer.

```
        ret = request_threaded_irq(irq,
                                   my_isr_quick,    /* hardirq */
                                   my_isr_thread,   /* threaded */
                                   IRQF_SHARED, "mydrv", dev);
```

`my_isr_quick` runs in hardirq, ACKs the device, returns `IRQ_WAKE_THREAD`. `my_isr_thread` runs in a kthread, can sleep.

22.6 amdgpu interrupt flow

amdgpu has many sources: vertical-blank, command-submission completion, SDMA, RAS errors, page-fault, IH (Interrupt Handler) ring.

The hardware writes interrupt cookies into the **IH ring** — a circular buffer in system memory. amdgpu's hardirq handler looks at the IH ring head/tail, copies entries, schedules a tasklet/work to process them. This avoids needing one IRQ per event.

You'll read this in `drivers/gpu/drm/amd/amdgpu/amdgpu_irq.c` once we get to Part VII.

22.7 Affinity

`/proc/irq/<n>/smp_affinity` lets the user pin an IRQ to a CPU set. For RT systems and high-throughput servers, irq affinity is critical. NVMe pins each MSI-X to a different CPU.

Driver code can hint via `pci_irq_get_affinity()`.

22.8 NMI and watchdogs

The Non-Maskable Interrupt is reserved for things that can't be ignored: hardware errors, watchdog timeouts. The kernel's hardlockup detector uses an NMI to detect when a CPU is stuck for too long. As a driver author you'll see NMI traces in the dmesg when the kernel panics on lockup.

22.9 IPIs

Inter-Processor Interrupts: a CPU sends an interrupt to another CPU. Used for TLB shootdown, kicking idle CPUs, RCU. Drivers rarely use IPIs directly — `smp_call_function*` wraps it.

22.10 Receiving interrupts in atomic vs. process context

One of the most common bugs: holding a regular spinlock when an interrupt fires that *also* takes the spinlock. Deadlock.

Fix: use **`spin_lock_irqsave`** when the same lock might be taken from IRQ context. Disables IRQs while held. Pair with `spin_unlock_irqrestore`. Not free; only when needed.

We'll spend a chapter on locks in Part V.

22.11 Polling fallback

When a chip's interrupt is broken (firmware bug, unsupported chip), drivers fall back to polling. amdgpu has knobs to disable certain interrupts and poll instead. Useful for triage.

22.12 Exercises

1. `cat /proc/interrupts` — note the columns: per-CPU counts. Observe which IRQs land on which cores. Find your GPU's IRQ.
2. Read `Documentation/core-api/genericirq.rst`.
3. Read `kernel/irq/manage.c::request_threaded_irq`.
4. In a kernel module, request an IRQ for a device you actually have (e.g. a serial port at IRQ 4 on QEMU). Print "irq fired" with `pr_info`. Free it on unload.
5. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_irq.c::amdgpu_irq_handler` once. Just orient yourself — we'll come back.

End of Part III. Move to Part IV — Multithreading & concurrency.

Part OST — Classic OS Theory (the academic foundation)

> **Stage 8 of the LEARNING PATH. 4–6 weeks at 1 hr/day, in parallel with Part III.**

You've done multi-threading and userspace systems. Now: **the academic theory that underlies every OS.**

This part complements Part III — OS Internals (which is Linux-specific). Part III tells you *how Linux works*; Part OST tells you *why it works that way and what alternatives exist.* Together they make you OS-fluent.

Why this part exists

If you walk into a kernel interview and say "I know Linux," good. If you can also discuss:

- "Why does CFS use vruntime instead of priority?"
- "What's the trade-off between MLFQ and lottery scheduling?"
- "When does LRU lose to ARC?"
- "What's the difference between strict 2PL and snapshot isolation?"

…then you sound like a senior engineer. **That's the goal of this part.**

The classic OS topics

Most OS textbooks (Tanenbaum, Silberschatz, OSTEP, Stallings) cover the same eternal topics:

1. Process model & threading.
2. Scheduling.
3. Synchronization (semaphores, monitors, lock-free).
4. Deadlock.
5. Memory management (paging, segmentation, swapping).
6. File systems & journaling.
7. I/O subsystem.
8. Distributed / concurrency basics.
9. Security & isolation.
10. Virtualization.

We do them at the right level for a kernel engineer (not too academic, not too cookbook).

Table of contents

#	Chapter
1	Process model — formal view

#	Chapter
2	CPU scheduling theory
3	Synchronization theory — Peterson, semaphores, monitors
4	Classic synchronization problems
5	Deadlock — prevention, avoidance, detection
6	Memory theory — paging, segmentation, swapping
7	Page replacement algorithms
8	File system theory & journaling
9	Crash consistency & write ordering
10	Distributed systems intro — consensus, RPC, time

Recommended companion text

- **OSTEP** (Operating Systems: Three Easy Pieces) by Remzi & Andrea Arpaci-Dusseau — **free PDF online**, beautifully written. The single best modern OS textbook. Read it alongside this part.
- *Modern Operating Systems* by Tanenbaum — encyclopedic.
- *Operating Systems: Internals and Design Principles* by Stallings — heavy on theory.

This Part OST is your **summary + Linux mapping**; OSTEP is your deeper read.

How this part fits

By the time you finish:

- You know **CFS** is one approach (Linux's). You know about MLFQ, lottery, deadline, EDF — and you can argue why CFS chose what it did.
- You know **Linux uses LRU** in the page cache (with twist). You know about ARC, 2Q, LFU; the trade-offs.
- You know **ext4 journals**. You know there are crash-consistency techniques (soft updates, COW like btrfs, WAL like databases).
- You know **TCP** is a reliable transport. You know the CAP theorem and why distributed consensus is hard.

This is what separates a "competent junior" from a "thinks at the level of architecture."

→ OST-01 — Process model: formal view.

OST 1 — Process model: the formal view

> The OS abstracts the CPU as "many processes time-sharing one or more CPUs." Here's the formal version.

The illusion

A modern computer has, say, 16 CPUs. Yet we run hundreds of programs **simultaneously**. The OS provides this illusion by **time-sharing** (multiplexing) and **multiplexing across cores**.

Two enabling abstractions:

- **Process**: an executing program with private memory, fds, etc.
- **Thread**: a separately schedulable execution within a process.

Process control block (PCB)

The kernel keeps a per-process **control block** with:

- Process ID, parent PID.
- Process state (Running, Ready, Blocked, Zombie, ...).
- Saved CPU registers (when not running).
- Memory map (page tables / VMAs).
- File descriptor table.
- Signal handlers + masks.
- Credentials (uid, gid).
- Accounting info (CPU time, peak memory).

Linux's PCB: `struct task_struct` in `include/linux/sched.h`. It's **massive** (3000+ lines for the struct + bits scattered). The OS books call it the "PCB"; in Linux it's `task_struct`.

Process state diagram

```
      New
       |
       v
+----------+   schedule       +-----------+
|  Ready   | <------------->  | Running   |
+----------+   preempt        +-----------+
    ^                               |
    |  wake                         v block / wait
    |                          +-----------+
    +------------------------- |  Blocked  |
                               +-----------+
                                    |
                               exit |
```

```
                                        v
                                   Zombie -> reaped -> Dead
```

A process is **always in exactly one state**. The kernel's scheduler moves it.

In Linux: `R` (running), `S` (sleep, interruptible), `D` (sleep, uninterruptible), `T` (stopped), `Z` (zombie).

Context switch

The OS pauses one process and runs another. Cost:

- Save old registers to PCB (all general regs, FP regs, page table base).
- Restore new registers.
- TLB flush (or partial — modern CPUs use ASIDs/PCIDs to avoid).
- Cache pollution as new process's data displaces old.

Cost: a few microseconds typically. **Not zero.** Excessive context switching is a real perf problem.

Process creation models

Different OSes use different primitives:

- **fork + exec** (Unix). Two-step, flexible.
- **CreateProcess** (Windows). Single-step, gives a fully constructed new process from scratch.
- **spawn (POSIX)**. Compromise; slowly replacing fork in many embedded systems.

`fork` is convenient but can be **expensive** for large processes (copy-on-write helps, but page table copying is real). On embedded with no MMU, fork is even harder.

`vfork` (mostly historical) — share parent's address space until exec. Dangerous, sometimes used for performance.

Threads — finer granularity

Two main models:

- **1:1** — every userspace thread is a kernel thread (Linux NPTL, Windows). Best.
- **N:1** — many user threads on one kernel thread (old "green threads"). Limited.
- **M:N** — pooling user threads onto kernel threads (Go's goroutines, Erlang processes). Best for huge concurrency.

Linux's NPTL pthreads is 1:1. The kernel scheduler sees every thread.

Go's runtime is M:N: millions of goroutines on a few kernel threads. Userspace scheduler atop kernel threads. Also true of Rust's `tokio`, Java's project Loom, etc.

Process vs thread vs fiber

- **Process**: separate address space, OS-scheduled.
- **Thread**: shared address space, OS-scheduled.

- **Fiber / coroutine**: shared address space, **userspace**-scheduled (no kernel involvement). Cheap (~stack size), bigger pool. Used in async runtimes.

A modern app may have N processes × M threads × K fibers.

Birth and death

- `fork`: copy yourself.
- `exec`: replace your code.
- `exit`: I'm done.
- `wait`: collect a child's exit status.

These four are the entire process model in Unix.

Resource isolation: namespaces (preview)

In Linux, even processes share the same kernel. To **isolate**, use namespaces:

- **PID namespace**: each container sees its own PID 1.
- **Mount namespace**: each container sees its own filesystem.
- **Network namespace**: each container sees its own NICs.
- **User namespace**: map UIDs.
- **IPC namespace**: separate POSIX/SysV IPC.
- **UTS namespace**: separate hostname.
- **Cgroup namespace**: hide cgroup hierarchy.

Containers (Docker, podman) = process + namespaces + cgroups (resource caps).

CPU scheduling preview

Once a process is **Ready**, the scheduler picks the next one to run. Goals:

- **Fairness** — every process gets some CPU.
- **Throughput** — total work done per unit time.
- **Latency** — interactive responsiveness.
- **Priority** — important tasks first.
- **Affinity** — keep tasks on cache-warm CPUs.
- **Power** — don't wake idle cores unnecessarily.

These goals **conflict.** Real schedulers compromise.

Next chapter dives in.

Linux specifics

- Every thread has a `task_struct`. Linux schedules at thread granularity, not process.
- A process is just "a group of tasks sharing memory, fds, etc." It's `task->mm` shared.
- Cgroup v2 controls CPU, memory, IO, devices.
- The `clone()` syscall is what `fork()`, `pthread_create()`, and `unshare()` are all built on.

Read `man 2 clone` for a glimpse of how flexible Linux's process model is.

Try it

1. Read `man 2 clone`. Look at the `CLONE_*` flags. Identify which combination = fork, which = pthread_create.
2. Read `/proc/PID/status` and `/proc/PID/stat`. Identify "state" character.
3. Check context-switch cost: write a benchmark using two threads pingpong'ing via futex; measure latency.
4. Read 2 chapters of OSTEP: "The Process" and "Process API."
5. Run a CPU-bound program. Watch in `top` how its state stays R. Run a sleep'ing program. Watch S/D states.

Read deeper

- OSTEP chapters 4–6.
- *Modern Operating Systems* (Tanenbaum) chapter 2.
- `Documentation/scheduler/` in the kernel.

Next → CPU scheduling theory.

OST 2 — CPU scheduling theory

> Many algorithms exist. Each prioritizes different goals. Linux's CFS is one branch on this tree.

The scheduling problem

Given: - a set of processes (or threads) waiting to run, - one or more CPUs, - some constraints (priorities, deadlines, fairness),

decide **which one runs next**, and **for how long**.

Metrics

Different schedulers optimize different metrics:

- **Throughput**: jobs completed per unit time.
- **Turnaround time**: completion time − arrival time.
- **Response time**: first-run time − arrival time.
- **Waiting time**: time spent in ready queue.
- **Utilization**: % of CPU busy.
- **Fairness**: how evenly time is shared.

A scheduler can't be best on all metrics. It picks a workload assumption.

Classic algorithms

FCFS (First-Come-First-Served)

Just a FIFO queue. Simple. **Convoy effect**: a long job blocks short jobs behind it. Bad for response time.

SJF (Shortest Job First)

Run the shortest available job. Optimal for average waiting time. Requires knowing burst length (oracle), so impractical for general scheduling. Used as a model for analysis.

SRTF (Shortest Remaining Time First) — preemptive SJF

If a new job has shorter remaining time than the current one, switch. Optimal in theory; impractical (oracle problem).

Round-Robin (RR)

Each ready job gets a fixed **time slice** (e.g., 10 ms). Then preempt and rotate. Fair; good response time. Tuning quantum is critical: too small → too many context switches; too big → poor latency.

Priority scheduling

Each job has a priority; highest priority runs. Possible **starvation** of low-priority jobs. Mitigation: **aging** (raise priority over time).

MLFQ (Multilevel Feedback Queue)

Multiple queues with different priorities. New jobs start at top; if they use full quantum, demoted. If they yield (I/O bound), kept high. Approximates SJF without oracle.

Used in Solaris, classic Unix BSD/SVR4. Good general-purpose.

Lottery scheduling

Each process holds "tickets." Scheduler picks a random ticket; that process runs. Probabilistic fairness, easy to implement, no aging needed.

Not used in real OSes (probabilistic fairness is hard to debug), but academically beautiful.

Stride scheduling

Deterministic version of lottery. Each process accumulates a "pass" with stride proportional to weight; lowest pass runs.

Fair-share / Proportional share

Allocate CPU **per user / per group**, not per process. Cgroups in Linux do this.

CFS (Completely Fair Scheduler)

Linux's default. Each task tracks **virtual runtime** (`vruntime`); the scheduler always runs the task with **least vruntime**. Effectively keeps everyone "equally behind." Implemented as an RB-tree on vruntime.

Weighted by `nice` value (priority): low-nice tasks accumulate vruntime slower. Result: fair sharing scaled by priority.

`Documentation/scheduler/sched-design-CFS.rst`.

EEVDF (Earliest Eligible Virtual Deadline First)

Newer Linux scheduler (replacing CFS in 6.6+). Improves on CFS for low-latency interactive workloads. More principled approach to wake-up balance.

Deadline scheduling (EDF, RM)

For real-time tasks with deadlines: - **EDF (Earliest Deadline First)**: highest priority = nearest deadline. - **RMS (Rate Monotonic)**: shorter period → higher priority. Static priorities.

Used in real-time kernels and Linux's `SCHED_DEADLINE`.

Linux's scheduling classes

```
SCHED_DEADLINE   (highest)  — deadline scheduler, real-time hard
SCHED_FIFO                  — real-time FIFO, run until yield
SCHED_RR                    — real-time RR, time-sliced
SCHED_NORMAL    (CFS)       — default, fair share
```

```
    SCHED_BATCH                 — slightly less interactive bias
    SCHED_IDLE      (lowest)    — only when nothing else
```

A task has a class. Within `SCHED_NORMAL`, CFS (or EEVDF). Real-time always preempts normal.

Multi-CPU scheduling

Per-CPU run queues. **Load balancing** moves tasks between CPUs to keep all busy. Trade-off: cache locality (don't move) vs balance (do move).

- **Affinity** (`sched_setaffinity`): pin a task to specific CPUs. Used for performance-critical code.
- **NUMA**: prefer local memory. Linux's NUMA balancer migrates pages to where threads run.

GPU work submission and isr handlers benefit from affinity. We'll see this in driver chapters.

Preemption — when does a task lose CPU?

- **Cooperative**: only when it yields. Old systems, JS engines, some real-time. Bad for misbehaving tasks.
- **Preemptive**: timer interrupt fires; scheduler may switch. Standard now.

Linux is preemptive in user-mode and (in `PREEMPT=y` kernels) in kernel-mode too. Real-time variants (PREEMPT_RT) make almost everything preemptible.

Wake-up & sleep

Tasks block on: - I/O (read from disk, network). - Mutex/condvar. - Sleep (`nanosleep`). - Wait for child.

When the event happens, the kernel **wakes** the task: changes state to Ready, possibly migrates CPU, possibly preempts current task.

Wake-up balancing decisions are subtle and are a frequent source of "scheduler tweaks" in patches.

Real-time scheduling

Hard real-time: missing a deadline is a failure (medical, automotive, GPU compositors arguably).

Linux ≠ hard RT, but with `PREEMPT_RT` patches and `SCHED_DEADLINE`, can be soft RT.

Key concept: **priority inversion** — a low-prio task holding a lock blocks a high-prio task. Solution: **priority inheritance** (the holder's priority is bumped to the highest waiter). Linux's RT mutexes implement this.

Schedulers in the wild

- Linux: CFS / EEVDF + SCHED_DEADLINE + RT classes.

- Windows: priority-based with quanta and dynamic boosts.
- macOS / iOS: Multi-Level Feedback Queue (Mach derivative).
- AWS Firecracker: Linux scheduler, but constrained.
- AWS Lambda: cgroup-isolated containers.

What's hard

Scheduling is full of edge cases:

- Wake-up storms (1000 tasks ready at once).
- Cache + NUMA (move = cache miss explosion).
- Power management (waking a sleeping CPU costs energy).
- Priority inversion + livelock.
- Thermal throttling (the CPU runs slower; scheduler unaware).

Read the changelogs of `kernel/sched/`. Almost every release tweaks something.

Try it

1. Read OSTEP chapters on Scheduling (introduction + MLFQ + CFS).
2. Run `chrt` to set a process to SCHED_FIFO. Watch how it preempts CFS tasks. **Don't run anything broken with high RT priority — it can hang the whole system.**
3. Pin a thread with `sched_setaffinity` to CPU 0; observe in `htop`.
4. Read `Documentation/scheduler/sched-design-CFS.rst`.
5. Pick a long-form `lkml` thread on a recent scheduler patch. Skim it.

Read deeper

- OSTEP chapters 7–10 (Scheduling).
- Linux: `kernel/sched/` and `Documentation/scheduler/`.
- "EEVDF: A Latency-Aware Scheduler for Linux" — recent overview articles.

Next → Synchronization theory.

OST 3 — Synchronization theory: Peterson, semaphores, monitors

> The academic foundation for the locks you've already used. Why they work; where they fail.

Why we need sync

Two threads incrementing a shared counter:

```
load r, [counter]
add r, 1
store [counter], r
```

If interleaved arbitrarily, results are wrong. The textbook word: **race condition**. The fix: ensure the read-modify-write happens **atomically** or that **only one thread is in the critical section at a time**.

Mutual exclusion — the goal

Three properties for a correct mutual exclusion algorithm:

1. **Mutual exclusion**: at most one thread in the critical section.
2. **Progress**: if no one is in CS and someone wants in, they enter (no spurious blockage).
3. **Bounded waiting**: a thread waiting won't wait forever.

Peterson's algorithm (theoretical, software-only)

For two threads, with shared `flag[2]` and `turn`:

```
flag[i] = true;
turn = j;          // let the other go
while (flag[j] && turn == j) ;   // wait
// critical section
flag[i] = false;
```

Provably correct **on a sequentially consistent memory model**. Doesn't work on real CPUs without memory barriers — a beautiful illustration of why memory ordering matters.

It's mostly of historical / educational interest; real systems use hardware atomics.

Hardware support

Modern CPUs offer atomic primitives:

- **Test-and-set (TAS)**: atomically read and set a bit. Set means "I have the lock."
- **Compare-and-swap (CAS)**: atomically replace value if it equals expected.

- **Load-linked/Store-conditional (LL/SC)** on RISC: load with reservation; store succeeds only if no other write happened.
- **Fetch-and-add**: atomic increment.

x86: `LOCK CMPXCHG`, `LOCK XADD`, `XCHG` (implicit lock). ARM: LL/SC pair `LDAXR`/`STLXR`.

These are the building blocks of every synchronization primitive.

Spinlock

```
volatile int lock = 0;

void lock_acquire(void) {
    while (__atomic_test_and_set(&lock, __ATOMIC_ACQUIRE)) {
        cpu_relax();   // pause / yield hint
    }
}

void lock_release(void) {
    __atomic_clear(&lock, __ATOMIC_RELEASE);
}
```

Pros: very fast on uncontended; no syscall. Cons: spinning wastes CPU on contention. Bad for long critical sections. **Good** in the kernel for **short** sections (especially in interrupt context where you can't sleep).

`cpu_relax()` issues `pause` on x86 — hints to the CPU that this is a spin-wait, reduces power and improves SMT sibling performance.

Mutex (sleeping lock)

If contention is likely, **sleep** instead of spinning. Linux's `pthread_mutex_t`:

1. Try CAS to acquire (uncontended path → no syscall).
2. If contended, futex syscall to sleep.
3. On unlock, wake one waiter via futex.

Result: uncontended ≈ free; contended pays one syscall.

Reader-writer lock

Many readers OR one writer. Useful when reads dominate. Implementation tricks:

- Counter for # readers + bit for writer.
- Avoid writer starvation (queue readers behind a waiting writer).
- Sequence locks (seqlock) — reads don't block writers; reader retries on conflict.

Linux: `rwlock_t`, `rw_semaphore`, `seqlock_t`.

Semaphore (Dijkstra, 1965)

A counter: - `wait()` (P): decrement; if becomes negative, block. - `signal()` (V): increment; wake one waiter.

A binary semaphore (count {0, 1}) is essentially a mutex.

A counting semaphore is great for “limit N concurrent users”:

```
        sem_init(&sem, 0, 4);   // 4 concurrent
        sem_wait(&sem); /* do work */ sem_post(&sem);
```

Monitors (Hoare/Hansen)

Higher-level abstraction: a class with implicit mutex around methods + condition variables for waiting on conditions.

```
monitor BoundedQueue {
    queue q;
    cond not_empty, not_full;

    void put(x) { while (full(q)) wait(not_full); q.push(x); signal(not_empty); }
    int take() { while (empty(q)) wait(not_empty); int x = q.pop(); signal(not_full); return x; }
}
```

Java’s `synchronized` + `wait/notify` is essentially monitors. C/POSIX uses mutex + condvar to build them.

Two semantic flavors: - **Hoare**: when a `signal` happens, the signaler immediately yields to the waiter (stack-order priority). Cleaner reasoning. - **Mesa**: signal just makes a waiter Ready; signaler keeps running. Waiter must re-check the condition (hence the `while`!). This is what POSIX does.

The “while loop around `wait`” in your pthreads code is **specifically because** of Mesa semantics + spurious wakeups.

Memory ordering — the modern wrinkle

Compiler and CPU may reorder reads/writes. On x86, mostly OK (TSO). On ARM/POWER, much weaker.

C11 / C++11 memory orderings:

- `relaxed`: atomic, no ordering with other accesses.
- `acquire`: subsequent reads see prior writes from the releaser.
- `release`: prior writes become visible to acquiring threads.
- `acq_rel`: both.
- `seq_cst`: total order.

A correctly written mutex uses `acquire` on lock and `release` on unlock — that’s why prior writes by the unlocker are visible to the next locker.

We’ll see real lock-free patterns in Part IV.

Lock-free vs wait-free

- **Lock-free**: at least one thread always makes progress. No deadlock.
- **Wait-free**: every thread always makes progress within a bounded number of steps. No starvation.

Wait-free is much harder. Lock-free is what most “lock-free queue / map” papers actually achieve.

Common synchronization primitives — summary

Primitive	Use
Atomic ops	counters, flags, lock-free DS
Spinlock	short critical sections, especially in IRQ context
Mutex	normal critical sections
RW lock	read-heavy critical sections
Seqlock	rare writes, frequent reads
Semaphore	counting / signaling
Condition variable	wait-until-condition
Barrier	“all threads reach here before continuing”
RCU	read-mostly, deferred reclaim
Hazard pointers	lock-free pointer reclamation
Monitor	C-style: mutex + condvar + invariants

Common bugs

1. **Deadlock**: cyclic lock acquisition.
2. **Livelock**: threads constantly retry but no progress.
3. **Starvation**: some thread never gets the lock.
4. **Priority inversion** (RT): low-prio holds, high-prio blocked, mid-prio runs forever.
5. **Lost wakeup**: signaling without holding the right lock; receiver missed it.
6. **Memory ordering bug**: flag set after data without release barrier; reader sees flag, not data.

Try it

1. Read OSTEP chapter “Locks” + “Lock-based Data Structures.”
2. Implement Peterson’s algorithm in C with **memory barriers**. Then strip the barriers; observe failure on ARM (or under TSan).
3. Implement a spinlock with `__atomic_test_and_set`. Compare speed to pthread_mutex on a benchmark.
4. Implement a counting semaphore using mutex + condvar.
5. Implement a producer/consumer via monitor (mutex + condvar). Write a class-style header.
6. Read `Documentation/locking/` in the kernel — `mutex-design.rst`, `rt-mutex.rst`, `spinlocks.rst`.

Read deeper

- OSTEP chapters 28–34 (Locks, Semaphores, Condition Variables, Monitors).
- *The Art of Multiprocessor Programming* (Herlihy, Shavit) — academic but readable.
- Linus’s mailing-list rant about `volatile` not being for synchronization (it’s not!).

Next → Classic synchronization problems.

OST 4 — Classic synchronization problems

> Dining philosophers. Readers/writers. Producer/consumer. The textbook problems exist because their solutions teach you patterns you'll reuse for life.

Producer / Consumer (bounded buffer)

The most-used pattern in real systems. Producer adds; consumer removes; bounded queue.

Solution with semaphores

```
sem empty = N    // empty slots
sem full  = 0    // filled slots
mutex  m

producer:
    sem_wait(empty);
    sem_wait(m);
    push(x);
    sem_post(m);
    sem_post(full);

consumer:
    sem_wait(full);
    sem_wait(m);
    x = pop();
    sem_post(m);
    sem_post(empty);
```

Two semaphores ensure correct count; mutex protects the queue itself.

Solution with mutex + condvar

Already shown in USP-09. Use `while (full)` and `while (empty)` — Mesa semantics.

Lock-free version

Specialized: SPSC ring buffer. Multi-producer multi-consumer is much harder.

Readers / Writers

Multiple readers OK simultaneously. Writers exclusive.

Naive (writer starvation)

```
sem_wait(rw_mutex) on first reader
sem_post(rw_mutex) on last reader
```

Writers wait while any reader exists. If readers always come, writer never gets in → starvation.

Writer-preference

Bias toward writers; readers wait if any writer is queued.

Fair version (no starvation)

"Bus-stop" / Hoare's solution: queue-based; whoever's been waiting longest goes next. Linux's `rw_semaphore` is approximately fair.

Seqlock

Optimized for "rare writes, frequent reads." Reader reads, validates by sequence counter, retries if needed. No reader blocking.

Trade-offs: seqlock → readers can be in violent retry loops if writers churn. Picks "scalability" over "writer fairness."

Dining philosophers

Five philosophers, five forks, each needs both neighboring forks to eat. Classic deadlock setup.

Failure mode (deadlock)

All grab left fork simultaneously → all wait for right fork → forever.

Solutions

1. **Asymmetric**: one philosopher picks right first; others left first. Breaks the symmetry.
2. **Resource hierarchy**: total order on resources; always acquire in increasing order. (Generalizes to n forks.)
3. **Arbitrator**: a "headwaiter" (mutex) lets at most n-1 philosophers be picking up forks at once.
4. **Tannenbaum's algorithm**: state-machine — only eat when both neighbors aren't eating.

The lesson: **lock ordering** prevents deadlock. The kernel has the same rule.

Sleeping barber

A barber sleeps; customers arrive, wake him; only N waiting chairs — overflow leaves.

Classic example of "synchronizing with a service provider." Solution: 3 semaphores (customers, barber_ready, mutex_for_chair_count).

Pattern shows up in real-life: Apache prefork, web server worker pools, kernel work-queue model.

Cigarette smokers (a teaching trap)

Three smokers each have one ingredient (paper, matches, tobacco); an agent randomly produces two ingredients and the smoker with the third must form a cigarette.

The puzzle: solve **without conditional waiting / without changing the agent**. Forces students to learn `pthread_cond_t` properly.

It's a teaching device. In real systems you just refactor.

H2O problem

Two H atoms and one O atom must combine. When 2 Hs and 1 O meet, they make a water molecule and proceed.

Solved with semaphores or barriers. Generalizes to "form groups before proceeding."

Once initialization

Multiple threads might be the first to call `init`; only one should run it.

```
once_t once = ONCE_INIT;
pthread_once(&once, init);
```

Internally: a lock + a flag. First thread runs `init`; others block until done; subsequent calls are flag-only fast path.

C++11: `std::call_once`, `std::once_flag`.

Token-passing (the producer/consumer's cousin)

A token rotates around N processes. Holder may do work; passes the token to the next. Used in distributed mutual exclusion.

Barrier

All N threads stop at a point until all arrive; then all proceed.

```
        pthread_barrier_t b;
        pthread_barrier_init(&b, NULL, N);
        // each thread:
        pthread_barrier_wait(&b);
```

Used in: parallel computations (each phase synchronized), GPU dispatches across thread groups.

Reader-Writer with priority

Writers high-prio: starve readers. Readers high-prio: starve writers. Fair: alternate.

Linux's choice for `rwsem` is "writer-priority by default with reader fairness tweaks." See `kernel/locking/rwsem.c`.

Banker's algorithm (deadlock avoidance)

A theoretical algorithm for granting resources only if the system stays in a "safe state" — one where some sequence of process completions exists.

Practical relevance: low. Real systems use **prevention** (lock ordering) or **detection + recovery** (rare).

What this teaches

If you can solve dining philosophers and producer/consumer cleanly in C, **you can write robust threaded code anywhere**. These problems aren't toys — they're the kernel's daily workload in miniature.

Try it

1. Producer/consumer (bounded buffer) — N producers, M consumers, capacity K. Verify count with stress test.
2. Readers/writers — both naive and fair versions. Show writer starvation in naive.
3. Dining philosophers — three solutions: ordered locking, arbiter, asymmetric. Confirm no deadlock.
4. H2O problem — combine 2H + 1O via semaphores.
5. Implement a `barrier` from scratch using mutex + condvar.
6. Bonus: dining philosophers with 1000 philosophers; measure throughput with each strategy.

Read deeper

- *The Little Book of Semaphores* (Allen Downey) — free PDF, dozens of these problems.
- OSTEP chapter on "Common Concurrency Problems."
- `Documentation/locking/lockdep-design.rst` (Linux's deadlock detector).

Next → Deadlock theory.

OST 5 — Deadlock: prevention, avoidance, detection

> When threads or processes hold resources and wait for resources others hold. Forever. The textbook hazard.

The four conditions for deadlock (Coffman, 1971)

A deadlock requires **all four**:

1. **Mutual exclusion** — at least one resource is non-shareable.
2. **Hold and wait** — a thread holding one resource waits for another.
3. **No preemption** — resources can't be forcibly taken from holders.
4. **Circular wait** — chain of threads waiting on each other.

Eliminate any one → no deadlock.

Prevention strategies

1. Eliminate hold-and-wait

Acquire all resources up front. Drawbacks: poor utilization, may not know set in advance.

2. Allow preemption

Release a held lock if needed. `pthread_mutex_trylock` + back-off. Used in some lockless / wait-die schemes.

3. Impose a total order on resources (the standard fix)

If every thread acquires locks in the **same global order**, circular wait is impossible.

```
// Always acquire A then B, never the other order.
pthread_mutex_lock(&A);
pthread_mutex_lock(&B);
// ...
pthread_mutex_unlock(&B);
pthread_mutex_unlock(&A);
```

Linux kernel does this religiously. **Lockdep** (kernel) verifies this at runtime.

For complex code, define **lock classes** and document order. E.g., "always acquire `mm->mmap_lock` before `vma->vm_lock`."

Avoidance — the Banker's algorithm

Decide whether to grant a resource based on whether granting keeps the system in a **safe state** (i.e., some serial completion order exists).

Theoretical; rarely used in practice. Modern OSes prefer prevention.

Detection + recovery

Allow deadlocks; periodically check the resource-allocation graph. If found, kill or roll back a process.

Used in: - Databases (deadlock detection in InnoDB, Postgres, Oracle — kill the youngest victim, retry). - Filesystems with complex locking. - Linux **lockdep** is a static-style detector — runs in development kernels.

Livelock

Like deadlock, but threads are **active** — retrying, backing off, retrying — making no progress.

Common pattern: two threads each `trylock; if fail, release and retry`. They keep failing in lockstep.

Mitigate with exponential backoff or randomization.

Priority inversion (and inheritance)

Low-prio holds lock; high-prio blocks on it; mid-prio (no lock dependency) preempts low → high stuck. Famous example: NASA's Mars Pathfinder reset loop.

Fix: **priority inheritance** — when high-prio waits on a lock, the holder's effective priority is bumped. Linux's RT mutexes (`rt_mutex`) implement this for `SCHED_DEADLINE` / `SCHED_FIFO` .

Lockdep — Linux's deadlock detector

`CONFIG_PROVE_LOCKING=y` activates lockdep. Every lock acquisition is recorded; lockdep tracks the **partial order** observed at runtime.

If you ever take **A→B** in one place and **B→A** in another, lockdep prints a "possible recursive locking" warning, even before a deadlock occurs. Saves countless hours.

```
============================================
[ INFO: possible recursive locking detected ]
[...]
```

Always run kernel development kernels with lockdep on.

Common deadlock-shaped bugs

1. **Re-entrant locking** — same thread tries to acquire a non-recursive mutex it already holds. Hangs.

2. **Calling a sleeping function with a spinlock held** — kernel BUG, system can stall.
3. **Allocating memory with `GFP_KERNEL` while holding a lock that the writeback path needs** — circular dependency on memory pressure.
4. **Two processes locking each other's PI-mutexes** — only avoided with priority inheritance.

Strategies in real systems

System	Approach
Databases (InnoDB, Postgres)	Detection + abort youngest
Linux kernel	Prevention via lock ordering + lockdep
Java	Wait-with-timeout, plus Java's `tryLock`
Go	Cooperative — use channels, avoid shared-state locks where possible
Erlang/Akka	No shared state; actors only

Algorithms tangentially related

- **Wait-die / wound-wait** schemes (in DBMS): age-based decisions to avoid deadlock.
- **Two-phase locking (2PL)**: acquire all locks, then release. Common in DB transactions; with strict ordering, prevents deadlock.

Common bugs

1. **Unannotated lock order** — different files take locks differently.
2. **Lock cycles via callback** — calling user-supplied function while holding lock; user's callback re-enters your lock.
3. **Implicit lock dependency** — `printk` may take internal locks; calling from inside spinlock context can deadlock if not careful.
4. **Leaked lock** — early return with lock held.

Try it

1. Construct a deadlock between two threads with two mutexes. Detect it with `pthread_mutex_timedlock` returning ETIMEDOUT.
2. Run with TSan; observe the "lock order inversion" report.
3. Add a third mutex; create a 3-cycle deadlock. Detect.
4. Fix all three by enforcing a global order; rerun.
5. Read `Documentation/locking/lockdep-design.rst`. Try to summarize how lockdep records "lock-acquired-after" pairs.
6. Bonus: build a tiny "lock checker" tool that records pairs `(L1, L2)` from your test program and reports cycles.

Read deeper

- OSTEP chapter on “Common Concurrency Problems” (deadlock + non-deadlock bugs).
- “Eraser: A Dynamic Data Race Detector for Multi-Threaded Programs” (the paper behind Helgrind/TSan).
- `Documentation/locking/` in Linux.

Next → Memory management theory.

OST 6 — Memory theory: paging, segmentation, swapping

> The OS lies to every process: each thinks it has the whole address space to itself. Here's how the lie is sustained.

The illusion

Every process sees a contiguous range of addresses, e.g., 0x0 .. 0x7fffffffffff (128 TB on Linux x86-64). Reality: - Memory is **fragmented** physical RAM. - **Pieces** (pages) of the process are mapped where physical RAM has room. - Some pages may be **on disk** (swapped out) until needed. - All processes share the same physical RAM but **don't see** each other's.

The hardware **MMU** + OS **page tables** maintain the illusion.

Address translation

Virtual address → physical address. The MMU translates on every memory access (assisted by TLB caching).

Two main schemes historically:

Paging

Memory divided into **fixed-size pages** (typically 4 KB; bigger for huge pages). Both virtual and physical address spaces are paginated. A **page table** maps virtual page → physical frame.

Example, 4 KB pages, 48-bit virtual address: - bits [47:39] index level 1 (PML4) - bits [38:30] index level 2 (PDPT) - bits [29:21] index level 3 (PD) - bits [20:12] index level 4 (PT) - bits [11:0] offset within page.

Each level holds 512 entries (9 bits each). Walking 4 levels gives ~256 GB of indirect address space; the kernel may set up the same on multiple processes' page tables.

This is the **multi-level page table** structure used by x86-64. Also called "radix tree of page tables." Other architectures use slightly different splits (ARM's 4-level, RISC-V's Sv39/Sv48).

Segmentation

Memory divided into **variable-size segments** (code, data, stack). Address = (segment, offset). x86 had it; modern Linux x86-64 has flat segments and uses only paging. Segmentation is mostly historical.

Combined

80386 supported both. Most systems today use paging only.

Page table entries (PTEs)

Each PTE typically holds: - Physical frame number. - Present bit (else, page fault on access). - Read/Write/Execute permissions. - User/Supervisor (kernel can access, user can or can't). - Accessed (set by hardware on read). - Dirty (set on write). - NX bit (no execute).

Page faults

On any access not satisfied by current page tables, the CPU raises a **page fault**. The OS handler decides:

- **Demand-paged**: the page exists logically but isn't loaded yet → load from disk, fix PTE, retry.
- **Copy-on-write**: page is shared but writer wants to modify → copy + remap.
- **Stack growth**: access just below the stack → expand stack, retry.
- **Swap-in**: page was swapped to disk → fetch back.
- **Bad address**: not part of any valid mapping → SIGSEGV the process.

TLB (Translation Lookaside Buffer)

Cache of recent virtual-to-physical translations. Hit → ~1 cycle. Miss → page table walk (4 memory loads on x86, possibly all the way to RAM if not cached).

TLB shootdown: when the OS unmaps a page, must invalidate the TLB on every CPU that might have it. Expensive — IPI-based. A common scalability bottleneck.

ASIDs / PCIDs: hardware tag in the TLB so different processes' TLB entries don't collide. Reduces flushing on context switch.

Page sizes

- 4 KB: standard.
- 2 MB: x86-64 large page.
- 1 GB: x86-64 huge page (1 GB pages).
- 64 KB / 1 MB / 16 GB on ARM, depending on translation granule.

Bigger pages → fewer TLB entries needed → lower TLB pressure for big workloads. But more **internal fragmentation** if you use only part. Linux's THP (Transparent Huge Pages) tries to give big pages opportunistically.

Swapping

When physical RAM fills, the OS may **swap out** less-used pages to disk: - mark PTE non-present; - write page contents to swap area; - on next access, page-fault → swap back in.

A modern Linux thin distinction: - **Page cache**: pages backed by files. “Drop them” means just discard; can be re-read from file. - **Anonymous pages**: pages not backed by files (heap, stack, malloc). Need a swap area to evict.

Working set & locality

The **working set** of a process is the set of pages it’s actively using. If the working set fits in RAM, performance is good. If not, **thrashing** — constant swapping, the system grinds.

Programs exhibit **temporal** and **spatial** locality: - Recently accessed pages likely accessed again soon. - Nearby pages likely accessed together.

Caches and prefetchers exploit this.

Memory zones (Linux)

Physical memory is divided into zones with different properties:

- **DMA32**: for devices that can only address 32-bit addresses.
- **DMA**: even more restricted (legacy 24-bit).
- **NORMAL**: most RAM; freely usable.
- **HIGHMEM**: on 32-bit, RAM beyond what kernel can directly map.
- **MOVABLE**: pages that can be migrated (for memory hot-unplug, defrag).

Allocators (buddy, slab) consult zone preferences.

NUMA

Multiple memory nodes per machine; access to remote nodes is slower. Linux’s NUMA-aware allocators try to allocate on the local node. Page migration may move pages closer to the CPU using them.

Allocation strategies (the OS picks frames)

- **Buddy allocator**: power-of-two sized blocks; split on alloc, merge on free. Fast, low fragmentation. Linux page allocator is buddy.
- **Slab / SLUB**: caches of small fixed-size objects; built on top of pages. (Part V/III.)
- **vmalloc**: virtually contiguous, physically scattered — good for big allocations that can’t get a contiguous frame.

Real address spaces (Linux x86-64)

```
0x0000000000000000  user space (lower half, ~128 TB)
0x0000800000000000  ... (canonical hole; sign extension)
0xffff800000000000  kernel mapping starts
0xffff800000000000  direct map of all physical memory
0xffffea0000000000  vmemmap (struct page array)
0xffffffff80000000  kernel image, text, modules
```

Userspace processes see the lower half. Kernel is mapped to upper half (always present) so syscalls don’t need a page table swap.

What about virtualization?

A guest OS runs in a VM. It has its own page tables (guest virtual → guest physical). The hypervisor adds a second translation (guest physical → host physical), via **EPT** (Intel) / **NPT** (AMD). This is "**nested paging**" — two levels of MMU walks. SR-IOV GPUs and other PCIe passthrough need IOMMU support too.

What you should remember

- Every memory access is **virtually** addressed; the MMU translates with caching.
- Page tables are a hierarchical data structure; most are kernel-managed; some have hardware semantics.
- Page faults are normal; the OS handles them.
- TLB is critical to speed; flushes are expensive.
- The kernel chooses what stays in RAM (page cache + LRU); the next chapter is about which pages to evict.

Try it

1. Read `/proc/self/maps` and decode the regions you see.
2. Watch a long-running program with `pmap -X PID`. What's its biggest segment?
3. Cause a deliberate page fault: `mmap` a file but `MADV_DONTNEED` it; access pages and watch in `/proc/PID/status` (RSS / faults).
4. Use `time` (with -v) on a workload; look at "minor page faults" vs "major page faults." Major = went to disk.
5. Try `numactl --hardware` (if NUMA system). Identify nodes and distances.
6. Read OSTEP chapters on Address Spaces (13–17), Beyond Physical Memory (21–24).

Read deeper

- OSTEP chapters 13–24 — entire memory section.
- *What Every Programmer Should Know About Memory* (Drepper, 2007) — deep, classic.
- `Documentation/admin-guide/mm/` and `mm/` source in the kernel.

Next → Page replacement algorithms.

OST 7 — Page replacement algorithms

> When RAM is full, which page do you evict? The choice can affect performance by an order of magnitude.

The setup

A process accesses pages over time: A, B, C, A, D, B, A, … If RAM fits 3 pages and we want a 4th, we must evict one. The evicted page may be re-fetched later, costing a page fault.

Goal: minimize page faults.

The optimal (Belady's MIN)

Evict the page that will not be used for the longest time in the future.

Provably optimal — proven by Belady (1966).

But it requires **future knowledge**. So: it's our **upper bound**; real algorithms approximate it.

FIFO (First-In, First-Out)

Evict the oldest page.

Simple. **Belady's anomaly**: more memory can sometimes cause **more** faults under FIFO (counter-intuitive). Avoid FIFO.

LRU (Least Recently Used)

Evict the page **not used for longest** in the past.

Approximates optimal under temporal locality (assumes recent past predicts near future). Generally good.

Implementation: - **Naive**: timestamp on each access. Scanning is O(pages) per replacement. - **Better**: linked list — most recent at front. On access, move to front. Plus a hash map for O(1) lookup. (DSA-26.) - **Hardware-supported**: PTE has "accessed" bit; OS scans periodically.

Approximate LRU — the practical reality

Real systems can't update a list on **every** memory access (too expensive). They use **clock-style** or **second-chance** algorithms.

Clock algorithm (NRU, “Not Recently Used”)

Pages in a circular list. A “hand” sweeps: - If page’s “accessed” bit is 0 → evict. - If 1 → clear it; move on.

Cheap; approximates LRU. Used in Linux (with tweaks).

Two-handed clock

Two hands; the leading hand clears bits, trailing hand evicts. Tunable rate.

Page-aging (Linux)

Each page has an “age” counter, periodically decayed. Access bumps age. Evict lowest age.

LFU (Least Frequently Used)

Evict the page with the smallest access count.

Pros: handles “hot” pages well. Cons: pages with one-time burst of accesses become hard to evict.

Often combined with aging (decay counter).

Working-set algorithm

Compute the set of pages accessed in a recent window (e.g., last 1 second). Keep working set in RAM; evict outside.

Predicts thrashing — if total working sets > RAM, system is doomed.

Modern variants

ARC (Adaptive Replacement Cache)

Maintains two LRU lists (recently used, frequently used) plus “ghost” lists tracking what was evicted recently. Adapts the partition between them. Used by ZFS, IBM DB2.

2Q

Splits cache into “in” (newcomers) and “out” (proven hot). Approximates LRU’s ability while being simpler.

TinyLFU / W-TinyLFU

Recent (last decade) approach. Tracks frequency via count-min sketch. Used in modern Java caches (Caffeine, Guava).

LIRS

Distinguishes pages by **recurrence interval** (how often used). Better than LRU for some loop-heavy workloads.

Linux's actual policy

Linux maintains **two LRU lists** per memory zone: - **Active list**: pages thought to be hot. - **Inactive list**: pages possibly cold; first to be evicted.

Page transitions: - Newly mapped → inactive. - Accessed once → moved to active. - Active page not accessed for a while → demoted to inactive. - Inactive + not accessed → eligible for eviction.

This is approximately a **two-handed clock with frequency awareness**.

In `mm/vmscan.c`, `mm/swap.c`. Read these once you finish Part V.

Anonymous vs file-backed

Linux's eviction logic differs: - **File-backed** (page cache for regular files): clean → just drop; dirty → write back, then drop. - **Anonymous** (heap, stack): need a swap area to write to before eviction.

Both go on LRU lists. Tunable balance via `vm.swappiness` (0 = avoid swap, 100 = swap freely).

Major vs minor faults

- **Minor fault**: page already in RAM (e.g., lazily mapped). Cheap; just fix PTE.
- **Major fault**: page must be fetched from disk. Slow.

Minimizing **major faults** is the goal of page replacement.

Thrashing

When the working set exceeds RAM, the system spends most time swapping pages in/out — making no real progress. Throughput collapses.

Mitigations: - More RAM. - Reduce process count. - Better algorithms (working-set aware). - The OOM killer (Linux: kills a process when memory pressure is severe).

In databases

DB engines often **manage their own buffer pool**, ignoring the OS page cache (with `O_DIRECT`). They tune replacement (often LRU-K, ARC) for SQL workloads. The OS page cache becomes a thin shim.

Common pitfalls

1. **Mistaking LRU for optimal**. LRU works on locality; for sequential huge scans (bigger than RAM), LRU is the worst — every page evicts before re-use. ARC and 2Q help.
2. **Using ages too coarsely**. Age = 1 vs Age = 0 distinguishes nothing useful at scale.
3. **Bypass the cache** with O_DIRECT and forget that **you** now have to manage replacement.

Try it

1. Simulate FIFO, LRU, OPT (oracular optimal) on a hand-crafted access trace. Count faults for each.
2. Reproduce Belady's anomaly: a trace + page count where FIFO gets **more** faults with **more** memory.
3. Implement an LRU cache with O(1) ops (DSA-26 again). Measure hit rate on a real trace (e.g., a website's request log).
4. Read `mm/vmscan.c` introduction comments. Identify the two-list model.
5. Tune `vm.swappiness` on a test box. Run a memory-heavy workload; observe swap usage with `vmstat`.

Read deeper

- OSTEP chapters 21–24 (Beyond Physical Memory).
- Megiddo & Modha "ARC: A Self-Tuning, Low Overhead Replacement Cache" (2003).
- Linux kernel `Documentation/admin-guide/mm/` — workingset.rst, transhuge.rst.

Next → File system theory & journaling.

OST 8 — File system theory & journaling

> A filesystem turns “blocks of bytes on a disk” into “files in a tree.” The data structures and crash-safety stories are gold.

What a filesystem must provide

- **Naming**: turn names into files (path resolution).
- **Storage**: efficient layout on disk.
- **Metadata**: size, owner, permissions, timestamps.
- **Concurrency**: many processes, one FS.
- **Durability**: survive crashes.
- **Performance**: reads, writes, sync.

These goals interact. Real FSes pick trade-offs.

On-disk layout (classic ext-style)

A disk is divided into blocks (4 KB typical):

```
+----------------+--------------------+----------------+--------------------+
| Boot block(s)  |  Superblock        | inode table    | data blocks        |
+----------------+--------------------+----------------+--------------------+
```

- **Superblock**: FS metadata (block size, total blocks, FS UUID, journal location).
- **Inode**: metadata for one file (size, owner, perms, list of data blocks). NO filename.
- **Data block**: actual file contents.
- **Directory**: a file whose contents are name → inode pairs.

Names live in directories, not in inodes. **Hard linking** (multiple names → same inode) is therefore natural.

Inode

Typical fields: - mode (file type + permissions). - uid, gid. - size. - atime, mtime, ctime. - direct block pointers (e.g., 12). - single, double, triple indirect block pointers.

ext4 uses **extents** (start + length) instead of block pointers — far more efficient for large contiguous files.

Path resolution

`/usr/local/bin/foo` → start from root inode → look up “usr” in dir entries → its inode → look up “local” → … → final inode.

Each lookup involves potentially: - Reading the directory file. - Reading the inode. - Permission checks. - Symlink following.

Linux's **dentry cache** caches resolved paths. Critical for performance.

Allocation strategies

How does the FS pick a block for a new write?

- **Bitmap free list**: one bit per block. Find first zero.
- **Buddy-like**: bigger free regions tracked separately.
- **Extents**: track contiguous free regions.

Locality matters — placing a file's blocks close together reduces seek time on rotational disks. SSDs care less but cache lines / erase blocks still exist.

Concurrency in FS

Multiple processes modifying same file or same directory → race. Handled with: - per-inode locks, - per-directory locks, - journaling for crash atomicity.

Crash consistency — the hard part

A crash mid-write can leave the FS in an inconsistent state: - Inode says the file is 10 KB, but only 5 KB of data is on disk. - Directory has entry for inode that doesn't exist. - Free list says block free, but it's actually in use.

Approaches:

`fsck` — scan and repair

After crash, scan everything; fix inconsistencies. Slow (minutes to hours on big disks). Old-style.

Journaling — write-ahead log

Before modifying real structures, write the **intent** to a journal. After crash, replay journal.

Two flavors: - **Data journaling**: log both metadata and data. Safe; double-writes (slow). - **Metadata journaling**: only metadata in journal; data flushed before. Faster; data ordering invariant: data on disk before metadata refers to it.

ext3/ext4 default: metadata journal with `data=ordered` (data before metadata). xfs similar.

Soft updates (FreeBSD's UFS2)

Carefully order writes so the FS is always consistent. No journal. Complex to implement. Mostly historical.

Copy-on-write (ZFS, btrfs)

Never overwrite. Write new blocks; update pointers atomically (sometimes a single sector write at the root). Eternal consistency. Snapshots are free. Cost: more I/O, more fragmentation; needs garbage collection.

Log-structured (LFS, F2FS for SSDs)

Treat the disk as a log: append everything; periodically clean (compact) the log. Great for SSDs (avoids in-place writes that wear out flash).

fsync & friends — durability boundaries

`write` puts data in the page cache. `fsync(fd)` waits until data is on disk. Without `fsync`, a crash can lose recent writes.

Some disks lie about being persistent (power-loss-protection-less SSDs). `O_SYNC` and barriers ensure ordering, but you trust the device.

File system zoo

FS	Notes
ext4	Linux default for years. Extent-based, journaled. Mature.
xfs	Originally SGI; great for large files / high concurrency.
btrfs	Copy-on-write; snapshots, RAID, checksums. Mature; some users still prefer ZFS.
ZFS	OpenZFS on Linux. CoW + checksums + RAID-Z + ARC. Heavy.
F2FS	Flash-friendly, log-structured. Used in some Android devices.
tmpfs	RAM-backed; POSIX shared memory uses it.
FAT32 / exFAT	USB drives, SD cards. Simple, no journaling.
NTFS	Windows. Linux can read/write via ntfs3 driver.
iso9660	CDs. Read-only.
nfs / cifs	Network file systems.
FUSE	“Filesystem in Userspace”; lets you write FSes as regular processes. SSHFS, gocryptfs, etc.

VFS — Virtual Filesystem Switch

Linux’s abstraction: an in-kernel API that all filesystems implement. Userspace `read("/somefile")` flows through:

```
syscall: sys_read
        -> vfs_read
        -> file->f_op->read_iter   (depends on FS)
        -> ext4_read_iter (e.g.)
        -> page cache or disk
```

VFS objects: **superblock** (FS instance), **inode**, **dentry** (cached path entry), **file** (open file).

We’ll cover VFS in Part V.

Network filesystems

NFS: simple RPC; client caches inodes/dentries. Coherence is “loose” (close-to-open). Used in HPC clusters.

CIFS/SMB: Windows file sharing.

Object stores (S3, Ceph): not POSIX — different API.

Modern distributed: Lustre, GlusterFS, CephFS for parallel HPC.

Performance issues

- **fsync latency**: SSDs ~1ms, HDDs 10ms+. Critical for DB commit.
- **Metadata-heavy** (lots of small files): inode + dentry cache misses dominate.
- **Fragmentation**: ext4/xfs handle well; severe on FAT.
- **Compression / encryption**: can speed up (less I/O) or slow down (CPU bound).
- **RAID**: parity calc costs writes; raid-1 doubles writes, raid-5/6 has read-modify-write penalty.

Try it

1. `df -hT` — what FS is each mount? Identify ext4, tmpfs, etc.
2. Make a 100 MB file with `dd`. `fdisk -l` (or `lsblk`); `tune2fs -l /dev/...` to inspect ext4 metadata.
3. Run a benchmark: `dd if=/dev/zero of=test bs=1M count=1000 oflag=direct` vs without `oflag=direct`.
4. Use `iotop` while doing massive writes; observe.
5. Read OSTEP chapters 39–43 (file systems).
6. Bonus: write a tiny FUSE filesystem (e.g., a “memory FS” that lives in RAM only).

Read deeper

- OSTEP chapters 39–45 — entire FS section.
- *The Linux Programming Interface* chapters on FS.
- For btrfs: “An introduction to btrfs” — multiple LWN articles.
- For ZFS: “End-to-End Data Integrity for File Systems” (Bonwick & Moore).

Next → Crash consistency & write ordering.

OST 9 — Crash consistency & write ordering

> Power dies. RAM goes. Disk has half-old, half-new state. How do real systems survive that?

The problem

You issue these writes:

```
1. write inode (file size = 10K)
2. write data block #1
3. write data block #2
```

Crash anywhere → state is partially applied. Possible bad outcomes: - (1) without (2,3) → inode says 10K, file is junk after the first 0 bytes. - (2,3) without (1) → data on disk but no inode pointing to it = leak. - (3) without (1,2) → orphan data block.

You need either **all-or-nothing** or **a known correct intermediate state**.

Tools the OS uses

Atomic single-sector write

Most disks guarantee a single sector (512 B or 4 KB) write is atomic — either fully succeeds or doesn't. Bigger writes can be torn.

Barriers / ordering

Tell the disk "don't reorder this past that point." Implemented via FUA (Force Unit Access) flags, cache flush commands.

The kernel's block layer enforces these for FSes that ask. When you do `fsync`, the FS issues a flush.

Checkpoints

Periodically declare the disk consistent. After crash, recover from the latest checkpoint forward.

Approaches

1. fsck-after-crash (very old)

Run `fsck` after every crash. Slow but simple. ext2 worked this way.

2. Write-ahead log (WAL) / journal

Before modifying actual data, write a record of what you're about to do.

Sequence: 1. Write the journal entry. Issue a barrier. 2. Apply the changes to FS metadata + data. 3. Mark the journal entry as committed. (Or just rely on transaction order.) 4. Periodically truncate the journal.

After crash: - Replay all uncommitted journal entries to redo (or to undo, depending on protocol).

ext3/ext4, NTFS, XFS, ZFS (zil), most SQL DBs.

Modes (ext4): - `data=journal`: journal everything, including data. Safe. Slow. - `data=ordered` (default): journal metadata; **flush data before metadata** that points to it. Common balance. - `data=writeback`: journal metadata only; **no data ordering**. Fastest, files can have stale data after crash.

3. Soft updates

Order writes carefully so the FS is **always** consistent on-disk. E.g., for "create file": 1. Write inode (with refcount=0). 2. Write directory entry. 3. Write inode (refcount=1).

If crash between 1 and 2 → orphan inode (recoverable via fsck pass).

Implementations have **dependency tracking** in memory; complex.

4. Copy-on-write (CoW)

Never overwrite. Build new state in new blocks; update root pointer atomically (one sector). All previous state remains intact.

ZFS, btrfs.

Pros: trivial consistency. Snapshots are free (just keep old root). Checksums can verify. Cons: more writes (every change = new blocks all the way to root). Garbage collection. Fragmentation.

5. Log-structured

Treat the disk like a log; append everything. Old data left in place; cleaner / GC reclaims periodically.

Excellent for SSDs (only forward writes). LFS (1992), F2FS (Linux), modern flash FSes.

Atomic rename — the user's friend

POSIX guarantees `rename(old, new)` is atomic on the same filesystem (within one inode/path). So safe-write-and-replace pattern:

```
1. write data to "foo.tmp"
2. fsync("foo.tmp")
3. rename("foo.tmp", "foo")     // atomic
```

Now `foo` is either fully old or fully new — never half-written.

This is **the most-used safe-write pattern**. Editors use it; package managers; everywhere.

Disk surprises you should know

- Some disks have **volatile write caches**: they ack the write before it's actually durable. Cache flushes are slow but necessary.
- **Power loss during firmware-internal operation** can corrupt disks (rare; mostly cheap drives).
- **Bit rot**: data can decay over time. Filesystems with checksums (ZFS, btrfs) detect; others silently corrupt.
- **SSDs**: write amplification, garbage collection, wear leveling. Operating systems issue **TRIM** to inform the SSD which blocks are free.

Real-world durability (databases)

PostgreSQL, MySQL, etc. write a **WAL** (write-ahead log). Commit = write WAL + fsync. Asynchronous flush of data pages.

After crash: 1. Read WAL. 2. Re-apply committed transactions. 3. Roll back uncommitted ones.

Same idea as FS journal, deeper structure. **Never lose committed data.**

`fsync` is the **transaction commit's** durability boundary.

Common pitfalls

1. **`write()` without `fsync()` ≠ durable.**
2. **Renaming on a different filesystem** silently degrades to copy + delete. Atomic rename only **within** an FS.
3. **Renaming a file pointed to by an open fd** — the fd still works, fine.
4. **Caches lie**: enterprise SSDs have power-loss capacitors; consumer SSDs may not.
5. **fsync on a directory**: needed if you want the directory entry change (e.g., new file) durable.
6. **NFS**: `fsync` over NFS has historical bugs; check the man page for your distro.

Try it

1. Implement "safe save": write to `tmp`, `fsync`, `rename`. Test by SIGKILLing the writer; verify file is either old or new, never half.
2. Stress-test crash safety: in a VM, periodically fsync after every commit, then reboot the VM hard. Verify your data is intact.
3. Read the ext4 journal log: `tune2fs -l /dev/...` shows journal device and inode. Use `debugfs` to inspect.
4. Read OSTEP chapter 42 (FSCK and Journaling).
5. Read the `dwarfs` or `bcachefs` design docs (modern Linux FSes).

Read deeper

- "Crash Consistency: FSCK and Journaling" — OSTEP chapter 42.

- “All File Systems Are Not Created Equal” by Pillai et al. (OSDI 2014) — devastating analysis of fsync usage.
- “Files are hard” — Dan Luu’s blog post.
- Postgres’s “Write-Ahead Logging” docs.

Next → Distributed systems intro.

OST 10 — Distributed systems intro: consensus, RPC, time

> "A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable." — Leslie Lamport. A short tour for the kernel-bound engineer.

Why a kernel engineer should care

GPUs run in clusters. Multi-GPU systems share work. Drivers talk to firmware over rings. Firmware runs its own scheduler. Even within a single machine, the **GPU and CPU are a distributed system** (no shared coherent memory in many cases; messages, fences, queues).

Concepts here will recur: - consensus → who's writing to a shared queue? - time → ordering events without a global clock. - failure modes → silent corruption vs explicit error. - caching → invalidation and consistency.

Definitions

A **distributed system** is multiple independent computers communicating over a network, usually: - without shared memory, - with possible network failures (lost, delayed, reordered messages), - with possible node failures (crash-stop, crash-recover, byzantine), - with no global clock.

These properties make distributed systems **harder than threads**.

The fundamental impossibility — FLP

Fischer, Lynch, Paterson (1985): in an asynchronous system with even one faulty process, no algorithm can guarantee both **safety** and **liveness** for consensus.

Practical consequence: every consensus protocol either: - has timeouts (gives up liveness eventually), - assumes synchrony, - weakens guarantees.

CAP theorem

Brewer: in a distributed system, you can pick at most **2 of**: - **Consistency** (all reads see most recent write), - **Availability** (every request gets a response), - **Partition tolerance** (works when network splits).

Since networks **always** partition (briefly, eventually), the real choice is **C vs A** during a partition.

- CP: refuse responses to ensure consistency (e.g., consensus-based systems like etcd, Spanner).
- AP: respond with possibly-stale data (e.g., DynamoDB, Cassandra).

CAP is a frame; nuance lives in how a system handles partitions when they happen.

Time and ordering

Without a global clock, “what happened first?” is hard.

Lamport timestamps

Each message: sender’s logical time + 1 → receiver’s. Gives a partial order (“happened-before”). Doesn’t detect concurrency.

Vector clocks

Each node tracks a vector of seen events from every node. Detects concurrent events. Used in Dynamo, Riak.

Hybrid logical clocks (HLC)

Combine wall clock + counter; tightly bounded skew + monotonic. Used in CockroachDB, MongoDB.

Real clocks

GPS / atomic-clock synced clocks (Google Spanner’s TrueTime). Provides bounded uncertainty windows. Linearizable transactions across continents.

Consensus algorithms

The classic problem: a set of nodes must agree on a value.

Two-phase commit (2PC)

Coordinator asks all participants “can you commit?” If all yes → commit. Else abort.

Vulnerability: coordinator dies after some commits but before all → blocked. Real DBs use 3PC or external resolution.

Paxos

Lamport, 1998. Solves consensus despite up-to-f failures with 2f+1 nodes. Notoriously hard to understand in original paper. “Paxos Made Simple” is the canonical readable version.

Raft

1. Designed to be **understandable**. Leader-based; if the leader dies, an election picks a new one. Replicates a log via simple AppendEntries RPC.

Used in: etcd, Consul, CockroachDB, RethinkDB, TiKV.

Byzantine fault tolerance (BFT)

For nodes that may **lie** (malicious / compromised). PBFT, Tendermint. Used in some blockchains. Heavier (3f+1 nodes for f Byzantine failures).

RPC (Remote Procedure Call)

Make a function call appear remote. Pioneered by Birrell & Nelson (1984). Modern incarnations: gRPC, Cap'n Proto, Thrift.

Key issues: - Failures: did the call succeed if you got no response? You don't know. - Idempotency: can you safely retry? Design APIs so retry is safe. - Latency: vastly higher than local calls; don't pretend they're free. - Serialization: protobuf, JSON, msgpack.

Replication

Multiple copies of data for availability + durability.

Single-leader (primary/backup)

All writes go to leader; leader replicates to followers. Simple. Strong consistency on reads from leader.

Multi-leader

Multiple leaders accept writes; sync to each other. Used in geo-replication. Conflict resolution is hard (CRDTs, last-writer-wins, manual merge).

Leaderless

Writes go to multiple nodes; reads query multiple. Quorum-based. Dynamo, Cassandra.

Quorums

W = write replicas, R = read replicas, N = total. For strong consistency: **W + R > N**. Common choice: W = R = N/2 + 1.

Eventual consistency

"After a quiescent period, all replicas converge." Used in DynamoDB, Cassandra. Cheap, scalable, occasional inconsistency. Useful when application can tolerate it (shopping cart, social posts).

Stronger models (reading)

- **Linearizability**: every operation appears to take effect at a single instant between its invocation and response. Like a single shared variable.
- **Sequential consistency**: all operations appear in some total order, consistent with each thread's order.
- **Causal consistency**: causally related operations are seen in order.

- **Eventual consistency**: weakest non-trivial.

In practice, **read your own writes** is what users actually expect.

CRDTs (Conflict-free Replicated Data Types)

Data structures that always converge under arbitrary merge order. Examples: G-counter (grow-only counter), OR-set (observed-remove set), LWW-register.

Used in Riak, Redis enterprise, collaborative editors (Yjs, Automerge).

Failure detectors

How do you know a remote node is dead? - **Heartbeats**: periodic "I'm alive." - **Phi-accrual**: probabilistic measure of likelihood of failure.

False positives (think dead, actually alive) → split-brain. False negatives (alive, really dead) → unavailability.

Saga / transactions across services

Microservice "transactions" can't use 2PC across DBs. Pattern: **saga** — sequence of local transactions with compensating actions. Tolerate intermediate states.

Real-world systems to know about

- **etcd / Consul** — Raft-based config stores. Used by Kubernetes for cluster state.
- **ZooKeeper** — older Zab-based. Used by Hadoop, Kafka.
- **Spanner** — Google's globally-consistent DB. TrueTime-based.
- **CockroachDB / TiKV / FoundationDB** — distributed SQL with serializability.
- **Cassandra / DynamoDB / ScyllaDB** — eventually-consistent KV.
- **Kafka** — distributed log (per-partition leader, replicated).

What this means for GPUs

GPU drivers contain "small distributed systems": - CPU and GPU as two nodes (with messaging via rings). - Multi-GPU coherence (NCCL, RCCL). - SR-IOV virtualization (host + N guest "drivers" sharing one physical GPU). - Firmware on the GPU (PSP, SMU) running concurrently.

Concepts like fences, sequence numbers, timeouts directly mirror distributed systems patterns.

Try it (mostly reading)

1. Read the Raft paper "In Search of an Understandable Consensus Algorithm." Watch the Raft visualization (https://raft.github.io/).
2. Read Brewer's "CAP twelve years later" article — short, by the originator.
3. Try `etcd` locally: start 3 nodes; partition one with `iptables`; observe behavior.

4. Read “There’s just no getting around it: you’re building a distributed system” — Mark Cavage.
5. Read *Designing Data-Intensive Applications* by Kleppmann at least chapters 5, 7, 9.

Read deeper

- *Designing Data-Intensive Applications* — Kleppmann. Best modern survey.
- *Distributed Systems* by van Steen & Tanenbaum.
- Lamport’s papers (free online): “Time, Clocks, and the Ordering of Events,” “Paxos Made Simple.”
- Raft paper: https://raft.github.io/raft.pdf

Closing the OS theory part

You now have: - **Process model** (formal). - **Scheduling theory** (algorithms beyond CFS). - **Synchronization theory** (Peterson, semaphores, monitors, modern atomics). - **Classic problems** (producer/consumer, dining philosophers, etc.). - **Deadlock** (prevention, detection, lockdep). - **Memory** (paging, address spaces, NUMA). - **Page replacement** (LRU and beyond). - **Filesystems** (inodes, extents, journaling, CoW). - **Crash consistency** (atomic rename, fsync rules). - **Distributed systems** (consensus, time, CAP).

That’s a real OS theory foundation.

The **next stage** of the LEARNING PATH is **Stage 9 — Linux kernel foundations** (Part V). You’re ready.

Chapter 23 — Threads, the C/C++ memory model

Multithreading is where most candidates fall down in an AMD/NVIDIA/Qualcomm interview. Memory ordering, atomics, and “what does volatile do?” come up nearly every time. This chapter and the next four make sure you can answer correctly.

23.1 What a thread is

A thread is an independent flow of execution that *shares the address space* with its peers. They each have their own:

- **registers** (including PC and SP),
- **stack**,
- **errno** (in glibc, via TLS).

They share:

- the **heap**,
- **globals**,
- **file descriptors**,
- **memory mappings**.

Two threads can read the same variable at the same time — that’s safe. If at least one writes and at least one other reads or writes, you have a **data race**, and the result is **undefined behavior** unless you use synchronization (atomics, mutexes, etc.).

23.2 pthreads

```c
#include <pthread.h>
#include <stdio.h>

void *worker(void *arg) {
    printf("hi from %ld\n", (long)arg);
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, (void *)1);
    pthread_create(&t2, NULL, worker, (void *)2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Compile: `gcc -pthread thread.c -o thread`.

API:

- `pthread_create` — start.
- `pthread_join` — wait for end + collect result.

- `pthread_detach` — give up join right; thread cleans itself up.
- `pthread_self` — get current thread id.

In modern C++ you'd write `std::thread t(worker, 1); t.join();` — the underlying call is the same.

23.3 The data race

```
int counter = 0;

void *inc(void *_) {
    for (int i = 0; i < 1000000; i++) counter++;   /* RACE */
    return NULL;
}
```

Run with two threads. Expected `2000000`. Actual: anywhere from ~1.0M to 2.0M. Why? `counter++` is three machine ops:

```
load   eax, [counter]
add    eax, 1
store  [counter], eax
```

Two threads interleave: T1 loads 5, T2 loads 5, both add to 6, both store 6 — one increment lost.

The fix: atomicity (Chapter 25) or a mutex (Chapter 24).

But the C standard goes further: a *data race* is **undefined behavior** even if "you got lucky." The compiler is allowed to assume your code has no races, and may transform racy code arbitrarily.

23.4 The C/C++ memory model

C11 and C++11 standardized a memory model: **what is observable in a program's executions across multiple threads.**

Key rule: **a program with a data race has UB.** Period.

To not have a race: every access to shared mutable data must be done via:

- a **mutex** held by all participants,
- an **atomic** with appropriate ordering,
- a **volatile** for hardware (special case; not a substitute for atomics).

The atomics offer **memory orderings**:

Order	Cost	Guarantees
`relaxed`	cheapest	only atomicity, no ordering
`acquire`	mid	reads after this, see writes that happened before the matching release
`release`	mid	writes before this become visible to a subsequent acquire
`acq_rel`	mid	both, on a RMW
`seq_cst`	most expensive	sequential consistency, total order

C11/`<stdatomic.h>`:

```
#include <stdatomic.h>
atomic_int counter = 0;

atomic_fetch_add_explicit(&counter, 1, memory_order_relaxed);
```

C++11/`<atomic>`:

```
std::atomic<int> counter{0};
counter.fetch_add(1, std::memory_order_relaxed);
counter.fetch_add(1);   /* default seq_cst */
```

For a counter you only read at the end, `relaxed` is fine. For a flag that signals "data is ready to read," you need `release` on store and `acquire` on load.

23.5 The acquire-release pattern

The single most important pattern.

Producer:

```
data = make_data();
ready.store(true, std::memory_order_release);
```

Consumer:

```
while (!ready.load(std::memory_order_acquire)) { }
use(data);
```

Release publishes prior writes; acquire ensures subsequent reads see them. Without the release/acquire, the compiler or CPU could reorder writes and `use(data)` could see uninitialized.

This pattern underlies every lock-free queue, every double-buffering scheme, every ring buffer.

23.6 Sequential consistency

`seq_cst` (the default) gives a single total order of all atomic ops across all threads. Easy to reason about. Expensive on weak-memory CPUs (ARM, POWER) because it requires full barriers. On x86, it's almost free.

For perf-sensitive code, use `acquire`/`release`/`relaxed` explicitly. For correctness-first code, use the default.

23.7 Hardware memory ordering

CPUs reorder loads/stores (within their own constraints). x86 is "TSO" (Total Store Order): loads can pass earlier stores from the same CPU only via the **store buffer**, and only with respect to *other CPUs*. Stores aren't reordered with stores; loads aren't reordered with loads.

ARM and POWER are weakly ordered: nearly any reorder is allowed unless a barrier is inserted.

`std::memory_order_*` is a **portable abstraction** over what each hardware needs. Use it; don't insert manual barriers in portable code unless you know exactly what you're doing.

The kernel uses its own families: `smp_mb()`, `smp_rmb()`, `smp_wmb()`, `barrier()`, `READ_ONCE/WRITE_ONCE`, `smp_load_acquire`/`smp_store_release`. We discuss those in Part V.

23.8 What `volatile` is and isn't

- `volatile` prevents the compiler from optimizing away or coalescing reads/writes.
- `volatile` does **not** insert any CPU barrier.
- `volatile` does **not** make a read-modify-write atomic.
- `volatile` is correct for: MMIO, signal handlers, `setjmp/longjmp` locals.

So `volatile int counter; counter++;` is still racy. Use `_Atomic int` (C) or `std::atomic<int>` (C++).

23.9 Memory model in practice — a tiny lock-free queue

A single-producer single-consumer ring buffer:

```
template <class T, size_t N>
class spsc {
    static_assert((N & (N-1)) == 0, "N must be power of 2");
public:
    bool push(const T& v) {
        size_t h = head_.load(std::memory_order_relaxed);
        size_t t = tail_.load(std::memory_order_acquire);
        if (((h + 1) & (N - 1)) == t) return false;   /* full */
        data_[h] = v;
        head_.store((h + 1) & (N - 1), std::memory_order_release);
        return true;
    }
    bool pop(T& out) {
        size_t t = tail_.load(std::memory_order_relaxed);
        size_t h = head_.load(std::memory_order_acquire);
        if (h == t) return false;                     /* empty */
        out = data_[t];
        tail_.store((t + 1) & (N - 1), std::memory_order_release);
        return true;
    }
private:
    T                       data_[N];
    std::atomic<size_t>     head_{0};   /* producer writes */
    std::atomic<size_t>     tail_{0};   /* consumer writes */
};
```

Producer publishes new `head_` with release after writing data; consumer reads with acquire to see the data. No mutex; no kernel call. The simplest non-trivial lock-free structure.

23.10 Thread-local storage

`thread_local` (C11/C++11) gives each thread its own copy of a variable.

```
        thread_local int my_id = 0;
```

Used for per-thread allocators, per-thread error state. Cheap on x86 (FS/GS segment register) but not free.

In the kernel, the equivalent is **per-CPU variables** (`DEFINE_PER_CPU`, `this_cpu_read`, `this_cpu_write`).

23.11 False sharing

Two threads write to two variables that happen to be on the same cache line. Each write invalidates the other's cache. Performance dies.

Fix: align hot per-thread/per-CPU data to a cache line:

```
        struct alignas(64) Slot { std::atomic<int> v; };
```

Kernel: `____cacheline_aligned`.

23.12 Exercises

1. Implement the racing counter from §23.3. Confirm corruption with `-O2`.
2. Replace `int counter` with `std::atomic<int>`; confirm correctness with `relaxed` ops.
3. Implement the SPSC queue. Pump 1e8 ints through it on two threads. Measure throughput.
4. Add a producer-consumer with a mutex; benchmark. Compare to lock-free.
5. Read `Documentation/memory-barriers.txt` from the kernel. It is long, dense, and the authoritative source on memory ordering for kernel work.

Next: Chapter 24 — Mutexes, condition variables, semaphores.

Chapter 24 — Mutexes, condition variables, semaphores

You will use these every day. We cover what they are, when each fits, and the dozen common bugs.

24.1 Mutex (mutual exclusion)

A mutex protects a *critical section* — a code region only one thread executes at a time.

```
#include <mutex>
std::mutex m;
int counter = 0;

void inc() {
    std::lock_guard<std::mutex> g(m);
    counter++;
}
```

`lock_guard` is RAII: it locks on construction, unlocks on destruction. Always use RAII. Manual `m.lock()/unlock()` will leak the lock on early return or exception.

Underneath: `pthread_mutex_t` on POSIX, `std::mutex` wraps it. Linux uses **futexes** — fast userspace mutexes. Uncontended `lock` / `unlock` is a single CAS, no syscall. Contended path enters the kernel via `futex_wait`.

24.2 Recursive mutexes

`std::recursive_mutex` allows the same thread to lock multiple times (and must unlock the same number). Useful when callbacks may recurse into your locked code. Kernel rule: **avoid recursion-friendly designs.** A recursive mutex usually indicates an unclear ownership model.

24.3 Reader/writer locks

`std::shared_mutex` allows multiple readers OR one writer.

```
std::shared_mutex sm;
{
    std::shared_lock<std::shared_mutex> g(sm);  /* read */
    /* … */
}
{
    std::unique_lock<std::shared_mutex> g(sm);  /* write */
    /* … */
}
```

Useful when reads vastly outnumber writes (cache lookups). Cost: a shared lock is more expensive than a mutex; on contention they often perform worse than expected.

In the kernel: `rwlock_t` (spinning) and `rw_semaphore` (sleeping). Even more relevant: **RCU** (Chapter 26).

24.4 Spin lock

A spin lock busy-waits instead of sleeping. Useful when:

- the critical section is very short,
- sleeping is forbidden (kernel atomic context).

In userspace, prefer mutex (which spins briefly then sleeps). In kernel, `spinlock_t` is the workhorse for short critical sections in IRQ context or per-CPU paths.

24.5 Deadlock

Two threads each hold a lock the other needs. Both wait forever.

Classic example:

```
T1: lock(A); lock(B); …
T2: lock(B); lock(A); …
```

Prevention rules:

1. **Lock ordering.** Always take locks in a consistent global order. Document it.
2. **Don't call user code while holding a lock.** They might call back into you and try to lock something.
3. **Limit lock scope.** Hold locks for the shortest time possible.
4. **Use `std::scoped_lock(a, b)`** (C++17) for multi-lock cases — it deadlock-avoids.

The kernel's `lockdep` runtime validator finds lock-ordering violations automatically. Run kernels with `CONFIG_PROVE_LOCKING=y` while developing.

24.6 Condition variables

A condition variable lets a thread wait for a *predicate* to become true. Always paired with a mutex.

```
std::mutex m;
std::condition_variable cv;
std::queue<int> q;

void producer() {
    {
        std::lock_guard<std::mutex> g(m);
        q.push(42);
    }
    cv.notify_one();
}

void consumer() {
    std::unique_lock<std::mutex> g(m);
    cv.wait(g, []{ return !q.empty(); });
    int v = q.front(); q.pop();
}
```

Rules:

1. **Always wait with a predicate** (`cv.wait(g, pred)`). Prevents spurious wakeups.
2. **Hold the mutex when modifying the predicate** and when calling `wait`.
3. **Notify after releasing or while holding the mutex** — both are correct, but the latter avoids the lost-wakeup race. Common practice: hold while modifying state, release, then notify.

Spurious wakeups: `wait` may return without anyone calling `notify_*`. The predicate-form loop handles this.

`notify_one` wakes one waiter; `notify_all` wakes all (then they each re-check predicate).

24.7 The producer/consumer queue

```
template <class T>
class BlockingQueue {
public:
    void push(T v) {
        {
            std::lock_guard<std::mutex> g(m_);
            q_.push(std::move(v));
        }
        cv_.notify_one();
    }
    T pop() {
        std::unique_lock<std::mutex> g(m_);
        cv_.wait(g, [&]{ return !q_.empty(); });
        T v = std::move(q_.front()); q_.pop();
        return v;
    }
private:
    std::mutex               m_;
    std::condition_variable  cv_;
    std::queue<T>            q_;
};
```

Reused everywhere. Many GPU runtime worker threads use this exact pattern.

24.8 Semaphore

A counting semaphore is a counter you can `acquire` (decrement, wait if 0) and `release` (increment, wake one). C++20 introduced `std::counting_semaphore`. Useful for resource pools and bounded queues.

```
std::counting_semaphore<8> slots(8);
slots.acquire();   /* one fewer slot */
do_thing();
slots.release();
```

In the kernel: `struct semaphore` (deprecated for new code; prefer mutex or completion).

24.9 Once and lazy init

```
std::once_flag flag;
std::call_once(flag, [&]{ heavy_init(); });
```

Guarantees `heavy_init` runs exactly once, even with concurrent callers. Replaces the double-checked-locking pattern, which is hard to get right.

Kernel: `static_branch_*`, `static_key`, simple atomic flags.

24.10 Barrier and latch (C++20)

`std::latch` — counter you decrement; threads `wait()` until counter hits 0. One-shot.

`std::barrier` — same, but resets each phase. Useful for parallel-loop synchronization.

In CUDA/HIP, `__syncthreads()` is a hardware-level analog at the warp/wave level. Do not confuse.

24.11 Common bugs

Bug	Symptom	Fix
Forgot to lock	Race; wrong values	Add lock / atomic
Held lock during signal/notify	Receiver wakes immediately, can't acquire	Notify after unlock (sometimes)
Predicate not re-checked	Spurious wakeup → wrong state	Use `cv.wait(g, pred)` form
Lock order inconsistent	Deadlock	Globally order locks; lockdep
Recursive lock with std::mutex	Self-deadlock	Use recursive_mutex or restructure
Holding lock too long	Latency spikes	Shrink critical section
Sleeping with spinlock	Kernel BUG	Drop lock first

24.12 Exercises

1. Implement `BlockingQueue<T>` from above. Test with multiple producers and consumers.
2. Implement a thread pool: N workers, a `BlockingQueue<std::function<void()>>` of tasks.
3. Implement a reader/writer simulation; benchmark `std::shared_mutex` vs `std::mutex`.
4. Construct a deadlock with two threads/two locks. Then fix with `std::scoped_lock`.
5. Read `kernel/locking/mutex.c`; identify the slow-path enter into the kernel.

Next: Chapter 25 — Atomics, lock-free data structures.

Chapter 25 — Atomics, lock-free data structures

Atomics are the building block of lock-free programming, of every mutex implementation, and of every kernel concurrency primitive. Get them right and you can write data structures that scale across cores. Get them wrong and you produce subtle bugs that show up only on weak-memory CPUs.

25.1 The atomic operations

The C++/C atomic library exposes:

- **load**, **store**, **exchange** (atomic swap),
- **fetch_add / fetch_sub / fetch_and / fetch_or / fetch_xor**,
- **compare_exchange_weak / compare_exchange_strong** (CAS — compare-and-swap).

Each can take a memory order (`relaxed`, `acquire`, `release`, `acq_rel`, `seq_cst`).

CAS is the keystone:

```
std::atomic<int> v{0};
int expected = 0;
if (v.compare_exchange_strong(expected, 1)) {
    /* succeeded: v was 0, now 1 */
} else {
    /* failed: expected was overwritten with the actual value of v */
}
```

`compare_exchange_weak` may spuriously fail (it's used in retry loops).
`compare_exchange_strong` only fails if the value differs.

Almost any lock-free algorithm decomposes into "load expected; do work; CAS to commit; on fail, retry."

25.2 The lock-free counter

```
std::atomic<int64_t> counter{0};
void inc() { counter.fetch_add(1, std::memory_order_relaxed); }
```

`fetch_add` compiles to `lock xadd` on x86 — a single locked instruction. Cost: ~10 cycles uncontended; spirals to thousands under contention from cache-line bouncing.

If you see a hot global counter, consider sharding (per-CPU/per-thread counters with a periodic rollup). That's how `vmstat`'s counters work in the kernel.

25.3 Lock-free stack (Treiber stack)

```
template <class T>
class TreiberStack {
```

```
        struct Node { T v; Node *next; };
        std::atomic<Node*> head_{nullptr};
    public:
        void push(T v) {
            Node *n = new Node{std::move(v), nullptr};
            Node *h = head_.load(std::memory_order_relaxed);
            do {
                n->next = h;
            } while (!head_.compare_exchange_weak(h, n,
                        std::memory_order_release,
                        std::memory_order_relaxed));
        }
        bool pop(T& out) {
            Node *h = head_.load(std::memory_order_acquire);
            while (h) {
                if (head_.compare_exchange_weak(h, h->next,
                        std::memory_order_acquire,
                        std::memory_order_acquire)) {
                    out = std::move(h->v);
                    delete h;       /* WARNING: ABA / use-after-free risk */
                    return true;
                }
            }
            return false;
        }
    };
```

Subtle: this works in single-threaded tests but is **broken** in the multithreaded case because of the **ABA problem**. Thread T1 reads `head=A`; T2 pops A and B, pushes A back. T1's CAS succeeds — it points head to old A's `next`, which has been freed.

Fixes:

- Hazard pointers.
- Epoch-based reclamation (used by Folly, RCU).
- Double-word CAS with a counter (limited to platforms with `cmpxchg16b`).

In the kernel, lock-free structures use **RCU** for reclamation: deleted nodes wait until all readers have passed a grace period.

> **Rule:** lock-free isn't free. Don't use it unless you've measured a lock to be the bottleneck, and you accept the complexity.

25.4 Single-producer single-consumer queue (revisited)

We saw it in Chapter 23. SPSC is the *one* lock-free pattern you can use confidently because it has no ABA: only the producer modifies head, only the consumer modifies tail.

This is exactly the model amdgpu uses for the **command ring**: CPU is producer; GPU is consumer.

25.5 The MCS / queue lock

A **mutex** that scales to many cores: each waiter spins on its own cache line; on release, the holder writes to the next waiter's line. No cache-line bouncing.

The kernel's `qspinlock` is a refined version. You don't write one — but understand the *idea*: scalable locks reduce contention by giving each waiter a distinct cache line.

25.6 Read-Copy-Update (RCU)

RCU lets multiple readers proceed concurrently with a writer, lock-free, with a small write-side cost. The trick:

1. **Readers** dereference the pointer, then assume the data is valid until they leave the read critical section.
2. **Writers** allocate a new version, install it via an atomic store, and *wait for a grace period* (until every CPU has gone through a quiescent state) before reclaiming the old.

```
struct foo *p;

/* reader */
rcu_read_lock();
struct foo *f = rcu_dereference(p);
use(f);
rcu_read_unlock();

/* writer */
struct foo *new_f = make_new();
struct foo *old = rcu_replace_pointer(&p, new_f);
synchronize_rcu();
free(old);
```

Reads are *zero-cost* on a preemption-disable kernel: `rcu_read_lock()` is `preempt_disable()`, the grace-period machinery handles waiting elsewhere. Writes pay a synchronization cost.

Used pervasively in: VFS, networking, the route cache, every list with read-mostly access. The amdgpu IRQ vector list uses RCU.

Read `Documentation/RCU/whatisRCU.rst`. RCU questions show up in kernel interviews.

25.7 Hazard pointers (briefly)

A reader publishes a “hazard” pointer to a memory address it’s reading; writers check the global hazard list before reclaiming. Used in concurrent libraries (Folly, ROCm runtime in places). RCU is generally simpler in the kernel.

25.8 Memory barriers in the kernel

Userland uses C++ atomics with memory orders. The kernel uses macros:

- `READ_ONCE(x)` / `WRITE_ONCE(x, v)` — atomic load/store on a plain word, prevents tearing and certain compiler reorderings.
- `smp_rmb()` / `smp_wmb()` / `smp_mb()` — runtime CPU read/write/full barriers (SMP-only; UP-no-op).
- `smp_load_acquire(p)` / `smp_store_release(p, v)` — analogous to C++ acquire/release.
- `barrier()` — compiler barrier only.

For atomics: `atomic_t`, `atomic64_t`. Operations: `atomic_inc`, `atomic_add`, `atomic_cmpxchg`, `atomic_dec_and_test`. Most are **fully ordered** by default (unlike C++); explicit relaxed variants exist as `atomic_*_relaxed`.

For reference counting, kernel uses `refcount_t` (with overflow check):

```
refcount_t r;
refcount_inc(&r);
if (refcount_dec_and_test(&r)) free(obj);
```

25.9 The `seq_lock`

Optimistic concurrency. Reader checks a sequence number before and after; if it changed, retry. Writer increments the sequence (odd = in progress). No reader wait, no shared cacheline writes from readers.

```
unsigned seq;
do {
    seq = read_seqbegin(&lock);
    /* read snapshot */
} while (read_seqretry(&lock, seq));
```

Used in `xtime` (gettimeofday) and a few other read-heavy paths.

25.10 Practical guidance

For our work:

1. **Default to a lock.** A mutex or spinlock is correct, simple, and 99% of the time fast enough.
2. **Use RCU** for read-mostly lookups (a list of registered objects, for example).
3. **Use SPSC ring buffers** for producer/consumer where you know the topology (CPU↔GPU command ring).
4. **Reach for full lock-free with CAS** only when the above three don't fit. Document carefully and add stress tests.
5. **Run with TSan / KCSAN** during development.

25.11 Exercises

1. Build the Treiber stack. Run with two threads doing 1M push/pop each. Check against ASan/TSan.
2. Demonstrate the ABA problem: pop A, pop B, free A, push A back. Show the bug.
3. Implement a tagged-pointer CAS (low bits as counter) to fix ABA. Run again under TSan.
4. Implement a per-CPU sharded counter; benchmark vs. a single atomic counter on 16 threads.
5. Read `kernel/rcu/tree.c::synchronize_rcu`; understand at a high level the grace-period detection.

Next: Chapter 26 — The Linux kernel concurrency toolbox.

Chapter 26 — The Linux kernel concurrency toolbox

The kernel has its own concurrency primitives, optimized for kernel constraints (no general malloc in IRQ context, preemption rules, hardirq vs. softirq vs. process). This chapter is the cheat-sheet of which to use when.

26.1 Spinlock

```
spinlock_t lock;
spin_lock_init(&lock);
spin_lock(&lock);
/* … critical section … */
spin_unlock(&lock);
```

- Spins (busy-waits) — never sleeps.
- Disables **preemption** while held.
- Variants:
  - `spin_lock_irq` — disable IRQs (use only in known-non-IRQ context).
  - `spin_lock_irqsave(&l, flags)` / `spin_unlock_irqrestore(&l, flags)` — save and disable IRQs, restore on unlock. Use whenever the same lock might be taken from IRQ context.
  - `spin_lock_bh` / `spin_unlock_bh` — disable softirqs.

Critical-section rules:

- **No sleeping**, ever. No `kmalloc(GFP_KERNEL)`, no `mutex_lock`, no `copy_to_user`.
- **Hold for microseconds.**
- **Don't acquire two unrelated spinlocks** without a documented order.

Use spinlock when: small critical sections; hot paths; required from IRQ.

26.2 Mutex

```
struct mutex m;
mutex_init(&m);
mutex_lock(&m);
/* … critical section … */
mutex_unlock(&m);
```

- May sleep — process context only.
- Cannot be used from hardirq, softirq, or with interrupts disabled.
- Variants: `mutex_trylock`, `mutex_lock_interruptible` (returns `-EINTR` if signaled).

Use mutex when: critical section may sleep, do I/O, allocate `GFP_KERNEL`, or is long.

26.3 Semaphore (rare in modern kernel)

`struct semaphore`. Counting. `down`, `up`. Mostly replaced by `struct mutex` for binary use. New code rarely uses.

26.4 Completion

A one-shot "wait until done" primitive. Cleaner than condvars for "wait for IRQ to fire" or "wait for thread to finish" patterns.

```
struct completion done;
init_completion(&done);

/* waiter */
wait_for_completion(&done);

/* completer (often in IRQ) */
complete(&done);
```

`complete_all(&done)` wakes all waiters and the next `wait_for_completion` returns immediately (until reinit).

amdgpu uses completions for one-shot reset and suspend/resume sync points.

26.5 Wait queues

Generalized "wait until predicate." More flexible than completion.

```
DECLARE_WAIT_QUEUE_HEAD(wq);
int condition = 0;

/* waker */
condition = 1;
wake_up(&wq);

/* waiter */
wait_event_interruptible(wq, condition);
```

Use waitqueues for arbitrary wait-on-state patterns.

26.6 Reader-writer locks

- `rwlock_t` — spinning rwlock; prefer never; cache-thrashes on contention.
- `struct rw_semaphore` — sleeping rwsem; useful for read-heavy data.

For most cases prefer **RCU** instead.

26.7 RCU primitives

```
struct foo *p = some_global;

/* readers */
rcu_read_lock();
struct foo *q = rcu_dereference(p);
use(q);
rcu_read_unlock();
```

```
        /* writers */
        struct foo *new_p = kmalloc(sizeof(*new_p), GFP_KERNEL);
        *new_p = ...;
        rcu_assign_pointer(p, new_p);
        synchronize_rcu();
        kfree(old_p);
```

Lists & hash:

- `list_for_each_entry_rcu(pos, head, member)` — read traversal.
- `list_add_rcu`, `list_del_rcu` — write side.

RCU works because the kernel knows the readers go through *quiescent states* (return to user, schedule, idle) at which they cannot hold an old pointer.

26.8 Atomic ops in the kernel

```
        atomic_t a = ATOMIC_INIT(0);
        atomic_inc(&a);
        atomic_dec(&a);
        atomic_read(&a);
        atomic_set(&a, 5);
        int old = atomic_xchg(&a, 1);
        int prev = atomic_cmpxchg(&a, 0, 1);
```

64-bit: `atomic64_t`.

Bit ops: `set_bit`, `clear_bit`, `test_and_set_bit`, `change_bit`. Atomic by default.

Refcounts: `refcount_t` (for safety, refuses to overflow), `kref` (RAII-ish ref structure with put callback).

26.9 Per-CPU variables

```
        DEFINE_PER_CPU(int, my_counter);
        get_cpu_var(my_counter)++;
        put_cpu_var(my_counter);
```

Or:

```
        this_cpu_inc(my_counter);     /* preempt-safe shorthand */
```

Used to scale: each CPU writes its own copy; aggregate later. amdgpu uses per-CPU caches in some allocator paths.

26.10 Memory barriers

Barrier	Effect
`barrier()`	compiler only
`smp_rmb()`	runtime read barrier (SMP)
`smp_wmb()`	runtime write barrier
`smp_mb()`	full barrier
`smp_load_acquire(p)`	acquire-load

Barrier	Effect
`smp_store_release(p, v)`	release-store
`mb()`, `rmb()`, `wmb()`	unconditional (for MMIO)

Pair acquires with releases. Most subsystems prefer `smp_load_acquire` / `smp_store_release` over raw barriers.

26.11 `lockdep`

Run-time detector for:

- Lock-order inversion.
- Recursive lock attempts.
- Sleeping with spinlock.
- Misuse of irq-safe vs irq-unsafe locks.

Enable with `CONFIG_PROVE_LOCKING=y`. Whenever you boot a development kernel, **enable lockdep**. Many subtle deadlocks become detectable on the first run.

26.12 KCSAN

Kernel Concurrency Sanitizer. Like TSan for the kernel. `CONFIG_KCSAN=y`. It finds data races on shared memory.

26.13 Practical patterns

- **Per-device state protected by one mutex.** Default starting point.
- **Fast path read with RCU; slow update with mutex.** For lookups (registered objects, etc.).
- **IRQ → workqueue with spinlock between.** For interrupt-driven event delivery.
- **Atomic counter for `n_users`; mutex for invariants.** A common combo.
- `refcount_t`, `kref`, `kref_get`, `kref_put` for object lifetime.

26.14 Exercises

1. Write a tiny kernel module with a `spinlock_t` protecting a global counter. Increment from a kthread; ensure no race (atomics would be cleaner — practice both).
2. Add a workqueue handler that processes items from a queue protected by a mutex.
3. Set up an `rcu_read_lock` reader and an `rcu_assign_pointer + synchronize_rcu + kfree` writer over a small list. Watch with `lockdep` to confirm no warnings.
4. Boot your dev kernel with `CONFIG_PROVE_LOCKING=y` and provoke a lock-order inversion deliberately. Read the lockdep splat in dmesg.
5. Read `Documentation/locking/locktypes.rst`. It is the official summary and matches this chapter closely.

Next: Chapter 27 — GPU concurrency: fences, queues, doorbells.

Chapter 27 — GPU concurrency: fences, queues, doorbells

GPU programming is *fundamentally* a multithreaded problem: thousands of GPU lanes, several command queues, multiple engines (graphics, compute, copy), and an asynchronous host. The synchronization primitives you've learned in CPU-land map onto GPU-land but with new names. Internalizing this map is the gateway into Part VII.

27.1 The mental model

The CPU produces work; the GPU consumes it.

```
┌─────────┐        submit cmd          ┌──────────┐
│  CPU    │ ──────────────────────►    │   Ring   │ ─► GPU CP fetches
│  app    │                            │  buffer  │
└─────────┘                            └──────────┘
   │                                       │
   │              completion               │
   │       ◄──────────────────────────────┘
   ▼
 fence signaled
```

The **ring buffer** is a shared memory queue between CPU (producer) and GPU (consumer). The CPU writes commands into the ring and updates the **write pointer**. The GPU's command processor (CP) reads from the **read pointer** and advances. Same SPSC pattern as Chapter 23.

27.2 Doorbells

The GPU might be napping. To kick it: ring a "doorbell" — write to a special MMIO/PCIe address that wakes the CP and tells it new work is available. Doorbells are a *notification* mechanism, not data.

amdgpu maps a doorbell page to userspace for fast userspace submission (userspace queues / KFD). When userspace writes the new write-pointer value to that page, the GPU is notified directly without a syscall. Each process has its own doorbell page (protected by IOMMU + the GPU's own per-process VM).

27.3 Fences

A **fence** is a "this work has completed" signal. The CPU waits on a fence; multiple GPU operations can have multiple fences. Fences are the GPU equivalent of C++ futures.

In Linux, the universal kernel-side fence type is `struct dma_fence`:

```
struct dma_fence {
    spinlock_t                *lock;
    const struct dma_fence_ops *ops;
    u64                        seqno;
    unsigned long              flags;
```

```
        /* … */
    };
```

`dma_fence` has methods:

- `dma_fence_wait()` — block until signaled.
- `dma_fence_signal()` — mark as signaled; wake waiters.
- `dma_fence_add_callback()` — register a callback to fire on signal.
- `dma_fence_get()` / `dma_fence_put()` — refcount.

Drivers implement the `ops` (`enable_signaling`, `signaled`, `wait`, `release`) for their fence subclass.

In amdgpu: `struct amdgpu_fence` wraps a hardware sequence number written into a memory location by the GPU. When the value reaches a target, the fence is signaled.

27.4 Sync objects (`drm_syncobj`)

A **drm_syncobj** is a userspace handle to a fence. It allows fences to flow through ioctls and across drivers. Vulkan's semaphores are usually backed by syncobjs; modern Mesa uses `DRM_IOCTL_SYNCOBJ_*` heavily.

Two flavors:

- **Binary syncobj** — like a single fence: signaled or not.
- **Timeline syncobj** — a monotonically increasing 64-bit value; "wait for value ≥ N" (Vulkan timeline semaphores).

27.5 dma-buf and cross-driver sharing

A `dma-buf` is a kernel object representing memory shared between drivers. amdgpu can export a dma-buf, V4L2 (camera) can import it; zero-copy. Each dma-buf carries a list of fences (for synchronization across drivers).

Vulkan external memory and Wayland zero-copy clients use dma-bufs.

27.6 Memory ordering CPU↔GPU

The CPU's writes to system memory are not magically visible to the GPU. Three ordering domains:

1. **CPU caches** — the CPU's writes might still be in store buffers or L1.
2. **PCIe write buffers** — even after the writes hit DRAM, the GPU might have buffered them in its own memory controller.
3. **GPU's own caches** — the GPU has L1, L2 caches that may be stale relative to its memory controller.

To make a CPU write visible to the GPU:

- Write to memory.
- `wmb()` (kernel) — ensures store ordering.
- Issue a doorbell (an MMIO write) — this acts as a heavy barrier on PCIe.
- The GPU CP, on processing, may need to invalidate its L2 (via a packet like `KCACHE_INVALIDATE`).

To make a GPU write visible to the CPU:

- GPU executes a `RELEASE_MEM` packet (writes a fence value to a known memory address with a flush).
- CPU reads the fence; on seeing the value, the corresponding data is also globally visible.

This is the per-engine version of acquire/release.

In amdgpu's `IB` (Indirect Buffer) submission, the prologue/epilogue contains cache invalidate / writeback packets. They are not optional. If you forget them, you get random graphical corruption — the most painful class of GPU bugs.

27.7 Coherent vs. write-combining vs. uncached for GPU

CPU-mapped GPU memory has three common attributes:

Attribute	Use
WB (write-back, cacheable)	rare for GPU memory; risk of CPU cache inconsistency
WC (write-combining)	command buffers, vertex buffers — write-only from CPU
UC (uncached)	doorbells, MMIO registers, fence read-back

GTT memory in amdgpu is often allocated as WC. Carving up: `_PAGE_PCD | _PAGE_PWT` etc. (all hidden behind `ioremap_wc` / `pgprot_writecombine`).

When you do `mmap` on a GPU buffer in userspace, the kernel chooses page protection based on the buffer's flags. Choose wrong, you corrupt data.

27.8 GPU scheduler in the kernel

The DRM scheduler (`drivers/gpu/drm/scheduler/sched_main.c`) sits between userspace submissions and hardware queues:

- Each hardware queue has a `drm_gpu_scheduler`.
- Each open file context has a `drm_sched_entity`.
- Submitting a job creates a `drm_sched_job` with input fences (deps) and an output fence.
- Scheduler picks ready jobs, respects round-robin or priority across entities.

We dedicate Chapter 50 to this. It is one of the trickiest pieces of code to read in amdgpu and one of the most-asked-about in interviews.

27.9 Memory operations from the GPU's perspective

The GPU has its own MMU (the GPUVM). Each process's GPU buffers are mapped at a per-process GPU virtual address. Buffers can live in:

- **VRAM** (GPU local memory).
- **GTT** (system memory mapped via the IOMMU, accessible to both CPU and GPU).
- **CPU pinned pages** (via `userptr` BOs — the kernel pins user memory and maps it into GPU VM).

Migrating between these is expensive. TTM (Chapter 46) is the placement engine.

27.10 Practical takeaways for our future driver work

1. CPU↔GPU is a producer-consumer system; SPSC ring buffers, fences, doorbells.
2. Memory barriers between CPU and GPU are explicit; the OS APIs hide most.
3. Most GPU bugs are concurrency bugs in disguise — wrong fence ordering, wrong cache flush, wrong placement.
4. Userspace drivers do their own scheduling, but the **kernel** scheduler exists to enforce isolation between processes.

27.11 Exercises

1. Read `include/linux/dma-fence.h` from the kernel. Identify the ops, the flags, the seqno.
2. Read `drivers/gpu/drm/scheduler/sched_main.c::drm_sched_main` once. Don't worry about understanding everything; pick out the loop and the call to `run_job`.
3. Build a userspace SPSC queue between two threads using shared memory and futex-based fences. Note how it mirrors a GPU command ring.
4. With libdrm, write a tiny program that opens `/dev/dri/card0` and queries `DRM_IOCTL_AMDGPU_INFO`. We'll use libdrm in Part VII.

End of Part IV. Move to Part V — Linux kernel foundations.

Chapter 28 — The kernel source tree and how to build it

If you can't build, boot, and modify a kernel, you can't write drivers. This chapter walks you from "no kernel checked out" to "I just booted my own bzImage with `printk("hello\n")` in `start_kernel`."

28.1 The tree

```
git clone --depth 1 \
    https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux
ls
```

The top-level dirs:

Dir	Contents
`arch/`	per-arch code (x86, arm64, riscv, ...)
`block/`	block layer
`crypto/`	kernel crypto
`drivers/`	every device driver — including `drivers/gpu/drm/amd/`
`fs/`	filesystems and VFS
`include/`	public kernel headers
`init/`	start_kernel, early boot
`ipc/`	sysv ipc, shm, semaphores
`kernel/`	scheduler, time, lockdep, RCU, signals
`lib/`	data structures, sort, etc.
`mm/`	memory management
`net/`	networking
`samples/`	example modules
`scripts/`	build helpers, checkpatch, etc.
`security/`	LSM, SELinux, AppArmor
`sound/`	ALSA
`tools/`	userspace tools (perf, bpftool)
`Documentation/`	hundreds of essential docs

For our work the deep targets are:

- `drivers/gpu/drm/` (DRM core)
- `drivers/gpu/drm/scheduler/` (DRM scheduler)
- `drivers/gpu/drm/ttm/` (TTM memory manager)
- `drivers/gpu/drm/amd/amdgpu/` (~800k lines of C)

- `drivers/gpu/drm/amd/display/` (DC — display core)
- `drivers/gpu/drm/amd/pm/` (power management)
- `drivers/gpu/drm/amd/include/` (hardware register defs)

You'll spend years here.

28.2 Configure

Pick a starting config:

```
make defconfig                    # generic
make x86_64_defconfig
make olddefconfig                 # bring an old .config forward
make menuconfig                   # interactive ncurses
```

Typical recipe for development:

```
cp /boot/config-$(uname -r) .config
make olddefconfig
scripts/config --enable DEBUG_KERNEL \
               --enable DEBUG_INFO \
               --enable PROVE_LOCKING \
               --enable DEBUG_ATOMIC_SLEEP \
               --enable KASAN \
               --enable KASAN_GENERIC \
               --enable DEBUG_VM \
               --enable DEBUG_LIST \
               --enable DRM_AMDGPU
make olddefconfig
```

Keep amdgpu **=m** (module) if you want to load/unload during dev:

```
scripts/config --module DRM_AMDGPU
```

28.3 Build

```
make -j$(nproc)
```

Output:

- `vmlinux` — the kernel itself, ELF.
- `arch/x86/boot/bzImage` — compressed kernel for boot.
- `*.ko` — modules.

To install (real machine):

```
sudo make modules_install
sudo make install              # copies bzImage, runs update-grub
```

Then reboot. Pick the new kernel from GRUB. `uname -a` confirms.

28.4 Boot in a VM (faster + safer)

Strongly recommended for development. QEMU + a small rootfs:

```
qemu-system-x86_64 \
    -kernel arch/x86/boot/bzImage \
    -append "console=ttyS0 nokaslr" \
    -nographic \
    -enable-kvm \
    -m 4G -smp 4 \
    -initrd ../rootfs.cpio
```

Boot. You're in your kernel.

For amdgpu on QEMU, you need GPU passthrough (VFIO) which is more work. Easier path: build amdgpu, install to your real Linux box, reboot.

28.5 The first patch you write

Find `init/main.c::start_kernel()`. Add:

```
asmlinkage __visible void __init __no_sanitize_address start_kernel(void)
{
    pr_info("hello from my custom kernel\n");
    /* … existing body … */
}
```

Rebuild, reboot, `dmesg | head` — your message is at the top. You have just modified the world's most-deployed C codebase.

28.6 Out-of-tree module

For driver development, build *your* module out-of-tree against the installed kernel:

```
sudo apt install linux-headers-$(uname -r)
mkdir -p /tmp/hello && cd /tmp/hello
```

`hello.c`:

```
#include <linux/init.h>
#include <linux/module.h>

static int __init hello_init(void) { pr_info("hello, world\n"); return 0; }
static void __exit hello_exit(void) { pr_info("goodbye, world\n"); }

module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("you");
MODULE_DESCRIPTION("hello world");
```

`Makefile`:

```
obj-m := hello.o
KDIR := /lib/modules/$(shell uname -r)/build
PWD  := $(shell pwd)

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Build and load:

```
make
sudo insmod hello.ko
dmesg | tail
sudo rmmod hello
dmesg | tail
```

If `dmesg` shows your messages, you've successfully:

- Written a kernel module.
- Built it against your running kernel's headers.
- Loaded and unloaded it.

The hardest barrier in driver development is past.

28.7 Reading the source — tools

- `cscope -R -k` then `cscope` — cross-reference jump.
- `git grep -n SYMBOL` — fast grep.
- `make tags` then vim `:tag SYMBOL`.
- LSP: `clangd` with `bear -- make -j`. Modern editors (vscode/cursor) work with the kernel.
- `kernel.org/doc/html/latest/` — official docs.
- `lwn.net/Kernel/Index/` — articles by topic.

28.8 Patch flow

The kernel uses git, plain mailing list patches (not GitHub PRs):

```
git checkout -b my-fix
# … edit …
git add -p
git commit -s -m "drivers/foo: fix bar"
git format-patch -1 -o patches/
scripts/checkpatch.pl patches/0001-*.patch
scripts/get_maintainer.pl patches/0001-*.patch
git send-email patches/0001-*.patch --to=<maintainers> --cc=<lists>
```

`-s` adds your `Signed-off-by`. `checkpatch.pl` enforces style. `get_maintainer.pl` tells you whom to email.

For amdgpu the relevant list is `amd-gfx@lists.freedesktop.org`. Subscribe.

28.9 Exercises

1. Clone, configure, build, install, boot a custom kernel. Verify with `uname -a`.
2. Add a `pr_info` to `start_kernel`; observe in dmesg.
3. Build the `hello.ko` from §28.6. Insmod, rmmod, observe dmesg.
4. Run `scripts/get_maintainer.pl drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c`. Note the names.
5. Skim `Documentation/process/submitting-patches.rst`. You will follow this when you submit your first amdgpu patch.

Next: Chapter 29 — Loadable kernel modules — your first module.

Chapter 29 — Loadable kernel modules: your first module

A module is a relocatable object (`.ko`) the kernel loads at runtime. We extend the hello-world from Chapter 28 to teach the full module API: parameters, init/exit ordering, dependencies, exported symbols.

29.1 Anatomy of a module

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/moduleparam.h>

static int  burst = 1;
static char *name = "world";

module_param(burst, int, 0644);
MODULE_PARM_DESC(burst, "how many greetings");
module_param(name, charp, 0644);
MODULE_PARM_DESC(name, "who to greet");

static int __init mod_init(void)
{
    int i;
    for (i = 0; i < burst; i++)
        pr_info("hello, %s (%d)\n", name, i);
    return 0;
}

static void __exit mod_exit(void)
{
    pr_info("goodbye, %s\n", name);
}

module_init(mod_init);
module_exit(mod_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("you");
MODULE_DESCRIPTION("greeter");
MODULE_VERSION("0.1");
```

Build with the Makefile from §28.6.

```
sudo insmod hello.ko name=AMD burst=3
dmesg | tail
sudo rmmod hello
```

You can also pass params via modprobe.conf or the kernel command line.

29.2 `__init` and `__exit` annotations

`__init` and `__exit` are *section attributes*. They place the function in special sections (`.init.text` , `.exit.text`). After init runs, the kernel **frees** the `.init.text` memory. So:

- `__init` functions cannot be called after init.
- `__init` data is free'd too.

For modules, the rules are similar but `__exit` may be elided if the module is built-in (kernel can't unload built-in modules).

`__initdata` for data, `__exitdata` for exit-only data.

29.3 The `printk` levels

```
        pr_emerg(...), pr_alert(...), pr_crit(...), pr_err(...),
        pr_warn(...), pr_notice(...), pr_info(...), pr_debug(...)
```

`pr_debug` is compiled out unless `DEBUG` is defined or `CONFIG_DYNAMIC_DEBUG` is on. `dmesg` shows everything; userspace klogd may filter to syslog.

In drivers, `dev_*` versions tag with the device:

```
        dev_info(dev, "frobbing %d\n", n);
```

29.4 Module dependencies

If your module calls a function exported by another module, modprobe loads the dependency first. The build records dependencies via `EXPORT_SYMBOL` of called functions.

```
        EXPORT_SYMBOL(my_helper);
        EXPORT_SYMBOL_GPL(my_gpl_helper);   /* GPL-only */
```

`modinfo hello.ko | grep depends` shows the result.

29.5 Symbol naming and collisions

All exported symbols share a global namespace. Conflicts result in module-load failure. Prefix everything with your module name (e.g. `amdgpu_*`). Use `static` for internal helpers.

29.6 Module parameters

```
        module_param_array(name, type, &count, perm);
        module_param_named(public_name, internal_var, type, perm);
        module_param_string(name, buf, sizeof(buf), perm);
```

Permissions: 0 = no sysfs file; 0644 = world-readable, root-writable. Read in `/sys/module/<name>/parameters/<param>` . Writeable params can be changed at runtime.

amdgpu has many params (`amdgpu_dpm` , `amdgpu_pcie_gen2` , `amdgpu_runpm`). Read `drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c` once.

29.7 Auto-loading via `MODULE_DEVICE_TABLE`

Drivers tell the kernel which devices they bind to via a table:

```
static const struct pci_device_id mydrv_ids[] = {
    { PCI_DEVICE(0x1002, 0x73a3) },   /* example AMD device */
    { },
};
MODULE_DEVICE_TABLE(pci, mydrv_ids);
```

The build system extracts these into `modules.alias`. udev matches devices to aliases on hotplug and runs `modprobe`.

29.8 Init order

The kernel's `start_kernel` runs `do_initcalls` in order:

```
core_initcall      — early
postcore_initcall
arch_initcall
subsys_initcall
fs_initcall
device_initcall    — module_init defaults here
late_initcall
```

For drivers, `module_init` becomes `device_initcall`. If you need earlier (e.g. you provide a clock or bus), use `subsys_initcall(my_init)`.

29.9 The `THIS_MODULE` and refcount

```
.owner = THIS_MODULE,
```

Each `struct file_operations` should have an owner so that modules with open file descriptors aren't unloaded out from under users. `try_module_get` / `module_put` increments/decrements the count.

29.10 Loading and unloading machinery

Userspace tools:

- `insmod` — load a `.ko` file directly.
- `modprobe` — resolve dependencies and load.
- `rmmod` — unload.
- `lsmod` — list.
- `modinfo` — print metadata.

Module loading goes through `init_module(2)` and `finit_module(2)` syscalls. The `finit_module` variant takes a file descriptor instead of a buffer (preferred since 3.8).

29.11 A non-trivial module: chardev skeleton

We won't fully implement a char device until Chapter 35, but here's the bone:

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/uaccess.h>

#define DEVNAME "skel"

static dev_t           dev_no;
static struct cdev     cdev;
static struct class   *cls;

static int     skel_open(struct inode *i, struct file *f)  { return 0; }
static int     skel_release(struct inode *i, struct file *f){ return 0; }
static ssize_t skel_read(struct file *f, char __user *u, size_t n, loff_t *o) {
    return 0;
}
static ssize_t skel_write(struct file *f, const char __user *u, size_t n, loff_t *o) {
    return n;
}

static const struct file_operations skel_fops = {
    .owner   = THIS_MODULE,
    .open    = skel_open,
    .release = skel_release,
    .read    = skel_read,
    .write   = skel_write,
};

static int __init skel_init(void)
{
    int ret = alloc_chrdev_region(&dev_no, 0, 1, DEVNAME);
    if (ret) return ret;

    cdev_init(&cdev, &skel_fops);
    ret = cdev_add(&cdev, dev_no, 1);
    if (ret) goto unreg;

    cls = class_create(DEVNAME);
    if (IS_ERR(cls)) { ret = PTR_ERR(cls); goto del; }
    device_create(cls, NULL, dev_no, NULL, DEVNAME);
    return 0;

del:    cdev_del(&cdev);
unreg:  unregister_chrdev_region(dev_no, 1);
    return ret;
}

static void __exit skel_exit(void)
{
    device_destroy(cls, dev_no);
    class_destroy(cls);
    cdev_del(&cdev);
    unregister_chrdev_region(dev_no, 1);
}

module_init(skel_init);
module_exit(skel_exit);
MODULE_LICENSE("GPL");
```

Insmod, then `ls -l /dev/skel`. You created a real device file. Read returns 0; write returns n. We'll fill in the body in Chapter 35.

29.12 Exercises

1. Write the param-greeter from §29.1; run with multiple param values; observe.

2. Build the chardev skeleton; verify `/dev/skel` appears; `cat /dev/skel` returns nothing; `echo hi > /dev/skel` accepts the bytes.
3. Add an `EXPORT_SYMBOL(my_func)` and a second module that calls it; observe the dependency in `modinfo`.
4. Examine `lsmod | grep amdgpu`; identify dependencies (`drm_kms_helper`, `drm`, `iommu_v2`, etc.).
5. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c::amdgpu_init`. Recognize `module_param`, `module_init` patterns.

Next: Chapter 30 — Kernel data structures: lists, rbtrees, xarray, idr.

Chapter 30 — Kernel data structures: lists, rbtrees, xarray, idr

The kernel does not use STL. It has its own intrusive data structures, optimized for in-place embedding, no allocation in the hot path, and zero RTTI. These show up everywhere in amdgpu.

30.1 The intrusive doubly-linked list

`struct list_head` is the most-used data structure in the kernel.

```
struct list_head { struct list_head *next, *prev; };
```

It is meant to be **embedded** in your struct:

```
struct obj {
    int data;
    struct list_head node;
};

LIST_HEAD(my_list);    /* declares + inits an empty list head */

struct obj *o = kmalloc(sizeof *o, GFP_KERNEL);
o->data = 5;
INIT_LIST_HEAD(&o->node);
list_add(&o->node, &my_list);
```

To iterate:

```
struct obj *o;
list_for_each_entry(o, &my_list, node) {
    pr_info("%d\n", o->data);
}
```

Macros:

- `list_add(new, head)` — front.
- `list_add_tail(new, head)` — back.
- `list_del(node)` — remove.
- `list_empty(head)` — is empty.
- `list_first_entry(head, type, member)` — first.
- `list_entry(ptr, type, member)` — same as `container_of`.
- `list_for_each_entry_safe(pos, n, head, member)` — safe to delete current.
- `list_splice` — merge lists.

The trick is `container_of`: `list_for_each_entry` walks `list_head` pointers but yields `struct obj *` because it computes the offset at compile time.

amdgpu uses lists for: BO lists, fence lists, IRQ source lists, scheduler entities, idle queues.

30.2 hlist — hashed linked list

`struct hlist_head` is a single-pointer head for buckets in hash tables. Same idea as `list_head`, smaller memory.

`include/linux/hashtable.h` provides static-size hash tables:

```
DEFINE_HASHTABLE(table, 8);    /* 256 buckets */
hash_init(table);
hash_add(table, &obj->node, obj->key);
struct obj *o;
hash_for_each_possible(table, o, node, key) {
    if (o->key == key) /* match */;
}
```

30.3 Red-black trees (`rb_tree`)

For ordered lookups, the kernel has `struct rb_root` and `struct rb_node`. Self-balancing BST.

```
struct obj { int key; struct rb_node node; };

struct rb_root tree = RB_ROOT;

static int insert(struct obj *o) {
    struct rb_node **p = &tree.rb_node, *parent = NULL;
    while (*p) {
        struct obj *t = rb_entry(*p, struct obj, node);
        parent = *p;
        if (o->key < t->key) p = &(*p)->rb_left;
        else if (o->key > t->key) p = &(*p)->rb_right;
        else return -EEXIST;
    }
    rb_link_node(&o->node, parent, p);
    rb_insert_color(&o->node, &tree);
    return 0;
}
```

A bit verbose. The kernel doesn't generate templates, so each user repeats the pattern. amdgpu uses rbtrees for VM (GPU virtual memory) range lookups.

30.4 Interval trees

Augmented rb-tree for `[start, last]` intervals. `include/linux/interval_tree.h`. Used for VM area overlaps. amdgpu's `amdgpu_vm.c` uses this for buffer-object placements in GPU virtual memory.

30.5 IDR / IDA / xarray

Sparse arrays keyed by integers — useful for "give me a small integer ID for this pointer."

- **`struct ida`** — allocates IDs (just bits).
- **`struct idr`** — pointer ↔ id mapping (older API).
- **`struct xarray`** — modern replacement for `idr` and the radix tree.

```
DEFINE_XARRAY(my_xa);
```

```
int id = xa_alloc(&my_xa, &id, ptr, XA_LIMIT(0, 1024), GFP_KERNEL);
void *p = xa_load(&my_xa, id);
xa_erase(&my_xa, id);
```

GPU drivers use IDR/xarray to give userspace handles to kernel objects. DRM core itself uses them for object handles (BOs, syncobjs, contexts).

30.6 Lookup-and-insert atomic patterns

```
xa_lock(&xa);
void *p = xa_load(&xa, id);
if (!p) {
    p = make();
    __xa_store(&xa, id, p, GFP_KERNEL);
}
xa_unlock(&xa);
```

`xa_lock` is internal; use it for compound operations. For simple ops, the atomic variants suffice.

30.7 Bitmaps (review)

`DECLARE_BITMAP(map, N)`, `set_bit`, `clear_bit`, `find_first_zero_bit`. From Chapter 6, but worth restating: when a kernel struct needs to track "which N things are in use" cheaply, a bitmap is the right answer.

30.8 The `kfifo`

A kernel SPSC ring buffer:

```
DECLARE_KFIFO(my_fifo, u8, 4096);

INIT_KFIFO(my_fifo);
kfifo_in(&my_fifo, buf, n);
kfifo_out(&my_fifo, buf, n);
```

Useful when one context produces and one consumes — typical of IRQ → workqueue paths.

30.9 The `llist` — lock-less list

`struct llist_head`, `llist_add`, `llist_del_all`. A *singly-linked* lock-free list using CAS. Fast for "many writers, single drainer." Used in workqueue internals and elsewhere.

30.10 Reference counting

- **`refcount_t`** — saturating refcount with overflow detection.
- **`struct kref`** — `refcount_t` plus a `release` callback pattern.

```
struct foo {
    struct kref ref;
    /* … */
};
```

```
static void foo_release(struct kref *kref) {
    struct foo *f = container_of(kref, struct foo, ref);
    kfree(f);
}

kref_get(&f->ref);
kref_put(&f->ref, foo_release);
```

Used everywhere ownership is shared. `dma_fence` uses kref. `drm_gem_object` uses kref. `pci_dev` uses kref.

30.11 Putting it together — driver-internal indices

A typical driver:

```
struct mydrv {
    struct mutex     lock;
    struct list_head bo_list;          /* all BOs */
    struct xarray    handles;          /* uint32 → BO  */
    struct rb_root   vm_tree;          /* GPU VM */
    DECLARE_BITMAP(free_qids, 64);     /* which queue IDs free */
};
```

This is a *mini-amdgpu* in five lines. Almost every kernel subsystem composes from these primitives.

30.12 Exercises

1. Build a small kernel module that creates 10 `struct obj` with `list_head node` and `int data`. Insert into a list, sort by data using `list_sort`, iterate, free.
2. Build a `struct rb_root` of `(key, value)` pairs. Insert, lookup, delete. Print in order.
3. Use an `xarray` to map small integer handles to dynamically-allocated objects. Allocate IDs, look up, release.
4. Use `kfifo` between a kthread (producer) and a workqueue (consumer); pump 1M bytes through.
5. Read `include/linux/list.h`. It is short and beautiful. Memorize the pattern.

Next: Chapter 31 — Kernel memory: kmalloc, vmalloc, slab, page allocator.

Chapter 31 — Kernel memory: kmalloc, vmalloc, slab, page allocator

The kernel has its own allocators with their own rules. Picking the wrong one leaks memory, crashes the system, or DMAs garbage. This chapter is short, concrete, and one you'll re-read often.

31.1 The hierarchy

```
                    ┌──────────────────────────┐
                    │      Buddy allocator     │   physical pages
                    │  alloc_pages, free_pages │   1, 2, 4, …, 1024 pgs
                    └──────────────────────────┘
                                 │
                ┌────────────────┼────────────────┐
                ▼                                 ▼
   ┌──────────────────┐                  ┌──────────────────┐
   │ Slab / SLUB / SLOB │                  │     vmalloc      │
   │  caches of fixed  │                  │ contiguous virt, │
   │  small objects    │                  │ scattered phys   │
   └──────────────────┘                  └──────────────────┘
            │                                     │
            ▼                                     ▼
      kmalloc(size, flags)                  vmalloc(size)
      kmem_cache_alloc(c, flags)
```

Above kmalloc lives `dma_alloc_coherent`, `dma_pool`, special arenas — used in driver paths. We get there in Part VI.

31.2 GFP flags

Every allocation specifies what it can do:

Flag	Meaning
`GFP_KERNEL`	normal, may sleep, may swap
`GFP_ATOMIC`	no sleep — for IRQ/atomic context
`GFP_NOWAIT`	no sleep, no oom-kill
`GFP_DMA`	low memory zone (legacy 16MB)
`GFP_DMA32`	< 4 GiB physical for 32-bit DMA
`__GFP_ZERO`	zero memory
`__GFP_NOWARN`	suppress oom warnings
`__GFP_HIGH`	use emergency reserves
`GFP_HIGHUSER_MOVABLE`	userspace pages, movable

You can combine: `GFP_KERNEL | __GFP_ZERO`. `kzalloc` is shorthand.

In atomic context (IRQ, spinlock-held) you **must** use `GFP_ATOMIC`. Using `GFP_KERNEL` will warn and may deadlock.

31.3 `kmalloc` and `kfree`

```
void *p = kmalloc(size, GFP_KERNEL);
if (!p) return -ENOMEM;
/* … */
kfree(p);
```

`kmalloc` returns physically contiguous memory. Aligned to the *natural alignment* of `size` (or page if size ≥ page). Backed by SLUB caches for small sizes; falls back to `alloc_pages` for big ones. Max size by default ~4 MB; beyond, you want `vmalloc` or `kvmalloc`.

`kzalloc(size, GFP_KERNEL)` = zeroed.

`krealloc(p, new_size, flags)` like userland realloc.

31.4 `kvmalloc`

Tries `kmalloc` first (contiguous), falls back to `vmalloc` (non-contiguous). Use for buffers that may be > a few pages and whose physical contiguity doesn't matter.

```
void *p = kvmalloc(size, GFP_KERNEL);
kvfree(p);
```

31.5 `vmalloc`

```
void *p = vmalloc(size);
vfree(p);
```

Returns memory whose **virtual** addresses are contiguous, but **physical** pages may be scattered. Use when:

- size is very large (multi-MB).
- physical contiguity isn't needed.
- you're willing to pay TLB cost (no huge pages).

Cannot be DMA'd from a typical PCI device (which expects contiguous physical or scatter-gather).

31.6 The slab allocator

`kmalloc` is built on slab caches. For your own fixed-size objects, create a cache:

```
struct kmem_cache *c = kmem_cache_create("my_obj",
    sizeof(struct my_obj), 0, SLAB_HWCACHE_ALIGN, NULL);

struct my_obj *o = kmem_cache_alloc(c, GFP_KERNEL);
kmem_cache_free(c, o);

kmem_cache_destroy(c);
```

Benefits over `kmalloc`: per-CPU magazines, predictable layout, optional constructor, sysfs stats (`/sys/kernel/slab/my_obj`).

amdgpu uses slab caches for `dma_fence`, `amdgpu_bo`, `drm_sched_job` etc.

31.7 The page allocator

Lower level than `kmalloc`. Returns whole pages.

```
struct page *p = alloc_pages(GFP_KERNEL, order);
void *addr = page_address(p);
__free_pages(p, order);
```

Order = $\log_2$ of page count. `alloc_pages(GFP_KERNEL, 0)` gives 1 page, `…, 4` gives 16 pages. Internally a buddy allocator.

For a `void *` interface: `__get_free_pages` / `free_pages`.

DMA paths often want pages: `dma_map_page` takes a `struct page *`.

31.8 `devm_*` — auto-free on detach

For driver probe-time allocations that must be freed when the device unbinds, the kernel offers `devm_kzalloc`, `devm_kmalloc`, `devm_request_irq`, `devm_kvasprintf`, etc. They auto-clean on driver detach.

```
struct mydrv *m = devm_kzalloc(dev, sizeof *m, GFP_KERNEL);
m->mmio = devm_ioremap_resource(dev, res);
```

Greatly reduces `goto out_*` chains.

31.9 Reading memory pressure

- `cat /proc/meminfo` — overall memory state.
- `cat /proc/slabinfo` — per-cache stats (root only).
- `slabtop` — interactive view.
- `/sys/kernel/slab/<cache>/...` — per-cache stats.
- `vmstat 1` — page activity.

Driver leaks usually show up as “this slab cache keeps growing during long runs.”

31.10 KASAN, KFENCE, and DEBUG_VM

Build with these in dev:

- **KASAN** — like ASan; detects use-after-free and out-of-bounds in kernel allocations.
- **KFENCE** — sampled, low-overhead use-after-free / OOB; fine for production debug.
- **DEBUG_VM** — many extra invariant checks in MM.
- **DEBUG_PAGEALLOC** — unmaps freed pages so use-after-free oopses immediately.

For driver work: KASAN. Kernel will boot ~30% slower but find your bugs.

31.11 The `dma-buf` allocator (preview)

GPU drivers also use DMA-related allocators. We discuss in Chapter 38 and 45:

- `dma_alloc_coherent(dev, size, &dma_handle, gfp)` — coherent DMA buffer (the device sees it consistent without flushing).
- `dma_pool_create()` for many small DMA allocations.
- `dma_map_single` / `dma_map_sg` for streaming DMA on existing memory.

31.12 Practical guidance

- Small struct: `kzalloc(sizeof *p, GFP_KERNEL)`.
- Many of one struct: `kmem_cache_create` + `kmem_cache_alloc`.
- 1+ MB buffer not for DMA: `kvmalloc`.
- Multi-MB DMA buffer: `dma_alloc_coherent`.
- Any allocation in IRQ/atomic: `GFP_ATOMIC`, but consider preallocating in process context first.

31.13 Exercises

1. In a kernel module, allocate `kmalloc(1024, GFP_KERNEL)`; print the address; free. Try with `GFP_ATOMIC` from a tasklet.
2. Create a slab cache for a small struct; allocate 1000 objects; check `/sys/kernel/slab/<cache>/`; free; check again.
3. Compare physical contiguity: `kmalloc(1<<22)` (4 MB) vs `vmalloc(1<<22)`. Translate to physical via `virt_to_phys` / `vmalloc_to_page` for each.
4. Build a kernel with KASAN; insmod a module that does `int *p = kmalloc(4); kfree(p); *p = 0;`. Read the dmesg splat.
5. Read `mm/slub.c::kmem_cache_alloc` — see the per-CPU magazine fast path.

Next: Chapter 32 — Time, timers, workqueues, kthreads.

Chapter 32 — Time, timers, workqueues, kthreads

Drivers need to wait for things, schedule deferred work, run periodic background tasks, and time-out hardware operations. This chapter covers the API.

32.1 Time bases

```
        ktime_t t = ktime_get();                    /* CLOCK_MONOTONIC, ns */
        unsigned long j = jiffies;                  /* tick count */
        unsigned long ns = ktime_get_real_ns();     /* CLOCK_REALTIME */
```

- **jiffies** — incremented at `HZ` (typically 250 or 1000 on Linux). Use for coarse waits (seconds).
- **ktime_t** — nanoseconds, monotonic. Use for everything else.
- `ktime_get_boottime` — survives suspend.

Convert:

```
        msecs_to_jiffies(500)
        jiffies_to_msecs(j)
        nsecs_to_jiffies(n)
```

Don't compare jiffies with `<`; use `time_before(a, b)` (handles wrap-around).

32.2 Sleeping (process context only)

```
        msleep(100);                /* milliseconds, may sleep more */
        usleep_range(100, 200);     /* microseconds, sleeping */
        schedule_timeout_interruptible(msecs_to_jiffies(500));
```

Tip: `usleep_range(min, max)` lets the kernel batch wakeups; better than `udelay` which busy-waits.

32.3 Busy-waiting (atomic context)

```
        udelay(50);     /* microseconds */
        mdelay(2);      /* milliseconds — discouraged */
        ndelay(100);    /* nanoseconds */
        cpu_relax();    /* PAUSE / YIELD inside spin loops */
```

Use only when you must, in atomic context, and only briefly. Long busy waits eat CPU.

32.4 Timers — one-shot deferred work

```
struct timer_list t;
timer_setup(&t, my_callback, 0);
mod_timer(&t, jiffies + msecs_to_jiffies(500));

/* … */
del_timer_sync(&t);
```

`my_callback(struct timer_list *t)` runs in **softirq** context (atomic). Cannot sleep. Use for short, rare deferred work — for anything substantive prefer workqueues.

32.5 High-resolution timers (`hrtimer`)

```
struct hrtimer ht;
hrtimer_init(&ht, CLOCK_MONOTONIC, HRTIMER_MODE_REL);
ht.function = my_hr_cb;
hrtimer_start(&ht, ms_to_ktime(1), HRTIMER_MODE_REL);
```

Nanosecond-resolution wakeups. `my_hr_cb` returns `HRTIMER_NORESTART` or `HRTIMER_RESTART`.

32.6 Workqueues — sleeping deferred work

```
struct work_struct w;
INIT_WORK(&w, my_work_fn);

schedule_work(&w);

/* in IRQ */
schedule_work(&dev->w);
```

`my_work_fn` runs in a **kthread** (process context). It can:

- sleep,
- take mutexes,
- copy from/to user (well, almost — but there's no userspace context, so no `copy_to_user` to *the calling* user).

Variants:

- `schedule_work(w)` — system work queue.
- `schedule_delayed_work(dw, delay)` — like timer + work.
- `queue_work(wq, w)` — on your own workqueue.
- `alloc_workqueue("mywq", WQ_UNBOUND, 0)` — for ordered or per-cpu wq.
- `cancel_work_sync(w)` / `flush_work(w)` — wait for completion.

amdgpu has many workqueues for: hot-plug, AMD Vega CG, RAS, scheduler, etc.

32.7 Tasklets (deprecated)

`tasklet_init`, `tasklet_schedule` — softirq-context deferred work. No new code should use them; prefer threaded IRQs and workqueues. amdgpu still has some.

32.8 Kthreads

A kernel-only "thread":

```
struct task_struct *t;

t = kthread_run(my_thread_fn, dev, "amdgpu-cleanup");
/* … */
kthread_stop(t);
```

Inside `my_thread_fn`:

```
while (!kthread_should_stop()) {
    wait_event_interruptible(wq, condition || kthread_should_stop());
    /* do work */
}
```

`kthread_stop` sets `should_stop` and wakes; thread should exit.

amdgpu uses kthreads for: GPU scheduler workers (`drm_sched_main`), suspend helper, RAS, etc.

32.9 RCU's `call_rcu`

If you need to defer freeing until current RCU readers finish:

```
struct foo {
    struct rcu_head rcu;
    /* … */
};

static void foo_release(struct rcu_head *r) {
    struct foo *f = container_of(r, struct foo, rcu);
    kfree(f);
}

call_rcu(&f->rcu, foo_release);
```

Or simpler `kfree_rcu(p, rcu)`.

32.10 The completion (review)

For "wait until something finishes once":

```
struct completion done;
init_completion(&done);

/* IRQ */
complete(&done);

/* waiter */
wait_for_completion(&done);
wait_for_completion_timeout(&done, msecs_to_jiffies(1000));
```

amdgpu uses completions for power-management transitions and reset stages.

32.11 Time-outs in waits

Always bound your waits. Hardware can hang, fences can be lost. Use timeout variants:

```
long ret = wait_event_interruptible_timeout(wq, cond, msecs_to_jiffies(500));
if (ret == 0)        /* timed out */;
else if (ret < 0)    /* signal */;
```

```
unsigned long t = dma_fence_wait_timeout(f, true, msecs_to_jiffies(1000));
if (t == 0) /* TDR */;
```

If a fence times out, the GPU has hung — you trigger reset (Chapter 53). This is a real and ugly path that you will work on.

32.12 Exercises

1. Write a module that schedules a workqueue from a timer; the work prints jiffies. Verify it runs.
2. Create a kthread that prints “tick” every second, and stops on rmmod.
3. Use `wait_event_interruptible_timeout` waiting on a flag toggled from another kthread; observe behavior on timeout.
4. Use `hrtimer` for a 100µs periodic — check accuracy with `ktime_get`.
5. Read `kernel/workqueue.c::process_one_work`. See where the worker invokes your callback.

Next: Chapter 33 — Locking in the kernel.

Chapter 33 — Locking in the kernel

We've already met spinlocks, mutexes, and RCU in Part IV. This chapter is the **driver-author's checklist**: which lock to pick, which traps to avoid, and how to satisfy lockdep on your first patch.

33.1 Decision tree

```
Need a lock around state shared between contexts?
│
├─ Held in IRQ handler (hardirq)?            → spin_lock_irqsave
├─ Held in softirq/tasklet only?             → spin_lock_bh
├─ Held only in process context, short?      → spinlock_t
├─ Held only in process context, may sleep?  → mutex
├─ Mostly read, few writes?                  → rwsem (for read-many) or RCU (read-mostly)
└─ Single producer, single consumer?         → atomic + memory barriers, no lock
```

33.2 Order of locks

Always document. Place a comment at the top of your driver:

```
/*
 * Lock ordering:
 *  dev->lock     -> bo->lock     -> fence->lock
 *  pm_lock       -> dev->lock
 */
```

Take in that order. Never the reverse.

`lockdep` will validate this empirically, but documentation is the contract for readers.

33.3 Common lock idioms

Per-device mutex protecting state

```
struct mydrv {
    struct mutex     lock;       /* protects fields below */
    struct list_head bos;
    int              n_bos;
    bool             initialized;
};
```

A single mutex is the right starting point. Optimize later.

Spin in IRQ; mutex elsewhere

If the same data is touched from IRQ and process context:

```
spinlock_t  lock;       /* IRQ-safe */
```

In process context: `spin_lock_irqsave(&lock, flags); … spin_unlock_irqrestore(&lock, flags);`.

In IRQ: `spin_lock(&lock); … spin_unlock(&lock);` (already in IRQ context).

Reader/writer with seqlock

For read-heavy timing-like data:

```
seqlock_t sl = __SEQLOCK_UNLOCKED(sl);

unsigned seq;
do {
    seq = read_seqbegin(&sl);
    /* read snapshot */
} while (read_seqretry(&sl, seq));
```

Writer: `write_seqlock` / `write_sequnlock`.

RCU for registered objects

```
list_for_each_entry_rcu(p, &irq_sources, node) {
    if (p->matches(...)) p->handle(...);
}
```

Writer: `list_add_rcu`, `synchronize_rcu`, `kfree`.

33.4 Sleeping vs. atomic context — the bug

The most common kernel bug:

```
spin_lock(&dev->lock);
buf = kmalloc(64, GFP_KERNEL);   /* BUG: may sleep */
```

Replace with `GFP_ATOMIC`, or move the alloc outside the spinlock.

`CONFIG_DEBUG_ATOMIC_SLEEP` catches this at runtime. **Always** boot dev kernels with this on.

33.5 IRQ-safe variants

When the same lock is taken from IRQ:

- `spin_lock_irqsave(lock, flags)` — saves IF flag, disables IRQ on this CPU.
- `spin_lock_irq(lock)` — disables IRQ; only safe if you know IRQs are currently enabled.
- `spin_lock_bh(lock)` — disables softirq.

Mismatched pairs (lock with IRQ-save, unlock without restore) are a bug. lockdep notices.

33.6 Read-side cost

- **Spinlock read-only**: still a CAS + memory write. Cache line is owned exclusively when held.
- **rwlock**: read paths still write the rwlock state; cache-line bouncing under reader contention.
- **rw_semaphore**: better for sleepers; still an atomic op.

- **RCU**: zero atomic ops on the read side (preempt_disable is free on PREEMPT_NONE). Best for read-heavy.

For a global structure read by every userspace IO and modified rarely (e.g. registered IRQ source list), RCU is the right answer.

33.7 Lock-free counters (per-CPU)

For statistics counters that get incremented hot and read rarely:

```
DEFINE_PER_CPU(u64, my_stat);

this_cpu_inc(my_stat);

u64 sum = 0;
int cpu;
for_each_possible_cpu(cpu) sum += *per_cpu_ptr(&my_stat, cpu);
```

Each CPU writes only its own; aggregation walks once. Extremely scalable.

33.8 lockdep features

- **Lock order tracking**: complains about A→B then B→A.
- **Soft/hard IRQ tracking**: a lock taken from IRQ must always be `irq_save`'d everywhere.
- **Recursive lock detection** for non-recursive locks.
- **Unlock-without-lock and lock-with-no-init checks.**

Read its splats carefully — they tell you exactly which call sites disagreed.

33.9 Deadlock avoidance recipes

- Lock annotation comments.
- `mutex_trylock` for "best-effort" paths; if you fail, drop everything and retry.
- `__must_check` and clean unwind.
- Avoid calling foreign code (callbacks, user code) under a lock.

33.10 Exercises

1. Build the chardev skeleton (Chapter 29) and add a per-device mutex protecting an internal counter. Increment from `write`, read from `read`.
2. Add a kthread that increments the same counter; ensure the lock is correct.
3. Make a deliberate lock-order inversion across two mutexes between two contexts; observe lockdep splat.
4. Replace the mutex with a per-CPU counter; verify functional equivalence.
5. Read `Documentation/locking/lockdep-design.rst`. Understand what each splat term means.

Next: Chapter 34 — The driver model: kobject, sysfs, devices, drivers, buses.

Chapter 34 — The driver model: kobject, sysfs, devices, drivers, buses

The Linux **driver model** is the kernel's universal abstraction: every device is a `struct device`, every driver is a `struct device_driver`, and they're matched on a bus. This is the spine that holds DRM, PCI, USB, and amdgpu together.

34.1 The objects

```
                  ┌──────────┐
                  │ kobject  │   refcounted, sysfs-visible
                  └──────────┘
                       │ embedded in
        ┌──────────────┴──────────────┐
        ▼                             ▼
   ┌─────────┐                   ┌─────────┐
   │ device  │                   │bus_type │
   └─────────┘                   └─────────┘
       │
   ┌───┴─────────┐
   ▼             ▼
device_driver  attribute_group  (sysfs files)
```

- **`struct kobject`** — the base. Has refcount, parent, name, ktype, release.
- **`struct device`** — embeds kobject. Represents a physical or logical device.
- **`struct device_driver`** — drivers register here. Has `probe`, `remove`, `id_table`.
- **`struct bus_type`** — PCI, USB, platform, I2C, SPI, etc. Match function bridges drivers and devices.

When a device appears (boot, hotplug), the bus calls each driver's `match` against the device. If matched, `probe` is called. If `probe` returns 0, the driver "binds" — the device is operational.

34.2 Buses you'll touch

Bus	Used by
`pci`	PCIe devices: GPUs, NVMe, NICs
`platform`	SoC IP blocks (Qualcomm, Tegra, ARM SoCs)
`usb`	USB devices
`i2c`	low-speed sensors, EDID readers, GPU side-band
`spi`	SPI flash, sensors
`mdio`	PHY chips
`dma-buf`	(not a bus; cross-driver buffer sharing)

amdgpu binds to PCI. AMD APUs may also have platform-bus children for sub-IPs.

34.3 sysfs

Every kobject creates a directory under `/sys`. The tree mirrors the device hierarchy.

```
ls /sys/devices/pci0000:00/0000:00:01.0/0000:01:00.0/
```

That's a typical AMD GPU PCIe path. Inside you find: `vendor`, `device`, `revision`, `power/`, `resource`, `enable`, `subsystem` (symlink to bus type), `driver` (symlink to bound driver).

Drivers expose attributes via:

```
static ssize_t my_attr_show(struct device *dev,
                            struct device_attribute *attr, char *buf)
{
    return sysfs_emit(buf, "%d\n", value);
}

static ssize_t my_attr_store(struct device *dev,
                             struct device_attribute *attr,
                             const char *buf, size_t count)
{
    int v;
    if (kstrtoint(buf, 0, &v)) return -EINVAL;
    set_value(v);
    return count;
}
DEVICE_ATTR_RW(my_attr);   /* generates dev_attr_my_attr */
```

Register:

```
device_create_file(dev, &dev_attr_my_attr);
```

Or via attribute groups:

```
static struct attribute *my_attrs[] = {
    &dev_attr_my_attr.attr,
    NULL,
};
ATTRIBUTE_GROUPS(my);     /* makes my_groups */

driver.dev_groups = my_groups;
```

amdgpu exposes hundreds of sysfs attributes for power management, tuning, and debugging. `cat /sys/class/drm/card0/device/pp_dpm_sclk` etc.

34.4 The probe/remove flow

```
static int my_probe(struct pci_dev *pdev, const struct pci_device_id *id)
{
    struct mydrv *m = devm_kzalloc(&pdev->dev, sizeof *m, GFP_KERNEL);
    if (!m) return -ENOMEM;

    pci_set_drvdata(pdev, m);

    if (pcim_enable_device(pdev)) return -ENODEV;
    if (pcim_iomap_regions(pdev, BIT(0), "mydrv")) return -ENOMEM;
    m->mmio = pcim_iomap_table(pdev)[0];

    if (devm_request_irq(&pdev->dev, pdev->irq, my_isr, IRQF_SHARED, "mydrv", m))
        return -EIO;
```

```
        /* register chardev / drm */

        return 0;
    }

    static void my_remove(struct pci_dev *pdev)
    {
        /* drm/chardev unregister */
        /* devm cleanup is automatic */
    }

    static const struct pci_device_id my_ids[] = {
        { PCI_DEVICE(0x1234, 0x5678) },
        { },
    };
    MODULE_DEVICE_TABLE(pci, my_ids);

    static struct pci_driver my_pci_driver = {
        .name      = "mydrv",
        .id_table  = my_ids,
        .probe     = my_probe,
        .remove    = my_remove,
    };

    module_pci_driver(my_pci_driver);   /* expands init/exit */
```

This is the *real* shape of a PCI driver. We'll write a working version against the QEMU `edu` device in Chapter 37.

34.5 Hot-plug

When the bus detects a new device (e.g. PCI hot-plug, USB plug-in), the kernel calls `bus->match()` against each registered driver. If matched, `driver->probe()`. On removal, `remove`.

amdgpu doesn't typically hot-plug, but eGPU systems do. Code must work for "device disappears" — that's where `remove` runs and you must not crash on in-flight ioctls.

34.6 The component framework

Some subsystems need multiple devices to become operational together (e.g. a display IP block + a DDC IP block + a clock source). The **component** framework defers binding until all required components register, then calls `master_bind`. Drivers using DC for AMD displays touch this.

34.7 Class

A *class* groups devices with similar function for userspace. `/sys/class/drm/`, `/sys/class/gpu/`, `/sys/class/net/`. `class_create()` registers; `device_create()` adds a device.

DRM uses class to build `/sys/class/drm/cardN`. You usually don't touch `class_create` directly when writing a DRM driver — it does it for you.

34.8 Power-management hooks

The driver model includes PM ops (we'll cover in Chapter 40):

```
static const struct dev_pm_ops mydrv_pm_ops = {
    .suspend = mydrv_suspend,
    .resume  = mydrv_resume,
    SET_RUNTIME_PM_OPS(mydrv_runtime_suspend, mydrv_runtime_resume, NULL)
};

static struct pci_driver my_pci_driver = {
    /* … */
    .driver.pm = &mydrv_pm_ops,
};
```

amdgpu has a complex PM ops table — runtime PM, system suspend, hibernate, ACPI integration.

34.9 Attaching to the device tree (briefly, for SoCs)

On embedded ARM/RISC-V platforms (Qualcomm, MediaTek, NXP), devices are described in a **device tree** (DT) blob the bootloader passes. Platform-bus drivers match DT compatible strings:

```
static const struct of_device_id my_of_ids[] = {
    { .compatible = "vendor,my-ip" },
    { },
};
```

`amdgpu` is PCI-only; for Qualcomm-style SoC GPUs you'd be in `drivers/gpu/drm/msm/` (Adreno) which uses platform/DT.

34.10 Reading source

- `drivers/base/core.c` — device model core.
- `drivers/base/bus.c`, `drivers/base/driver.c` — bus matching.
- `drivers/pci/pci-driver.c` — PCI driver registration.
- `include/linux/device.h`, `include/linux/pci.h` — the headers.

Once these are familiar, every driver in the tree follows the same shape.

34.11 Exercises

1. Read the directory tree of one PCI device under `/sys/devices/pci0000:00/...`. Identify the symlinks `driver`, `subsystem`, `power`.
2. Write a platform driver that matches a fake compatible string (you don't need a real device — just register the driver and register a `platform_device` from another module).
3. Add a `DEVICE_ATTR_RW(foo)` to your skeleton driver from Chapter 29. Write/read it via `/sys/devices/...`.
4. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c::amdgpu_pci_driver`. Trace `probe` to identify the first ten things amdgpu does on bind.
5. `udevadm monitor --kernel` while plugging/unplugging USB; observe events. The same machinery underlies `amdgpu` reset paths.

End of Part V. Move to Part VI — Linux device drivers.

Chapter 35 — Character drivers from scratch

A character device is the simplest driver shape and the foundation for all “byte stream” interfaces — including DRM. We build a complete one with read/write/ring buffer/wait queue, then later layer DMA, mmap, and ioctl on top.

35.1 Char vs. block

- **Char**: byte-stream; sequential or arbitrary access; no required block size. `/dev/null`, `/dev/zero`, `/dev/random`, `/dev/dri/card0`.
- **Block**: fixed sectors; managed by the block layer; supports read-ahead, scheduler, file systems.

DRM, evdev, V4L2, audio, GPIO — all are char.

35.2 The major and minor numbers

`/dev` entries have a (major, minor) pair. Major identifies the driver, minor the instance. Old kernels statically assigned majors; we now `alloc_chrdev_region` to get one dynamically.

```
dev_t dev_no;
alloc_chrdev_region(&dev_no, 0, 1, "mydrv");   /* one minor, starting from 0 */
unregister_chrdev_region(dev_no, 1);
```

35.3 `cdev` — the kernel object

```
struct cdev cdev;
cdev_init(&cdev, &my_fops);
cdev.owner = THIS_MODULE;
cdev_add(&cdev, dev_no, 1);
/* … */
cdev_del(&cdev);
```

Then create the `/dev` node via the device-class machinery so udev populates `/dev`:

```
struct class *cls = class_create("mydrv");
device_create(cls, NULL, dev_no, NULL, "mydrv");
/* ... */
device_destroy(cls, dev_no);
class_destroy(cls);
```

(Newer kernels: `class_create("mydrv")` without `THIS_MODULE` argument.)

35.4 The read/write contract

The kernel calls your `read(file, ubuf, count, offp)`:

- `ubuf` is a **userspace** pointer. Do not deref directly.
- Use `copy_to_user(ubuf, kbuf, n)` — it handles page faults.
- Return number of bytes copied, or negative errno.
- `*offp` is the file offset; update if you support seeking.

Write is symmetric.

```
static ssize_t my_read(struct file *f, char __user *ubuf,
                       size_t count, loff_t *offp)
{
    char tmp[64];
    int n;

    n = snprintf(tmp, sizeof(tmp), "value=%d\n", value);
    if (*offp >= n) return 0;             /* EOF */
    if (count > n - *offp) count = n - *offp;
    if (copy_to_user(ubuf, tmp + *offp, count))
        return -EFAULT;
    *offp += count;
    return count;
}
```

The `__user` annotation is checked by **sparse**. Your build will warn if you mix `__user` and kernel pointers.

35.5 A real ring-buffer chardev

We build a producer/consumer driver: writers push bytes; readers pop. Includes proper blocking, wait queues, locking.

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/wait.h>
#include <linux/uaccess.h>
#include <linux/mutex.h>

#define BUFSZ 4096

struct mydev {
    struct cdev   cdev;
    struct mutex  lock;
    wait_queue_head_t  rwq, wwq;
    char  buf[BUFSZ];
    size_t head, tail;        /* head=write, tail=read */
};

static struct mydev *g_dev;
static dev_t           g_dev_no;
static struct class   *g_class;

static size_t available_to_read(struct mydev *d) {
    return (d->head + BUFSZ - d->tail) % BUFSZ;
}
static size_t available_to_write(struct mydev *d) {
    return BUFSZ - 1 - available_to_read(d);
}
```

```
static int my_open(struct inode *i, struct file *f) {
    f->private_data = g_dev;
    return 0;
}

static ssize_t my_read(struct file *f, char __user *ubuf,
                       size_t count, loff_t *offp)
{
    struct mydev *d = f->private_data;
    size_t avail, chunk, total = 0;
    int ret;

    if (count == 0) return 0;

    if (mutex_lock_interruptible(&d->lock)) return -ERESTARTSYS;

    while (available_to_read(d) == 0) {
        mutex_unlock(&d->lock);
        if (f->f_flags & O_NONBLOCK) return -EAGAIN;
        ret = wait_event_interruptible(d->rwq, available_to_read(d) > 0);
        if (ret) return ret;
        if (mutex_lock_interruptible(&d->lock)) return -ERESTARTSYS;
    }

    avail = available_to_read(d);
    if (count > avail) count = avail;

    while (count > 0) {
        chunk = min(count, BUFSZ - d->tail);
        if (copy_to_user(ubuf + total, d->buf + d->tail, chunk)) {
            mutex_unlock(&d->lock);
            return -EFAULT;
        }
        d->tail = (d->tail + chunk) % BUFSZ;
        total  += chunk;
        count  -= chunk;
    }

    mutex_unlock(&d->lock);
    wake_up_interruptible(&d->wwq);
    return total;
}

static ssize_t my_write(struct file *f, const char __user *ubuf,
                        size_t count, loff_t *offp)
{
    struct mydev *d = f->private_data;
    size_t avail, chunk, total = 0;
    int ret;

    if (count == 0) return 0;

    if (mutex_lock_interruptible(&d->lock)) return -ERESTARTSYS;

    while (available_to_write(d) == 0) {
        mutex_unlock(&d->lock);
        if (f->f_flags & O_NONBLOCK) return -EAGAIN;
        ret = wait_event_interruptible(d->wwq, available_to_write(d) > 0);
        if (ret) return ret;
        if (mutex_lock_interruptible(&d->lock)) return -ERESTARTSYS;
    }

    avail = available_to_write(d);
    if (count > avail) count = avail;

    while (count > 0) {
        chunk = min(count, BUFSZ - d->head);
        if (copy_from_user(d->buf + d->head, ubuf + total, chunk)) {
            mutex_unlock(&d->lock);
            return -EFAULT;
```

```
            }
            d->head = (d->head + chunk) % BUFSZ;
            total  += chunk;
            count  -= chunk;
        }

        mutex_unlock(&d->lock);
        wake_up_interruptible(&d->rwq);
        return total;
    }

    static const struct file_operations my_fops = {
        .owner   = THIS_MODULE,
        .open    = my_open,
        .read    = my_read,
        .write   = my_write,
        .llseek  = no_llseek,
    };

    static int __init my_init(void)
    {
        int ret;

        g_dev = kzalloc(sizeof *g_dev, GFP_KERNEL);
        if (!g_dev) return -ENOMEM;
        mutex_init(&g_dev->lock);
        init_waitqueue_head(&g_dev->rwq);
        init_waitqueue_head(&g_dev->wwq);

        ret = alloc_chrdev_region(&g_dev_no, 0, 1, "myring");
        if (ret) goto err_free;

        cdev_init(&g_dev->cdev, &my_fops);
        g_dev->cdev.owner = THIS_MODULE;
        ret = cdev_add(&g_dev->cdev, g_dev_no, 1);
        if (ret) goto err_unreg;

        g_class = class_create("myring");
        if (IS_ERR(g_class)) { ret = PTR_ERR(g_class); goto err_del; }

        device_create(g_class, NULL, g_dev_no, NULL, "myring");
        pr_info("myring loaded major=%d minor=%d\n", MAJOR(g_dev_no), MINOR(g_dev_no));
        return 0;

    err_del:    cdev_del(&g_dev->cdev);
    err_unreg:  unregister_chrdev_region(g_dev_no, 1);
    err_free:   kfree(g_dev);
        return ret;
    }

    static void __exit my_exit(void)
    {
        device_destroy(g_class, g_dev_no);
        class_destroy(g_class);
        cdev_del(&g_dev->cdev);
        unregister_chrdev_region(g_dev_no, 1);
        kfree(g_dev);
    }

    module_init(my_init);
    module_exit(my_exit);
    MODULE_LICENSE("GPL");
```

Test:

```
    sudo insmod myring.ko
    sudo chmod 666 /dev/myring
    echo "hello world" > /dev/myring
    cat /dev/myring        # in another terminal blocks, then prints
```

You now have a working char driver with proper blocking I/O.

35.6 What we did right

- **Mutex** protecting state.
- **Two wait queues** — one for readers, one for writers; correct for producer/consumer.
- **Interruptible** waits — signals work.
- `O_NONBLOCK` support for `-EAGAIN`.
- `copy_from_user` / `copy_to_user` for userspace buffers.
- **Proper init/exit unwind**.
- `__user` annotation in API.

35.7 What's missing (next chapters)

- `poll` / `epoll` support — Chapter 36.
- `ioctl` for control — Chapter 36.
- `mmap` — Chapter 36.
- Multiple opens with shared data — Chapter 36.

35.8 Exercises

1. Build the driver above; test basic read/write.
2. Add `poll` (next chapter teaches; for now: `poll_wait(f, &d->rwq, p); poll_wait(f, &d->wwq, p);` and return appropriate `EPOLLIN/EPOLLOUT`).
3. Add an `ioctl` (`MYRING_GET_STATE`) returning current head/tail to userspace via a struct.
4. Run multiple readers/writers; observe correctness. Then run under TSan-like stress (loop 1M ops/thread).
5. Read `drivers/char/mem.c`; especially `read_zero` / `write_null` — minimal char drivers.

Next: Chapter 36 — `ioctl`, `mmap`, polling, async I/O.

Chapter 36 — `ioctl`, `mmap`, `poll`, async I/O

`read` / `write` are not enough for real devices. Drivers expose control via `ioctl`, expose memory via `mmap`, expose readiness via `poll`. DRM does all three. This chapter covers them.

36.1 `ioctl` — control commands

Userspace:

```
int ret = ioctl(fd, MYDRV_GET_STATE, &state);
```

Kernel:

```
static long my_ioctl(struct file *f, unsigned int cmd, unsigned long arg);
```

Place in `file_operations.unlocked_ioctl` (and `compat_ioctl` for 32-bit-on-64-bit).

Encoding the command

`<linux/ioctl.h>` provides macros:

```
#define MYDRV_IOC_MAGIC 'm'

#define MYDRV_NOP        _IO(MYDRV_IOC_MAGIC, 0)
#define MYDRV_SET_FOO    _IOW(MYDRV_IOC_MAGIC, 1, struct mydrv_foo)
#define MYDRV_GET_BAR    _IOR(MYDRV_IOC_MAGIC, 2, struct mydrv_bar)
#define MYDRV_RW_QUUX    _IOWR(MYDRV_IOC_MAGIC, 3, struct mydrv_quux)
```

`_IOW` (write to driver, i.e. user→kernel), `_IOR` (read from driver, kernel→user), `_IOWR` (both). The macros encode magic byte, command number, and direction/size — so the kernel can validate.

A safe ioctl handler

```
static long my_ioctl(struct file *f, unsigned int cmd, unsigned long arg)
{
    struct mydev *d = f->private_data;
    void __user *uarg = (void __user *)arg;

    if (_IOC_TYPE(cmd) != MYDRV_IOC_MAGIC)
        return -ENOTTY;

    switch (cmd) {
    case MYDRV_GET_BAR: {
        struct mydrv_bar bar;
        bar.value = atomic_read(&d->state);
        if (copy_to_user(uarg, &bar, sizeof(bar))) return -EFAULT;
        return 0;
    }
    case MYDRV_SET_FOO: {
```

```
            struct mydrv_foo foo;
            if (copy_from_user(&foo, uarg, sizeof(foo))) return -EFAULT;
            return apply_foo(d, &foo);
        }
        default:
            return -ENOTTY;
        }
    }
```

Always:

- copy_from_user / copy_to_user for the entire struct.
- validate before use.
- return `-ENOTTY` for unknown commands.

DRM extends this with a registration table where each ioctl has a flag bit (DRM_RENDER_ALLOW, DRM_AUTH, DRM_MASTER) and a struct size; the core copies in/out automatically and dispatches.

36.2 `mmap`

User:

```
    void *p = mmap(NULL, size, PROT_READ|PROT_WRITE, MAP_SHARED, fd, offset);
```

Kernel `file_operations.mmap`:

```
    static int my_mmap(struct file *f, struct vm_area_struct *vma);
```

You must populate the VMA so that future accesses get the right physical pages. Two approaches:

Approach A: pre-populate with `remap_pfn_range`

For a contiguous physical buffer (e.g. `dma_alloc_coherent` memory):

```
    static int my_mmap(struct file *f, struct vm_area_struct *vma)
    {
        struct mydev *d = f->private_data;
        unsigned long pfn = virt_to_phys(d->dma_buf) >> PAGE_SHIFT;
        unsigned long size = vma->vm_end - vma->vm_start;

        if (size > d->dma_size) return -EINVAL;

        vma->vm_page_prot = pgprot_writecombine(vma->vm_page_prot);
        if (remap_pfn_range(vma, vma->vm_start, pfn,
                            size, vma->vm_page_prot))
            return -EAGAIN;
        return 0;
    }
```

`remap_pfn_range` walks the VMA and writes PTEs pointing to the contiguous physical pages.

Approach B: demand-fault with `vm_ops->fault`

For a buffer that's not contiguous, or whose pages should be allocated lazily:

```
static vm_fault_t my_fault(struct vm_fault *vmf)
{
    struct mydev *d = vmf->vma->vm_private_data;
    pgoff_t pgoff = vmf->pgoff;
    struct page *page = d->pages[pgoff];

    get_page(page);
    vmf->page = page;
    return 0;
}

static const struct vm_operations_struct my_vm_ops = {
    .fault = my_fault,
};

static int my_mmap(struct file *f, struct vm_area_struct *vma)
{
    struct mydev *d = f->private_data;
    vma->vm_private_data = d;
    vma->vm_ops = &my_vm_ops;
    return 0;
}
```

When user accesses a page, the CPU faults; kernel calls `my_fault`; we return the page; kernel installs the PTE.

GEM (Chapter 45) uses this pattern: GPU buffers may be in VRAM, GTT, or system memory; the fault handler decides what to install.

Cache attributes

```
vma->vm_page_prot = pgprot_writecombine(vma->vm_page_prot);
vma->vm_page_prot = pgprot_noncached(vma->vm_page_prot);
```

For GPU command buffers and frame buffers being written by CPU and read by GPU, write-combining is correct: the CPU coalesces writes into 64-byte buffers and flushes to the device.

For doorbells and MMIO, uncached.

For system-memory buffers being read by both, the default writeback works *if* you flush carefully (`dma_sync_single_for_*`).

Choosing wrong → silent corruption.

36.3 `poll` / `epoll` / `select`

Userspace:

```
struct pollfd p = { .fd = fd, .events = POLLIN };
poll(&p, 1, 1000);
```

Kernel:

```
static __poll_t my_poll(struct file *f, struct poll_table_struct *pt)
{
    struct mydev *d = f->private_data;
    __poll_t mask = 0;

    poll_wait(f, &d->rwq, pt);
    poll_wait(f, &d->wwq, pt);
```

```
        if (available_to_read(d) > 0)  mask |= EPOLLIN  | EPOLLRDNORM;
        if (available_to_write(d) > 0) mask |= EPOLLOUT | EPOLLWRNORM;
        return mask;
    }
```

`poll_wait` registers the wait-queue with the polling system without actually waiting. When the wait queue is woken (e.g. `wake_up_interruptible(&d->rwq)`), the polling system rechecks the masks.

DRM exposes `poll` for vblank events and async fence completion notifications.

36.4 Async I/O / `O_NONBLOCK` / signal-driven

Modern apps use **`epoll(7)`** or **`io_uring(7)`**. Drivers see them as variants of `poll` and async submission. For our purposes:

- `O_NONBLOCK` set: read/write return `-EAGAIN` if would block.
- Standard pattern: drivers expose `read`, `write`, `poll` correctly; epoll just works.
- `io_uring` integration is opt-in via `IORING_OP_*`; rarely necessary in driver code.

DRM uses fences and event fds for completion delivery, not raw async I/O.

36.5 Notifying user via fd

Two common patterns:

- **Read interface**: when an event happens, queue an event packet; user `read`s the packet. DRM does this for vblank, page-flip-complete.
- **eventfd**: a counting fd; the driver wakes user via `eventfd_signal()`. Simpler to integrate into existing event loops.
- **drm_syncobj**: fence-handle exposure that integrates with kernel's fence layer.

36.6 Compat ioctls

A 32-bit userspace can call into a 64-bit kernel. Many ioctl structs have implicit padding/alignment differences. Provide `compat_ioctl`. For pure-uint32 structs, often `compat_ptr_ioctl` suffices. DRM provides core helpers.

36.7 Exercises

1. Add `MYRING_GET_HEAD_TAIL` ioctl to your driver from Chapter 35; copy out a small struct.
2. Add `mmap` to expose the ring buffer's RAM page to userspace; user can read directly. (Hint: kmalloc'd memory is not contiguous physically; you may need `vmalloc_user` and `remap_vmalloc_range`.)
3. Add `poll` so `epoll` apps can wait for read/write availability.
4. Write a small userspace program that opens `/dev/myring`, mmaps, polls — and benchmark vs. `read`.
5. Read `drivers/gpu/drm/drm_ioctl.c::drm_ioctl`. Note how it dispatches by table.

Next: Chapter 37 — The PCI/PCIe subsystem and writing a PCI driver.

Chapter 37 — The PCI/PCIe subsystem and writing a PCI driver

Every modern AMD GPU is a PCIe device. Mastering PCI is a hard prerequisite. This chapter walks the bus, enumeration, BARs, MSI/MSI-X, and ends with a working driver against QEMU's `edu` test device.

37.1 The PCI fabric

PCIe is a tree of point-to-point serial links rooted at the **Root Complex** in the CPU. Switches branch the tree; endpoints are leaves.

Each device has:

- **Bus / Device / Function** address (`BB:DD.F`): `lspci` shows them.
- **Configuration space** (256 B legacy + 4 KiB extended): vendor/device IDs, BARs, capabilities.
- **Memory BARs** — Base Address Registers: regions of physical address space mapped to the device. The OS programs the addresses; the device responds to them.
- **I/O BARs** (legacy x86 port I/O) — rarely used today.
- **Interrupts** — INTx (legacy), MSI, or MSI-X.

`lspci -tv`:

```
-[0000:00]-+-00.0  AMD Family 17h Root Complex
           +-01.0-[01]----00.0  AMD Navi 10 [Radeon RX 5700]
```

`lspci -vv -s 01:00.0` dumps everything.

37.2 The configuration space

A PCI config-space header looks like:

```
00: Vendor ID  Device ID
04: Command    Status
08: Class      SubClass  ProgIF  RevID
0C: …
10: BAR0
14: BAR1
18: BAR2
1C: BAR3
20: BAR4
24: BAR5
28: CardBus CIS pointer
2C: SubVendor SubDevice
30: ROM BAR
34: Cap pointer    (chain into MSI/MSI-X/PM/PCIe extended caps)
3C: IRQ            Pin  Min_Gnt  Max_Lat
```

The kernel's PCI core parses this and populates `struct pci_dev`.

37.3 The driver model on PCI

```
static const struct pci_device_id mydrv_ids[] = {
    { PCI_DEVICE(0x1234, 0x11e8) },     /* QEMU edu device */
    { },
};
MODULE_DEVICE_TABLE(pci, mydrv_ids);

static int my_probe(struct pci_dev *pdev, const struct pci_device_id *id);
static void my_remove(struct pci_dev *pdev);

static struct pci_driver my_pci_driver = {
    .name     = "mydrv",
    .id_table = mydrv_ids,
    .probe    = my_probe,
    .remove   = my_remove,
};

module_pci_driver(my_pci_driver);
```

`module_pci_driver` macro declares `module_init` / `module_exit` that register/unregister the PCI driver.

37.4 The QEMU `edu` device

QEMU ships a pedagogic PCI device called `edu` (in `hw/misc/edu.c`). Vendor `0x1234`, device `0x11e8`. It's a minimal device with:

- a 1 MB BAR0 with control registers and a 4 KiB scratchpad,
- DMA support (memcpy from/to host RAM),
- a doorbell-style "raise IRQ" mechanism.

To attach it to a QEMU VM:

```
qemu-system-x86_64 -enable-kvm -m 2G -smp 2 \
    -kernel arch/x86/boot/bzImage \
    -append "root=/dev/sda console=ttyS0 nokaslr" \
    -hda rootfs.qcow2 \
    -nographic \
    -device edu
```

Inside the VM you'll see `lspci` print the edu device. We will write a real driver for it now.

37.5 Probe path

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/pci.h>
#include <linux/io.h>
#include <linux/interrupt.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/uaccess.h>

struct edu {
    struct pci_dev *pdev;
    void __iomem   *bar0;
    int             irq;

    struct cdev     cdev;
```

```
        dev_t           dev_no;
        struct class   *cls;
        spinlock_t      lock;
        wait_queue_head_t wq;
        u32             last_factorial;
        bool            irq_seen;
    };

    #define EDU_REG_ID         0x00
    #define EDU_REG_CARD       0x04
    #define EDU_REG_FACT       0x08      /* write to compute factorial */
    #define EDU_REG_STATUS     0x20
    #define EDU_REG_INTR_RAISE 0x60
    #define EDU_REG_INTR_ACK   0x64
    #define EDU_REG_INTR_STATUS 0x24

    static irqreturn_t edu_isr(int irq, void *cookie)
    {
        struct edu *e = cookie;
        u32 s = readl(e->bar0 + EDU_REG_INTR_STATUS);
        if (!s) return IRQ_NONE;

        writel(s, e->bar0 + EDU_REG_INTR_ACK);

        spin_lock(&e->lock);
        e->last_factorial = readl(e->bar0 + EDU_REG_FACT);
        e->irq_seen       = true;
        spin_unlock(&e->lock);

        wake_up_interruptible(&e->wq);
        return IRQ_HANDLED;
    }

    static ssize_t edu_write(struct file *f, const char __user *ub,
                             size_t n, loff_t *off)
    {
        struct edu *e = f->private_data;
        char buf[16];
        u32  v;

        if (n == 0 || n > sizeof(buf) - 1) return -EINVAL;
        if (copy_from_user(buf, ub, n)) return -EFAULT;
        buf[n] = '\0';
        if (kstrtouint(buf, 0, &v)) return -EINVAL;

        spin_lock_irq(&e->lock);
        e->irq_seen = false;
        spin_unlock_irq(&e->lock);

        /* "factorial" register: writing N triggers IRQ when N! is computed */
        writel(v, e->bar0 + EDU_REG_FACT);
        return n;
    }

    static ssize_t edu_read(struct file *f, char __user *ub,
                            size_t n, loff_t *off)
    {
        struct edu *e = f->private_data;
        char  buf[32];
        int   len;
        u32   val;

        if (wait_event_interruptible(e->wq, READ_ONCE(e->irq_seen)))
            return -ERESTARTSYS;

        spin_lock_irq(&e->lock);
        val = e->last_factorial;
        spin_unlock_irq(&e->lock);

        len = scnprintf(buf, sizeof(buf), "%u\n", val);
        if (*off >= len) return 0;
```

```
        if (n > len - *off) n = len - *off;
        if (copy_to_user(ub, buf + *off, n)) return -EFAULT;
        *off += n;
        return n;
    }

    static int edu_open(struct inode *i, struct file *f)
    {
        struct edu *e = container_of(i->i_cdev, struct edu, cdev);
        f->private_data = e;
        return 0;
    }

    static const struct file_operations edu_fops = {
        .owner  = THIS_MODULE,
        .open   = edu_open,
        .read   = edu_read,
        .write  = edu_write,
        .llseek = no_llseek,
    };

    static int edu_probe(struct pci_dev *pdev, const struct pci_device_id *id)
    {
        struct edu *e;
        int ret;

        e = devm_kzalloc(&pdev->dev, sizeof(*e), GFP_KERNEL);
        if (!e) return -ENOMEM;
        e->pdev = pdev;
        spin_lock_init(&e->lock);
        init_waitqueue_head(&e->wq);

        ret = pcim_enable_device(pdev);
        if (ret) return ret;

        ret = pcim_iomap_regions(pdev, BIT(0), "edu");
        if (ret) return ret;
        e->bar0 = pcim_iomap_table(pdev)[0];

        pci_set_master(pdev);

        ret = pci_alloc_irq_vectors(pdev, 1, 1, PCI_IRQ_MSI | PCI_IRQ_INTX);
        if (ret < 0) return ret;
        e->irq = pci_irq_vector(pdev, 0);

        ret = devm_request_irq(&pdev->dev, e->irq, edu_isr, IRQF_SHARED, "edu", e);
        if (ret) goto err_free_irq;

        ret = alloc_chrdev_region(&e->dev_no, 0, 1, "edu");
        if (ret) goto err_free_irq;

        cdev_init(&e->cdev, &edu_fops);
        e->cdev.owner = THIS_MODULE;
        ret = cdev_add(&e->cdev, e->dev_no, 1);
        if (ret) goto err_unreg;

        e->cls = class_create("edu");
        if (IS_ERR(e->cls)) { ret = PTR_ERR(e->cls); goto err_del; }
        device_create(e->cls, &pdev->dev, e->dev_no, NULL, "edu");

        pci_set_drvdata(pdev, e);
        dev_info(&pdev->dev, "edu probed: bar0=%p irq=%d id=0x%x\n",
                 e->bar0, e->irq, readl(e->bar0 + EDU_REG_ID));
        return 0;

    err_del:    cdev_del(&e->cdev);
    err_unreg:  unregister_chrdev_region(e->dev_no, 1);
    err_free_irq:
        pci_free_irq_vectors(pdev);
        return ret;
    }
```

```
static void edu_remove(struct pci_dev *pdev)
{
    struct edu *e = pci_get_drvdata(pdev);
    device_destroy(e->cls, e->dev_no);
    class_destroy(e->cls);
    cdev_del(&e->cdev);
    unregister_chrdev_region(e->dev_no, 1);
    pci_free_irq_vectors(pdev);
}

static const struct pci_device_id edu_ids[] = {
    { PCI_DEVICE(0x1234, 0x11e8) },
    { },
};
MODULE_DEVICE_TABLE(pci, edu_ids);

static struct pci_driver edu_pci_driver = {
    .name     = "edu",
    .id_table = edu_ids,
    .probe    = edu_probe,
    .remove   = edu_remove,
};

module_pci_driver(edu_pci_driver);
MODULE_LICENSE("GPL");
```

Userspace test:

```
sudo insmod edu.ko
echo 5 | sudo tee /dev/edu     # 5! = 120
sudo cat /dev/edu              # prints 120
```

You wrote a complete PCI driver: enumerated, ioremap'd a BAR, requested an IRQ, built a chardev front-end, and validated with hardware (well, virtual hardware).

37.6 What's important

- `pcim_*` (managed) variants auto-clean on detach. Prefer them.
- Always `pci_set_master` before doing DMA.
- `pci_alloc_irq_vectors` — modern API; tries MSI-X, MSI, INTx in priority.
- `readl/writel` on `__iomem` memory; never deref directly.
- Use `dev_*` for prints — they include the BDF.

37.7 Beyond `edu`

Real GPUs have:

- many BARs (frame buffer, MMIO registers, doorbells),
- 64-bit BARs (`PCI_BASE_ADDRESS_MEM_TYPE_64`),
- multiple MSI-X vectors per IRQ source,
- PCIe link training / state,
- ASPM (Active State Power Management),
- Resizable BAR (so the CPU can map all of VRAM if the system supports it).

The amdgpu probe path enumerates all of these; you'll read it in Chapter 48.

37.8 Exercises

1. Build the edu driver against your kernel; run in QEMU; verify factorial.
2. Add an ioctl to read `EDU_REG_ID` (a fixed value 0xed).
3. Use the edu's DMA registers (read QEMU's `hw/misc/edu.c`) — kick a host→device→host memcpy.
4. Read `Documentation/PCI/pci.rst`. Long but useful.
5. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c::amdgpu_pci_probe`. Note how it differs from edu in scope; identify the analog of `pcim_iomap_regions`, `pci_alloc_irq_vectors`, `pci_set_drvdata`.

Next: Chapter 38 — DMA, IOMMU, coherent vs streaming mappings.

Chapter 38 — DMA, IOMMU, coherent vs streaming mappings

A device that can't DMA is useless. amdgpu DMAs constantly: pulling commands from system memory, pushing rendered frames to a scanout buffer in VRAM, copying textures, even running compute kernels via the SDMA engine. This chapter is the foundation.

38.1 What DMA is

The CPU could copy data between RAM and a device, but that wastes cycles. **Direct Memory Access** lets the device move data on its own — it has its own bus master that issues memory transactions. The CPU just programs "from here, this many bytes, to here" and goes to do something else.

Three actors:

- **CPU**: prepares buffers, kicks the DMA, optionally waits.
- **Device**: reads/writes memory.
- **IOMMU** (when present): translates *device-side* addresses to physical RAM.

38.2 The two DMA mappings

Type	API	Use
Coherent	`dma_alloc_coherent`	Long-lived buffers shared in both directions; descriptors, ring buffers, completion areas.
Streaming	`dma_map_single`, `dma_map_sg`, `dma_map_page`	One-shot transfers; data buffers; the device "owns" the buffer until you unmap.

Coherent memory is allocated by the kernel, mapped both into the CPU and the device, and is **automatically synchronized** (typically uncached or write-combining). No explicit flushes.

Streaming requires you to:

- map the buffer (`dma_map_single` returns a `dma_addr_t` — what the device sees),
- transfer ownership to the device,
- do the DMA,
- transfer back to the CPU via `dma_sync_*` if you re-read,
- unmap when done.

Why streaming? Coherent memory is expensive on platforms that need uncached or noncacheable mappings. For one-shot transfers, the cache flush + map is cheaper.

38.3 Coherent example

```
        dma_addr_t dma_handle;
        void *cpu_addr = dma_alloc_coherent(&pdev->dev, size, &dma_handle, GFP_KERNEL);
        if (!cpu_addr) return -ENOMEM;

        /* Tell device its physical-ish address (after IOMMU translation, this is what it sees). */
        writel(lower_32_bits(dma_handle), regs + DMA_LO);
        writel(upper_32_bits(dma_handle), regs + DMA_HI);

        /* CPU and device can read/write cpu_addr; no flushes. */

        dma_free_coherent(&pdev->dev, size, cpu_addr, dma_handle);
```

`dma_handle` is a **device-side address** (often called "bus address" or "DMA address"). With an IOMMU enabled, it's a virtual address in the device's IOMMU domain. Without IOMMU, it's the physical address (or platform-dependent translation).

38.4 Streaming example

```
        dma_addr_t dma_addr = dma_map_single(&pdev->dev, kbuf, size, DMA_TO_DEVICE);
        if (dma_mapping_error(&pdev->dev, dma_addr)) return -ENOMEM;

        writel(lower_32_bits(dma_addr), regs + DMA_LO);
        writel(upper_32_bits(dma_addr), regs + DMA_HI);
        writel(size,                    regs + DMA_LEN);
        writel(START,                   regs + DMA_CTL);

        /* device DMAs … completes via IRQ */

        dma_unmap_single(&pdev->dev, dma_addr, size, DMA_TO_DEVICE);
```

Direction:

- `DMA_TO_DEVICE` — CPU writes, device reads.
- `DMA_FROM_DEVICE` — device writes, CPU reads.
- `DMA_BIDIRECTIONAL` — both.

Direction tells the kernel what cache flushes to do.

38.5 Scatter-gather

For a big buffer that's many physically-discontiguous pages (e.g. a userspace mmap'd buffer):

```
        struct scatterlist sg[N];
        sg_init_table(sg, N);
        for (i = 0; i < N; i++) sg_set_page(&sg[i], pages[i], PAGE_SIZE, 0);

        unsigned int nents = dma_map_sg(&pdev->dev, sg, N, DMA_TO_DEVICE);
        /* program device with scatter list */
        dma_unmap_sg(&pdev->dev, sg, N, DMA_TO_DEVICE);
```

`dma_map_sg` may *coalesce* adjacent entries when the IOMMU can. Always use the returned `nents`, not your input `N`.

For modern code, prefer `sg_table` plus the `dma_buf_map_attachment` paths.

38.6 DMA mask

A device can DMA to a limited address range. For a 32-bit DMA-only device, only 4 GiB. Set the mask early:

```
        ret = dma_set_mask_and_coherent(&pdev->dev, DMA_BIT_MASK(40));
        if (ret) {
            dev_err(&pdev->dev, "no 40-bit DMA support\n");
            return ret;
        }
```

If the mask isn't satisfiable, the kernel will use the bounce buffer (SWIOTLB) — slow. Modern GPUs are 40+ bit; amdgpu uses 44-bit.

38.7 The IOMMU

The IOMMU translates **device addresses → physical RAM**:

- Programs the device with IO virtual addresses (IOVAs).
- Maps IOVAs to RAM via IOMMU page tables managed by the kernel.
- Devices with bugs or hostile firmware can't reach memory you didn't map.

AMD-Vi: AMD's IOMMU. VT-d: Intel's. Both expose the same Linux API.

Group: devices behind the same IOMMU group share the same IOVA space (e.g., functions of a multi-function PCI device). For VFIO passthrough, you must pass the entire group.

For driver code, you mostly *don't* care: `dma_*` APIs hide IOMMU behind them. You only care if:

- A weird IOMMU configuration breaks transfers (e.g. SWIOTLB engaged).
- You're integrating with VFIO (PCIe passthrough).
- You're touching the SVM (Shared Virtual Memory) path: an IOMMU feature where a device shares the CPU's process page tables — used by KFD for ROCm.

38.8 SWIOTLB / bounce buffer

When the device DMA mask can't reach the buffer's physical address, the kernel allocates a **bounce buffer** in low memory, copies the data, then DMAs. Slow. Common with old 32-bit devices.

If you see `swiotlb` in dmesg under load, your device or kernel is missing IOMMU support. amdgpu users sometimes hit this on platforms with broken Re-bar configurations.

38.9 dma-buf

A cross-driver buffer-sharing primitive. One driver allocates and *exports*; another *imports* and gets DMA mappings into its own context. Used heavily for:

- Wayland and Vulkan WSI (zero-copy framebuffers).
- V4L2 → GPU (camera → display).
- amdgpu BO ↔ another DRM driver via PRIME.

We discuss in Chapter 45 alongside GEM. The key API:

- `dma_buf_export(...)` — exporter creates.
- `dma_buf_get(fd)` — importer attaches.
- `dma_buf_attach`, `dma_buf_map_attachment` — get a `sg_table`.
- `dma_buf_vmap` — kernel virtual mapping.

38.10 Cache management

For coherent memory: nothing to do.

For streaming with `_TO_DEVICE`: CPU's writeback caches are flushed before device reads. The kernel does this in `dma_map_*`.

For streaming with `_FROM_DEVICE`: CPU caches are invalidated after device writes. `dma_unmap_*` does it.

For mid-DMA reads/writes by CPU on a streaming buffer:

```
dma_sync_single_for_cpu(&pdev->dev, dma_addr, size, DMA_FROM_DEVICE);
/* read */
dma_sync_single_for_device(&pdev->dev, dma_addr, size, DMA_FROM_DEVICE);
```

Forget these and you'll get random data corruption.

38.11 Minimal end-to-end DMA in our edu driver

QEMU's edu device supports DMA via registers `0x80..0x9C`. We add:

```
/* allocate coherent buffer */
dma_addr_t dh;
void *kva = dma_alloc_coherent(&pdev->dev, 4096, &dh, GFP_KERNEL);

/* fill */
memset(kva, 0xAB, 4096);

/* program edu DMA: src=dh, dst=intra-device offset, len, direction */
writeq(dh,          e->bar0 + 0x80);
writeq(0x40000,     e->bar0 + 0x88);  /* device-internal address */
writel(4096,        e->bar0 + 0x90);
writel(0x1,         e->bar0 + 0x98);  /* fire */

/* wait for IRQ, etc. */

dma_free_coherent(&pdev->dev, 4096, kva, dh);
```

You can extend Chapter 37's edu driver to do this — a useful exercise.

38.12 Exercises

1. Add a DMA path to the edu driver from Chapter 37: allocate coherent, write a pattern, DMA into device, DMA back, verify.
2. Repeat with `dma_map_single` on a `kmalloc` buffer; observe SWIOTLB on machines without IOMMU support enabled in firmware.
3. Read `Documentation/core-api/dma-api.rst`.
4. Read `Documentation/driver-api/dma-buf.rst`.

5. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_dma_buf.c`. We will revisit.

Next: Chapter 39 — MMIO, register access, barriers.

Chapter 39 — MMIO, register access, barriers

You've already used `readl` / `writel`. This chapter goes deeper: the memory model of MMIO, write coalescing, posted writes, and how to talk safely to GPU registers.

39.1 What MMIO is

Memory-Mapped I/O. The chipset and PCIe routing make some physical address ranges resolve not to DRAM but to a device. CPU loads/stores to those addresses become bus transactions to the device.

The CPU's caches typically *don't* cache MMIO ranges; they're marked **UC** (uncached) by the kernel.

To use:

```
void __iomem *base = ioremap(phys, size);
/* … readl/writel … */
iounmap(base);
```

Or the `pcim_iomap_regions` (Chapter 37) for PCI BARs.

39.2 The accessors

```
u8  readb(addr);   void writeb(u8  v, addr);
u16 readw(addr);   void writew(u16 v, addr);
u32 readl(addr);   void writel(u32 v, addr);
u64 readq(addr);   void writeq(u64 v, addr);
```

Why not just `*(u32*)addr = v`? Two reasons:

1. **`__iomem` annotation** — sparse static analysis catches accidental dereferences.
2. **Endianness and barrier semantics** — these are encoded in the accessor.

Variants:

- `readl_relaxed` / `writel_relaxed` — no DMA barrier; faster; you must add manual barriers.
- `ioread32` / `iowrite32` — alternate name; same thing.
- `readsl` / `writesl` — fixed-source/dest "string" loops; useful for FIFOs.

39.3 Posted writes

PCIe writes are **posted** by default: the CPU's write returns as soon as the write reaches the root complex's outbound buffer. The data has not yet reached the device; the device hasn't acknowledged.

Implication: a sequence like

```
        writel(START, regs + CTRL);
        status = readl(regs + STATUS);
```

…does not guarantee the device has *seen* the START write before the STATUS read. The read pushes the write through (because reads are non-posted) — that's the famous "read after write to flush a posted write" idiom.

Your code:

```
        writel(value, regs + CTRL);
        (void)readl(regs + CTRL);   /* flush posted write */
```

amdgpu uses register-poll waits everywhere; the read implicitly flushes.

39.4 Memory barriers between CPU and device

The kernel has DMA barriers:

- `wmb()` — write memory barrier; ensures stores complete before subsequent stores.
- `rmb()` — read memory barrier.
- `mb()` — full barrier.

`writel` already includes the necessary write barrier on most architectures. `writel_relaxed` does not.

Pattern:

```
        /* CPU: write descriptor in DRAM */
        desc->addr = ...;
        desc->len  = ...;
        wmb();                              /* descriptor visible */
        writel(cur_idx, regs + RING_HEAD);    /* doorbell */
```

Without the `wmb()`, the device might fetch the descriptor with old contents.

For "device wrote to memory; CPU reads":

```
        fence_val = readl(regs + FENCE);   /* device has updated mem before this */
        rmb();
        data = ring->shared[fence_val % N];
```

amdgpu's IB handlers and fence read paths follow exactly this.

39.5 Write combining

Some MMIO ranges are mapped **WC** (write-combining). The CPU's write-combining buffer accumulates 64-byte aligned writes and flushes them in a burst. Used for:

- Large frame-buffer writes (CPU drawing into VRAM scanout area).
- Command rings in WC system memory.

Write a partial 64 bytes? It still flushes when the buffer is full or you hit a barrier. Use `__iowrite32_copy` / `memcpy_toio` to leverage WC for big copies.

```
        memcpy_toio(dst_iomem, src_kernel, len);
```

For doorbell pages, WC is wrong (you want the write to go through immediately). Use UC.

39.6 Endianness

`readl` is **always** little-endian (most platforms; explicitly specified in the API). For a big-endian device on a little-endian CPU, swap manually:

```
        u32 v = be32_to_cpu(__raw_readl(addr));
```

amdgpu hardware is little-endian; you don't worry about this.

39.7 Address validation

When userspace passes a register offset (via ioctl/debugfs), validate it:

```
        if (offset > pci_resource_len(pdev, 0) - 4) return -EINVAL;
```

Otherwise userspace can read MMIO outside your BAR — which may map to *another device* and crash the system or read sensitive data.

amdgpu has explicit "register whitelists" for what userspace can read via the `AMDGPU_INFO` ioctl. You will read this code in Part VII.

39.8 Debugging MMIO

- `lspci -s 01:00.0 -v` — see BARs and memory regions.
- `cat /proc/iomem` — system address map.
- `mmiotrace` — kernel feature to log all MMIO accesses to a debugfs file. Build with `CONFIG_MMIOTRACE=y`.
- `devmem2 0xfb00000c 32` — read a register from userspace (root only). Dangerous; use only on devices you understand.

For amdgpu, set `amdgpu.mmio_debug=1` parameter to log register accesses. Useful when debugging hangs.

39.9 amdgpu register naming

Each ASIC family has thousands of registers, defined in headers like `gc_10_3_0_offset.h`, `gc_10_3_0_sh_mask.h`. The macros `RREG32_SOC15`, `WREG32_SOC15`, `WREG32_FIELD_SOC15`, etc. abstract over family.

```
        WREG32_SOC15(GC, 0, mmCP_RB_CNTL, value);
```

The driver builds register addresses dynamically based on chip family and instance. We'll dive in Chapter 48.

39.10 Exercises

1. In your edu driver, add a `relaxed` variant that drops the implicit barrier; measure throughput. Confirm the regular variant orders correctly.

2. Map an MMIO BAR with `ioremap_wc` and `memcpy_toio` 16 KB. Time it vs `readl`-loop equivalent.
3. Read `arch/x86/include/asm/io.h::readl` to see the exact instruction emitted.
4. `cat /proc/iomem` — find your GPU's BARs by name.
5. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_device.c::amdgpu_device_indirect_rreg`. Identify the register-access primitive amdgpu uses when MMIO BAR isn't directly accessible (over PCIe config-space MMIO bridge).

Next: Chapter 40 — Power management, runtime PM, suspend/resume.

Chapter 40 — Power management, runtime PM, suspend/resume

A driver that doesn't suspend cleanly is a driver that won't ship. Laptops, servers, and even VMs go through suspend/resume regularly. amdgpu's PM code is one of the gnarliest in the tree. We learn the framework now.

40.1 Two axes of PM

- **System suspend / resume** — user closes laptop lid. Kernel suspends every device, then the CPU. On resume, every device gets `resume()` called.
- **Runtime PM** — when a device is idle, the kernel calls `runtime_suspend`; on next access, `runtime_resume`. Per-device, while system is up.

amdgpu does both. ROCm compute systems also rely on runtime PM to reduce idle power.

40.2 The `dev_pm_ops` struct

```
static const struct dev_pm_ops mydrv_pm_ops = {
    .suspend         = mydrv_suspend,        /* system suspend */
    .resume          = mydrv_resume,         /* system resume */
    .freeze          = mydrv_freeze,         /* hibernate prep */
    .thaw            = mydrv_thaw,           /* hibernate undo */
    .poweroff        = mydrv_poweroff,
    .restore         = mydrv_restore,
    SET_RUNTIME_PM_OPS(mydrv_runtime_suspend,
                       mydrv_runtime_resume,
                       mydrv_runtime_idle)
};

static struct pci_driver my_driver = {
    /* … */
    .driver.pm = &mydrv_pm_ops,
};
```

The kernel calls these at the appropriate moments.

40.3 What suspend does, in order

System suspend (`pm_suspend`):

1. Freeze user processes.
2. Late callbacks for each device — quiesce work.
3. Disable non-boot CPUs.
4. Suspend last "noirq" callbacks.
5. CPU suspends (S3 / S0ix on x86, off on ARM).

Resume reverses.

For your driver, in `suspend`:

- Cancel all pending work; flush workqueues.
- Wait for in-flight DMA / IRQ events to complete.
- Save register state you'll need to restore.
- Disable interrupts at the device.
- Optionally power down the PHY.

In `resume`:

- Re-enable PCIe (`pci_enable_device`, `pci_set_master`).
- Restore registers.
- Re-arm interrupts.
- Resume scheduling.

40.4 Runtime PM lifecycle

Reference-counted. When the count drops to 0 and the device is "idle", runtime suspend may be called.

```
        pm_runtime_get(&pdev->dev);     /* +1, may resume if needed */
        /* … use device … */
        pm_runtime_put(&pdev->dev);     /* -1 */
```

For sync versions: `pm_runtime_get_sync` / `pm_runtime_put_sync`.

For an autosuspend pattern (don't suspend immediately on idle):

```
        pm_runtime_set_autosuspend_delay(&pdev->dev, 5000);   /* 5s */
        pm_runtime_use_autosuspend(&pdev->dev);
        pm_runtime_put_autosuspend(&pdev->dev);
```

In every IO entrypoint, `pm_runtime_get_sync` to ensure the device is up. amdgpu's ioctl wrappers do this implicitly.

40.5 amdgpu PM landscape

The amdgpu PM code spans several systems:

- **`amdgpu_device.c::amdgpu_pmops_*`** — PCI `dev_pm_ops` callbacks.
- **`amdgpu/pm/`** — DPM (Dynamic Power Management): voltage/frequency scaling.
- **SMU** — System Management Unit: a microcontroller in the GPU running power firmware. The driver communicates via mailboxes.
- **DC** — Display Core: does its own modeset/PSR (Panel Self Refresh) for power.
- **runtime PM** — used for D3cold (full power off) when the GPU is idle and no display attached.

This is one of the most-touched areas of amdgpu in production patches. Understanding PM is a fast path to ownership of features.

40.6 The “atomic suspend” rule

In `*_noirq` callbacks, IRQs are disabled. You must not sleep. In `*_late` callbacks (one phase earlier), userspace is frozen but IRQs work. Pick the right phase.

amdgpu uses `pmops_prepare`, `pmops_complete`, `pmops_suspend`, `pmops_resume` plus the `_noirq` variants for the very-late ordering of GPU power.

40.7 D-states (PCIe)

PCIe defines D0 (active) through D3hot (deepest with config space) and D3cold (power off). The CPU can put a PCIe device in any state via config-space writes.

```
        pci_save_state(pdev);
        pci_set_power_state(pdev, PCI_D3hot);
        /* … later … */
        pci_set_power_state(pdev, PCI_D0);
        pci_restore_state(pdev);
```

For runtime suspend, amdgpu typically goes to D3hot or D3cold (the latter via ACPI hooks) depending on platform.

40.8 ACPI integration

ACPI provides platform-specific suspend hooks. The GPU’s ACPI methods (`_PR3`, `_DSM`) are called by the kernel’s PM core or directly by amdgpu. Cross-checking ACPI on a particular platform is a real debugging effort when “the lid closes but the GPU never comes back.”

40.9 Practical advice

When you start landing patches in amdgpu PM:

- Read `Documentation/power/runtime_pm.rst`.
- Read `Documentation/driver-api/pm/devices.rst`.
- Test with `rtcwake -m mem -s 10` (RTC-triggered S3) and `echo platform | sudo tee /sys/power/disk; echo disk | sudo tee /sys/power/state` (hibernate).
- Check the platform variant: laptops use ASPM and L1.X states; desktops sometimes don’t.
- Watch dmesg for `PM:` lines; they tell you what’s happening at each phase.

40.10 Exercises

1. Add `suspend` / `resume` to your edu driver: save/restore the contents of EDU_REG_FACT.
2. Add runtime PM with autosuspend; instrument with `pm_runtime_*` calls in your read/write paths.
3. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c::amdgpu_pmops_suspend`.
4. Read `Documentation/power/runtime_pm.rst`.
5. Use `rtcwake -m mem -s 30` to trigger suspend; observe your driver’s callbacks; verify state.

Next: Chapter 41 — debugfs, sysfs, tracing, ftrace, perf.

Chapter 41 — debugfs, sysfs, tracing, ftrace, perf

You will spend more hours debugging than coding. The kernel has world-class instrumentation; learning to wield it is a force multiplier.

41.1 dmesg / printk

Default output channel. Use `pr_info`, `pr_err`, `dev_info`, `drm_info`. Levels at top (Chapter 29).

```
dmesg -w        # follow new messages
dmesg -T        # human time
dmesg -l err,warn   # filter
journalctl -k   # systemd's kmsg view
```

Boot-time `loglevel=8` raises console verbosity.

`dynamic debug` lets you toggle individual `pr_debug`s at runtime:

```
echo 'file amdgpu_cs.c +p' | sudo tee /sys/kernel/debug/dynamic_debug/control
```

41.2 sysfs vs debugfs

	sysfs (`/sys`)	debugfs (`/sys/kernel/debug`)
Stable ABI?	Yes — versioned, documented	No — develop-only
One value per file	Yes	No (anything goes)
Use for	User config, status	Driver dev, deep inspection
Helpers	`DEVICE_ATTR_*`, `sysfs_emit`	`debugfs_create_*`

debugfs example:

```
static int amdgpu_fence_info_show(struct seq_file *m, void *p)
{
    seq_printf(m, "current seqno %u\n", current_seqno);
    return 0;
}
DEFINE_SHOW_ATTRIBUTE(amdgpu_fence_info);

debugfs_create_file("fence_info", 0444, parent, dev, &amdgpu_fence_info_fops);
```

amdgpu has dozens of debugfs files — fence rings, BO lists, pm voltage tables, register dumps.

41.3 ftrace

Kernel function tracer. The most powerful day-to-day tool.

```
sudo trace-cmd record -p function_graph -l 'amdgpu_*'
# … reproduce …
sudo trace-cmd stop
sudo trace-cmd report > trace.txt
```

You see every call into amdgpu, indented by depth. For a hung path, this is gold.

Higher-level: `tracepoint`s — events drivers emit at named points. amdgpu adds tracepoints in `amdgpu_trace.h`. View with:

```
sudo trace-cmd list -e amdgpu
sudo trace-cmd record -e amdgpu_cs
```

41.4 perf

CPU profiler. Used to find hotspots:

```
sudo perf record -F 99 -ag -- sleep 5
sudo perf report
sudo perf script | flamegraph.pl > flame.svg
```

For driver work, attach to your process or system-wide. `perf top` for live.

`perf trace` is `strace` on perf — much faster.

41.5 BPF / bpftrace

```
sudo bpftrace -e 'tracepoint:amdgpu:amdgpu_cs { @[comm] = count(); }'
```

One-liners for dynamic tracing. `bcc` and `bpftrace` are essential for production tracing.

41.6 GDB on the kernel

For VM-based dev:

```
qemu … -s -S
gdb vmlinux
(gdb) target remote :1234
(gdb) hbreak start_kernel
(gdb) c
```

For real machines, **kgdb** over serial. Set up early or you'll never reach a kernel panic.

41.7 crash and pstore

When a panic happens on a real machine you have:

- **kdump**: another kernel boots and saves the crashed kernel's memory; analyze with `crash`.
- **pstore**: persistent ramoops; reads dmesg from RAM after reboot.

You will need these once you hit a real GPU panic on a customer machine. AMD's bug tracker has many "please attach kdump."

41.8 KASAN, KCSAN, lockdep, DEBUG_VM

Already mentioned. Build with these on for development. They catch bugs you'd otherwise spend days hunting.

41.9 Memory leak detection

- **kmemleak** — `CONFIG_DEBUG_KMEMLEAK=y`. Scans memory for orphaned allocations periodically.

```
echo scan | sudo tee /sys/kernel/debug/kmemleak
sudo cat /sys/kernel/debug/kmemleak
```

Best when you can reliably reproduce.

41.10 amdgpu-specific debug knobs

Module parameters (boot or rmmod/insmod) controlling amdgpu:

```
amdgpu.debug_mask=0x...
amdgpu.runpm=0      # disable runtime PM
amdgpu.dpm=0        # disable DPM
amdgpu.gpu_recovery=2
amdgpu.mes=0        # disable MES (microcontroller scheduler)
```

debugfs files:

- `/sys/kernel/debug/dri/0/amdgpu_pm_info`
- `/sys/kernel/debug/dri/0/amdgpu_gpu_recover` — write 1 to force a reset.
- `/sys/kernel/debug/dri/0/amdgpu_fence_info`
- `/sys/kernel/debug/dri/0/amdgpu_gem_info`
- `/sys/kernel/debug/dri/0/amdgpu_vm_info`

When triaging a hang, dump these.

41.11 ROCm-side tools

- `rocm-smi` — voltage, temperature, fan, clocks.
- `rocminfo` — discovery.
- `rocprof` — kernel-level profiler for HIP.
- `rocgdb` — GDB for AMDGPU device code.

You'll cross between kernel and userspace tools constantly.

41.12 Reading dmesg traces

A typical amdgpu hang trace:

```
[  XXX.XXX] amdgpu 0000:01:00.0: [drm:amdgpu_ring_test_helper] *ERROR* ring sdma0 timeout
[  XXX.XXX] amdgpu 0000:01:00.0: [drm] GPU recovery starts
[  XXX.XXX] amdgpu 0000:01:00.0: [drm] failed to suspend display engine
[  XXX.XXX] Call Trace:
```

```
[  XXX.XXX]  ? amdgpu_device_gpu_recover.cold+...
[  XXX.XXX]  ? drm_sched_main+...
```

The `Call Trace` is the kernel stack at panic. Resolve symbol offsets to source line with:

```
addr2line -e vmlinux 0xffffffffc04a5e3a
```

Or use `gdb`:

```
gdb vmlinux
(gdb) list *(amdgpu_device_gpu_recover+0x1f8)
```

Mastery of stacks → mastery of triage.

41.13 Exercises

1. Enable dynamic_debug on your driver; toggle pr_debug live.
2. Add a tracepoint to your edu driver: `TRACE_EVENT(edu_factorial, TP_ARGS(u32 input, u32 output), …)`. Use `trace-cmd record -e edu_factorial`.
3. Run `perf record -ag` on a stress workload of your driver; identify the hot function.
4. Build a kernel with KASAN+KMEMLEAK; introduce a leak; observe.
5. Write down the contents of three amdgpu debugfs files on your machine. Identify what each tells you.

End of Part VI. Move to Part VII — The Linux GPU stack (DRM + amdgpu).

Chapter 42 — GPU architecture for software engineers

You can't write GPU drivers without a clear picture of the hardware. This chapter is a fast tour, AMD-specific where it matters.

42.1 What a GPU is

A GPU is a wide SIMD machine optimized for *throughput*. CPUs minimize latency; GPUs maximize throughput.

Trade-offs:

- CPU: 8–96 cores, deep pipelines, big caches, branch predictors.
- GPU: hundreds to thousands of "lanes", shallow pipelines, small per-core caches but enormous register files, hardware multithreading to hide memory latency.

Modern AMD GPUs (RDNA / CDNA):

- **Compute Units (CUs)** — the basic block. Each has 64 ALUs (lanes) organized as 4 SIMDs (RDNA: 32-wide × 2 sub-issue; CDNA: 64-wide).
- **Wavefronts (waves)** — group of 32 (RDNA) or 64 (CDNA) lanes executing in lock-step.
- **Workgroups** — collections of waves that share **LDS** (Local Data Share, a.k.a. shared memory in CUDA terms).
- **Shader Engines / Shader Arrays** — clusters of CUs, sharing fixed-function units.
- **L1, L2 caches** — L1 per CU; L2 globally shared, banked.
- **Memory controllers** — connected to GDDR / HBM.
- **DCN** (Display Core Next) — display engines, modesetting.
- **VCN** (Video Core Next) — video encode/decode.

A high-end GPU has dozens of CUs and an enormous (~2 TB/s) memory subsystem.

42.2 The blocks you'll touch

Inside the chip, the units of interest to a driver writer:

- **GFX (graphics)** — graphics + general compute pipeline. Has the **CP** (Command Processor) that fetches and dispatches commands.
- **MEC** (Micro-Engine Compute) — older compute engine; queue-driven.
- **MES** (Micro-Engine Scheduler) — newer firmware-based scheduler.
- **SDMA** — System DMA engine; copy buffers between GTT/VRAM/host.
- **DCN** — display.
- **VCN** — video.
- **PSP** (Platform Security Processor) — secure firmware loader.
- **SMU** — System Management Unit; power/thermal/voltage.
- **IH** — Interrupt Handler (the hardware that delivers IRQs).

- **GMC / VMID** — GPU Memory Controller; VM IDs for multi-process.

amdgpu has a directory per IP block: `gfx_v10_0.c`, `sdma_v4_4_2.c`, `vcn_v3_0.c`, `mes_v11_0.c`, etc. The driver is structured around these.

42.3 The command-processor model

The CPU produces work; the GPU consumes it via a ring buffer of "packets." Common packet types:

- **PM4** (graphics CP packets) — the GFX language. Examples:
  - `INDIRECT_BUFFER` — jump to an IB and execute it.
  - `WRITE_DATA` — write a dword to memory (often used as fence).
  - `WAIT_REG_MEM` — poll a memory location for a value.
  - `RELEASE_MEM` — write fence + flush caches.
  - `ACQUIRE_MEM` — invalidate caches.
- **SDMA packets** — copy, fill, write.
- **VCN packets** — video.
- **MES packets** — submit/scheduler control.

A "command buffer" is a sequence of packets in memory. Userspace builds one (an **IB** — Indirect Buffer), then submits via ioctl. The kernel:

1. Validates the IB (relocations, register white-list).
2. Wraps it with prologue/epilogue PM4 (cache flushes, fence).
3. Pushes onto a ring (or hands to MES).
4. Triggers a doorbell.

We dedicate Chapter 49 to this.

42.4 The ISA (briefly)

AMD's GPU ISA — RDNA, CDNA, GCN — is documented in PDFs ("RDNA 3 ISA", "CDNA 3 ISA"). You don't need to read it day one, but if you do GFX bring-up or compute optimization, you will.

The userspace shader compiler (LLVM AMDGPU backend) emits ISA. The kernel rarely cares about ISA contents — it just shovels code into VRAM and tells the CP "here's a shader at address X."

42.5 GPU memory architecture

Memory the GPU can see, in priority order:

1. **VRAM** (GDDR6/HBM): high-bandwidth, GPU-local, *not* CPU-coherent.
2. **GTT** (Graphics Translation Table): system RAM the GPU can access via the IOMMU; CPU-shared.
3. **System memory via userptr**: pinned user pages mapped into GPU VM.
4. **DOORBELL**: tiny, special PCIe pages mapped into the GPU's MMIO.

GPU virtual addresses are *per-process* on modern AMD parts — every userspace process has its own GPUVM with its own VMID. The GPU's MMU walks page tables prepared by the kernel.

When userspace allocates a BO (buffer object), the kernel chooses placement (VRAM, GTT, or both with eviction) and maps it into the process's GPUVM. We cover in Chapter 51.

42.6 The whole stack, top to bottom

```
 OpenGL / Vulkan / OpenCL / HIP             (Mesa, ROCclr)
              ▲
userspace driver: IB build, BO mgmt          (mesa/radeonsi, rocclr)
              ▲
     libdrm + ioctl                          ( libdrm_amdgpu )
              ▲ syscall
──────────────┼──────────────────
amdgpu.ko   ▼
  │
  ├── DRM core: file_ops, ioctl dispatch
  ├── GEM: buffer object handles
  ├── TTM: VRAM/GTT placement, eviction
  ├── DRM scheduler: per-engine queues
  ├── IRQ: IH ring drain → tasklet
  ├── PM: DPM, runtime pm
  ├── DC: display modesetting
  ├── psp: firmware load + security
  ├── gfx/sdma/vcn/mes: per-IP ring submit
  └── KFD: HSA / ROCm path (separate fd)
              │ MMIO + DMA + IRQ
              ▼
            GPU silicon
```

We will build out each box in subsequent chapters.

42.7 The KFD path (ROCm)

AMD has *two* ways into the GPU from userspace:

- **DRM/render** path (`/dev/dri/renderD*`): graphics, Vulkan, OpenGL, OpenCL via Mesa/Radeon, etc.
- **KFD path** (`/dev/kfd`): for ROCm — HSA queues, signals, SVM. Used by HIP, OpenCL ROCm runtime.

KFD is in `drivers/gpu/drm/amd/amdkfd/`. It shares device with amdgpu (KFD is initialized by amdgpu). The KFD interface offers user-mode queues — userspace fills doorbells directly and the GPU's MES runs them. This is how ROCm achieves low submission overhead.

42.8 What a GPU "kernel" is

In compute parlance, a **kernel** is a small function that runs on the GPU, dispatched as a grid of workgroups. CUDA calls them `__global__` functions; HIP/OpenCL/CUDA all call them kernels.

In Linux/driver parlance, "kernel" means the OS kernel.

We disambiguate by context. In Part VII, "GPU kernel" or "compute kernel" or "shader" = the workgroup function. The OS kernel = "the kernel."

42.9 Bring-up overview (preview of Chapter 48)

When amdgpu binds to a GPU at boot:

1. Detect ASIC family from PCI ID.
2. Map BARs.
3. Enable PCIe.
4. Init PSP and load firmware (every IP has a `.bin`).
5. Init GMC, VMIDs, page tables.
6. Init GFX (CP), SDMA, VCN, JPEG, MES, etc.
7. Init DC (display).
8. Init DPM/SMU.
9. Register DRM device, expose ioctls and `/dev/dri/cardN` + `/dev/dri/renderDN`.

Each step has hundreds of lines. The driver is *enormous* because GPUs are enormous.

42.10 Exercises

1. `lspci -vv -s 01:00.0` (your GPU's BDF). Identify all BARs and capabilities.
2. `cat /sys/class/drm/card*/device/uevent`.
3. Read `Documentation/gpu/amdgpu/index.rst` from the kernel.
4. Skim the AMD RDNA 3 architecture white-paper (publicly available). You don't need to memorize; orient yourself.
5. Read `drivers/gpu/drm/amd/amdgpu/amdgpu.h` — the omnibus header. Identify `struct amdgpu_device` and its many sub-structs.

Next: Chapter 43 — The Linux graphics stack from app to silicon.

Chapter 43 — The Linux graphics stack from app to silicon

Before we dive into the kernel side, we need to know what's on top: how a Vulkan call reaches amdgpu, what Mesa is, what libdrm is, what KMS is. Then the kernel work makes sense.

43.1 The big picture

```
┌──────────────────────────────────────────────────────────────┐
│   Application: glmark2, vkcube, dav1d, blender, hipify-app   │
└──────────────────────────────────────────────────────────────┘
                        │
              OpenGL / Vulkan / OpenCL / HIP
                        │
┌──────────────────────▼───────────────────────────────────────┐
│   API loader: libGL.so / libvulkan.so / libamdhip64.so       │
└──────────────────────────────────────────────────────────────┘
                        │
                        │
┌──────────────────────▼───────────────────────────────────────┐
│   Vendor userspace driver:                                   │
│     - Mesa radeonsi  (OpenGL, GLES)                          │
│     - Mesa radv      (Vulkan)                                │
│     - AMDVLK         (AMD's open Vulkan)                     │
│     - ROCclr / ROCr  (HIP, OpenCL ROCm)                      │
│     - PAL / DXC      (proprietary on Windows)                │
│   builds command buffers, manages BOs, submits via libdrm    │
└──────────────────────────────────────────────────────────────┘
                        │ libdrm_amdgpu.so
┌──────────────────────▼───────────────────────────────────────┐
│   /dev/dri/renderDxxx  (or /dev/kfd)                         │
└──────────────────────────────────────────────────────────────┘
                        │
              syscalls (ioctl, mmap, …)
┌──────────────────────▼──────────────────────────────────────┐
│   amdgpu.ko (kernel)                                         │
│      DRM core, GEM, TTM, DRM-Sched, KMS, PM, IRQ             │
└──────────────────────────────────────────────────────────────┘
                        │ MMIO + DMA + doorbells
┌──────────────────────▼───────────────────────────────────────┐
│   GPU silicon                                                │
└──────────────────────────────────────────────────────────────┘
```

You will work in the `amdgpu.ko` row, but understanding the rows above lets you debug from "Vulkan game flickers" to "dma-buf import fails."

43.2 The userspace driver's job

Given a Vulkan command list:

1. **Compile shaders** — the driver embeds an LLVM-backed compiler producing AMDGPU ISA.
2. **Allocate buffers** — vertex, index, descriptor, image, uniform. Each becomes a BO via `DRM_IOCTL_AMDGPU_GEM_CREATE`.
3. **Build a PM4 / SDMA stream** in CPU-mapped memory.

4. **Submit** via `DRM_IOCTL_AMDGPU_CS` with the IB and a list of BOs ("BO list") and dependency fences.
5. **Wait** on the returned fence (`DRM_IOCTL_AMDGPU_CS` returns a sequence number; userspace can poll or `DRM_IOCTL_AMDGPU_WAIT_FENCES`).

amdgpu's userspace partner, `libdrm_amdgpu`, wraps these ioctls in a clean C API.

43.3 The display path: DRM/KMS

For displaying a rendered frame:

1. Userspace allocates a "scanout-able" BO.
2. Renders into it.
3. Calls `DRM_IOCTL_MODE_CRTC_PAGE_FLIP` (or atomic: `DRM_IOCTL_MODE_ATOMIC`) to flip the active framebuffer.
4. Kernel programs the display engine.

This is the **KMS** (Kernel Mode Setting) part of DRM. amdgpu's display piece is **DC** (Display Core) — a huge separate codebase under `drivers/gpu/drm/amd/display/`.

For headless / compute-only systems, KMS is not used; userspace just opens `/dev/dri/renderD*`.

43.4 `/dev/dri/cardN` vs `/dev/dri/renderDN`

- **cardN** — primary node. Has KMS ioctls. Requires `DRM_AUTH` for sensitive things.
- **renderDN** — render-only node. No KMS, no auth. Cheaper, used by compute and many Vulkan render-only paths.

For compute-only scenarios, `renderD*` is the entry. ROCm primarily uses `renderD*` and `/dev/kfd`.

43.5 Key userspace bits

- **Mesa**: open-source GL/Vulkan/OpenCL stack. `radeonsi` (GL), `radv` (Vulkan), `clover`/`rusticl` (OpenCL).
- **AMDVLK**: AMD's official Vulkan driver, also open-source.
- **libdrm**: thin wrapper over DRM ioctls. `libdrm_amdgpu` is its amdgpu-specific subset.
- **wlroots / Mutter / Sway / KWin**: Wayland compositors using KMS+DRM directly.
- **ROCm runtime (ROCr/HSA)**: `librocm-hsa-runtime64.so`. Talks to `/dev/kfd`.
- **HIP runtime**: `libamdhip64.so`. Built on ROCr.

43.6 Patches typically span layers

A single feature (say, a new compression mode) may require:

- **HW spec / ISA** updated (silicon team).
- **kernel** ioctl tag and new packet generation in amdgpu.
- **libdrm_amdgpu** API addition.
- **Mesa radv** updated to use the new mode.

- **Vulkan tests** in `vkcts` updated.

Working on one layer demands at least a reading-knowledge of the rest. Pick a feature you care about and follow its path through all layers — that's the fastest way to develop perspective.

43.7 Wayland and X11 differences (briefly)

- **X11**: monolithic server. DRI3 lets clients render to dma-bufs, server scans them out.
- **Wayland**: compositor is the server; clients hand dma-bufs to it. Direct, less latency. Linux is migrating to Wayland.

For driver work, both look the same: a process opens `/dev/dri/cardN`, becomes DRM master, programs KMS. Below KMS the driver doesn't see who the client was.

43.8 ROCm path

For HIP/ROCm:

```
HIP app → libamdhip64.so → libhsa-runtime64.so → /dev/kfd → amdkfd/amdgpu
```

The KFD path bypasses graphics: it allocates BOs through KFD ioctls, registers user-mode queues with MES, and userspace writes commands directly into shared queues without per-submit ioctl. Doorbells trigger the MES to run them.

This drives the very low submission overhead ROCm needs.

43.9 Putting your first userspace prog together

A "hello GPU" with libdrm_amdgpu (we'll write fully in Chapter 52):

```
amdgpu_device_handle dev;
amdgpu_device_initialize(fd, &major, &minor, &dev);

amdgpu_bo_handle bo;
amdgpu_bo_alloc_request req = { .alloc_size = 4096, .preferred_heap = AMDGPU_GEM_DOMAIN_GTT };
amdgpu_bo_alloc(dev, &req, &bo);

amdgpu_bo_cpu_map(bo, &cpu_addr);
((uint32_t*)cpu_addr)[0] = 0xCAFEBABE;
amdgpu_bo_cpu_unmap(bo);

amdgpu_bo_free(bo);
amdgpu_device_deinitialize(dev);
```

Less than 30 lines and you're talking to the GPU. We'll extend this to actually submit a command stream.

43.10 Exercises

1. `vulkaninfo` (install `vulkan-tools`). Read every line. Note the device name, queues, memory heaps.
2. `glxinfo | grep OpenGL` — the renderer string from Mesa.
3. `ls /dev/dri` — see your nodes. Read `/sys/class/drm/renderD*/device/driver`.

4. `strace -f -e trace=ioctl glxgears 2>&1 | head -200` — observe the DRM ioctls.
5. Read `Documentation/gpu/drm-uapi.rst`.

Next: Chapter 44 — DRM, the kernel's GPU framework.

Chapter 44 — DRM, the kernel's GPU framework

DRM (Direct Rendering Manager) is the kernel subsystem every Linux GPU driver lives in. amdgpu, i915, nouveau, msm, etnaviv, lima, panfrost — all are DRM drivers. This chapter lays out DRM's structure and the boilerplate every driver writes.

44.1 What DRM provides

DRM provides:

- A **char device** (`/dev/dri/cardN`, `/dev/dri/renderDN`).
- A **standard ioctl set** (open/close, GEM, syncobj, prime, KMS).
- **Per-driver ioctls** registered as a vendor table (the AMDGPU_* ioctls).
- **GEM** — a generic buffer-object framework.
- **Atomic KMS** — modesetting.
- **DRM scheduler** — generic GPU job scheduler used by amdgpu.
- **PRIME / dma-buf** — cross-driver buffer sharing.
- **DRM master** — exclusive control of display.
- **GEM helpers**, **GEM SHMEM**, **GEM CMA** — buffer-management helpers for simpler drivers.

The core lives in `drivers/gpu/drm/`. amdgpu under `amd/amdgpu/`.

44.2 Where DRM glues in

```
drivers/gpu/drm/drm_*.c              ← DRM core
drivers/gpu/drm/scheduler/*.c        ← DRM scheduler
drivers/gpu/drm/ttm/*.c              ← TTM (used by amdgpu)
drivers/gpu/drm/amd/amdgpu/*.c       ← amdgpu IPs
drivers/gpu/drm/amd/display/*        ← DC display
drivers/gpu/drm/amd/include/*        ← register defs
include/drm/*.h                       ← DRM API headers
```

44.3 The driver-author's deliverables

A DRM driver provides at minimum:

1. A `struct drm_driver` (the vtable for everything).
2. An `unsigned long driver_features` mask (`DRIVER_GEM`, `DRIVER_RENDER`, `DRIVER_MODESET`, `DRIVER_ATOMIC`, …).
3. Per-driver ioctls: `struct drm_ioctl_desc ioctls[]`.
4. `file_operations` glue (mostly stock helpers).
5. `drm_device` allocation (`drm_dev_alloc`) and registration (`drm_dev_register`).
6. Optional: KMS implementation, BO management.

amdgpu's `struct drm_driver`:

```
static struct drm_driver kms_driver = {
    .driver_features  = DRIVER_ATOMIC | DRIVER_GEM | DRIVER_RENDER
                      | DRIVER_MODESET | DRIVER_SYNCOBJ
                      | DRIVER_SYNCOBJ_TIMELINE,
    .open             = amdgpu_driver_open_kms,
    .postclose        = amdgpu_driver_postclose_kms,
    .lastclose        = amdgpu_driver_lastclose_kms,
    .ioctls           = amdgpu_ioctls_kms,
    .num_ioctls       = ARRAY_SIZE(amdgpu_ioctls_kms),
    .fops             = &amdgpu_driver_kms_fops,
    .gem_prime_import = amdgpu_gem_prime_import,
    /* … */
};
```

We'll see this struct repeatedly.

44.4 DRM file_operations

`amdgpu_driver_kms_fops` is mostly stock helpers:

```
static const struct file_operations amdgpu_driver_kms_fops = {
    .owner    = THIS_MODULE,
    .open     = drm_open,
    .release  = drm_release,
    .unlocked_ioctl = amdgpu_drm_ioctl,
    .mmap     = drm_gem_mmap,
    .poll     = drm_poll,
    .read     = drm_read,
    .compat_ioctl = amdgpu_kms_compat_ioctl,
    .llseek   = noop_llseek,
};
```

`drm_open`, `drm_release` — DRM core's open/close. They allocate the per-fd `struct drm_file`, hook into authentication, and call `drv->open` when present.

44.5 The ioctl table

```
static const struct drm_ioctl_desc amdgpu_ioctls_kms[] = {
    DRM_IOCTL_DEF_DRV(AMDGPU_GEM_CREATE, amdgpu_gem_create_ioctl, DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_CTX,        amdgpu_ctx_ioctl,        DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_BO_LIST,    amdgpu_bo_list_ioctl,    DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_CS,         amdgpu_cs_ioctl,         DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_INFO,       amdgpu_info_ioctl,       DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_GEM_METADATA,amdgpu_gem_metadata_ioctl, DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_GEM_VA,     amdgpu_gem_va_ioctl,     DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_GEM_OP,     amdgpu_gem_op_ioctl,     DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_GEM_USERPTR,amdgpu_gem_userptr_ioctl,DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_WAIT_CS,    amdgpu_cs_wait_ioctl,    DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_WAIT_FENCES,amdgpu_cs_wait_fences_ioctl, DRM_AUTH | DRM_RENDER_ALLOW),
    DRM_IOCTL_DEF_DRV(AMDGPU_SCHED,      amdgpu_sched_ioctl,      DRM_MASTER),
    DRM_IOCTL_DEF_DRV(AMDGPU_VM,         amdgpu_vm_ioctl,         DRM_AUTH | DRM_RENDER_ALLOW),
    /* … */
};
```

Each entry points at a function and declares permissions. `DRM_RENDER_ALLOW` lets the ioctl be used on `renderD*` (no DRM-master required).

DRM core handles the copy_from/to_user, validates struct sizes, and dispatches.

44.6 The `drm_device`

`struct drm_device` is DRM's device handle. Allocated with `drm_dev_alloc()`, registered with `drm_dev_register()`. Embedded inside a driver-specific struct (e.g. `struct amdgpu_device`):

```
struct amdgpu_device {
    struct drm_device         ddev;     /* embedded */
    struct pci_dev           *pdev;
    /* … hundreds more fields … */
};
```

`container_of(ddev, struct amdgpu_device, ddev)` recovers the parent.

44.7 The `drm_file`

Per-open file. Points to `drm_device`. Has `driver_priv` for the driver's per-fd state (`struct amdgpu_fpriv`).

`amdgpu_fpriv` holds:

- a per-process VM (GPUVM),
- a context array (`amdgpu_ctx_mgr`),
- BO list table,
- syncobj table.

Multiple opens by one process can have multiple file pointers; usually one process opens once.

44.8 Authentication and master

For KMS ioctls, DRM master controls the display. Only one process at a time. X11 server, Wayland compositor become master.

Render-only clients (Vulkan apps, ROCm runtime) don't need master — they use `renderD*`.

In `amdgpu_ioctl`, DRM core checks the flags. Drivers don't re-check.

44.9 dma-buf / PRIME

DRM's dma-buf integration goes by the name **PRIME**. Two ioctls:

- `DRM_IOCTL_PRIME_HANDLE_TO_FD` — turn a GEM handle into a dma-buf fd.
- `DRM_IOCTL_PRIME_FD_TO_HANDLE` — import a dma-buf fd as a GEM handle.

Used for hand-off between two GPUs (hybrid graphics, prime offload), or between GPU and codec/display.

amdgpu provides `gem_prime_import` / `gem_prime_export` callbacks that work with TTM's dma-buf integration.

44.10 KMS at a glance (full chapter later)

For display:

- **Connector**: physical port (HDMI, DP, eDP).
- **Encoder**: signal generator (TMDS, DP).
- **CRTC**: frame timing generator + scanout pipe.
- **Plane**: image source layered onto a CRTC (primary, cursor, overlay).
- **Framebuffer**: GEM BO with format/pitch metadata.

Atomic modeset: state is a struct; you build a desired state; commit atomically. Either everything goes or nothing.

amdgpu DC implements all of this.

44.11 The DRM scheduler (full chapter later)

`drivers/gpu/drm/scheduler/` provides per-hardware-queue scheduling: takes `drm_sched_job`s, respects input fences, runs them in order, signals output fences. Each amdgpu engine ring has a `drm_gpu_scheduler`.

We treat in Chapter 50.

44.12 The minimal DRM driver

For comparison, `vkms` (Virtual KMS) in `drivers/gpu/drm/vkms/` is ~1500 lines and a complete DRM driver. Read it; it is a wonderful exemplar.

```
static struct drm_driver vkms_driver = {
    .driver_features = DRIVER_MODESET | DRIVER_ATOMIC | DRIVER_GEM,
    .open            = vkms_postclose,
    .fops            = &vkms_driver_fops,
    .gem_prime_import_sg_table = vkms_prime_import_sg_table,
    DRM_GEM_SHMEM_DRIVER_OPS,
    .name            = "vkms",
    /* … */
};
```

`DRM_GEM_SHMEM_DRIVER_OPS` is a macro that fills in the GEM SHMEM helpers. amdgpu uses TTM instead, which is the heavier-weight path.

44.13 Exercises

1. Read the entire `vkms` driver (4–6 files). Identify how it implements every required hook.
2. Read `include/drm/drm_drv.h::struct drm_driver`. Understand each field.
3. Read `drivers/gpu/drm/drm_drv.c::drm_dev_alloc`, `drm_dev_register`. Note refcounting.
4. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c::amdgpu_ioctls_kms`. Identify all user-visible ioctls.
5. Open `/dev/dri/renderD128` from a userspace program; call `DRM_IOCTL_VERSION` to get the driver name and major/minor.

Next: Chapter 45 — GEM — buffer objects.

Chapter 45 — GEM — buffer objects

A GPU is, more than anything, a buffer-eating machine. Vertex buffers, textures, framebuffers, command buffers, ring buffers, descriptor sets, intermediate render targets — every GPU operation reads or writes a buffer. **GEM** (Graphics Execution Manager) is DRM's framework for naming, owning, sharing, and mapping those buffers.

45.1 What a GEM object is

A `struct drm_gem_object` is a kernel object representing one buffer:

```
struct drm_gem_object {
    struct kref         refcount;
    unsigned int        handle_count;
    struct drm_device  *dev;
    struct file        *filp;
    struct kobject     *kobj;

    struct dma_resv    *resv;          /* fences attached to this buffer */
    struct dma_resv     _resv;
    size_t              size;
    int                 name;          /* legacy global handle */
    /* … */
};
```

Driver embeds it inside the driver's BO struct:

```
struct amdgpu_bo {
    struct ttm_buffer_object  tbo;     /* TTM embeds drm_gem_object */
    /* … placement, mappings, metadata … */
};
```

Hierarchy: `drm_gem_object` < `ttm_buffer_object` < `amdgpu_bo`.

45.2 Handles vs. objects

Userspace doesn't see a kernel pointer — it sees a small **handle** (uint32). Handles are allocated per `drm_file`; they are NOT shared across processes.

To pass a buffer to another process: convert handle → fd via PRIME (`DRM_IOCTL_PRIME_HANDLE_TO_FD`), pass the fd over a Unix socket, then the receiver does `DRM_IOCTL_PRIME_FD_TO_HANDLE` to get a handle valid in *its* `drm_file`.

45.3 The lifecycle

1. `DRM_IOCTL_AMDGPU_GEM_CREATE` (or generic `DUMB_CREATE`):
   - Allocate a BO of the requested size and placement.
   - Allocate a handle in the file's idr.
   - Return the handle.

2. `DRM_IOCTL_GEM_CLOSE`:
   - Drop the handle. If last handle, drop the kref. Last kref triggers `gem_free_object`.
3. **Mmap path**:
   - `DRM_IOCTL_AMDGPU_GEM_MMAP` returns a fake offset.
   - User does `mmap(fd, …, offset)`; kernel resolves offset → BO and installs `vm_ops->fault`.
4. **VA bind path**:
   - `DRM_IOCTL_AMDGPU_GEM_VA` maps the BO into the process's GPUVM at a given GPU VA.
5. **Submit path**:
   - User builds an IB referencing GPU VAs.
   - `DRM_IOCTL_AMDGPU_CS` validates and submits.

45.4 amdgpu BO placement

Each BO has a **placement**: VRAM, GTT, CPU, or hybrid. Common flags:

```
AMDGPU_GEM_DOMAIN_CPU        /* system memory, CPU access primary */
AMDGPU_GEM_DOMAIN_GTT        /* system memory mapped via IOMMU */
AMDGPU_GEM_DOMAIN_VRAM       /* GPU local */
AMDGPU_GEM_DOMAIN_DOORBELL   /* small doorbell pages */
AMDGPU_GEM_DOMAIN_OA         /* off-chip APU memory */
```

A BO can have *preferred* and *allowed* domains. TTM (next chapter) chooses where to actually place it, may evict to make room, may keep it pinned.

45.5 Reserving and fences

A BO can be in flight (GPU is reading/writing) when userspace tries to use it. Synchronization uses `dma_resv`:

```
struct dma_resv {
    struct dma_fence       *fence_excl;   /* exclusive (writer) */
    struct dma_resv_list   *fence;        /* shared (readers)   */
    struct ww_mutex         lock;
};
```

When a job starts, it adds its fence to the BOs it touches (write → exclusive, read → shared). When another job submits, it waits for the existing fences before issuing its work.

This is the foundation for cross-process synchronization without explicit semaphores.

45.6 The mmap path in depth

User:

```
struct drm_amdgpu_gem_mmap mmap_req = { .handle = handle };
ioctl(fd, DRM_IOCTL_AMDGPU_GEM_MMAP, &mmap_req);
void *cpu = mmap(NULL, size, PROT_READ|PROT_WRITE, MAP_SHARED, fd, mmap_req.addr_ptr);
```

Kernel:

1. `amdgpu_gem_mmap_ioctl` returns a fake offset that encodes the GEM object.
2. `drm_gem_mmap` (in `file_operations.mmap`) looks up the offset in the device's `vma_manager`, finds the GEM object, calls `drv->vm_ops->fault` on access.
3. `amdgpu_gem_fault` resolves where the page lives (VRAM / GTT / migrated) and installs the PTE.

For a write-combined CPU mapping into VRAM, the PTE has WC attribute. For GTT, it can be cached. The driver must pick correctly.

45.7 The VA bind path (GPU-side mapping)

A BO must be mapped into the GPU's address space before the GPU can dereference it.

```
struct drm_amdgpu_gem_va va = {
    .handle    = handle,
    .operation = AMDGPU_VA_OP_MAP,
    .flags     = AMDGPU_VM_PAGE_READABLE | AMDGPU_VM_PAGE_WRITEABLE,
    .va_address= 0x100000000ULL,
    .offset_in_bo = 0,
    .map_size  = size,
};
ioctl(fd, DRM_IOCTL_AMDGPU_GEM_VA, &va);
```

Kernel updates the per-process GPUVM page tables to map `va_address` → BO's pages. We discuss in Chapter 51.

A BO can have multiple VA mappings (in different processes, different addresses). The driver maintains a list of `amdgpu_bo_va_mapping`.

45.8 dma-buf export/import

Export:

```
struct drm_prime_handle p = { .handle = handle, .flags = O_RDWR };
ioctl(fd, DRM_IOCTL_PRIME_HANDLE_TO_FD, &p);
/* p.fd is a dma-buf fd. */
```

Import (other process or other driver):

```
struct drm_prime_handle p = { .fd = received_fd };
ioctl(fd, DRM_IOCTL_PRIME_FD_TO_HANDLE, &p);
/* p.handle is a new GEM handle in this drm_file. */
```

The dma-buf is what's sent over Wayland / VAAPI / V4L2 / Vulkan external memory.

amdgpu's `gem_prime_import` and `gem_prime_export` drive this.

45.9 BO list

A submission may touch dozens of BOs. The kernel needs to validate (lock, get into fast memory, attach fences) every BO before running the work. Userspace prepares a **BO list** ahead of time:

```
struct drm_amdgpu_bo_list_in {
    uint32_t operation;
    uint32_t list_handle;
    uint32_t bo_number;
    uint32_t bo_info_size;
    uint64_t bo_info_ptr;        /* user pointer to entries */
};
```

Each entry is `(handle, priority)`. The kernel hashes them, sorts by priority for eviction decisions.

Modern amdgpu has the **inline BO list** (passed in the CS ioctl directly) as well — avoids the round-trip to create a BO list handle.

45.10 Userptr BOs

A "userptr" BO is a BO that wraps existing user pages. The kernel uses `get_user_pages` to pin them and maps them into GPUVM.

Used by: HIP `hipHostMalloc`, OpenGL EXT_buffer_storage, scientific codes.

The kernel must register an MMU notifier (`mmu_interval_notifier`): if the user munmaps or mremaps, the kernel must unmap from GPUVM and possibly stall in-flight work.

`drivers/gpu/drm/amd/amdgpu/amdgpu_mn.c` implements this. Beautiful, treacherous code.

45.11 Reserved and pinned BOs

Some BOs cannot move:

- The page tables themselves.
- Doorbells.
- The CSA (Compute Shader Associations) buffer.
- BOs being scanned out (display) — must stay in VRAM.

Kernel pins via `amdgpu_bo_pin()`. Counts; unpinning lets TTM reclaim.

45.12 GEM helpers and "shmem" path

Lighter drivers (vkms, simpledrm, panfrost) use `drm_gem_shmem_helper.c` for buffer management — system-memory only, no migration. Faster to implement.

amdgpu's needs (VRAM/GTT/eviction) require TTM, which is heavier but more capable.

45.13 Exercises

1. Write a userspace program that opens `/dev/dri/renderD128`, calls `DRM_IOCTL_AMDGPU_GEM_CREATE` to allocate 64 KB GTT BO, mmaps it, writes a pattern, frees.
2. Extend it to map the BO into GPU VA at 0x100000000 via `DRM_IOCTL_AMDGPU_GEM_VA`.
3. Export to dma-buf fd; in another process, import as GEM handle; mmap; verify the same memory.
4. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_gem.c::amdgpu_gem_create_ioctl`. Trace how a request flows to TTM.

5. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_mn.c::amdgpu_mn_invalidate_range_start`. This is one of the hardest files in amdgpu; just orient yourself.

Next: Chapter 46 — TTM, the memory manager amdgpu uses.

Chapter 46 — TTM — the memory manager amdgpu uses

TTM (Translation Table Manager) is DRM's bigger-brother memory manager, designed for GPUs with VRAM, paging between system memory and VRAM, and eviction under pressure. amdgpu, radeon, nouveau (recent), vmwgfx, qxl all use TTM.

46.1 The problem TTM solves

A GPU has finite VRAM. Workloads allocate more BO memory than VRAM. We must:

- Place BOs where they're cheapest to access ("preferred placement").
- **Evict** lower-priority BOs from VRAM to GTT when needed.
- Migrate transparently with respect to in-flight GPU work.
- Track per-BO **fences** so we don't evict something currently being used.

TTM does all of this generically.

46.2 The pieces

```
ttm_resource_manager  per-pool placement (VRAM, GTT, …)
      |
ttm_buffer_object     one BO; embeds drm_gem_object
      |
ttm_resource          where this BO currently lives
      |
ttm_tt                CPU-side page array (system RAM)
```

- `ttm_buffer_object` **(TBO)**: the BO type. Driver-specific BO embeds it.
- `ttm_resource_manager`: per-pool allocator (one for VRAM, one for GTT, etc.).
- `ttm_resource`: the *current* placement; pointer to the manager and within-pool offset.
- `ttm_tt`: backing pages when in system memory.

amdgpu has:

- `mgr_vram` for VRAM (a buddy allocator).
- `mgr_gtt` for GTT (linear allocator with eviction).
- `mgr_doorbell`, `mgr_oa`, etc. for special pools.

46.3 Placement

A `ttm_placement` struct says "this BO is allowed in these pools, prefer that one":

```
struct ttm_place {
    uint32_t  fpfn, lpfn;   /* page-frame range */
    uint32_t  mem_type;     /* TTM_PL_VRAM | TTM_PL_TT | ... */
    uint32_t  flags;
};

struct ttm_placement {
```

```
        unsigned          num_placement;
        const struct ttm_place *placement;
        unsigned          num_busy_placement;
        const struct ttm_place *busy_placement;
    };
```

`busy_placement` is fallback when the primary is full and can't evict.

46.4 Migration and fences

When TTM moves a BO from VRAM → GTT, it issues a GPU copy job (using SDMA — fast, asynchronous). The copy returns a fence. TTM hangs the fence on the BO. Subsequent GPU work that touches the BO will wait on the fence.

If the BO is being read while moving: TTM serializes via `dma_resv` (Chapter 45).

Driver supplies a `move()` callback that knows how to copy. `amdgpu_bo_move()` issues an SDMA copy and returns a fence.

46.5 Eviction

When `mgr_vram` runs out:

- It picks a victim (LRU, lowest priority, not pinned).
- Calls `ttm_bo_validate(victim, &placement_to_GTT, ctx)` — moves it.
- Now there's space; the requested allocation succeeds.

Eviction is the single subtlest path in any GPU driver. Bugs here look like graphical glitches, hangs under memory pressure, or "the GPU reset itself."

46.6 The "reservation" lock

Every BO has a `dma_resv` whose lock is a `ww_mutex` — a "wait-wound" mutex that supports deadlock-free multi-lock acquisition (you can lock 100 BOs at once and TTM ensures progress).

When you bind multiple BOs for a CS, you take the wound-mutex on all of them via `ttm_eu_reserve_buffers`. If a deadlock would occur, you "wound" yourself, drop and retry — guaranteed to eventually succeed.

This is one of the cleverest pieces of synchronization in the kernel.

46.7 The amdgpu_bo struct (real)

```
    struct amdgpu_bo {
        struct ttm_buffer_object        tbo;
        struct ttm_bo_kmap_obj          kmap;
        u64                             flags;
        unsigned                        prime_shared_count;
        struct list_head                shadow_list;
        struct amdgpu_bo               *parent;
        struct amdgpu_bo               *shadow;
        struct amdgpu_mn               *mn;
        union {
            struct dma_buf_attachment  *attach;
            struct {
                struct drm_mm_node     *bin;       /* binning */
```

```
                uint64_t                preferred_domains;
                uint64_t                allowed_domains;
            };
        };
        /* … */
    };
```

Many fields. The key ones:

- `tbo`: TTM half.
- `flags`: BO flags (no-eviction, contiguous, persistent).
- `parent`: for sub-allocated regions (sub-allocator atop a big BO).
- `mn`: MMU notifier for userptr BOs.

46.8 The buddy VRAM allocator

amdgpu's VRAM manager is a buddy. Same structure as the kernel's page allocator. Free lists by power-of-two block sizes; split on alloc, merge on free.

Live in `drivers/gpu/drm/amd/amdgpu/amdgpu_vram_mgr.c`. Worth reading after this textbook.

46.9 Sub-allocator

Many small descriptor-sized allocations would fragment VRAM. amdgpu's `amdgpu_sa_bo` carves a single big BO into smaller chunks.

```
    struct amdgpu_sa_manager {
        struct amdgpu_bo  *bo;
        struct list_head   olist;     /* in-use chunks */
        struct list_head   flist[…];  /* free chunks per ring */
        /* … */
    };
```

Used for IBs and small kernel-internal structures.

46.10 Performance pitfalls

- **Migration thrash**: a BO bouncing between VRAM and GTT under pressure. Symptom: low FPS that improves when VRAM has slack. Triage: instrument migrations with tracepoints.
- **Eviction deadlock**: two CSes each need each other's BO evicted. ww_mutex prevents true deadlock but you may stall. Watch fence wait latencies.
- **Pinned-too-long**: scanout BOs pinned, fragmentation grows. Mitigate with DC's "underflow recovery" path.

46.11 The kernel call you'll see most

```
    amdgpu_bo_create(adev, &bp, &bo);
    amdgpu_bo_reserve(bo, false);
    amdgpu_bo_pin(bo, AMDGPU_GEM_DOMAIN_VRAM);
    amdgpu_bo_kmap(bo, &cpu);
    /* … use … */
    amdgpu_bo_kunmap(bo);
    amdgpu_bo_unpin(bo);
```

```
        amdgpu_bo_unreserve(bo);
        amdgpu_bo_unref(&bo);
```

That's the rhythm of every kernel-internal BO use.

46.12 Exercises

1. Read `drivers/gpu/drm/ttm/ttm_bo.c` — focus on `ttm_bo_validate` and `ttm_bo_move`.
2. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_vram_mgr.c`. Identify the buddy structure.
3. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_sa.c`. Identify the suballocator.
4. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_object.c::amdgpu_bo_create`. Trace through what flags drive what placements.
5. Run a workload that exhausts VRAM (`vkmark` heavy textures, or `dmesg`-watch under games). Observe migration and eviction in `/sys/kernel/debug/dri/0/`.

Next: Chapter 47 — KMS — display, planes, CRTCs, atomic modesetting.

Chapter 47 — KMS — display, planes, CRTCs, atomic modesetting

KMS (Kernel Mode Setting) lives in DRM and is how Linux puts pixels on the screen. amdgpu's display side is its own giant subsystem (DC), but it sits behind a standard KMS API that we walk here. If you join the AMD display team you'll be inside DC; otherwise this chapter is enough to debug display issues from the rest of the driver.

47.1 The objects

```
   Connector  →  Encoder  →  CRTC  →  [Plane(s)] → Framebuffer (BO)
   "the port"   "signals"    "timer"   "image"     "memory"
```

- **Connector**: a physical port (HDMI, DP, eDP, VGA). Has detection, EDID, supported modes.
- **Encoder**: the signal generator (TMDS for HDMI, DisplayPort transport, etc.). 1:1 with a connector usually.
- **CRTC** (Cathode Ray Tube Controller): the pipe that times pixels. Has a *mode* (resolution, timings) and a list of planes.
- **Plane**: an image source. Primary plane is the main framebuffer. Cursor plane is the cursor. Overlay planes for video.
- **Framebuffer**: a `drm_framebuffer` referencing a GEM BO with format/pitch.

A modern desktop has typically 1 CRTC active per display, with 1 primary plane + maybe 1 cursor plane.

47.2 The display path

```
GPU rendered into BO
        |
   plane.fb = BO
        |
   CRTC scans plane(s) according to mode
        |
   Encoder produces TMDS/DP
        |
   Connector outputs to physical port
        |
       Monitor
```

47.3 Atomic modesetting

The legacy KMS API had separate calls for each thing (`SET_CRTC`, `PAGE_FLIP`, `SET_PROPERTY`). Atomic is the modern way:

1. Build a `drm_atomic_state` containing the desired state of every modified object (CRTC, plane, connector).
2. Validate via `atomic_check`.

3. Commit via `atomic_commit`.

If anything fails validation, nothing changes. Multi-display config changes are atomic — you don't see a half-state.

ioctl: `DRM_IOCTL_MODE_ATOMIC`. `libdrm` exposes a friendly wrapper.

47.4 The driver's job

For each of CRTC/plane/connector/encoder, the driver implements callbacks via `drm_*_funcs` and helpers. Roughly:

- `*_atomic_check` — validate proposed state against hardware constraints.
- `*_atomic_update` (planes) — program the hardware for new image source.
- `*_atomic_enable` / `disable` (CRTC) — turn a pipe on/off.
- `*_helper_funcs` — DRM core's helpers fill in standard machinery.

amdgpu DC's task is to *map* the abstract state onto AMD's display hardware (DPP, DPG, MPC, OPP, OTG, etc.).

47.5 EDID and modes

When a monitor is plugged in, the connector reads the **EDID** (Extended Display Identification Data) over DDC (an I2C side channel on the display port). EDID lists supported resolutions, timings, color spaces, HDR metadata.

Driver: `connector_funcs->detect`, `connector_helper->get_modes`. `drm_get_edid()` does the I2C. `drm_add_edid_modes` parses.

47.6 vblank

Each CRTC emits a "vertical blank" event roughly every frame. Drivers expose:

- vblank-counter to userspace,
- vblank-event delivery via the DRM file (read `drm_event` packets).

Used for v-sync, frame pacing, and timing measurement. KMS clients call `DRM_IOCTL_WAIT_VBLANK`.

47.7 Page flip

The bread-and-butter operation: replace the visible framebuffer with a new one, in sync with vblank.

Atomic flip:

```
        plane.fb = new_bo;
        ATOMIC_COMMIT(state, DRM_MODE_PAGE_FLIP_EVENT);
```

Driver programs the scanout-base register such that on the next vblank, the new BO becomes active. Sends a flip-complete event to userspace (over the DRM fd) so the compositor can begin building the next frame.

47.8 Cursor and overlay planes

- **Cursor**: small, software-positioned. AMD HW has a dedicated cursor.
- **Overlay**: secondary image surfaces, often used for video (a YUV plane behind/above primary RGB). Saves CPU/GPU time vs. compositing.

Modern compositors (Wayland) increasingly use planes to offload composition; the kernel's atomic API exposes "is this layer combination feasible?" via atomic_check.

47.9 HDR, color, gamma

Each plane and CRTC has properties for color management: gamma LUT, CSC matrix, degamma, color space, HDR metadata. Atomic state carries these.

amdgpu DC implements the AMD-specific color pipeline. The kernel exposes a generic API; userspace (e.g. `colord`, GameScope) drives.

47.10 The DC architecture (briefly)

`drivers/gpu/drm/amd/display/` contains:

- `amdgpu_dm/` — the DRM-facing layer (KMS bridge).
- `dc/` — the platform-independent DC core (also runs on Windows). Built around abstract pipe blocks.
- `dc/dcn*/` — per-generation DCN (Display Core Next) hardware code.
- `modules/` — color, freesync, hdr, mst, info_packet, etc.

DC is enormous. If you join DC, expect to spend months ramping. If you don't, treat DC as a black box callable via the standard KMS hooks.

47.11 Headless / KFD

GPUs without displays (server compute) or with disabled KMS (`amdgpu.modeset=0`) still load amdgpu but only expose `renderD*` and `/dev/kfd`. KMS code is bypassed. Most ROCm workloads run headless.

47.12 Exercises

1. `cat /sys/class/drm/card0-*/status`. Identify connected vs. disconnected.
2. Use `drmModeGetResources` (libdrm) to enumerate connectors/encoders/CRTCs/planes from userspace.
3. Read `Documentation/gpu/drm-kms.rst`.
4. Read `drivers/gpu/drm/amd/display/amdgpu_dm/amdgpu_dm.c::amdgpu_dm_atomic_check`. Don't try to absorb everything — orient.
5. Use `modetest` (from libdrm) to enumerate modes and force a particular mode (be careful, can blank your screen).

Next: Chapter 48 — The amdgpu driver — top-level architecture.

Chapter 48 — The amdgpu driver — top-level architecture

We've assembled all the prerequisites. Now we walk amdgpu from top to bottom.

48.1 Source map

```
drivers/gpu/drm/amd/amdgpu/
├── amdgpu.h                      omnibus header
├── amdgpu_drv.c                  module init, pci_driver, fops
├── amdgpu_device.c               probe, init/fini orchestration
├── amdgpu_kms.c                  ioctl entry points (open/info/etc.)
├── amdgpu_ttm.c                  TTM glue
├── amdgpu_object.c               BO create/destroy/move
├── amdgpu_gem.c                  GEM ioctls
├── amdgpu_cs.c                   command-submit ioctl  ← star of the show
├── amdgpu_ring.c                 rings, IB submission
├── amdgpu_fence.c                HW seqno → dma_fence
├── amdgpu_sync.c                 fence dependency tracking
├── amdgpu_vm.c                   per-process GPU page tables
├── amdgpu_irq.c                  IH ring drain, IRQ subsystem registration
├── amdgpu_ctx.c                  user-visible scheduling context
├── amdgpu_sched.c                DRM-sched glue
├── gfx_v*.c                      per-generation GFX IP
├── sdma_v*.c                     per-generation SDMA IP
├── vcn_v*.c                      video codec IP
├── jpeg_v*.c                     JPEG IP
├── mes_v*.c                      MES (microcontroller scheduler) IP
├── psp_v*.c                      PSP (firmware loader) IP
├── soc15.c, nbio_v*.c, gmc_v*.c …  per-IP HW shims
├── amdgpu_pm/                    power management
├── amdgpu_dm/, dc/               display core (separate Makefile)
└── amdgpu_kfd_*.c                KFD bridge (compute path)
```

`drivers/gpu/drm/amd/amdkfd/` — the KFD/HSA path (separate fd `/dev/kfd`).

`drivers/gpu/drm/amd/include/` — register definitions per ASIC.

48.2 The probe path

`amdgpu_pci_probe(pdev, id)` (in `amdgpu_drv.c`) — entry. High-level flow:

```
        adev = devm_drm_dev_alloc(&pdev->dev, &kms_driver, struct amdgpu_device, ddev);
        adev->pdev = pdev;
        amdgpu_driver_load_kms(adev, ent->driver_data);
        drm_dev_register(&adev->ddev, ent->driver_data);
```

`amdgpu_driver_load_kms` calls `amdgpu_device_init` which does the heavy lifting:

1. Detect ASIC family / variant.
2. Map BARs (`pci_iomap`).
3. Init `amdgpu_irq`.
4. `amdgpu_atombios_init` (legacy, for older ASICs) — ATOM BIOS parsing.
5. Init `amdgpu_smu` — system management.

6. `psp_init` — load all per-IP firmware.
7. `amdgpu_device_ip_init`: walks every "IP block" and calls its `early_init`, then `sw_init` (alloc rings, fences), then `hw_init` (program HW).
8. `amdgpu_ttm_init` — set up VRAM and GTT memory managers.
9. `amdgpu_vm_init` — set up VM machinery.
10. `amdgpu_amdkfd_device_init` — KFD setup.
11. Register DRM device.

48.3 IP blocks

Each "IP" (Intellectual Property block) — GFX, SDMA, VCN, MES, PSP, GMC, SMU, DC — implements a vtable:

```
struct amdgpu_ip_block_version {
    enum amd_ip_block_type type;
    u32 major, minor, rev;
    const struct amd_ip_funcs *funcs;
};

struct amd_ip_funcs {
    char *name;
    int (*early_init)(void *handle);
    int (*late_init)(void *handle);
    int (*sw_init)(void *handle);
    int (*sw_fini)(void *handle);
    int (*hw_init)(void *handle);
    int (*hw_fini)(void *handle);
    int (*suspend)(void *handle);
    int (*resume)(void *handle);
    bool (*is_idle)(void *handle);
    int (*wait_for_idle)(void *handle);
    int (*soft_reset)(void *handle);
    /* … */
};
```

`amdgpu_device.c::amdgpu_device_ip_init` walks the table and calls each in order. Suspend/resume walks in the right order. This is essentially OOP-in-C with each IP as a plugin.

To bring up a new ASIC, you (typically) port the IP block files to the new register addresses and add an entry. The driver's bring-up structure absorbs new generations cleanly.

48.4 The amdgpu_device struct

`struct amdgpu_device` is the **god-object** that holds everything. ~1000+ fields. Notable groups:

- `pdev`, `ddev` — PCI and DRM cores.
- `mmio[]`, `rio_mem`, `doorbell` — register banks.
- `firmware`, `psp`, `smu` — firmware/control state.
- `vm_manager` — GPUVM.
- `gmc` — GPU memory controller; describes VRAM/GTT regions.
- `gfx`, `sdma[]`, `mes`, `vcn[]`, `jpeg[]` — per-IP state.
- `irq` — IRQ subsystem.
- `mode_info` — KMS mode info.
- `dm`, `dc` — display core glue.

- `pm` — power management.
- `ras` — RAS (reliability, availability, serviceability).
- `ip_blocks[]` — the IP table.

A function rarely needs more than a few fields. But the struct is your map.

48.5 The IH (Interrupt Handler) pipeline

Hardware interrupts to amdgpu come through an **IH ring** — circular buffer in memory the CPU pre-allocates. When an event happens, hardware writes a 16-byte cookie into the ring and updates the write pointer.

```
amdgpu_irq_handler(pci_irq) {
    /* read IH ring tail */
    while (head != tail) {
        process_one_cookie(ring[head]);
        head++;
    }
    schedule(tasklet);   /* deferred */
}
```

`process_one_cookie` dispatches to the registered IRQ source — vblank, ring-completion, page-fault, RAS, etc. Each `amdgpu_irq_src` has an `id_table` mapping (client_id, src_id) and a `process` callback.

This is the closest thing in the kernel to a publish/subscribe bus.

48.6 The CS path (preview)

`DRM_IOCTL_AMDGPU_CS` is the ioctl that submits work. We dedicate Chapter 49. High-level:

1. Userspace passes “chunks”: IBs, deps (input fences), syncobjs.
2. Driver looks up BOs, takes ww_mutex on all (`ttm_eu_reserve_buffers`).
3. Validates IBs (security: no privileged register access, no out-of-bounds memory writes).
4. Builds a `drm_sched_job`.
5. Hands to DRM scheduler (Chapter 50).
6. Returns a fence handle to userspace.

48.7 The amdgpu_ring abstraction

Each engine has a ring (or several): GFX ring, compute rings, SDMA rings, VCN rings. The `struct amdgpu_ring`:

```
struct amdgpu_ring {
    struct amdgpu_device  *adev;
    const struct amdgpu_ring_funcs *funcs;
    struct drm_gpu_scheduler  sched;
    struct amdgpu_bo  *ring_obj;
    volatile uint32_t  *ring;        /* CPU mapping of ring */
    unsigned   wptr, rptr;
    uint64_t   gpu_addr;
    /* … */
};
```

`funcs` is the per-ring vtable: how to write a "begin IB" packet, "fence" packet, etc. It abstracts PM4 vs. SDMA vs. VCN packet formats.

We discuss in Chapter 49.

48.8 The fence path

amdgpu's HW writes a sequence number to a memory address when each job finishes. The driver:

- Allocates a `dma_fence` for each submission.
- Polls (and registers an interrupt source) for sequence number progression.
- Signals the fence when the seqno passes.

Code: `amdgpu_fence_emit`, `amdgpu_fence_process`. We discuss in Chapter 50.

48.9 GPUVM

Each open file (or KFD process) has its own GPU virtual address space. Page tables live in VRAM (or system memory). Levels: PD0 → PD1 → PD2 → PT.

When userspace `MAP`s a BO, the driver writes PTEs. When the BO migrates (TTM moves), PTEs are updated. When a BO is freed, PTEs are zeroed.

Updates can go via SDMA copy (asynchronously, very fast for batches) or CPU writes. amdgpu uses both depending on circumstance.

48.10 RAS, reset, recovery

When the GPU hangs (job timeout) or a hardware error fires, amdgpu issues a **reset**. Two flavors:

- **Soft reset** — reset specific IPs, reload firmware, recover.
- **Mode-1 / Mode-2 / FLR** — bigger hammers, up to PCI Function-Level Reset.

After reset, all in-flight jobs are killed (their fences signaled with error), VM PTEs may need rebuilding, displays must be re-enabled. amdgpu does all of this without losing the desktop.

`amdgpu_device.c::amdgpu_device_gpu_recover` is where it happens. Massive function. Worth reading once you've absorbed the rest.

48.11 Reading the source: a recommended path

1. `amdgpu_drv.c` — module init, ioctls table.
2. `amdgpu_device.c` — probe.
3. `amdgpu_kms.c` — open/close.
4. `amdgpu_gem.c` — BO create/free/mmap.
5. `amdgpu_ttm.c` — TTM glue.
6. `amdgpu_cs.c` — command submission.
7. `amdgpu_ring.c`, `amdgpu_ib.c` — IB push.
8. `amdgpu_fence.c` — fences.
9. `amdgpu_vm.c` — VM (long).

10. `amdgpu_irq.c` — IH ring.

Then dive into one IP (say, `gfx_v10_0.c`) to see how a generation handles GFX init.

48.12 Exercises

1. Boot with `amdgpu.debug_mask=0xfff` and capture the dmesg from `amdgpu` load. Identify the IP-block sequence.
2. Read `amdgpu_drv.c::amdgpu_pci_probe`. Take notes on the 50 lines.
3. Read `amdgpu_device.c::amdgpu_device_ip_init`. Spot the call to each IP's `sw_init` then `hw_init`.
4. Read `amdgpu_ring.c::amdgpu_ring_init`. Note the BO allocation for the ring buffer + fences.
5. Run `cat /sys/kernel/debug/dri/0/amdgpu_firmware_info`. See every firmware version loaded.

Next: Chapter 49 — Command submission, IBs, rings, doorbells.

Chapter 49 — Command submission, IBs, rings, doorbells

This is the path on which every game, every CUDA-equivalent compute kernel, every Vulkan submit travels. Master it and you understand how AMD GPUs are programmed from Linux.

49.1 The big picture

```
userspace builds an IB (PM4/SDMA packets) in a BO
   │
   │ DRM_IOCTL_AMDGPU_CS (chunks: IBs, deps, syncobjs, BO list)
   ▼
amdgpu_cs_ioctl
   ├── parse chunks
   ├── reserve BOs (ww_mutex via ttm_eu)
   ├── validate IBs (whitelist)
   ├── build drm_sched_job
   ├── hand to drm-sched (Chapter 50)
   │
   ▼
drm_sched picks job ready
   ├── waits on input fences
   ├── runs `run_job` → amdgpu_job_run
   ▼
amdgpu_job_run
   ├── emits prologue (cache flushes, VM switch, etc.)
   ├── pushes IB packet onto HW ring
   ├── emits fence packet
   ├── kicks doorbell
   │
   ▼
GPU CP fetches packets, executes IBs
   ├── eventually reaches fence packet
   ├── writes seqno to memory + raises IRQ
   ▼
amdgpu_fence_process → signals dma_fence → wakes waiters
```

We go through each piece.

49.2 An IB (Indirect Buffer)

An IB is a buffer of GPU packets the userspace driver builds. The kernel-side `amdgpu_ib`:

```
struct amdgpu_ib {
    struct amdgpu_sa_bo  *sa_bo;
    uint32_t              length_dw;
    uint64_t              gpu_addr;
    uint32_t             *ptr;
    uint32_t              flags;
};
```

The IB lives in CPU-mapped memory (sub-allocated). The CPU writes `ptr[i] = packet`; on submit, the driver tells the ring "go execute the IB at `gpu_addr` for `length_dw` dwords."

PM4 example: write 0xCAFEBABE to memory at GPU VA 0x1000:

```
        ptr[0] = PACKET3(PACKET3_WRITE_DATA, 3);
        ptr[1] = WRITE_DATA_DST_SEL(WRITE_DATA_DST_SEL_MEM)|
WRITE_DATA_ENGINE_SEL(WRITE_DATA_ENGINE_SEL_ME);
        ptr[2] = lower_32_bits(0x1000);
        ptr[3] = upper_32_bits(0x1000);
        ptr[4] = 0xCAFEBABE;
        ib->length_dw = 5;
```

In practice userspace builds these. Mesa has `radeonsi_emit_*` and ACO compiles shaders into IB-embedded packets.

49.3 The ring

A ring is a circular buffer in memory (CPU- and GPU-readable). The CPU writes packets at `wptr`; the GPU's CP reads at `rptr`. Hardware registers hold the pointers.

```
        struct amdgpu_ring {
            volatile uint32_t  *ring;     /* CPU map */
            uint64_t            gpu_addr; /* GPU VA */
            unsigned            ring_size;
            unsigned            wptr, rptr;
            /* … */
        };
```

Pushing a packet:

```
        amdgpu_ring_write(ring, packet);
        /* increments wptr */
```

After all packets:

```
        amdgpu_ring_commit(ring);   /* writes wptr to hardware MMIO */
```

The kernel uses `amdgpu_ring_emit_*` helpers per ring type. PM4 packets go to GFX/compute rings; SDMA packets to SDMA rings; VCN packets to VCN rings. Each ring's `funcs` knows how.

49.4 The CS ioctl in detail

User struct (simplified):

```
        struct drm_amdgpu_cs_in {
            uint32_t  ctx_id;
            uint32_t  bo_list_handle;
            uint32_t  num_chunks;
            uint32_t  flags;
            uint64_t  chunks;          /* user array of chunk pointers */
        };

        struct drm_amdgpu_cs_chunk {
            uint32_t  chunk_id;
            uint32_t  length_dw;
            uint64_t  chunk_data;       /* user pointer */
        };
```

Chunks include:

- `IB`: IB descriptor (which engine, which BO/offset, length).
- `DEPENDENCIES`: input fences from same context.
- `SYNCOBJ_IN` / `SYNCOBJ_OUT`: external syncobj deps and outputs.
- `FENCE`: where to put the user-fence value on completion.
- `BO_HANDLES`: inline BO list.

Kernel `amdgpu_cs_ioctl`:

```
int amdgpu_cs_ioctl(struct drm_device *dev, void *data, struct drm_file *filp)
{
    struct amdgpu_cs_parser parser = { 0 };
    int r;

    r = amdgpu_cs_parser_init(&parser, ...);     /* alloc context */
    r = amdgpu_cs_parser_bos(&parser);           /* validate + reserve BOs */
    r = amdgpu_cs_parser_pass1(&parser);         /* parse IB chunks */
    r = amdgpu_cs_parser_pass2(&parser);         /* parse dep/syncobj chunks */
    r = amdgpu_cs_dependencies(&parser);         /* attach input fences */
    r = amdgpu_cs_submit(&parser, ...);          /* push to scheduler */
    amdgpu_cs_parser_fini(&parser);
    return r;
}
```

(This is conceptual; modern code is split across `amdgpu_cs.c` and tweakable.)

49.5 BO reservation with ww_mutex

```
ttm_eu_reserve_buffers(&parser->ticket, &parser->validated, true, &parser->fences_to_wait);
```

This locks every BO in the parser's list using a "wait-wound" protocol. It guarantees forward progress even when many CSes contend.

After reservation, the driver:

- Validates each BO (moves it to a domain the engine can access; e.g., GTT for SDMA).
- Reads in-flight fences.
- Adds the new submission's fence to each BO via `dma_resv_add_fence`.

Upon submission completion, the BOs' resv lists are updated and other CSes that were waiting can proceed.

49.6 Validation

The kernel must ensure userspace cannot:

- Read/write privileged registers.
- DMA outside its own BOs.
- Access another process's memory.

amdgpu validates IBs by:

- A **whitelist** for register addresses in `WRITE_DATA` / `SET_*` packets.
- BO-list checks: every memory address used must be backed by a BO the userspace owns.
- VMID isolation: each process has its own VMID; the GPU's MMU enforces.

This validation is the most security-critical code in amdgpu. CVEs here are kernel-level.

49.7 Building the drm_sched_job

```
struct drm_sched_job {
    struct list_head        list;
    struct dma_fence        s_fence;
    struct dma_fence_cb     finish_cb;
    /* … */
};
```

The driver wraps with `struct amdgpu_job` (extends `drm_sched_job`).

```
amdgpu_job_alloc(adev, num_ibs, &job, ...);
job->ibs[0] = ib;
amdgpu_job_submit(job, entity, ...);
```

`drm_sched_job_init`, `drm_sched_job_arm`, `drm_sched_entity_push_job`. We treat in Chapter 50.

49.8 Doorbells

Once the scheduler decides "this job is next," it calls the driver's `run_job(struct drm_sched_job *)` (= `amdgpu_job_run`). The driver:

1. Emits prologue PM4 (VM switch, cache flushes).
2. Pushes the IB packet.
3. Emits a fence packet.
4. Commits the ring (updates `wptr` register or doorbell).

The doorbell write is the magic moment the GPU is told "look, more work." On modern GPUs, doorbells live in MMIO pages mapped per-process; userspace can write directly when using KFD-mode queues.

49.9 Multiple engines

amdgpu has multiple rings per engine:

- 1 GFX ring (graphics + compute).
- ~8 compute rings (older parts).
- ~2-4 SDMA rings (system DMA).
- ~2-4 VCN rings (video decode).

The DRM scheduler runs one job per ring at a time. Multiple engines run in parallel.

User can pick which engine via `chunk_ib.ip_type`:

```
AMDGPU_HW_IP_GFX, AMDGPU_HW_IP_COMPUTE, AMDGPU_HW_IP_DMA,
AMDGPU_HW_IP_UVD, AMDGPU_HW_IP_VCE, AMDGPU_HW_IP_VCN_DEC,
AMDGPU_HW_IP_VCN_ENC, AMDGPU_HW_IP_VCN_JPEG,
```

A modern Vulkan game uses GFX + COMPUTE + DMA + VCN simultaneously.

49.10 MES — the firmware scheduler

Newer chips have a MES microcontroller running firmware that does scheduling on-chip. The kernel sends "submit queue X" commands to MES via a mailbox; MES handles preemption, priority, and queue management. KFD's user-mode queues route through MES.

For graphics, MES is being introduced gradually; many chips still use the kernel-side ring driving model.

49.11 Performance notes

- Submission latency: 5-50 µs per `amdgpu_cs_ioctl` typical. Userspace queues (KFD) get this down to <1 µs.
- BO list size: large lists cost validation time; userspace can pre-reserve "BO-list handles" and reuse.
- Kernel-side compilation of IBs is zero — they're pre-built in userspace.

49.12 Exercises

1. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_cs.c::amdgpu_cs_ioctl`. Walk through all phases.
2. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_ring.c::amdgpu_ring_alloc` and `_commit`.
3. Read `gfx_v10_0.c::gfx_v10_0_ring_emit_ib` to see how an IB packet is composed for GFX10.
4. Use `bpftrace` to hook `amdgpu_cs_ioctl` and count submissions per second under a benchmark:

   ```
   bpftrace -e 'kprobe:amdgpu_cs_ioctl { @ = count(); } interval:s:1 { print(@); clear(@); }'
   ```

5. Read `Documentation/gpu/amdgpu/cs.rst` (if present) or follow a CS through `amdgpu_trace.h` tracepoints.

Next: Chapter 50 — The GPU scheduler and dma-fence.

Chapter 50 — The GPU scheduler and dma-fence

The DRM scheduler (`drivers/gpu/drm/scheduler/`) and the kernel's `dma_fence` are the dual pillars of GPU concurrency in Linux. Every Vulkan timeline semaphore, every Wayland surface flip, every cross-process buffer hand-off lives on top of these. Read this chapter twice.

50.1 The DRM scheduler

A `struct drm_gpu_scheduler` is created per **hardware ring**. It has:

- a list of `drm_sched_entity` (one per app context per ring),
- a worker thread that pulls jobs and runs them,
- timeout machinery for hung jobs.

```
struct drm_gpu_scheduler {
    const struct drm_sched_backend_ops *ops;
    uint32_t                            hw_submission_limit;
    long                                timeout;
    const char                         *name;
    struct list_head                    sched_rq[DRM_SCHED_PRIORITY_COUNT];
    wait_queue_head_t                   job_scheduled;
    atomic_t                            hw_rq_count;
    struct task_struct                 *thread;
    /* … */
};
```

Backend ops the driver implements:

```
struct drm_sched_backend_ops {
    struct dma_fence *(*prepare_job)(struct drm_sched_job *, struct drm_sched_entity *);
    struct dma_fence *(*run_job)(struct drm_sched_job *sched_job);
    enum drm_gpu_sched_stat (*timedout_job)(struct drm_sched_job *sched_job);
    void (*free_job)(struct drm_sched_job *sched_job);
};
```

amdgpu's implementations: `amdgpu_job_run`, `amdgpu_job_timedout`, `amdgpu_job_free`. We saw `run_job` in the previous chapter.

50.2 Entities

A `drm_sched_entity` represents "one stream of work from one client." A single application context typically has one entity per ring it uses (one GFX entity, one compute entity, etc.).

```
struct drm_sched_entity {
    struct list_head    list;
    struct drm_sched_rq *rq;
    struct drm_gpu_scheduler **sched_list;
    /* … */
    struct spsc_queue   job_queue;
```

```
        struct dma_fence  *last_scheduled;
        /* … */
    };
```

`spsc_queue` is a *single-producer single-consumer* lock-free FIFO. The producer is your `amdgpu_cs_ioctl` thread; the consumer is the scheduler's worker. The same SPSC pattern from Chapter 23.

50.3 The lifecycle of a job

```
    amdgpu_job_alloc();                       /* allocate */
    drm_sched_job_init(&job->base, entity);  /* attach to entity */
    drm_sched_job_arm(&job->base);           /* generate fence */
    job->base.s_fence ← new dma_fence

    drm_sched_entity_push_job(&job->base);   /* into entity SPSC queue */
       ▼
    scheduler thread loops:
        job = drm_sched_entity_pop_job(entity);
        /* wait on job->dependencies */
        fence = ops->run_job(job);
        /* attach `finish_cb` to `fence` so when HW fence signals, sched fence does too */
       ▼
    HW fence signals on completion (IRQ → amdgpu_fence_process)
       ▼
    sched fence is signaled
       ▼
    ops->free_job(job)
```

The scheduler's *sched fence* (`s_fence`) has two parts: a **scheduled** fence (signaled when the job has been pushed to HW) and a **finished** fence (signaled when HW completes). Userspace usually waits on `finished` .

50.4 Dependencies

Each job has a list of input fences. `drm_sched` waits on all of them before calling `run_job` . Dependencies come from:

- Same context, prior IB on a ring (sequencing).
- Cross-engine dependencies (compute → GFX).
- Cross-process syncobj signals.
- BO resv lists (writer fence on a BO must complete before next reader).

amdgpu's `amdgpu_cs_dependencies` builds this fence list during CS parsing.

50.5 dma-fence

```
    struct dma_fence {
        spinlock_t           *lock;
        const struct dma_fence_ops *ops;
        struct rcu_head       rcu;
        struct list_head      cb_list;
        union {
            struct list_head  cb_list;
            ktime_t           timestamp;
        };
        u64                   context;
        u64                   seqno;
```

```
        unsigned long          flags;
        struct kref            refcount;
        int                    error;
    };
```

State machine:

- Created: not signaled.
- Enable signaling: driver arms hardware notification.
- Signaled: `flags |= SIGNALED`. Wakers run.

Operations:

- `dma_fence_get` / `put` — refcount.
- `dma_fence_signal()` — driver signals from IRQ or wherever.
- `dma_fence_add_callback(f, cb, fn)` — async callback.
- `dma_fence_wait[_timeout]()` — sync wait.
- `dma_fence_default_wait()` — kernel default.

`dma_fence_context_alloc()` reserves a unique 64-bit context ID. Within a context, sequence numbers must be monotonically increasing.

50.6 amdgpu fences

Each amdgpu ring has a per-ring dma-fence sequence:

```
    struct amdgpu_fence_driver {
        uint32_t                  num_fences_mask;
        spinlock_t                lock;
        struct dma_fence         **fences;     /* circular buffer of pending */
        uint64_t                  sync_seq;
        atomic_t                  last_seq;   /* latest signaled */
        bool                      initialized;
        /* … */
    };
```

When a CS submits, the driver picks the next seqno, embeds a `RELEASE_MEM` packet that writes that seqno to a known memory location with cache flush. When `amdgpu_fence_process()` runs (from IRQ or polling), it reads `last_seq` from the memory and signals all dma-fences up to that point.

50.7 Timeouts and reset

If a job doesn't finish within its timeout (`drm_gpu_scheduler.timeout`, default 10 s for amdgpu), `timedout_job()` is called. amdgpu:

1. Logs the hang (dmesg + debugfs).
2. Triggers a GPU reset (`amdgpu_device_gpu_recover`).
3. Marks the job and its fence with `-ECANCELED`.
4. After reset, scheduler resumes; subsequent jobs run.

This is a critical reliability property: a single bad shader doesn't take down the whole desktop.

50.8 Priority and preemption

Each entity has a priority. Higher priority preempts lower at the queue level (the scheduler picks high-prio first when both are ready). For *true* hardware preemption (mid-job interruption), the GPU's CP must support it; modern amdgpu uses MES for compute preemption.

User-visible priority via `DRM_IOCTL_AMDGPU_SCHED` — set context priority. `niceness` for graphics under PCs is rarely changed; for compute servers, it's important.

50.9 Syncobjs and timeline semaphores

`struct drm_syncobj` wraps a single dma-fence (binary syncobj) or a timeline of fences (timeline syncobj). It exposes a userspace handle and supports:

- Waiting for a fence to signal.
- Importing/exporting via fd.
- Timeline `wait for value ≥ N`.

amdgpu plugs syncobj into the CS path: `chunk SYNCOBJ_IN` adds dependencies; `chunk SYNCOBJ_OUT` materializes the output fence.

50.10 Locking model

The scheduler thread holds:

- `entity->lock` while popping the job.
- No global scheduler lock during `run_job`.
- `s_fence->lock` (spinlock) when signaling.

Driver's `run_job` runs in the scheduler kthread context. It can sleep, take mutexes, do DMA, but it's the hot path — keep it fast.

`free_job` runs from the scheduler thread or from the fence callback context.

50.11 amdgpu jobs in code

```
struct amdgpu_job {
    struct drm_sched_job  base;
    struct amdgpu_vm     *vm;
    struct amdgpu_ring   *ring;
    struct amdgpu_sync    sync;
    uint32_t              gds_base, gds_size, gws_base, gws_size, oa_base, oa_size;
    uint32_t              vmid;
    uint64_t              uf_addr;
    bool                  is_killed;
    /* … */
    struct amdgpu_ib      ibs[];
};
```

`run_job` (= `amdgpu_job_run`) does:

1. VM switch (program VMID).
2. Emit per-engine prologue.
3. For each IB, push `INDIRECT_BUFFER` packet.

4. Emit RELEASE_MEM (the fence).
5. Commit ring.
6. Return the fence.

50.12 Exercises

1. Read `drivers/gpu/drm/scheduler/sched_main.c::drm_sched_main`.
2. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_job.c`.
3. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_fence.c::amdgpu_fence_emit` and `_process`.
4. Read `include/linux/dma-fence.h`. Memorize the ops table.
5. Use bpftrace on `amdgpu_fence_emit` and `amdgpu_fence_signal`; count rates under a load. Should be roughly equal at steady state.

Next: Chapter 51 — VM, page tables, and unified memory.

Chapter 51 — VM, page tables, and unified memory

GPUVM is amdgpu's most intricate piece. Per-process page tables mapping user GPU virtual addresses to physical pages (VRAM, GTT, host pinned). This is the Linux MMU, but for GPUs.

51.1 Why per-process VM

Without per-process VM:

- Two processes' GPU buffers would share a single address space; collisions possible; no isolation.

With per-process VM (since Vega/Navi):

- Each process has its own GPU virtual memory.
- The hardware MMU enforces.
- Processes can use the same GPU VAs without conflict.
- A bug in one process can't read another's memory.

Implementation: each process gets a **VMID** (a small integer). The GPU's MMU has a table indexed by VMID giving the page-table root. Submissions tag the VMID; the engine uses that VMID's page tables.

VMIDs are scarce (16 GFX VMIDs typical). The driver multiplexes (VMID 0 reserved for kernel, others rotated among contexts via flush + re-bind).

51.2 The structures

```
struct amdgpu_vm {
    struct rb_root_cached   va;          /* allocated ranges */
    struct list_head        evicted;     /* BOs to re-bind */
    struct list_head        moved;
    struct list_head        relocated;
    struct list_head        idle;
    struct list_head        invalidated;
    struct list_head        freed;
    /* page directory tree */
    struct amdgpu_vm_pt     root;
    struct dma_fence       *last_update;
    /* … */
};

struct amdgpu_vm_pt {
    struct amdgpu_vm_bo_base    base;
    struct amdgpu_vm_pt        *entries;
    /* … */
};
```

The PT tree is recursively defined, mirroring the hardware page-table format (4 levels for 48-bit GPUVM, etc.).

Each `amdgpu_bo_va` is a per-mapping struct: which BO maps where in this VM, with what flags.

```
struct amdgpu_bo_va {
    struct amdgpu_vm_bo_base    base;
    /* … */
    struct list_head            valids;
    struct list_head            invalids;
};
```

51.3 Mapping a BO

`DRM_IOCTL_AMDGPU_GEM_VA` with `op=MAP`:

1. Allocate a `bo_va` mapping range in the VM's `va` rb-tree.
2. Append to `bo->valids` / `invalids`.
3. Schedule a page-table update — actually writing PTEs is deferred.

`amdgpu_vm_bo_update()` performs PT updates, either via SDMA copy job (default — fast, async) or CPU writes (`amdgpu_vm_use_cpu_for_update`). PT updates produce a fence; subsequent CSes wait on it.

PT updates are batched per-VM: the kernel coalesces many incremental mappings into one SDMA submission.

51.4 The PDE / PTE format

Generation-specific. RDNA 3 uses 64-bit entries with bits for: present, valid, system/local, MTYPE (cache type), R/W, fragment size, large-page, address. Look in `include/asm-generic/sizes.h`-style headers in `drivers/gpu/drm/amd/include/`.

51.5 Eviction and re-binding

When TTM evicts a BO from VRAM to GTT (or system), its PTEs become invalid (the GPU physical pages have moved). The driver must:

1. Mark the BO's mappings as invalid.
2. Add the BO to `vm->evicted`.
3. On the next CS for this VM, walk `evicted` and re-update PTEs to point at the new physical pages.

This is the *most* concurrency-fraught path. Reading `amdgpu_vm.c::amdgpu_vm_handle_moved` is a rite of passage.

51.6 Userptr BOs and MMU notifiers

Userptr maps user pages into GPUVM. If the user munmaps or mremaps, those pages may go away. The kernel registers a **MMU interval notifier**:

```
struct mmu_interval_notifier {
    /* registered on user's mm covering [start, end) */
    bool (*invalidate)(struct mmu_interval_notifier *,
                       const struct mmu_notifier_range *,
```

```
                                unsigned long cur_seq);
        };
```

When the user changes mappings, the kernel calls `invalidate`. amdgpu must:

1. Stall any in-flight GPU work using the BO.
2. Unmap from GPUVM.
3. Possibly retry with new pages.

Code: `amdgpu_mn.c`. Treacherous because it can be called from any context, with various locks held by the MM core.

51.7 Shared Virtual Memory (SVM)

In ROCm, you can let the GPU access *any* address in the user's process address space. The GPU faults on missed pages; the kernel resolves via the same path the CPU's page-fault handler uses; pages get pinned and mapped on demand.

This needs:

- IOMMU SVA (Shared Virtual Addressing) — the IOMMU walks the CPU's page tables.
- KFD's SVM API for finer control.

Code: `drivers/gpu/drm/amd/amdkfd/kfd_svm.c`. Currently the most rapidly evolving part of the AMD GPU driver.

51.8 The GMC

GMC (GPU Memory Controller) is the IP block that owns:

- VRAM aperture, its size, its base.
- GTT aperture (system memory window).
- VMID flushing.
- The on-chip page-walk hardware.

`gmc_v*.c` per-generation files. Setting up GMC is one of `hw_init`'s steps for the IP.

51.9 Page faults

When the GPU dereferences an unmapped GPUVA, it raises a page fault. Two flavors:

- **Recoverable**: the kernel resolves (e.g. SVM), tells the GPU to retry.
- **Fatal**: the kernel tags the fault and signals an error. Affected job is canceled; reset may follow.

Page faults are delivered through the IH ring as a special cookie. `amdgpu_irq.c` and `gmc_v*.c::process_interrupt` handle them.

If you ever see "VM_L2 RDREQ" in dmesg, that's the GPU reporting a VM page fault.

51.10 Concurrency

GPUVM updates can happen from:

- CS ioctl threads (mapping new BOs).

- TTM eviction (re-mapping moved BOs).
- KFD context (process address-space changes).
- Reset path (rebuilding everything).

All of these synchronize through:

- `amdgpu_vm.lock` (`mutex` + `dma_resv` for the root).
- ww_mutex on each BO's resv.
- Per-VM update fences.

Reading `amdgpu_vm.c` is reading distributed-systems-grade synchronization.

51.11 Summary mental model

- Per-process GPU page tables, like CPU page tables, but updated by SDMA.
- TTM moves BOs; VM keeps PTEs current.
- Userptr + MMU notifiers tie GPUVM to CPU mm.
- VMID is a scarce hardware resource; flushed and recycled.
- Faults arrive via IH ring; recoverable or fatal.

51.12 Exercises

1. Read `drivers/gpu/drm/amd/amdgpu/amdgpu_vm.c::amdgpu_vm_init`. Watch how the root PT is allocated.
2. Read `amdgpu_vm.c::amdgpu_vm_bo_update`. Identify the PT walking and PTE write path.
3. Read `amdgpu_mn.c::amdgpu_mn_invalidate_hsa`. Understand what stalls the GPU on userspace munmap.
4. Read `drivers/gpu/drm/amd/amdkfd/kfd_svm.c::svm_range_set_attr`. Skim; this is hard code.
5. Trace `amdgpu_vm_update_ptes` with bpftrace under a workload.

Next: Chapter 52 — Userspace: libdrm, Mesa, ROCm, KFD.

Chapter 52 — Userspace: libdrm, Mesa, ROCm, KFD

A kernel driver that no userspace can talk to is dead. AMD's userspace stack has many layers; you'll need to navigate them when debugging anything visible.

52.1 libdrm and libdrm_amdgpu

`libdrm.so` is the thin DRM ioctl wrapper. `libdrm_amdgpu.so.1` is its amdgpu-specific subset; ships with the system.

Public headers: `xf86drm.h`, `xf86drmMode.h`, `amdgpu.h`, `amdgpu_drm.h`.

A minimal program:

```
#include <fcntl.h>
#include <stdio.h>
#include <amdgpu.h>
#include <amdgpu_drm.h>

int main(void) {
    int fd = open("/dev/dri/renderD128", O_RDWR);
    amdgpu_device_handle dev;
    uint32_t major, minor;
    amdgpu_device_initialize(fd, &major, &minor, &dev);

    struct amdgpu_gpu_info info;
    amdgpu_query_gpu_info(dev, &info);
    printf("family %u  asic_id 0x%x  pci %04x:%04x num_shader_engines %u\n",
           info.family_id, info.asic_id, info.vendor_id, info.device_id,
           info.num_shader_engines);

    amdgpu_device_deinitialize(dev);
    close(fd);
    return 0;
}
```

Build:

```
gcc t.c -ldrm_amdgpu -o t -I/usr/include/libdrm
```

You're talking to amdgpu without Mesa. This is what we'll extend in the labs.

52.2 Mesa

Mesa is the open-source graphics stack. For AMD:

- `src/gallium/drivers/radeonsi` — OpenGL.
- `src/amd/vulkan` (`radv`) — Vulkan.
- `src/amd/compiler` (`aco`) — shader compiler (besides LLVM AMDGPU).
- `src/amd/common` — shared utilities.
- `src/amd/llvm` — LLVM-backed compile path.

Build & install:

```
git clone https://gitlab.freedesktop.org/mesa/mesa.git
cd mesa
meson setup build -Dvulkan-drivers=amd -Dgallium-drivers=radeonsi
ninja -C build
sudo ninja -C build install
```

Test with `vkcube` (Vulkan) or `glmark2` (OpenGL).

52.3 ROCm runtime

ROCr / HSA runtime sits below HIP. For our purposes:

- `/dev/kfd` — KFD device.
- `librocm-hsa-runtime64.so` — talks to KFD.
- `libamdhip64.so` — HIP layer over ROCr.

KFD adds ioctls beyond DRM:

- `KFD_IOC_CREATE_QUEUE` — user-mode queue.
- `KFD_IOC_DESTROY_QUEUE`.
- `KFD_IOC_SET_MEMORY_POLICY`.
- `KFD_IOC_ALLOC_MEMORY_OF_GPU`, `_FREE`.
- `KFD_IOC_MAP_MEMORY_TO_GPU`, `_UNMAP`.
- `KFD_IOC_SVM`.
- … (~70 ioctls in modern KFD).

When you write HIP, every `hipMalloc` / `hipMemcpy` / `hipLaunchKernel` ultimately hits KFD ioctls.

52.4 Reading HIP/ROCm source

Useful repos (all on github.com/ROCm):

- `ROCR-Runtime` — HSA runtime.
- `clr` (formerly ROCclr / HIP clr) — ROCm common language runtime.
- `HIP` — HIP API.
- `ROCm-CompilerSupport`, `llvm-project` — compilers.
- `ROCm-Device-Libs`, `aomp` — device libs.
- `MIOpen`, `rocBLAS`, `RCCL` — math libraries.

The kernel-side counterpart is amdkfd; the userspace counterpart is HSA runtime.

52.5 A minimal compute submission via libdrm_amdgpu

Skeleton (full code in labs):

```
amdgpu_device_initialize(fd, &maj, &min, &dev);

amdgpu_context_handle ctx;
amdgpu_cs_ctx_create(dev, &ctx);
```

```
        /* allocate IB BO */
        amdgpu_bo_alloc_request req = { .alloc_size = 4096,
                                        .preferred_heap = AMDGPU_GEM_DOMAIN_GTT };
        amdgpu_bo_handle ib_bo;
        amdgpu_bo_alloc(dev, &req, &ib_bo);

        uint64_t ib_va = ...; /* via amdgpu_va */
        uint32_t *ib;
        amdgpu_bo_cpu_map(ib_bo, (void**)&ib);

        /* fill IB with PM4 packets: e.g. write 0xcafebabe to a known BO address */

        amdgpu_cs_request request = { .ip_type = AMDGPU_HW_IP_GFX,
                                      .number_of_ibs = 1,
                                      .ibs = &(struct amdgpu_cs_ib_info){
                                          .ib_mc_address = ib_va,
                                          .size = ib_dw_count } };
        uint64_t seq;
        amdgpu_cs_submit(ctx, 0, &request, 1);
        amdgpu_cs_query_fence_status(...);    /* wait for completion */
```

A lab exercise will turn this skeleton into a complete runnable example.

52.6 The Vulkan layer at AMD

`radv` is the dominant open-source Vulkan driver for AMD on Linux. AMDVLK is AMD's official open-source. Both submit through libdrm_amdgpu. `radv` uses the `aco` compiler which is faster to compile shaders than LLVM.

For your job: read `src/amd/vulkan/radv_cmd_buffer.c`. It builds command buffers identical in spirit to what we did with libdrm_amdgpu — much more elaborate.

52.7 The X11/Wayland integration

- **X11 / DRI3**: clients render to dma-bufs, X server (Xorg with xf86-video-amdgpu, or Xwayland) scans them out.
- **Wayland**: compositor uses libdrm directly; calls KMS atomic to flip; clients send dma-bufs over the wire.

For driver work, both look the same — it's all atomic + GEM + dma-buf below.

52.8 Tools you should have

- `glxinfo`, `vulkaninfo`, `clinfo` — capability dumps.
- `vkcube`, `glmark2`, `glxgears` — smoke tests.
- `mangohud` — overlay; FPS, GPU utilization.
- `rocm-smi`, `rocminfo`, `rocprof` — ROCm-side.
- `radeontop` — utilization per ring.
- `umr` (UMR-debugger) — register dump tool, official for amdgpu.

52.9 Debugging userspace ↔ kernel

A bug that "starts in Mesa" sometimes ends in amdgpu. Triage:

1. Reproduce with the simplest test (`vkcube`, `glxgears`).

2. `strace -e ioctl` — what ioctls?
3. `dmesg` — any warnings?
4. `cat /sys/kernel/debug/dri/0/amdgpu_*` — relevant state?
5. `rocm-smi` — GPU temp/fan/voltage.
6. If hang: `umr` to dump registers; `cat /sys/kernel/debug/dri/0/amdgpu_gpu_recover` to force reset.
7. If still mysterious: `ftrace amdgpu*` and `bpftrace`.

Never assume a bug is in one layer until you've ruled out the others.

52.10 Exercises

1. Build the skeleton from §52.5; submit a "WRITE_DATA" PM4 packet that writes a value to a BO; read back; verify.
2. Build a Vulkan minimal compute example with `radv`; observe ioctls with strace.
3. Build Mesa from source against your kernel headers. Run `vulkaninfo` with the new driver.
4. Read one ioctl entry-to-completion: `vkQueueSubmit` → `radv_queue_submit_normal` → `amdgpu_cs_submit_*` → kernel `amdgpu_cs_ioctl`.
5. Use `mangohud` while running a game; correlate FPS dips with `dmesg` events.

Next: Chapter 53 — Reset, recovery, RAS, and the real world.

Chapter 53 — Reset, recovery, RAS, and the real world

GPUs hang. Memory has bit flips. Firmware faults. Production systems must recover gracefully. Reset and RAS code is amdgpu's **highest-leverage** area for a junior — small, important patches in this region get attention and ownership.

53.1 What "reset" means

The driver detects something has gone wrong (job timed out, register-poll hung, memory error, page fault) and:

1. Stop new submissions.
2. Quiesce in-flight (mark fences with errors).
3. Reset hardware (one or more IPs, possibly the whole GPU).
4. Reload firmware.
5. Re-init state (rings, VMIDs, page tables, display).
6. Resume scheduling.

Done well, a single bad shader doesn't crash the desktop; only the offending GPU job's process sees `VK_ERROR_DEVICE_LOST`.

53.2 Reset levels (modes)

amdgpu defines several:

- **soft reset** — reset specific IPs (GFX, SDMA) only.
- **mode-1 reset** — full ASIC reset, but PCIe link stays.
- **mode-2 reset** — partial ASIC reset.
- **PCI FLR (Function-Level Reset)** — last resort; PCIe-level.
- **whole-GPU reset** — combines everything; full re-init.

Selection depends on ASIC, error severity, and platform support. Module param `amdgpu.gpu_recovery` controls.

53.3 The recovery code path

Entry: `amdgpu_device_gpu_recover()`. Long function. Outline:

```
amdgpu_device_lock_adev();              /* exclusive */
drm_sched_stop(all schedulers);
    ↓
quiesce all rings
amdgpu_device_pre_asic_reset();         /* kill jobs */
    ↓
amdgpu_asic_reset();                    /* the actual reset */
amdgpu_device_post_asic_reset();
    ↓
```

```
        amdgpu_device_ip_resume();              /* re-init IPs */
            ↓
        drm_sched_start(all schedulers);
        amdgpu_device_unlock_adev();
```

Sub-paths handle:

- per-process VMs (re-bind PTEs, evicted BOs reload).
- KFD queues (re-create, doorbell ring).
- DC display (re-program OTG, re-light displays).
- PSP firmware reload.

The function is large because every IP must contribute correct shutdown/restart code. Buggy IP code = recovery hangs.

53.4 What the userspace sees

A submitting process whose job was on a reset gets:

- The fence signaled with `-ECANCELED` (or `-ENODEV` for PCI death).
- Subsequent ioctls return errors until the process re-creates its resources.

Mesa/`radv` translates this to `VK_ERROR_DEVICE_LOST`. Vulkan apps must implement device-lost handling — most don't and just crash. Compositors usually re-init and recover.

53.5 RAS — Reliability, Availability, Serviceability

For data-center GPUs (Instinct MI series), bit flips matter. ECC memory, parity in caches, register-level error reporting. The driver:

- Gathers correctable / uncorrectable errors.
- Exposes via sysfs (`/sys/class/drm/card*/device/ras/`).
- Triggers a full reset on fatal errors.
- Reports counts to userspace (rocm-smi).

`drivers/gpu/drm/amd/amdgpu/amdgpu_ras.c` is the core. Per-IP `*_ras.c` files do the IP-specific error decoding.

53.6 Hang detection

Kicked in at multiple layers:

- `drm_sched_job` timeout — job exceeds `drm_gpu_scheduler.timeout`.
- Kernel watchdog on internal poll loops (`amdgpu_ring_test_helper`).
- Userspace MTBF heuristics (Mesa watchdog, KFD).
- Hardware-driven RAS errors via IH ring.

A "hang" can be: shader infinite loop, bad memory access, cache deadlock, link error.

53.7 Page-fault path

When a GPU dereferences an unmapped GPUVA:

1. The hardware raises a fault, posts to IH ring.

2. amdgpu's `gmc_v*::process_interrupt` decodes (which client, address, RW).
3. If recoverable (SVM): kernel resolves; tells GPU to retry.
4. If not: log + kill the offending job (or whole context); reset if necessary.

Most recoverable faults today come from SVM/KFD. Graphics typically uses pre-bound memory.

53.8 The watchdog and the "TDR" experience

Windows-style **TDR** (Timeout Detection and Recovery) — a job stuck for >2 s triggers reset; user sees a brief screen flicker. Linux's amdgpu has equivalent via `drm_sched`. The default 10 s timeout is conservative; for compute, you may want longer.

Module param `amdgpu_lockup_timeout=` lets users configure per ring.

53.9 Production-quality testing

For driver work, your patches will be expected to handle:

- Stress tests (`piglit`, `vkcube --persistent`, `glmark2` long runs).
- Reset injection — `echo 1 > /sys/kernel/debug/dri/0/amdgpu_gpu_recover` triggers a fake reset; observe that everything recovers.
- Fault injection — kfd-ras has interfaces to inject errors.
- Memory pressure — fill VRAM, force eviction, observe correctness.
- Suspend/resume loops — `rtcwake -m mem -s 5` in a loop.

Doing this on your own machine *before* submitting upstream patches is the difference between "merged in v2" and "rejected with months of back-and-forth."

53.10 What junior engineers can land

If you want quick, real, valuable patches:

- Improve dmesg messages in error paths (more useful info, less noise).
- Add tracepoints for newly-tracked events.
- Fix small `WARN_ON`s that fire under specific configurations.
- Improve reset coverage for less-tested combinations (e.g. multi-GPU setups).
- Add new sysfs/debugfs files exposing internal state for triage.
- Document something that's not documented (`Documentation/gpu/amdgpu/...`).

Each is low-risk, high-value, and teaches you the surrounding code.

53.11 Example real-world patch shape

```
Subject: drm/amdgpu: avoid a NULL deref in ras_recovery on init failure

When the PSP fails to initialize during early probe, ras_recovery
attempts to dereference psp->ring before it is allocated, causing a
kernel oops.

Fix it by checking for ring presence before recovery.

Signed-off-by: <you>
```

```
---
 drivers/gpu/drm/amd/amdgpu/amdgpu_ras.c | 3 +++
 1 file changed, 3 insertions(+)
```

A two-line fix gets you on the contributor list. From there, slowly take on bigger pieces.

53.12 Exercises

1. Read `amdgpu_device.c::amdgpu_device_gpu_recover` in full. Sketch a flow diagram on paper.
2. Trigger a manual reset: `echo 1 > /sys/kernel/debug/dri/0/amdgpu_gpu_recover`. Observe dmesg. Confirm the desktop recovers.
3. Read `amdgpu_ras.c::amdgpu_ras_block_late_init`.
4. Search the amd-gfx mailing list for “regression” patches; observe how they’re discussed and merged.
5. Pick one open bug from the AMD bug tracker and attempt to reproduce. (You’ll be surprised how often the answer is “didn’t reproduce, please test.”)

End of Part VII. Move to Part VIII — Hardware/software interface.

Chapter 54 — Computer architecture for systems programmers

You don't need a master's degree in computer architecture, but to write fast driver code you must know how a CPU executes your C, where time goes, and where the silicon is hiding from you.

54.1 What a single core actually does

A modern x86_64 core (Zen 4 / Golden Cove) runs at ~5 GHz, executes 4–6 micro-ops per cycle. That means: a single core does roughly **20 billion small operations per second**. Latency vs. throughput:

- Add two registers: 1 cycle, throughput 4/cycle.
- Multiply: 3 cycles, throughput 1/cycle.
- L1 hit: 4 cycles.
- L2 hit: 12 cycles.
- L3 hit: 30 cycles.
- DRAM: 100–300 cycles.
- Locked atomic: 10–30 cycles uncontended.
- Branch mispredict: 15–20 cycles wasted.
- TLB miss + page-walk: 100–500 cycles.

Memory is the bottleneck. Code that fits in L1 is 100x faster than code that misses to DRAM.

54.2 The pipeline

A core pipelines its work:

1. **Fetch** instructions from L1I.
2. **Decode** into micro-ops.
3. **Rename** registers (so independent ops can run in parallel).
4. **Issue** to execution units.
5. **Execute**.
6. **Retire** in program order.

Modern cores can have 100+ in-flight micro-ops. They speculate past unresolved branches; they execute out-of-order; they predict cache misses. All of this is invisible to your C, but it determines whether your code runs at 10% or 80% of peak.

54.3 The memory hierarchy

```
┌──────────┐
│ Registers │   ~16 GP + 32 SIMD; 0 cycles
├──────────┤
│   L1d     │   ~32 KB; 4 cycles
│   L1i     │   ~32 KB
├──────────┤
│    L2     │   ~512 KB-1MB; 12 cycles
├──────────┤
│    L3     │   ~32 MB shared; 30 cycles
├──────────┤
│   DRAM    │   GBs; 100-300 cycles
├──────────┤
│ NVMe / GPU│   ~µs / ms
└──────────┘
```

Cache lines are **64 bytes**. Loads bring a whole line. Two threads writing different fields in the same line cause **false sharing** (Chapter 23) — performance dies because the line bounces.

54.4 Branch prediction and speculation

Branches are predicted by the BPU. Mispredict = pipeline flush. Branchless code (`cmov`, masking) is sometimes faster than branchy.

Speculative execution = the CPU runs ahead of unresolved branches. Spectre/Meltdown-class attacks abused this. The kernel mitigates with retpolines, IBRS, etc. — visible as slight perf cost on every kernel call.

54.5 Out-of-order and barriers

The CPU may reorder loads/stores within its model. On x86_64 (TSO):

- Loads don't get reordered with earlier loads.
- Stores don't get reordered with earlier stores.
- A store can be reordered with a *later* load to a different address.

For SMP visibility, the kernel uses barriers (`smp_mb`, `smp_rmb`, `smp_wmb`). For a single core, it's the as-if rule.

54.6 SIMD

Modern x86 has SSE (128-bit), AVX (256-bit), AVX-512 (512-bit). ARM has NEON (128) and SVE (variable). One AVX-512 instruction does 8 `double`s in parallel.

GPU "SIMD" is much wider (32 or 64 lanes). The CPU's SIMD is tiny by comparison but used in libc memcpy/memset, hot kernels, video codecs, etc.

For driver work you rarely write SIMD intrinsics; the compiler auto-vectorizes simple loops. For runtime work (Mesa shader compiler, video codecs), you'll see lots of intrinsics.

54.7 ARM/AArch64 differences

- 31 GP regs (`x0-x30`), all 64-bit. `sp` separate.

- LDP/STP load/store pair instructions.
- Weakly-ordered memory model — needs `dmb`, `ldar`, `stlr` for SC.
- 4 KB / 16 KB / 64 KB pages possible.
- Fixed 4-byte instruction width (no variable-length encoding).

If you write driver code that needs to run on Qualcomm SoCs, you'll know AArch64. amdgpu is x86-mostly today but a few amdgpu APUs run on ARM in some form factors.

54.8 RISC-V briefly

The newcomer. Open ISA. Few production GPU-attached systems but presence growing. Same general patterns as ARM: weak ordering, RISC, paged.

54.9 GPU architecture vs. CPU

Already in Chapter 42. Re-anchor here:

- CPU: low latency, deep cache, branch prediction, OoO.
- GPU: high throughput, hardware multithreading hides DRAM latency, SIMD-of-SIMD.

GPUs hate divergent control flow within a wave. CPUs handle it fine.

54.10 What this means for you

- **Profile, don't guess.** `perf` will tell you where cycles go.
- **Cache > algorithm**, often. A linear scan of a vector beats a tree until n is large.
- **Avoid false sharing.** Pad hot per-CPU/per-thread data to a cache line.
- **Use atomics correctly.** Wrong memory order will be slow on ARM, "work" on x86.
- **For MMIO**: the CPU caches don't help; every read is a PCIe transaction. Batch where possible.

54.11 Exercises

1. Write a microbenchmark that times accessing a 64 MB array vs a 16 KB array in a tight loop. Compare; relate to L3 vs. L1.
2. Write code that has false sharing; pad to fix; measure.
3. Run `perf stat -e cache-misses,branch-misses your_program` — interpret.
4. Disassemble (`objdump -d`) a hot inner loop; identify the schedule of micro-ops.
5. Read AMD's "Software Optimization Guide for Zen 4." Skim — internalize that out-of-order is real.

Next: Chapter 55 — Caches, coherence, and memory ordering.

Chapter 55 — Caches, coherence, and memory ordering

The single most-asked AMD/NVIDIA/Qualcomm interview topic outside of basic C. We deepen Chapter 23 here with hardware specifics.

55.1 Cache structure

Modern CPUs have multi-level caches:

- **L1d / L1i** per core, ~32 KB each, ~4 cycles, 8-way associative.
- **L2** per core, ~1 MB, ~12 cycles.
- **L3** shared across cores, MBs, ~30 cycles.

Cache lines: 64 bytes (essentially universal for x86 and ARM).

A line may be in one of (MESI / MOESI):

- **M** — modified (dirty, I have the only copy).
- **O** — owner (modified, but shared).
- **E** — exclusive (clean, only mine).
- **S** — shared (clean, others may have).
- **I** — invalid (no copy).

When core 1 writes, it must transition the line to M, evicting others' copies via a **coherence protocol** (snoops or directories). Bouncing a hot line between cores is expensive — it's the cost of contended atomics.

55.2 Coherence vs. consistency

- **Coherence**: every memory location appears to have a single global ordering; once written, all readers eventually see the new value.
- **Consistency** (memory model): how reads and writes to *different* locations may be reordered.

x86: TSO (Total Store Order). Strong: stores from one core appear in the same order on all cores. Loads can be reordered with earlier stores to different addresses.

ARM: more permissive. Loads/stores can be reordered freely unless barriered.

POWER, Alpha, RISC-V: even more permissive.

55.3 The store buffer

When a core stores, the value goes to a **store buffer** before reaching cache. This lets the core continue executing without waiting. Other cores don't see the store until it drains.

The store buffer is the reason `mfence` exists — to drain it.

55.4 Memory barriers

Barrier	x86 spelling	ARM spelling
Compiler only	`__asm volatile("":::"memory")`	same
Read	(free on x86)	`dmb ld`
Write	(free on x86)	`dmb st`
Full	`mfence` / `lock`-prefixed	`dmb sy`
Acquire load	(free on x86)	`ldar`
Release store	(free on x86)	`stlr`

C++/`<atomic>` orders compile to the right thing per arch. The kernel has `smp_*mb`, `smp_load_acquire`, etc.

55.5 The infamous "double-checked locking"

```
        if (!instance) {
            lock();
            if (!instance) instance = create();
            unlock();
        }
        return instance;
```

In Java pre-5 this was broken because the publish wasn't release. In modern C/C++ with atomic + acquire/release, it works. In the kernel, prefer `cmpxchg` or `static_branch_*`.

55.6 The CPU↔device case

Caches are not coherent with **all** devices. PCIe is, in a sense, by default coherent on x86 (Snoop, ACS), but:

- Whether a device sees CPU writes from the CPU's caches depends on the device's transaction type and the IOMMU.
- For *write-combined* CPU mappings, writes are buffered — `wmb()` flushes.
- For *uncached*, writes are immediate.

amdgpu manages this with `wmb()` after writing descriptors and before doorbell.

55.7 GPU caches

The GPU has L1 caches per CU, L2 globally, sometimes Infinity Cache (very large). These are NOT coherent with the CPU.

To make GPU writes visible to CPU:

- **Flush** GPU L2 (cache flush packet, like `RELEASE_MEM` with EOP).
- The CPU read after fence-signaled is then guaranteed to see the data.

Driver responsibility: emit cache-flush packets in IB epilogues. amdgpu does. If you forget, you get garbage.

55.8 IOMMU and coherence

With IOMMU, devices can choose coherent or non-coherent traffic. For GPU traffic to system memory (GTT), x86 systems usually use snooped writes — the CPU caches stay consistent. For VRAM traffic, the CPU isn't involved.

ARM systems sometimes use I/O coherency (ACE-Lite). When designing drivers for ARM SoC GPUs (Qualcomm Adreno via msm), you must explicitly handle.

55.9 Practical guidance

- For shared CPU-CPU data: use C++ `std::atomic` or kernel atomics; pick correct order.
- For CPU↔device shared data via DMA: trust `dma_alloc_coherent` for descriptors, use `dma_sync_single_for_*` on streaming.
- For CPU↔GPU command rings: use WC mappings and explicit barriers.
- For perf-critical code: use `perf stat` to measure cache-miss rate; restructure data layout for locality.

55.10 Exercises

1. Write code that increments two adjacent `int`s from two threads. Pad them to separate cache lines. Compare throughput.
2. On x86, write a release-acquire publish/consume; verify with TSan.
3. On ARM (in QEMU or a real device), repeat — note that without barriers, the same code is broken.
4. Read `Documentation/atomic_t.txt` and `Documentation/memory-barriers.txt`.
5. Read AMD's GPU "Cache Hierarchy" section in the architecture white-paper for whichever generation you have.

Next: Chapter 56 — PCIe deep dive.

Chapter 56 — PCIe deep dive

Almost every interview at AMD/NVIDIA/Qualcomm asks about PCIe. We took a tour in Chapter 37; here we go deep enough to discuss bandwidth, ordering, and pitfalls.

56.1 PCIe is a layered packet protocol

```
┌──────────────────────┐
│  Software API        │   pci_dev, dma_*
├──────────────────────┤
│  Transaction Layer   │   TLPs (Transaction Layer Packets)
├──────────────────────┤
│  Data Link Layer     │   reliability, DLLPs, ACK/NAK, flow ctrl
├──────────────────────┤
│  Physical Layer      │   8b/10b or 128b/130b, lanes
└──────────────────────┘
```

A PCIe transaction is a **TLP**: header + optional payload, addressed to an endpoint by Bus/Device/Function or by memory address.

56.2 Lanes and generations

Bandwidth per lane (per direction):

Gen	Encoding	Raw GT/s	Effective GB/s/lane
1	8b/10b	2.5	0.25
2	8b/10b	5.0	0.5
3	128b/130b	8.0	0.985
4	128b/130b	16.0	1.97
5	128b/130b	32.0	3.94
6	PAM4+FEC	64.0	7.56
7	PAM4+FEC	128.0	15.13

Cards advertise width (x1/x2/x4/x8/x16). Aggregate = lanes × per-lane.

A modern GPU is x16 PCIe Gen4 or Gen5: ~32 GB/s or ~63 GB/s per direction.

`lspci -vv -s 01:00.0 | grep -E 'LnkSta|LnkCap'` reports actual vs. capable.

56.3 TLP types

- **Memory read/write** (MRd / MWr).
- **I/O read/write** (legacy x86 port I/O, rare today).
- **Configuration read/write** (cfgrd0/cfgwr0).
- **Message** (interrupts, errors, hot-plug).
- **Completion** for non-posted requests.

Most GPU traffic is MWr (descriptor + payload writes from GPU to host RAM, doorbell writes from CPU to GPU) and MRd (GPU fetching commands, fences).

56.4 Posted vs. non-posted

- **Posted**: write goes, no completion expected. Includes Memory writes.
- **Non-posted**: requires a completion. Reads (memory and config) and config writes.

A read pushes posted writes through ordering rules → fence semantics on PCIe.

56.5 Ordering rules (briefly)

- Within a TC (Traffic Class), strong ordering by default for posted-vs-posted: writes don't pass writes.
- Reads can pass writes only with **Relaxed Ordering** bit set.
- Completions are unordered with each other in some cases.

GPU drivers assume default (strong) ordering. RO is rarely used.

56.6 BARs

Each device has up to 6 BARs declaring address windows:

```
BAR[i]:
  bit 0: 0=memory, 1=I/O
  bit 1-2: 64-bit (10) or 32-bit (00)
  bit 3: prefetchable (1) — write-combinable
  bits 4+: address (read after writing 1s gives size mask)
```

GPU has typically:

- BAR0 (Memory, prefetchable, large): VRAM aperture (or part of it).
- BAR1: doorbell/MMIO mix (varies by gen).
- BAR2: registers.
- BAR5: ROM.

Resizable BAR (ReBAR / SAM): firmware tells the chipset to allocate a 16 GB BAR0 on systems supporting it; the CPU can map all of VRAM.

56.7 ATS, PRI, ACS

- **ATS (Address Translation Service)**: device caches address translations. Reduces IOMMU pressure for GPUs.
- **PRI (Page Request Interface)**: device asks the OS to handle a page fault. Enables SVM.
- **ACS (Access Control Services)**: prevents peer-to-peer DMA between devices unless OS allows.

amdgpu, with KFD, uses ATS+PRI for SVM on platforms that support it. ACS interacts with VFIO IOMMU groups (security boundary).

56.8 Speed / latency

Round-trip MMIO read latency over PCIe is ~hundreds of ns to ~1 µs depending on platform. **Avoid hot-path reads.** Writes are posted, much faster.

DMA for bulk transfer is ~1 µs setup, then GB/s steady-state. Doorbells are µs-scale.

56.9 Common PCIe issues

- **Link not at full speed**: BIOS, PCIe re-training, faulty cable/slot. Diagnose with `lspci -vv`.
- **Hot-plug failure**: ACS, PCIe slot config. Linux's `setpci` and DSDT tweaks help.
- **DMA mask too restrictive**: SWIOTLB engaged, slowness.
- **Bandwidth-bound workload**: PCIe gen too low, too few lanes.

56.10 Multi-GPU and P2P

Two GPUs on the same root complex can DMA directly between each other (peer-to-peer) without going through host RAM. amdgpu supports P2P DMA with explicit setup (`amdgpu_p2p` flags). Used by RCCL for multi-GPU collectives.

ACS interferes (it forces P2P traffic through the root complex). On servers, BIOS has knobs to disable ACS for performance, accepting reduced isolation.

56.11 Reading a real GPU's PCIe state

```
lspci -vv -s 01:00.0
01:00.0 VGA compatible controller: Advanced Micro Devices, Inc. [AMD/ATI] Navi 31 [Radeon RX 7900
XTX] (rev c8) (prog-if 00 [VGA controller])
    Subsystem: Sapphire Technology Limited Navi 31 [Radeon RX 7900 XTX]
    Flags: bus master, fast devsel, latency 0, IRQ 156
    Memory at 7c000000000 (64-bit, prefetchable) [size=16G]   ← VRAM via ReBAR
    Memory at 7d000000000 (64-bit, prefetchable) [size=2M]
    I/O ports at 1000 [size=256]
    Memory at fcc00000 (32-bit, non-prefetchable) [size=512K] ← MMIO regs
    Capabilities: [48] Vendor Specific Information
    Capabilities: [50] Power Management version 3
    Capabilities: [58] Express (v2) Endpoint, MSI 00
    LnkSta: Speed 16GT/s, Width x16
    Capabilities: [a0] MSI: Enable+ Count=1/1 Maskable+ 64bit+
    Capabilities: [c0] MSI-X: Enable+ Count=8 Masked-
    Capabilities: [120] AER: ...
    Capabilities: [200] Resizable BAR <?>
    Kernel driver in use: amdgpu
```

Read every line. Each capability has a code path in the kernel.

56.12 Exercises

1. `lspci -vv` on your machine. Identify each capability for your GPU.
2. `cat /proc/iomem | grep amdgpu`. Identify each BAR.
3. Read `Documentation/PCI/pcieaer-howto.rst` (Advanced Error Reporting).
4. Read `drivers/pci/setup-bus.c::pci_resize_resource` (ReBAR).
5. With `pcm-pcie` (Intel; runs on AMD too) measure GPU PCIe bandwidth under a workload.

Next: Chapter 57 — Performance: profiling, flamegraphs, GPU profilers.

Chapter 57 — Performance: profiling, flamegraphs, GPU profilers

You can't optimize what you can't measure. This chapter teaches the toolset for both CPU-side driver perf and GPU-side workload perf.

57.1 The scientific method

1. **Have a hypothesis** ("submission overhead is the bottleneck").
2. **Measure** with a tool appropriate to the hypothesis.
3. **Confirm or reject.**
4. **Optimize the actual hotspot, not what you guessed.**

90% of perf claims I've seen in industry were wrong before measurement.

57.2 perf

CPU sampling profiler.

```
sudo perf record -F 999 -ag -- sleep 5
sudo perf report
```

Top consumers, per function. With `-g`, callgraphs.

`perf top` — live view.

`perf stat` — counters: cycles, instructions, cache misses, branch misses.

```
sudo perf stat -e cycles,instructions,cache-misses,cache-references \
                ./your_program
```

`perf trace` — strace replacement, much faster.

57.3 Flame graphs

```
sudo perf record -F 99 -ag -- sleep 30
sudo perf script | stackcollapse-perf.pl | flamegraph.pl > flame.svg
```

Open in browser. Hot stacks are wide. The clearest visualization.

Run during a Vulkan game; identify amdgpu functions in the hot path.

57.4 ftrace

Kernel function tracer (Chapter 41). For driver-internal latency:

```
sudo trace-cmd record -p function_graph -l 'amdgpu_*'
```

Examine in `kernelshark`.

57.5 bpftrace / bcc

Dynamic tracing. One-liner:

```
sudo bpftrace -e '
kprobe:amdgpu_cs_ioctl { @start[tid] = nsecs; }
kretprobe:amdgpu_cs_ioctl /@start[tid]/ { @lat = hist(nsecs - @start[tid]); delete(@start[tid]); }'
```

Get a histogram of CS ioctl latency.

57.6 GPU profiling

For ROCm:

- `rocprof --hip-trace --hsa-trace ./your_app` — collect kernel times, HIP events, HSA-side.
- Output JSON; visualize in `chrome://tracing`.

For graphics:

- **`renderdoc`** — frame capture and profiling.
- **`Radeon GPU Profiler (RGP)`** + `radeonprofiler` — AMD's official.
- **`gputrace`** — Mesa-internal.

Identify per-shader cost, draw call cost, memory pressure.

57.7 GPU performance counters

Modern GPUs expose hundreds of counters: cache hits, ALU utilization, wave occupancy, memory bandwidth. AMD's interface: `rocprof -i input.txt`.

Counters tell you *why* a shader is slow:

- Low **VALU utilization** + high stall: memory-bound.
- High register pressure → low occupancy: cut register usage.
- LDS bank conflicts: rearrange access pattern.

57.8 Where time goes in amdgpu

Typical CS ioctl breakdown (rough):

Phase	µs
ioctl entry, copy_from_user	1
BO list resolve + ww_mutex	1–10

Phase	µs
IB validation	0.5
build sched job	0.5
schedule_main wakeup	1
ring emit + doorbell	1

So 5–15 µs per submit on a not-very-contended system. KFD user-mode queues bypass the ioctl: <1 µs.

For graphics, draw-call rate is bound by GPU (state changes, shader compiles), not submission.

57.9 Memory bandwidth

DRAM bandwidth: ~25 GB/s per channel (DDR4-3200). Modern desktops have 2 channels: 50 GB/s aggregate. Workstations 4-8 channels. GPU memory: 1–2 TB/s for HBM3 / GDDR6X.

PCIe Gen4 x16: ~32 GB/s per direction. A GPU-bound application moving data in and out of VRAM is gated here.

`pcm-memory` and AMD's `uprof` can measure DRAM bandwidth.

57.10 Power and thermal

`rocm-smi` shows clocks, voltages, temperatures, power. Throttling shows up as drops in `sclk` (shader clock). `journalctl -k | grep dpm` for kernel-side PM logs.

Many "perf bugs" are actually thermal throttling under inadequate cooling.

57.11 Practical tips

- **Always rebuild with optimizations** (`-O2`); profiling `-O0` code is meaningless.
- **Disable address randomization** for reproducible profiles: `setarch x86_64 -R`.
- **Pin the workload** to a CPU set; keep noise out: `taskset 0xff`.
- **Run for long enough**; tools need samples.
- **Compare apples to apples**; always have a baseline measurement.

57.12 Exercises

1. Profile `vkcube` with `perf record -ag`. Identify the top 5 functions.
2. Profile `glmark2 -bbuild`; flamegraph.
3. Use bpftrace to histogram `amdgpu_cs_ioctl` latency under load.
4. Run `rocprof --stats` on a HIP sample (`rocm-examples` repo); identify the longest kernel.
5. Use `rocm-smi -f` while running a workload; observe if clocks throttle.

Next: Chapter 58 — Debugging: gdb, kgdb, crash, ftrace, dmesg, lockdep.

Chapter 58 — Debugging: gdb, kgdb, crash, ftrace, dmesg, lockdep

This is the toolbox you'll reach for when something is broken. Master each.

58.1 dmesg first, always

The kernel speaks via dmesg. Read every word of the relevant section.

Look for:

- **Call traces** — kernel stack at oops/warning.
- `KASAN` **reports** — memory bugs.
- `Lockdep` **reports** — locking bugs.
- `amdgpu` prefixes — driver-specific.

Save the full dmesg before any reboot or rmmod for analysis.

58.2 gdb on userspace

```
gdb --args ./your_program arg1 arg2
(gdb) run
(gdb) bt
(gdb) print var
(gdb) frame N
(gdb) up / down
(gdb) info threads
(gdb) thread 3
(gdb) watch *(int*)0x1234
```

For C++:

- `(gdb) p/r obj` — raw, skip pretty-printer.
- Auto-load libstdc++ pretty-printers in your `.gdbinit`.

58.3 gdb on the kernel (under QEMU)

```
qemu-system-x86_64 -kernel bzImage -append "nokaslr" -s -S
gdb vmlinux
(gdb) target remote :1234
(gdb) break start_kernel
(gdb) c
```

`-S` stops on entry; `-s` opens GDB stub on port 1234. `nokaslr` matters because gdb uses absolute addresses.

Set `lx-symbols` and other helpers from `scripts/gdb/`.

58.4 kgdb on real hardware

Boot kernel with `kgdboc=ttyS0,115200` and `kgdbwait`. On serial host, `gdb vmlinux; target remote /dev/ttyS0`. Step through real machines.

Less convenient than VM but sometimes the only way for hardware-specific bugs.

58.5 crash and kdump

When a kernel panics:

```
echo "kdump_panic = 1" >> /etc/default/kdump   # distro-specific
service kdump start
```

After a panic, the second kernel saves `/proc/vmcore`. Analyze:

```
        sudo crash vmlinux /var/crash/vmcore
        crash> bt
        crash> log
        crash> ps
        crash> mod -d            # list modules
```

`crash` is GDB-on-steroids for postmortem.

58.6 ftrace for kernel function tracing

```
        echo function_graph > /sys/kernel/tracing/current_tracer
        echo 'amdgpu_*' > /sys/kernel/tracing/set_ftrace_filter
        echo 1 > /sys/kernel/tracing/tracing_on
        # … reproduce …
        echo 0 > /sys/kernel/tracing/tracing_on
        cat /sys/kernel/tracing/trace > my.trace
```

Or use `trace-cmd`:

```
        sudo trace-cmd record -p function_graph -l 'amdgpu_*'
        sudo trace-cmd report > log.txt
```

For a hung path, ftrace is gold. You see every entry/exit; if a function never returns, the indent never closes — that's your hang.

58.7 lockdep, KASAN, KCSAN, KMSAN, UBSAN, KFENCE, DEBUG_VM

Build with these on for development. Each finds a specific bug class:

- **lockdep** — lock ordering and irq-safety.
- **KASAN** — use-after-free, OOB.
- **KFENCE** — sampled, low-overhead OOB.
- **KCSAN** — data races.
- **KMSAN** — uninitialized reads (clang only).
- **UBSAN** — undefined behavior.
- **DEBUG_VM** — invariant checks in MM.

- **DEBUG_KOBJECT** — sysfs lifetime.

Boot with all of them on at least once before sending a patch series.

58.8 Driver-specific: amdgpu

- `amdgpu.debug_mask=0x...` — turn on dev-only logging.
- `/sys/kernel/debug/dri/0/amdgpu_*` — internal state.
- `umr` — register-level dump tool. `sudo umr -ls` lists rings; `sudo umr -gr 0 0 0x6118` reads register.
- `amdgpu_gpu_recover` — force a reset.

For hangs:

1. Capture `dmesg` (full).
2. Dump `/sys/kernel/debug/dri/0/amdgpu_fence_info`, `_gem_info`, `_vm_info`.
3. `umr -O halt_waves; umr -wa -O wave_status -i ...` — examine in-flight waves.
4. Submit bug report with above.

58.9 Reproducing intermittent bugs

- **Run the test in a tight loop** with logging.
- **Increase concurrency** to compress timing windows.
- **Add `udelay`** in suspicious paths to widen race windows.
- **Use KCSAN** for data races.
- **Use `ftrace`** to pinpoint when the divergence began.

A bug you can't reproduce is a bug you can't fix.

58.10 Reading core dumps

Userspace SIGSEGV → core dump (if `ulimit -c unlimited`):

```
gdb ./your_app core.12345
(gdb) bt
(gdb) info threads
```

For amdgpu-induced segfaults from misuse of libdrm, the core often points right at the bad ioctl call site.

58.11 Static analysis

- `sparse` — kernel's own. Catches `__user` / `__iomem` mismatches, endian bugs.
- `clang --analyze`, `clang-tidy`.
- `cppcheck`.
- `coccinelle` (`make coccicheck`) — semantic patches.

Each finds different bugs. Run them periodically.

58.12 The triage checklist

When a bug lands on your desk:

1. Reproduce locally?
2. dmesg full, before+after?
3. lspci, kernel version, distro?
4. Last known good kernel?
5. Simple reproducer?
6. `git bisect` if regression.
7. ftrace, then bpftrace, then crash dump if needed.

A clean triage report goes a long way to getting the right person to look.

58.13 Exercises

1. Boot a debug kernel (KASAN+lockdep+DEBUG_VM). Insmod a module that intentionally takes locks in two orders; observe lockdep.
2. Cause a userspace SEGV; load core in gdb; navigate frames.
3. Trace your edu driver with ftrace; observe the call sequence on a write/IRQ.
4. Set up kdump on a real machine; force a panic with `echo c > /proc/sysrq-trigger`; reboot; analyze with `crash`.
5. Read `Documentation/admin-guide/bug-hunting.rst`. Internalize.

End of Part VIII. Move to Part IX — Getting hired (AMD / NVIDIA / Qualcomm / Intel).

Chapter 59 — What AMD / NVIDIA / Qualcomm / Intel actually look for

Time to be blunt. This chapter is the playbook for a junior with a contractor / new-grad position who wants to convert to FTE in the systems / GPU / SoC parts of these companies. The advice is gathered from real hiring loops and panel feedback; it is the same shape across all four.

59.1 The roles

- **Kernel / driver engineer (Linux)**: writes amdgpu / open-gpu-kernel-modules / msm / i915 code. Heavy C, kernel APIs, hardware bring-up, debugging, upstream contribution.
- **Runtime / compiler engineer**: ROCm, CUDA, Mesa, MIOpen, MIGraphX. C++ and LLVM-flavored work.
- **Tools / profiling**: rocprof, RGP, NSight. C++ and visualization.
- **GPU verification / silicon bringup**: writes test code that runs on FPGA emulators of yet-unreleased silicon.
- **HW/SW co-design**: bridges silicon and software.

This book primarily targets the first. The skills overlap heavily with the second.

59.2 What they're really screening for

The actual signal at AMD / NVIDIA / Qualcomm interview loops:

1. **Can you write correct, idiomatic C** and explain memory layout? (Almost every loop tests this.)
2. **Do you understand pointers, lifetimes, and undefined behavior** at the level of "I can read kernel patches and not be confused"?
3. **Concurrency**: mutexes, atomics, memory ordering, deadlocks. Always asked.
4. **Operating systems**: VM, paging, syscalls, scheduling, IRQ. Always asked.
5. **Computer architecture**: caches, coherence, branch prediction. Often asked.
6. **PCIe / DMA / MMIO**: at least one question.
7. **GPU concepts** (if you're claiming GPU exposure): wavefronts, occupancy, SIMD, fences, buffers.
8. **Debugging story**: "Tell me about the hardest bug you fixed." Real, honest answer expected.
9. **Code review / collaboration**: "How would you review this patch?" or "Walk me through your patch process."

Things that *don't* matter much for systems hires:

- LeetCode hard. (You'll see one or two graph problems, not 300.)
- Frontend / web framework knowledge.
- Cloud / kubernetes (unless the team is platform team).

- Latest ML buzzwords.

59.3 The bar for “competent junior” in 2026

You will be expected to:

- Build a custom kernel and out-of-tree module without help.
- Debug a kernel oops by reading a stack trace.
- Implement a producer/consumer buffer with proper synchronization.
- Write a small `ioctl`-based driver against QEMU.
- Read and explain a real DRM/amdgpu function.
- Discuss `dma-fence`, MMU, and how a Vulkan submit hits the kernel.

This book has prepared every one of those.

59.4 What earns you FTE conversion

Internal offers happen to people who:

- **Become the local expert** on something. “Ask Sudh about reset.” Pick a sub-area; own it.
- **Land upstream patches** with their AMD email. Even tiny ones. Visible to managers.
- **Help others**: review patches, answer team questions, write internal docs.
- **Communicate clearly**. Standups, design docs, emails. Strong written communication is often the deciding factor for converting.
- **Don’t argue with hardware.** When the silicon does something weird, you accept and adapt; you don’t blame the spec.
- **Understand customer use cases.** A patch that solves a real game/app issue reads better than a “cleanup.”

A blunt truth: the contractor-to-FTE path at AMD/NVIDIA/Qualcomm is a **performance review** dressed up as recruitment. By the time the requisition opens, the team usually knows whom they want. Be that person before the req opens.

59.5 The 90-day plan when you join

If you’ve just joined as a contractor in the amdgpu team:

Days 1-30: build, boot, and load a custom amdgpu on hardware you have. Read every IP block’s `_init` function. Triage one open bug in your area to the line.

Days 31-60: ship one small upstream patch (typo, dmesg improvement, missing const). Get it merged. Onboard to internal tooling, the test farm, the bug tracker.

Days 61-90: own one component (say, `gfx_v11_0.c`) at the level of “I can review patches there.” Write or update one piece of internal documentation. Pick a real customer-facing bug and own it through fix.

By day 100, your manager should be saying “this contractor is doing FTE-grade work.” That gets the requisition opened.

59.6 Companies' cultures (rough)

- **AMD**: open-source-heavy, mailing-list culture, distributed teams, Indian + Eastern-European + N. American hubs. Output-focused; hardware-spec discipline.
- **NVIDIA**: closed-source kernel module historically; OSS now (open-gpu-kernel-modules). High intensity, deep specialization. Expect HW NDA early.
- **Qualcomm**: SoC-centric (Adreno GPU on Snapdragon). msm DRM, more downstream Android-focused. Carrier requirements drive a lot of work.
- **Intel**: i915/Xe DRM open. Strong in tooling, perf engineering. Different products different teams.

Pick your battles where the culture fits.

59.7 The non-technical signals

- **Punctuality** matters. Don't be late to scheduled syncs.
- **Replying** to email/Slack quickly with thoughtful answers.
- **Asking** good questions: scoped, with context, with what you've tried.
- **Saying "I don't know"** when you don't, then finding out.
- **Avoiding** bikeshedding (style nits in code review when they don't matter).

These are real factors in conversion decisions.

59.8 Exercises

1. List three areas of amdgpu you could plausibly own in 6 months. Pick one.
2. Subscribe to amd-gfx@lists.freedesktop.org. Spend a week just reading patches. Notice the rhythm.
3. Pick one open issue from `gitlab.freedesktop.org/drm/amd/-/issues` and read every comment. Form an opinion.
4. Identify two engineers on your team whose work you admire. Read their last 20 commits. Note what they value.
5. Write a one-page personal "what I will own this quarter" doc. Share with your manager.

Next: Chapter 60 — System-software interview question bank with answers.

Chapter 60 — System-software interview question bank with answers

A curated list of questions taken from real loops. Each has a model answer that demonstrates senior-level thinking. Memorize the answers, but also be able to riff.

60.1 C language

Q1. Why does this print 0?

```
unsigned u = 1;
int s = -1;
if (s < u) puts("a"); else puts("b");
```

A: Mixed signed/unsigned comparison promotes both to `unsigned int`. `(unsigned)-1` is `UINT_MAX`, greater than `1`. So we go to "b", not "a." This is integer-promotion + 2's-complement.

Q2. What's the difference between `int *p` and `int * const p`?

A: `int *p` is a pointer to int; both `p` and `*p` are mutable. `int * const p` is a const pointer to int; `*p` is mutable but `p` cannot be reassigned. Read declarations right-to-left.

Q3. Why is this UB?

```
int a = INT_MAX;
a = a + 1;
```

A: Signed integer overflow. Compiler may assume it doesn't happen and remove subsequent overflow checks.

Q4. What is strict aliasing?

A: The compiler may assume two pointers of incompatible types don't point to the same memory; only `char*` and `unsigned char*` may alias anything. Use `memcpy` to bit-cast.

Q5. Implement `container_of`.

A:

```
#define offsetof(T, m) ((size_t)&((T*)0)->m)
#define container_of(ptr, type, member) \
    ((type*)((char*)(ptr) - offsetof(type, member)))
```

60.2 C++

Q1. Rule of 5.

A: If you define one of {dtor, copy ctor, copy assign, move ctor, move assign}, define (or `=delete`) all five. Rule of 0: prefer to need none, by composing types that manage themselves.

Q2. What does `std::move(x)` do?

A: It is `static_cast<T&&>(x)`. It does nothing at runtime. It changes the type so move overloads are picked.

Q3. When would you reach for `shared_ptr` over `unique_ptr`?

A: When ownership is genuinely shared across multiple owners with non-trivial lifetimes. Otherwise prefer `unique_ptr` — it's cheaper, single-ownership is clearer.

Q4. Why does `std::vector<T>::push_back` move when `T`'s move is `noexcept`?

A: Strong exception guarantee: if the move can't throw, growing the vector and moving elements is safe to roll back. If move can throw, fall back to copy.

Q5. Difference between `std::function` and a templated callable parameter?

A: `std::function` is type-erased — fixed type, indirect call, may allocate. Template is monomorphized — generic but produces a separate instantiation per callable.

60.3 Operating systems

Q1. fork vs. exec.

A: `fork` duplicates the process (COW). `exec` replaces the image. `fork` + `exec` is the standard way to start a new program with custom env/argv.

Q2. What is virtual memory and why?

A: Each process sees its own address space mapped via the MMU and page tables to physical RAM. Provides isolation, over-commit, and copy-on-write fork. We discussed in Chapter 19.

Q3. What's the difference between user and kernel mode?

A: A CPU bit (CPL=0 vs 3 on x86) decides privileged-instruction access. User code can only enter kernel via syscall/exception/interrupt. Kernel pages are not accessible to user code.

Q4. What does `mmap` actually do?

A: Adds a VMA to the process's mm; doesn't allocate physical pages. On first access, page-fault handler allocates and installs PTEs. Backing can be a file, anonymous, or device.

Q5. CFS in one paragraph.

A: Linux's default scheduler. Each task has a `vruntime`; scheduler picks the leftmost in a red-black tree keyed by `vruntime`. Tasks accumulate `vruntime` weighted by nice value.

60.4 Multithreading and concurrency

Q1. Difference between `std::atomic` and `volatile`.

A: `volatile` prevents compiler caching of the value but does not insert any CPU memory ordering. `std::atomic` provides both atomicity and ordering. Use `atomic` for thread synchronization; `volatile` for MMIO and signal handlers.

Q2. Acquire-release semantics.

A: A `release` store makes prior writes in the same thread visible to a thread doing an `acquire` load on the same atomic. The standard pairing for hand-off: producer writes data then `release`-stores a flag; consumer `acquire`-loads the flag, then reads data.

Q3. Implement a SPSC ring buffer.

A: head and tail are atomics. Producer writes data, then release-stores new head. Consumer acquire-loads head; if differs from tail, reads data, advances tail with relaxed store. Power-of-2 size for cheap modulo.

Q4. ABA problem.

A: A thread reads a pointer A, plans to CAS, but in between another thread pops A, pushes A again with different state. CAS succeeds, but state has changed. Mitigations: tagged pointers, hazard pointers, RCU.

Q5. Spinlock vs mutex.

A: Spinlock busy-waits; cheap if uncontended and held briefly; required in atomic contexts. Mutex sleeps; better for longer or contention-heavy critical sections. In the kernel, spinlock is the default for short, IRQ-safe sections; mutex for sleepable.

Q6. Deadlock — how to prevent?

A: Establish a global lock order; always acquire in the same order. Tools: lockdep. Use `std::scoped_lock` / `mutex_trylock` retry.

Q7. RCU in 30 seconds.

A: Readers traverse without locking. Writers publish via atomic store, then wait for a grace period (every reader has passed a quiescent state) before reclaiming the old. Trades write-side latency for zero read-side cost. Used for read-mostly data.

60.5 Memory model and architecture

Q1. Why is `cache_line_size` 64 bytes?

A: Industry consensus tradeoff between coherence overhead and bandwidth utilization. Larger lines mean more false sharing; smaller lines mean more tag overhead. 64 hits a sweet spot.

Q2. What is false sharing?

A: Two threads write different variables on the same cache line. The line bounces between cores' caches via coherence, killing performance. Fix: align hot per-thread data to a cache line.

Q3. TSO vs weakly ordered (x86 vs ARM).

A: x86 TSO: stores ordered, loads can pass earlier stores to other addresses. ARM weak: nearly anything can be reordered; explicit barriers required. Portable code uses C++ `<atomic>` orders.

Q4. What is the store buffer?

A: A small per-core FIFO holding pending writes that haven't yet reached the cache. Lets the core continue executing without waiting. On x86, the store buffer can reorder a younger load past an older store to a different address.

60.6 PCIe and DMA

Q1. Posted vs. non-posted PCIe writes.

A: Memory writes are posted: no completion expected. Reads and config writes are non-posted: require completion. A read is the way to "fence" prior posted writes.

Q2. Coherent vs. streaming DMA.

A: Coherent (`dma_alloc_coherent`): kernel returns a buffer with consistent CPU/device view, no flushing required, often uncached. Streaming (`dma_map_single`): existing buffer mapped for one transfer; CPU caches flushed/invalidated at map/unmap.

Q3. Why might DMA fail with old hardware?

A: Device's DMA mask doesn't cover the buffer's physical address. Kernel falls back to bounce buffers (SWIOTLB) — slow.

Q4. What does the IOMMU do?

A: Translates device addresses (IOVA) to physical RAM via IO page tables managed by the kernel. Provides protection (devices can only DMA to mapped regions) and abstraction (devices see virtual addresses).

60.7 GPU concepts

Q1. What's a wavefront?

A: AMD's SIMD execution unit — 32 (RDNA) or 64 (CDNA) lanes executing the same instruction in lock-step. Like a CUDA "warp."

Q2. What's a fence in DRM?

A: A `dma_fence` — an opaque signal-once primitive representing "this GPU work has completed." Drivers signal from IRQ; userspace and other drivers wait or attach callbacks.

Q3. What's TTM?

A: Translation Table Manager. A DRM-internal memory manager that handles VRAM/GTT pools, eviction, migration, and cross-driver sharing. amdgpu uses it.

Q4. What's the lifecycle of an `amdgpu_cs_ioctl`?

A: Parse chunks → reserve BOs (ww_mutex) → validate IBs (whitelist) → build `drm_sched_job` → push to scheduler entity → eventually scheduler runs `amdgpu_job_run` → emits PM4 packets to the ring → doorbell → GPU executes → fence packet writes seqno → IRQ → `amdgpu_fence_process` → `dma_fence_signal` → userspace's wait wakes.

Q5. What's a doorbell?

A: A small MMIO/PCIe page the GPU uses as a "wake up" notification channel. CPU writes a value (typically the new wptr); GPU's CP/MES sees it and runs new work. With KFD, doorbells are mapped per-process to userspace for low-overhead submission.

Q6. Per-process VM — why?

A: Each process has its own GPU virtual address space, isolated by the GPU's MMU + per-process VMID. Lets two processes share GPU VAs without conflict and protects against cross-process DMA.

60.8 Debugging

Q1. Tell me about the hardest bug you debugged.

A: Pick one real bug. Tell it as a story:

- Symptom (intermittent crash under suspend on certain laptops).
- Triage (dmesg showed a NULL deref after PSP firmware reload).
- Hypothesis (race between PSP completion and IH handler).
- Tools (ftrace, kdump, lockdep).
- Root cause (callback could race with init; needed seqcount).
- Fix (added barrier + null check; tested with stress suspend loop).
- Lessons (init paths must guard against early IRQs).

If you don't have one yet, build one through the labs.

Q2. Walk me through what you do when you see a kernel oops.

A: Capture full dmesg; find the offending function; resolve symbol via `addr2line` against vmlinux; identify the source line; reason about what state is missing. If reproducible, attach gdb / use ftrace. If a regression, `git bisect`.

Q3. How do you find a memory leak in a long-running driver?

A: Build with kmemleak; run the workload; let it scan; read the report. Cross-check with `slabinfo` for our cache growing. Add more `kmemleak_no_leak` annotations or fix the leak.

60.9 System design

Q1. Design an event-driven I/O subsystem for a device that does both reads and writes asynchronously.

A: Per-fd state: ring buffer for completions, two wait queues (read/write), IRQ handler enqueues. Userspace via `read` / `poll`. Async via eventfd for io_uring integration. Locking: per-fd spinlock (IRQ-safe). Fast path: tracked metric counters via `this_cpu_inc`. Safety: bounded queues, drop-or-block policy.

Q2. Design a GPU command-submission API that minimizes ioctl overhead.

A: User-mode queues: doorbell pages mapped per-process; user writes commands and rings the doorbell directly. Kernel only involved for setup/teardown/error. Sync via shared memory fences. Fairness via firmware scheduler (MES). This is essentially KFD.

Q3. How would you guarantee a 1080p 60Hz display with no tearing on a system under heavy GPU load?

A: Use atomic KMS with `DRM_MODE_PAGE_FLIP_EVENT` synced to vblank. Ensure scanout BO is pinned in VRAM (or fast GTT). Use front/back buffer chain. Set high-priority DRM scheduler for the compositor's submissions. Monitor throttling; ensure DPM stays at high power state during animation.

60.10 Exercises

1. Pick five questions above; record yourself answering them aloud. Cringe at the first take; iterate.

2. Find five more real questions from glassdoor / blind for the team you're targeting; add answers.
3. Pair up with a friend and do mock interviews; both sides.
4. For every C/C++ snippet here, type it out and predict; run; explain discrepancies.
5. Write a one-page "the bug I'm proudest of fixing." This becomes part of your STAR-method answer to the inevitable question.

Next: Chapter 61 — Projects that get you hired.

Chapter 61 — Projects that get you hired

Resumes with hardware/kernel projects pass screens that résumés with "built a CRUD app in Django" don't. Here is a graded list, easiest first, all relevant to the AMD/NVIDIA/Qualcomm pipeline.

61.1 Tier-0 (you must have at least these)

1. **Custom Linux kernel build + boot** — your own bzImage with a `pr_info("hi from <name>")` in `start_kernel`. Document on github.
2. **Out-of-tree hello world `.ko`** — Chapter 28-29 builds it; productionize.
3. **Char driver with read/write/poll/ioctl/mmap** — Chapter 35-36. Add a userspace test program.
4. **PCI driver against QEMU's `edu`** — Chapter 37; with full DMA support.
5. **Userspace libdrm_amdgpu submission program** — opens `/dev/dri/renderD*`, builds an IB, submits, reads result.

If you've done all five, you've passed the bar of "actually wrote driver code."

61.2 Tier-1 (interview gold)

1. **Producer/consumer ring buffer** — both lock-based and lock-free SPSC. Benchmark, write up.
2. **Tiny memory allocator** — arena + free-list. Pass leak/overflow tests under ASan.
3. **Lock-free hash table** — open addressing, linear probing, with hazard pointers or epoch reclamation.
4. **Fictional GPU simulator** — a userspace daemon that pretends to be a GPU; speaks a custom command-buffer language; lets you write a "driver" against it. Forces you to design the API surface.
5. **Tiny KMS-only DRM driver against `vkms`** — enable a virtual scanout, page-flip, write a minimal compositor.

61.3 Tier-2 (top-tier)

1. **A real upstream patch in `drivers/gpu/drm/amd/`** — even a typo or a comment cleanup. The `Signed-off-by` is what counts. Walks ahead of 95% of resumes.
2. **A bigger upstream patch** — maybe a missing `__must_check`, an obvious refactor, an extension to debugfs. After your first patch lands, this is straightforward.
3. **Mesa/`radv` patch** — same, on userspace side. Adds a Vulkan extension or fixes a bug.
4. **A blog post or `Documentation/` patch** — writeup of how some part of amdgpu works. Not duplicating existing docs; explain something poorly-documented. Pulls eyeballs and demonstrates deep understanding.

5. **A reproducer + bug report on amd-gfx** for an open issue you triaged.

61.4 Tier-3 (signature work)

1. **A new feature in amdgpu** — pick something from the bug tracker labeled enhancement: a new ioctl, a sysfs knob, support for a new chip variant.
2. **A subsystem you understand end-to-end** — e.g. you can explain reset path top-to-bottom, write a slide deck, fix bugs in it.
3. **Performance work** — measure, identify, fix. With before/after numbers.

People who land tier-3 work are FTE within a year.

61.5 What to put on github

For each project, a clean repo with:

- **README**: the elevator pitch — what, why, how to build/run.
- **Build instructions** that actually work on a fresh Ubuntu 24.04.
- **Test/example program** showing usage.
- **Design notes** in a `docs/` folder: what you tried, what didn't work.
- **License**: GPL-2.0 for kernel work, MIT/BSD for userspace.

Commits should be small, atomic, with descriptive messages. `git log --oneline` should read like a story.

61.6 Anti-patterns (avoid these)

- **AI-generated code without your understanding** — interviews quickly expose it.
- **A 50-step monorepo with one giant commit** — looks like dump.
- **"Cleaned up codebase"** PRs to popular repos with no real change — adds nothing.
- **Buzzword-laden descriptions** — "blockchain DRM driver." Recruiters' eyes glaze.

61.7 The resume

For systems / GPU roles, a one-page resume with:

- Contact + linkedin + github at top.
- Education, brief.
- Experience: 3-5 bullets per role. Each bullet should have *measurable impact* ("reduced ioctl latency by 40%") or *technical specificity* ("implemented threaded IRQ + workqueue handler for X subsystem").
- Projects (the github ones above): 2-4 bullets each.
- Skills section short: C, C++, Linux kernel, x86_64/ARM64 asm, gdb, perf, git.
- No "soft skills" cliches.

Resume is for the recruiter to forward, not for the interviewer to read. Give it the strongest first two lines.

61.8 The cover letter

Optional. If used:

- 3 paragraphs, max.
- Why this team specifically (named) — show you read about them.
- One concrete piece of evidence for each claimed strength.
- Close with a sentence about what you want to learn there.

For an internal conversion, the “cover letter” is the conversation you have with your manager.

61.9 Exercises

1. Build the five Tier-0 projects this month. Push to github with a clean README.
2. Pick one Tier-1 project; spec it; build it; write up.
3. Subscribe to amd-gfx; spend a week reading; then submit a one-line patch fixing something obvious. Get it merged.
4. Refactor your resume to the systems-job template.
5. Write the “bug I’m proudest of” story; trim to 90 seconds spoken.

Next: Chapter 62 — Contributing to the upstream kernel and Mesa.

Chapter 62 — Contributing to the upstream kernel and Mesa

Landing patches upstream is the single best signal of competence in our field. This chapter is the workflow.

62.1 The kernel patch flow

1. **Find something to fix.** Bug tracker (`gitlab.freedesktop.org/drm/amd`), checkpatch warnings, sparse warnings, regressions on the lists.
2. **Write the patch.** Small, atomic, one logical change.
3. **Test it.** Build, boot, run the relevant workload.
4. **Format it.** `git format-patch -1`. `scripts/checkpatch.pl`. Polish the commit message.
5. **Find maintainers.** `scripts/get_maintainer.pl`.
6. **Send via email** with `git send-email`.
7. **Address feedback.** Respin as v2, v3, etc. with `--in-reply-to`.
8. **Maintainer applies.** It hits a `-next` branch, then linus.

62.2 The commit message

Short subject line: `subsystem: short verb-led description (max ~70 chars)`. Examples:

- `drm/amdgpu: fix NULL deref in ras_recovery on early init failure`
- `drm/amdgpu: add tracepoint for fence wait latency`
- `drm/amdgpu/gfx11: correct CP_HQD_PIPE_PRIORITY register offset`

Body (mandatory, paragraphs separated by blank lines):

- **What** does the patch do.
- **Why** is it needed.
- **How** was it tested.
- **Caveats**, follow-ups, related bugs.

End with `Signed-off-by: Your Name <you@amd.com>` (`git commit -s`).

If fixing a regression, add `Fixes: <12-char SHA> ("subject")`.

If fixing a reported bug: `Reported-by:`, `Tested-by:`, `Reviewed-by:` accumulated as the patch travels.

62.3 The DRM/amdgpu specifics

- Mailing list: `amd-gfx@lists.freedesktop.org`.
- CC: `dri-devel@lists.freedesktop.org` for cross-cutting changes.
- Maintainers: Alex Deucher, Christian König, et al. (`get_maintainer.pl` will tell you).

- Pull-request rhythm: Christian (or Alex) collects to a `drm-next-amd` branch, then PR upstream weekly.

Big-picture rules:

- **Don't break in-tree drivers** with API changes. Convert them in the same series.
- **Add tests** when adding ioctls.
- **Get acks** from outside maintainers when touching DRM core.
- **Mind backwards compatibility** of UAPI. Adding fields to ioctl structs is fine if size grows; never reorder existing ones.

62.4 Mesa patch flow

Mesa lives on GitLab with merge requests, not mailing lists.

1. Fork `gitlab.freedesktop.org/mesa/mesa`.
2. Branch from `main`.
3. Make changes; commit with `Signed-off-by` (Mesa uses DCO too).
4. Push; open MR.
5. CI runs (Marge bot).
6. Reviewers comment.
7. Marge merges when approved.

Subsystems within Mesa: `radv` (Vulkan), `radeonsi` (GL), `aco` (compiler). Each has owners.

62.5 ROCm and KFD specifics

- ROCm code is on `github.com/ROCm`. PRs.
- KFD kernel code is in the Linux kernel tree but rapidly developed; the AMD Kfd-specific list is `amd-gfx`.
- The `kfd-staging` and `amd-staging-drm-next` branches at gitlab.freedesktop.org are where in-flight work lives before going upstream.

For ROCm-side patches, sign the CLA, follow GitHub flow.

62.6 Code review etiquette

- **Address every comment** even if just to say "fixed in v2" or "I disagree because…".
- **Keep diffs small.** A 100-line patch reviews; a 5000-line patch sits.
- **Don't argue** for the sake of arguing. The maintainer's word is final on style.
- **Threadlocking**: when reviewers say "this is racy," accept; show your synchronization analysis or fix.

62.7 Mistakes that get rejected

- Tabs/spaces wrong; line length wrong.
- Missing `Signed-off-by`.
- Function bigger than necessary; mixing logical changes.
- Not running `checkpatch.pl --strict`.

- Not testing.
- “Fixed warning” when the warning hides a real bug.
- Not handling all error paths.

62.8 An example first patch

```
Subject: [PATCH] drm/amdgpu: log the firmware version we attempted to load

When firmware loading fails during PSP init, we log the path but
not the version we expected.  This makes triage harder when users
report load failures on systems with mixed BIOS/PSP versions.

Add the expected version to the existing dev_err.

Tested on RX 7900 XTX with both correct and intentionally-wrong
firmware copies; in the failure case the new log line allows
quick identification of the version mismatch.

Signed-off-by: <you>
---
 drivers/gpu/drm/amd/amdgpu/psp_v13_0.c | 6 ++++--
 1 file changed, 4 insertions(+), 2 deletions(-)
```

Notice: short, concrete, tested, motivated. That patch will land.

62.9 Long-term contribution rhythm

Set yourself a goal: **one merged patch per month** for six months. Then **one per week**. By month six, you’ll be on first-name basis with the maintainer of your area.

The transformation that happens to you: you read code differently, you suggest patches in your own internal codebase to the same standard, your code review improves dramatically. This is the actual “becoming a kernel engineer” — the upstream loop trains you faster than any internal mentorship.

62.10 Resources

- `Documentation/process/submitting-patches.rst` — the bible.
- `Documentation/process/coding-style.rst`.
- `lwn.net/Articles/` — search for “amdgpu,” learn the ecosystem’s narrative.
- `airlied.blogspot.com` — Dave Airlie’s posts about DRM evolution.
- `phoronix.com` — news about kernel/Mesa changes.
- `freedesktop.org` git repos.

62.11 Exercises

1. Set up `git send-email`. Send a test patch to yourself.
2. Open three pages in browser tabs, then close them: `lwn.net/Kernel`, `lore.kernel.org/amd-gfx`, `gitlab.freedesktop.org/drm/amd/-/issues`. Make this a habit.
3. Pick one open issue with `kind/regression` label. Reproduce. Bisect. Send a `Reported-by`-worthy email even if you can’t fix it.
4. Read every patch on `amd-gfx` for one week. Don’t reply, just observe.

5. Submit one trivial patch this month. Get it merged.

Next: Chapter 63 — The 12-month plan from contractor to FTE.

Chapter 63 — The 12-month plan from contractor to FTE

Concrete, week-by-week. Adjust to your starting level. The goal: by month 12 you have an FTE offer in hand or you're being prepared for one.

Months 1–2: foundations

Goal: write fluent C and confident kernel modules.

- Re-read Parts I–II of this book; do every exercise.
- Build a custom kernel; load `hello.ko`; modify init message in `start_kernel`.
- Implement: arena allocator, free-list allocator, generic linked list with `container_of`.
- Build, run, and load the chardev skeleton (Part VI Chapter 35) on real hardware.
- Pass through 5 of these exercises *each week*: small, real, tracked.

Output: github repo with five small kernel modules, clean READMEs.

Months 3–4: kernel and concurrency

Goal: confident with the kernel's concurrency, memory, and IRQ.

- Re-read Parts III–V; exercises.
- Build the producer/consumer chardev with proper blocking I/O, poll, ioctl, mmap.
- Build the QEMU-`edu` PCI driver from Chapter 37; extend with DMA.
- Read the Linux kernel coding style cover-to-cover. Run `checkpatch.pl` on everything you write.
- **Subscribe to amd-gfx@lists.freedesktop.org**; read patches every day.

Output: a "real" PCI driver with DMA + IRQ + chardev.

Months 5–6: DRM and amdgpu orientation

Goal: read amdgpu source comfortably; submit your first upstream patch.

- Re-read Part VII through Chapter 49; do exercises.
- Read every `amdgpu_*.c` file's intro and one function from each.
- Pick one IP block (e.g. `gfx_v10_0.c`); spend a week reading.
- Build Mesa from source; run `vkcube` and `vulkaninfo` against your build.
- **Submit your first kernel patch upstream.** Even a typo. Get it merged.

Output: one merged patch with your `Signed-off-by`.

Months 7–8: ownership area

Goal: become the local expert on a specific subsystem.

- Choose one: `amdgpu_cs` path, `amdgpu_vm`, reset/recovery, RAS, runtime PM, debugfs.
- Read every line. Take notes. Diagram on whiteboard.
- Find three open issues in that area; reproduce; comment with analysis.
- Pair with a senior engineer; offer to do "boring fixes" they don't want.
- Read every commit to that area for the last 6 months on `git log`.

Output: written internal doc on the subsystem; first non-trivial upstream patch.

Months 9–10: real bug ownership

Goal: end-to-end ownership of one customer-impacting bug.

- Pick an open issue, ideally with multiple comments and no resolution.
- Reproduce locally. Debug systematically. Capture full state on each attempt.
- Communicate: status updates to manager weekly.
- Land the fix upstream.

The first end-to-end bug ownership is the single biggest career step in the first year. It is the proof that you can be trusted with real work.

Months 11–12: convert

Goal: signal FTE-readiness; convert.

- Patches landed: aim for 5+ upstream this year, of varying sizes.
- Review at least 10 patches from peers (publicly, on the list).
- Write one polished internal design doc proposing a feature improvement.
- Have a 1:1 with your manager: "I want FTE conversion. What's needed?"
- Know your salary band and the full comp structure before negotiation.

The conversation with your manager *should not be a surprise* to them. By now you've shipped, you've reviewed, you're known, you've helped. The req is a formality.

A weekly cadence

Every week, even slow weeks:

- **Read patches** (1 hr).
- **Read source** (1-2 hr).
- **Write code** (3-5 hr).
- **Test** (1-2 hr).
- **Review** at least one peer patch.
- **Triage** one bug, even if you don't own it.

That's 8-12 hours of "growth" work. Add to your normal job duties.

On burnout

Driver work can be punishing. Bugs that take weeks. Reset paths that hurt your soul. Reviewers who nitpick. A few rules:

- **Take real weekends**. Reading patches "just for fun" still counts as work.
- **Own one thing at a time** to the depth you can sustain.
- **Rotate areas** when one becomes oppressive.
- **Talk** to your manager when stuck. Better than spinning silently.

On asking for help

When stuck:

- Spend at most **a day** spinning solo.
- Then write a focused question: what you're seeing, what you've tried, what hypotheses you have.
- Send to whoever is the relevant expert (use #channel or person directly).
- Almost always: 80% of help comes from formulating the question well.

On luck

There's a luck component: which team you join, who your manager is, which silicon ships. You can stack the deck:

- **Pick teams** with throughput. (Visible commits, working products.)
- **Pick managers** known for promoting their reports.
- **Pick areas** that matter to the company (not pet projects).
- **Be patient** with the actual hire/promote cycles. Many requisitions are 3-6 months from "approved" to "open."

Final words

Two years from now you should:

- Have ~50 upstream commits.
- Be the person someone else asks "how does X work?"
- Have shipped a real feature in a real release.
- Be on a hiring panel for new hires.
- Be paid well, sleeping well, learning fast.

That's the destination this book aimed at.

You have everything you need. The path is clear. **Now go build.**

End of Part IX. Move to Labs for hands-on exercises, or to Appendix for cheat sheets.

Part X — The Senior / Principal / PMTS Track

> **You are competing with someone who has spent 24 years in this stack.** This part is the map of what they know.

This part is *not* a tutorial. It is a reference. Each chapter is a dense overview of an area where staff- and principal-level engineers spend years. You should:

1. **Read each chapter once for shape** — get the vocabulary, see the diagrams, know the parts.
2. **Pick one or two areas as your specialty** and go deep using the references in each chapter.
3. **Re-visit** the rest as you encounter the topics in your day job.

You will not master Part X in a year. **Nobody masters all of Part X.** The 24-year PMTS you're competing with is *deep* in 4–6 of these areas and *fluent* in the rest. That's the realistic target.

Sections

Hardware foundations (the silicon side)

#	Chapter
64	GPU microarchitecture: RDNA
65	GPU microarchitecture: CDNA & matrix engines
66	Wavefronts, occupancy, VGPRs/SGPRs
67	GPU memory subsystem: HBM, GDDR, controllers
68	GPU cache hierarchy & coherence
69	PM4 packets, command processor, firmware engines

The graphics pipeline, deeply

#	Chapter
70	Geometry pipeline: vertex → tess → geom → mesh shaders
71	Rasterization, hierarchical Z, primitive culling
72	Pixel pipeline: RBE, ROPs, blending, MSAA
73	Ray tracing hardware

Compute, runtime, queues

#	Chapter
74	KFD, HSA, AQL packets, MES
75	CWSR, priorities, preemption
76	ROCm runtime deep

Display, multimedia, power

#	Chapter
77	Display Core (DCN) deep dive
78	Multimedia (VCN), codecs
79	SMU, DPM, voltage/frequency, telemetry

Userspace stack deep

#	Chapter
80	Mesa, radv, ACO, NIR
81	LLVM AMDGPU backend
82	Vulkan & DX12 deep

Driver advanced topics

#	Chapter
83	SR-IOV and GPU virtualization
84	VM faults, page tables, recovery
85	Security: PSP, FW signing, attestation, SEV

Performance & debug at scale

#	Chapter
86	SQTT, RGP, perf counters, RGD
87	Cross-stack debug & regression triage

Engineering at staff/principal level

#	Chapter
88	Customer / ISV / game studio interaction
89	Maintainership and upstream leadership
90	Silicon bring-up, post-silicon validation
91	Cross-vendor architecture: NVIDIA, Apple, ARM, Imagination
92	The PMTS career arc — the 24-year view

How to use this part for FTE conversion

The realistic plan, given you are a fresh contractor:

Year	Goal	Part X focus
Year 0–1	Become a competent junior. Ship 5+ patches.	Read 64–69, 84, 86, 89 once. Don't try to internalize all of it.
Year 1–2	FTE conversion. Specialize.	Pick **one** sub-area (e.g., GPUVM, or RBE, or display) and become *the person* on your team for it. Re-read its Part X chapters monthly.
Year 2–4	Senior engineer.	Add a second specialty. Start reviewing patches. Cover 2–3 areas.
Year 4–8	Staff engineer.	Cross-cut: lead a feature spanning 3+ subsystems. Cover 6–8 areas of Part X.
Year 8+	Principal / PMTS.	Architecture, silicon influence, mentoring. Cover 10+ areas; deep in some you helped *design*.

The 24-year PMTS started exactly the same way. Do not skip steps.

What this part is not

- It's not a substitute for AMD's internal architecture documents (which are NDA-protected and 1000s of pages).
- It's not a tutorial — most chapters have minimal exercises.
- It's not exhaustive — some areas (e.g., specific HW codecs, PSP firmware internals) are intentionally surface-level because the knowledge is gated behind NDA.

It *is* the most useful pre-NDA map of "what you'll be expected to know" that exists outside AMD.

➡️ Start with Chapter 64 — GPU microarchitecture: RDNA.

Chapter 64 — GPU microarchitecture: RDNA

This chapter is the *physical structure* of an AMD graphics GPU at the level a senior engineer must be able to draw on a whiteboard. We focus on **RDNA** (Radeon DNA — RDNA 1/2/3/4 — desktop and mobile graphics). CDNA (compute) is chapter 65.

Top-level block diagram

```
                    +-----------------------------------+
                    |           Host (CPU / PCIe)         |
                    +--+----------------------+-----------+
                       |                      |
                 PCIe TLPs               Doorbells / IRQ
                       v                      ^
                 +----------+           +-----------+
                 |  IH      |<----------|  Engines  |
                 |(Interrupt|           +-----------+
                 | Handler) |
                 +----+-----+
                      |
   +------------------+-----------------------------------+
   |             AMD GPU SoC                               |
   |                                                       |
   |  +-----------+   +-----------+   +-------------+     |
   |  | GFX (CP)  |   | SDMA      |   | VCN (codec) |     |
   |  | + MEC/MES |   | (DMA)     |   |             |     |
   |  +-----+-----+   +-----+-----+   +------+------+     |
   |        |               |                |            |
   |        v               v                v            |
   |  +---------------------------------------------+     |
   |  |     Shader Engines (SE0..SE_n-1)            |     |
   |  |   each containing:                          |     |
   |  |     - Workgroup Processors (WGPs)           |     |
   |  |     - Compute Units (CUs) made of SIMDs     |     |
   |  |     - LDS (Local Data Share)                |     |
   |  |     - L0$ / L1$                             |     |
   |  +---------------------------------------------+     |
   |                       |                               |
   |                       v                               |
   |             +----------------+                        |
   |             |   GL2 / L2$    |                        |
   |             +----------------+                        |
   |                       |                               |
   |                       v                               |
   |             +----------------+                        |
   |             |  GMC (memory   |                        |
   |             |  controller)   |                        |
   |             +----------------+                        |
   |                       |                               |
   +-----------------------+-------------------------------+
                           |
                 +-----------------+
                 |  HBM / GDDR     |
                 |  (VRAM)         |
                 +-----------------+
```

Memorize this. Every conversation at AMD references one of those boxes.

The hierarchy from top to bottom

1. SoC

The "GPU chip" is a system-on-chip with multiple **IP blocks**: GFX (graphics+compute), SDMA (DMA copy engines), VCN (video encode/decode), DCN (display), SMU (power), PSP (security). Each is a separate piece of silicon with its own firmware and its own kernel-driver IP block (`amd_ip_funcs`, see chapter 48).

2. Shader Engine (SE)

A shader engine is a coarse cluster of CUs/WGPs sharing a primitive front-end (geometry/rasterization) and a render back-end (RBE). RDNA chips have 1 to 6+ SEs. Big GPUs have multiple **Shader Arrays (SAs)** per SE; each SA holds a number of WGPs.

3. Workgroup Processor (WGP) — the RDNA innovation

In **GCN** (the older arch) the unit was the **Compute Unit (CU)**. RDNA introduced the **WGP**, which is **two CUs paired** sharing a 128-KB LDS, an instruction cache, a scalar cache, and a texture path. RDNA's "wave32" mode uses the WGP-pair to issue 32 lanes per cycle; legacy "wave64" uses 64-lane execution that splits work across the WGP halves.

(In docs you may see "WGP mode" vs "CU mode"; CU mode treats the two CUs of a WGP as independent units with their own LDS halves. WGP mode pairs them and shares the full 128 KB.)

4. Compute Unit (CU)

Inside a CU, the actual ALUs:

- **2× SIMD32** (or **4× SIMD16** in GCN). Each SIMD32 is a 32-lane vector ALU. It executes 32 work-items in lockstep — a "wave32."
- **1× scalar ALU** for control flow, address calc, branch decisions.
- **VGPR file** (vector general-purpose registers) — typically 1024 VGPRs of 32 bits per SIMD, dynamically allocated per wave (more VGPRs/wave → fewer waves resident → lower occupancy; chapter 66).
- **SGPR file** (scalar registers) — ~108 per wave.
- **Branch/exec mask** — 32-bit (or 64-bit in wave64) execution mask saying which lanes are "alive" through divergent branches.
- **LDS** — fast local memory, 64 KB per CU (or 128 KB shared per WGP).

5. Caches at the CU level

```
L0 instruction $    per WGP   ~32 KB
L0 scalar $         per WGP   ~16 KB
L0 vector / data $  per WGP   ~32 KB     (used to be called "L1$" in GCN)
L1 cache            per SE    ~128 KB    (shared between WGPs in the SE)
L2 / GL2            chip-wide several MiB
L3 / Infinity Cache chip-wide tens-to-hundreds of MiB (RDNA2/3/4 only)
```

The naming changed across generations. Always confirm with the specific chip's whitepaper. The **Infinity Cache** (L3) is a huge SRAM behind L2 to absorb VRAM bandwidth pressure — it's RDNA2's biggest win for high resolutions.

6. Front-end / fixed function

In addition to programmable shaders, each SE has fixed-function blocks:

- **Geometry processor / primitive assembler** — vertex fetch, primitive assembly, primitive culling (small-triangle/zero-area, frustum, backface).
- **Rasterizer** — turns a primitive into pixels.
- **Hierarchical Z (HiZ) / Stencil cull** — early reject before pixel shader.
- **RBE (Render Backend)** — depth/stencil/color writes, blending, MSAA resolve.

We dedicate chapters 70–72 to these.

Wave32 vs wave64 (the RDNA shift)

GCN executed **wave64** (64 lanes per wave) on a 16-lane SIMD over 4 cycles. Worked, but a divergent or short shader wasted lanes.

RDNA introduced **wave32** as the default — a 32-lane SIMD that issues in 1 cycle. Wave64 is still supported (backward compat, some workloads). UMDs (Mesa, DXC, drivers) pick wave size per shader stage based on heuristics.

Implications:

- Compiler (ACO, LLVM) produces different code for wave32 vs wave64.
- Occupancy math (chapter 66) differs.
- LDS bank conflicts differ (chapter 67).

RDNA generations at a glance

Gen	Architecture name	First products	Notable feature
GCN1..5	Southern Islands → Vega	HD 7000 (2012) → Vega (2017)	Wave64 only, 4× SIMD16, GDS
RDNA 1	Navi 10/14	RX 5700/5500 (2019)	First WGP/wave32, no Infinity Cache
RDNA 2	Navi 21/22/23/24	RX 6000 (2020)	Infinity Cache, ray accel units, mesh shaders, VRS
RDNA 3	Navi 31/32/33	RX 7000 (2022)	Chiplet (GCD+MCDs), dual-issue VOPD, AI accelerators
RDNA 4	Navi 4x	RX 9000 (2025)	Improved RT, AI accel, monolithic

You should know the rough release year and one differentiator each. Interviewers ask.

How a triangle becomes pixels — the path through RDNA

1. CP receives draw IB. Vertex shader (VS, or merged “GS-NGG” / “ESGS” pipeline) is dispatched as wave32 across CUs.
2. VS reads vertex buffers via L0/L1/L2 caches; outputs go to LDS or directly to next stage.

3. Primitive assembler combines vertices into triangles. **Primitive culling** discards backfacing/zero-area early (NGG path).
4. Rasterizer emits pixel quads to **PS (pixel shader)** wavefronts.
5. PS waves run on CUs, sample textures via TC paths.
6. PS output → RBE: depth/stencil test, blend, write to color buffer in VRAM.

Every box is a place where a senior engineer has spent months optimizing.

What you should be able to do after this chapter

- Sketch this diagram from memory in 90 seconds.
- Explain the difference between wave32 and wave64 in one sentence.
- Explain what an SE, WGP, CU are and why they exist as separate boxes.
- Name three caches and roughly their sizes.
- Identify, given a workload, which part of the pipeline is likely the bottleneck (geometry-heavy → front-end; high-res → RBE/bandwidth; small shaders → wave occupancy).

Read deeper

- AMD whitepapers for each gen ("RDNA 1 architecture," "RDNA 3 architecture") — public, dense, **read all of them.**
- AMD ISA manuals (RDNA1, RDNA2, RDNA3.5 ISA documents) — the bible.
- ChipsAndCheese articles on RDNA (excellent independent analysis).
- `drivers/gpu/drm/amd/amdgpu/gfx_*.c` — kernel side for each chip family.
- `mesa/src/amd/common/` — userspace knows about the same blocks.

Chapter 65 — GPU microarchitecture: CDNA & matrix engines

CDNA (Compute DNA) is AMD's *compute-only* GPU line — the **Instinct** accelerators. No display, no RBE, no ray tracing units. Everything that gets cut goes back into more compute, more memory bandwidth, and **matrix engines** for AI.

If you work on consumer graphics, CDNA is "the cousin." If you work on data center, AI, or HPC, CDNA is your daily life.

How CDNA differs from RDNA

Feature	RDNA	CDNA
Target	Gaming, content creation, mobile	HPC, AI training/inference
Display block (DCN)	yes	**no**
RBE / blending	yes	**no**
Ray-tracing accel	yes (RDNA2+)	no
Wave size	wave32 default, wave64 supported	wave64 native (uses 4×SIMD16)
Core unit	WGP (paired CUs)	CU
Matrix engine	small AI accel (RDNA3+)	**MFMA / large MFMA** is the marquee feature
Memory	GDDR / Infinity Cache	HBM2/HBM2e/HBM3
Form factor	PCIe card, APU	OAM module, MI series PCIe
Driver path	amdgpu (DRM/GFX rings)	amdgpu + KFD (HSA queues)

CDNA is closer to the GCN ancestor in many ways (wave64, 4×SIMD16) than to RDNA. Mental model: CDNA = "GCN with matrix engines bolted on, and HBM."

CDNA lineage

Gen	Architecture	Products	Headline
Pre-CDNA	Vega 20 / GCN5	MI50, MI60 (2018)	First matrix accel (V_DOT, packed FP16)
CDNA 1	Arcturus	MI100 (2020)	First true matrix engine (MFMA), no display
CDNA 2	Aldebaran	MI200 (MI210/250/250X)	Multi-die (2 GCDs), Infinity Fabric, FP64 matrix

Gen	Architecture	Products	Headline
CDNA 3	Aldebaran successor	MI300 series (MI300X, MI300A)	APU+GPU package, 8-die, HBM3, sparse FP8
CDNA 4+	future	MI350+	even bigger AI focus

The matrix engine: MFMA

The single biggest CDNA feature is the **MFMA (Matrix Fused Multiply-Add)** family of instructions. They compute a small dense matrix product C += A × B in *one* hardware instruction.

```
v_mfma_f32_16x16x16f16   v[acc], v[a], v[b], v[acc]
```

Reads:

- `v[a]` is a 16×16 block of FP16 values (distributed across the wave's lanes).
- `v[b]` is a 16×16 block of FP16.
- Multiplies into a 16×16 FP32 accumulator.
- Adds into `v[acc]` and writes back.

A wave executing this in parallel keeps the matrix multipliers fed for many cycles. **This is the engine of every modern AI workload.**

Variants: different shapes (16×16×16, 32×32×8), data types (FP16, BF16, FP32, FP64, INT8, INT4, FP8 on MI300+), accumulation widths.

You don't write MFMA by hand often. You either:

- Use **rocBLAS / rocSPARSE / rocFFT / MIOpen / hipBLASLt** which ship hand-tuned kernels.
- Use **Composable Kernel** (CK) — AMD's templated C++ library that emits MFMA-laden HIP kernels.
- Write HIP and let the compiler pick MFMA via intrinsics or pattern matching.
- Build a kernel in Triton or PyTorch's compiler, which emit MFMA via ROCm.

Sparse MFMA (CDNA 3)

MI300 introduced 2:4 structured sparsity — if 2 of every 4 weights are zero, the matrix engine can skip them and double effective throughput. Critical for AI inference.

Memory subsystem on CDNA

- **HBM2/2e/3** stacked DRAM, gigabytes-wide. ~1.5 to 5+ TB/s on modern parts.
- **Infinity Fabric** between dies on multi-die parts.
- **xGMI** links between GPUs (8 GPUs per host, all-to-all on flagship platforms).
- No Infinity Cache on CDNA. The HBM bandwidth is high enough.

Multi-GPU is *the* CDNA topic. RCCL (the AMD analog of NCCL) does collectives — all-reduce, broadcast, all-gather — over xGMI rings/trees.

Software differences vs RDNA

KFD path

CDNA workloads almost always go through **KFD** (`/dev/kfd`) and the HSA runtime. User-mode queues, AQL packets, MES scheduling. See chapter 74.

Graphics drivers (`/dev/dri/card*`) are still there but rarely used on a headless MI300.

Compiler

LLVM AMDGPU backend has separate code generation paths for `gfx9*` (CDNA1/2), `gfx940/942` (CDNA 3 family), `gfx1030+` (RDNA2/3). MFMA intrinsics live behind `__builtin_amdgcn_mfma_*`. When you write HIP, the compiler decides; when you write inline GCN/RDNA assembly, you use the actual mnemonic.

Tools

- `rocm-smi` — telemetry (clock, voltage, temp, ECC).
- `rocprofiler-sdk` / `rocprof` — perf counters, kernel traces.
- `omniperf`, `omnitrace` — higher-level analysis tools.
- `rocgdb` — GPU debugging (single-stepping wavefronts!).

We cover these in chapter 86.

Multi-die, chiplets, packaging (CDNA 2 onward)

MI200 was AMD's first multi-die compute GPU: two **GCDs** (Graphics Compute Dies) connected by an Infinity Fabric link. Software sees them either as two separate devices ("HOST" mode) or one logical device with NUMA-style memory access ("XGMI" mode). Both modes have their bug surface area.

MI300X is even more aggressive: 8 compute chiplets ("XCDs") + 4 I/O dies + 8 HBM stacks on one package. MI300A adds a CPU chiplet (Zen 4) sharing the same HBM coherently — a true APU at data-center scale.

The driver implications:

- VRAM is split across HBM stacks; allocator must NUMA-balance.
- Cross-die latency matters; placement affects perf.
- Reset domains: a single XCD can reset, others stay alive (helpful for cluster reliability).

ECC

CDNA always has ECC on HBM and key SRAMs (vs RDNA which often doesn't). The RAS subsystem (chapter 53) reports correctable errors (CE) and uncorrectable errors (UE), with EEPROM-backed bad-page tables.

What you should be able to do after this chapter

- Tell apart RDNA and CDNA in one sentence each.
- Explain what MFMA does and why it dominates AI throughput.

- Sketch an MI300 package: chiplets, HBM stacks, fabric.
- Name three RCCL collectives and roughly when each is used.
- Say what “sparse 2:4” means and which gen introduced it.

Read deeper

- AMD CDNA whitepapers (CDNA1, CDNA2, CDNA3) — public.
- MI300 architecture deck (HotChips 2023 talk is excellent).
- LLVM AMDGPU backend `lib/Target/AMDGPU/AMDGPUGenSubtargetInfo.inc` (autogen) shows feature flags per gfx target.
- Composable Kernel (`github.com/ROCm/composable_kernel`) — read a couple of kernel templates to see how MFMA gets emitted.
- ROCm RCCL repository.

Chapter 66 — Wavefronts, occupancy, VGPRs/SGPRs

The GPU's parallelism unit is the **wavefront** (or "wave"). Mastering occupancy — how many waves are simultaneously resident on the hardware — is the single biggest lever for compute performance. PMTS-level engineers think about this constantly.

What a wave actually is

A **wave** is a group of work-items (lanes) that execute in lockstep on a SIMD. Lockstep means: every lane executes the same instruction at the same time. If lanes diverge (some take a branch, some don't), the SIMD masks off lanes that aren't supposed to execute and runs both sides — the dreaded **divergence cost**.

Architecture	Wave size	SIMD width	Issue cycles
GCN	64	16	4
CDNA 1/2	64	16	4
CDNA 3 (MI300)	64	16	4 (with double-rate ops)
RDNA wave32 (default)	32	32	1
RDNA wave64	64	32	2 (issued back-to-back)

A wave has its own:

- 32-bit (or 64-bit on wave64) **execution mask** (`exec`) — which lanes are alive.
- **PC** (program counter).
- VGPR allocation (private to this wave).
- SGPR allocation (private to this wave).
- A slot in the wave scheduler.

Multiple waves on the same SIMD time-slice. The scheduler picks a ready wave each cycle.

Workgroup → wave mapping

A **workgroup** (compute) or **draw of N pixels** (graphics) is broken into waves by the hardware:

- Workgroup size 256, wave32 → 8 waves.
- Workgroup size 256, wave64 → 4 waves.
- Workgroup size 64, wave64 → 1 wave (this is the GCN-classic case).

Workgroup-level shared memory (LDS) is shared by all waves of the workgroup that land in the same CU/WGP. Synchronization between waves of a workgroup uses LDS barriers (`s_barrier`).

Occupancy

Occupancy is the number of waves resident on each SIMD (or each WGP / CU, depending on convention). Higher occupancy = the scheduler has more work to hide latency with.

```
Latency hiding analogy:
- 1 wave on the SIMD, it's waiting for memory → SIMD idles.
- 8 waves on the SIMD, one waits for memory → scheduler runs another.
```

What limits occupancy

A wave occupies *all* of:

1. **VGPRs** — RDNA SIMDs have ~1024 VGPRs each. If a shader uses 80 VGPRs/wave, max waves/SIMD = 1024/80 ≈ 12. The compiler quantizes VGPR counts (e.g., to multiples of 4).
2. **SGPRs** — usually less of a bottleneck; ~108 per wave; the file holds many.
3. **LDS** — per workgroup. If you allocate 32 KB LDS, only 2 workgroups can be resident in a 64 KB CU.
4. **Wave slots** — the scheduler has a hard cap on simultaneous waves (e.g., 16 or 20 per SIMD).
5. **Workgroup slots** — a separate cap on number of distinct workgroups.

The actual occupancy is the **minimum** of all these limits.

Compiler reports occupancy

When you compile a HIP kernel:

```
hipcc -save-temps -O3 my_kernel.hip
# look at *.s file, see comments like:
;   VGPRs: 80
;   SGPRs: 32
;   ScratchSize: 0
;   Occupancy: 8
```

Or use `--save-temps` plus `roc-obj-ls` / `roc-obj-extract` to inspect the HSACO. RGP (chapter 86) shows occupancy per-wave per-time.

Spilling (the enemy)

If a shader exceeds the architectural VGPR limit (e.g., wants 300 VGPRs, max is 256), the compiler **spills** registers to **scratch memory** (a per-wave region in VRAM, accessed via the scratch path). Every spill = a DRAM round-trip = horrible.

You'll see this as `ScratchSize` > 0 in compile output.

Mitigations (in order of preference):

1. **Reduce live ranges** — restructure code so fewer values are live simultaneously.
2. **Wave64** — twice the registers per work-item (since the wave is bigger).
3. **Reduce inlining** — sometimes hurts perf elsewhere; profile.
4. **Bigger workgroup, smaller per-thread state**.

A senior engineer reads HSACO and counts "live VGPRs" by hand to find spill culprits.

Divergence

Different lanes taking different branches:

```
if (lane_id < 16) {
    do_thing_a();
} else {
    do_thing_b();
}
```

The wave executes `do_thing_a` with half the lanes masked off, then `do_thing_b` with the other half masked off. **Cost ≈ both arms.** When divergence happens at every iteration, you've serialized the wave.

Compiler/runtime tricks:

- Reorder data so adjacent lanes take the same branch.
- Sort work by branch type before dispatch.
- Use predication for tiny branches (faster than masking).

Wave-level intrinsics

Modern shaders use intrinsics that reach across lanes within a wave:

- `__shfl`, `__shfl_xor`, `__shfl_up`, `__shfl_down` — read another lane's value.
- `__ballot` — broadcast a per-lane bool to a 32/64-bit mask.
- `__any`, `__all` — quick reductions.
- DPP (Data-Parallel Primitives) instructions — fast lane permutations.

These exist because the hardware can copy values between lanes within the SIMD without going through LDS or memory. Use them for wave reductions, prefix sums, sorting.

RDNA's dual-issue (RDNA 3+)

RDNA 3 added **VOPD** (vector op dual-issue): one instruction encodes two scalar-add or vector-mul-add type ops that execute on the same cycle. Compiler must pair eligible ops; you can hit ~2× throughput on float-heavy kernels. Doesn't apply to MFMA (CDNA-only).

CDNA's MFMA pipeline

On CDNA, MFMA instructions occupy the matrix engine for many cycles. While MFMA is running, the SIMD can issue other vector ops (the matrix unit and SIMD are partially decoupled). The compiler interleaves MFMA with VGPR-to-VGPR shuffling and address computation to keep both fed. Hand-written kernels exploit this by software-pipelining loads, MFMAs, and stores.

Practical occupancy tuning

Symptom	Likely cause	Fix
Low utilization, high cache hit rate	Memory-bound, not occupancy-bound	Coalesce loads, reduce traffic

Symptom	Likely cause	Fix
Scratch > 0	Spilling	Reduce live VGPRs
Many waves but slow	Divergence	Reorder data
Few waves and slow	Low occupancy	Reduce VGPR/LDS use
LDS-bound	Bank conflicts	Pad arrays, swizzle indices

A good debugging approach:

1. Look at occupancy in RGP / `rocprof`.
2. Check spill / scratch.
3. Look at wave issue stalls per cycle (HW counters).
4. Look at L2 hit rate vs HBM bandwidth.
5. Hypothesize, change *one* thing, measure.

What the kernel driver has to know

- VGPR/SGPR/LDS limits per chip family — used to validate shader compiles.
- Scratch buffer allocation — per-wave private memory in VRAM.
- Reset/recovery behavior on hung waves (chapter 84).
- Per-context priority (graphics vs compute vs realtime) so a high-priority wave can preempt.

The driver does not micro-manage occupancy. The compiler decides VGPR count; the driver allocates scratch and validates.

What you should be able to do after this chapter

- Define wave, workgroup, occupancy.
- Explain why high VGPR usage hurts performance.
- Identify spilling from a `.s` listing.
- Know roughly how to read RGP's occupancy view.
- Reason about divergence cost.

Read deeper

- RDNA 3 ISA manual, §3 "Wavefront execution" and §6 "VGPRs/SGPRs."
- AMD blog: "RGP profiling for game developers" — shows occupancy and stalls.
- LLVM `AMDGPUSubtarget` & `SIRegisterInfo` — VGPR allocation logic.
- Mesa ACO: `aco_register_allocation.cpp` — independent allocator that often beats LLVM on wave32.
- `rocm/HIP-Examples` — kernels with comments about occupancy.

Chapter 67 — GPU memory subsystem: HBM, GDDR, controllers, coalescing

The GPU is a parallel arithmetic engine bolted to a *very* aggressive memory system. Bandwidth, not compute, is the bottleneck for most real workloads. This chapter is the memory side.

Memory types

Type	Where it lives	Role
VGPRs	inside SIMD	per-lane fastest
SGPRs	inside CU	per-wave scalar
LDS	per CU/WGP	workgroup-shared scratch (“shared memory” in CUDA terms)
GDS	chip-wide	global atomics, less used in modern UMDs
L0 / L1 / L2	per-WGP / per-SE / chip	caches
Infinity Cache (L3)	chip	huge SRAM behind L2 (RDNA2+ desktop)
VRAM	HBM stacks or GDDR chips	gigabytes of off-chip DRAM
System RAM (via GTT)	host DRAM	mapped to GPU through IOMMU/GART

A wave can address all of those, but with very different latencies and bandwidths.

VRAM: HBM vs GDDR

	GDDR (5/5X/6/6X/7)	HBM (2/2e/3/3e)
Form factor	individual DRAM chips around the GPU on the PCB	stacked die on the same package as the GPU
Per-pin speed	very high (e.g., 21–32 Gbps GDDR6/6X/7)	medium (~3–6 Gbps)
Channel width	narrow (32-bit per chip, 256–384 bits aggregate)	very wide (1024 bits per stack)
Bandwidth	hundreds of GB/s	up to several TB/s
Power per bit	higher	lower
Cost	cheaper	expensive
Used in	RDNA gaming GPUs, APUs	CDNA, MI250/MI300, datacenter

Why both exist: gaming workloads tolerate higher latency for cheaper memory. AI/HPC needs sustained TB/s, so the $$$ HBM stacks are worth it. RDNA2/3 high-end uses **Infinity Cache** as an SRAM staircase to bridge GDDR's bandwidth gap.

HBM packaging

HBM stacks contain 4–12 DRAM dies vertically connected by **TSVs** (through-silicon vias) and connected to the GPU through a **silicon interposer** (or **CoWoS** at TSMC). The interposer carries the wide buses without needing PCB-level routing. This is why HBM is "expensive": you're paying for advanced packaging.

GDDR particulars

- GDDR6X / GDDR7 uses PAM3 / PAM4 signaling (multi-level voltages per cycle).
- Memory controller has to handle high-frequency training, refresh, ZQ calibration.
- Power consumption is dominated by I/O at high speeds.

Memory controllers

The GPU has multiple **memory channels** (HBM stack channels or GDDR chip channels). Each channel has its own controller. Together they form the **GMC (Graphics Memory Controller)**.

The kernel driver `amdgpu_gmc` handles:

- aperture programming (where in the GPU's address space VRAM and GTT live),
- VM page-table walker config,
- ECC scrubber init,
- bad-page handling.

A typical config:

```
VRAM aperture: 0x000000_00000000 .. 0x0000FF_FFFFFFFF   (16 TB virtual, mostly unmapped)
GTT  aperture: 0x010000_00000000 .. 0x01FFFF_FFFFFFFF
MMIO aperture: ...
```

GPUs have a **48-bit virtual address space** for the shader VM. CPU and GPU virtuals are *separate* unless using SVM / ATS (chapter 51, 75).

Coalescing — the most important access pattern

When a wave issues a load `ld a[lane_id]`, every lane in the wave generates one address. Hardware **coalesces** them: if all 32 (or 64) addresses fall into a single cache line, it issues one cache-line read. If they're scattered, it issues many.

```
Coalesced (good):
  lane 0 reads &a[0]      ┐
  lane 1 reads &a[1]      ├ all in one 64-byte cache line, 1 transaction
  ...                     │
  lane 31 reads &a[31]    ┘

Scattered (bad):
  lane 0 reads &a[0]      ┐
  lane 1 reads &a[1024]   ├ 32 different cache lines, 32 transactions
  lane 2 reads &a[2048]   │
  ...
```

Strided access by 1 element of the right size = good. Random access = order of magnitude slower. AOS vs SOA layouts matter enormously here.

Bank conflicts in LDS

LDS is divided into **banks** (typically 32 banks in RDNA, 4-byte wide each). Two lanes that hit the same bank with different addresses serialize. Memorize:

```
addr % 32  →  bank
```

If `lane_id` reads `lds[lane_id * 4]`, that hits bank 0 from every lane → 32-way conflict. Use `lds[lane_id * 4 + (lane_id / 4)]` or pad to break the stride.

This is the single biggest LDS performance trap; senior engineers profile bank conflict rates routinely.

Address translation on the GPU

The GPU has its own MMU. Every shader memory access is a GPU virtual address. Translation:

```
GPU VA  →  GPUVM page tables  →  device physical (VRAM) or system PA (via IOMMU)
```

The page tables have several levels (PD0/PD1/PT in AMD terms). Walked by hardware. The driver builds and updates them via SDMA writes (chapter 51, 84).

TLB caches translations; flush is needed when mappings change. amdgpu issues TLB flushes as part of fence completion when VM pages are remapped.

Cache coherence on the GPU

GPUs are *not* fully coherent the way CPUs are. Within one wave, after a write, you must explicitly invalidate or flush caches before another wave (or block) reads. Compute often uses:

- `s_waitcnt vmcnt(0) lgkmcnt(0)` — wait for outstanding memory ops.
- `buffer_wbinvl1` — write-back & invalidate L1 (varies per gen).
- `s_dcache_wb` — flush scalar cache.

The compiler emits these around atomics, barriers, kernel boundaries, and explicit `__threadfence` calls.

CPU↔GPU coherence:

- "Coherent" mappings (host-allocated, mapped with HSA_AMD_MEMORY_POOL_PINNED_FLAG_*) — slower but no manual flush.
- "Write-combined" mappings — burst writes only; reads are very slow.
- "Uncached" — for MMIO-like access.

The kernel driver picks the right pgprot/cache mode in `mmap` callbacks.

Bandwidth math

Always do this back-of-envelope before optimizing.

```
Effective BW needed = (input bytes + output bytes) per kernel * launch frequency
Available BW       = HBM/GDDR peak * efficiency (50-80% real-world)
```

```
If needed >> available → memory-bound. Compute changes don't help.
If needed << available → compute-bound. Adding waves / better ALU code helps.
```

For an MI300X with ~5 TB/s HBM, a kernel that touches 50 GB once per call takes ≥10 ms just to move bytes — irreducible.

VRAM layout decisions made by the kernel

- **VRAM (local memory)** — fastest GPU access; CPU has slower BAR-mapped access.
- **GTT (system memory mapped through IOMMU)** — CPU-fast, GPU has to round-trip over PCIe.
- **VRAM with CPU-write-combined coherent map** — for upload streams.
- **Visible PCI BAR** — historically limited to 256 MB; **Resizable BAR (rebar)** lifts this so the CPU can directly address all of VRAM. Modern systems should have rebar enabled.

Allocator (TTM, GEM, see chapters 45–46) picks placement based on `domain` flags.

Memory hot paths a senior engineer cares about

1. **Texture sampling** — high-throughput TC/TCC path with format conversion + filtering.
2. **Image stores** — coalesced or atomic; impacts RBE bandwidth.
3. **Buffer atomics** — global atomic to L2 or to memory; serialization point.
4. **LDS reductions** — bank-aware indexing for parallel reductions.
5. **Streaming loads** — bypass caches with `glc` (NOLD/streaming) bits to avoid pollution.

What you should be able to do after this chapter

- Sketch the memory hierarchy with rough sizes and bandwidths.
- Explain why HBM is more expensive but worth it for AI.
- Define coalescing and bank conflicts.
- Reason about whether a kernel is memory-bound vs compute-bound.
- Know the difference between VRAM, GTT, and resizable BAR.

Read deeper

- AMD CDNA 3 architecture (HotChips slides) — best memory subsystem diagrams.
- HBM3 standard (JESD238) — terse but the source of truth.
- `drivers/gpu/drm/amd/amdgpu/gmc_v*.c` — GMC IP version for each chip.
- `drivers/gpu/drm/ttm/ttm_resource.c` — placement logic.
- ChipsAndCheese articles on caches/L2/Infinity Cache.

Chapter 68 — GPU cache hierarchy & coherence

GPUs and CPUs share concepts (L1, L2, lines, hit rate) but their cache *semantics* are different. Misunderstanding this is the source of most GPU memory bugs.

The hierarchy on RDNA (gen-typical, varies)

```
                    [VGPRs / SGPRs]                  per-lane / per-wave
                           |
                  [L0 vector $ ~32 KB]               per WGP
                  [L0 scalar $ ~16 KB]               per WGP
                  [L0 instr  $ ~32 KB]               per WGP
                           |
                   [L1 $ ~128 KB]                    per Shader Engine
                           |
                   [L2 / GL2 several MiB]            chip-wide
                           |
              [Infinity Cache (L3) tens-hundreds MiB] (RDNA2+ desktop)
                           |
                       [VRAM]
```

Naming has shifted across gens (the old “L1” became the new “L0”, the old “GL2” became “L2”). Whitepaper-cite the specific gen when you discuss; don’t guess.

On CDNA

```
[VGPRs] / [SGPRs] / [LDS 64-128KB per CU]
         |
   [L1 vector $] per CU       (smaller than RDNA's WGP L0)
         |
   [L2 $] chip-wide, large    (no Infinity Cache, but big L2)
         |
       [HBM]
```

CDNA leans more on raw HBM bandwidth, less on cache layers.

Cache-line size

Most AMD GPU caches use **64-byte** lines (a wave32 with each lane reading a `u16` = 64 bytes total — matches perfectly). CDNA may have wider effective fetch granularity due to the 64-lane wave. Always 64 B unless the spec says otherwise; design data layouts around it.

Coherence: the GPU model

CPUs have **MESI-style coherence**: every cache snoops every other, writes propagate transparently. GPUs **do not** scale this — too many cache instances, too high cost. Instead:

- Within a single wave/CU: cache is coherent with itself.

- Between waves on the same CU/SE: depends on level and gen; often coherent at the L1/L2 boundary with explicit instructions.
- Between waves across the chip: not automatically coherent; **explicit invalidate / write-back instructions** required.
- Between GPU and CPU: zero coherence by default; must use coherent mappings or flush.

Translated to programmer-visible rules:

1. After a global write, before another wave reads the same address, you must:
   - On many AMD gens: write-back the producer's L1 (`buffer_wbinvl1`) **and** invalidate the consumer's L1.
   - Atomics handle this implicitly (they go to L2 or VRAM).
2. `memory_scope_workgroup` — flush only enough to keep the workgroup consistent (cheaper).
3. `memory_scope_device` — flush so any wave on this device sees it.
4. `memory_scope_system` — flush so the CPU sees it (slow; only for completion fences).

The compiler emits the right wait/flush combos based on the C++ atomic memory orders or HIP `__threadfence_*`. As a driver dev, you must understand what's emitted.

Wait counters and barriers

GPU memory ops are *issued* but their completion is asynchronous. The wave keeps a few counters:

- `vmcnt` — count of outstanding vector memory ops (loads/stores).
- `lgkmcnt` — outstanding LDS / GDS / scalar / constant-mem ops.
- `expcnt` — outstanding export (output) ops.

Synchronizing is `s_waitcnt vmcnt(0)` etc. — wait until that counter reaches the given value. The compiler inserts these.

Workgroup barriers (`s_barrier`) ensure all waves of a workgroup reach a point and that LDS writes are visible. They include the right `s_waitcnt`.

Caching policy — `glc`, `slc`, `dlc`

Memory ops on RDNA carry policy bits:

- **glc (globally coherent)** — bypass L0/L1, go to L2; for inter-CU communication.
- **slc (system-level coherent)** — bypass L2; for CPU-coherent paths.
- **dlc (device-level coherent / disable L1)** — RDNA1+ extra knob.

The compiler emits these per access. For example, atomics use `glc`. Streaming loads (won't be re-read) might use `slc` to avoid polluting L2. RDNA3 adds even finer policy control.

A senior engineer reads HSACO and notes the policy bits; wrong policy can cause subtle correctness bugs (e.g., a producer wave's writes not visible because the consumer reads from a stale L1).

Texture cache (TC / TCC / TCP)

Reads through the texture path go via a separate cache — TCP/TCC — optimized for 2D locality and format conversion. Image-typed reads benefit from TCC's 2D-aware addressing (Z-order/Morton ordering on the lines). Buffer reads use the regular vector $ path.

For graphics: textures live in TCC; render targets are written through RBE which has its own color cache (CB) and depth cache (DB) — chapter 72.

Cache misses and how to fix them

Common patterns:

Pattern	Cause	Fix
Strided / scattered read	Non-coalesced or non-2D-local layout	Restructure data; transpose; tile
Read-then-write same line many times	False sharing / atomics on adjacent words	Pad to line; use private accumulator
Double-buffered streaming	Producer/consumer on same data	Use `glc` and explicit wait
Texture cache thrash	Working set > TCC	Tile/swizzle textures; reduce footprint

Inspecting cache behavior

- **rocprof / omniperf** counters: `L1_HIT_RATE`, `L2_HIT_RATE`, `TCC_MISS`, `TCC_HIT`.
- **RGP** for graphics: shows shader memory stalls, cache hit graphs.
- **umr** can dump some counters live.
- Microbenchmark with `streamBench` / `babelstream` to know your peak.

Coherence with the CPU (for SVM / ATS)

When SVM (Shared Virtual Memory) is in play, GPU and CPU share an address space with hardware coherence between IOMMU-backed pages and GPU caches. Implementation:

- IOMMU translates GPU device VAs to host PAs.
- Coherent fabric links GPU caches to CPU cache hierarchy (Infinity Fabric on AMD APUs, NVLink/CCIX/CXL on others).
- Atomics with `system` scope use this path.

MI300A (CDNA3 APU) is the most coherent today — Zen 4 CPU and CDNA GPU share HBM with hardware coherence; you can pass pointers freely.

For discrete GPUs over PCIe, "coherence" usually means *managed* coherence: explicit fences and flushes hidden behind a runtime API.

Implications for the driver

The kernel driver:

- Sets up MTYPE bits in page-table entries that select cache policy per-page (CDNA has many MTYPEs).

- Programs shader engine cache config registers (cache size partitions, prefetch policies).
- Issues blanket cache flushes during reset or page-table changes.
- Picks pgprot for `mmap` carefully (write-combined for streaming, cached for read-mostly host buffers).

`amdgpu_vm_pte.c` and `amdgpu_gmc_*.c` are the relevant files.

What you should be able to do after this chapter

- Draw the cache hierarchy of a recent RDNA chip.
- Explain why GPUs aren't fully MESI-coherent.
- Use vmcnt/lgkmcnt vocabulary in conversation.
- Reason about glc/slc/dlc bits.
- Know what 2:4 sparse and `MTYPE` mean.

Read deeper

- AMD ISA manuals (RDNA2/3, CDNA3) — especially sections on memory operations.
- "Atomics on GPUs" papers (any year).
- `drivers/gpu/drm/amd/amdgpu/gmc_v*.c` — MTYPE programming.
- LLVM AMDGPU `SIMemoryLegalizer.cpp` — emits the wait/flush sequences. **Read this file.** It is the operational definition of the GPU memory model.
- `mesa/src/amd/common/ac_nir_lower_global_access.c` — Mesa's similar lowering.

Chapter 69 — PM4 packets, command processor, firmware engines

The kernel doesn't actually drive the GPU's pixel pipeline. **Firmware does.** Specifically a small army of microcontrollers running AMD-signed firmware, communicating with the host through structured packet streams called **PM4**.

Understanding PM4 + the CP firmware is the *real* difference between a junior driver dev and a senior one. This chapter is the bridge.

The firmware engines (one chip, many CPUs)

A modern AMD GPU contains, in addition to the shader cores:

Engine	Role	Firmware filename pattern
CP / ME (Command processor / Micro Engine)	parses PM4, drives GFX ring	`*_me.bin`
PFP (Pre-Fetch Parser)	fetches/decodes PM4 ahead	`*_pfp.bin`
CE (Constant Engine, GCN)	older constant-buffer path; gone in RDNA	`*_ce.bin`
MEC (Micro Engine Compute)	drives compute rings	`*_mec.bin`, `*_mec2.bin`
MES (Micro Engine Scheduler)	newer scheduler engine, manages user queues	`*_mes.bin`, `*_mes_2.bin`
RLC (Run List Controller)	power gating, clock gating, save/restore	`*_rlc.bin`
SDMA	DMA engines, also FW-driven	`*_sdma.bin`
VCN	video codec engines	`*_vcn.bin`
PSP	security boot, FW signing	`*_sos.bin`, `*_ta.bin`
SMU	power/clock controller	`*_smc.bin`, `*_pp_table.bin`
DMCUB (DCN)	display microcontroller	`*_dmcub.bin`

You can list yours:

```
ls /lib/firmware/amdgpu/
```

PSP loads, verifies signatures on, and brings up all the others. If PSP says no, the GPU doesn't run.

What PM4 is

PM4 is a **packetized command format** parsed by ME/PFP/MEC. A PM4 packet is a 32-bit header followed by N data dwords:

```
 31         16 15  8  7   0
+-------------+-----+------+
| length-1    | OP  | type |   header
+-------------+-----+------+
| dword 1     |
+-------------+
| dword 2     |
+-------------+
...
+-------------+
| dword N     |
+-------------+
```

`type=3` is the only one used in modern flows (PM4 type-3). The opcode (`OP`) selects the packet. There are 100+ packet types; you'll meet a few constantly.

Most common PM4 packets

Packet	What it does
`PACKET3_NOP`	filler
`PACKET3_INDIRECT_BUFFER`	"switch to this IB and execute it" — chains UMD work into the ring
`PACKET3_WRITE_DATA`	write N dwords to memory or registers
`PACKET3_COPY_DATA`	copy a register to memory or vice versa
`PACKET3_WAIT_REG_MEM`	wait until a memory/register value matches a condition (polling)
`PACKET3_RELEASE_MEM`	post a fence: flush caches, then write a value to memory
`PACKET3_ACQUIRE_MEM`	invalidate caches before reading
`PACKET3_DRAW_INDEX_*` , `PACKET3_DRAW_INDIRECT`	submit draws
`PACKET3_DISPATCH_DIRECT`	submit a compute dispatch
`PACKET3_SET_*_REG`	program register state (CONTEXT, SH, UCONFIG, CONFIG groups)
`PACKET3_EVENT_WRITE_*`	issue events (cache flush, pipeline flush, end-of-pipe)
`PACKET3_PREEMPT`	request mid-task preemption
`PACKET3_COND_EXEC`	execute next packet only if a condition is true

You will read PM4 streams when debugging driver bugs. Tools to do so:

- `umr` (`umr -RS gfx_0.0.0`) can dump ring contents.
- Mesa has trace replay (`AMD_DEBUG=hang`) which dumps the IB on hang.
- `amdgpu_ring` debugfs files.

A senior engineer reads PM4 like English.

The command processor pipeline

When the kernel writes a PM4 stream into a GFX ring buffer and rings the doorbell:

```
1. CP wakes up (was idle since last work).
2. PFP fetches packets ahead of ME.
3. ME executes packets:
     - SET_CONFIG_REG → writes a register
     - WAIT_REG_MEM → CP polls until condition
```

```
      - RELEASE_MEM → CP issues an EOP event, which after pipeline flush writes a fence value
      - INDIRECT_BUFFER → CP jumps into UMD-emitted IB
4. UMD IB has draws/dispatches → CP launches waves on shader engines.
5. End of UMD IB → return to kernel ring.
6. Kernel emits a final RELEASE_MEM that signals a fence in memory.
7. CP raises an interrupt (IH path).
```

The kernel's role is *mostly*: build small "trampoline" PM4 sequences that bracket UMD's work — set up VMID, set up some context state, jump into UMD IB, emit a fence, raise an IRQ. UMD does all the actual draws.

How the kernel emits PM4 (in `amdgpu`)

Each ring type has a `funcs` table:

```
struct amdgpu_ring_funcs {
    void (*emit_ib)(struct amdgpu_ring *ring, struct amdgpu_job *job, ...);
    void (*emit_fence)(struct amdgpu_ring *ring, u64 addr, u64 seq, unsigned flags);
    void (*emit_pipeline_sync)(struct amdgpu_ring *ring);
    void (*emit_vm_flush)(struct amdgpu_ring *ring, unsigned vmid, u64 pd_addr);
    void (*emit_reg_wait)(struct amdgpu_ring *ring, u32 reg, u32 val, u32 mask);
    /* lots more */
};
```

Per-IP files (`gfx_v10_0.c`, `gfx_v11_0.c`, ...) provide implementations that emit PM4 dwords with `amdgpu_ring_write(ring, dw)`. Reading these tells you exactly what the kernel pushes for each operation.

MES — the next-generation scheduler

Older AMD GPUs did all queue management in the kernel: the kernel mapped one global GFX ring and a few KIQ/MEC compute rings, and sequentially streamed PM4 into them.

With **MES** (introduced gradually, GFX10 onward), the firmware itself runs a scheduler. User-mode queues (UMQs) become real: each ROCm process can have its own ring, doorbell, and queue, and MES schedules among them in firmware. KFD (`amdkfd`) heavily relies on MES for compute queue scheduling.

Implications:

- The host can submit through user queues (no syscall per submit) — huge for low-latency compute.
- Reset and recovery becomes more nuanced (MES-managed vs host-managed queues).
- The kernel's role shifts toward setup + reset, not constant scheduling.

We treat MES + user queues in chapter 74.

RLC — the unsung hero

The RLC handles **power and clock gating**: dynamically turns off CUs/SEs when idle. It also handles **CWSR** (Compute Wave Save/Restore — chapter 75) which saves wave state when a higher-priority job preempts. Without RLC, no power efficiency.

The kernel programs RLC tables for:

- which CUs to gate when load drops,
- save/restore image layout for CWSR,

- microcode version expectations.

If RLC firmware is wrong, GPU hangs in non-obvious ways during idle transitions.

PSP boot order

```
1. PSP ROM (silicon ROM) verifies and runs PSP SOS firmware from ROM/flash.
2. SOS verifies and loads:
   - SMU firmware
   - GFX firmware (PFP/ME/MEC/MES)
   - SDMA firmware
   - VCN firmware
   - DCN/DMCUB firmware
3. SOS may load Trusted Applications (TAs) for content protection (HDCP, ASD).
4. SOS hands control: SMU brings clocks online; GFX waits for kernel.
```

If you see “PSP failed to load” in `dmesg`, the GPU is dead until reboot. Failure modes: corrupt firmware blob, signature mismatch, broken PCIe link.

Versioning and pain

Every chip family ships its own firmware versions. The kernel has tables of “minimum acceptable version” and refuses to bind if the system firmware is older. Production kernels fight this by either bundling FW with the kernel package or upstreaming compatibility checks.

`/sys/kernel/debug/dri/0/amdgpu_firmware_info` lists current versions.

Attack surface

Firmware is the GPU’s TCB (trusted computing base). PSP signing prevents tampered firmware. RAS handles SRAM/cache ECC. SR-IOV virtualization adds a separate trust boundary (chapter 83, 85).

A recent class of GPU exploits goes through tricking the kernel into corrupting PM4 streams — the firmware doesn’t fully validate every field, and a cleverly crafted submission can read other VMID’s pages, etc. Mitigations live in the CS validation path (`amdgpu_cs.c`).

What you should be able to do after this chapter

- Name the firmware engines and what each does.
- Explain a PM4 type-3 packet structure.
- Read a PM4 dump and identify draws / fences / VM flushes.
- Describe what MES changed about queue scheduling.
- Trace from kernel `amdgpu_ring_commit` to silicon execution.

Read deeper

- AMD `pm4` headers in the kernel: `drivers/gpu/drm/amd/include/asic_reg/gfx_*/gfx_*_offset.h` and `pm4_packet*.h`.
- `drivers/gpu/drm/amd/amdgpu/gfx_v11_0.c` for a recent-gen ring funcs implementation.
- `drivers/gpu/drm/amd/amdgpu/mes_v11_0.c` for MES setup.

- `drivers/gpu/drm/amd/amdgpu/psp_*.c` for PSP boot sequencing.
- `umr` source: https://gitlab.freedesktop.org/tomstdenis/umr — has packet decoders.
- AMD GPUOpen blog posts on MES.

Chapter 70 — Geometry pipeline: vertex → tess → geom → mesh shaders

The “front end” of the graphics pipeline takes vertex data from buffers and emits triangles ready for rasterization. It is hardware + programmable shaders + a *lot* of API politics.

The classic pipeline (DX10/OpenGL 3 era)

```
Vertex Shader (VS)
        │
   Hull Shader (HS) ──┐
        │             │
  Tessellator (FF)    │ tess optional
        │             │
  Domain Shader (DS) ─┘
        │
 Geometry Shader (GS) (optional, slow on AMD)
        │
   Primitive Assembly + Clipping/Culling
        │
       Rasterizer
```

Five programmable stages plus fixed-function tessellation. AMD GPUs *have* hardware for all of these but for performance they’re collapsed.

How AMD actually maps stages — the merged-shader pipeline

Modern AMD HW has **two** dedicated pipeline slots between primitive setup and rasterization:

- **HS (Hull Shader)** stage — runs on the geometry processor.
- **GS (Geometry Shader)** stage — also where vertex shading lands.

The driver/compiler **merges** API stages into these slots:

API stages used	Mapped to AMD HW
VS only	HS=skipped, GS=VS-as-GS (“ESGS” / “NGG” path)
VS + HS + DS	HS=HS, GS=DS-as-GS
VS + HS + DS + GS	HS=HS, GS=GS (worst case)
Mesh shader	special “NGG mesh” path

This merging is invisible to the application but the driver and compiler do real work to make it happen — see `mesa/src/amd/common/ac_nir_lower_*` and the radv stage merging logic.

NGG (Next-Generation Geometry) — RDNA's big change

NGG, introduced with Vega and matured in RDNA, replaces the legacy ESGS path. Key idea: the hardware no longer needs a separate "ES" stage in front of GS; vertex shading and primitive culling happen together in NGG-format wavefronts that emit position+attributes for **culled** primitives only.

Wins:

- Primitive culling on-chip before any rasterization work.
- Better wave utilization.
- Native support for **mesh shaders** (a more flexible front-end).

NGG is the only path on RDNA2+. Mesa's `radv` and AMD's PAL both target NGG.

Mesh shaders

Mesh shaders (DX12/Vulkan VK_EXT_mesh_shader) replace the entire VS/HS/DS/GS chain with two stages:

- **Task shader** (optional): emits work for downstream mesh shaders, can amplify work.
- **Mesh shader**: outputs vertices and primitives directly, with explicit primitive connectivity, in a compute-shader-like programming model.

Why it's a big deal:

- App controls the topology (no API limits like GS amplification cap).
- LOD, culling, and small-triangle reject can be done per-meshlet.
- Better fits modern rendering (Nanite-style virtualized geometry).

AMD has supported mesh shaders since RDNA2 (DXR 1.0 era) via NGG.

Primitive culling

Several culls happen before rasterization:

1. **Frustum cull** — primitive entirely outside view frustum.
2. **Backface cull** — primitive facing away (CW/CCW depending on state).
3. **Small-primitive cull** — sub-pixel triangles that won't generate any pixels.
4. **Zero-area cull** — degenerate.
5. **Cluster culling** (modern, app-controlled) — entire meshlet cull.

These happen on the geometry processor; rejected primitives never enter the rasterizer. NGG made them efficient by doing them on the same wave that did vertex shading.

Index/vertex fetch

Vertex inputs come from buffers in VRAM. Two paths:

- **Index buffer + vertex buffer**: the geometry processor fetches indices, then issues vertex fetches. Cache-friendly with an indexed cache (PA_PRIM cache).
- **Direct draw**: fetches vertices in order.

Vertex format conversion (e.g., `R8G8B8_UNORM` → `vec4`) happens either in fixed function (legacy) or in the VS (modern, generated by the compiler).

Tessellation (HS + Tess + DS)

DX11-era feature. Hull shader emits per-patch and per-control-point data and tessellation factors. The tessellator (fixed function) generates new vertices according to the factors. The domain shader runs per generated vertex.

Use cases: terrain, character LOD, smoother surfaces. In practice, tessellation has fallen out of fashion vs mesh shaders + virtual geometry. Most AAA engines now use mesh-shader pipelines.

Geometry shader — the cautionary tale

GS lets you read a primitive and emit zero or more primitives. It looked great on paper but turned out to be **slow on AMD** (and most vendors): the variable output rate forces serialization. NGG path tries to avoid GS when possible, mapping VS/DS-only chains to non-GS hardware.

Avoid GS if you control the renderer.

Position-only / streamout

Some setups fetch only position attributes for shadow/depth-only passes. AMD supports a "position-only" wave shape that skips fetching unused attributes. Saves bandwidth and BW-bound stages.

Streamout writes vertex shader outputs back to a buffer (transform feedback). RDNA implements via direct memory writes from the geometry pipeline. Performance depends on coalescing.

Driver responsibilities

The kernel driver doesn't really program the pipeline; it just provides the GPU and dispatches submitted IBs. The **userspace driver** (Mesa/radv, AMD PAL/Vulkan, etc.) does:

- Compile API shaders (NIR → ACO/LLVM → AMDGPU ISA).
- Set up RBE/CB state via context registers.
- Emit vertex/index buffer descriptors.
- Pick wave size (32 vs 64), NGG mode, optimal stage merging.
- Emit PM4 to set context state and submit draws.

Kernel just: validates BOs, schedules IBs, signals fences.

Performance traps in geometry

Symptom	Cause	Fix
Vertex throughput collapses	Many small draws	Batch / instance / indirect draws

Symptom	Cause	Fix
GS used heavily	Slow on AMD	Refactor without GS or to mesh shaders
Excessive vertex re-fetch	Index buffer not localized	Mesh-optimize index ordering
Position passes slow	Full vertex shader for shadow	Use position-only path
Tessellation hot	Over-tessellation	Cap tess factors
Mesh shader hangs	Wave divergence in task→mesh emission	Check group sizes, stride in `OutputVertices`

What you should be able to do after this chapter

- Sketch the legacy and NGG geometry pipeline.
- Define mesh / task shader in one sentence.
- Explain why GS is slow on AMD.
- List four primitive culls.
- Identify which stages a given API setup will use.

Read deeper

- `mesa/src/amd/common/ac_nir_lower_ngg.c` — NGG lowering.
- `radv` mesh-shader implementation in Mesa.
- DX12 / Vulkan mesh shader specs (Khronos, Microsoft).
- AMD GPUOpen blog: “Mesh shaders” + “GPU geometry processing.”
- “GeForce 8800 architecture” — historical context for unified shader pipelines (works for AMD too).

Chapter 71 — Rasterization, hierarchical Z, primitive culling

The rasterizer takes a primitive (post-clipped, post-projected) and figures out which pixels (or sub-pixel samples) it covers. It then schedules pixel-shader waves on the CUs. This is fixed-function silicon — you don't program it directly, but you live with its rules forever.

What rasterization actually computes

Given a triangle defined by its three screen-space vertices, the rasterizer:

1. Computes the bounding box.
2. For each pixel/sample inside the bounding box, evaluates **edge functions** to test inside/outside.
3. For inside samples, computes barycentric coordinates and interpolates per-vertex attributes.
4. Emits a **pixel quad** (2×2 group of pixels) to the pixel shader, with attribute-interpolated values, derivative info (`ddx` / `ddy`), and a coverage mask.

It does this in parallel across multiple **rasterizers** per shader engine. Throughput is measured in pixels (or quads) per clock.

Pixel quads — the 2×2 unit

Pixel shaders execute on **2×2 quads** even at the edge of triangles. Why? Because hardware needs neighbors for derivative computations (`ddx` / `ddy`) used in mip-level selection for textures. So even a 1-pixel triangle occupies a full quad, with 3 of the 4 lanes "helper invocations" that don't write final color but participate in derivative math.

This means small triangles are wasteful — you're paying for 4 lanes' worth of shading to color 1 pixel. **Small-triangle culling matters.**

Hierarchical Z (HiZ) and Hi-Stencil

Before pixel shading runs, the **HiZ** unit can reject entire pixel tiles by comparing the triangle's depth range against the pre-computed minimum/maximum Z of each tile in the depth buffer.

```
Screen tile (e.g., 8x8 pixels) has stored:
   ZMIN, ZMAX

Triangle covers tile with depths [zlo, zhi].
- If zhi < ZMIN  → entire tile passes (closer than everything stored).
- If zlo > ZMAX  → entire tile rejected (entirely behind everything stored).
- Otherwise     → fall through to per-pixel Z test.
```

HiZ buffers are compressed metadata maintained alongside the depth buffer.

EarlyZ is the per-pixel depth test that runs before the pixel shader (when possible), skipping shader invocations whose Z fails. Disabled if the shader writes Z, uses discard heavily, or has side effects. Modern shaders use `[earlydepthstencil]` (HLSL) / `layout(early_fragment_tests)` (GLSL) to force EarlyZ when safe.

A senior engineer always asks: "is EarlyZ enabled here? If not, why?"

Render targets, depth, MSAA

Rasterizer outputs land in render targets (RTs) and a depth buffer. Multiple render targets (MRT) — up to 8 typical. Each may have:

- **Format** (R8G8B8A8_UNORM, R16G16B16A16_FLOAT, R32_UINT, ...).
- **Compression**: AMD calls this **DCC (Delta Color Compression)**. Lossless tile-level compression that reduces bandwidth on render and texture-fetch.
- **Tiling**: GPUs don't store textures linearly; they use 2D-swizzled tiling for cache locality. AMD's **DCC + Display tiles** swizzling has many flavors per gen.

MSAA (Multi-sample antialiasing)

Multiple **samples per pixel** (2/4/8) at the depth/coverage level, but the pixel shader runs *once per pixel* (not per sample). The single shaded color is broadcast to covered samples. Resolve at end-of-frame compresses to 1×.

EQAA (Enhanced Quality AA) increases coverage samples beyond color samples for cheaper AA.

MSAA is bandwidth-heavy. RDNA introduced **VRS (Variable Rate Shading)** — shade at coarser-than-pixel rate (e.g., 1 shader invocation per 2×2 block) for areas with low detail. Saves shader work without proportionally hurting visible quality.

Primitive culling, again — at the rasterizer level

Even after geometry-stage culling, the rasterizer can reject:

- Primitives outside the scissor.
- Sample-mask zero coverage.
- Conservative-rasterization edge cases.

Conservative rasterization (rare feature) marks a pixel as covered if *any part* of the triangle touches it. Used for voxelization, occlusion culling.

Tile-based vs immediate-mode

Mobile GPUs (Mali, Adreno, PowerVR) are **tile-based deferred renderers (TBDR)**: they bin all primitives into screen tiles, then process tiles serially. Saves bandwidth.

AMD desktop GPUs are **immediate-mode** with tiles being a cache/tiling concern, not a rendering primitive. RDNA introduced **DSBR (Draw Stream Binning Rasterizer)** as a hybrid: defers rasterization within a small stream window to extract primitive overlap, then rasterizes; smaller wins than full TBDR but no app changes needed.

RDNA-specific: WGP-pair issuing

Pixel-shader waves run on the same WGPs as compute. The graphics scheduler picks WGPs for new PS waves based on RBE bandwidth and CU occupancy. Senior debugging looks at "PS wave issue stalls" to detect rasterizer-front-end bottlenecks.

Common rasterization perf issues

Symptom	Likely cause	Fix
GPU bound on tiny triangles	Sub-pixel geometry	Proper LOD / mesh-shader cull
Z-bound	EarlyZ disabled	Avoid `discard`; mark `earlydepthstencil`
RBE bandwidth pegged	Heavy MSAA / no DCC	Enable DCC, reduce MSAA, use VRS
Overdraw shows in RGP	No depth pre-pass	Add depth-only pass for opaque geo
Quad utilization < 50%	Small triangles	Mesh-shader cluster cull

Driver responsibilities

Kernel: nothing rasterization-specific; it just allocates BOs and submits IBs.

UMD (Mesa, PAL, Windows driver):

- Configure RBE state (PA_SC_, *DB_*, CB_* registers) from API state.
- Pick depth/color compression (DCC) modes per RT.
- Manage **HTILE** (HiZ metadata) and **CMASK** (color compression metadata) buffers.
- Emit clears that update HTILE/DCC metadata in O(1) instead of touching every pixel.
- Submit fast-clear PM4 sequences.

A *lot* of UMD optimization lives in metadata management.

Display interaction

The final color RT often becomes a presented surface:

- KMS receives a `drm_framebuffer` referencing the BO.
- Display Core (DCN, chapter 77) reads it. DCN supports DCC-compressed inputs, so the render output doesn't need a decompression pass before scanout — nice bandwidth saving.

Cross-driver dma-buf scenarios (composited via Wayland) require care: the consumer must understand the modifier (tiling+DCC layout). DRM **format modifiers** describe these layouts so consumer drivers know what they're getting.

What you should be able to do after this chapter

- Explain pixel quads and helper invocations.
- Describe HiZ and EarlyZ in one sentence each.

- Define DCC, HTILE, CMASK and what each compresses.
- Know when EarlyZ gets disabled.
- Explain DSBR vs TBDR.
- Identify rasterizer perf issues from a profile.

Read deeper

- AMD GPUOpen: “DCC primer,” “Render Backend Engine (RBE),” “DSBR.”
- `mesa/src/amd/common/ac_surface.c` — surface tiling/compression metadata.
- `mesa/src/amd/vulkan/radv_meta_*.c` — fast-clear, decompress meta paths.
- `drivers/gpu/drm/amd/include/asic_reg/db_*.h`, `cb_*.h` — depth and color register sets.
- “A Trip Through the Graphics Pipeline” — Fabian Giesen’s classic 13-part series. Required reading.

Chapter 72 — Pixel pipeline: RBE, ROPs, blending, MSAA

The pixel shader's output doesn't go straight to memory. It funnels through the **Render Back End (RBE)** — a fixed-function block that does depth testing, blending, MSAA resolve, and compression-aware writes.

RBE is where memory bandwidth is most actively managed. PMTS engineers spend years tuning RBE behavior.

Block diagram of the RBE

```
PS wave outputs (color, depth, coverage)
              |
              v
      +------------------+
      |  Pre-shader Z/   |   EarlyZ (when applicable)
      |   stencil tests  |
      +------------------+
              |
              v   (PS runs if needed, on the CUs)
              |
              v
      +------------------+
      |  Post-shader Z/  |
      |   stencil tests  |
      +------------------+
              |
              v
      +------------------+
      |  Coverage / MSAA |
      +------------------+
              |
              v
      +------------------+
      |    Blending      |
      +------------------+
              |
              v
      +-----------------------+
      | Color / Depth caches  |   CB$, DB$
      +-----------------------+
              |
              v
      +-----------------------+
      | DCC/HTILE/CMASK paths |   compression engines
      +-----------------------+
              |
              v
            VRAM
```

CB = Color Buffer pipe, DB = Depth Buffer pipe. Each has its own caches, separate from the shader caches.

ROPs ("Raster OPerations" units)

A **ROP** is the unit that takes a quad of post-shader pixels and writes them to memory after testing/blending. AMD GPUs have many ROPs distributed across shader engines. ROP count is often advertised in marketing specs ("64 ROPs").

ROP throughput limits fillrate: how many pixels/sec you can write. With heavy blending or MSAA resolve, ROPs can become the bottleneck even when shaders are idle.

Depth and stencil

Depth test happens twice (potentially): pre-shader (EarlyZ when allowed) and post-shader (always). DB pipe handles:

- depth-test (less, less-equal, greater, etc.),
- depth write (with mask),
- HTILE updates,
- depth compression.

Stencil is a per-sample 8-bit value stored interleaved with depth (typical D24S8 or D32_S8 formats). Operations: keep, replace, increment, decrement, increment-wrap, decrement-wrap, invert.

Common uses for stencil:

- shadow volumes (legacy),
- decal stencil,
- masking deferred-shading lighting passes,
- mirror/portal effects.

Blending

The CB pipe applies the blend equation:

```
final = src * srcFactor  OP  dst * dstFactor
```

`OP` is add/sub/min/max. Factors are zero/one/srcAlpha/oneMinusSrcAlpha/etc. The hardware reads the destination value, computes, writes back. Blending is a read-modify-write; with order-dependent blending (typical alpha), it serializes pixel writes within a pixel.

Independent blend allows different blend state per RT. **Dual-source blending** (two outputs from PS into one RT) supports advanced effects like correct subpixel AA.

DCC (Delta Color Compression)

AMD's tile-level lossless RT compression. For each tile (e.g., 8×8 pixels in BC-style layouts), a small metadata block stores either "this tile is a constant" or "delta-encoded against a reference." When a write would change the tile from "compressed" to "non-compressible," the hardware silently decompresses.

Properties:

- **Lossless** — pixel-exact.
- **Bandwidth saving**: typical 30-60% on real frames.

- **Correct on read**: any consumer aware of DCC can read directly; non-aware needs **decompress** pass.

DCC requires extra metadata buffers (called **DCC keys**) tied to the RT. Modifier-aware compositors (KWin, Mutter on recent Mesa) can scan out DCC-compressed buffers without intermediate decompression.

DCC is on by default in Mesa for Vulkan/RADV; some old games or unusual access patterns disable it.

CMASK and FMASK

- **CMASK (Color Mask)**: 1-bit-per-sample (or 4-bit-per-tile) buffer that records whether a sample equals the "fast-clear color" — enables 1-cycle clears.
- **FMASK (Fragment Mask)**: only for MSAA RTs; encodes which sample at each pixel got which shader output. Lets the resolve pass do the right thing without per-sample stores.

Fast clears

Clearing an RT to a fixed color is a frequent operation. The naive "write every pixel" path costs `width × height × bpp` bytes. AMD's fast clear:

1. Set the CMASK to "all clear."
2. Store the clear color in a small register (per RT).
3. Future reads through CB return the clear color when CMASK says so.

Cost: O(metadata size) instead of O(pixel count). Massive win.

The driver/UMD must emit the clear and update metadata correctly, including coordination with subsequent compositors.

MSAA on the back end

Multi-sample paths in RBE:

- **Coverage**: which samples does this fragment cover? (1, 2, 4, 8 samples per pixel.)
- **Storage**: typically one shaded color per pixel + per-sample coverage; FMASK encodes which sample got which color when there's overlap.
- **Resolve**: at end-of-frame, average per-sample colors into 1× color buffer for display.

MSAA bandwidth = N× single-sample. RBE must maintain throughput; modern AMD parts have lots of CB area dedicated to MSAA.

VRS (Variable Rate Shading) interaction

VRS reduces shading rate to coarser-than-pixel (1×2, 2×1, 2×2, 4×4). The PS runs once for a VRS region, and the RBE writes the same color to all covered pixels. Coverage and depth are still per-sample.

Three sources of VRS rate:

1. **Per-draw**: API call sets a global rate.
2. **Per-primitive**: vertex shader output `SV_ShadingRate` per primitive.

3. **Per-screen-region**: a low-res "shading rate image" attached to the framebuffer.

Common use: foveated rendering for VR.

Pixel shader output paths

A PS can write:

- **Up to 8 RT colors** (`SV_Target0..7`).
- **Depth**: rare (`SV_Depth`); kills EarlyZ.
- **Stencil**: very rare (`SV_StencilRef` in DX12).
- **UAV writes** (storage images / structured buffers): goes through the regular shader memory path, **not** RBE.
- **Discard / kill**: drops the fragment (no write); may interact with EarlyZ.

UAV writes from PS are useful for compute-graphics interop (e.g., GPU particles updated from PS). Order isn't guaranteed across pixels of the same triangle; if you need ordering, atomics or **ROV (Raster-Order Views)** — a hardware feature on some chips that orders UAV writes per-pixel.

Cross-stage dependencies

When a render pass writes to a texture and a subsequent draw reads it, the producer's RBE writes must drain to memory and downstream caches must invalidate before the consumer can fetch. The driver/UMD inserts:

- **Cache flushes** (RB→L2 writeback).
- **Stage barriers** (Vulkan `vkCmdPipelineBarrier`).
- **Decompress** if consumer doesn't understand DCC.

Mis-emitting barriers is one of the top game-driver bug categories.

Driver responsibilities

UMD work:

- Choose CB compression mode (CMASK on/off, DCC on/off, FMASK on/off, fast-clear color register).
- Emit fast-clear PM4 sequences.
- Manage Vulkan barriers → PM4 cache flush events.
- Pick the right shadow/depth-only path.

Kernel work: BO allocation for metadata, atomic update of metadata BO when scanned out (compositor/DRM).

What you should be able to do after this chapter

- Explain RBE, CB, DB.
- Define DCC, CMASK, FMASK, HTILE.
- Walk through a fast-clear sequence.
- List PS output kinds.

- Reason about MSAA bandwidth.

Read deeper

- AMD whitepapers on RBE/DCC.
- `mesa/src/amd/common/ac_surface.c` — surface metadata.
- `mesa/src/amd/vulkan/radv_image.c`, `radv_meta_clear.c`, `radv_meta_decompress.c`.
- DX12 / Vulkan barrier specs (Microsoft, Khronos).
- “A Trip Through the Graphics Pipeline” — RBE chapters.

Chapter 73 — Ray tracing hardware

Ray tracing on AMD started with **RDNA 2** (RX 6000 series, 2020). RDNA 4 is the third generation. Each gen significantly improves per-ray throughput, traversal performance, and BVH features. PMTS-level engineers in the graphics group must speak this fluently.

What ray tracing actually computes

Given a ray (origin + direction) and a scene of geometry, find:

- **First hit** (or "any hit," depending on shader): which triangle/primitive does the ray first intersect?
- Trigger the appropriate **shader** (closest-hit, miss, any-hit, intersection) with hit info.

The naive approach (test every triangle) is O(N) per ray — unusable. The standard solution is a **BVH (Bounding Volume Hierarchy)**: a tree of axis-aligned bounding boxes containing geometry. Traversal is roughly O(log N) per ray.

The BVH

```
                Root AABB
                /       \
             AABB        AABB
            /    \      /    \
          ...   ...   ...   ...
          /\
        Leaf  Leaf       ← references to triangles
```

Each interior node has a few children (binary, quad, octree variants); each leaf references geometry.

A ray descends the tree:

1. Test ray against each child AABB.
2. Push hit children onto a traversal stack.
3. At leaves, intersect against actual triangles.
4. Return closest hit.

Modern GPUs have **hardware** for BVH traversal — the part that's expensive in software but a few cycles per node in HW.

AMD's ray-accelerator approach

RDNA 2/3/4 add **Ray Accelerators** (RAs) inside each CU. Compared to NVIDIA's RT cores, AMD's RAs are *hybrid*: hardware for box and triangle intersection tests, **but traversal control flow runs in shader code**.

```
On AMD:
- Shader executes traversal loop.
- For each node, calls hardware "intersect 4 boxes" or "intersect triangle" intrinsic.
```

```
- Hardware returns hit/no-hit / barycentrics.
- Shader pushes/pops the stack, decides next node.
```

This means:

- The compiler emits the traversal loop ("inline traversal").
- Optimization (which node to visit next, sorting hits) is software-controlled.
- More flexible than fully-fixed-function approaches; can be slower if the loop is poorly compiled.

NVIDIA's RT cores do traversal in HW too; AMD trades flexibility for less area. Real-world performance differences are workload-dependent and gen-dependent.

RDNA 3 and 4 improvements

RDNA 3:

- Larger LDS-resident traversal stack option.
- Better sorting of hit candidates.
- More instructions per ray.

RDNA 4:

- Major increase in box-tests-per-cycle.
- Improved BVH traversal compaction.
- Better hit/miss latency hiding.

Comparable to NVIDIA's per-gen RT improvements; the gap has narrowed.

DXR / VKR — the API picture

DirectX Raytracing (DXR) and **Vulkan KHR_ray_tracing_pipeline** define the same conceptual model:

- App builds **acceleration structures (AS)**: bottom-level (BLAS, per mesh) and top-level (TLAS, the scene).
- App writes **HLSL/GLSL shaders** for the various RT shader stages.
- App dispatches a "trace rays" call: width × height × depth threads, each tracing rays.

Shader stages:

Stage	When
Ray generation (RGen)	First, per ray-thread; emits primary rays
Closest-hit (CHit)	When a ray's closest hit is found
Any-hit (AHit)	For each hit considered (alpha-tested geometry)
Miss	When the ray misses everything
Intersection (Isect)	Custom primitive types (procedural geometry)
Callable	App-defined utility shaders, like function calls

These compile into PSOs (DX) or pipeline objects (Vulkan) bundled with shader binding tables (SBTs) the GPU uses to dispatch the right shader for each hit.

Inline ray tracing (a.k.a. “Ray Queries”)

DXR 1.1 / Vulkan `VK_KHR_ray_query` allow ray traversal **inside** a regular pixel/compute shader without a full RT pipeline:

```
RayQuery<RAY_FLAG_NONE> rq;
rq.TraceRayInline(scene, ...);
while (rq.Proceed()) { ... }
if (rq.CommittedStatus() == COMMITTED_TRIANGLE_HIT) {
    /* shade */
}
```

Inline RT is preferred when you don’t need many shaders or complex SBTs. AMD’s hardware loves this — compiler embeds the traversal loop directly.

BVH build

AS construction itself is GPU-accelerated:

- Driver/runtime calls into a BVH builder kernel (provided by AMD’s runtime / Mesa’s radv).
- Builder partitions geometry, computes AABBs, lays out nodes in BVH-format buffers.
- App sees an opaque BO holding the AS.

The BVH layout is **AMD-private** — apps must rebuild ASes on driver update if AMD changes the format. Mesa and AMD’s official driver may have incompatible BVH layouts; that’s why an AS built by one isn’t usable by the other.

Build modes:

- **Build** — initial.
- **Update** — refit AABBs after small per-frame motion (cheap).
- **Compact** — make a smaller copy after build to save memory.

Performance considerations

Ray tracing throughput depends on:

1. **BVH quality** — how well it minimizes ray-box overlap. SAH (Surface Area Heuristic) builders are slower but produce better BVHs.
2. **Coherence** — adjacent rays should mostly visit the same nodes. Random rays = thrash.
3. **Wave occupancy** — RT shaders tend to have high VGPR usage; spills hurt a lot.
4. **Hit shader divergence** — different lanes hit different materials → divergent shader execution.
5. **Cache locality** — BVH traversal is pointer-chase-y; L0/L1 hit rate matters.

PMTS engineers spend tens of thousands of hours collectively on these.

Mitigations

- **Sort rays** before tracing (by direction or origin) to improve coherence.
- **Bin shading** — sort hits into bins by material, then dispatch coherent shading kernels. (NVIDIA’s “SER” feature; partially software-emulated on AMD.)
- **Reduce VGPR pressure** in hit shaders.
- **Use ray queries** for simple cases (avoids shader-binding-table indirection).

- **Avoid AnyHit** if possible (each AnyHit invocation re-enters shader).

Driver responsibilities

Kernel:

- Allocate BVH BOs.
- Map them with right MTYPE (uncached on writes from BVH-builder kernel that is, then cached for reads — actually usually fully cached).
- Provide enough VRAM headroom (BVHs are big).

UMD (Mesa, PAL):

- Compile RT shaders (ACO/LLVM) with traversal loop inlined.
- Build the SBT.
- Build BVHs — radv currently does this with its own builder; AMD's PAL has another. Quality varies.
- Manage AS update vs rebuild.

A real PMTS contribution is improving the BVH builder. radv's builder has gotten substantially better over the last 2 years through community contributions.

Lumen, RTGI, path tracers

Real-world workloads using AMD RT:

- **Cyberpunk 2077** path-tracing mode (RDNA3/4 territory).
- **Alan Wake 2**, **Indiana Jones** — heavy RT.
- **Unreal Engine 5 Lumen** — uses inline RT for software-traced and hardware-traced flavors.
- **Blender Cycles** — production path tracer with HIP RT backend.

Knowing what these workloads stress (BVH update cost? Shader divergence?) is the day-to-day of RT-team engineers.

What you should be able to do after this chapter

- Explain BVH and traversal.
- Describe AMD's hybrid ray-accelerator approach.
- List the DXR shader stages.
- Define inline RT vs RT pipeline.
- Identify performance traps (divergence, BVH quality, VGPR pressure).

Read deeper

- AMD GPUOpen "Ray Tracing in DXR" series.
- `mesa/src/amd/vulkan/radv_acceleration_structure.c` — radv BVH builder.
- "Ray Tracing Gems" volumes 1 and 2 (free PDFs).
- Microsoft DXR spec.
- Khronos `VK_KHR_ray_tracing_pipeline` and `VK_KHR_ray_query` specs.

- PIX / RGP captures of any RT-enabled game.

Chapter 74 — KFD, HSA, AQL packets, MES

The compute path on AMD GPUs is fundamentally different from the graphics path. Graphics goes through DRM (`/dev/dri/card*`, libdrm, Mesa). Compute goes through **KFD** (`/dev/kfd`) using the **HSA** model with **AQL packets** scheduled by **MES**.

Senior compute / runtime engineers at AMD live in this chapter.

The dual-driver story

Historically:

- `/dev/dri/cardN` — the DRM/amdgpu node, used by Mesa, Vulkan, OpenGL, OpenCL-on-Mesa.
- `/dev/kfd` — the KFD node, used by ROCr (HSA runtime), HIP, ROCm tools.

Both nodes talk to the same `amdgpu` kernel driver, but expose different ioctls and a different programming model. KFD has **always-on user-mode queues**; DRM uses kernel-mediated submission. Recent MES-driven kernels are blurring the line, but the userspace API split remains.

What HSA is

Heterogeneous System Architecture (HSA) — an industry standard from the early 2010s, championed by AMD. The full vision (cross-vendor unified compute) didn't materialize, but AMD's runtime (ROCr) is HSA-shaped.

Core HSA ideas:

- **Agents**: each device (CPU, GPU) is an agent.
- **Queues**: software-visible ring buffers per process per agent.
- **AQL (Architected Queuing Language)**: standardized 64-byte packets describing work.
- **Signals**: hardware-coordinated 64-bit values for sync.
- **Shared virtual memory** (SVM): CPU and GPU share addresses.

ROCr is the user-space runtime implementing HSA on AMD. HIP (CUDA-API-shaped) sits above ROCr. PyTorch, TensorFlow, JAX run on top.

AQL packet format

Every AQL packet is **64 bytes**, with a 16-bit header and packet-type-specific body.

```
+--------+--------+---------------------------+
| header |  type  |    type-specific data     |
+--------+--------+---------------------------+
   16b      16b              480 bits
```

Packet types include:

Packet	What it does
`KERNEL_DISPATCH`	run a compute kernel with grid dimensions
`BARRIER_AND`	wait for all listed signals to reach 0
`BARRIER_OR`	wait for any
`AGENT_DISPATCH`	run a runtime-internal task on the kernel agent
`INVALID`	written into the slot until the producer is ready (consumer waits)

Kernel dispatch carries: grid size, workgroup size, kernel object pointer, kernarg pointer, completion signal handle, barrier flags.

The user-mode queue model

A queue is a **ring buffer in shared memory**. Producer (the host runtime, in user mode) writes packets at `write_index`. Consumer (the GPU's MES firmware, or older HW's HQD) consumes at `read_index`. A **doorbell** notifies the GPU.

```
+-------------------------------------+
| AQL packet 0 (INVALID until ready)  |
+-------------------------------------+
| AQL packet 1                        |
+-------------------------------------+
| AQL packet 2                        |
+-------------------------------------+
| ...                                 |
+-------------------------------------+
| AQL packet N-1                      |
+-------------------------------------+
   ^                   ^
   read_index          write_index
   (consumed by HW)    (advanced by host)

Doorbell write rings the queue's doorbell (PCIe write to a magic page).
```

To submit:

1. Reserve a slot (atomic increment of `write_index`).
2. Fill all packet fields **except** `header.type`.
3. Memory barrier.
4. Atomically write `header.type = KERNEL_DISPATCH` (transitions packet from INVALID to ready).
5. Ring the doorbell.

Steps 4+5 must be atomic-in-meaning so the GPU can't see a partially-filled packet.

Doorbells

A **doorbell** is a small PCIe-mapped page where each queue has its own dword. Writing to the doorbell wakes up the GPU scheduler. The mapping is per-process and protected by VMID/PASID — process A can't ring process B's doorbell.

Kernel allocates doorbell pages and grants `mmap` access to userspace. Each queue gets a slot; thousands of queues share one doorbell page.

MES: scheduling user-mode queues

MES (Micro Engine Scheduler) is firmware running on a dedicated microcontroller in the GPU. It manages the **runlist**: set of queues currently mapped to hardware. Each VMID slot can have at most one mapped queue per pipe per ME instance.

When a queue gets work (doorbell rings):

1. MES checks if the queue is mapped (has a real HW slot).
2. If yes, MES schedules execution: dispatches workgroups based on AQL packet.
3. If no, MES decides whether to swap out a lower-priority queue and map this one.

Scheduling considers:

- Per-queue priority (HSA `HSA_QUEUE_PRIORITY_*`).
- Per-process priority (set via KFD ioctl).
- Wave time-slicing for long-running kernels.
- CWSR (chapter 75) for preemption.

Pre-MES: kernel did this in software via "process queue manager" with limited slots. MES scales much better.

KFD ioctls — the compute API

`/dev/kfd` exposes ioctls that are *complement* to amdgpu's:

Ioctl	Purpose
`AMDKFD_IOC_CREATE_QUEUE`	allocate a user queue (gets back a doorbell offset)
`AMDKFD_IOC_DESTROY_QUEUE`	tear down a queue
`AMDKFD_IOC_SET_MEMORY_POLICY`	per-VMID memory policies
`AMDKFD_IOC_GET_CLOCK_COUNTERS`	timestamps for profiling
`AMDKFD_IOC_ALLOC_MEMORY_OF_GPU`	allocate GPU-visible memory (VRAM or coherent system)
`AMDKFD_IOC_MAP_MEMORY_TO_GPU`	bind memory to a GPU VM
`AMDKFD_IOC_SVM`	manage shared virtual memory regions and migration policies

All of these manipulate kernel-side `struct kfd_process` / `struct kfd_process_device` state. The KFD source is in `drivers/gpu/drm/amd/amdkfd/`.

Kernel object & code object

A compiled HIP kernel is an **HSACO** (HSA code object) — an ELF blob containing AMDGCN machine code, kernel descriptors, metadata. ROCr loads it into VRAM and produces a **kernel object pointer** — passed in `KERNEL_DISPATCH` packets.

Compute waves spawned by the GPU read code from VRAM via the L1 instruction cache. Updating the same kernel object (e.g., during JIT recompile) requires icache invalidation.

Signals

HSA signals are 64-bit memory locations tracked with their producer/consumer semantics. Used for:

- Completion notification (kernel dispatch carries a completion signal handle).
- Barriers (wait until signal == X).

Two flavors:

- **Default signals** — a memory location plus a kernel-managed waiter event.
- **Doorbell signals** — backed by GPU doorbells; very low-latency for inter-queue sync.

ROCr exposes them via `hsa_signal_*` API. Internally they are atomic 64-bit values.

SVM (Shared Virtual Memory)

In SVM mode, GPU and CPU share an address space. Allocations made by `malloc()` (or `hsa_amd_svm_attribute_set`) are accessible from both, with the kernel handling page faults via:

- **HMM (Heterogeneous Memory Management)** in Linux — provides page migration callbacks.
- **MMU notifiers** — invalidate GPU mappings when CPU side mprotects/unmaps.
- **GPU page faults** — when GPU touches an unmigrated page, IOMMU/GMC raises a fault, kernel handles it (chapter 84).

SVM is heavy machinery and where the most subtle compute bugs live. CDNA APUs (MI300A) make this trivial because of hardware coherence; discrete GPUs need explicit migration.

Userspace stack

```
Application (Python / C++)
        |
   PyTorch / TF / JAX / HIP code
        |
        v
  HIP runtime (libamdhip64.so)
        |
        v
  ROCr / HSA runtime (libhsa-runtime64.so)
        |
        v
  KFD ioctls + doorbells (no syscall on submit)
        |
        v
   amdgpu kernel (KFD module) + amdgpu IH/MES
        |
        v
   GPU hardware
```

Once a queue is created, **submission has zero syscalls**. AQL packet write + doorbell write = the entire submission path. This is critical for AI workloads issuing thousands of kernels/sec.

Driver responsibilities (KFD)

Kernel must:

- Manage `kfd_process` lifecycle (per-process resources).
- Allocate/deallocate user queues, doorbell slots, runlist entries.
- Coordinate VMID allocation with the rest of `amdgpu`.
- Implement SVM with HMM/MMU notifiers.
- Implement `kfd_dev_dbg_*` for debug events when rocgdb attaches.
- Handle GPU page faults (recover or signal app).

These are deep waters. `drivers/gpu/drm/amd/amdkfd/kfd_*.c` is the codebase.

What you should be able to do after this chapter

- Explain the dual-node (DRM + KFD) architecture.
- Describe the AQL packet format.
- Walk through user-mode queue submission step-by-step.
- Define MES, runlist, doorbell, kernel object.
- Describe how SVM uses HMM and MMU notifiers.

Read deeper

- AMD HSA runtime (open-source ROCr): https://github.com/ROCm/ROCR-Runtime.
- HSA spec PDFs (HSA Foundation, public).
- `drivers/gpu/drm/amd/amdkfd/kfd_chardev.c` — every ioctl handler.
- `drivers/gpu/drm/amd/amdkfd/kfd_process_queue_manager.c`.
- `drivers/gpu/drm/amd/amdkfd/kfd_svm.c` — SVM with HMM.
- AMD GPUOpen blog: “AQL packet processing,” “MES queues.”

Chapter 75 — CWSR, priorities, preemption

A modern GPU runs work for many processes (and many queues per process). Each kernel can take milliseconds to hours. Without preemption, an evil or just-long compute kernel would block all interactive graphics. Solving this is **CWSR**: Compute Wave Save / Restore.

This chapter is dense and important — preemption interacts with reset, scheduling, security, and performance.

The problem

Consider:

- Process A: training a 30-second compute kernel (BFS over 10 GB of graph).
- Process B: a Wayland compositor wanting to render the next frame in 16 ms.

If A's wave occupies the GPU and we can't interrupt it, B misses the deadline → user-visible stutter. Worse: in some platforms (laptops, VR), the watchdog will eventually reset the GPU thinking it hung — interrupting A *and* breaking everyone.

We need to *preempt*: stop A, start B, later resume A from where it left off.

Granularity options

GPU preemption can happen at different granularities:

Granularity	Meaning	Cost	Used for
Pipeline boundary	Wait for current draw/dispatch to finish, then switch	low overhead, high latency	OK for most graphics
Wavefront boundary	Drain in-flight waves, switch	medium	better for mixed loads
Mid-wave (CWSR)	Save wave state mid-instruction, switch, restore later	high overhead, low latency	required for real-time + long compute

CWSR is the heaviest, used when latency requirements are tight.

What CWSR saves

When a wave is preempted mid-execution, the hardware/RLC must save:

- All VGPRs (~1024×4 B = 4 KB per SIMD wave).
- All SGPRs (~108×4 B = ~432 B per wave).
- LDS contents (up to 64-128 KB per workgroup).
- Wave PC, exec mask, status registers, scc, vcc.

- Scratch context.

The save area is in **VRAM** (or sometimes GTT), pre-allocated by the runtime. RLC firmware copies wave state to/from this area when preemption happens. Restore is the same in reverse.

CWSR images are **chip-specific** in layout — every gen has its own format documented in AMD's headers.

When CWSR runs

The host requests preemption via PM4 packet (`PACKET3_PREEMPT`) on the GFX or compute ring, or via MES API for user queues. Hardware:

1. Selects waves to preempt (by priority criteria).
2. RLC writes their save areas.
3. Idle indication signals back to MES/CP.
4. New work runs (via newly mapped queue or new dispatch).
5. When the preempted work is to resume, RLC reads the save areas and the waves continue.

Time-slicing

Without CWSR, very-long-running kernels would block everything. The kernel watchdog (TDR — Timeout Detection and Recovery, chapter 53) considers a long-running kernel as "hung" and resets the GPU.

Time-slicing combats this: every N milliseconds, MES preempts and yields to other queues. Long-running kernels see "pauses" but eventually finish. The watchdog measures *forward progress*, not wall time, so legitimate long kernels don't trigger it.

Priorities

ROCr exposes:

- `HSA_QUEUE_PRIORITY_LOW`
- `HSA_QUEUE_PRIORITY_NORMAL`
- `HSA_QUEUE_PRIORITY_HIGH`
- `HSA_QUEUE_PRIORITY_REALTIME` (rare, requires permissions)

Vulkan/DX exposes context priorities. Internally these map to MES scheduler priority bands.

The **realtime** priority is special: it preempts everything else aggressively, used for compositors, VR, low-latency audio (where AMD GPU is doing mixing).

CU masking and partitioning

Sometimes you want **physical isolation** instead of time-slicing. Two mechanisms:

- **CU mask**: a bitmap saying which CUs a queue may run on. Process A gets CU0..CU15, Process B gets CU16..CU31. Hard partition.

- **Compute partitioning** (CDNA-2/3): an entire XCD or SE can be carved out as a logical sub-device.

CU masks are popular in datacenter (e.g., shared GPU between containers). Set via KFD `set_cu_mask` ioctl.

Realtime + display interaction

Display (chapter 77) operates on hard real-time deadlines (vblank intervals). Page flips need a few hundred microseconds of GPU compute (composition). The graphics scheduler escalates these to high priority and uses CWSR if a long compute task is mid-flight.

Mistakes in this code path produce visible glitches — frame skips, tearing, "compositor hang." Reviewing display ↔ compute scheduling fixes is a senior responsibility.

What kernels and runtimes worry about

- **Save area sizing**: too small → CWSR fails; too big → wastes VRAM.
- **Restore correctness**: every shadow register must round-trip exactly. Bugs here are catastrophic.
- **Atomic write windows**: queue manipulation must be atomic from MES's perspective. Race → wrong queue runs.
- **Priority inversion**: low-prio queue holds a resource the high-prio one needs. Detect and avoid.

Preemption + reset

If preemption itself hangs (RLC firmware bug, hardware glitch), the whole reset machinery (chapter 53) kicks in. Tiered reset:

1. **Soft reset**: just the offending IP (compute), preserves graphics.
2. **Mode-1 reset**: more of the GPU.
3. **Mode-2 reset**: full reset, all userspace contexts get `VK_ERROR_DEVICE_LOST`.

CWSR-induced hangs typically lead to soft reset.

Debug interactions

When `rocgdb` attaches to a HIP process, it can:

- Set breakpoints inside running compute waves.
- Cause a wave to halt at a breakpoint.
- Step through GPU code instruction-by-instruction.

This requires the kernel to inject debug interrupts and use CWSR-like state save to read VGPR/SGPR/LDS values. The KFD debug ioctls (`AMDKFD_IOC_DBG_*`) implement this. It's a beautiful pile of complexity.

Performance considerations

- CWSR overhead per preemption: dozens of microseconds. Not free.

- Frequent preemption hurts throughput for the preempted workload.
- The scheduler tries to minimize preemptions consistent with priority.
- **Avoid mixing realtime with long compute** if you can place them on different GPUs.

Driver responsibilities

- Allocate CWSR buffers per-process, per-queue.
- Program RLC tables that say where to find per-chip state.
- Coordinate with MES on queue priorities.
- Handle CWSR failures (saved wave can't be restored: hangs → reset).
- Expose KFD ioctls for CU masks and priority.

`drivers/gpu/drm/amd/amdkfd/kfd_packet_manager.c`, `kfd_priv.h`, `kfd_mqd_manager_*.c`, and `gfx_v*.c` (per-gen RLC programming) are the relevant files.

What you should be able to do after this chapter

- Explain why GPUs need preemption.
- Define CWSR and what it saves.
- Describe priority bands.
- Differentiate CU mask vs time-slicing.
- Reason about preemption + reset interactions.

Read deeper

- AMD GPUOpen: "GPU Compute Preemption," "Realtime Compute on Radeon."
- `drivers/gpu/drm/amd/amdkfd/kfd_mqd_manager_*.c` — MQD (Memory Queue Descriptor) handling.
- `drivers/gpu/drm/amd/amdgpu/gfx_v11_0.c` — RLC microcode load + CWSR setup.
- `rocgdb` documentation (the GPU debug API).
- HSA runtime spec, "queue priority and preemption" section.

Chapter 76 — ROCm runtime deep

ROCm is AMD's "CUDA equivalent" — a stack of libraries, runtimes, compilers, and tools that lets developers write and run GPU compute on AMD hardware. PMTS engineers in the compute team work somewhere in this stack.

The big picture

```
Application: PyTorch / TensorFlow / JAX / Blender / DCC apps / scientific code
                  |
                  v
        ┌──────────────────────────┐
        | Frameworks integration:  |  PyTorch ROCm backend, XLA on ROCm
        | MIOpen, hipBLASLt, RCCL  |
        └──────────────────────────┘
                     |
                     |
                     v
        ┌──────────────────────────┐
        |  HIP runtime             |  CUDA-API-shaped C++ for AMD
        |  (libamdhip64.so)        |
        └──────────────────────────┘
                     |
                     |
                     v
        ┌──────────────────────────┐
        |  ROCr / HSA runtime      |  HSA agents, queues, signals
        |  (libhsa-runtime64.so)   |
        └──────────────────────────┘
                     |
                     |
       Doorbells / KFD ioctls
                     |
                     v
        ┌──────────────────────────┐
        |  amdgpu kernel + amdkfd  |
        └──────────────────────────┘
                     |
                     |
                     v
                  GPU silicon
```

Plus a forest of libraries:

Library	Role
rocBLAS	dense linear algebra (GEMM, BLAS levels 1-3)
hipBLASLt	newer GEMM with offline-tuned solutions
rocSPARSE	sparse linear algebra
rocFFT	FFT
rocSOLVER	linear solvers (LU, QR, SVD)
MIOpen	DNN primitives (conv, pool, batchnorm)
RCCL	NCCL-shape collectives over xGMI/IB/PCIe
Composable Kernel	templated C++ kernel generators
ROCgdb / rocprofiler-sdk / omniperf / omnitrace	debug + perf tools

HIP — the CUDA-shaped API

HIP is C++ with kernel launch syntax very close to CUDA:

```
__global__ void saxpy(float a, const float *x, float *y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] += a * x[i];
}

int main() {
    float *dx, *dy;
    hipMalloc(&dx, N*sizeof(float));
    hipMalloc(&dy, N*sizeof(float));
    hipMemcpy(dx, hx, N*sizeof(float), hipMemcpyHostToDevice);
    saxpy<<<(N+255)/256, 256>>>(2.0f, dx, dy, N);
    hipMemcpy(hy, dy, N*sizeof(float), hipMemcpyDeviceToHost);
    hipFree(dx); hipFree(dy);
}
```

`hipify` is a tool that translates CUDA source to HIP source — usually an automated `s/cuda/hip/` plus a few semantic fixes. AI frameworks rely on this.

Key semantic differences vs CUDA:

- Wave size 64 (CDNA) or 32 (RDNA) — not always 32 like NVIDIA. Reductions/shfl semantics shift.
- No `__syncwarp` (use `__builtin_amdgcn_wave_barrier()` or HIP equivalents).
- Cooperative groups subset.
- Different driver APIs for stream/event semantics in some edge cases.

hipcc and the compiler chain

```
*.hip → hipcc (driver) → clang (with -x hip)
                         → produces both host code (compiled normally)
                         and device code (compiled to AMDGCN)
                         linked into a fat binary

Device code path:
  *.hip → frontend (clang) → LLVM IR → AMDGPU backend → AMDGCN ISA
                                                      → ELF inside the host binary
```

At runtime, ROCr extracts the AMDGCN code object for the matching gfx target, loads it, hands a kernel object pointer to dispatch.

The AMDGPU backend is shared between graphics and compute (chapter 81).

Streams and events

Like CUDA, HIP has streams (queues) and events (signals).

- A stream is an HSA queue.
- A default stream is created per-thread (or per-context, configurable).
- Events are HSA signals.
- `hipStreamSynchronize`, `hipEventRecord` map straight to HSA primitives.

Async memcpy uses SDMA queues internally — the runtime selects the right engine.

Multi-GPU

Multi-GPU on AMD uses:

- **xGMI / Infinity Fabric** for in-package or in-board GPU-GPU links (high bandwidth).
- **PCIe peer-to-peer** for cross-board.
- **RDMA over IB / RoCE** for cross-node clusters.

RCCL (`librccl.so`) implements:

- AllReduce
- AllGather
- ReduceScatter
- Broadcast
- AlltoAll
- ReduceScatter / All-to-All-v
- Send/Recv (point-to-point)

It picks topology-aware algorithms (rings, trees) and stages buffers via SDMA. PyTorch DDP / DeepSpeed / Megatron all rely on it.

hipBLASLt and rocBLAS

GEMM is the most-tuned kernel in the world. Both libraries ship pre-tuned solutions for many shapes, indexed by (M, N, K, dtype, layout, gfx-arch).

hipBLASLt adds an offline solution-tuning workflow: an app records its hot shapes, you run a tuner, the tuner picks per-shape MFMA configs, fused activations, persistent kernels — all stored in a tuning DB. Then production runs hit the tuned path.

When PyTorch on AMD beats expectations on a model, it's usually because hipBLASLt has the shapes already tuned.

MIOpen

Equivalent of cuDNN. Convolution, pooling, batchnorm, dropout, RNNs. Has multiple convolution algorithms (direct, GEMM, Winograd, implicit-GEMM, FFT) and a chooser that picks per (input shape, filter shape, layout, dtype). Auto-tuning at first run + cache.

Composable Kernel (CK)

A C++ template metaprogramming library. You instantiate templates of "an MFMA-based GEMM with these tile sizes, this fusion, this prologue/epilogue," and CK emits an optimized HIP kernel. Used by AMD library teams to scale tuning effort across many ops without writing N×M hand-kernels.

If you want to write a *fast* fused kernel and don't want to touch ISA, CK is the tool.

PyTorch on ROCm

PyTorch's CUDA backend is replicated as a ROCm backend — same Python API, same `torch.cuda.*` (yes, really named "cuda" even on AMD for compatibility), but underneath calls HIP runtime, hipBLASLt, MIOpen, RCCL.

A ROCm-aware PyTorch wheel is provided by AMD or community. Major models work out of the box on MI300; performance for some small / unusual shapes may need hipBLASLt tuning.

Triton, JAX, TF, XLA

- **Triton**: JIT compiles Python tile-language to GPU. AMD upstream support is improving each release; targets MFMA on CDNA.
- **JAX**: via XLA, which has an AMDGPU backend.
- **TF**: ROCm builds maintained.

PMTS-level engineers spend cycles ensuring these toolchains pick MFMA, avoid unnecessary copies, choose the right layouts, and don't fall off cliffs.

Debugging compute

- **rocgdb**: GPU-side gdb. Stop a kernel mid-flight, inspect VGPRs, step.
- **rocprof / rocprofiler-sdk**: hardware counters per kernel.
- **omnitrace**: kernel timeline + CPU profile interleaved.
- **omniperf**: roofline, occupancy, cache hit views.

We dive into these in chapter 86.

Production realities

- **Driver/firmware version coupling**: ROCm release N expects amdgpu kernel ≥ X and firmware ≥ Y. Mismatch = breakage. Documentation of compat matrices is critical.
- **Container support**: ROCm ships container images; users mount `/dev/kfd` and `/dev/dri/render*`. Permissions/cgroups matter.
- **Reset behavior**: a process hang that triggers GPU reset kills everyone on that GPU. Multi-tenant scheduling needs to handle this.

What you should be able to do after this chapter

- Sketch the ROCm stack from app to kernel.
- Distinguish ROCr, HIP, hipBLASLt, MIOpen, RCCL.
- Explain how a HIP kernel becomes machine code.
- Outline RCCL's role.
- Know which tools to reach for when a workload is slow.

Read deeper

- ROCm documentation (rocm.docs.amd.com) — tutorial-quality.
- ROCr-Runtime, HIP, hipBLASLt, MIOpen, RCCL repos on github.com/ROCm.
- Composable Kernel readme + a couple of sample kernels.
- AMD blog “ROCm fundamentals” series.
- Latest MI300X performance papers and HotChips talks.

Chapter 77 — Display Core (DCN) deep dive

The display side of an AMD GPU is its own world: a real-time pixel pipeline running independently of compute/graphics, with its own microcontroller, its own clock domains, and a giant codebase (`drivers/gpu/drm/amd/display/`). This chapter is the high-level map.

What "display" actually does

Once a frame is rendered to a framebuffer, the **display engine** must:

1. Read the framebuffer at the right pixel rate (e.g., 4K@120Hz = ~1.06 GP/s).
2. Apply scaling, blending of multiple planes (cursor, video overlay, UI).
3. Apply per-pixel transformations (color space conversion, gamma, HDR tone-mapping).
4. Send pixels out a physical link (DisplayPort, HDMI, eDP, VGA-equivalent legacy).
5. Manage timing (HSync, VSync, blanking intervals).
6. Coordinate hot-plug detection, link training, power states.

All in real time, with hard deadlines: missing a vblank = visible glitch.

DCN block diagram (RDNA / CDNA-A era)

```
                  Memory (framebuffer in VRAM)
                          |
              +-----------+
              | MMHUB / GFXHUB read clients (HUBP / DCHUBBUB)
              |
              v
+--------+     +--------+     +--------+
|  DPP   |     |  DPP   |     |  DPP   |    DPPs - per-plane processors
|        |     |        |     |        |    (scaling, color conv, gamma)
+--------+     +--------+     +--------+
     \             |             /
      \            |            /
       v           v           v
              +---------+
              |   MPC   |    Multi-Plane Controller (blending, OVL)
              +----+----+
                   |
                   v
              +---------+
              |   OPP   |    Output Pixel Processing (final color, dither)
              +----+----+
                   |
                   v
              +---------+
              |   OPTC  |    Output Pipe Timing Controller (timing gen)
              +----+----+
                   |
                   v
              +---------+
              |   DIO   |    Display I/O (link encoder, audio mux)
              +----+----+
                   |
```

```
                v
        +---------------+
        | Stream encoder|   DP / HDMI / eDP transport
        +---------------+
                |
                v
            physical pins
```

Plus:

- **DCCG**: DC Clock Generator. Produces pixel clocks, link clocks.
- **DMCUB**: a microcontroller running display firmware ("DMUB"); offloads PSR (panel self-refresh), brightness control, IRQ handling, etc.
- **DSC**: Display Stream Compression block (visually-lossless compression for high-bandwidth modes).
- **HPD**: Hot-Plug Detect lines.

Pipes

A "pipe" is a chain HUBP → DPP → MPC → OPP → OPTC → DIO bound to one display. Modern GPUs have 4–6 pipes; you can scan out to 4–6 displays simultaneously.

Pipe split: a single high-resolution display (e.g., 8K) can use *multiple* pipes in parallel, splitting the frame horizontally to meet bandwidth. Display Core decides this dynamically based on the validation engine's bandwidth math.

Planes

A plane is one input layer into MPC blending. Typical planes per pipe:

- 1 primary plane (full screen, the desktop).
- 0–N overlay planes (video, cursor, secondary UI elements).
- 1 cursor plane.

Overlay planes save memory bandwidth and allow YUV video to be scanned out without conversion via the GPU shader (DPP does the conversion in flight).

Atomic KMS exposes planes as `drm_plane`s; userspace compositors use them for low-latency video playback and "client-mode" cursors.

Atomic modeset, in DC's view

When userspace calls `drmModeAtomicCommit`:

1. DRM core builds an atomic state.
2. amdgpu_dm validates the state: can these planes/CRTCs actually work together at these rates?
3. If yes, DC's **validate engine** (`dcn*_resource.c`, `dml.c` — the famous Display Mode Library) computes detailed bandwidth/clock requirements.
4. If validate passes, DC programs all the registers to switch to the new state, atomically with vblank if requested.

DML is hundreds of formulas implementing AMD's pixel-bandwidth model. Wrong DML = pipe under-runs (display goes black, glitches). Production-quality DML is *extremely* hard.

Modes, timings, links

A mode is a tuple: (resolution, refresh, scan polarity, vblank size, link rate, lane count, color depth, color space, ...).

For DisplayPort:

```
1080p60 SDR  → ~6 Gbps payload  → 4 lanes × HBR (2.7 Gbps) is plenty
4K60  HDR   → ~17 Gbps          → 4 lanes × HBR2 (5.4) or 4 × HBR3 (8.1)
4K120 HDR   → ~32 Gbps          → DP 1.4 + DSC, or DP 2.0 UHBR
8K60  HDR   → ~50 Gbps          → DP 2.0 UHBR (10/13.5/20)
```

Link training: the source and sink negotiate the highest stable lane count + symbol rate. Failure modes are many — AMD-FreeSync or specific TVs at 4K120 are common pain points.

EDID / DisplayID

The monitor advertises its capabilities through **EDID** (a 128-byte blob, sometimes with extensions) over the DDC line at hot-plug. DC parses EDID to know modes, color spaces, HDR metadata, audio capabilities. **DisplayID** is the modern, larger replacement.

EDID parsing has 30 years of bugs and quirks. AMD has tables of monitor quirks in `drm_edid.c` / `drm_displayid.c` and amdgpu-specific overrides.

HDR and color

DC supports:

- **Color spaces**: sRGB, Rec.709, Rec.2020, P3.
- **HDR transfer functions**: PQ (HDR10), HLG.
- **3D LUTs** for color management — applied per-plane in DPP.
- **Tone mapping**: usually compositor-side or display-side, some hardware acceleration via DPP.

Color management ABI: per-plane / per-CRTC properties — `IN_FORMATS`, `COLOR_ENCODING`, `HDR_OUTPUT_METADATA`, etc. Userspace (KWin, weston, monado) reads these and configures.

Variable refresh (FreeSync / VRR / Adaptive Sync)

The display can be told “refresh whenever the GPU has a new frame” rather than every 16.67 ms. Implemented over DP/HDMI VRR. DC programs the OPTC to allow extending the vblank until a flip arrives.

Driver involvement:

- Detect VRR-capable panel from EDID/DisplayID.
- Expose VRR property to userspace.
- Handle “min/max refresh” range; clamp frame timing in compositor.
- Avoid jittering between refresh-locked modes.

PMTS-level work: chasing FreeSync flicker bugs that only manifest on specific panels at specific brightness levels.

DSC (Display Stream Compression)

For modes that exceed link bandwidth, DSC compresses ~3:1 with visually lossless quality. Pipeline:

```
DPP output → DSC encoder in DCN → compressed stream over DP → DSC decoder in monitor
```

DSC is per-pipe; resource validation accounts for DSC slices. Programming it has many tiny knobs (slice height, bits-per-pixel, line buffer config).

DMCUB — display firmware microcontroller

DMUB firmware runs on DMCUB, handles:

- IRQ steering (HPD, vblank, pflip).
- PSR1/PSR2 (panel self-refresh, eDP only — saves power on idle frames).
- Brightness control (eDP backlight).
- Some atomic update sequences (microcontroller commits to silicon at exact timing).

You'll see `dmub_*.c` files in the DC tree. Updating DMUB firmware is part of the GPU firmware load (PSP-signed).

Audio

DCN handles audio over DP/HDMI: pulls audio samples from PCM buffers, interleaves into the display data stream. The Linux audio side is `snd-hda-amdgpu` or `snd-hda-codec-hdmi`. Drivers cooperate over a small ALSA-DRM bridging interface.

Reset and recovery

Display reset is **independent** from GFX reset where possible. If GFX hangs, the user should still see the screen. DC's validate engine and dependency tracking enables this.

A full PCIe-level link reset takes everything; finer mode-1/mode-2 resets might keep DC alive.

Driver responsibilities (massive)

`drivers/gpu/drm/amd/display/` is one of the largest subdirectories of the kernel. Roughly:

- `dc/` — vendor-shared display core (no Linux-isms).
- `dc/dcn*/` — per-DCN-version specific code.
- `dc/dml/` — display mode library (math).
- `amdgpu_dm/` — Linux DRM bridging code (atomic helpers, plane/CRTC drivers).

The bring-up of a new chip's display is a months-long multi-team effort: DC engineers + ASIC + firmware + Linux integration. DC is typically maintained by a dedicated AMD team.

What you should be able to do after this chapter

- Sketch the DCN pipeline.

- Define DPP, MPC, OPP, OPTC, DIO, DCCG, DMCUB, DML.
- Walk through atomic modeset → DC validate → register program.
- Define DSC and PSR.
- Identify VRR / FreeSync pieces.

Read deeper

- AMD GPUOpen blog: “Introduction to Display Core.”
- `drivers/gpu/drm/amd/display/dc/dml/` — DML formulas.
- `drivers/gpu/drm/amd/display/amdgpu_dm/amdgpu_dm.c` — main bridge.
- DRM atomic KMS docs (`Documentation/gpu/drm-kms.rst`).
- DisplayPort 2.0, HDMI 2.1, EDID, DisplayID specs.

Chapter 78 — Multimedia (VCN), codecs

The GPU is also a video processor. AMD GPUs have a dedicated **VCN (Video Core Next)** block that decodes and encodes video at hardware speed — many gigapixels per second per stream, dozens of streams concurrently.

Multimedia is its own corner of the driver, with codec details, latency requirements, and DRM (digital-rights-management) interactions.

What VCN does

A VCN block is a fixed-function + programmable engine that handles:

- **Decode**: H.264/AVC, H.265/HEVC, AV1, VP9 (legacy MJPEG on older parts).
- **Encode**: H.264, H.265, AV1 (depending on gen).
- **JPEG decode** (separate JPEG engine on most parts).

Each codec is a hardware pipeline implementing the standard's bitstream parsing, motion compensation, transforms, quantization, entropy (CABAC/CAVLC/range), in-loop filtering.

VCN generations

VCN	Year	Codecs
VCN 1.x	Vega (2017–)	H.264, HEVC dec/enc, VP9 dec
VCN 2.x	Renoir, Navi 1x	+ AV1 dec
VCN 3.x	Navi 2x	+ improved AV1 dec
VCN 4.x	Navi 3x, MI300	+ AV1 enc
VCN 5.x	Navi 4x	improved AV1 enc, dual streams

Datacenter parts (CDNA) often have multiple VCN instances per chip for transcoding farms.

How decode works

The application (e.g., Firefox, mpv, VLC):

1. Demuxes the container (mp4, mkv, …) → bitstream buffers.
2. Parses bitstream headers (sequence parameter set, picture parameter set, slice header) — usually in software.
3. For each frame, builds a **picture parameter buffer** describing references, motion vectors, slice data.
4. Submits decode work via VA-API or VDPAU.
5. The userspace driver (Mesa's `radeonsi` / VAAPI) translates to PM4 and SDMA writes to VCN's compute-style engine.

6. VCN reads bitstream + reference frames, decodes, writes YUV output to a buffer.
7. Output buffer is composited onto display (via DPP YUV→RGB conversion) or fed into a downstream pipeline.

How encode works

Encode is harder — encoder must make rate/quality trade-offs in real time.

1. App provides raw YUV input frames.
2. Userspace driver (radeonsi VAAPI encoder) configures rate-control, GOP structure, target bitrate.
3. Submits frames to VCN encode pipeline.
4. VCN computes motion vectors against references, transforms, quantizes, entropy-codes, emits a bitstream.
5. App writes bitstream to file or streams it.

VCN encode is used for screen recording (OBS), game streaming (Twitch/Discord overlays), low-latency desktop sharing (Parsec, AMD ReLive), Steam Remote Play, transcoding pipelines.

Latency targets

Use case	End-to-end latency	Notes
Cloud gaming	< 50 ms	sub-frame encode, low-delay GOP
Game streaming local	10–20 ms	low-delay AVC/HEVC, slim references
Transcoding	best-effort	optimize for throughput
Screen capture	16–30 ms	balanced
Real-time video calls	100–200 ms	usually CPU; VCN can help

Achieving very-low-latency encode is constrained — small intra-period, no B-frames, slim references; lower compression efficiency.

VA-API and the userspace stack

```
mpv / OBS / Firefox / FFmpeg
        |
   libva  (VA-API)
        |
  vdpau / vaapi user driver (Mesa: src/gallium/drivers/r600/, src/gallium/winsys/amdgpu/, src/
gallium/frontends/va/)
        |
    libdrm_amdgpu
        |
    /dev/dri/card*
        |
     amdgpu kernel driver
        |
        VCN engine
```

The VAAPI driver builds VCN command streams (PM4 packets specific to VCN: `RBBM_REGISTER_*`, decoded-by-firmware indirect buffers). It manages reference picture lists, allocates DPB (Decoded Picture Buffer) memory, handles error recovery.

DRM (digital rights management)

Protected content (Netflix 4K HDR, etc.) requires:

- A **trusted application (TA)** running in the PSP — verifies firmware, holds keys.
- An encrypted bitstream that only the TA can decrypt.
- Decode that *never* exposes plaintext to host RAM. The decoded buffer is in a "secure" region.
- Display path enforces HDCP — the link to the monitor is encrypted.

Linux's content-protection support is rudimentary; full Widevine / PlayReady L1 paths are limited. AMD's mainline kernel supports the underlying primitives; full DRM-laden playback usually requires Windows or specific Chrome OS configurations.

PMTS engineers in this area work with content providers, OEMs, and standards bodies. Lots of NDA layers.

DPB management

Modern codecs reference **multiple previous (and sometimes future) frames** for prediction:

- H.264: up to 16 reference frames.
- HEVC: up to 16 short-term + 16 long-term.
- AV1: explicit reference signaling, up to 7.

The driver/runtime must allocate and recycle these reference frame buffers, flag their lifetimes correctly, and ensure VCN reads exactly the right frames for each decode step. DPB bugs cause subtle visual artifacts ("ghosting," "color bleed").

JPEG engine

Many parts have a separate JPEG decoder. Useful for thumbnail decoding in image-heavy workloads (web browsing, photo apps), or for AI inference pipelines with image input. Programmable similarly to VCN but a smaller engine.

VCN driver pieces in kernel

`drivers/gpu/drm/amd/amdgpu/`:

- `vcn_v1_0.c` … `vcn_v5_0.c` — per-VCN-version IP block code (set up rings, interrupts, firmware load).
- `amdgpu_vcn.c` — common VCN logic, ring handling.
- `jpeg_v*.c` — JPEG engine setup.

`amdgpu_vcn.c` exposes `vcn_dec_ring` and `vcn_enc_ring` to the scheduler; UMD submits IBs through these the same way GFX rings work.

Cross-IP coordination

VCN cooperates with:

- **GFX/compute** for any post-processing (denoise, tone-mapping) that's not VCN-resident.
- **SDMA** for buffer copies in/out.
- **DCN** for direct scan-out of decoded YUV (no copy through compositor).
- **PSP** for protected-content keys.

Power management for VCN

VCN is power-hungry when active and gateable when idle:

- Runtime PM auto-suspends VCN when no decode/encode for ~seconds.
- DMUB or RLC handles clock-gating.
- Encoded streams have hard latency budgets, so wake-up time matters.

Testing and validation

- **Bitstream conformance**: AMD validates against codec-spec test vectors.
- **Real content**: AAA games, streaming services, broadcast-grade test patterns.
- **Stress**: many concurrent streams, long-running transcode farms.
- **Inter-vendor interop**: encoded content must play on every device.

When a customer reports "AV1 stream stutters in Firefox on Linux at 4K," that bug starts here.

What you should be able to do after this chapter

- Identify VCN gen on a given chip.
- Sketch decode and encode flows.
- Define VA-API and Mesa's role.
- Describe DPB and why it matters.
- Reason about latency vs quality vs bitrate trade-offs.

Read deeper

- `drivers/gpu/drm/amd/amdgpu/vcn_v*_0.c`.
- `mesa/src/gallium/frontends/va/` — Mesa VAAPI implementation.
- VAAPI (libva) docs.
- AV1 / HEVC / H.264 specs (ITU-T / ISO/IEC).
- AMD AMF (Advanced Media Framework) documentation for cross-platform API.
- OBS Studio source as a real-world consumer example.

Chapter 79 — SMU, DPM, voltage/frequency, telemetry

The GPU's clocks aren't fixed. They scale up under load, down at idle, and adjust per-IP block all the time. The thing controlling them is the **SMU (System Management Unit)** — a microcontroller running AMD-signed firmware that's effectively the GPU's autonomic nervous system.

PMTS engineers in power management spend years tuning what SMU does.

What SMU manages

- **Clock frequencies** for every clock domain on the chip:
  - GFXCLK (shader engine clocks)
  - SOCCLK (uncore / interconnect)
  - MEMCLK (HBM/GDDR clock)
  - FCLK (Infinity Fabric)
  - VCN clocks
  - DCN pixel clock generators
  - PCIe link state
- **Voltages** for each rail (VDDGFX, VDDSOC, VDDMEM, …).
- **Power consumption budgeting** to stay within TGP (Total Graphics Power) and TBP (Total Board Power).
- **Thermal management** — fan speed curves, thermal throttling.
- **Telemetry** — exposes current clock/voltage/temp/power readings to the kernel.
- **Boot sequencing** — brings up clocks in order during init.

DPM — Dynamic Power Management

DPM is the algorithm SMU runs to pick clocks. Inputs:

- Activity counters per IP (how busy is GFX? VCN? DCN?).
- Temperature sensors (junction temp, edge temp, hotspot, mem temp).
- Power readings (estimated at chip + measured at board if available).
- User policy ("performance," "balanced," "power saver").

Output: a (frequency, voltage) operating point for each clock.

DPM is implemented as a small state machine with hysteresis to avoid clock thrashing. SMU updates clocks at sub-millisecond rates.

OPPs and DPM tables

A list of allowed (frequency, voltage) pairs lives in firmware as a "DPM table." There are typically 8 levels per clock — DPM_0 (lowest) through DPM_7 (peak). Levels are hardware-binned per chip — silicon at the high end can hit DPM_7 cleanly; weaker silicon may cap at DPM_6.

```
GFXCLK DPM table:
   DPM_0: 500 MHz, 0.65 V
   DPM_1: 800 MHz, 0.70 V
   DPM_2: 1100 MHz, 0.75 V
   ...
   DPM_7: 2700 MHz, 1.05 V
```

Userspace sees these via `/sys/class/drm/card*/device/pp_dpm_*`:

```
        cat /sys/class/drm/card0/device/pp_dpm_sclk
        0: 500Mhz
        1: 800Mhz
        ...
        7: 2700Mhz *     ← * = currently active
```

You can write to these files to force a level (force_performance_level=manual) for testing.

Telemetry

`rocm-smi` reads SMU telemetry over a kernel interface (`hwmon` plus AMD-specific paths). Available metrics:

- Junction temperature (Tj_max usually 105–110 °C; throttle starts before that).
- Edge temp.
- HBM temp (CDNA only).
- GFX activity %.
- Memory activity %.
- Power draw (W).
- Fan RPM.
- ECC counts.

Production monitoring (Prometheus exporters, Datadog) scrape these.

SVI3 and serial voltage

Modern AMD GPUs use **SVI3** (Serial Voltage Identification, version 3) — a serial protocol between SMU and the on-board voltage regulator. SMU sends "set rail X to Y mV"; the regulator complies. Faster transitions, finer granularity.

Old SVI2 / SVI1 were slower, coarser. Datacenter AMD GPUs may have multiple regulators per rail for current density.

Thermal management

SMU monitors:

- **Tj** (junction): from on-die diodes. The most critical — controls throttle.

- **Tedge**: PCB edge temp.
- **Thotspot**: max of all on-die diodes.
- **Tmem**: HBM temp.
- **Tboard**: chassis sensor.

Policy:

- Below threshold: free-running.
- Threshold reached: lower DPM level. Fans speed up.
- Tj_max: emergency clock cuts.
- Critical: shutdown.

Fan curves are programmable but constrained by AMD's safe envelope. OEMs (board partners) tune their own curves on top.

Power capping

Each chip has:

- **TGP** (Total Graphics Power) — the GPU's headline TDP target.
- **TBP** (Total Board Power) — including memory and VRMs.
- **PPT** (Package Power Tracking) — measured estimate.
- **EDC** (Electrical Design Current) — current limits.
- **TDC** (Thermal Design Current).

SMU continuously balances clocks against these so the chip never exceeds its envelope. If a workload would push past PPT, clocks drop until the budget fits.

Customer/ISV-specific tuning

Some ISVs (e.g., Blender, professional CAD) request bespoke DPM behavior — minimum sustained clocks, less aggressive idle. The driver may tag a process as "compute" vs "graphics" via context flags and feed that to SMU.

PMTS engineers field weird requests: "our app on MI250 runs slower at 70 °C than 80 °C" — yes, because at 70 °C SMU is in a transitional DPM band. Investigate, fix.

Power features per platform

- **Mobile / APU**: SMU coordinates with PMU for shared TDP between CPU and GPU. STAPM (Skin Temperature-Aware Power Management) limits long-running power based on chassis temp.
- **Desktop**: GPU has its own envelope; PCIe slot limits separately enforced.
- **Datacenter (CDNA)**: tighter constraints, ECC mandatory, often no fan (passive, server cools the slot).

RuntimePM in the kernel

Kernel-side, runtime PM (`pm_runtime_*`) interacts with SMU:

- When the GPU is idle, kernel suspends → SMU drops to deep idle.

- Wake on activity (a draw call, a poll) → SMU spins clocks up.
- Mobile parts have additional **PX (PowerXpress)** or **D3Cold** sleeps where the whole GPU is power-gated, only PSP/SMU stays on.

Suspend/resume bugs are common: a discovery sequence times out, SMU misses an ACK, GPU comes back broken. Lots of debugging life is here.

SMU interface — kernel side

The kernel talks to SMU through:

- A doorbell-style mailbox in MMIO.
- Message + arg + response protocol: kernel writes message ID, SMU responds with status.

`drivers/gpu/drm/amd/pm/swsmu/` has per-chip SMU drivers. Each chip family has its own message numbers, table layouts, and quirks.

```
        amdgpu_dpm_set_soft_freq_range(adev, AMD_SMU_GFXCLK, min, max);
        amdgpu_dpm_get_clk_freq_by_index(adev, ...);
```

These wrap message exchanges.

SMU firmware updating

SMU firmware ships in `*_smc.bin`; PSP loads it during boot. New firmware fixes regulator timing, adds new clock states, or addresses errata. Out-of-tree distros sometimes lag firmware → people complain about “fan ramping wildly” until firmware updates.

What goes wrong

- DPM oscillates between levels (hysteresis bug) → audible coil whine + jitter.
- Tj reading wrong → either over-throttle or thermal damage.
- Sleep/wake races → corrupted SMU state requiring full reset.
- Power-cap miscalibrated → underperformance even with thermal headroom.
- Voltage transitions glitch under load → silicon errors / hangs.

What you should be able to do after this chapter

- Define SMU and DPM in one sentence each.
- Explain TGP/TBP, telemetry sources.
- Read `pp_dpm_sclk` and interpret it.
- Reason about thermal vs power throttling.
- Sketch the kernel↔SMU message flow.

Read deeper

- `drivers/gpu/drm/amd/pm/swsmu/smu13/smu_v13_*.c` — recent gen.
- `drivers/gpu/drm/amd/pm/inc/smu_*.h` — table structures.

- AMD GPUOpen: “Power Management on Radeon.”
- `rocm-smi` source for the telemetry queries.
- HotChips slides on “AMD power architecture” (any year).

Chapter 80 — Mesa, radv, ACO, NIR

If the kernel is half of an AMD GPU stack, **Mesa** is the other half. It's the open-source OpenGL/Vulkan/OpenCL implementation. For AMD, the marquee components are `radeonsi` (OpenGL), `radv` (Vulkan), and **ACO** (a hand-rolled compiler). PMTS engineers in graphics live here.

Mesa structure (quick map)

```
mesa/
├── src/
│   ├── compiler/
│   │   ├── glsl/        ← GLSL frontend
│   │   ├── nir/         ← NIR (Mesa's portable IR) — main IR
│   │   └── spirv/       ← SPIR-V frontend
│   ├── amd/
│   │   ├── common/      ← shared AMD code (NIR lowering, surfaces, AC reg model)
│   │   ├── compiler/    ← ACO compiler
│   │   ├── llvm/        ← LLVM AMDGPU backend frontend (alternative to ACO)
│   │   ├── vulkan/      ← radv (Vulkan)
│   │   └── ci/          ← per-target test rules
│   ├── gallium/
│   │   ├── drivers/
│   │   │   ├── radeonsi/   ← OpenGL/EGL/OpenCL on AMD
│   │   │   └── ...
│   │   ├── frontends/      ← VAAPI, VDPAU, OpenCL frontends
│   │   └── winsys/amdgpu/  ← libdrm_amdgpu glue
│   └── intel/, broadcom/, freedreno/, etc.
```

The split: `gallium` is a state-tracker framework for OpenGL/OpenCL drivers; `radv` is a standalone Vulkan driver. Both call into `src/amd/common/` and the compiler.

NIR — Mesa's IR

NIR (NIR Intermediate Representation) is Mesa's portable shader IR, used by every driver. Frontends (GLSL, SPIR-V, HIPCC) emit NIR; backends consume it.

NIR features:

- SSA form for vector and scalar values.
- Block-structured control flow (`if/loop/break/continue`).
- Generic intrinsics for things like `image_load`, `barrier`, `vote`.
- Lowering passes: convert high-level ops into target-friendly ops.

Key NIR passes for AMD:

- `nir_lower_io`: split stage outputs/inputs into per-component locations.
- `nir_lower_subgroups`: implement subgroup ops (ballot, shuffle, vote).
- `nir_opt_dce`, `nir_opt_cse`, `nir_opt_loop_unroll` — generic optimizations.
- `ac_nir_lower_ngg`: AMD-specific NGG lowering.
- `ac_nir_lower_resinfo`, `ac_nir_lower_image`: AMD-specific image-fetch lowering.

ACO (and the LLVM backend) consume the final NIR and produce machine code.

ACO — the AMD-compute optimizer

ACO is a from-scratch shader compiler written for AMD targets. Started as a community/alternative to LLVM (which has a huge surface area and slower compile). ACO focuses on:

- Fast compile time.
- Good register allocation, especially for wave32.
- AMD-specific IR (`aco_ir.h`).
- Generates AMDGCN/RDNA ISA directly (no LLVM dependency).

ACO is the default for radv. The LLVM backend remains as a fallback (and is mandatory for some cases — e.g., RT shaders, which require complex CFG transforms ACO doesn't yet have).

ACO IR

ACO's IR sits between NIR and AMDGCN ISA. Each instruction is one of:

- **VOPx** (vector op forms): `v_add_f32`, `v_mul_f32`, `v_mfma_*`, etc.
- **SOPx** (scalar op forms).
- **SMEM, MUBUF, MIMG, FLAT** load/store forms.
- **EXP** (export — vertex outputs / pixel outputs).
- Pseudo-ops for spills, copies, parallel-copies during regalloc.

Compile pipeline (simplified):

```
NIR → instruction selection → live-range analysis → register allocation
    → scheduling → assembly → ELF
```

Why ACO matters

- 30–50% faster compile times than LLVM in many cases (matters for shader compile stutter in games).
- Can produce equally good or better code on common shader shapes.
- AMD-aware — knows about wave size, NGG, MFMA, mode bits.
- Easier to extend by Mesa contributors than LLVM.

PMTS-level work in ACO: improving regalloc, adding new MFMA forms, lowering new Vulkan extensions, fixing pathological shaders.

radv — Vulkan on AMD

radv implements the Vulkan API:

- `vkCreateDevice` etc. → opens `/dev/dri/renderD*`.
- Pipeline state objects (PSOs) → compile + layout shaders into ISA.
- Descriptor sets → AMD descriptor format (image/sampler/SSBO descriptor structs in 32-byte blobs).
- Command buffers → assemble PM4 streams.
- Submit → `amdgpu_cs_submit` ioctl.
- Queues, fences, semaphores → drm_syncobj / dma_fence.

radv is impressively fast. Several AAA games run *better* on radv than on AMDVLK (AMD's official Vulkan), because radv's compiler and lower-overhead command-buffer building win on workloads that hammer the API.

radv source

File	Role
`radv_device.c`	device init, capability query
`radv_pipeline*.c`	pipeline (PSO) creation, shader compile entry
`radv_cmd_buffer.c`	recording PM4
`radv_descriptor_set.c`	descriptor management
`radv_meta_*.c`	"meta" passes for clears, decompresses
`radv_image.c`	DCC/HTILE/CMASK setup
`radv_acceleration_structure.c`	RT BVH builder

radeonsi — OpenGL on AMD

radeonsi implements OpenGL 4.6, OpenGL ES, EGL on top of Gallium. Compiler = LLVM AMDGPU backend (older code path; some pieces use ACO via `amd/llvm`).

OpenGL is much harder to make fast than Vulkan because of API state changes and validation. radeonsi has years of legacy and is genuinely impressive code.

OpenCL on Mesa: rusticl

The new OpenCL frontend, written in Rust. Replaces the old `clover`. Uses NIR → ACO/LLVM. Aimed at filling the OpenCL gap for users who don't want full ROCm.

Compositor / WSI

WSI (Window System Integration) binds Vulkan/EGL outputs to Wayland or X11 surfaces. Mesa has WSI code in `src/vulkan/wsi/` (Vulkan WSI extensions) and EGL platforms.

WSI cooperates with KMS atomic on the kernel side. Direct scan-out from Vulkan to KMS planes (no compositor copy) is supported and increasingly used.

How a draw call becomes silicon (Mesa view)

```
1. App: vkCmdBindPipeline + vkCmdDraw
2. radv: append PM4 dwords to a command buffer BO
   - SET_CONTEXT_REG for blend, depth, viewport
   - SET_SH_REG for shader resources (descriptor sets)
   - DRAW_INDEX_2 packet
3. App: vkQueueSubmit
4. radv: submit BO list + IB to amdgpu_cs ioctl
5. Kernel: validate, schedule via DRM scheduler
6. CP runs IB on the silicon
```

PMTS-level optimization: minimize the work in step 2. Cache state vectors, hash them, reuse precomputed PM4 sequences. radv has many such tricks (`radv_emit_*` functions reuse cached state).

Testing

Mesa CI runs:

- `dEQP-VK` and `dEQP-GLES`: Khronos test suites (10s of thousands of tests).
- `piglit`: OpenGL test suite.
- `crucible`: Vulkan-specific tests.
- Game replay (Chrome's `vk_replay`).

CI runs on real AMD hardware in shared test farms (FreeDesktop CI). Failing a CI run blocks merge.

How to contribute

1. Find a small bug — check Mesa's GitLab issue tracker, filter `radv` label.
2. Clone Mesa: `git clone https://gitlab.freedesktop.org/mesa/mesa.git`.
3. Build: `meson setup build -Dvulkan-drivers=amd -Dgallium-drivers=radeonsi`; `ninja -C build`.
4. Reproduce, fix, test (run dEQP locally for the affected feature).
5. Submit MR on GitLab. Be ready for review iterations.

Mesa is one of the most welcoming open-source communities in graphics. Junior contributions land regularly.

What you should be able to do after this chapter

- Sketch the Mesa source tree.
- Define NIR, ACO, LLVM AMDGPU backend.
- Distinguish radv, radeonsi, rusticl.
- Describe a draw-call's path from API to ISA.
- Know where to find a specific feature (RT, DCC, descriptors).

Read deeper

- Mesa source (you must skim this).
- Bas Nieuwenhuizen's blog on radv internals.
- Connor Abbott's writings on ACO design.
- "Mesa 3D: Behind the scenes" FOSDEM talks (multiple years).
- `mesa/docs/` — surprisingly good architecture docs.

Chapter 81 — LLVM AMDGPU backend

LLVM is the compiler infrastructure used by Clang, Mesa (legacy radeonsi), and ROCm (HIP). Its **AMDGPU backend** generates AMDGCN/RDNA machine code from LLVM IR. Understanding it is required if you want to work on shader/compute compilers at AMD.

Why both LLVM and ACO?

Mesa's `radv` uses ACO. ROCm/HIP uses LLVM. Two reasons LLVM still dominates compute:

1. **Massive infrastructure**: LLVM has decades of optimization passes (loop transforms, vectorizers, SLP, register allocators, SSA-based passes, autovec). Hard to reimplement.
2. **Cross-vendor flow**: HIP, OpenCL, OpenMP offload all use LLVM. Reusing the same backend simplifies development.

Graphics moved to ACO because Vulkan compile latency matters. Compute is throughput-bound; LLVM's slower compile is OK because kernels are AOT-compiled.

LLVM source structure (AMDGPU side)

```
llvm-project/
└── llvm/lib/Target/AMDGPU/
    ├── AMDGPU.h                ← public-ish header
    ├── AMDGPU.td               ← TableGen — instruction definitions
    ├── AMDGPUSubtarget.cpp     ← per-chip features
    ├── SIInstructions.td       ← VOP, SOP, SMEM, MUBUF, etc. instructions
    ├── SIInstrInfo.cpp         ← instruction info
    ├── SIRegisterInfo.cpp      ← VGPR/SGPR allocation
    ├── SIShrinkInstructions.cpp ← compress 64-bit forms when possible
    ├── SIMemoryLegalizer.cpp   ← emits the right cache flush sequences
    ├── GCNHazardRecognizer.cpp ← inserts s_waitcnt and other waits
    ├── AMDGPUInstructionSelector.cpp
    ├── AMDGPULegalizerInfo.cpp
    ├── AMDGPULowerKernelArguments.cpp
    └── ...
```

Reading `SIInstructions.td` gives you the canonical list of ISA opcodes per RDNA/CDNA gen.

Compile flow

LLVM IR (LL or BC) → AMDGPU backend → ELF object file containing AMDGCN code.

Stages:

1. **IR-level passes**: GVN, CSE, LICM, loop-unroll, autovec, AMDGPU-specific lowering.
2. **Instruction Selection (ISel)**: lowers LLVM IR to MIR (machine IR), with virtual registers.
3. **Register allocation**: assigns VGPRs/SGPRs.

4. **Scheduling**: orders instructions to minimize stalls (latency, RAW/WAW hazards).
5. **Hazard recognition**: inserts `s_waitcnt`, NOPs.
6. **Memory legalizer**: inserts `buffer_wbinvl1`, `s_dcache_wb`, etc., per memory model.
7. **Final emission**: produces ELF.

Each stage has dozens of files and configuration knobs.

TableGen — the instruction database

`AMDGPU.td` and friends use **TableGen**, LLVM's declarative DSL, to describe:

- All instructions (mnemonics, encodings, operand types, latency, predicates).
- Calling conventions.
- Register classes.
- Patterns to match LLVM IR onto instructions during ISel.

A snippet of TableGen looks like:

```
def V_ADD_F32_e32 : VOP2_Pseudo<"v_add_f32",
    (outs VGPR_32:$vdst),
    (ins VSrc_f32:$src0, VGPR_32:$src1)>;
```

The build runs `llvm-tblgen` to generate C++ from these. PMTS-level work often involves modifying `.td` files when a new ISA gen lands.

Memory model handling

Compute kernels use C++ atomics with memory orders. LLVM's `SIMemoryLegalizer.cpp` translates these into AMD-specific sequences:

```
atomic load with acquire        → load + s_waitcnt + invalidate caches
atomic store with release       → wb caches + s_waitcnt + store
atomic_thread_fence(seq_cst)    → wb + invalidate + barrier
```

The exact sequence depends on the gfx target and memory scope. **Read this file once.** It is the operational definition of the AMD GPU memory model.

Wave size handling

LLVM tracks wave size (`-mattr=+wavefrontsize32` or `+wavefrontsize64`) per function or module. It influences:

- Vector register width (effectively).
- DPP/permute lane patterns.
- Active-lane mask handling.
- Subgroup/wavefront-level intrinsics.

Compute kernels on RDNA can pick wave32 for short kernels with high parallelism, wave64 for long kernels with high register pressure. The HIP frontend sometimes guesses; explicit `__attribute__((amdgpu_waves_per_eu))` forces.

Register allocation challenges

VGPRs are dynamically allocated per wave. The compiler picks the *minimum* count; the runtime tells the GPU. Allocating too few → spilling. Allocating too many → reduced occupancy.

LLVM uses a "sliding window" approach: try multiple register counts, pick the best estimated occupancy/perf trade.

Pathological cases:

- **Long basic blocks** with many live values → high VGPR pressure.
- **Loop-carried** values across many iterations.
- **Inline kernels with deep callees**.

Senior engineer work: re-reading allocator logs (`-Rpass=regalloc`), identifying spill culprits, restructuring kernels.

Optimization levels

Flag	What it does
`-O0`	no opts, slow code, fast compile, debuggable
`-O1`	basic
`-O2`	most game-relevant
`-O3`	aggressive (loop unroll, autovec)
`-Os`	size
`-Ofast`	`-O3 + -ffast-math` (relaxed FP semantics)

Most production HIP code uses `-O3`. Be careful with `-ffast-math`: it can change results subtly and cause divergent output between platforms.

AMDGCN-specific intrinsics

`__builtin_amdgcn_*` exposes raw ISA features. Examples:

- `__builtin_amdgcn_mbcnt_lo`, `_hi` — compute wave lane id.
- `__builtin_amdgcn_ds_bpermute` — DS permute (fast lane permute via LDS).
- `__builtin_amdgcn_mfma_*` — matrix engine.
- `__builtin_amdgcn_buffer_load_format_*` — typed buffer loads.
- `__builtin_amdgcn_image_*` — texture path.
- `__builtin_amdgcn_s_setreg_*` — set hardware control registers.

PMTS engineers use these regularly to wring every cycle out of a hot kernel.

Code object layout

LLVM emits an ELF (an "HSACO" — HSA code object) with sections:

- `.text` — machine code.
- `.rodata` — constants.
- `.note.amd.amdgpu` — metadata (wave size, kernel descriptors, kernel argument layout).

- `.debug_*` — debug info (if `-g`).

`roc-obj-ls`, `llvm-readelf`, and `llvm-readobj` inspect HSACOs:

```
clang -x hip mykernel.hip -O3 -emit-static-lib -o mykernel.a
roc-obj-ls -V mykernel.a
roc-obj-extract -t gfx942 mykernel.a   # extract for one target
llvm-readobj --notes extracted.hsaco
```

Multi-arch fat binaries

A HIP build typically targets multiple gfx archs in one binary (`gfx906;gfx908;gfx90a;gfx940;gfx942;gfx1030;gfx1100`). Each is a separate compilation; the runtime picks the right code object at load time based on the device.

The wrapper format is `clang_offload_bundle` magic. If you see weird "no compatible code object" errors, your binary doesn't include the running GPU's gfx target.

ROCm-LLVM vs upstream LLVM

AMD ships its own LLVM fork with a few extra patches (newer mfma intrinsics, faster turn-around on backend changes). The patches eventually upstream. As a contributor: prefer upstream; AMD will pick it up.

What you should be able to do after this chapter

- Identify the AMDGPU backend's location in LLVM.
- Define ISel, regalloc, scheduling, legalizer in one sentence each.
- Read TableGen at a glance.
- Use `__builtin_amdgcn_*` intrinsics deliberately.
- Reason about wave size, occupancy, spilling at the LLVM level.

Read deeper

- LLVM source: `llvm/lib/Target/AMDGPU/`. **Just read it.**
- LLVM docs: `llvm.org/docs/AMDGPUUsage.html`.
- AMD blog: "Compiler optimization on AMDGPU."
- Talks at LLVM dev meeting on AMDGPU topics (yearly).
- `llvm-mc`, `llvm-objdump`, `llvm-readobj` man pages — use them.

Chapter 82 — Vulkan & DX12 deep

Modern explicit graphics APIs (Vulkan, DirectX 12, Metal) push state management, synchronization, and memory layout decisions out of the driver and into the application. This *should* mean less work for the driver. In practice, it just means *different* work, and the surface area of "things the driver must do well" is enormous.

PMTS engineers in the graphics group think in Vulkan/DX12 idioms even when working on hardware.

Why explicit APIs

OpenGL/DX11 hid almost everything: state changes, memory placement, synchronization, descriptor management. Drivers became huge optimizers that figured out what apps "really meant." Vulkan/DX12 say: the application *tells* the driver exactly what it wants, no inference.

Wins:

- Lower per-call CPU overhead (fewer driver decisions).
- Multi-thread submission.
- Predictable behavior (same code = same perf across drivers).
- Fine-grained memory control.

Costs:

- Apps must manage many things they used to ignore (barriers, layouts, descriptors).
- Misuse → undefined behavior or hard-to-debug crashes.

Pipeline state objects (PSOs)

A PSO snapshots most of the rendering state into one object: shaders, vertex format, blend, depth, MSAA. Apps build these once, bind them many times. The driver compiles shaders during PSO build (slow); subsequent binds are cheap.

This is why many AAA games have a "compiling shaders" loading screen — they're warming the PSO cache.

PSO compilation cost is the **#1 driver perceived issue**. Mesa/radv ACO improvements over the years have largely targeted this.

Descriptor sets / descriptors / descriptor heaps

Resources (textures, buffers, samplers) are bound through **descriptors**. Each descriptor is a hardware-specific structure (32 or 64 bytes on AMD) describing the resource: address, size, format, swizzle, sampler state.

Vulkan models this with **descriptor sets** (groupings of descriptors); DX12 with **descriptor heaps + tables/root-descriptors**.

AMD reads descriptors via the **scalar load (SMEM)** path — a wave's scalar unit fetches descriptors that all lanes share, then VGPRs fetch resources. Caching matters; descriptor pool layout matters.

Bindless descriptors (Vulkan `descriptorIndexing`, DX12 root signatures + heaps) let shaders index into a giant descriptor table at runtime. Modern engines (Unreal 5) use bindless heavily. The compiler must generate appropriate descriptor-indexing code.

Synchronization

Both Vulkan and DX12 force the app to express dependencies:

- **Pipeline barriers / Resource barriers**: "before subsequent ops can execute, complete previous ops, flush their caches, and transition resource layouts."
- **Events / Splits barriers**: barrier whose wait is in a separate command than the signal.
- **Semaphores / Fences**: cross-queue and CPU↔GPU sync.

Driver translates these into PM4: `RELEASE_MEM` (with cache-flush mask), `WAIT_REG_MEM`, `ACQUIRE_MEM`. Wrong barriers = corruption or stalls.

Image layouts

In Vulkan, an image can be in many layouts: `GENERAL`, `COLOR_ATTACHMENT_OPTIMAL`, `SHADER_READ_ONLY_OPTIMAL`, `TRANSFER_SRC_OPTIMAL`, etc. Layout dictates internal compression state (DCC on/off, HTILE compressed). Transitioning layouts may require hardware decompresses.

PMTS-level art: making layout transitions cheap (skip the decompress when the next stage understands the compression).

Render passes (Vulkan classic) vs dynamic rendering

Vulkan 1.0 had **render pass** objects: declare attachment topology and dependencies up-front. Tile-based GPUs (Mali, Adreno) love this. AMD doesn't really benefit; the abstraction was painful.

Vulkan 1.3 dynamic rendering removes the render pass, allowing direct begin/end. Most modern Vulkan apps use this.

DX12 always had a more direct model.

Pipeline barriers — the bug factory

Most game crashes / corruption on Linux Vulkan are barrier bugs:

- App didn't barrier between a write and a subsequent read → race.
- App over-synchronized → lost performance ("driver is slow") even though correctness is fine.

`VK_LAYER_KHRONOS_validation` catches many. radv has internal asserts and `RADV_DEBUG=*` knobs.

Compute in Vulkan

Vulkan compute is a strict subset of compute shading: a Vulkan compute pipeline ⇒ AMDGCN compute kernel running on a compute ring. Same hardware as graphics compute; just different API surface.

Many engines use Vulkan compute for postprocess, animation, particles. Some use it for everything (Path of Exile 2's renderer is heavily compute).

Mesh and ray tracing in Vulkan

Both as KHR extensions, both supported by radv:

- `VK_EXT_mesh_shader` (now KHR-tier in Vulkan 1.4 era).
- `VK_KHR_ray_tracing_pipeline` and `VK_KHR_ray_query`.

Both require recent radv + ACO + RDNA2+.

DX12 on Linux

DX12 doesn't run natively on Linux; you need a translation layer:

- **VKD3D / VKD3D-Proton**: translates DX12 to Vulkan. Used by Steam Proton and Wine.
- Performance often near-native because the APIs are similar.

PMTS-level engineers at AMD also work with Microsoft on the Windows D3D12 driver (separate codebase, NDA).

Translation challenges

- DX12 root signatures don't map 1:1 to Vulkan descriptor layouts.
- Some DX12 features need Vulkan extensions Mesa doesn't yet support.
- Resource barriers vary in semantics.

VKD3D maintains a translation layer that's grown to ~95% feature coverage.

Drawing best practices (what every AAA team learns)

1. **Batch draws** with indirect draw + bindless.
2. **Use compute for postprocess** rather than full-screen quads with PS.
3. **Layout transition coalescing** — fewer, broader barriers.
4. **PSO caching** — ship pipeline cache binaries with the game.
5. **Async compute** — overlap shadow rendering / tone mapping with main pass.
6. **Avoid render-pass-load-clear** — use fast-clear via meta path.
7. **Place transient resources in tile-memory tiers** — even on AMD, reuse memory for per-frame buffers.

A senior engineer can recite these from memory.

Driver architecture impact

When PSO compilation lags, the driver may:

- Cache PSOs to disk between game runs (`~/.cache/mesa_shader_cache`).
- Pre-compile in the background using a "shader threadpool."
- Fall back to a simpler shader (PSO "stuttering" mitigation).

When barriers stack up:

- Coalesce adjacent barriers into one PM4 fence.
- Emit cache flushes lazily (only flush masks that change).

When descriptor sets change rapidly:

- Cache the descriptor write layout, hash, and reuse PM4 sequences.

These optimizations are the bread and butter of game-driver work.

Validation, tools

- `VK_LAYER_KHRONOS_validation` — must run during dev.
- **RGP** (chapter 86) — captures GPU timeline.
- **PIX** (Microsoft, DX12) — equivalent for Windows.
- **Nsight Graphics** — supports Vulkan and DX12; works on AMD too.
- **vktrace / vkreplay** — capture and replay Vulkan workloads.

CI for Mesa/radv runs these against many real games (anonymized replays) to catch regressions.

What you should be able to do after this chapter

- Define PSO, descriptor set, pipeline barrier.
- Distinguish Vulkan render passes from dynamic rendering.
- Reason about barrier + image layout interactions.
- Sketch how DX12 maps to Vulkan via VKD3D.
- Recognize common best-practice patterns.

Read deeper

- Vulkan spec (vulkan.org/spec).
- DX12 reference (Microsoft Learn, gpuopen.com).
- "Vulkan Tutorial" by Alexander Overvoorde — best free intro.
- AMD GPUOpen blog series on Vulkan.
- VKD3D source on Wine GitLab.
- "The Road to Vulkan" — Khronos talks.

Chapter 83 — SR-IOV and GPU virtualization

In data centers and cloud, one physical GPU often serves many tenants. **SR-IOV (Single-Root I/O Virtualization)** is the PCIe standard that lets a single GPU expose multiple **virtual functions (VFs)**, each appearing as an independent device to a virtual machine. AMD's GPUs implement SR-IOV in firmware + hardware + a sophisticated kernel/host driver split.

PMTS engineers in the cloud team and the security team live in this chapter.

The problem

A bare-metal driver assumes it owns the GPU. A cloud customer wants:

- 8 customers sharing one MI300, each in their own VM.
- Strong isolation: customer A can't see customer B's framebuffer or VRAM.
- Performance: near-native, not "10× slower because everything is paravirtualized."
- Live migration support (in some scenarios).

SR-IOV is the answer. The hardware exposes a **PF (physical function)** that the **host hypervisor** drives, plus N **VFs**, one per VM.

PCIe-level model

```
            Host hypervisor (KVM / Hyper-V / VMware)
                        |
                        | PCIe config space access
                        v
          +--------------------------------+
          | GPU (one PCI device)            |
          | +----------------------------+ |
          | | PF: BDF 03:00.0             | |  driven by host PF driver
          | | VF: BDF 03:00.1, 03:00.2,...| |  driven by guest VF driver
          | +----------------------------+ |
          +--------------------------------+
```

Each VF has:

- Its own PCIe BDF.
- Its own MSI-X vectors.
- Its own BAR aperture (mapped from a slice of the PF's resources).
- Its own VMID for GPU work.

VFs use a **subset** of the GPU's resources — partitioned engines, partitioned VRAM, partitioned compute units. The host PF driver decides the partitioning.

Host vs guest drivers

In `amdgpu`:

```
adev->virt.caps == AMDGPU_SRIOV_CAPS_*
adev->virt.ops  == per-PF or per-VF callback table
```

The same driver compiles for both modes; runtime detection switches behavior.

Host (PF) driver:

- Initializes hardware fully.
- Allocates VRAM/GFX time slices to VFs.
- Mediates access to shared resources (DCN display, video encode pools).
- Listens for VF faults / health.
- Manages reset of individual VFs without disturbing others.

Guest (VF) driver:

- Cannot do many init operations (PSP boot, SMU programming).
- Talks to the PF driver via the **MailBox** interface — a small register-level RPC.
- Asks the PF for resources (memory, doorbell slots) when it can't allocate directly.

The Mailbox

VF↔PF communication goes through MMIO mailbox registers. Protocol is essentially:

```
VF writes: command, arg, "request"
PF interrupts, reads, services
PF writes: result, "complete"
VF reads result
```

Commands include: "get firmware version," "request reset," "I'm ready," "I'm gone." All initialization handshakes use mailbox.

This is a security boundary. The PF driver must validate every mailbox message — a malicious guest could send malformed commands trying to crash the PF or escape the VM.

VRAM and address-space partitioning

The PF carves out:

- A piece of VRAM dedicated to each VF (e.g., MI300X 192 GB → 8 VFs × 24 GB).
- A range of GPU virtual address space.
- A subset of HW queues / doorbells.

The VF's VRAM size is hard-fixed per scheduler config; resizing requires VM stop. Some platforms allow CMSA / MIG-style sub-partitions of compute units, exposed as different VF profiles.

Time-slice vs space-slice

Two scheduling models:

Model	How
Time-slice	All VFs share *all* compute units; MES rotates work between them every few ms. Maximizes per-VF peak performance, but cross-VF interference.

Model	How
Space-slice	Each VF gets dedicated CUs / SDMA / VCN. No interference, lower peak per VF. Used in cloud with strict QoS guarantees.

AMD's MI300X partitioning supports both depending on platform config.

Reset isolation

Customer A's VM hangs the GPU. Should customer B suffer?

- **Function-level reset (FLR)** of just the offending VF if hardware allows.
- Otherwise: **mode-1 reset** of the whole GPU (everyone takes a hit).

The PF driver tries hardest for FLR. RAS errors that mark a single VF's region "bad" can be quarantined.

Live migration

Migrating a running VM with a passthrough GPU is technically possible:

1. Quiesce the VF (stop new submissions).
2. Snapshot VF state (VRAM, queue state, fence values).
3. Transfer to destination host.
4. Restore VF state.

In practice, GPU live migration is rare today — too much state, too much work. Cloud usually evacuates GPU VMs on planned maintenance instead.

MIG-like vs partition

NVIDIA "MIG" pre-creates partitions at boot. AMD's approach is more flexible — partition profiles can be reconfigured (with downtime). For training (long-lived jobs), partition once and don't change. For inference (many small tenants), pick the right profile.

Security implications

SR-IOV is the single biggest GPU security concern:

- A VF must not access another VF's memory.
- A VF must not crash the PF.
- A VF must not see another VF's compute results in caches.
- A VF must not extract decryption keys from the PSP.

Every IH / mailbox / queue path is reviewed for cross-tenant leak. Bugs are CVEs (e.g., past advisory: VF VMID reuse without TLB flush).

Display & multimedia in SR-IOV

Most cloud configs disable display (no monitor on a server). Some:

- Use a separate "GIM" (Graphics Infrastructure Manager) host service that renders for VFs.
- Stream encode via VCN to the network (cloud gaming).
- Disable VCN in PF profile for compute-only deployments.

Display blocks are tricky because they're inherently global (one HDMI/DP cable per chip). Cloud gaming uses VCN encode + RDP/PCoIP/Parsec/AMD Cloud SDK to stream.

Tooling

- `lspci -vv` shows VF/PF relationships (`Capabilities: SR-IOV`).
- `cat /sys/bus/pci/devices/<BDF>/sriov_numvfs` — current VF count.
- `echo 8 > sriov_numvfs` enables 8 VFs (needs IOMMU enabled).
- `virsh attach-device` (libvirt) hands a VF to a VM.
- AMD's `amdgpu-virt` debug knobs in sysfs.

What kernel code to read

- `drivers/gpu/drm/amd/amdgpu/amdgpu_virt.c` — virtualization helpers.
- `drivers/gpu/drm/amd/amdgpu/amdgv_*.c` (when present) — virtualization-specific paths.
- `drivers/gpu/drm/amd/amdgpu/mxgpu_*` — per-chip SR-IOV mailbox implementations.
- `drivers/pci/iov.c` — Linux PCI SR-IOV core.

What you should be able to do after this chapter

- Define PF and VF.
- Sketch host+guest driver split.
- Describe the mailbox protocol.
- Distinguish time-slice from space-slice scheduling.
- Reason about reset isolation and security boundaries.

Read deeper

- Linux SR-IOV docs: `Documentation/PCI/pci-iov-howto.rst`.
- AMD GIM (Graphics Infrastructure Manager) repository for host-side VM cloud SDK.
- Linux KVM docs on PCI passthrough.
- AMD GPUOpen blog: "MI300 partitioning."
- Past CVE advisories for AMD GPU SR-IOV (read the patches; learn the patterns).

Chapter 84 — VM faults, page tables, recovery

GPU memory is **virtualized**: every shader access goes through GPU page tables. When the address isn't mapped, you get a **page fault** — and unlike CPU page faults, GPU page faults can be brutal. The way amdgpu handles them is a recurring source of complex bugs and a key area for senior engineers to own.

GPU page tables (recap from chapter 51)

```
GPU virtual address (48-bit typical)
        |
        v
   PD0 (top-level) ──┐
        |            |
        v            |
   PD1 (mid-level)   |
        |            | walked by hardware
        v            |
   PT (page table)   |
        |            |
        v            |
   page (4 KiB / 2 MiB / 1 GiB)
        |            |
        v            v
  GPU PA → VRAM, or → IOMMU → host PA
```

Each PTE is 64 bits, encoding:

- Physical address.
- Cache policy (MTYPE bits — chapter 68).
- R/W/X permissions.
- Valid bit.
- TLB-tag info.
- Per-VMID owner.

Page tables are stored in VRAM, edited via SDMA writes, and flushed via TLB-flush PM4 packets.

What "VM fault" means

A wave issues an address whose PTE has the **valid bit clear**, or fails permission checks. The hardware:

1. Aborts the load/store.
2. Records fault info (faulting VMID, address, access type, source ring, source CU/wave) in a fault log.
3. Raises an interrupt to the IH (Interrupt Handler) ring.

The kernel reads the IH ring, parses the fault payload, and decides what to do.

Two flavors

Fatal page faults

The default. The wave doesn't continue; the entire submission is in a bad state. Kernel:

1. Logs the fault (`dmesg`).
2. Marks the originating context as guilty.
3. Triggers a reset of the affected ring (and possibly broader).
4. The userspace process gets `VK_ERROR_DEVICE_LOST` / `hipDeviceUnreachable` / etc. on its next API call.

This is the classic "dmesg flood" people see when a game has a barrier bug.

Recoverable page faults

CDNA + recent RDNA support **GPU page-fault recovery**: the wave retries the access after the kernel has fixed the mapping. Used for:

- **SVM** lazy migration: HMM brings the page in.
- **Userptr** invalidation re-bind.
- **HMM-MMu-notifier**-driven migration.

Implementation: hardware stalls the wave; kernel handles fault (allocates a page, updates PTE, flushes TLB); kernel signals "retry"; wave continues.

This requires:

- HW that supports retry semantics (gfx9, some gfx10+ for compute).
- Kernel page-fault handler wired up.
- The page tables updated *atomically* before resume — TLB shootdown must complete.

The page-fault path in `amdgpu`

```
IH ring: fault entry written by GMC/IH
        |
        v
amdgpu_irq_dispatch()
        |
        v
gmc_v*.process_interrupt()
        |
        v
amdgpu_vm_handle_fault() / svm_range_restore_pages() / etc.
        |
        v
Either:
  - Resolve (find missing page, update PTE, retry)
  - Escalate (reset)
```

Multiple paths converge here. SVM has its own (`kfd_svm.c::svm_fault_handler`). DRM-context faults have another (`amdgpu_vm_fault_*`).

Why recovery is hard

- The fault must be classified accurately. Was this an SVM page (kernel can resolve) or a wild pointer (must reset)?

- Multi-thread races: the same page might be faulted by many waves simultaneously.
- TLB shootdown must complete before resume — wrong ordering and the wave reads stale.
- ECC can corrupt PTEs themselves; fault may be due to bad memory.
- Per-VMID isolation must be respected.

A single bug in the fault path can corrupt other contexts' memory. **Reviewing patches here is non-negotiable.**

TLB management

Each VMID slot has its own TLB. Flushing options:

- **Per-VMID TLB flush**: targeted, cheaper.
- **Global TLB flush**: nuclear, used after major re-mappings.
- **Specific-page TLB invalidation**: for SVM with retry support.

PM4 packet `EVENT_WRITE` with appropriate flags issues a TLB flush. The flush latency is in the hundreds of nanoseconds; do it sparingly.

Eviction and re-binding

When VRAM gets full, TTM evicts BOs to GTT. The BO's PTEs in every VM that mapped it must be *invalidated* before the eviction completes; new mappings emerge after the next bind. This dance requires:

- Reservation lock (`ww_mutex`) on every VM holding the BO.
- Walk the bo_va list.
- Submit SDMA writes to update PTEs.
- Wait for completion.
- Free the original VRAM.

A senior engineer reviews eviction-path patches very carefully — race conditions here can leak memory or corrupt other workloads.

Userptr and MMU notifiers

A "userptr BO" wraps a user-space malloc-ed buffer. The kernel pins the pages and maps them to GPU VA. But the user might `munmap()` or fork() those pages; the kernel registers an **MMU notifier** that fires on such events. The notifier:

- Removes the pages from GPU mappings.
- Marks the BO "evicted."
- Next GPU access faults; handler re-pins and re-maps.

This is critical for ROCm's "managed memory" pattern. Bugs here = silent memory corruption in user processes.

SVM (Shared Virtual Memory) extras

SVM mappings have richer metadata:

- **Migration policies**: prefer-VRAM, always-system, etc.
- **Per-page access counters**: GPU logs which pages it touches; kernel uses this to migrate hot pages to VRAM.
- **SVM ranges**: contiguous virtual ranges with a single policy.

`kfd_svm.c` has this logic. Cross-NUMA scheduling becomes interesting on multi-die GPUs (MI200/MI300).

Reset-and-recovery interactions (continued from chapter 53)

A GPU reset:

1. Quiesces all in-flight work.
2. Drops every in-flight fence with `EIO`.
3. Re-runs IP block init (per `amd_ip_funcs`).
4. Re-binds all BOs that survived the reset.
5. Resumes user processes — but their previously-pending work is gone (`VK_ERROR_DEVICE_LOST`).

For VM faults specifically, the reset path takes the guilty context and (if `amdgpu_gpu_recovery=1`) tries to keep other contexts alive. SR-IOV adds complexity (per-VF reset).

Watching for faults in production

```
# Persistent log target:
journalctl -k -f | grep -i 'amdgpu.*fault'

# debugfs detail:
cat /sys/kernel/debug/dri/0/amdgpu_gpuvm_info
cat /sys/kernel/debug/dri/0/amdgpu_evict_vram
```

Or use `umr -O ihn -s gfx_0.0.0` to dump the IH ring directly.

Common fault signatures

dmesg pattern	Cause
“GFX VMID 1 PASID 0x… … TC0 … HUB CLIENT GMC …”	wild pointer in shader
“process X has VM_FAULT at 0x…”	userspace bug, often barrier-related
“amdgpu: … gmc invalidate during …”	TLB flush path issue
Many faults on same address	SVM not migrating, or repeated wild access

What you should be able to do after this chapter

- Distinguish fatal from recoverable faults.
- Walk through the IH → handler → resolve/reset chain.
- Define VMID, TLB flush, MMU notifier in one sentence each.
- Read a fault dmesg line and identify the faulting client.
- Reason about eviction-path locking.

Read deeper

- `drivers/gpu/drm/amd/amdgpu/amdgpu_vm*.c`.
- `drivers/gpu/drm/amd/amdgpu/gmc_v11_0.c` (or relevant gen).
- `drivers/gpu/drm/amd/amdkfd/kfd_svm.c`.
- Linux HMM docs (`Documentation/mm/hmm.rst`).
- AMD blog: “GPU Page Faults and Recovery.”

Chapter 85 — Security: PSP, FW signing, attestation, SEV

The GPU is increasingly part of the system's trust boundary. AMD GPUs include a **PSP (Platform Security Processor)** — a separate ARM-based security CPU on-chip — that boots all firmware, verifies signatures, holds keys, and mediates protected operations.

This is one of the highest-NDA areas in the company. The pre-public summary below is what you can talk about at conferences; deeper details are AMD-internal.

Why GPUs need a security processor

- Firmware tampering would let an attacker exfiltrate memory across tenants.
- Cloud customers need confidence that another tenant can't read their model weights.
- Content protection (Widevine L1, PlayReady SL3000) requires hardware-rooted keys.
- Confidential computing (SEV-SNP, TDX) needs the GPU to prove its state.

PSP solves the first two; AMD's broader security stack handles the rest.

PSP basics

- ARM Cortex-A5 (older) / Cortex-A8 / Cortex-A9 / newer cores depending on gen.
- Tightly coupled to the rest of the GPU via internal bus.
- Has its own ROM, RAM, and persistent storage (in some packages, OTP fuses).
- Boots first, before any other engine.

PSP firmware (`*_sos.bin`) is signed by AMD with an asymmetric key whose root of trust is in PSP ROM. Successive firmware loads are chained:

```
PSP ROM (silicon) → verifies PSP SOS → loads PSP SOS
PSP SOS → verifies & loads SMU FW
        → verifies & loads GFX FW (PFP, ME, MEC, MES)
        → verifies & loads SDMA FW
        → verifies & loads DMUB FW
        → verifies & loads VCN FW
        → verifies & loads RAS TA, ASD TA, HDCP TA, others
```

Any failed signature → engine doesn't load → graceful degradation or full failure depending on which engine.

Trusted Applications (TAs)

PSP can run small AMD-signed programs called **Trusted Applications**:

TA	Role
ASD (AMD Secure Driver)	base TA all drivers use
HDCP	content protection key handling for DP/HDMI

TA	Role
RAS	error reporting (some flows go through PSP)
DTM (Display Topology Manager)	display security policy
SecureOS / Trustlets	per-feature mini OSes

Loaded by the kernel via PSP ring commands. Each TA runs in PSP's protected memory, isolated from PF/VF and from each other.

Kernel↔PSP interface

`drivers/gpu/drm/amd/amdgpu/psp_*.c` implements a ring-based command interface:

```
1. Kernel constructs a command (struct psp_gfx_cmd_resp).
2. Writes it to the PSP ring buffer.
3. Tells PSP "go" via a register or doorbell.
4. PSP processes, writes a response.
5. Kernel reads response.
```

Commands include: load a TA, invoke TA, load firmware, run RAS handlers, query firmware versions.

PSP responses use status codes; well-defined errors (`TA_OUT_OF_MEMORY`, `INVALID_CCD_ID`, etc.). Production code retries on transient errors and bails on persistent ones.

Firmware update flow

```
1. AMD signs new firmware blobs with offline keys.
2. Distributed in `linux-firmware` package (or Windows-side equivalents).
3. On boot, kernel walks driver IP table and asks PSP to load each.
4. PSP verifies signatures against ROM-rooted key.
5. PSP loads or rejects.
6. Driver waits for confirmation, then proceeds with init.
```

If a malicious user replaces a firmware blob: PSP rejects (signature mismatch), driver fails init, GPU is dead until valid firmware restored. **Cannot bypass at the kernel level** — PSP is the gate.

Anti-rollback

PSP enforces minimum firmware versions. Once a chip has run version N, version < N is rejected (to prevent attacker from downgrading to a version with a known vulnerability). Implemented via fuses or persistent monotonic counters in PSP storage.

Practical consequence: **never** update GPU firmware in a way that gets "downgraded" later. Production rollback plans must account for this.

Confidential Computing on GPU (SEV-SNP era)

AMD SEV (Secure Encrypted Virtualization) protects guest VM memory from the hypervisor by encrypting RAM with per-VM keys. **SEV-SNP** adds cryptographic integrity (no replay/relocation attacks).

Extending this to GPUs:

- The GPU memory must be unreadable by the hypervisor.

- The GPU must support memory encryption (TEE-on-GPU).
- DMA between CPU and GPU must use encrypted channels.

AMD has been progressively adding support. MI300 has memory encryption and some confidential-compute flows. Full end-to-end (encrypted training of an AI model on a hostile cloud) is the long-term aspiration.

Attestation

A confidential workload demands **proof** that the GPU is in a known-good state:

- A valid firmware version.
- A valid TA configured.
- Memory encryption enabled.

PSP can produce a **signed attestation report** including its measurements. The customer's enclave verifies the report against AMD's public key, and only then unseals secret data. This is the same model as TPM-based remote attestation, but for the GPU.

Quotes are emitted via PSP commands; kernel exposes them through ioctls / sysfs paths.

HDCP and content protection

For DP/HDMI displays of premium content:

1. Source (GPU) and sink (monitor) authenticate via HDCP 2.x.
2. Negotiate a session key.
3. Encrypt every pixel on the wire.

The kernel exposes HDCP property on the connector (`drm_property_create_enum` for "Content Protection"). DC programs the encrypter; PSP holds the keys; Trustlet handles negotiation.

Bug here = loss of HDCP certification for the platform. Slow walks through this code.

Side-channel concerns

Modern security thinking includes side-channels:

- **Timing**: a tenant's compute jobs might leak data via cache-timing variations.
- **Power**: a tenant could measure system power and infer another's activity.
- **Voltage glitching** (lab attacks): physical access to glitch the chip into bypassing checks.

Mitigations:

- Cache flush on context switch (slow, sometimes used in shared-tenancy).
- Power-gating to mask activity.
- Anti-glitch circuits in PSP (proprietary).

Threat-model for a PMTS reviewer

When reviewing a security-related patch, a senior engineer asks:

1. Is any input from userspace / VF / network used without validation?

2. Could a memory-mapping change race with TLB flush, exposing stale data to another tenant?
3. Can the patch be exploited to issue a forbidden PM4 packet?
4. Does it touch fence/semaphore handling — could a use-after-free leak data?
5. Is there a reset path that could leave the GPU in an exploitable state?

Approval requires confidence in all of these.

Where PSP code lives

Kernel side:

- `drivers/gpu/drm/amd/amdgpu/psp_*.c` — PSP rings.
- `drivers/gpu/drm/amd/amdgpu/amdgpu_psp.h`.
- TA-specific kernel code: e.g., `amdgpu_xgmi.c` uses TAs for cross-GPU link auth on MI200/300.

Firmware side: not in tree (pre-built, signed blobs in `linux-firmware`).

What you should be able to do after this chapter

- Sketch PSP boot order.
- Describe TAs and what they do.
- Explain anti-rollback and why downgrade is dangerous.
- Outline confidential-compute requirements on GPU.
- Identify HDCP and TA flows in DC.

Read deeper

- AMD security white papers (search “AMD PSP,” “AMD SEV-SNP whitepaper”).
- `drivers/gpu/drm/amd/amdgpu/psp_*.c`.
- AMD GPUOpen security blog posts.
- Google research on AMD platform security (yearly papers).
- OS-Conf / DEF CON talks on AMD platform security.

Chapter 86 — SQTT, RGP, perf counters, RGD

Performance work on AMD GPUs is dominated by a few profilers and a *lot* of hardware counter knowledge. Knowing what each tool sees and where its blind spots are is what separates serious profilers from people running `time ./game`.

The tools

Tool	Workload	What it shows
RGP (Radeon GPU Profiler)	Graphics (Vulkan/DX12)	wave-level timeline, occupancy, stalls, cache hits
rocprofiler-sdk / rocprof	Compute (HIP)	hardware counters per kernel, traces
omniperf	Compute	roofline, occupancy, cache hit views, AI focus
omnitrace	Compute + system	unified CPU+GPU timeline
RGD (Radeon GPU Detective)	Crashes	post-mortem GPU dump analysis
PIX (Microsoft)	DX12 graphics	similar to RGP, Windows-side
AMD AMF tools	Multimedia	encode/decode profiling

PMTS-level engineers know all of these. Junior engineers usually know one.

SQTT (Shader Quality Trace)

SQTT is the hardware feature that captures **per-wave events** in a circular buffer. Each event records:

- Wave issued / completed instruction
- Memory access started / completed
- Cache hit / miss
- Wave preempted
- LDS bank conflict
- Stall reasons

The volume is enormous (millions of events for a few-millisecond capture). Hardware writes them into a buffer; software post-processes.

RGP and Microsoft PIX both consume SQTT. RGD uses SQTT for crash forensics.

How RGP capture works

```
1. App runs, RADEON-PROFILER hotkey pressed (or programmatic capture).
2. driver opens SQTT buffers in VRAM.
3. enables SQTT collection on shader engines.
```

```
4. captures one or more frames.
5. disables collection; exports raw data.
6. user opens .rgp file in RGP UI.
```

In radv (Mesa): set `RADV_DEBUG=trace` or use `RADEON_DEBUG=hang` then trigger via Alt+Shift+P (with the RGP hotkey daemon).

Captures are huge (100s of MB per frame). Post-processing reconstructs:

- Per-wave timeline.
- Pipeline state per draw.
- Shader code with lane-count hot spots.
- Memory access patterns.
- Cache hit/miss rates.

Hardware perf counters

The GPU has hundreds of hardware counters: per shader engine, per CU, per memory channel, per RBE. Examples:

- `SQ_PERF_SEL_WAVES`: wave issues.
- `SQ_PERF_SEL_CYCLES`: SIMD active cycles.
- `TCC_HIT`, `TCC_MISS`: L2 hits/misses (texture cache cluster).
- `CB_PERF_SEL_*`: render backend counters.
- `GL2_*`: L2 cache cluster counters.

Tools program counter selects (via PM4 SET_UCONFIG_REG packets), let kernels run, then read accumulated counts. Counters are limited (e.g., 16 simultaneous), so multiple passes may be needed for full coverage.

`rocprofiler-sdk` (the modern AMD profiler API) wraps this in:

```yaml
metrics:
  - WAVES
  - L2CACHE_HIT_RATE
  - VALU_BUSY
```

```
rocprofv3 --kernel-trace --metrics-set=foo my_app
```

SPM — Streaming Performance Monitor

SPM streams selected counters to a buffer at a fixed sample interval (e.g., every 1024 cycles). Lets you build **counter timelines** synchronized with SQTT — see how L2 hit rate evolves over a frame.

Used by RGP's "Frame Summary" view.

Reading an RGP capture

For a graphics workload:

1. **Frame Summary**: total time, hottest stages.
2. **Pipeline Overview**: time per pipeline (geometry vs raster vs PS vs RBE).
3. **Most expensive draws**: top 10 draws by GPU time.

4. **Wave occupancy**: timeline showing waves on each SE/CU.
5. **Shader analysis**: per-shader stats (VGPR count, occupancy, cycles, stalls).
6. **Cache views**: hit rates per level, per draw.
7. **Async compute**: are compute and graphics overlapping?

A first-pass profile takes 30 seconds; a thorough one takes hours. Identifying *which* of N findings to actually address is the art.

Reading a rocprof / omniperf capture

For compute:

1. Total kernel time, top kernels.
2. Per-kernel: VGPRs, SGPRs, scratch, occupancy.
3. Cache hit rates per level.
4. Memory bandwidth realized vs peak.
5. Roofline: where does this kernel sit (memory-bound vs compute-bound)?
6. Wave issue stalls broken down by reason (memory, scalar, branch).

omniperf's roofline view is famous — plots arithmetic intensity vs achievable performance vs HW peak.

Hot-path identification: a checklist

When given a profile, walk down this list:

1. **Is the GPU saturated?** If GPU activity < 90%, find why (CPU-bound, sync-bound, page-flip waits).
2. **Where's the time?** Top draw / kernel / IP block.
3. **Is it memory-bound?** Compare BW realized vs peak. If saturated, optimize accesses; reduce footprint.
4. **Is it compute-bound?** Look at stalls. VALU stalls = compiler issue or true math limit.
5. **Is it occupancy-limited?** Spilling, LDS, VGPRs.
6. **Is it serial?** Bad async scheduling or barrier oversync.

Each finding gets one fix. Profile again.

RGD — crash forensics

When a GPU hangs or VM-faults, RGD analyzes the post-mortem dump:

- Last submitted IBs.
- Faulting wave PC and registers.
- Page-table state at fault time.
- Cache state.

Used by AMD's customer support to handle game-crash reports. PMTS-level engineers help train the heuristics RGD uses to isolate likely causes.

Capture tips

- **Capture short**: 1-3 frames is plenty. SQTT buffer fills fast.
- **Stable conditions**: same scene, same input, same clocks (force `pp_dpm_*` to a fixed level). Variability ruins comparisons.
- **Background noise**: kill compositors / browsers; their work shows up too.
- **Compare A/B**: capture before-fix and after-fix, diff in RGP.

Driver instrumentation

The driver exposes useful trace points:

- `tracepoint:amdgpu_cs_ioctl` — every CS submission.
- `tracepoint:amdgpu_vm_*` — VM events.
- `tracepoint:dma_fence_*` — fence events.
- `tracepoint:drm_sched_*` — scheduler events.

Use `trace-cmd` or `perf trace` to capture:

```
trace-cmd record -e 'amdgpu_cs_ioctl' -e 'dma_fence_*' my_app
trace-cmd report
```

Counter combinations to learn

```
cycles_active / cycles_total      → utilization
valu_busy_cycles / cycles_active  → ALU efficiency
tcc_hit / (tcc_hit + tcc_miss)    → L2 hit rate
gl2_hit_rate                      → effective with chip varies
mem_accesses / kernel_time        → BW used
```

Memorizing 8-10 derived metrics is enough to talk shop.

What you should be able to do after this chapter

- Define SQTT, SPM, RGP, RGD.
- Capture an RGP profile of a Vulkan game and explain at least one finding.
- Use rocprof to extract a counter set for a HIP kernel.
- Reason about memory-bound vs compute-bound.
- Identify when occupancy is the issue.

Read deeper

- RGP user guide and the AMD GPUOpen tutorials.
- ROCm rocprofiler-sdk documentation.
- AMD blog: “Optimizing your graphics on Radeon.”
- omniperf docs: the roofline model walkthrough.
- Reading other AAA engines’ RGP captures (some shared online).

Chapter 87 — Cross-stack debug & regression triage

A bug at PMTS level rarely has a single cause in a single layer. It spans:

```
App ↔ runtime ↔ UMD ↔ kernel ↔ firmware ↔ hardware ↔ device
```

You must move fluidly between all of them. This chapter is the playbook.

The triage funnel

```
Customer report
   |
   v
Reproduce
   |
   v
Localize  ←— which layer?  app / UMD / kernel / FW / HW
   |
   v
Bisect (in time, in config, in workload)
   |
   v
Root cause
   |
   v
Fix + test + push upstream + backport
   |
   v
Close
```

Each arrow is a *skill*. Most engineers are great at 2 of these and adequate at the rest. PMTS engineers are great at all of them.

Reproducing

Without a repro, you have a story, not a bug. First step is always “make it happen on demand.”

Tactics:

- Get exact game build, driver version, kernel, firmware, GPU, OS.
- Get a save file at the moment before crash.
- Capture the API call stream (`vkapidump`, `apitrace`, `vktrace`).
- Synthetic reproducers from API traces.
- Loops and stress tests.

If you can’t reproduce in a controlled environment, you can’t trust any fix. **Spend the time.**

Localizing the layer

Each layer has signatures:

Symptom	Likely layer
`dmesg: amdgpu: VM_FAULT...`	kernel saw GPU fault — UMD or app bug usually
`VK_ERROR_DEVICE_LOST`	reset happened — could be anywhere
`glxgears` works, game doesn't	game-specific UMD or shader compile bug
Always after game's "compiling shaders"	PSO / shader compiler issue
Random hang after hours	TDR or memory leak
Tearing, jitter on display	KMS / DC / VRR
Slow only on one GPU model	firmware version / SMU
Runs fine elsewhere except on AMD	Mesa / radv / kernel bug
Same crash on Windows D3D12	likely hardware / firmware

Differential diagnosis: change one variable and see what changes.

- Different GPU? Kernel? Firmware? Mesa version?
- Disable one feature: `RADV_DEBUG=nodcc nooptimize`.
- Boot with `amdgpu.gpu_recovery=0` to inspect post-hang state.

Bisecting

If a regression appeared between two known-good and known-bad versions, **`git bisect`** is your friend.

For Mesa / kernel:

```
cd ~/work/linux
git bisect start
git bisect bad master
git bisect good v6.6
# bisect builds, you boot/test, mark good or bad
git bisect good   # or: git bisect bad
# after ~log2(N) iterations, you have one commit
```

Build/boot cycles are slow. Tips:

- Pre-build a minimal kernel config that's fast.
- Use `make -j$(nproc)`.
- VM with virtfs sharing source/build avoids reinstalls.
- Script the bisect step: `git bisect run my-test.sh` if the test is automatable.

For firmware bisects: AMD ships discrete firmware versions; you bisect by manually swapping `*_smc.bin` etc. and rebooting.

Reading dmesg

Patterns to recognize at a glance:

```
[  120.123] amdgpu 0000:03:00.0: GPU fault detected: 147 0x09380000 ...
            ^                                        ^
            adev (PCI BDF)                           fault status code
```

```
[  120.124] amdgpu: VM_FAULT_FROM_ME at addr 0x0000000040004000 ...
                                            ^
                                            faulting GPU VA

[  120.125] amdgpu: GFX 0 wave 0 lane mask 0xffffffff ...
                               ^
                               offending wave
```

Decode the status code from `gmc_v*.h`. Sub-codes tell you whether it's a translation fault, a permission fault, a write-to-RO, etc. — narrows the hypothesis.

Reading trace-cmd output

```
trace-cmd record -e 'amdgpu_cs_ioctl' -e 'dma_fence_*' -e 'drm_sched_*' my_game
trace-cmd report > trace.txt
```

You get a microsecond-accurate timeline of submissions, fences, and scheduler events. A typical session reveals patterns like:

- Fence not signaling → `drm_sched_*` shows job stuck.
- Long latency between submit and start → CPU-bound or scheduler blocked.
- Many short submissions → CPU overhead opportunity.

umr — direct hardware introspection

```
sudo umr -lb gfx_0.0.0      # live wave + ring state for GFX
sudo umr -O ihn -s gfx_0.0.0 # IH ring contents
sudo umr -p REG_NAME         # program a register
```

`umr` reads/writes registers, decodes ring buffers, dumps wave state. Used when nothing else works.

rocgdb walkthrough

For a hung HIP kernel:

```
rocgdb my_app
(rocgdb) run
^C    # interrupt while kernel is running
(rocgdb) info threads
(rocgdb) thread <gpu thread>
(rocgdb) bt
(rocgdb) info locals
(rocgdb) p $vgpr0
```

You can step single instructions, inspect VGPRs, see exec mask. When it works, it's magic. Setup is painful (debug builds, matching ROCm/kernel/firmware versions).

Cross-stack reasoning example

Scenario: A new RDNA3 chip exhibits visual corruption only in Cyberpunk 2077, only with DXR enabled, only at 4K, only on Linux. (Hypothetical.)

Steps:

1. **Reproduce**: confirmed on a couple of machines.
2. **Layer hypothesis**: graphics-only? Compute? Display?
   - Disable DXR → bug gone → narrows to ray tracing path.
3. **Layer further**:
   - Disable `RADV_DEBUG=nooptimize` → bug remains. So not an ACO regression.
   - Try AMDVLK (closed-source AMD Vulkan) → bug remains. So not a radv-only bug.
   - Try Windows → bug not there. So Linux-specific.
4. **Hypothesis**: kernel or firmware path differs Linux vs Windows for RT features.
5. **Bisect kernel**: between two kernels, find the patch that introduced — turns out to be a TLB-flush optimization.
6. **Root cause**: optimization missed flushing for a specific BVH-update flow; stale TLB entries cause wrong reads.
7. **Fix**: revert or refine the optimization.
8. **Test**: reproduce no longer triggers across multiple machines + games.
9. **Push upstream**: with `Fixes:` tag, `Cc: stable@vger.kernel.org`.

A real example like this can take days to weeks. Senior engineers maintain a mental library of "where similar problems came from" so they can short-circuit the funnel.

When the bug is in firmware

If your bisect lands on "no kernel/UMD change, same firmware" — you may have a hardware/firmware errata. Steps:

1. Search AMD's internal bug DB for similar reports.
2. Check firmware release notes for related fixes.
3. Submit a bug to the appropriate AMD team with the repro.
4. While waiting: see if a kernel-side workaround is feasible (e.g., disable an HW feature on this chip, write extra TLB flushes).

When it's an upstream regression you didn't cause

Etiquette:

- File a bug on the appropriate tracker (Mesa GitLab, kernel bugzilla, lore.kernel.org).
- Tag the original commit author and any reviewers.
- Provide repro and bisect.
- Be polite. Software is hard.

Production patterns

A real PMTS workflow looks like:

```
Morning: triage 3-5 new bugs
   - reproduce 2, file the others for more info
Mid-morning: deep dive on biggest customer hit
   - bisect, root cause, prepare fix
```

```
Afternoon: review 3 patches from team
Evening: write tests, send fix patches upstream
```

Mixed with: meetings, planning, mentoring junior engineers, talking to customers.

What you should be able to do after this chapter

- Reproduce, localize, bisect, root-cause, fix, push upstream — independently.
- Use dmesg, trace-cmd, umr, rocgdb without panic.
- Follow a hypothesis tree across layers.
- Recognize firmware vs kernel vs UMD bug patterns.

Read deeper

- “Linux Kernel Debugging” book (recent editions).
- AMD GPUOpen “Crash debugging on Radeon.”
- `Documentation/admin-guide/bug-bisect.rst`.
- DEFCON / FOSDEM talks on real driver debug stories.
- The amd-gfx@lists.freedesktop.org archive — months of cross-stack debug discussions.

Chapter 88 — Customer / ISV / game studio interaction

A staff or PMTS engineer doesn't just write code — they're the company's technical face to important external partners. Game studios, AI companies, OEMs, cloud providers. How you handle these interactions decides whether AMD wins the next design.

Who the customers are

Segment	Example customers	What they want
AAA game studios	CDPR, Sony first-party, Microsoft Studios, Capcom	Their game runs great on AMD GPUs from launch
Engine vendors	Epic (Unreal), Unity, Activision, EA	Their engine extracts max perf on RDNA
AI / ML companies	Meta, OpenAI, Anthropic, Mistral	PyTorch / JAX / Triton on AMD; cluster reliability
Cloud providers	AWS, Azure, GCP, Oracle, Vultr	Multi-tenant SR-IOV, reliable reset, SLA-grade
HPC labs	ORNL, LLNL, CERN	Long-job stability, ECC, MPI/RCCL throughput
OEMs	Lenovo, Dell, HP, ASUS	Power, thermal, audio, suspend-resume on laptops
Operating system vendors	Microsoft, Red Hat, Canonical	Stable driver ABI, kernel integration
Standards bodies	Khronos, PCI-SIG, VESA	Spec contributions, conformance

Each has different priorities and different tolerance for bugs.

What they want from you

- **Show up prepared.** Have done your homework on their workload.
- **Be honest.** If a feature won't work for 6 months, say so. Don't promise.
- **Respect their time.** Customers' engineers are senior too. Don't waste their cycles with basic questions.
- **Follow up.** A bug filed is a bug owned. Update them weekly.

A typical engagement

"We're shipping our game in 4 months. Help us ship on AMD."

Phase 1 — Onboarding (weeks 0-2):

- NDA signed, legal sorted.

- Dedicated Slack/Teams/email channel.
- Get build access.
- Provide them tools: RGP, drivers, validation suites.

Phase 2 — Smoke test (weeks 2-4):

- Run their game, identify obvious problems.
- Prioritize: visual bugs > crashes > perf > polish.

Phase 3 — Deep dives (weeks 4-12):

- For each prioritized issue, root cause + fix or workaround.
- Sometimes the fix is in the game (you tell them). Sometimes the fix is in driver (you write it).
- Profile, optimize hot paths together.

Phase 4 — Ship (last 4 weeks):

- Lock the AMD driver version they ship against.
- Final certification.
- Marketing coordination ("optimized for Radeon").

After ship: support the title for years. Patches, regression fixes, new card support.

Profiling with a customer

You sit (virtually or physically) with their engineer. You both run RGP / PIX captures of their workload. You point out:

- "This shadow pass is taking 4 ms because every quad has 2 triangles — split rebatch."
- "Your post-process is back-to-back compute but with full barriers; convert to event-style."
- "DCC is being decompressed every frame because of layout transition in this pass; here's a fix."

This is the work. Hours of profile-and-suggest, sometimes with their game engine source open, sometimes through opaque closed-binary captures.

Reporting customer-found driver bugs

When a customer finds a driver bug:

1. They send you a repro (sometimes a 50 GB game build + savefile).
2. You reproduce locally.
3. You file an internal bug, link the customer's external ticket.
4. You either fix it or hand off to the right team.
5. Ship the fix in the next driver release window.
6. Update the customer.

The cycle time matters. A customer waiting 6 months for a fix to a launch-blocker is a *lost customer*.

Pre-silicon engagement

Months *before* a chip launches, ISVs need:

- Early hardware (or simulators).
- Driver builds with experimental support for the new arch.
- Documentation on new features.
- Direct line to AMD engineers.

PMTS engineers participate in pre-silicon co-development with key partners. The terms are strictly NDA. The relationship is built on trust over years.

OEM dynamics

Laptop OEMs care about:

- **Power**: every milliwatt matters. AMD's APU + dGPU mux logic must be perfect.
- **Thermal**: tight chassis, weird thermal solutions, audio coupling. Driver must hit thermal targets without throttling complaints.
- **Sleep/wake**: 1000+ daily resumes per laptop. Must work.
- **Brightness, hot keys, special buttons**: ACPI / vendor BIOS quirks.

OEMs file bugs you'd never see otherwise: "On our X1234 laptop, screen flickers when on battery between 20% and 30%." You learn to love EC firmware engineers.

Cloud customer dynamics

Cloud is performance + reliability. They care about:

- **MTBF** (mean time between failures): how often does a GPU fail in a fleet?
- **Reset behavior**: a single tenant must not take down the box.
- **Serviceability**: hot-swap, telemetry, RMA flow.
- **SR-IOV correctness**: no cross-tenant leak, ever.

Their bug reports come with population statistics ("0.1% of MI300s in our fleet show this pattern"). You become a statistician.

ISV best practices for AMD

You'll often write or co-author "ISV best practice" docs:

- Use mesh shaders rather than tessellation when possible.
- Pre-compile PSOs at startup.
- Use async compute for shadow passes.
- Avoid full RT on midrange GPUs by default.
- Implement upscaling (FSR) before HDR.

These docs go out via GPUOpen or directly to studios. PMTS engineers contribute, edit, and stand behind them.

Conferences

Regular ones:

- **GDC** — Game Developers Conference. AMD has a booth, a track of talks.
- **SIGGRAPH** — academic graphics.
- **HotChips** — hardware deep dives.
- **XDC** (X.Org Developers Conference) — open-source graphics.
- **FOSDEM** — open-source generally.
- **LPC** (Linux Plumbers Conference) — kernel-level.
- **DevSummit** — internal AMD events.

PMTS engineers speak at these. Either talks they've prepared, panels, or "office hours" answering questions.

Public communication

You'll write:

- Blog posts on GPUOpen.
- Conference talks.
- Open-source patches with detailed commit messages.
- Mailing-list responses to design proposals.
- Twitter/X / Mastodon / LinkedIn for visibility (career-facing).

Quality matters. A PMTS engineer's blog post is a small AMD endorsement; care is required.

Ethical considerations

You will sometimes:

- Be asked to optimize for *one* benchmark in a way that hurts others. Push back.
- Hear customer complaints about a competitor in confidence. Don't repeat.
- Receive proprietary game code under NDA. Treat it like radioactive material.
- Be tempted to over-promise. Don't.

Your reputation outlasts any single product cycle. Protect it.

What you should be able to do after this chapter

- Understand who AMD's customers are.
- Plan a 4-month engagement with a game studio.
- Triage a customer-reported bug correctly.
- Communicate effectively across NDA boundaries.
- Speak at a conference about AMD GPU work.

Read deeper

- AMD GPUOpen blog (gpuopen.com).

- GDC vault talks for “AMD Radeon” or “ATI” historical context.
- Books on technical leadership (“Staff Engineer” by Will Larson — chapter on stakeholder management).

Chapter 89 — Maintainership and upstream leadership

Sending a few patches doesn't make you a maintainer. **Owning a subsystem upstream** does. This is the trail PMTS engineers walk in the open-source world.

What "maintainer" means in Linux

A maintainer for a subsystem (e.g., `drivers/gpu/drm/amd/`):

- Reviews and merges patches into their tree.
- Sends pull requests up the chain (Alex Deucher → DRM tree → Linus).
- Ensures conformance with kernel coding/process standards.
- Handles bug reports affecting their area.
- Makes architectural decisions.
- Mentors new contributors.

In Linux, maintainership is *distributed*: there's a maintainer for `gpu/drm/amd/`, one for `gpu/drm/amd/display/`, one for `amdkfd/`, and so on.

`MAINTAINERS` file in the kernel root lists everyone. Look:

```
RADEON and AMDGPU DRM DRIVERS
M:  Alex Deucher <alexander.deucher@amd.com>
M:  Christian König <christian.koenig@amd.com>
L:  amd-gfx@lists.freedesktop.org
S:  Supported
T:  git https://gitlab.freedesktop.org/agd5f/linux.git
F:  drivers/gpu/drm/amd/
F:  drivers/gpu/drm/radeon/
F:  include/uapi/drm/amdgpu_drm.h
F:  include/uapi/drm/radeon_drm.h
```

`M:` are maintainers; `L:` is the mailing list; `T:` is the git tree.

How you become a maintainer

There's no "election." You earn the role:

1. Send many high-quality patches over a few years.
2. Review others' patches consistently and well.
3. Be active on the mailing list with helpful, constructive replies.
4. Show ownership of an area — when something breaks, you fix it.
5. Eventually a sitting maintainer says "I want X to take this over" or "X is my co-maintainer for Y."

It's a multi-year commitment. PMTS engineers at AMD are the standard maintainership pool.

Reviewing patches

Reviewing is at least as important as writing.

Review checklist (mental version)

- **Does the patch describe the problem?** Commit message is the documentation.
- **Does it fix what it claims?** Read the actual diff.
- **Side effects:** does it break other use cases? Other chips? Other configs?
- **Locking:** added new shared state — is it protected? Order maintained?
- **Memory safety:** any new allocations match deallocations? Error paths?
- **ABI:** any uapi headers changed — backwards compatible?
- **Tests:** could the change be tested? If yes, is there a test?
- **Style:** `checkpatch.pl` clean.
- **Performance:** does it regress anything?

A good review can take an hour. A great review for a complex patch can take a day. Worth it.

Review etiquette

- Be specific. "This locking is wrong" → "Line 42: `vm->lock` is taken without disabling IRQs, but the IRQ handler at line 78 also takes it; this can deadlock."
- Suggest fixes when you can.
- Distinguish "must fix" from "would be nice."
- Don't hold over personal taste — if the patch works and follows conventions, ship it.

Review tags

When you review, use these tags in your reply:

Tag	Meaning
`Reviewed-by:`	"I reviewed this carefully and it's correct."
`Acked-by:`	"I'm OK with this; not as deep as Reviewed-by."
`Tested-by:`	"I ran it and it works."
`Reported-by:`	(for the author to add) "X reported this bug."
`Suggested-by:`	"X suggested the fix."

These end up in the commit when applied. They're how you build reputation.

Sending pull requests

When you have a queue of patches ready for the next merge window:

1. Rebase on the appropriate base.
2. Run regression tests.
3. Tag a release: `git tag -s drm-misc-next-2026-05-07 -m "..."`.
4. `git request-pull <base> <url> <tag>` and email the result to the next-up maintainer.

The PR email gets reviewed; on acceptance, the upstream pulls.

Conflict resolution

Conflicts happen — between AMD's needs and other vendors', between maintainers, between technical visions. Senior contributors:

- State their position in writing on-list.
- Listen to counter-arguments.
- Compromise where possible.
- Escalate to broader maintainers if needed (Dave Airlie for DRM; Linus for kernel-wide).
- Don't let conflicts get personal.

You will see flame wars. Choose to be the calm one.

ABI stability — sacred

Linux's prime directive: **don't break userspace.**

Once a uapi header is in a release, you cannot:

- Remove a struct field.
- Change a field's meaning.
- Add a required field (existing apps don't fill it).
- Change ioctl numbers.
- Reduce a permission's scope.

You *can*:

- Add new optional fields (apps that don't fill it work as before).
- Add new ioctls.
- Add new flags to existing ioctls (with new bit meanings; old bits keep semantics).

A patch breaking userspace ABI gets nacked at lightning speed. Even if "no one uses it" — that's never proven.

Working with downstream distros

Distros (Ubuntu, Fedora, RHEL, Debian, Arch) maintain their own patches. As maintainer:

- Push fixes upstream **first**.
- Tag user-visible bugfixes with `Cc: stable@vger.kernel.org` so they auto-backport.
- Coordinate on out-of-tree patches that distros refuse to upstream (rare but happens).

Backports require careful patch isolation — your fix must be self-contained, not part of a chain.

The 6-month merge cycle

Linux releases every ~10 weeks. Within that:

- Weeks 0–2: merge window. Maintainers send PRs to Linus. Big features land.
- Weeks 3–10: stabilization. Only fixes accepted (`-rcN` releases).
- Final: `vX.Y` released.
- Next merge window opens.

For a feature you want in `vX.Y`, you must have it reviewed-and-acked **before** the merge window opens — usually weeks of lead time. PMTS engineers plan releases backwards from this calendar.

Mesa / Mesa-like maintainership

Mesa is GitLab-style merge requests rather than email patches. The community is smaller, more visible, and more conversational. Reviews happen in MR comments.

Becoming a Mesa maintainer:

- Many merged MRs in a specific area.
- Active in MR reviews.
- Project leadership grants Developer or Maintainer role.

Same general ethics; faster cadence (Mesa releases quarterly).

Working with bugbots

Modern Linux upstream uses automation:

- **`syzbot`** — fuzzer that finds kernel bugs and emails reports.
- **LKP (Linux Kernel Performance)** — automated performance regression detection.
- **regzbot** — tracks regressions across releases.
- **0day robot** — build/test on every patch.

Maintainers must respond to these; ignoring `syzbot` reports is a quick way to lose credibility. PMTS engineers triage bot output and either fix, suppress (with rationale), or hand off.

Mentoring

You'll be asked to onboard new contributors. Patient mentoring is part of the role:

- Pair-debug their first real patch.
- Walk them through how to use git format-patch / send-email.
- Explain patch comment etiquette.
- Encourage them when reviewers are harsh.

Good engineers grow good engineers. The teams that mentor well retain knowledge across decades.

What you should be able to do after this chapter

- Read MAINTAINERS, find the right person to send a patch to.
- Conduct a thorough patch review.
- Tag patches correctly (`Reviewed-by`, `Fixes`, `Cc: stable`).
- Plan a feature delivery against the merge window calendar.
- Handle a `syzbot` report.

Read deeper

- `Documentation/process/` in the kernel — every page.
- `Documentation/process/maintainer-handbooks.rst`.
- “Linux From Scratch — Patch Submission Howto” (multiple versions online).
- Maintainer summit talks (Linux Plumbers Conference).
- Greg KH’s “kernel maintainership” essays.

Chapter 90 — Silicon bring-up, post-silicon validation

The most “real-world” thing AMD does is bring up new silicon: a chip arrives from the fab, and a team has to make it boot, run, and ship in 6–12 months. PMTS engineers in this area drive the process.

You won’t do this in your first year. You’ll watch it; in your fourth or fifth year you may be doing pieces.

The phases

```
1. Tapeout         (chip's design is "frozen" and sent to fab — TSMC, Samsung)
2. Wafer return    (silicon comes back; first samples)
3. Lab bring-up    (initial validation; one chip on a test board)
4. Pre-production  (extending support, making OS work)
5. Production      (engineering samples ship; alpha and beta customers test)
6. Public launch   (consumer/datacenter availability)
```

Pre-tapeout, the design is being verified in simulation and emulation (Synopsys ZeBu, Cadence Palladium). Drivers are being prototyped against simulators that approximate the hardware.

Lab bring-up — the first hours

A wafer comes back. A small team in a lab in Austin / Markham / Bangalore / Shanghai puts a chip into a test PCB. Power on:

- Does the chip respond on PCIe at all? (read PCI vendor ID register)
- Does PSP boot?
- Does SMU bring clocks to base? (read clock counters)
- Can the kernel driver bind?

This is the **POR (Power On Reset)** moment. It’s a milestone celebrated company-wide.

If the chip doesn’t respond: hours of debug with logic analyzers, oscilloscopes, x-ray imaging, electrical measurements. Sometimes a re-spin (fix in next mask set).

Pre-tapeout simulation work

Drivers must work before silicon exists. Two main vehicles:

Software simulator (PAL-Sim)

A high-level functional model of the hardware. Slow (microseconds per simulated clock), but easy. The driver can run against it to validate basic flows.

FPGA / Emulator

A real hardware emulator (millions of dollars per box) that runs the design's RTL at ~1/1000 real speed. Drivers run against this for full register-accurate testing — TLB walks, IH interrupts, real DMA. Hours to run a kernel boot, days to run a full test.

PMTS engineers writing new IP block drivers do most of their early work here.

Bring-up artifacts

Bring-up generates a stream of artifacts:

- **Errata documents**: silicon bugs that software must work around.
- **Workaround patches**: kernel and UMD changes to mask errata.
- **Pre-public firmware**: SMU/PSP fixes for early issues.
- **Bring-up scripts** (often in Python): unique sequences to coax features alive.

Errata are taken seriously. Some are fixed in B0/C0/etc. silicon revs; some persist for the chip's life and become permanent driver code.

Communicating with hardware

When something doesn't work in bring-up, the question is: is it software, firmware, or silicon?

- **Software**: revert recent driver patches; bisect.
- **Firmware**: ask FW team for a debug build with logging.
- **Silicon**: provide a minimal repro to ASIC team, who run it against RTL/emulator and confirm.

Silicon bugs are scary because they can't be fixed by patches — only via mask revisions or driver workarounds. A senior PMTS engineer is often the bridge: speaks to silicon, kernel, FW, and UMD teams equally.

Post-silicon validation (PSV)

Once a chip is functional, PSV teams pound on it for months:

- **Stress**: sustained 100% load for days.
- **Power cycle**: thousands of resets.
- **Thermal**: full envelope from cold to hottest spec.
- **Voltage**: corners (low/nominal/high) per rail.
- **Process**: chips from different wafers, different process corners.
- **Real workloads**: real games, real AI training jobs.

PSV publishes failure rates. Engineering decides: ship it, fix it, defeature it.

Driver test plans for new silicon

For each new chip the driver must support:

Test class	Examples
Smoke	bind, query, allocate, run a hello-world kernel
Functional	every IP block (GFX, SDMA, VCN, DCN) end-to-end
Integration	real Vulkan apps, ROCm benchmarks
Stress	weeks of mixed workload
Reset/recovery	inject hangs, validate recovery
Power	suspend/resume, runtime PM
Multi-GPU	xGMI, peer-to-peer
Thermal	force throttle path
Long-tail	obscure chip variants and SKU configurations

PMTS engineers design and review these test plans.

Errata handling pattern

A typical erratum's lifecycle:

1. ASIC team identifies (e.g., "TLB invalidation can drop one entry under specific conditions").
2. Errata document published (internal).
3. Driver/FW team designs workaround (e.g., flush TLB twice in this path).
4. Patch lands with comment referencing errata ID.
5. Workaround stays for the chip's life unless fixed in next stepping.

You'll see comments like:

```
        /*
         * NavixxFamily Errata #123: TLB invalidate may miss entries
         * when targeted with specific VMID range. Issue a global
         * invalidate after each targeted one as a workaround.
         */
```

These accumulate. Reading the workarounds for a chip teaches you what actually went wrong in silicon.

Pre-launch software polish

In the months before launch:

- Performance tuning across N games and workloads.
- Polish: idle power, thermal acoustics, sleep latency.
- Compatibility: every monitor, every USB-C dock, every weird DP-MST setup.
- Compliance: FCC, EMC, drop tests (laptops), thermal certifications.

The driver stabilizes; new commits decline; the chip team hands off to long-term sustaining engineers.

After launch — sustaining

The first year after launch:

- Bug reports flood in from real-world users.
- Hot patches via driver releases.
- Backports to LTS kernels.
- Game-day bugs: a new AAA title launches, exposes a corner-case bug, must be fixed in 24 hours.

PMTS engineers in sustaining are often the same ones who did bring-up. They know the chip's quirks intimately.

Mentoring during bring-up

Bring-up is intense, hours-long. Senior engineers must:

- Calmly mentor junior engineers in the lab at 2 AM.
- Explain why "it works on my machine" isn't enough.
- Show how to bisect against silicon revs.
- Push for proper documentation of every fix.

Failure to mentor → next-gen bring-up has the same surprises.

What you should be able to do after this chapter

- Sketch the silicon-life-cycle phases.
- Distinguish errata from bugs.
- Outline a driver test plan for a new chip.
- Reason about RTL emulator vs software simulator.
- Recognize bring-up phases in a real org.

Read deeper

- "Hardware Implementation of Computer Architecture" — academic but useful background.
- AMD HotChips talks on chip bring-up (occasionally surface).
- Talks/papers on RISC-V chip bring-up (analogous, public).
- Cadence / Synopsys EDA tool docs (high-level).
- Internal AMD bring-up wikis (when you have access).

Chapter 91 — Cross-vendor architecture: NVIDIA, Apple, ARM, Imagination

You're competing not just with peers at AMD but with engineers at every other GPU vendor. The world is small; people move between companies; the same engineering ideas pop up in different costumes.

PMTS-level engineers are fluent in *all* major GPU architectures. They read each company's whitepapers when they're released, attend HotChips talks, and know the differences cold.

The roster

Vendor	Lines	Where it dominates
NVIDIA	GeForce, RTX, A/H/B series datacenter, Tegra	gaming high-end, AI, autonomous
AMD	Radeon, Instinct	gaming, datacenter AI (rising), HPC
Intel	Arc (dGPU), Xe iGPU, Habana	iGPU, attempting dGPU & AI
Apple	M-series GPUs	Mac platform; tightly coupled to OS
ARM Mali	Cortex-coupled mobile	Android phones, embedded
Imagination Tech (PowerVR)	embedded, automotive	tile-based mobile/embedded
Qualcomm Adreno	Snapdragon mobile	Android flagship phones
Samsung Xclipse	based on AMD RDNA IP	Samsung phones
Huawei Ascend / GPU	datacenter	China internal
MediaTek	various	mobile
Cerebras / SambaNova / Groq / Tenstorrent	non-GPU AI	training, inference accelerators

NVIDIA — the standard you measure against

NVIDIA's GPU architecture vocabulary:

AMD term	NVIDIA term
Compute Unit (CU) / WGP	**Streaming Multiprocessor (SM)**
Wavefront (32 or 64)	**Warp** (32, fixed)
VGPR / SGPR	(combined) registers per SM
LDS	**Shared Memory**
L0/L1 vector $	L1 cache (per SM)

AMD term	NVIDIA term
L2 $	L2 cache (chip-wide)
Infinity Cache	(none — but “L2 large” on H100/B100 is similar)
HBM	HBM (same standard)
MFMA / matrix engine	**Tensor Core**
Ray-accelerator	**RT Core**
MES	**GSP (GPU System Processor)** firmware
KFD / HSA	**CUDA driver / kernel module**
AQL packet	**CUDA stream commands**
ROCm	**CUDA / cuDNN / cuBLAS / NCCL**
Mesa radv	**NVIDIA Vulkan driver** (closed)
amdgpu kernel	**nvidia.ko** (closed; nouveau open)
Open kernel modules	recently NVIDIA opened a “GSP-only” kernel module

NVIDIA strengths:

- Tensor Cores are extremely well integrated; cuBLAS/cuDNN have decade+ optimization.
- CUDA ecosystem mindshare is unmatched.
- NVLink for multi-GPU.
- Early to RT Cores.
- Stable, mature drivers (closed-source).
- Massive software investment in ML frameworks.

NVIDIA weaknesses (from AMD’s POV):

- Closed-source drivers; harder to debug for users.
- Per-GPU pricing high; ROI argument open.
- Power efficiency story shifting as AMD catches up.
- Vendor-lockin frustrates the open-source ecosystem.

Apple — vertical integration

Apple GPUs (in M-series chips) are **TBDR (tile-based deferred renderers)** with custom ISA. Tightly integrated with macOS / Metal / Apple’s compilers. Strengths:

- Unified memory (CPU + GPU share LPDDR5X with hardware coherence).
- Excellent power efficiency.
- Metal API designed alongside hardware → minimal overhead.
- Apple Silicon GPUs scale from 8-core (laptop) to ~80-core (Ultra).

Weakness: locked to Apple platform; no Linux support (though Asahi project is making progress); no AI training-class part yet.

Architecture insight: TBDR partitions the screen into tiles. All draws are sorted by tile. Within a tile, depth is determined first, then only visible pixels are shaded. Saves bandwidth massively on overdraw-heavy scenes (mobile games).

ARM Mali

Mali GPUs (in mobile SoCs from Mediatek, Samsung Exynos, Google Tensor) are TBDR (Bifrost / Valhall / 5th-gen architectures). Open-source Linux driver available (Panfrost, then Panthor for Valhall+). Performance per watt is good; raw compute lower than desktop.

Architecture quirks:

- Quad-based execution (every "core" runs 4-wide vec).
- Cooperative caching across cores.
- Tile memory exposed to shaders for transient data.

ARM is also a key partner — they license the architecture, and AMD competes/coexists in the Android space.

Imagination Tech (PowerVR)

The original tile-based renderer (1990s onwards). PowerVR Series-A through Series-9X. Used in early iPhones, Apple drew heavy inspiration. Now mostly automotive and embedded. Open-source drivers via "PVRSGX" and modern "powervr" kernel driver.

Qualcomm Adreno

Mobile flagship. Forked from the AMD/ATI XGS — historical link! Adreno 600/700/800 series. Mostly closed userland; Mesa's `freedreno` is the open-source Vulkan/OpenGL driver, and very good. Architecture: tile-based with some unique tricks (low-power "bucket" mode for video).

Intel Arc / Xe

Intel's serious dGPU push. Open-source drivers (`xe` kernel driver, Mesa `iris` and `anv` Vulkan). Architecture: Xe Cores → Vector Engines + Matrix Engines (XMX). Matrix engines do BF16/INT8 matrix.

Intel's open-source story is excellent — they upstream aggressively. Their performance has improved release-over-release. Watch them.

How a PMTS uses cross-vendor knowledge

- **Standards work**: in Khronos Vulkan committee, you negotiate with NVIDIA's representatives to design extensions that work for both. Knowing their architecture's idiosyncrasies makes you a better negotiator.
- **Borrowing ideas**: NVIDIA's Tensor Cores were the public proof that matrix engines were the way; AMD's MFMA followed. NVIDIA's RT Cores; AMD's RA. The industry moves together.
- **Recruitment**: many AMD seniors are ex-NVIDIA, ex-Intel, ex-Apple. The conversation shifts when you understand where they came from.
- **Customer comparisons**: a customer says "NVIDIA does X better." You need to know whether they're right and what to do about it.

Reading whitepapers

Every gen, vendors release architecture whitepapers (free PDFs):

- AMD: GPUOpen “RDNA 1/2/3/4 architecture,” “CDNA 1/2/3.”
- NVIDIA: “Volta,” “Ampere,” “Hopper,” “Blackwell” architecture whitepapers.
- Apple: M-series HotChips talks (no full whitepaper; reverse-engineered details by Asahi team).
- Intel: Xe architecture whitepapers.

Read every one in your area when it appears. Compare. Notice the trade-offs each vendor made.

Industry standards work

Khronos:

- **Vulkan** specification working group.
- **OpenCL** WG.
- **SPIR-V** intermediate language.
- **glTF** asset format.

PCI-SIG:

- PCIe spec.
- CXL (Compute Express Link).

VESA:

- DisplayPort versions (1.4, 2.0, 2.1).
- AdaptiveSync (the basis for FreeSync / G-Sync compatibility).

PMTS engineers from AMD attend, contribute proposals, vote. Junior engineers attend a couple of sessions to learn how decisions get made.

What you should be able to do after this chapter

- Translate AMD vocabulary to NVIDIA / Apple / ARM equivalents.
- Identify each vendor’s strengths and weaknesses.
- Read a competitor whitepaper and summarize architecture for your team.
- Know which standards bodies shape the industry.
- Recognize ex-vendor colleagues by their approach to problems.

Read deeper

- HotChips proceedings (every year, free PDF) — the gold standard.
- ChipsAndCheese.com — independent chip analyses.
- Asahi Linux project blog — Apple GPU reverse engineering.
- Mesa project — has drivers for almost every vendor; reading them comparatively is enlightening.
- Khronos Vulkan extension list — see who pushes which features.

Chapter 92 — The PMTS career arc — the 24-year view

You are starting now. The 24-year PMTS you're competing with started in 2002. You can't compress that into 2 years. But you can plan your next 24 with intention, so when you're year 12, year 18, year 24, you're better than they were at the same year.

This is the closing chapter. It's about the long game.

The shape of a career

```
Years 0-1     :  fresh hire / contractor — competent on tools
Years 1-3     :  competent junior — ships features, fixes bugs reliably
Years 3-5     :  senior — owns a feature area; reviewer trusted by team
Years 5-8    :  staff — leads cross-team feature; respected on mailing lists
Years 8-12   :  principal — architectural decisions; mentors staff engineers
Years 12-20  :  PMTS / Distinguished — sets multi-year direction; speaks at major conferences
Years 20+    :  Fellow / VP-IC track — industry-shaping; few people in this seat
```

These titles vary by company. AMD ladder: Engineer I → II → III → MTS (Senior) → SMTS → PMTS → Fellow. Many MTSes are excellent; few become PMTS.

What changes at each level

0–1 year

You're being onboarded. Your tools are wobbly. Your knowledge of AMD GPUs is from this book (or worse). Your job is to **become a useful contributor**: ship 5–10 small patches; learn the tools; build relationships.

1–3 years

You ship features. You debug things others can't. You start to be the person juniors come to. You **specialize** — pick one area (GPUVM, RBE, VCN, KFD, display, SMU, …) and own it. Read every patch in that area, even if you aren't the author. Become the person whose `Reviewed-by:` matters.

3–5 years (senior)

You can land features without anyone holding your hand. You own design discussions. You handle escalations from customers. You become an upstream maintainer for a sub-area. You start mentoring juniors actively.

The transition from competent-junior to senior is **trust + ownership**: you're handed a hard bug, and you don't need follow-up emails. You finish.

5–8 years (staff)

You're now influencing things outside your immediate team. You're at planning meetings for the next chip. Your name is on industry talks. You spend more time on architecture and review than on writing. You **multiply** other engineers' output.

The transition is from "I do hard things" to "I make my whole team productive."

8–12 years (principal)

You decide multi-year directions for AMD. "Should we invest in mesh shaders or virtual geometry?" "Should we deprecate API X or keep maintaining it?" You sit in rooms with VPs and CTOs. You write strategy documents. You're sometimes the AMD voice at standards bodies.

You may write less code than you used to. You may miss it. The trade-off was inevitable.

12–20 years (PMTS, Distinguished)

Few people reach here. You are AMD's spokesperson on a topic — the person customers, press, and standards bodies believe. You **shape** the industry through papers, talks, code, opinions. Your career compounds: every year you build more pull on what AMD does.

20+ years (Fellow, VP-IC track)

Only a handful of seats. You're the technical conscience of the company. You can spend years on one big bet because the company trusts your judgment. Your retirement marks the end of an era.

What kills a career at each stage

0–1: not asking for help; faking understanding; bouncing between teams without depth.

1–3: not specializing; being a generalist forever; complaining about company politics instead of shipping.

3–5: not learning to mentor; not learning to communicate beyond the technical; not building reputation upstream.

5–8: not engaging with cross-team / cross-org problems; staying in one team forever; still wanting to write all the code yourself.

8–12: bad hires (you're hiring now, and bad hires are your fault); refusing to delegate; not making time for strategy.

12+: living in the past architectures; not learning new domains; not raising the next generation.

The compounding investments

Every year, do at least one of these. They compound.

1. **Write something durable** — a patch series, a paper, a blog post, a tool.
2. **Mentor one junior** — actively, weekly, with patience.
3. **Speak publicly once** — internal lunch-and-learn, conference, university recruiting.

4. **Read deeply outside your area** — reading a competitor's whitepaper is "outside your area" the first 5 years.
5. **Build a relationship with a customer or partner** — a name you can email anytime.

After 5 years of one-each-of-these, you have:

- 5 durable artifacts (people quote you).
- 5 engineers you trained (some now senior).
- 5 talks given (you're "known" in the niche).
- Broad cross-vendor literacy.
- A small network of customers who trust you.

Compound that for 20 years. That is what a PMTS *is*.

The non-technical skills

You will be evaluated for promotion on:

- Technical depth (yes, you knew this).
- **Influence**: do others change behavior because of you?
- **Communication**: written, spoken, presentation.
- **Judgment**: do your design decisions hold up over time?
- **Mentorship**: are juniors you work with getting better?
- **Reliability**: when you commit, does it land?

You can be the smartest person in the room and not get promoted to PMTS if your reliability is poor or you can't mentor.

Money, family, location

Practical realities for a long career:

- **Compensation**: PMTS-level total comp is high — hundreds of thousands USD/year with stock — but it's earned, not handed. Plan accordingly. Don't take golden-handcuff risks early; take career-defining risks early.
- **Family**: 24 years includes marriage, children, parental care. Those are not distractions; they are the reason. Work-life integration evolves. Companies that demand round-the-clock work eat their seniors.
- **Location**: AMD has offices in Markham (Toronto), Austin, Bangalore, Shanghai, Munich. Some teams are remote-first. Where you live affects which customers you talk to.
- **Health**: a sustained sprint of code-review-debug-mentor without exercise/sleep ends in burnout by year 8. The PMTS engineers I know who are still strong at year 24 made this investment early.

The horrible truth: not everyone makes PMTS

The pyramid is real. Most engineers stop at MTS or SMTS. That's still a great career. The leap to PMTS requires:

- Right-time-right-place (a chip cycle where your area exploded).
- Right manager (one who advocates for you).

- Right company (one that has a PMTS-level role open).
- Right portfolio of impact.

You can do everything right and not make it. **Don't make PMTS the only goal.** Make the *journey* the goal. The engineering you do, the people you mentor, the bugs you fix, the customers you delight — those are the value.

A 24-year plan, sketched

Year	Goal
Year 0–1	Land 10+ patches in `drivers/gpu/drm/amd/`. Convert from contractor to FTE. Pick a specialty.
Year 1–2	Become the team's GPUVM person (or RBE, or VCN, or…). One conference talk.
Year 2–4	Senior promotion. First mentee. Co-author a blog post.
Year 4–6	Lead a feature spanning 3+ files. Start reviewing patches outside your area.
Year 6–8	Staff promotion. Chair a standards working group sub-feature. Mentor 2–3 juniors.
Year 8–10	Cross-org influence. New chip's bring-up effort co-led by you.
Year 10–14	Principal promotion. Architectural decisions on next-gen.
Year 14–18	PMTS push. Mature mentor of staff engineers. Industry-known.
Year 18–24	Fellow track or VP-IC track. Decide. Either is OK.

Adjust everything by ±2-4 years for realities of life.

The alternative paths

You don't have to climb to PMTS to have a great life:

- **People-management**: lead a team of engineers; lead an org. Different skills; equally rewarded.
- **Founder / startup**: if AMD's ladder isn't fast enough, the next great GPU/AI company hasn't been founded yet.
- **Academia**: PhD at year 5, become a professor. Different reward function (papers, students).
- **Consulting**: senior expert with their own clients, no employer.

Don't let "I should be a PMTS" trap you in a path that doesn't fit you. Audit yourself yearly: am I still excited?

A final word

The 24-year PMTS you're competing with knew nothing in their year 0. They became what they are by 24 years of attention. **You can do the same.** It's not about IQ; it's about iteration. Show up; do good work; ship; mentor; repeat.

The book ends here. The career begins.

Welcome to it.

"I do not know what I may appear to the world; but to myself I seem to have been only like a boy playing on the seashore, and diverting myself in now and then finding a smoother pebble or a prettier shell than ordinary, whilst the great ocean of truth lay all undiscovered before me." — Isaac Newton.

Appendix — Toolchain cheat sheet

Quick reference for the daily commands.

Build

```
gcc -std=c11 -Wall -Wextra -Werror -O2 -g    # release
gcc -std=c11 -Wall -Wextra -Werror -O1 -g \
    -fsanitize=address,undefined             # dev
g++ -std=c++17 -Wall -Wextra -Werror -O2 -g
clang -fsanitize=memory -fno-omit-frame-pointer  # MSan (uninit reads)
```

Inspect

```
nm a.out                          # symbols (T = text, U = undef, …)
nm -C a.out                       # demangle C++
nm -D libfoo.so                   # dynamic symbols only
objdump -d a.out                  # disassemble
objdump -h a.out                  # sections
readelf -a a.out
strings a.out | head
file a.out
ldd a.out                         # dynamic deps
addr2line -e vmlinux 0xfff…       # symbol + line for an address
c++filt _ZNK3std9basic_…          # demangle one symbol
```

Debug

```
gdb --args ./prog arg1            # start gdb
(gdb) run / r                     # start
(gdb) break / b main              # set breakpoint
(gdb) bt                          # backtrace
(gdb) info threads / thread N     # threads
(gdb) print x / p x               # print
(gdb) watch *(int*)0xdead         # watchpoint
(gdb) finish                      # run until function returns
(gdb) layout asm/regs/src         # tui

valgrind --tool=memcheck ./prog
valgrind --tool=helgrind ./prog   # data races
strace -f -e ioctl ./prog
ltrace ./prog
perf stat ./prog
perf record -ag -- ./prog
perf report
perf top
```

Kernel build

```
cp /boot/config-$(uname -r) .config
make olddefconfig
scripts/config --enable DEBUG_KERNEL DEBUG_INFO PROVE_LOCKING KASAN
```

```
        make olddefconfig
        make -j$(nproc)
        sudo make modules_install
        sudo make install
```

Out-of-tree module Makefile

```
obj-m := mymod.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD  := $(shell pwd)

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Useful kernel debug knobs

```
        echo 1 > /proc/sys/kernel/sysrq
        dmesg -w
        echo 'file amdgpu* +p' | sudo tee /sys/kernel/debug/dynamic_debug/control
        echo function_graph > /sys/kernel/tracing/current_tracer
        echo 'amdgpu_*' > /sys/kernel/tracing/set_ftrace_filter
        echo 1 > /sys/kernel/tracing/tracing_on
```

git daily

```
        git status
        git diff
        git diff --stat HEAD~1
        git log --oneline -20
        git log --graph --oneline --all
        git blame file.c
        git bisect start; git bisect bad; git bisect good <sha>
        git rebase -i HEAD~5
        git commit -s -m "subject"     # -s for sign-off
        git format-patch -1 -o out/
        git send-email --to=user out/0001-*.patch
        git cherry-pick <sha>
```

DRM/amdgpu

```
        lspci -vv -s 01:00.0
        cat /sys/class/drm/card*/status
        cat /sys/kernel/debug/dri/0/name
        cat /sys/kernel/debug/dri/0/amdgpu_pm_info
        echo 1 > /sys/kernel/debug/dri/0/amdgpu_gpu_recover    # force reset
        glxinfo | grep -i 'renderer string'
        vulkaninfo --summary
        modetest -M amdgpu                  # libdrm test tool
```

ROCm

```
        rocm-smi
        rocminfo
        clinfo
        rocprof --hip-trace ./hip_app
```

Appendix — A QEMU + custom-kernel dev VM (do this once, save your life forever)

You will break the kernel. You don't want to brick your laptop. The right answer is a VM that boots a kernel *you* built, with `gdb` attached, and that you can blow away in 5 seconds.

1. Install host packages

Debian/Ubuntu:

```
sudo apt install -y \
    build-essential libncurses-dev libssl-dev libelf-dev bison flex \
    bc cpio rsync wget git \
    qemu-system-x86 qemu-utils \
    debootstrap
```

2. Get the kernel source

```
cd ~/work
git clone --depth=1 https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux
```

3. Configure and build

Start from the Debian/Ubuntu config (closest to a real distro), enable debug:

```
make defconfig
make kvm_guest.config             # adds virtio etc.
scripts/config --enable DEBUG_KERNEL --enable DEBUG_INFO_DWARF5 \
               --enable GDB_SCRIPTS --enable PROVE_LOCKING \
               --enable KASAN --enable KCSAN \
               --enable DEBUG_FS --enable FTRACE --enable DYNAMIC_FTRACE
make olddefconfig
make -j$(nproc)
```

You now have `arch/x86/boot/bzImage` and `vmlinux`.

4. Build a minimal rootfs

The classic "buildroot or debootstrap" choice. For a 60-second answer, use a busybox initramfs:

```
mkdir -p ~/work/rootfs && cd ~/work/rootfs
mkdir -p {bin,sbin,etc,proc,sys,dev,tmp,root,lib,lib64,usr/{bin,sbin}}
cp /bin/busybox bin/
```

```
for cmd in sh ls cat echo mount umount mkdir insmod rmmod dmesg lsmod modinfo poweroff; do
    ln -sf busybox bin/$cmd
done
cat > init <<'EOF'
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev
echo "ldd-textbook rootfs ready"
exec /bin/sh
EOF
chmod +x init
find . | cpio -o -H newc | gzip > ../initramfs.cpio.gz
```

(Get `busybox-static` from your package manager if `/bin/busybox` isn't statically linked.)

5. Boot it

```
qemu-system-x86_64 \
    -enable-kvm -cpu host -smp 4 -m 2G \
    -kernel ~/work/linux/arch/x86/boot/bzImage \
    -initrd ~/work/initramfs.cpio.gz \
    -append "console=ttyS0 nokaslr" \
    -nographic
```

`Ctrl-A x` to quit.

`nokaslr` is important if you want gdb addresses to match `vmlinux`.

6. Attach gdb to the running kernel

In the QEMU command, add `-s -S` (`-s` = gdbserver on :1234, `-S` = freeze at start). In another terminal:

```
cd ~/work/linux
gdb vmlinux
(gdb) target remote :1234
(gdb) hbreak start_kernel
(gdb) c
```

You're now stepping through the Linux boot. Welcome to god mode.

7. Add the QEMU `edu` device for the PCI lab

```
qemu-system-x86_64 ... \
    -device edu \
    -device intel-iommu                 # optional, exercises IOMMU
```

Inside the guest you'll see the `edu` PCI device. Lab 06 binds against it.

8. Enable AMD GPU passthrough (optional, advanced)

For real `amdgpu` work you can pass through a discrete GPU using VFIO. This is a longer setup, covered in Chapter 50. For most learning, the QEMU `edu` device + a real machine running `amdgpu` natively is enough.

9. Snapshots

Always make a snapshot of a working VM image before you go destructive:

```
qemu-img create -f qcow2 disk.qcow2 8G
qemu-img snapshot -c clean disk.qcow2
qemu-img snapshot -a clean disk.qcow2     # restore
```

10. The 30-second iterate loop

Once set up, the developer cycle is:

```
make -j$(nproc) bzImage modules           # rebuild
sudo make modules_install INSTALL_MOD_PATH=$ROOTFS
# rebuild initramfs (or use 9p / virtfs to share)
qemu-system-x86_64 ... -kernel … -initrd …
# inside guest: insmod / dmesg / test
# crash? keep dmesg, exit, edit code, repeat
```

If a change takes more than 30 seconds to test, fix the workflow before fixing the bug.

Appendix — Kernel coding style (the rules that get patches accepted)

The full canonical doc lives at `Documentation/process/coding-style.rst`. This is the operational subset you must internalize before sending a patch.

1. Indentation

- **Tabs**, exactly **8 columns wide**. Not 4. Not spaces. Tabs.
- This is non-negotiable. Run `:set ts=8 sw=8 noexpandtab` in vim or the equivalent in your editor.
- The wide tab is intentional: if you nest deeper than 3 levels, your function is too complex.

2. Line length

- **80 columns** is still the soft limit. Up to **100 columns** is tolerated for human readability when wrapping makes things worse. Don't wrap a printk message just to make it fit; let it run.

3. Braces

- Opening brace **on the same line** as `if`, `while`, `for`, `switch`, `do`.
- Opening brace **on its own line** for `function definitions` and `struct/union` definitions.

```
int foo(int x)
{
    if (x > 0) {
        do_thing();
    } else {
        do_other();
    }
}
```

- Single-statement bodies don't need braces *unless* one branch of an `if` / `else` has braces — then both must.

4. Naming

- `snake_case` for functions, variables, struct members.
- `SCREAMING_SNAKE` for `#define` and enum constants.

- Local variables: short. `i`, `n`, `len`, `tmp`. The kernel is not Java.
- Functions exposed in headers: prefix with the subsystem (`amdgpu_vm_init`, `drm_dev_register`).
- No camelCase. No `m_` Hungarian prefixes.
- `goto err_*:` labels for cleanup.

5. Comments

- `/* like this */` for multi-line. `//` is allowed but `/* */` is more common.
- Comments explain **why**, never **what**.
- Function header comments use **kernel-doc**:

```
/**
 * amdgpu_bo_create - allocate a BO
 * @adev: amdgpu device
 * @size: bytes
 * @domain: AMDGPU_GEM_DOMAIN_*
 *
 * Allocates a buffer object suitable for the requested domain.
 * Caller must drop the reference with amdgpu_bo_unref().
 *
 * Return: 0 on success, -ENOMEM on allocation failure.
 */
```

6. Functions

- One function does **one thing**.
- Aim for **<60 lines**. If it's longer, you almost certainly need to split.
- **<5 parameters**. Pack into a struct otherwise.
- Single exit point via `goto err_*` for cleanup. The "errors are forward gotos" pattern is universal:

```
int probe(struct device *dev)
{
    int ret;

    ret = step_a();
    if (ret)
        return ret;

    ret = step_b();
    if (ret)
        goto err_a;

    ret = step_c();
    if (ret)
        goto err_b;

    return 0;

err_b:
    undo_b();
err_a:
    undo_a();
    return ret;
}
```

7. Locking

- Document every lock in a comment near its declaration: what it protects, what may not be held while taking it, IRQ vs process context.
- Acquire in a **fixed order** project-wide. The kernel runs `lockdep` to enforce this; respect it.

8. Macros

- Lowercase macros that look like functions are *frowned on* unless you have a real reason.
- Always parenthesize macro arguments and the whole expansion:

```
#define MIN(a, b) ({ typeof(a) _a = (a); typeof(b) _b = (b); _a < _b ? _a : _b; })
```

(But prefer `min()` from `<linux/minmax.h>`.)

9. Don't

- **Don't typedef structs.** Use `struct foo`. (Kernel rule, opposite of glibc style.) Exceptions are well-established (`u32`, `pid_t`, opaque handles).
- **Don't use bool in structs** that pack tightly. Use `u8` or bitfields where size matters.
- **Don't add new printk's without a level**. Always `pr_err` / `pr_warn` / `pr_info` / `pr_debug` (or `dev_*`).
- **Don't break userspace.** "Don't break userspace" is the prime directive. If your patch makes an ABI change, justify it in the commit msg or it gets nacked.

10. Run before sending

```
scripts/checkpatch.pl --strict --codespell -g HEAD
scripts/get_maintainer.pl <patch.mbox>
make C=2 W=1 drivers/gpu/drm/amd/amdgpu/<your_file>.o
```

`checkpatch.pl` will catch 80% of style issues; the human reviewer will catch the rest.

11. Commit message

- **Subject line**: `subsystem: short imperative summary` ≤72 chars.
  - Good: `drm/amdgpu: Fix NULL deref in vm_pt_create()`
  - Bad: `Fixed bug`
- **Body**: explain *why*, in present tense imperative. Reference the bug, the failing path, the symptom, the fix.
- **Tags** at the bottom in this order:

```
Fixes: 1234567890ab ("drm/amdgpu: original commit subject")
Cc: stable@vger.kernel.org   # if user-visible bugfix
Reviewed-by: Reviewer Name <r@example.com>
Signed-off-by: Your Name <you@amd.com>
```

If you can write good kernel commit messages, you can write anything.

Appendix — Glossary (every acronym you will hit)

ABI — Application Binary Interface. The contract between compiled artifacts (calling convention, struct layout, symbol names). Stable kernel↔userspace; *unstable* in-kernel.

ACPI — Advanced Configuration and Power Interface. Firmware tables describing the platform.

APIC / LAPIC / IOAPIC — Advanced Programmable Interrupt Controller. Routes interrupts on x86.

ATS / PRI / PASID — PCIe address translation services / page request interface / process address space ID. Used for IOMMU + SVM.

BAR — Base Address Register. PCIe device's window into memory or I/O space.

BO — Buffer Object. A piece of GPU-visible memory in DRM/GEM.

CDNA / RDNA — AMD's compute / graphics architectures (Instinct vs Radeon).

CRTC — Cathode-Ray Tube Controller (vestigial name). The display pipeline driving a connector.

CS — Command Submission. The ioctl that hands a GPU work batch to the kernel.

DC — Display Core. AMD's display block (also known as DCN).

DKMS — Dynamic Kernel Module Support. Out-of-tree module manager.

DMA — Direct Memory Access. Device reads/writes RAM without CPU.

DRM — Direct Rendering Manager. Linux GPU driver framework.

EDID — Extended Display Identification Data. Monitor capability blob.

FLR — Function Level Reset (PCIe). Per-function reset.

GEM — Graphics Execution Manager. DRM's BO management layer.

GFP flags — Get Free Page flags. Tell kmalloc/alloc_pages how it can sleep, what zone, etc.

GMC — Graphics Memory Controller (AMD). Handles GPU MMU, page tables.

GPUVM — Per-process GPU virtual address space.

GTT — Graphics Translation Table. System RAM made GPU-accessible via IOMMU/GART.

hrtimer — High-resolution timer in kernel.

IB — Indirect Buffer. A buffer of GPU commands referenced from a ring.

IH — Interrupt Handler block (AMD). Receives interrupts from GPU sub-blocks.

IOMMU / AMD-Vi / VT-d / SMMU — I/O memory management unit. Translates device DMAs.

IP block — Intellectual Property block. A discrete hardware unit on the GPU (GFX, SDMA, VCN, …).

IRQ — Interrupt Request.

KFD — Kernel Fusion Driver. AMD's compute/HSA path (`/dev/kfd`).

KMD / UMD — Kernel-Mode Driver / User-Mode Driver.

KMS — Kernel Mode Setting. The display side of DRM.

MEC / MES — Micro Engine Compute / Scheduler. AMD GPU firmware engines.

MMIO — Memory-Mapped I/O. Device registers accessed by load/store.

MSI / MSI-X — Message-Signaled Interrupts. PCIe interrupts as memory writes.

NUMA — Non-Uniform Memory Access. Memory closer to some CPUs than others.

OPP — Operating Performance Point. (Voltage, frequency) pair.

PASID — see ATS.

PCIe — PCI Express. The bus.

PIMPL — Pointer to IMPLementation. C++ pattern hiding members for ABI stability.

PM4 — AMD GPU command packet format.

PRIME — DRM mechanism for sharing BOs between drivers via dma-buf.

PSP — Platform Security Processor (AMD). Boots GPU firmware.

RAII — Resource Acquisition Is Initialization. C++ resource pattern.

RAS — Reliability, Availability, Serviceability. ECC/UE handling.

RCU — Read-Copy-Update. Kernel concurrency primitive for read-mostly data.

SDMA — System DMA engine (AMD).

SMU — System Management Unit (AMD). Power/clock controller.

SVM — Shared Virtual Memory. CPU and GPU share the same VA space.

TLB — Translation Lookaside Buffer. MMU/SMMU page-table cache.

TLP — Transaction Layer Packet. PCIe wire packet.

TTM — Translation Table Manager. DRM’s memory manager for VRAM-class devices.

UE / CE — Uncorrectable / Correctable Error (RAS).

VA / PA — Virtual / Physical Address.

VCN — Video Core Next (AMD video codec block).

VFS — Virtual File System. Linux’s filesystem abstraction.

VMID — Virtual Memory ID. GPU’s process tag, indexes the GPU page table register file.

vblank — Vertical Blanking Interval. The “between-frames” event.

Appendix — Checklists you can paste into your dotfiles

These are the lists I personally walk down when I'm tired and the build is broken at 2 AM.

"I just hit a kernel oops" checklist

1. Save the full `dmesg`. Don't trust scrollback; pipe to a file: `dmesg > /tmp/oops.log`.
2. Note: PID, comm, RIP, CR2 (faulting address), call trace.
3. Decode the RIP: `./scripts/decode_stacktrace.sh vmlinux < oops.log`.
4. `addr2line -e vmlinux <RIP>` for file:line.
5. Look at the **last function in your driver** in the trace. That's almost always the bug site.
6. Reproduce with KASAN if not enabled. KASAN turns "oops" into "exact bad access + which alloc/free pair."
7. If you can't reproduce: enable `lockdep`, `KCSAN`, `slub_debug=FZP`. Reboot. Try again.

"I just hit a hang" checklist

1. Magic SysRq:
   - `Alt+SysRq+t` → all task stacks to dmesg.
   - `Alt+SysRq+l` → all CPU stacks.
   - `Alt+SysRq+w` → blocked tasks (uninterruptible sleep).
2. From sysrq output, find the offending CPU/task. Look for `mutex_lock`, `wait_for_completion`, `schedule`.
3. Check the GPU: `cat /sys/kernel/debug/dri/0/amdgpu_*ring*` — are all rings idle?
4. `cat /proc/<pid>/stack` of the hung process.
5. If it's a deadlock, `lockdep` would have caught it on a debug build → make sure you're running a debug build.

"I'm about to send a kernel patch" checklist

- [ ] One logical change per patch.
- [ ] Built it. (`make C=1 W=1` for sparse + extra warnings on touched files.)
- [ ] Tested it. (Mention exactly what test in the commit msg.)
- [ ] `scripts/checkpatch.pl --strict` clean (or justified).
- [ ] `scripts/get_maintainer.pl` to get To/Cc list.

- [ ] Subject is `subsystem: short imperative summary` (≤72 chars).
- [ ] Body explains *why* not *what*.
- [ ] `Signed-off-by:` line.
- [ ] `Fixes:` tag if it fixes a known commit (use `--abbrev=12`).
- [ ] `Cc: stable@vger.kernel.org` if user-visible bugfix worth backporting.
- [ ] No churn in unrelated files.
- [ ] Sent with `git send-email` (not GitHub PR, not attachment).
- [ ] Reply with v2/v3 in the same thread; include changelog under `---`.

“I’m reviewing someone’s patch” checklist

- [ ] Apply locally with `git am`. Build it. Skim runtime output if possible.
- [ ] Read the *whole* changed file, not just the hunk.
- [ ] Locking: every shared field touched, who holds what?
- [ ] Memory: who frees, on which path? Error paths cleaned up?
- [ ] Concurrency: can this fire in interrupt context? On another CPU during teardown?
- [ ] DMA/coherence: is the right sync called? Direction right?
- [ ] Userspace ABI: never break it. Never.
- [ ] Comments: necessary, accurate, not narrating.
- [ ] Reply on-list. Be technical, not personal. Quote specifically.

“I’m starting at AMD on Monday” checklist (Day 0)

- [ ] Kernel built from source from git, booting in a VM.
- [ ] AMD GPU in a desktop, latest distro kernel + Mesa, all userspace stacks (OpenGL/Vulkan/ROCm) verified working.
- [ ] `umr` installed and `umr -lb` runs.

- [ ] `git clone` of `linux`, `mesa`, `drm`, `libdrm`, `llvm-project` somewhere on disk.
- [ ] Read your team's onboarding docs end-to-end before Day 1.
- [ ] Email + git + IRC/Slack working. Sign-off + corp legal name in `git config`.
- [ ] One pre-built tracing/printing helper in `~/bin/` (e.g., a script that turns on dynamic_debug for amdgpu).
- [ ] LinkedIn says "Software Engineer, AMD" with the start date. (Soft thing, but the reality of FTE conversion is influenced by visibility.)

"I'm building a portfolio commit" checklist

- [ ] Solves a real bug or adds a real feature, not a toy.
- [ ] Commit message is interview-quality (problem, root cause, fix, impact, test).
- [ ] Test plan included (output, screenshot, perf number).
- [ ] No trailing whitespace, no debugging prints left over.
- [ ] Tagged in your portfolio repo's README with the *upstream* link.
- [ ] You can talk about it for 10 minutes without notes.

Print these. Tape them above your desk.

Appendix — Curated reading list

Books, talks, and source you will actually open. Curated, not a dump.

Books — read end-to-end

1. **The C Programming Language**, Kernighan & Ritchie. Still the cleanest C book. ~270 pages.
2. **Effective Modern C++**, Scott Meyers. 42 items, every one earns its keep. C++11/14, but the lessons map to C++17/20.
3. **Operating Systems: Three Easy Pieces** (free online). Best modern OS textbook for a beginner.
4. **Linux Device Drivers, 3rd ed.** (Corbet/Rubini/Kroah-Hartman, free PDF). Old (2.6 era) but still the best driver book ever written. Read it knowing some APIs have evolved.
5. **Linux Kernel Development, 3rd ed.**, Robert Love. Friendly tour of the kernel.
6. **Understanding the Linux Kernel**, Bovet & Cesati. Heavier reference; dip in.
7. **Computer Architecture: A Quantitative Approach**, Hennessy & Patterson. Skim early chapters; deep on memory hierarchy & ILP.
8. **The Linux Programming Interface**, Michael Kerrisk. The definitive Linux syscall book.

Books — reference (not cover-to-cover)

- **C++ Concurrency in Action**, Anthony Williams.
- **Is Parallel Programming Hard?**, Paul McKenney (free). RCU and memory models from the inventor.
- **Memory Barriers: a hardware view for software hackers**, McKenney (paper).
- **What Every Programmer Should Know About Memory**, Ulrich Drepper (free PDF). Old but the cache chapter is timeless.

Documentation you should bookmark

- `Documentation/` in the kernel tree — especially `Documentation/gpu/`, `Documentation/locking/`, `Documentation/driver-api/`, `Documentation/process/`.
- The DRM kernel-doc HTML: build it once with `make htmldocs` and read `gpu.html`.
- AMD GPUOpen + ROCm docs: https://gpuopen.com, https://rocmdocs.amd.com.
- PCIe base spec (you likely won't read it; you will *grep* it).

Talks (YouTube — search and watch)

- "The C++ Programming Language" — Bjarne Stroustrup, Going Native 2012.
- "atomic<> Weapons" parts 1 & 2 — Herb Sutter. Memory model, slow but worth it.

- “Linux Graphics, GPUs and DRM” — Daniel Vetter, FOSDEM. Multiple years; pick the latest.
- “How the Linux Kernel ARM page tables work” — Linus Walleij.
- “An introduction to the LLVM compiler infrastructure” — Chris Lattner.
- AMD ISA + GPU architecture deep dives (RDNA whitepapers, CDNA whitepapers).
- “Diagnosing realtime application issues” — Steven Rostedt, ftrace creator.

Source code (read this, not just docs)

- `drivers/gpu/drm/vkms/` — smallest complete DRM driver.
- `drivers/gpu/drm/drm_*.c` core files — `drm_drv.c`, `drm_ioctl.c`, `drm_gem.c`, `drm_atomic.c`, `drm_atomic_helper.c`.
- `drivers/gpu/drm/scheduler/sched_main.c` — the DRM scheduler.
- `drivers/gpu/drm/amd/amdgpu/amdgpu_drv.c`, `amdgpu_device.c`, `amdgpu_cs.c`, `amdgpu_vm.c`, `amdgpu_ring.c`, `amdgpu_fence.c`, `amdgpu_irq.c`, `amdgpu_gem.c`, `amdgpu_bo.c`.
- `drivers/gpu/drm/ttm/ttm_*.c` — buffer manager.
- `mesa/src/amd/` and `mesa/src/gallium/drivers/radeonsi/`, `mesa/src/amd/vulkan/` (radv) — the userspace side.
- libdrm `amdgpu/` directory.
- For RCU: `kernel/rcu/tree.c` — but read McKenney’s papers first.

Mailing lists / forums

- `linux-kernel@vger.kernel.org` (lore.kernel.org) — main kernel list.
- `dri-devel@lists.freedesktop.org` — DRM/KMS/GPU devs (this is where amdgpu patches go).
- `amd-gfx@lists.freedesktop.org` — AMD-specific.
- `mesa-dev` / Mesa GitLab (most discussion is on MRs now).
- `freedesktop.org` IRC: #dri-devel, #radeon (libera.chat).

Reading order for an absolute beginner

Months 1–2: K&R + arena/SPSC/thread-pool labs. Months 3–4: OSTEP + LDD3 chapters 1-7 + your own char driver + edu PCI lab. Months 5–6: Drepper memory paper + McKenney’s “Is Parallel Programming Hard?” + DRM core source. Months 7–9: amdgpu source crawl + Mesa radv + first upstream patch. Months 10–12: Specialize (RAS, KFD, scheduler, DC, …). Your specialty becomes your interview answer.

Appendix — A guided tour of `drivers/gpu/drm/amd/amdgpu/`

This is the file you open when you start at AMD and your manager says "have a look at the driver." Don't open the directory and panic at 800 files. Open these, in this order.

The path prefix `drivers/gpu/drm/amd/amdgpu/` is omitted below.

Tier 1 — read these first

File	Why
`amdgpu_drv.c`	Module entry, `pci_driver` struct, the `pci_device_id` table, module params (`amdgpu.dpm=` , etc.). The "front door" of the driver.
`amdgpu.h`	Mega-header. `struct amdgpu_device` lives here. ~1500+ lines of typedefs and forward decls. Skim, don't memorize.
`amdgpu_device.c`	`amdgpu_device_init` , `amdgpu_device_fini` , IP block bring-up & tear-down sequence. The probe path lives here.
`amdgpu_kms.c`	DRM driver `open` / `postclose` hooks, ioctl table, `drm_driver` struct, fpriv allocation.
`amdgpu_ioctl.c` (split across files)	The amdgpu-specific ioctls live in `amdgpu_kms.c` , `amdgpu_cs.c` , `amdgpu_gem.c` , `amdgpu_ctx.c` .

Tier 2 — the resource managers

File	Why
`amdgpu_object.{c,h}`	`amdgpu_bo` — wraps `ttm_buffer_object` , the BO type used everywhere.
`amdgpu_gem.c`	GEM ioctls (create, mmap, op, userptr) — the userspace BO API.
`amdgpu_ttm.c`	TTM glue — placement, eviction, blit-via-SDMA migration callbacks.
`amdgpu_vm.{c,h}`	Per-process GPUVM, page table walks, `amdgpu_bo_va` , mapping update path, eviction. The single most important file in the driver.
`amdgpu_vm_pt.c`	Page table allocation/update logic separated out.
`amdgpu_mn.c`	MMU notifiers for userptr (CPU page-table changes invalidate GPU mappings).

Tier 3 — command path

File	Why
`amdgpu_cs.c`	The `AMDGPU_CS` ioctl. Validates the BO list, builds an `amdgpu_job`, hands to scheduler. The hot path.
`amdgpu_job.c`	`amdgpu_job` lifecycle, scheduler integration.
`amdgpu_ring.{c,h}`	Generic ring abstraction for GFX/SDMA/UVD/VCN/MES.
`amdgpu_ib.c`	Indirect Buffer helpers.
`amdgpu_fence.c`	dma-fence implementation backed by GPU sequence counters. Very small, very important.
`amdgpu_sync.c`	Aggregating multiple fences for a single dependency.
`amdgpu_ctx.c`	`amdgpu_ctx` (per-userspace-context) — owns per-context priorities, hangs counters.

Tier 4 — interrupt + reset

File	Why
`amdgpu_irq.c`	IRQ source registration + dispatch table.
`amdgpu_ih.c` (and per-IP `vega20_ih.c`, `navi10_ih.c`, ...)	IH ring (interrupt aggregator).
`amdgpu_reset.c`	Reset logic, reset domain, links to `amdgpu_device.c::amdgpu_device_gpu_recover`.
`amdgpu_ras.c`	RAS (ECC) handling.
`amdgpu_ras_eeprom.c`	Bad page table persisted in EEPROM.

Tier 5 — display + per-IP

Path	Why
`amdgpu/dc/` (under `drivers/gpu/drm/amd/display/`)	The Display Core, *huge*. Atomic state, link training, programming sequences. Treat as separate codebase.
`amdgpu_dm.c` (under `display/amdgpu_dm/`)	The bridge between DRM atomic API and DC.
`amdgpu/gfx_*.c`, `gmc_*.c`, `sdma_*.c`, `vcn_*.c`, ...	Per-IP block instantiations. Each file owns the silicon-specific tables.
`amdgpu/mes_*.c`	MES (Micro Engine Scheduler) firmware glue.
`amdgpu/psp_*.c`	Platform Security Processor — owns firmware loading.
`amdgpu/sdma_*.c`	SDMA queues used for buffer copy/migration.

Tier 6 — KFD (compute)

`drivers/gpu/drm/amd/amdkfd/` is its own driver path used by ROCm:

File	Why
`kfd_module.c`, `kfd_device.c`	Bring-up.
`kfd_chardev.c`	The `/dev/kfd` ioctl table.
`kfd_process.c`, `kfd_process_queue_manager.c`	Per-process KFD state, queue management.
`kfd_svm.c`	Shared Virtual Memory.

How to navigate it

```
# 1. Use cscope or vim's tags + ripgrep.
cscope -Rb -s drivers/gpu/drm/amd
# 2. ALWAYS use git log for any function: who wrote it and why?
git log -p drivers/gpu/drm/amd/amdgpu/amdgpu_vm.c | less
# 3. Read commit messages of the file owners.
git log --format='%an' drivers/gpu/drm/amd/amdgpu/amdgpu_vm.c | sort | uniq -c | sort -rn | head
# 4. Check Documentation/gpu/amdgpu.rst.
```

Suggested reading sequence (your first month)

Week	Read
1	`amdgpu_drv.c`, `amdgpu_device.c::amdgpu_pci_probe()` end-to-end.
2	`amdgpu_kms.c` ioctl table, follow `AMDGPU_CS` into `amdgpu_cs.c`.
3	`amdgpu_vm.c` — read every function, draw the data structure on paper.
4	`amdgpu_fence.c` + `amdgpu_ring.c` + DRM scheduler `drivers/gpu/drm/scheduler/sched_main.c`.

By the end you'll have something to say in any AMD interview question.

Labs

Hands-on exercises with full source code. Each lab corresponds to a chapter or chapter group. They're designed so each builds on the previous.

Lab	Chapter	What you build
`01-arena/`	7	Arena and free-list allocator in C
`02-spsc/`	23	Lock-free SPSC ring buffer in C++
`03-thread-pool/`	24	Mutex+condvar thread pool
`04-hello-mod/`	29	First kernel module with parameters
`05-myring-chardev/`	35-36	Char driver with read/write/poll/ioctl/mmap
`06-edu-pci/`	37-38	PCI driver against QEMU's `edu` with DMA
`07-vkms-skeleton/`	44-47	Minimal DRM driver
`08-libdrm-amdgpu/`	49-52	Userspace amdgpu IB submission

Each directory has its own `README.md`, source, and `Makefile`. Run `make` and follow.

Prerequisites

```
sudo apt install build-essential gdb clang \
                 linux-headers-$(uname -r) \
                 git make qemu-system-x86 \
                 libdrm-dev mesa-utils \
                 valgrind strace
```

For amdgpu labs, you need a system with an AMD GPU and the in-tree `amdgpu` driver loaded. For kernel-module labs, any modern Linux works.

Tips

- Always test in a VM first if you're modifying anything that talks to your GPU. `amdgpu` reset paths are mostly safe but data loss is possible.
- Keep `dmesg -w` open in a separate terminal while testing kernel modules.
- Build with `-Werror -Wall -Wextra` for userspace; turn on KASAN for kernel.